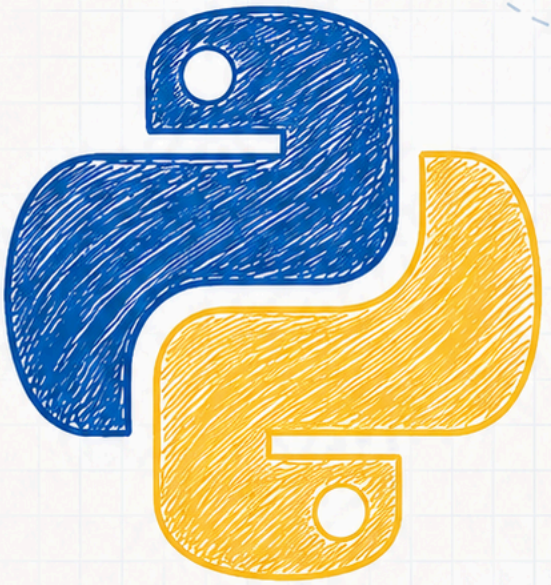
APUNTES

Programación de Servicios y Procesos

(Con Python)



```python
import time          # importaciones
import socket
from multiprocessing import Process

def trabajo(nombre):          # definición de función
    print(f"Iniciando {nombre}...")
    time.sleep(2)
    print(f"{nombre} finalizado.")

if __name__ == "__main__":          # ejecución del proceso
    p = Process(target=trabajo, args=("Proceso",))
    p.start()
    p.join()
```

Por
Sergi García Barea

licencia
CC BY SA

Contents

- Inicio
  - 📚 Unidades
  - 📝 Boletines de ejercicios
  - 📄 Licencia
- U01 — Python 3 básico
  - 🎯 Objetivo de la unidad
  - 🗺️ Mapa de la unidad
  - 📝 Boletines de la unidad
  - ✅ Criterios de evaluación cubiertos (Prerequisito)
  - 🚪 ¿Por dónde empiezo?
- 01 — Introducción a Python
  - 📬 La idea en una frase
  - 🐍 ¿Qué es Python?
  - 🤔 ¿Por qué Python para PSP?
  - 🖥️ Instalar Python
  - 📦 Sugerencia: entorno virtual (venv)
  - 💻 Tu primer programa
  - 💉 La consola interactiva
  - 🧠 Mini-chequeo
  - ✅ Resumen en 3 frases
  - 🦎 Vocabulario rápido
- 02 — Comentarios
  - 📬 La idea en una frase
  - 📝 Comentario de línea
  - 📄 Docstring (comentario multilinea)
  - 🔁 Comillas simples vs dobles
  - ⚠️ Errores comunes
  - 🧠 Mini-chequeo
  - ✅ Resumen en 3 frases
  - 🦎 Vocabulario rápido
- 03 — Tipos de datos y operadores
  - 📬 La idea en una frase

    - 🔢 Números enteros (int)
    - 🧑‍🚒 Números decimales (float)
    - ✅ Booleanos (bool)
    - 📝 Strings (str)
    - 🕳️ None
    - 🔧 Funciones de conversión
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Variables y colecciones
    - 📬 La idea en una frase
    - 📌 Variables
    - 📋 Listas
    - 🔒 Tuplas
    - 📖 Diccionarios
    - 🔵 Conjuntos (set)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Control de flujo
    - 📬 La idea en una frase
    - 🔀 Condicional: if / elif / else
    - 🔁 Bucle: for
    - 🔁 Bucle: while
    - ⚠️ Manejo de errores: try / except
    - 🔃 Iterables e iteradores
    - 🧩 Comprensiones (preview)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Funciones
    - 📬 La idea en una frase
    - 🔨 Definir y llamar funciones

    - 📦 Argumentos variables
    - 🏭 Funciones de primera clase
    - 🔹 Funciones lambda
    - 🔁 map, filter y reduce
    - 🧩 Comprensiones
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Clases
    - 📬 La idea en una frase
    - 📐 Definir una clase
    - 🎭 Crear y usar objetos
    - 🔑 El método __init__
    - 📋 Métodos útiles
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Módulos y avanzado
    - 📬 La idea en una frase
    - 📦 Importar módulos
    - 🔍 Explorar módulos
    - 🛒 pip: instalar paquetes
    - ⚡ Generadores
    - 🎀 Decoradores
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé el código
    - 🔥 Fireside Chat: Variables vs Constantes
    - 🕵️ ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO
    - ⚡ Laboratorio de tortura

    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (Prerequisito)
- UD 2 — Gestión de procesos con Python
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA1)
    - 🚪 ¿Por dónde empiezo?
- 01 — Qué es un proceso
    - 📬 La idea en una frase
    - 🫧 La burbuja de memoria
    - 🪪 El PID, el carnet de identidad
    - 📋 Características de un proceso
    - 🍳 La analogía de la cocina
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Estados de un proceso
    - 📬 La idea en una frase
    - 🔄 El ciclo de vida de un proceso
    - 🚦 Las transiciones, una a una
    - ☕ El ejemplo de la cafetería
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Paralela vs Distribuida
    - 📬 La idea en una frase
    - 🤹 La tabla de los tres conceptos

    - 🍳 Paralela: cuatro sartenes
    - 🌆 Distribuida: varios restaurantes
    - ⚖️ CPUs vs máquinas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — subprocess.run()
    - 📬 La idea en una frase
    - 🚀 El primer lanzamiento
    - 🎛️ Los parámetros importantes
    - ⏱️ Timeout: no esperes para siempre
    - 🦎 Un ejemplo con error controlado
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — subprocess.Popen()
    - 📬 La idea en una frase
    - 🚀 El primer Popen: abrir el bloc de notas
    - 🎛️ Los métodos de Popen
    - 🤸 Lanzar varios procesos a la vez
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Comunicación con procesos
    - 📬 La idea en una frase
    - 🔌 Conectar los tubos
    - 📨 Enviar y recibir con communicate()
    - 🗣️ El mismo ejemplo completo, con sentido
    - 💡 ¿Cuándo usar cada herramienta?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Compatibilidad Windows / Linux

    - 📬 La idea en una frase
    - 🪟 🐧 La tabla de comandos equivalentes
    - 🧩 El truco del shell
    - 🌍 Lo que nunca cambia
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Procesos nativos con multiprocessing
    - 📬 La idea en una frase
    - ⚖️ subprocess vs multiprocessing
    - 🚀 Lanzando funciones: Process, start() y join()
    - 🧱 El muro del aislamiento (la burbuja de memoria)
    - 🤝 Rompiendo el muro con Manager
    - 🔒 Sincronización con Lock (condiciones de carrera)
    - 🏋️ Reto avanzado (no evaluable)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Ejemplo resuelto: estado compartido con procesos
    - 📬 La idea en una frase
    - 🎯 El problema
    - 🧱 Paso 1: la estructura base del programa
    - 👷 Paso 2: el trabajador
    - 💾 Paso 3: persistencia (guardar y cargar el almacén)
    - 🚀 Paso 4: el main que lo pone todo en marcha
    - 🔍 Traza paso a paso
    - 🧨 Paso 5: quita el candado y mira la catástrofe
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 10 — Procesos en la práctica
    - 📬 La idea en una frase
    - ⭐ Sé el código: abrir bloc de notas y calculadora

    - 🥊 El ring de los conceptos — run() vs Popen()
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 11 — Cierre: consolida lo aprendido
    - ⭐ Sé el proceso
    - 🔥 Fireside Chat: Proceso vs Hilo
    - 🦹 ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “maté el proceso y no volvió”
    - ⚡ Laboratorio de tortura: el lanzador con cierre automático
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA1)
- UD 3 — Hilos y concurrencia con Python
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2)
    - 🚪 ¿Por dónde empiezo?
- 01 — De proceso a hilo
    - 📬 La idea en una frase
    - 🛎️ ¿Qué es un hilo?
    - 🥊 Hilos vs Procesos
    - 🍳 La analogía de los chefs en una cocina
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Tu primer hilo

    - 📬 La idea en una frase
    - 🪄 Crear y lanzar un hilo
    - 👑 El hilo principal vs los hilos secundarios
    - ⏳ join() — esperar a que termine
    - 🧰 Propiedades de un hilo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Hilos con argumentos
    - 📬 La idea en una frase
    - 🎒 Pasar argumentos con args
    - 🎁 kwargs: argumentos con nombre
    - 🏷️ Nombrar hilos: .name = "hilo-" + str(n)
    - 👯 Varios hilos a la vez
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Hilos daemon
    - 📬 La idea en una frase
    - 😈 ¿Qué es un hilo daemon?
    - 🧚 Be the code, my friend — daemon en acción
    - 🥊 El ring de los conceptos — Hilo normal vs Hilo daemon
    - ⏳ join() para esperar a un daemon (si acaso lo necesitas)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Timer
    - 📬 La idea en una frase
    - ⏰ El ejemplo mínimo
    - 🧭 Cancelar un Timer
    - 🎁 Pasar argumentos al Timer
    - 💡 Usos típicos del Timer
    - 🧠 Mini-chequeo

    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — El GIL
    - 📬 La idea en una frase
    - 🔒 ¿Qué es el GIL?
    - ⚡ CPU-bound: los hilos NO sirven de nada
    - 🌐 I/O-bound: los hilos SÍ aceleran
    - 📊 La tabla que hay que saberse
    - 🔨 Cuándo sirven de verdad los hilos
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Estados del hilo
    - 📬 La idea en una frase
    - 🔁 El diagrama del ciclo de vida
    - 🗺️ Los estados, uno a uno
    - 🧍 El estado BLOQUEADO, el más importante
    - 🕐 ¿Cuándo termina un hilo?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Hilos en la práctica
    - 📬 La idea en una frase
    - 🧍 Be the code, my friend — El hilo viajero
    - 🥊 El ring de los conceptos — Hilo vs Proceso
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé el hilo
    - 🔥 Fireside Chat: Hilo vs Proceso
    - 🕵️ ¿Quién soy?

    - 👹 CONRAD VS EL MUNDO: “el programa termina antes que mis hilos”
    - ⚡ Laboratorio de tortura: la sala de descargas
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA2)
- UD 4 — Sincronización en Python
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2)
    - 🚪 ¿Por dónde empiezo?
- 01 — Condición de carrera
    - 📬 La idea en una frase
    - 🔍 ¿Por qué falla? El += no es atómico
    - 💰 La analogía: dos cajeros y una caja
    - 🌍 Dónde aparece la condición de carrera
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Lock
    - 📬 La idea en una frase
    - 🔑 Métodos de Lock
    - 🤝 Siempre with lock:
    - 🎭 Be the code: contador con y sin Lock
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — RLock
    - 📬 La idea en una frase

    - 🪃 ¿Cuándo lo necesitas? Funciones que se llaman entre sí
    - ⚖️ Lock vs RLock
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Semaphore
    - 📬 La idea en una frase
    - ⏱️ Semáforo con timeout
    - 🏭 ¿Para qué sirve un semáforo?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Barrier
    - 📬 La idea en una frase
    - 🧱 Sincronizar fases de un trabajo
    - ⚖️ Barrier frente al resto
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Condition
    - 📬 La idea en una frase
    - 📚 Métodos de Condition
    - ⚠️ La regla de oro: while, no if
    - 🦸 ¿Cuándo usarla?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Productor-Consumidor
    - 📬 La idea en una frase
    - 🎬 Paso a paso
    - 📏 Con varios productores y consumidores
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases

    - 🦎 Vocabulario rápido
- 08 — Buenas prácticas
    - 📬 La idea en una frase
    - 🎲 El deadlock: todos esperan al Lock
    - 🛡️ Cómo evitar deadlocks: las 3 reglas
    - ⚡ Preguntas rápidas (y sus respuestas)
    - 🥊 El ring de los conceptos — Lock vs Semáforo vs Barrera
    - 🖍️ Aprieta el lápiz
    - 🧩 Pool Puzzle de sincronización
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Python Free-Threaded (PEP 703)
    - 📬 La idea en una frase
    - 🔐 Qué es el free-threaded (y por qué tardó 15 años)
    - 🦸 ¿Mi Python es free-threaded? (3 comprobaciones)
    - 🔬 Mira el bytecode: contador += 1 bajo el capó
    - 🏃 demo_race_condition.py
    - 🛎️ Por qué Lock y queue.Queue siguen mandando
    - 🗺️ Matriz de decisión (¿qué uso y cuándo?)
    - 🛠️ Actividades sencillas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 10 — Cierre: consolida lo aprendido
    - ⭐ Sé el lock
    - 🔥 Fireside Chat: Lock vs Semaphore
    - 🦸 ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “olvidé el release() y todos esperan al lock”
    - ⚡ Laboratorio de tortura: la caja común de la cooperativa
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits

    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA2)
- UD 5 — Sockets TCP y UDP
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA3)
    - 🚪 ¿Por dónde empiezo?
- 01 — Qué es un socket
    - 📬 La idea en una frase
    - 📇 IP y puerto: el “quién” y el “dónde”
    - 🐍 Crear un socket en Python
    - ✈️ La analogía: carta certificada contra avión de papel
    - ⚖️ TCP vs UDP cara a cara
    - 📞 La analogía del teléfono, al detalle
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Cliente TCP
    - 📬 La idea en una frase
    - 🔁 Ciclo de vida del cliente
    - 🐍 El cliente mínimo resuelto
    - 🧩 Cada método, al detalle
    - 🎭 Be the code, my friend — Cliente TCP
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Servidor TCP
    - 📬 La idea en una frase
    - 🔁 Ciclo de vida del servidor
    - 🐍 El servidor mínimo resuelto

    - 🧩 Cada método, al detalle
    - 🔁 El bucle de atención
    - 🧩 Pool Puzzle — El ciclo de vida del socket
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Ciclo de vida y errores
    - 📬 La idea en una frase
    - 🤝 Three-way handshake (establecer conexión)
    - 👋 Cierre de la conexión: la despedida
    - 😡 TIME_WAIT y “Address already in use”
    - 📸 El ciclo completo, en una mirada
    - 🛡️ Las redes fallan: try/except y timeouts
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Cliente y servidor UDP
    - 📬 La idea en una frase
    - 📤 El cliente UDP: sendto() y recvfrom()
    - 📥 El servidor UDP: bind() y recvfrom()
    - 🪨 Datagramas: cada envío es un mundo aparte
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — HTTP y NTP
    - 📬 La idea en una frase
    - 🌐 HTTP desde cero: el protocolo que mueve la web
    - 🔢 El orden de los bytes (byte ordering)
    - ⏰ NTP: sincronizar relojes con UDP
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Cuándo usar cada protocolo

    - 📬 La idea en una frase
    - 🧭 El criterio: velocidad o fiabilidad
    - 📡 Casos reales: la tabla que se repite en las entrevistas
    - 🔀 Y si quieres ambas cosas: QUIC
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Práctica eco
    - 📬 La idea en una frase
    - 🛠️ Eco en TCP
    - 📡 Eco en UDP
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé el socket
    - 🔥 Fireside Chat: TCP vs UDP
    - 🕵️ ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “conexión reiniciada y paquetes perdidos”
    - ⚡ Laboratorio de tortura: eco TCP y UDP
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA3)
- UD 6 — Servidores concurrentes con Python
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4c-d)

    - 🚪 ¿Por dónde empiezo?
- 01 — Servidor secuencial
    - 📬 La idea en una frase
    - 🐢 El servidor secuencial
    - 📜 La traza: secuencial vs concurrente
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — El problema de la espera
    - 📬 La idea en una frase
    - 🥶 Un cliente lento congela la cola
    - 🏛️ La analogía: la ventanilla única
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Hilo por cliente
    - 📬 La idea en una frase
    - 🚀 El servidor multihilo
    - 📜 Be the code — Servidor multihilo en acción
    - ⚖️ Ventajas y el primer cuidado
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — ThreadPoolExecutor
    - 📬 La idea en una frase
    - ⚠️ El problema: miles de hilos
    - ⚡ El servidor con pool
    - 🎯 ¿Qué tamaño de pool elegir?
    - 🧩 Pool Puzzle — Servidor con ThreadPool
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Benchmark

    - 📬 La idea en una frase
    - 🧪 El experimento
    - 📊 Los resultados
    - 🚀 El lanzador masivo de clientes
    - 🔍 Cómo interpretar los resultados
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Sincronización en servidores
    - 📬 La idea en una frase
    - 💥 La condición de carrera en el servidor
    - 🔒 El Lock en acción
    - 🔍 ¿Y el contador de bytes?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Límites y buenas prácticas
    - 📬 La idea en una frase
    - 🔢 ¿Cuántos hilos puede tener un servidor?
    - 🦹 ¿Qué pasa si un hilo se cuelga?
    - 🏭 ¿ThreadPool o hilo por cliente en producción?
    - 🌊 ¿Y si el servidor recibe 10.000 conexiones?
    - 🔀 ¿Puedo mezclar hilos y asyncio?
    - ✅ Guía rápida de buenas prácticas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Servidor concurrente completo con hilos
    - 📬 La idea en una frase
    - 🏗️ El servidor: servidor.py
    - 💬 El cliente: cliente.py
    - 🚀 El lanzador: lanzaclientes.py
    - 🤝 Cómo probarlos juntos

    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Servidor concurrente completo con procesos
    - 📬 La idea en una frase
    - 🧐 ¿Por qué procesos y no hilos?
    - 🏗️ El servidor: servidor.py
    - 💬 El cliente: cliente.py
    - 🚀 El lanzador: lanzaclientes.py
    - 🤝 Cómo probarlos juntos
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 10 — Cierre: consolida lo aprendido
    - ⭐ Sé el servidor
    - 🔥 Fireside Chat: Hilo por cliente vs ThreadPool
    - 🕵️‍♀️ ¿Quién soy?
    - 😡 CONRAD VS EL MUNDO: “un cliente lento bloquea el servidor”
    - ⚡ Laboratorio de tortura: servidor concurrente con ThreadPool
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA4c-d)
- UO7 — APIs REST y HTTP
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4a-b)

    - 🚪 ¿Por dónde empiezo?
- 01 — Web y HTTP
    - 📬 La idea en una frase
    - 🧩 ¿Qué es una API?
    - 🕸️ La web en tres piezas
    - 📬 Petición y respuesta
    - 🛵 La analogía del pedido a domicilio
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Métodos HTTP
    - 📬 La idea en una frase
    - 🚦 Los verbos de HTTP
    - 📃 ¿Qué es la idempotencia?
    - 🐍 Métodos en Python
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Principios REST
    - 📬 La idea en una frase
    - 🧱 Recursos: las cosas, no las acciones
    - 🗄️ Los cuatro pilares de REST
    - 🦚 Sin estado (stateless)
    - 🏷️ ¿Qué significa “RESTful”?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Códigos de estado
    - 📬 La idea en una frase
    - 🎨 Las familias de códigos
    - 📋 La tabla completa
    - 🐍 Códigos en Python
    - 🧠 Mini-chequeo

    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — JSON
    - 📬 La idea en una frase
    - 📦 El formato
    - 🔄 De JSON a Python y viceversa
    - 🔁 JSON dentro de requests
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — requests: el GET
    - 📬 La idea en una frase
    - 📥 El GET más básico
    - 📄 La respuesta y sus propiedades
    - 🔎 Parámetros de consulta
    - 🧢 Cabeceras personalizadas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — requests: POST, PUT y DELETE
    - 📬 La idea en una frase
    - 📤 POST: crear un recurso
    - 📝 PUT y PATCH: actualizar
    - 🗑️ DELETE: borrar
    - 🛡️ Gestión de errores: status_code y raise_for_status
    - 🧩 Pool Puzzle — Petición API con errores
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Práctica: mini cliente de API
    - 📬 La idea en una frase
    - 🏗️ El mini cliente de API
    - ⭐ Sé el código: una petición GET, milisegundo a milisegundo

    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé la petición HTTP
    - 🔥 Fireside Chat: GET vs POST vs PUT vs DELETE
    - 🦹 ¿Quién soy?
    - 🤬 CONRAD VS EL MUNDO: “404 que nadie entiende”
    - ⚡ Laboratorio de tortura: mini cliente contra una API pública
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA4a-b)
- U08 — APIs Comerciales
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4a-b)
    - 🚪 ¿Por dónde empiezo?
- 01 — API Keys
    - 📬 La idea en una frase
    - 🎫 ¿Qué es una API key?
    - 🔑 ¿Cómo se obtiene?
    - 📍 ¿Dónde va la clave?
    - 🔑 La analogía del llavero de acceso
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Variables de entorno

    - 📬 La idea en una frase
    - 📁 El archivo .env
    - ⚙️ Cargarlo con python-dotenv
    - 🚫 No subir claves al repo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — OpenWeatherMap
    - 📬 La idea en una frase
    - 🗺️ Registro y API key
    - 📍 La URL y sus parámetros
    - 🍳 Llamada y parseo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — OpenAI
    - 📬 La idea en una frase
    - 🧠 Registro y clave
    - 💬 Chat completions
    - 📜 Mensajes y roles
    - 🔄 Alternativa con httpx (sin librería oficial)
    - 💰 Consumo y coste
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Rate limiting
    - 📬 La idea en una frase
    - ⏳ ¿Qué es el rate limit?
    - 🚨 El error 429
    - 😴 Esperar antes de reintentar
    - 🔁 Reintentos inteligentes (backoff)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases

    - 🦎 Vocabulario rápido
- 06 — Errores HTTP
    - 📬 La idea en una frase
    - 🚨 Los códigos que verás
    - 🧯 raise_for_status() y try/except
    - 🛡️ Una llamada segura de verdad
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Seguridad y buenas prácticas
    - 📬 La idea en una frase
    - 🚫 Lo que nunca debes hacer
    - 🗂️ El .gitignore y el commit
    - 🔁 Rotación de claves
    - 🛡️ Checklist de buenas prácticas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Práctica: APIs comerciales
    - 📬 La idea en una frase
    - ⏳ El programa completo: tiempo + cerebro
    - ⭐ Sé el código: la consulta meteorológica
    - 🛠️ Herramientas para probar APIs
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé la petición
    - 🔥 Fireside Chat: REST vs SOAP vs GraphQL
    - 🕵️ ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “clave subida a GitHub”
    - ⚡ Laboratorio de tortura: OpenWeatherMap + OpenAI

    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - ✅ Criterios de evaluación cubiertos (RA4a-b)
- UD 9 — Seguridad y cifrado con Python
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA5)
    - 🚪 ¿Por dónde empiezo?
- 01 — Principios de seguridad
    - 📬 La idea en una frase
    - 🏰 Los seis principios que no puedes olvidar
    - 🔑 La analogía de las cerraduras de una casa
    - 📖 La cita que debes saber
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Hash y huellas digitales
    - 📬 La idea en una frase
    - 👣 La huella digital
    - 🐉 Hash en Python: tres sabores
    - ⚖️ La tabla comparativa
    - 📄 Hash de un archivo: verificar integridad
    - 🐉 SHA-512 en acción
    - 📐 Las cinco propiedades del hash
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Contraseñas seguras

    - 📬 La idea en una frase
    - 🎭 Be the code: registro y login con hash
    - 🗄️ La regla de oro
    - 🕳️ El problema: hashes idénticos
    - 🧂 La solución: salt
    - 🌈 Por qué la sal destroza las tablas rainbow
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — Cifrado clásico
    - 📬 La idea en una frase
    - 🏛️ La historia: Julio César y su desplazamiento de 3
    - 🐍 Cifrar y descifrar en Python
    - 💥 Romperlo por fuerza bruta
    - 🥊 Hash vs Cifrado: el ring de los conceptos
    - ⚖️ La moraleja: cada uno a lo suyo
    - 🎬 La escena que lo aclara todo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Cifrado simétrico (AES)
    - 📬 La idea en una frase
    - 🔑 Cifrado simétrico: una sola clave
    - 🔑 Cifrado asimétrico: el par público/privado
    - ⚖️ AES vs RSA
    - 🧩 Componentes de AES
    - 🐍 AES con cryptography
    - ⛓️ Modos de operación: ECB, CBC, GCM
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Cifrado asimétrico (RSA)
    - 📬 La idea en una frase

    - 🔑 Generar el par de claves
    - 🔒 Cifrar con la pública, descifrar con la privada
    - ⚠️ El límite de RSA
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Firmas digitales
    - 📬 La idea en una frase
    - ✍️ Firmar (con tu clave privada)
    - ✅ Verificar (con la clave pública del firmante)
    - 🎭 Be the code: Firma y verificación
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Cifrado híbrido y práctica
    - 📬 La idea en una frase
    - 🔁 El intercambio, paso a paso
    - 🐉 Código completo
    - 🌐 Por qué HTTPS usa esto
    - 🔐 RBAC: Control de acceso basado en roles
    - 🧬 El sistema completo: cifrar + firmar
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé el cifrado
    - 🔥 Fireside Chat: Hash vs Cifrado
    - 🔥 Fireside Chat: AES vs RSA
    - 🦸 ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “guardé las contraseñas con MD5 y sin sal”
    - 👹 CONRAD VS EL MUNDO: “cifré el mensaje pero no puedo descifrarlo”
    - ⚡ Laboratorio de tortura: registro + login + cifrado híbrido

    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - ✅ Criterios de evaluación cubiertos (RA5)
- UD 10 — Alta disponibilidad
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4e-g)
    - 🚪 ¿Por dónde empiezo?
- 01 — Event Loop
    - 📬 La idea en una frase
    - 🚫 El problema de esperar
    - ☕ La analogía de la cafetería
    - ⚙️ Qué es el event loop, de verdad
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Corrutinas
    - 📬 La idea en una frase
    - 🌀 async def y await
    - 🤔 Corrutina no es una función normal
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — create_task y gather
    - 📬 La idea en una frase
    - 🚀 asyncio.gather: todas a la vez
    - 🎯 asyncio.create_task: en segundo plano
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases

    - 🦎 Vocabulario rápido
- 04 — Timeouts
    - 📬 La idea en una frase
    - ⏰ El problema del cuelgue
    - ⏱️ wait_for con respaldo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Heartbeat
    - 📬 La idea en una frase
    - 💓 El latido en asyncio
    - 🩺 Heartbeat con estado
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Backoff
    - 📬 La idea en una frase
    - 📈 El problema: reintentar sin piedad
    - 🔢 La secuencia del backoff
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Threads vs asyncio
    - 📬 La idea en una frase
    - 🥊 El ring de los conceptos
    - ⚖️ La tabla comparativa
    - 🧭 ¿Cuándo usar cada uno?
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — Disponibilidad y práctica
    - 📬 La idea en una frase
    - 🩺 El monitor de servicio

    - 🎭 Be the code: el event loop en un servidor
    - 🖍️ Aprieta el lápiz
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Cierre: consolida lo aprendido
    - ⭐ Sé la corrutina
    - 🔥 Fireside Chat: hilos vs asyncio
    - 🦸 ¿Quién soy?
    - 👺 CONRAD VS EL MUNDO: “el servidor se cuelga si un cliente no responde”
    - ⚡ Laboratorio de tortura: el monitor de servicio definitivo
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
    - 🎬 PRÓXIMAMENTE EN UD 11 · Spring Boot con Java
    - ✅ Criterios de evaluación cubiertos (RA4e-g)
- UD 11 — Spring Boot con Java
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 ¿Qué necesitas?
    - 📝 Boletines de la unidad
    - 🚪 ¿Por dónde empiezo?
- 01 — Java para desarrolladores Python
    - 📬 La idea en una frase
    - 🔄 Python vs Java: la comparativa rápida
    - 🔄 Tipos básicos
    - 📦 Colecciones
    - 📐 Clases y objetos
    - ⚡ Concurrencia
    - 🧠 Mini-chequeo

    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Introducción a Spring Boot
    - 📬 La idea en una frase
    - 🌱 ¿Qué es Spring Boot?
    - 🆚 Flask vs Spring Boot
    - 📁 Estructura de un proyecto Spring Boot
    - 🚀 Crear tu primer proyecto
    - 🏃 El punto de entrada
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Inyección de dependencias
    - 📬 La idea en una frase
    - 🍽️ La analogía del restaurante
    - 🏭 El contenedor IoC
    - 🏷️ Las anotaciones principales
    - 🔄 El ciclo de vida
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — RESTController y APIs REST
    - 📬 La idea en una frase
    - 🆚 Flask vs @RestController
    - 🏷️ Las anotaciones de routing
    - 📝 Ejemplo completo: API de tareas
    - 🔑 Las anotaciones clave
    - 📦 El modelo
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — JPA y bases de datos
    - 📬 La idea en una frase

    - 🆚 SQL vs JPA
    - 🏷️ Las anotaciones de JPA
    - 📦 El repositorio
    - 🔍 Consultas personalizadas
    - ⚙️ Configuración (application.properties)
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Ejemplo completo: API de tareas
    - 📬 La idea en una frase
    - 🏗️ El proyecto completo
    - 💉 Probar la API
    - 📊 La arquitectura en capas
    - 🧠 Mini-chequeo
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — Cierre: Python vs Java/Spring Boot
    - ⭐ Sé el framework
    - ⚖️ La comparativa final
    - 🔥 Fireside Chat: Python vs Java
    - 🦸 ¿Quién soy?
    - 📚 Recursos para seguir
    - 🧠 Atrévete a pensar
    - 🎬 Poscréditos
    - ✅ Criterios de la unidad
- Boletín UD 1 — Inicial
    - 1. Hola mundo con f-string
    - 2. Calculadora básica
    - 3. Par o impar
    - 4. Tabla de multiplicar
    - 5. Invertir una cadena
    - 6. Lista de la compra
    - 7. Diccionario de alumnos

    - 8. Clase Coche
- Boletín UD 1 — Inicial (Resuelto)
    - 1. Hola mundo con f-string
    - 2. Calculadora básica
    - 3. Par o impar
    - 4. Tabla de multiplicar
    - 5. Invertir una cadena
    - 6. Lista de la compra
    - 7. Diccionario de alumnos
    - 8. Clase Coche
- Boletín UD 1 — Avanzado
    - 1. Comprensiones de lista
    - 2. Manejo de archivos
    - 3. Lambda con reduce
    - 4. Clase Cuenta Bancaria
    - 5. Decorador de rendimiento
    - 6. Generador de Fibonacci
    - 7. Gestor de errores personalizado
    - 8. Mini inventario con set operations
    - 9. Parser de CSV sencillo
- Boletín UD 1 — Avanzado (Resuelto)
    - 1. Comprensiones de lista
    - 2. Manejo de archivos
    - 3. Lambda con reduce
    - 4. Clase Cuenta Bancaria
    - 5. Decorador de rendimiento
    - 6. Generador de Fibonacci
    - 7. Gestor de errores personalizado
    - 8. Mini inventario con set operations
    - 9. Parser de CSV sencillo
- Boletín UD 2 — Inicial
    - 1. Listar el directorio actual
    - 2. Saber el nombre del equipo

    - 3. Abrir la calculadora
    - 4. Mi primer PID
    - 5. Versión de Python
    - 6. Abrir el bloc de notas y esperar
    - 7. Ordena el ciclo de vida
    - 8. Ping con timeout
- Boletín UD 2 — Inicial (Resuelto)
    - 1. Listar el directorio actual
    - 2. Saber el nombre del equipo
    - 3. Abrir la calculadora
    - 4. Mi primer PID
    - 5. Versión de Python
    - 6. Abrir el bloc de notas y esperar
    - 7. Ordena el ciclo de vida
    - 8. Ping con timeout
- Boletín UD 2 — Avanzado
    - 1. Filtrar ipconfig
    - 2. Comprobar si un programa existe
    - 3. Abrir el navegador
    - 4. Lanzador múltiple
    - 5. Captura de ping
    - 6. PID del padre
    - 7. Comunicación en dos direcciones
    - 8. Timeout con reintentos
    - 9. Mini monitor de procesos
    - 10. Contador compartido con candado
    - 11. Almacén compartido con Manager
    - 12. Persistencia del almacén
- Boletín UD 2 — Avanzado (Resuelto)
    - 1. Filtrar ipconfig
    - 2. Comprobar si un programa existe
    - 3. Abrir el navegador
    - 4. Lanzador múltiple

    - 5. Captura de ping
    - 6. PID del padre
    - 7. Comunicación en dos direcciones
    - 8. Timeout con reintentos
    - 9. Mini monitor de procesos
    - 10. Contador compartido con candado
    - 11. Almacén compartido con Manager
    - 12. Persistencia del almacén
- Boletín UD 3 — Inicial
    - 1. Hilo que cuenta
    - 2. Múltiples hilos con retardo
    - 3. Hilo con argumento
    - 4. Hilo que saluda
    - 5. Dos hilos en paralelo
    - 6. Hilo con nombre
    - 7. Esperando con join()
    - 8. Varios hilos con nombre y argumentos
- Boletín UD 3 — Inicial (Resuelto)
    - 1. Hilo que cuenta
    - 2. Múltiples hilos con retardo
    - 3. Hilo con argumento
    - 4. Hilo que saluda
    - 5. Dos hilos en paralelo
    - 6. Hilo con nombre
    - 7. Esperando con join()
    - 8. Varios hilos con nombre y argumentos
- Boletín UD 3 — Avanzado
    - 1. Join con timeout
    - 2. Listar hilos activos
    - 3. Identificar el hilo actual
    - 4. 🎯 Carrera de mensajes
    - 5. 🫧 Simular descarga con progreso
    - 6. 🧩 Detener hilo con bandera

    - 7. 🎭 Hilo que devuelve un valor
    - 8. ⏱️ Timer con repetición
    - 9. 🏗️ Pool de hilos manual
- Boletín UD 3 — Avanzado (Resuelto)
    - 1. Join con timeout
    - 2. Listar hilos activos
    - 3. Identificar el hilo actual
    - 4. 🎯 Carrera de mensajes
    - 5. 🫧 Simular descarga con progreso
    - 6. 🧩 Detener hilo con bandera
    - 7. 🎭 Hilo que devuelve un valor
    - 8. ⏱️ Timer con repetición
    - 9. 🏗️ Pool de hilos manual
    - 🎁 Bonus: tres clásicos resueltos
- Boletín UD 4 — Inicial
    - 1. Lock protege una lista
    - 2. RLock básico
    - 3. Semáforo con timeout
    - 4. Carrera sin Lock
    - 5. Contador protegido con Lock
    - 6. Sección crítica con sleep
    - 7. Semáforo simple
    - 8. Barrera de 3
- Boletín UD 4 — Inicial (Resuelto)
    - 1. Lock protege una lista
    - 2. RLock básico
    - 3. Semáforo con timeout
    - 4. Carrera sin Lock
    - 5. Contador protegido con Lock
    - 6. Sección crítica con sleep
    - 7. Semáforo simple
    - 8. Barrera de 3
- Boletín UD 4 — Avanzado

    - 1. Condition con notify_all
    - 2. Deadlock entre dos hilos
    - 3. Semáforo con recursos limitados
    - 4. 🎯 Banco con cuentas compartidas
    - 5. 🎱 Semáforo con timeout
    - 6. 🧩 Barrera con fases múltiples
    - 7. 🎭 Condition con timeout
    - 8. ⏱️ Deadlock provocado y solución
    - 9. ⏳ Productor-Consumidor múltiple
- Boletín UD 4 — Avanzado (Resuelto)
    - 1. Condition con notify_all
    - 2. Deadlock entre dos hilos
    - 3. Semáforo con recursos limitados
    - 4. 🎯 Banco con cuentas compartidas
    - 5. 🎱 Semáforo con timeout
    - 6. 🧩 Barrera con fases múltiples
    - 7. 🎭 Condition con timeout
    - 8. ⏱️ Deadlock provocado y solución
    - 9. ⏳ Productor-Consumidor múltiple
- Boletín UD 5 — Inicial
    - 1. Servidor saludo
    - 2. Cliente que envía y recibe
    - 3. Servidor eco
    - 4. IP y puerto
    - 5. Cliente UDP con entrada de usuario
    - 6. Servidor UDP con eco personalizado
    - 7. TCP vs UDP: clasifica
    - 8. Cliente HTTP manual
- Boletín UD 5 — Inicial (Resuelto)
    - 1. Servidor saludo
    - 2. Cliente que envía y recibe
    - 3. Servidor eco
    - 4. IP y puerto

    - 5. Cliente UDP con entrada de usuario
    - 6. Servidor UDP con eco personalizado
    - 7. TCP vs UDP: clasifica
    - 8. Cliente HTTP manual
- Boletín UD 5 — Avanzado
    - 1. Servidor que cuenta caracteres
    - 2. Cliente interactivo
    - 3. Servidor de mayúsculas
    - 4. Cliente con reconexión
    - 5. Servidor que gestiona múltiples conexiones (sin hilos)
    - 6. Timeout personalizado
    - 7. Cliente NTP manual
    - 8. Servidor UDP multimensaje
    - 9. Ping UDP
    - 10. Servidor en todas las interfaces
    - 11. Compara TCP y UDP
    - 12. Mini servidor web
- Boletín UD 5 — Avanzado (Resuelto)
    - 1. Servidor que cuenta caracteres
    - 2. Cliente interactivo
    - 3. Servidor de mayúsculas
    - 4. Cliente con reconexión
    - 5. Servidor que gestiona múltiples conexiones (sin hilos)
    - 6. Timeout personalizado
    - 7. Cliente NTP manual
    - 8. Servidor UDP multimensaje
    - 9. Ping UDP
    - 10. Servidor en todas las interfaces
    - 11. Compara TCP y UDP
    - 12. Mini servidor web
- Boletín UD 6 — Inicial
    - 1. Servidor eco básico
    - 2. Servidor eco con bucle

    - 3. Cliente eco
    - 4. Servidor secuencial
    - 5. Servidor multihilo
    - 6. Cliente con respuesta
    - 7. Servidor con ThreadPoolExecutor
    - 8. Lanzador de 5 clientes
- Boletín UD 6 — Inicial (Resuelto)
    - 1. Servidor eco básico
    - 2. Servidor eco con bucle
    - 3. Cliente eco
    - 4. Servidor secuencial
    - 5. Servidor multihilo
    - 6. Cliente con respuesta
    - 7. Servidor con ThreadPoolExecutor
    - 8. Lanzador de 5 clientes
- Boletín UD 6 — Avanzado
    - 1. Servidor con timeout
    - 2. Cliente con timeout
    - 3. Contador de bytes totales
    - 4. Servidor con límite de clientes
    - 5. Prueba de carga
    - 6. Servidor con cola de espera
    - 7. Estado del servidor
    - 8. Heartbeat en servidor
    - 9. Balanceador de carga simple
    - 10. Servidor con un proceso por cliente
    - 11. Contador de conexiones entre procesos
    - 12. Aislamiento de fallos
- Boletín UD 6 — Avanzado (Resuelto)
    - 1. Servidor con timeout
    - 2. Cliente con timeout
    - 3. Contador de bytes totales
    - 4. Servidor con límite de clientes

    - 5. Prueba de carga
    - 6. Servidor con cola de espera
    - 7. Estado del servidor
    - 8. Heartbeat en servidor
    - 9. Balanceador de carga simple
    - 10. Servidor con un proceso por cliente
    - 11. Contador de conexiones entre procesos
    - 12. Aislamiento de fallos
- Boletín UD 7 — Inicial
    - 1. GET a JSONPlaceholder
    - 2. Título del post
    - 3. Contar usuarios
    - 4. GET a GitHub
    - 5. Mostrar JSON
    - 6. Nombre real
    - 7. Métodos HTTP
    - 8. Códigos de estado
- Boletín UD 7 — Inicial (Resuelto)
    - 1. GET a JSONPlaceholder
    - 2. Título del post
    - 3. Contar usuarios
    - 4. GET a GitHub
    - 5. Mostrar JSON
    - 6. Nombre real
    - 7. Métodos HTTP
    - 8. Códigos de estado
- Boletín UD 7 — Avanzado
    - 1. PUT para actualizar
    - 2. DELETE un recurso
    - 3. HEAD request
    - 4. 🎯 Info de usuario de GitHub
    - 5. 🔎 Buscador de repositorios
    - 6. 🧩 API con paginación

    - 7. 🎭 POST y respuesta
    - 8. ⏱️ Tiempo de respuesta
    - 9. 🏗️ Cliente de API con caché
- Boletín UD 7 — Avanzado (Resuelto)
    - 1. PUT para actualizar
    - 2. DELETE un recurso
    - 3. HEAD request
    - 4. 🎯 Info de usuario de GitHub
    - 5. 🔎 Buscador de repositorios
    - 6. 🧩 API con paginación
    - 7. 🎭 POST y respuesta
    - 8. ⏱️ Tiempo de respuesta
    - 9. 🏗️ Cliente de API con caché
- Boletín UD 8 — Inicial
    - 1. Humedad actual
    - 2. Descripción del clima
    - 3. Temperatura mínima y máxima
    - 4. Temperatura y sensación térmica
    - 5. Error 401
    - 6. ¿Dónde va la API key?
    - 7. Clave desde el .env
    - 8. Protege el repo
- Boletín UD 8 — Inicial (Resuelto)
    - 1. Humedad actual
    - 2. Descripción del clima
    - 3. Temperatura mínima y máxima
    - 4. Temperatura y sensación térmica
    - 5. Error 401
    - 6. ¿Dónde va la API key?
    - 7. Clave desde el .env
    - 8. Protege el repo
- Boletín UD 8 — Avanzado
    - 1. Clima con .env para dos ciudades

    - 2. GPT con temperatura creativa
    - 3. Capturar errores HTTP
    - 4. Comprobación de claves
    - 5. Chat con GPT y rol system
    - 6. Timeout
    - 7. 🎯 Pronóstico extendido
    - 8. 🔍 Múltiples ciudades
    - 9. 🧩 GPT: explicador automático
    - 10. 🎭 Conversación con contexto
    - 11. ⏱️ Monitor de uptime
    - 12. ⏳ Agregador de APIs
- Boletín UD 8 — Avanzado (Resuelto)
    - 1. Clima con .env para dos ciudades
    - 2. GPT con temperatura creativa
    - 3. Capturar errores HTTP
    - 4. Comprobación de claves
    - 5. Chat con GPT y rol system
    - 6. Timeout
    - 7. 🎯 Pronóstico extendido
    - 8. 🔍 Múltiples ciudades
    - 9. 🧩 GPT: explicador automático
    - 10. 🎭 Conversación con contexto
    - 11. ⏱️ Monitor de uptime
    - 12. ⏳ Agregador de APIs
- Boletín UD 9 — Inicial
    - 1. SHA1 de tu nombre
    - 2. MD5 de una frase
    - 3. Longitud de los hashes de “Hola mundo”
    - 4. Cifrado César sencillo
    - 5. Determinismo del hash
    - 6. Nonce y tag
    - 7. AES: cifrar y descifrar completo
    - 8. Simétrico vs asimétrico

- Boletín UD 9 — Inicial (Resuelto)
  - 1. SHA1 de tu nombre
  - 2. MD5 de una frase
  - 3. Longitud de los hashes de “Hola mundo”
  - 4. Cifrado César sencillo
  - 5. Determinismo del hash
  - 6. Nonce y tag
  - 7. AES: cifrar y descifrar completo
  - 8. Simétrico vs asimétrico
- Boletín UD 9 — Avanzado
  - 1. Cifrado César con espacios
  - 2. Fuerza bruta César
  - 3. Hash con salt
  - 4. Mini gestor de contraseñas
  - 5. Cifrado híbrido simplificado
  - 6. Firma alterada
  - 7. RBAC con permisos cifrado
  - 8. Cifrar archivo completo
  - 9. RSA: cifrar mensajes largos
  - 10. Firma con verificación de integridad
  - 11. RSA vs AES benchmark
  - 12. Sistema de cifrado de extremo a extremo
- Boletín UD 9 — Avanzado (Resuelto)
  - 1. Cifrado César con espacios
  - 2. Fuerza bruta César
  - 3. Hash con salt
  - 4. Mini gestor de contraseñas
  - 5. Cifrado híbrido simplificado
  - 6. Firma alterada
  - 7. RBAC con permisos cifrado
  - 8. Cifrar archivo completo
  - 9. RSA: cifrar mensajes largos
  - 10. Firma con verificación de integridad

    - 11. RSA vs AES benchmark
    - 12. Sistema de cifrado de extremo a extremo
- Boletín UD 10 — Inicial
    - 1. Mensaje diferido
    - 2. Saludo y despedida
    - 3. Temporizador
    - 4. Corrutina que devuelve
    - 5. Tres tareas a la vez
    - 6. Tarea en segundo plano
    - 7. Heartbeat básico
    - 8. Timeout básico
- Boletín UD 10 — Inicial (Resuelto)
    - 1. Mensaje diferido
    - 2. Saludo y despedida
    - 3. Temporizador
    - 4. Corrutina que devuelve
    - 5. Tres tareas a la vez
    - 6. Tarea en segundo plano
    - 7. Heartbeat básico
    - 8. Timeout básico
- Boletín UD 10 — Avanzado
    - 1. Backoff exponencial
    - 2. Timeout con respaldo
    - 3. Dos heartbeats
    - 4. 🎯 Web scraper asíncrono
    - 5. 🎱 Servidor asyncio con heartbeat
    - 6. ⏱️ Monitorización de servidores
    - 7. 🧩 Semáforo asyncio
    - 8. 🎭 Timeout con fallback
    - 9. 🏗️ Chat asíncrono
- Boletín UD 10 — Avanzado (Resuelto)
    - 1. Backoff exponencial
    - 2. Timeout con respaldo

    - 3. Dos heartbeats
    - 4. 🎯 Web scraper asíncrono
    - 5. 🔍 Servidor asyncio con heartbeat
    - 6. ⏱️ Monitorización de servidores
    - 7. 🧩 Semáforo asyncio
    - 8. 🎭 Timeout con fallback
    - 9. ⏳ Chat asíncrono
- Boletín UD 11 — Inicial
    - 1. Hola, Java
    - 2. Variables y tipos
    - 3. Lista de la compra
    - 4. Clase Alumno
    - 5. Tu primer @RestController
    - 6. Parámetro en la ruta
    - 7. JSON de respuesta
    - 8. Entidad JPA mínima
- Boletín UD 11 — Inicial (Resuelto)
    - 1. Hola, Java
    - 2. Variables y tipos
    - 3. Lista de la compra
    - 4. Clase Alumno
    - 5. Tu primer @RestController
    - 6. Parámetro en la ruta
    - 7. JSON de respuesta
    - 8. Entidad JPA mínima
- Boletín UD 11 — Avanzado
    - 1. ⏳ Servicio entre controlador y repositorio
    - 2. 📋 Consultas personalizadas JPA
    - 3. ✅ Validación con anotaciones
    - 4. 🧪 Primer test con JUnit
    - 5. 🔍 Test de integración del controlador
    - 6. ❌ Gestión de errores centralizada
    - 7. 🗃️ Migración con Flyway

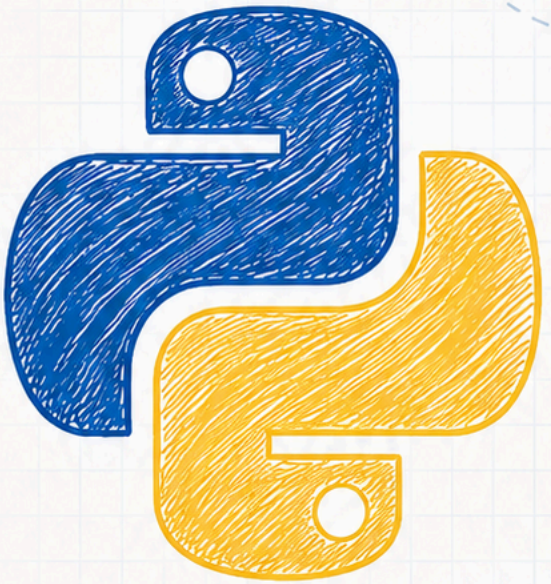
    - 8. 🔐 Seguridad básica con Spring Security
    - 9. 📦 Paginación en la API
    - 10. 🚀 Mini-proyecto completo
- Boletín UD 11 — Avanzado (Resuelto)
    - 1. 🧬 Servicio entre controlador y repositorio
    - 2. 📋 Consultas personalizadas JPA
    - 3. ✅ Validación con anotaciones
    - 4. 💉 Primer test con JUnit
    - 5. 🔍 Test de integración del controlador
    - 6. ❌ Gestión de errores centralizada
    - 7. 🗃️ Migración con Flyway
    - 8. 🔐 Seguridad básica con Spring Security
    - 9. 📦 Paginación en la API
    - 10. 🚀 Mini-proyecto completo

Inicio

APUNTES

Programación de Servicios y Procesos

(Con Python)



```python
import time          # importaciones
import socket
from multiprocessing import Process

def trabajo(nombre):          # definición de función
    print(f"Iniciando {nombre}...")
    time.sleep(2)
    print(f"{nombre} finalizado.")

if __name__ == "__main__":          # ejecución del proceso
    p = Process(target=trabajo, args=("Proceso",))
    p.start()
    p.join()
```

Por
Sergi García Barea

licencia
CC BY SA

Empezar por la Unidad 1 | Ver en GitHub

Sergi Garcia Barea — CC BY-SA 4.0

¿Prefieres estudiar sin conexión? Descarga el curso completo

PDF | EPUB

Unidades

UNIDAD 1 — Prerequisito

Python 3 básico

Variables, tipos, colecciones, control de flujo, funciones, clases, módulos, generadores y decoradores.

Ver unidad

UNIDAD 2 — RA1

Gestión de procesos con Python

Módulo `subprocess`, ejecución de comandos, captura de salida, control de errores, tuberías (pipes).

Ver unidad

UNIDAD 3 — RA2

Hilos y concurrencia con Python

Módulo `threading`, creación de hilos, `join`, `daemon`, `Timer`, GIL, ciclo de vida.

Ver unidad

UNIDAD 4 — RA2

Sincronización

`Lock`, `Semaphore`, `Barrier`, `Condition`, `RLock`, productor-consumidor.

Ver unidad

UNIDAD 5 — RA3

Sockets TCP y UDP

TCP/UDP, cliente/servidor, `socket`, `bind`, `listen`, `accept`, errores, `SO_REUSEADDR`.

Ver unidad

UNIDAD 6 — RA4c-d

Servidores concurrentes con Python

Servidor multihilo, `ThreadPool`, `concurrent.futures`, benchmark.

Ver unidad

UNIDAD 7 RA4a-b

HTTP y APIs REST

REST, métodos HTTP, `requests`, JSON, códigos de estado, consumo de APIs.

👉 Ver unidad 👈

UNIDAD 8 RA4a-b

APIs Comerciales

OpenWeatherMap, OpenAI, `python-dotenv`, rate limit, gestión de errores, autenticación.

👉 Ver unidad 👈

UNIDAD 9 RA5

Seguridad y cifrado con Python

Hash, contraseñas con sal, AES, RSA, cifrado híbrido, firmas digitales, RBAC.

👉 Ver unidad 👈

UNIDAD 10 RA4e-g

Alta disponibilidad

`asyncio`, `async` / `await`, heartbeat, reintentos con backoff, timeouts, alta disponibilidad.

👉 Ver unidad 👈

UNIDAD 11

Spring Boot con Java

Del Python al ecosistema Java: Spring Boot, DI, REST, JPA.

👉 Ver unidad 👈

📝 Boletines de ejercicios

Cada unidad tiene **4 boletines** (44 en total): inicial y avanzado, con y sin solución.

UNIDAD 1

- 🟢 Inicial por resolver
- ✅ Inicial resuelto
- ⭐ Avanzado por resolver
- 💪 Avanzado resuelto

UNIDAD 2

- 🟢 Inicial por resolver
- ✅ Inicial resuelto
- ⭐ Avanzado por resolver
- 💪 Avanzado resuelto

UNIDAD 3

- 🟢 Inicial por resolver
- ✅ Inicial resuelto
- ⭐ Avanzado por resolver
- 💪 Avanzado resuelto

UNIDAD 4

- 🟢 Inicial por resolver
- ✅ Inicial resuelto
- ⭐ Avanzado por resolver
- 💪 Avanzado resuelto

UNIDAD 5

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 6

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 7

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 8

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 9

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 10

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

UNIDAD 11

Inicial por resolver | Inicial resuelto | Avanzado por resolver | Avanzado resuelto

Licencia

CC BY-SA 4.0 — *Sergi Garcia Barea*

Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional

Puedes compartir y adaptar el material para cualquier propósito, incluso comercial, siempre que reconozcas la autoría y compartas las modificaciones bajo la misma licencia.

U01 — Python 3 básico

La base de todo: variables, funciones, clases y módulos en Python

> 🗺️ **Ruta del viaje:** **Python básico** → 🚀 Procesos y Subprocess → 🔀 Hilo → 🔒 Sincronización → 🔌 TCP → 📡 UDP → 🌐 API REST → 💉 APIs comerciales → 🔐 Hash → 🧬 Cifrado → 🏗️ Servidores → ⏱️ asyncio

---

Python es el lenguaje que da nombre al módulo y el que vas a usar en todas las unidades siguientes. Esta unidad es prerequisito: cubre todo lo que necesitas saber de Python antes de lanzarte a crear procesos, hilos y servidores. Si ya dominas variables, funciones, clases y módulos, puedes saltarla y empezar directamente por la UD 2. Si tienes dudas, quédate aquí: cada concepto se explica con ejemplos cortos que puedes copiar y ejecutar en tu terminal.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Escribir comentarios de línea y multilinea en Python.
- Distinguir entre tipos primitivos (entero, float, boolean, string, None) y usar operadores aritméticos, de comparación y lógicos.
- Crear y manipular variables, listas, tuplas, diccionarios y conjuntos.
- Usar slicing, comprensiones y el operador `in`.
- Controlar el flujo de un programa con `if`, `for`, `while` y `try/except`.
- Definir funciones con argumentos posicionales, `*args`, `**kwargs` y funciones lambda.
- Crear clases con atributos, métodos, `@classmethod` y `@staticmethod`.
- Importar módulos, instalar paquetes con `pip` y usar generadores y decoradores.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Introducción a Python	Qué es Python, por qué aprenderlo, características e instalación	Todos
02 · Comentarios	Comentario de línea, docstring multilinea	Todos
03 · Tipos de datos y operadores	Enteros, floats, booleans, strings, None, operadores	Todos
04 · Variables y colecciones	Variables, listas, tuplas, diccionarios, conjuntos, slicing	Todos
05 · Control de flujo	if, for, while, range, try/except, iteradores	Todos
06 · Funciones	def, argumentos, lambda, map, filter, comprensiones	Todos
07 · Clases	class, **init**, métodos, classmethod, staticmethod	Todos
08 · Módulos y avanzado	import, pip, generadores, decoradores	Intermedio
09 · Cierre	Sé el código, Fireside, Laboratorio de tortura…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (Prerequisito)

Esta unidad es **prerequisito** del módulo PSP. No corresponde a ningún RA directamente, pero sin ella no podrás seguir las unidades siguientes.

CE	Criterio	Dónde se cubre
—	Domina la sintaxis básica de Python (variables, tipos, operadores)	✅ Puntos 3 y 4
—	Controla el flujo de ejecución (condicionales, bucles, excepciones)	✅ Punto 5
—	Define y llama a funciones con diferentes tipos de argumentos	✅ Punto 6
—	Crea y usa clases con atributos y métodos	✅ Punto 7
—	Importa módulos y gestiona paquetes externos	✅ Punto 8

> Estos conocimientos son **necesarios** para todas las unidades del módulo (UD 1–UD 10).

🚪 ¿Por dónde empiezo?

¿Nunca has tocado un ordenador o un lenguaje de programación? No empieces por aquí: primero repasa los fundamentos en el curso de Pensamiento Computacional (variables, bucles, condicionales). Una vez lo domines, vuelve a esta unidad para afianzar Python.

¿Nunca has programado en Python? Perfecto, esta unidad es para ti. Empieza por el punto 1 y avanza despacio: cada capítulo tiene ejemplos que puedes copiar y ejecutar en tu terminal. No te saltes los boletines: la práctica es la clave.

¿Ya sabes algo de Python pero no estás seguro? Haz el mini-chequeo del punto 3. Si lo sacas sin problemas, puedes saltar a los puntos que menos domines y dedicar más tiempo al punto 8 (generadores y decoradores), que suelen ser los que más cuestan.

📍 **Primer punto:** 01 · Introducción a Python ⏭️ **Al acabar la unidad, continúa en U02 · Procesos y Subprocess.**

01 — Introducción a Python

Qué es Python, por qué aprenderlo y cómo empezar

> 🗺️ **Estás en:** **U01 · Python 3 básico** → 01 · Introducción a Python

📫 La idea en una frase

> Python es un lenguaje interpretado, de tipado dinámico y sintaxis limpia, diseñado para que programas se lean como pseudocódigo. Es la puerta de entrada ideal a la programación y el lenguaje que usaremos en todo el módulo PSP.

¿Por qué Python y no otro lenguaje? Porque en pocas líneas consigues mucho resultado, la comunidad es enorme y la documentación, excelente. Además, es el lenguaje más usado en servidores, automatización y ciencia de datos. Si aprendes bien Python, no solo apruebas este módulo: abres puertas a medio mundo.

¿Qué es Python?

Python es un lenguaje de programación de **alto nivel**, **interpretado** y **multiplataforma**. Creado por Guido van Rossum en 1991, hoy es uno de los lenguajes más populares del mundo.

Algunas características clave:

Característica	Qué significa en la práctica
Interpretado	No necesitas compilar. Escribes el código y lo ejecutas directamente.
Tipado dinámico	No declares tipos: `x = 5` y luego `x = "hola"` funciona sin problemas.
Sintaxis limpia	El código se lee casi como inglés. Las indentaciones obligatorias fuerzan código limpio.
Multiplataforma	Funciona en Windows, Linux y macOS sin cambiar nada.

Característica	Qué significa en la práctica
Biblioteca estándar enorme	`os`, `sys`, `threading`, `socket`, `json`… todo incluido.
Comunidad masiva	Casi cualquier duda que tengas, ya alguien la respondió en Stack Overflow.

🤔 ¿Por qué Python para PSP?

En este módulo vamos a crear procesos, hilos, servidores, clientes y APIs. Todo eso se hace con Python porque:

- **`subprocess`** y **`threading`** están en la biblioteca estándar: no necesitas instalar nada.
- **`socket`** permite crear servidores TCP y UDP en pocas líneas.
- **`requests`** y **`httpx`** hacen peticiones HTTP fáciles.
- **`asyncio`** permite programación asíncrona para servidores concurrentes.
- La sintaxis clara reduce errores cuando estás aprendiendo conceptos nuevos.

🖥️ Instalar Python

Windows

1. Ve a python.org/downloads.
2. Descarga la última versión (3.10 o superior).
3. **Importante:** al instalar, marca la casilla **"Add Python to PATH"**.
4. Abre una terminal y verifica:

```
python --version
```

Deberías ver algo como `Python 3.11.5`.

Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install python3 python3-pip
python3 --version
```

macOS

```
brew install python3
python3 --version
```

📦 Sugerencia: entorno virtual (venv)

Antes de instalar paquetes con `pip`, merece la pena crear un **entorno virtual**. Un entorno virtual es una carpeta aislada con su propia instalación de Python y sus propios paquetes, separada del sistema. Así evitas conflictos entre proyectos.

Windows

```
# Crear el entorno (una sola vez por proyecto)
python -m venv venv

# Activarlo
venv\Scripts\activate

# Verificar que funciona (deberías ver "(venv)" al inicio del prompt)
python --version

# Desactivar cuando acabes
deactivate
```

Linux / macOS

```
# Crear el entorno
python3 -m venv venv

# Activarlo
source venv/bin/activate

# Verificar
python --version

# Desactivar
deactivate
```

¿Por qué usar venv?

- **Aislamiento:** los paquetes que instales solo están en ese entorno, no en todo el sistema.
- **Reproducibilidad:** puedes exportar la lista de paquetes con `pip freeze > requirements.txt` y recrear el entorno en otro ordenador.
- **No rompes nada:** si un paquete actualizado te rompe un proyecto, los demás proyectos no se ven afectados.

> 💡 **Consejo:** crea siempre un entorno virtual antes de empezar un proyecto. La carpeta `venv/` se añade al `.gitignore` porque no se sube al repositorio: cada quien instala sus paquetes localmente.

💻 Tu primer programa

Abre un editor de texto (VS Code, PyCharm o incluso el Bloc de notas) y escribe:

```
print("¡Hola, mundo!")
```

Guarda como `hola.py` y ejecuta en la terminal:

```
python hola.py
```

Salida:

```
¡Hola, mundo!
```

¿Ha funcionado? Enhorabuena: ya tienes Python instalado y sabes ejecutar un programa.

🧪 La consola interactiva

Python ofrece una consola interactiva donde puedes probar código línea a línea. Abre una terminal y escribe:

```
python
```

Verás el prompt `>>>`. Ahí puedes escribir expresiones y ver el resultado al instante:

```
>>> 2 + 3
5
>>> "hola".upper()
'HOLA'
>>> exit()
```

La consola interactiva es perfecta para probar cosas rápido antes de meterlas en un archivo.

🧠 Mini-chequeo

1. ¿Qué pasa si no marcas “Add Python to PATH” al instalar en Windows?
2. ¿Cuál es la diferencia entre el archivo `.py` y la consola interactiva?
3. ¿Python necesita compilar los programas antes de ejecutarlos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Python es un lenguaje interpretado, de tipado dinámico y sintaxis limpia, ideal para empezar.

- Para instalarlo, ve a python.org (Windows) o usa el gestor de paquetes (Linux/macOS) y verifica con `python --version`.
- Tu primer programa es un `print("¡Hola, mundo!")`: guárdalo en un `.py` y ejecútalo con `python hola.py`.

🐛 Vocabulario rápido

Término	Idea general
Interpretado	Se ejecuta línea a línea, sin compilar a binario
Tipado dinámico	Las variables no tienen tipo fijo; puede cambiar en cualquier momento
Alto nivel	Se acerca al lenguaje humano; el intérprete se encarga de la máquina
PATH	Variable del sistema que dice dónde buscar ejecutables
Consola interactiva	Modo de Python para probar código línea a línea
Módulo	Archivo `.py` con funciones y clases reutilizables
venv	Entorno virtual: carpeta aislada con su propio Python y paquetes

---

📚 Volver al índice de la unidad · **Siguiente:** 02 · Comentarios

02 — Comentarios

Comentarios de línea y docstrings multilinea 📝

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 02 · Comentarios

📬 La idea en una frase

> Los comentarios explican el código a quien lo lea (incluido tú dentro de tres meses). En Python se escriben con # para una línea y con """ para bloques multilinea.

Un comentario es texto que Python **ignora por completo** al ejecutar. Sirve para documentar, explicar la lógica o dejar notas. Es la diferencia entre código que funciona y código que funciona **y se entiende**.

📝 Comentario de línea

El comentario de línea se escribe con el carácter #. Todo lo que haya a la derecha de # en esa línea se ignora:

```
# Este es un comentario completo
x = 5  # Este comentario va al final de la línea de código
```

Buena práctica: separa el comentario del código con al menos un espacio:

```
# Bien
x = 5  # valor inicial

# Mal
x = 5  #valor inicial
```

Cuándo usar comentarios

- Para explicar **por qué** haces algo, no **qué** hace el código (eso se lee solo).
- Para dejar una nota temporal: `# TODO: mejorar esto más adelante`.
- Para bloquear código que no quieres ejecutar de momento:

```
# print("Esta línea no se ejecuta")
x = 10
```

📄 Docstring (comentario multilinea)

Para comentarios de varias líneas se usan **tres comillas dobles** `"""`. Esto se llama **docstring** y se usa especialmente para documentar funciones, clases y módulos:

```
"""
Este es un comentario multilinea.
Puede ocupar las líneas que necesites.
Python lo ignora completamente.
"""
```

Docstrings en funciones

Cuando pones un docstring como **primera instrucción** de una función, se convierte en su documentación oficial:

```
def sumar(a, b):
    """
    Suma dos números y devuelve el resultado.

    Parámetros:
        a (int o float): primer número
        b (int o float): segundo número

    Retorna:
        int o float: la suma de a y b
    """
    return a + b
```

Puedes acceder a ese docstring con `help()` o con `sumar.__doc__`:

```
help(sumar)
# Salida: Suma dos números y devuelve el resultado...
```

Docstrings en clases

Lo mismo aplica a las clases:

```
class Persona:
    """
    Representa una persona con nombre y edad.

    Atributos:
        nombre (str): nombre de la persona
        edad (int): edad en años
    """

    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
```

🔄 Comillas simples vs dobles

Python acepta comillas simples `'` y dobles `"` para strings. Para docstrings se usan `"""` (tres comillas dobles) por convención, aunque `'''` (tres comillas simples) también funciona:

```
# Todos son strings válidos
'Hola'
"Hola"
'''Hola'''
"""Hola"""
```

La convención del PEP 257 (guía de estilo de Python) recomienda `"""` para docstrings.

⚠️ Errores comunes

Olvidar la segunda comilla

```
# Error de sintaxis
"Hola mundo
# Falta la comilla de cierre
```

Comentario sin espacio tras

```
# Funciona pero es feo
x = 5  #esto funciona
x = 5  # esto es mejor
```

Confundir string con docstring

```
# Esto es un string asignado a una variable, no un docstring
mensaje = """
Esto es un string multilinea.
No documenta nada.
"""

# Esto SÍ es un docstring (primera instrucción de la función)
def saludar():
    """
    Saluda al usuario.
    """
    print("Hola")
```

🧠 Mini-chequeo

1. ¿Qué personaje se usa para comentarios de una línea en Python?
2. ¿Cómo se escribe un docstring de varias líneas?
3. ¿Un docstring es obligatorio en una función?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Los comentarios de línea usan # y se ignoran al ejecutar.
- Los docstrings usan """ y documentan funciones, clases y módulos.
- Comenta el **por qué**, no el **qué**: el código debe ser autoexplicativo.

🐛 Vocabulario rápido

Término	Idea general
Comentario	Texto ignorado por Python que explica el código
Docstring	Comentario multilinea con """ que documenta funciones/clases
PEP 257	Convención de Python para escribir docstrings
Autoexplicativo	Código que se entiende solo por su nombre y estructura

📚 Volver al índice de la unidad · **Anterior:** 01 · Introducción a Python · **Siguiente:** 03 · Tipos de datos y operadores

03 — Tipos de datos y operadores

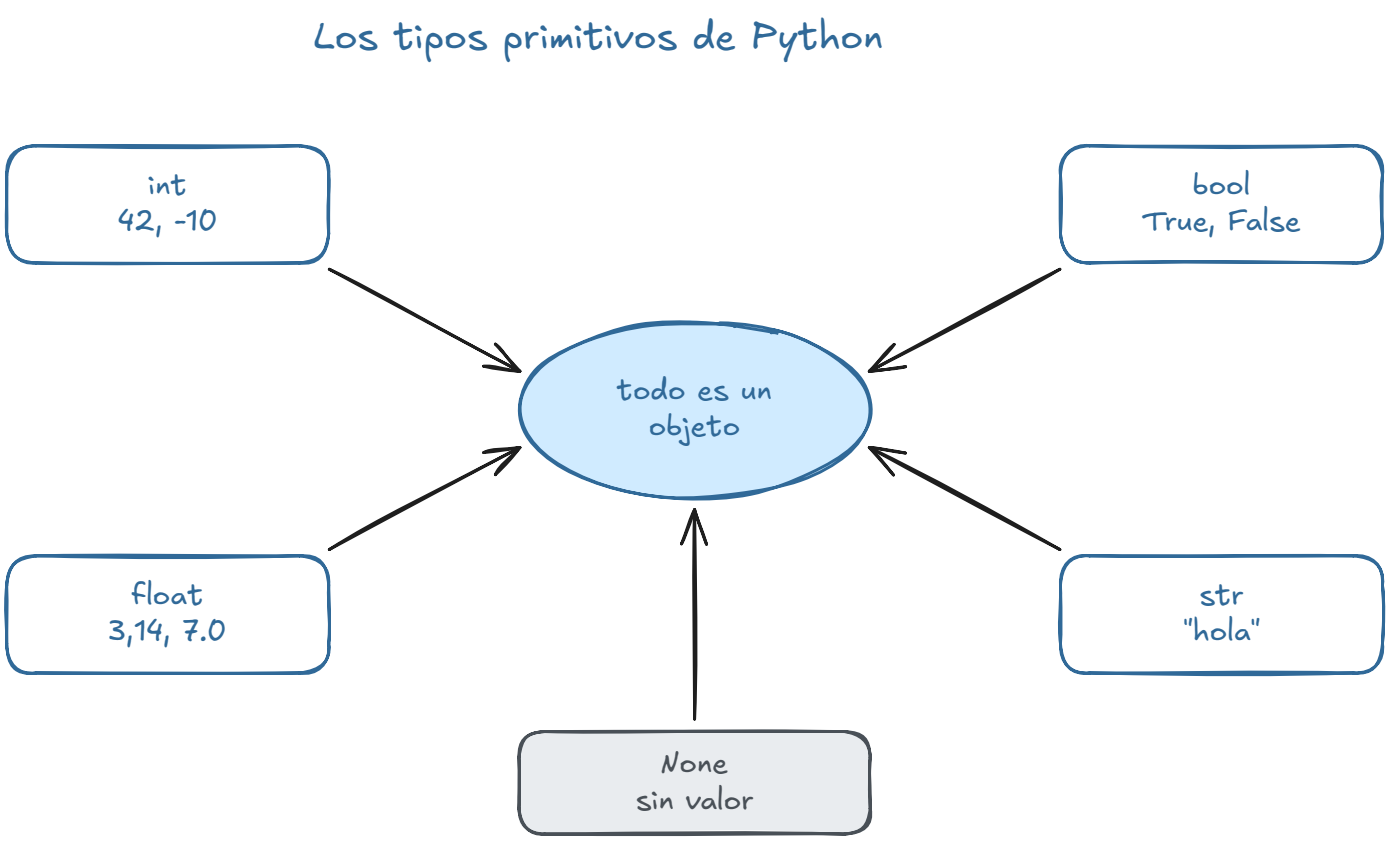
Enteros, floats, booleans, strings, None y sus operadores 🔢

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 03 · Tipos de datos y operadores

📬 La idea en una frase

> Todo en Python es un objeto con un tipo. Los tipos primitivos son enteros, floats, booleans, strings y None. Cada tipo tiene sus operadores, y Python deduce el tipo automáticamente.

No necesitas declarar tipos como en C o Java. Python **adivina** el tipo cuando asignas un valor. Si escribas `x = 5`, `x` es un entero. Si luego escribes `x = "hola"`, pasa a ser un string. Eso es el **tipado dinámico**.



tipado dinámico: Python deduce el tipo al asignar, sin declararlo

🔢 Números enteros (int)

Los enteros no tienen límite de tamaño: puedes tener números enormes sin preocuparte.

```
3       # => 3
-10     # => -10
1000000 # => 1000000
```

Operaciones aritméticas

```
1 + 1       # => 2
8 - 1       # => 7
10 * 2      # => 20
(1 + 3) * 2  # => 8   (paréntesis refuerzan la precedencia)
```

División

La división con `/` **siempre** devuelve un float (decimal), incluso entre enteros:

```
35 / 5   # => 7.0  (float, no int)
34 / 5   # => 6.8
```

Si quieres un resultado entero con truncado de decimales, usa `//`:

```
34 // 5  # => 6   (trunca los decimales)
35 // 5  # => 7
```

Potencia y módulo

```
2 ** 3    # => 8   (2 elevado a 3)
10 % 3    # => 1   (resto de la división: 10 / 3 = 3 resto 1)
```

🏊 Números decimales (float)

Cuando un operador es float, el resultado siempre es float:

```
3 * 2.0    # => 6.0
3.5 + 1.2  # => 4.7
```

Cuidado con la precisión de los floats:

```
0.1 + 0.2  # => 0.30000000000000004  (no exacto)
```

Esto no es un error de Python: es cómo funciona el formato de punto flotante en todos los lenguajes. Para cálculos que requieren precisión exacta (dinero, por ejemplo), se usa el módulo `decimal`.

✅ Booleanos (bool)

Dos valores: `True` y `False` (con mayúscula inicial).

```
True
False
```

Operador lógico `not`

```
not True   # => False
not False  # => True
```

Comparaciones (devuelven bool)

```
1 == 1     # => True    (igualdad)
2 == 1     # => False
1 != 1     # => False   (desigualdad)
2 != 1     # => True
1 < 10     # => True
1 > 10     # => False
2 <= 2     # => True
2 >= 2     # => True
```

Comparaciones encadenadas

Python permite encadenar comparaciones, como en las matemáticas:

```
1 < 2 < 3    # => True   (equivalente a 1 < 2 and 2 < 3)
2 < 3 < 2    # => False
```

📝 Strings (str)

Los strings son cadenas de texto. Se crean con comillas simples ' o dobles ":

```
"Hola mundo"
'Hola mundo'
```

Concatenación

```
"Hola " + "mundo!"   # => "Hola mundo!"
```

Acceso a caracteres

Un string se puede tratar como una lista de caracteres (los índices empiezan en 0):

```
"Hola"[0]    # => 'H'
"Hola"[-1]   # => 'o'  (el último carácter)
```

Formato de strings

Método `.format()`

```
"{} pueden ser {}".format("strings", "interpolados")
# => "strings pueden ser interpolados"

# Reutilizar argumentos por posición
"{0} sé ligero, {0} sé rápido".format("Jack")
# => "Jack sé ligero, Jack sé rápido"

# Usar palabras clave
"{nombre} quiere comer {comida}".format(nombre="Bob", comida="lasaña")
# => "Bob quiere comer lasaña"
```

f-strings (Python 3.6+)

La forma moderna y más cómoda: pon `f` antes de la comilla e incluye variables entre `{}`:

```
nombre = "Bob"
comida = "Lasaña"
f"{nombre} quiere comer {comida}"   # => "Bob quiere comer lasaña"
```

Las f-strings son la forma recomendada en Python moderno.

Métodos útiles

```
"Hola Mundo".lower()      # => "hola mundo"
"Hola Mundo".upper()      # => "HOLA MUNDO"
"Hola Mundo".replace("Mundo", "Python")  # => "Hola Python"
"hola mundo".split()      # => ["hola", "mundo"]
"  hola  ".strip()        # => "hola" (elimina espacios al inicio y final)
```

None

`None` es el valor que representa “nada”. Es un objeto único de tipo `NoneType`:

```
None   # => None
```

Comparar con None

No uses `==` para comparar con None. Usa `is`:

```
"etc" is None   # => False
None is None    # => True
```

Valores que se evalúan como False

`None`, `0`, strings vacíos, listas vacías, diccionarios vacíos y conjuntos vacíos se evalúan como `False` en un contexto booleano:

```
bool(0)     # => False
bool("")    # => False
bool([])    # => False
bool({})    # => False
bool(set()) # => False
```

Todo lo demás se evalúa como `True`.

🔧 Funciones de conversión

```
int(3.7)      # => 3    (trunca, no redondea)
float(5)      # => 5.0
str(100)      # => "100"
bool(1)       # => True
bool(0)       # => False
bool("hola")  # => True
bool("")      # => False
```

🧠 Mini-chequeo

1. ¿Qué devuelve `10 / 2` en Python 3? ¿Y `10 // 2`?
2. ¿Cuál es la diferencia entre `==` y `is`?
3. ¿Por qué `0.1 + 0.2` no da exactamente `0.3`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Los tipos primitivos son `int`, `float`, `bool`, `str` y `None`; Python deduce el tipo automáticamente.
- La división `/` devuelve float; `//` devuelve entero con truncado.
- Usa `is` para comparar con None y f-strings para formatear texto de forma moderna.

🐛 Vocabulario rápido

Término	Idea general
int	Número entero, sin límite de tamaño
float	Número decimal (punto flotante)
bool	Booleano: `True` o `False`
str	String, cadena de texto
None	Valor que representa “nada”
Tipado dinámico	El tipo se deduce al asignar, no se declara
f-string	Formato de strings moderno con `f"..."`
Concatenación	Unir strings con +

📚 Volver al índice de la unidad · **Anterior:** 02 · Comentarios · **Siguiente:** 04 · Variables y colecciones

04 — Variables y colecciones

Variables, listas, tuplas, diccionarios, conjuntos y slicing 📦

> 🗺️ **Estás en:** **U01 · Python 3 básico** → 04 · Variables y colecciones

📬 La idea en una frase

> Las variables almacenan datos y las colecciones agrupan varios datos en una sola estructura. Python ofrece listas (mutables), tuplas (inmutables), diccionarios (clave:valor) y conjuntos (sin duplicados).

Las colecciones son la base de casi todo programa. Si sabes manejarlas bien, el resto del módulo será mucho más fácil.

📌 Variables

En Python no hace falta declarar variables antes de usarlas. Simplemente asignas un nombre a un valor:

```
nombre = "Ana"
edad = 22
activa = True
```

Convenciones de nombres

La convención oficial (PEP 8) es usar **snake_case**: palabras en minúscula separadas por guion bajo:

```
# Bien
mi_variable = 5
nombre_completo = "Ana García"

# Mal (pero funciona)
miVariable = 5        # CamelCase
MiVariable = 5        # PascalCase
```

Errores comunes

Si accedes a una variable que no existe, Python lanza un error `NameError`:

```
otra_variable  # NameError: name 'otra_variable' is not defined
```

📋 Listas

Las listas son la colección más usada en Python. Almacenan una secuencia de elementos y son **mutables** (puedes cambiarlas después de crearlas).

```
# Lista vacía
lista = []

# Lista prellenada
otra_lista = [4, 5, 6]
```

Añadir y eliminar elementos

```
lista.append(1)     # lista = [1]
lista.append(2)     # lista = [1, 2]
lista.append(4)     # lista = [1, 2, 4]
lista.append(3)     # lista = [1, 2, 4, 3]

lista.pop()         # => 3  (elimina y devuelve el último)
# lista ahora es [1, 2, 4]
```

Acceso por índice

Los índices empiezan en 0. Python también permite índices negativos (-1 es el último):

```
lista = [1, 2, 4, 3]

lista[0]     # => 1
lista[-1]    # => 3
lista[4]     # IndexError: list index out of range
```

Slicing (rebanado)

El slicing extrae una porción de la lista con la sintaxis `lista[inicio:final:pasos]`:

```
lista = [1, 2, 4, 3]

lista[1:3]    # => [2, 4]      (del índice 1 al 2, sin incluir el 3)
lista[2:]     # => [4, 3]      (desde el índice 2 hasta el final)
lista[:3]     # => [1, 2, 4]   (desde el inicio hasta el índice 2)
lista[::2]    # => [1, 4]      (cada dos elementos)
lista[::-1]   # => [3, 4, 2, 1]  (invertir la lista)
```

Otras operaciones

```
del lista[2]           # Elimina el elemento en índice 2 (el 4): lista ahora es [1,
2, 3]

lista + otra_lista     # => [1, 2, 3, 4, 5, 6] (no modifica las originales)

lista.extend(otra_lista)  # Añade otra_lista al final de lista (modifica lista)

1 in lista             # => True (comprobar existencia)

len(lista)             # => 6 (longitud)
```

🔒 Tuplas

Las tuplas son como las listas, pero **inmutables**: una vez creadas, no puedes cambiar nada.

```
tupla = (1, 2, 3)
tupla[0]       # => 1
tupla[0] = 3   # TypeError: 'tuple' does not support item assignment
```

Operaciones válidas

```
len(tupla)              # => 3
tupla + (4, 5, 6)       # => (1, 2, 3, 4, 5, 6)
tupla[:2]               # => (1, 2)
2 in tupla              # => True
```

Desempaquetado

Una de las cosas más elegantes de Python: puedes asignar múltiples variables de golpe:

```
a, b, c = (1, 2, 3)    # a=1, b=2, c=3

# Sin paréntesis (Python crea la tupla automáticamente)
d, e, f = 4, 5, 6

# Intercambiar valores sin variable temporal
e, d = d, e             # d=5, e=4
```

📖 Diccionarios

Los diccionarios almacenan pares **clave:valor**. Son ideales para datos estructurados.

```
# Diccionario vacío
dicc_vacio = {}

# Diccionario prellenado
dicc_lleno = {"uno": 1, "dos": 2, "tres": 3}
```

Acceder a valores

```
dicc_lleno["uno"]       # => 1

# Buscar una clave inexistente lanza KeyError
dicc_lleno["cuatro"]    # KeyError
```

Método get (seguro)

```
dicc_lleno.get("uno")        # => 1
dicc_lleno.get("cuatro")     # None (no lanza error)
dicc_lleno.get("cuatro", 4)  # => 4 (valor por defecto)
```

Claves y valores

```
list(dicc_lleno.keys())    # => ["uno", "dos", "tres"]
list(dicc_lleno.values())  # => [1, 2, 3]
```

Añadir, modificar y eliminar

```
dicc_lleno["cuatro"] = 4       # Añadir o modificar
dicc_lleno.setdefault("cinco", 5)  # Solo si la clave no existe

del dicc_lleno["uno"]          # Eliminar una clave
```

Comprobar existencia

```
"uno" in dicc_lleno    # => True
1 in dicc_lleno        # => False (1 es un valor, no una clave)
```

🔵 Conjuntos (set)

Los conjuntos almacenan elementos **sin duplicados** y permiten operaciones de teoría de conjuntos.

```
# Conjunto vacío (¡cuidado! {} crea un dict, no un set)
conjunto_vacio = set()

# Conjunto con valores (los duplicados se eliminan automáticamente)
un_conjunto = {1, 2, 2, 3, 4}   # => {1, 2, 3, 4}
```

Operaciones de conjuntos

```
otro_conjunto = {3, 4, 5, 6}

# Intersección (elementos comunes)
un_conjunto & otro_conjunto    # => {3, 4}

# Unión (todos los elementos)
un_conjunto | otro_conjunto    # => {1, 2, 3, 4, 5, 6}

# Diferencia (en uno pero no en otro)
{1, 2, 3, 4} - {2, 3, 5}     # => {1, 4}
```

Añadir y comprobar

```
un_conjunto.add(5)    # => {1, 2, 3, 4, 5}
2 in un_conjunto      # => True
10 in un_conjunto     # => False
```

🧠 Mini-chequeo

1. ¿Cuál es la diferencia entre una lista y una tupla?
2. ¿Qué devuelve `{1, 2, 2, 3}`?
3. ¿Cómo accedes al último elemento de una lista `mi_lista`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Las variables no se declaran: simplemente asignas un valor con `=`.
- Las listas son mutables y usan índices; las tuplas son inmutables; los diccionarios usan claves; los conjuntos eliminan duplicados.
- El slicing `lista[inicio:final:pasos]` extrae porciones de listas y strings de forma elegante.

🐛 Vocabulario rápido

Término	Idea general
snake_case	Convención de nombres: minúsculas con guion bajo
Mutable	Que puede cambiar después de creado
Inmutable	Que no puede cambiar
Slicing	Extraer una porción de una secuencia
Desempaquetado	Asignar múltiples variables de golpe
Clave:valor	Par de datos en un diccionario
Set	Conjunto sin duplicados

📚 Volver al índice de la unidad · **Anterior:** 03 · Tipos de datos y operadores · **Siguiente:** 05 · Control de flujo

05 — Control de flujo

if, for, while, range, try/except e iteradores 🔀

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 05 · Control de flujo

---

📬 La idea en una frase

> El control de flujo decide **qué código se ejecuta y cuándo**. Las tres estructuras básicas son la condición (`if`), la iteración (`for` y `while`) y la gestión de errores (`try/except`).

Sin control de flujo, todo programa sería una lista lineal de instrucciones. Con él, tu programa puede tomar decisiones, repetir tareas y sobrevivir a los errores.

🔀 Condicional: if / elif / else

```
una_variable = 5

if una_variable > 10:
    print("Es mayor que 10")
elif una_variable < 10:    # elif es opcional
    print("Es menor que 10")
else:                       # else también es opcional
    print("Es exactamente 10")
```

Reglas clave

- **La indentación es obligatoria.** Python usa sangría (4 espacios por convención) para saber qué código pertenece a cada bloque.
- `elif` y `else` son opcionales.
- Puedes encadenar cuantos `elif` quieras:

```python
nota = 7

if nota >= 9:
    print("Sobresaliente")
elif nota >= 7:
    print("Notable")
elif nota >= 5:
    print("Aprobado")
else:
    print("Suspenso")
```

🔄 Bucle: for

`for` itera sobre cualquier **iterable**: listas, tuplas, strings, diccionarios, rangos…

```python
for animal in ["perro", "gato", "ratón"]:
    print(f"{animal} es un mamífero")
```

Salida:

```
perro es un mamífero
gato es un mamífero
ratón es un mamífero
```

La función range

`range(n)` genera una secuencia de números del 0 al n-1:

```python
for i in range(4):
    print(i)
```

Salida:

```
0
1
2
3
```

`range` acepta hasta 3 argumentos: `range(inicio, final, pasos)`:

```
range(0, 10, 2)   # => 0, 2, 4, 6, 8
range(5, 0, -1)   # => 5, 4, 3, 2, 1
```

Iterar con índice

Si necesitas el índice y el valor, usa `enumerate()`:

```
frutas = ["manzana", "plátano", "cereza"]

for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
```

Salida:

```
0: manzana
1: plátano
2: cereza
```

🔁 Bucle: while

`while` itera mientras una condición sea `True`:

```
x = 0
while x < 4:
    print(x)
    x += 1    # x = x + 1
```

Salida:

```
0
1
2
3
```

Cuidado con los bucles infinitos

Si la condición nunca se hace `False`, el bucle no acaba nunca:

```
# ¡Esto bloquea tu terminal!
while True:
    print("Esto no para nunca...")
```

Para salir de un bucle infinito, pulsa **Ctrl+C** en la terminal.

⚠️ Manejo de errores: try / except

Python lanza **excepciones** cuando algo falla. Puedes capturarlas con `try/except`:

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("No se puede dividir por cero")
```

Capturar el error

Puedes guardar la excepción en una variable para inspeccionarla:

```
try:
    numeros = [1, 2, 3]
    print(numeros[10])
except IndexError as e:
    print(f"Error: {e}")
```

Múltiples excepciones

```
try:
    x = int(input("Dame un número: "))
    resultado = 10 / x
except ValueError:
    print("No es un número válido")
except ZeroDivisionError:
    print("No se puede dividir por cero")
```

Bloque finally

`finally` se ejecuta **siempre**, haya fallado o no:

```
try:
    f = open("archivo.txt")
    contenido = f.read()
except FileNotFoundError:
    print("Archivo no encontrado")
finally:
    print("Esto se imprime siempre")
```

🔃 Iterables e iteradores

Un **iterable** es cualquier objeto sobre el que puedes hacer un `for`: listas, tuplas, strings, diccionarios, conjuntos, `range`…

```
mi_lista = [1, 2, 3]

# La función iter() crea un iterador
mi_iterador = iter(mi_lista)

next(mi_iterador)   # => 1
next(mi_iterador)   # => 2
next(mi_iterador)   # => 3
next(mi_iterador)   # StopIteration (ya no hay elementos)
```

Los iteradores son la base de los bucles `for`. Cuando escribes `for x in iterable`, Python usa un iterador por debajo.

🧩 Comprensiones (preview)

Las comprensiones permiten crear listas, diccionarios y conjuntos de forma concisa. Ya las veremos en detalle en el punto 6, pero aquí un adelanto:

```python
# Comprensión de lista
[ x ** 2 for x in range(5) ]   # => [0, 1, 4, 9, 16]

# Con condición
[ x for x in range(10) if x % 2 == 0 ]   # => [0, 2, 4, 6, 8]
```

🧠 Mini-chequeo

1. ¿Qué imprime `for i in range(3): print(i)`?
2. ¿Qué pasa si olvidas el `:` al final de un `if`?
3. ¿Cómo capturas un `ValueError` en un `try/except`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `if/elif/else` toma decisiones; la indentación define los bloques.
- `for` itera sobre iterables; `while` itera mientras haya condición.
- `try/except` captura errores sin que el programa se rompa.

🐛 Vocabulario rápido

Término	Idea general
Iterable	Objeto sobre el que se puede iterar (lista, string, range…)
Iterador	Objeto que recorre un iterable elemento a elemento
Excepción	Error que se lanza durante la ejecución
Indentación	Sangría (4 espacios) que define bloques de código
Comprensión	Sintaxis concisa para crear colecciones con un bucle

📚 Volver al índice de la unidad · **Anterior:** 04 · Variables y colecciones · **Siguiente:** 06 · Funciones

06 — Funciones

def, argumentos, lambda, map, filter y comprensiones ⚡

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 06 · Funciones

---

📬 La idea en una frase

> Una función es un bloque de código reutilizable que hace una sola cosa. Se define con `def`, se llama con paréntesis y puede recibir argumentos y devolver valores.

Las funciones son el primer paso hacia el código **modular**: escribes una vez, usas cuantas veces quieras. Sin funciones, cada programa sería un monolito imposible de mantener.

🔨 Definir y llamar funciones

```
def sumar(x, y):
    print(f"x es {x} y y es {y}")
    return x + y

# Llamada con argumentos posicionales
sumar(5, 6)   # imprime "x es 5 y y es 6", devuelve 11

# Llamada con argumentos de palabra clave (el orden no importa)
sumar(y=6, x=5)   # => 11
```

Reglas clave

- `def` seguido del nombre, paréntesis y dos puntos.
- El bloque de la función va indentado.
- `return` devuelve un valor. Sin `return`, la función devuelve `None`.

📦 Argumentos variables

*args

Recoge un número **variable** de argumentos posicionales en una tupla:

```
def varargs(*args):
    return args

varargs(1, 2, 3)   # => (1, 2, 3)
```

**kwargs

Recoge un número **variable** de argumentos de palabra clave en un diccionario:

```
def keyword_args(**kwargs):
    return kwargs

keyword_args(pie="grande", lago="ness")   # => {"pie": "grande", "lago": "ness"}
```

Combinar ambos

```
def todos_los_argumentos(*args, **kwargs):
    print(f"Args: {args}")
    print(f"Kwargs: {kwargs}")

todos_los_argumentos(1, 2, a=3, b=4)
# Args: (1, 2)
# Kwargs: {'a': 3, 'b': 4}
```

Desempaquetar al llamar

```
args = (1, 2, 3, 4)
kwargs = {"a": 3, "b": 4}

todos_los_argumentos(*args)          # equivalente a todos_los_argumentos(1, 2, 3, 4)
todos_los_argumentos(**kwargs)       # equivalente a todos_los_argumentos(a=3, b=4)
todos_los_argumentos(*args, **kwargs)
```

🏭 Funciones de primera clase

En Python, las funciones son objetos. Puedes pasarlas como argumentos, devolverlas y guardarlas en variables:

```
def crear_suma(x):
    def suma(y):
        return x + y
    return suma

sumar_10 = crear_suma(10)
sumar_10(3)   # => 13
```

Esto se llama **clausura**: la función `suma` "recuerda" el valor de `x` aunque `crear_suma` ya ha terminado.

🔹 Funciones lambda

Una función lambda es una función **anónima** (sin nombre) de una sola expresión:

```
# lambda argumentos: expresión
(lambda x: x > 2)(3)   # => True
```

Son útiles para argumentos de otras funciones:

```
# Funciones de orden superior
map(sumar_10, [1, 2, 3])               # => [11, 12, 13]
filter(lambda x: x > 5, [3, 4, 5, 6, 7])  # => [6, 7]
```

🔄 map, filter y reduce

map

Aplica una función a cada elemento de un iterable:

```
numeros = [1, 2, 3, 4]
cuadrados = list(map(lambda x: x ** 2, numeros))
# => [1, 4, 9, 16]
```

filter

Filtra elementos según una condición:

```
numeros = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
# => [2, 4, 6]
```

reduce

Acumula los elementos de un iterable en un solo valor (necesita importar `functools`):

```
from functools import reduce

numeros = [1, 2, 3, 4]
suma_total = reduce(lambda a, b: a + b, numeros)
# => 10
```

🧩 Comprensiones

Las comprensiones son la forma **pythonica** de crear listas, diccionarios y conjuntos a partir de iterables:

Comprensión de lista

```
# Bucle clásico
cuadrados = []
for x in range(5):
    cuadrados.append(x ** 2)

# Comprensión de lista (lo mismo en una línea)
cuadrados = [x ** 2 for x in range(5)]   # => [0, 1, 4, 9, 16]
```

Con condición

```
pares = [x for x in range(10) if x % 2 == 0]   # => [0, 2, 4, 6, 8]
```

Comprensión de diccionario

```
cuadrados_dict = {k: k ** 2 for k in range(3)}   # => {0: 0, 1: 1, 2: 4}
```

Comprensión de conjunto

```
letras = {c for c in "la cadena"}   # => {'d', 'l', 'a', 'n', ' ', 'c', 'e'}
```

🧠 Mini-chequeo

1. ¿Cuál es la diferencia entre `*args` y `**kwargs`?
2. ¿Qué devuelve `map(lambda x: x * 2, [1, 2, 3])` sin envolver en `list()`?
3. ¿Cómo se escribe una comprensión de lista que filtre solo los números impares del 1 al 10?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Se define con `def`, se llama con paréntesis y se devuelve valor con `return`.
- `*args` y `**kwargs` permiten argumentos variables; las lambda son funciones anónimas de una expresión.
- Las comprensiones crean listas, diccionarios y conjuntos de forma concisa y legible.

🐛 Vocabulario rápido

Término	Idea general
def	Palabra clave para definir una función
return	Devuelve un valor desde la función
args	Argumentos posicionales variables (tupla)

Término	Idea general
kwargs	Argumentos de palabra clave variables (diccionario)
Lambda	Función anónima de una sola expresión
Cláusura	Función que recuerda variables de su contexto
Comprensión	Sintaxis concisa para crear colecciones
map	Aplica una función a cada elemento
filter	Filtra elementos según una condición

📚 Volver al índice de la unidad · **Anterior:** 05 · Control de flujo · **Siguiente:** 07 · Clases

07 — Clases

class, __init__, métodos, classmethod y staticmethod 🏗️

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 07 · Clases

📬 La idea en una frase

> Una clase es un plano para crear objetos. Los objetos tienen atributos (datos) y métodos (funciones que operan con esos datos). Las clases organizan el código y permiten la reutilización.

Python es un lenguaje **orientado a objetos**. Todo es un objeto: un entero, un string, una lista… En esta unidad aprenderás a crear tus propias clases, que es donde la potencia de Python brilla de verdad.

🏗️ Definir una clase

```python
class Humano:
    # Atributo de clase (compartido por todas las instancias)
    especie = "H. sapiens"

    # Constructor
    def __init__(self, nombre):
        # Atributo de instancia (propio de cada objeto)
        self.nombre = nombre

    # Método de instancia
    def decir(self, msg):
        return f"{self.nombre}: {msg}"

    # Método de clase (compartido, recibe la clase como primer argumento)
    @classmethod
    def get_especie(cls):
        return cls.especie

    # Método estático (no recibe ni instancia ni clase)
    @staticmethod
    def roncar():
        return "*roncar*"
```

Diferencia entre tipos de métodos

Tipo	Primer argumento	Para qué sirve
Instancia	`self`	Opera con los datos de un objeto concreto
Clase	`cls`	Opera con la clase en general (no necesita instancia)
Estático	Ninguno	Función utilitaria que no depende de la clase ni de la instancia

🎭 Crear y usar objetos

```
# Instanciar una clase
i = Humano(nombre="Ian")
print(i.decir("hola"))     # => "Ian: hola"

j = Humano("Joel")
print(j.decir("hey"))      # => "Joel: hey"
```

Métodos de clase

```
i.get_especie()   # => "H. sapiens"

# Cambiar el atributo de clase
Humano.especie = "H. neanderthalensis"
i.get_especie()   # => "H. neanderthalensis"
j.get_especie()   # => "H. neanderthalensis"
```

Métodos estáticos

```
Humano.roncar()   # => "*roncar*"
```

🔑 El método `__init__`

`__init__` es el **constructor** de la clase. Se ejecuta automáticamente cuando creas un nuevo objeto:

```
class Coche:
    def __init__(self, marca, modelo, año):
        self.marca = marca
        self.modelo = modelo
        self.año = año

mi_coche = Coche("Seat", "León", 2022)
print(mi_coche.marca)   # => "Seat"
```

Atributos por defecto

Puedes dar valores por defecto a los parámetros:

```
class Coche:
    def __init__(self, marca, modelo, año=2024):
        self.marca = marca
        self.modelo = modelo
        self.año = año

mi_coche = Coche("Seat", "León")   # año = 2024 por defecto
```

📋 Métodos útiles

`__str__` y `__repr__`

Estos métodos especiales definen cómo se muestra un objeto:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __str__(self):
        return f"{self.nombre}, {self.edad} años"

    def __repr__(self):
        return f"Persona('{self.nombre}', {self.edad})"

p = Persona("Ana", 25)
print(p)          # => "Ana, 25 años" (usa __str__)
repr(p)           # => "Persona('Ana', 25)" (usa __repr__)
```

Herencia

Una clase puede **heredar** de otra:

```python
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hablar(self):
        raise NotImplementedError("Subclase debe implementar hablar()")

class Perro(Animal):
    def hablar(self):
        return "Guau"

class Gato(Animal):
    def hablar(self):
        return "Miau"

perro = Perro("Toby")
print(perro.hablar())   # => "Guau"
```

🧠 Mini-chequeo

1. ¿Cuál es la diferencia entre un atributo de clase y uno de instancia?
2. ¿Qué hace `self` en un método de instancia?
3. ¿Cuándo se usa `@staticmethod` en lugar de un método normal?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Una clase es un plano; un objeto es una instancia de ese plano.
- `__init__` inicializa los atributos; `self` referencia al objeto actual.
- Los métodos de clase (`@classmethod`) operan sobre la clase; los estáticos (`@staticmethod`) no dependen de nada.

🐛 Vocabulario rápido

Término	Idea general
Clase	Plano o plantilla para crear objetos
Objeto/Instancia	Ejemplo concreto de una clase
Atributo	Dato que pertenece a un objeto
Método	Función que pertenece a una clase
self	Referencia al objeto actual dentro de un método
Herencia	Una clase hijo hereda atributos y métodos de una clase padre
init	Constructor: se ejecuta al crear un objeto
@classmethod	Método que opera sobre la clase, no sobre una instancia
@staticmethod	Método que no depende ni de la clase ni de la instancia

📚 Volver al índice de la unidad · **Anterior:** 06 · Funciones · **Siguiente:** 08 · Módulos y avanzado

08 — Módulos y avanzado

import, pip, generadores y decoradores 📦

> 🗺️ **Estás en:** **U01 · Python 3 básico** → 08 · Módulos y avanzado

📬 La idea en una frase

> Los módulos son archivos `.py` con código reutilizable. Python trae cientos en su biblioteca estándar y `pip` te permite instalar miles más. Los generadores y decoradores son herramientas avanzadas que simplifican el código.

Esta unidad cierra la parte teórica de Python básico. Con lo que ya sabes (tipos, colecciones, control, funciones, clases) puedes escribir programas completos. Los módulos amplían tus posibilidades y los generadores/decoradores te dan superpoderes.

📦 Importar módulos

Un módulo es simplemente un archivo `.py`. Para usarlo, lo importas:

```
import math

print(math.sqrt(16))   # => 4.0
```

Formas de importar

```python
# Importar el módulo completo
import math
math.sqrt(16)

# Importar funciones específicas
from math import ceil, floor
ceil(3.7)    # => 4.0
floor(3.7)   # => 3.0

# Importar con alias
import math as m
m.sqrt(16)

# Importar todo (¡no recomendado!)
from math import *
```

¿Por qué no usar `from math import *`?

Porque puede haber **conflictos de nombres**. Si tu archivo define una función `sqrt` y haces `from math import *`, la tuya se pierde.

🔍 Explorar módulos

```python
import math
dir(math)   # Lista todas las funciones y atributos del módulo
```

También puedes ver la documentación con `help()`:

```python
help(math.sqrt)
```

🛒 pip: instalar paquetes

`pip` es el gestor de paquetes de Python. Con él puedes instalar librerías creadas por otros:

```
# Instalar un paquete
pip install requests

# Instalar una versión concreta
pip install requests==2.28.0

# Listar paquetes instalados
pip list

# Desinstalar
pip uninstall requests
```

Los paquetes se instalan desde PyPI (Python Package Index), el repositorio oficial con más de 400.000 paquetes.

El archivo requirements.txt

En proyectos reales, se guarda la lista de dependencias en un archivo `requirements.txt`:

```
requests==2.28.0
cryptography==3.19.0
openai==1.0.0
```

Para instalar todas las dependencias de golpe:

```
pip install -r requirements.txt
```

⚡ Generadores

Los generadores crean valores **sobre la marcha** en lugar de generarlos todos de golpe. Son ideales para secuencias grandes.

La palabra clave yield

```
def duplicar_numeros(iterable):
    for i in iterable:
        yield i + i
```

Cuando una función contiene `yield`, se convierte en un generador. No devuelve todos los valores de golpe; los va **produciendo** uno a uno:

```
_rango = range(1, 900000000)

for i in duplicar_numeros(_rango):
    print(i)
    if i >= 30:
        break
```

Esto es **eficiente**: no crea una lista de 900 millones de números en memoria. Va produciendo valores uno a uno y, cuando haces `break`, deja de generar.

Generador vs lista

```
# Lista: ocupa memoria para todos los elementos
cuadrados_lista = [x ** 2 for x in range(1000000)]

# Generador: ocupa casi nada de memoria
cuadrados_gen = (x ** 2 for x in range(1000000))
```

La diferencia es el operador: `[]` crea una lista, `()` crea un generador.

🎀 Decoradores

Un decorador es una función que **envuelve** a otra función para añadirle comportamiento sin modificarla.

Ejemplo básico

```python
from functools import wraps

def mi_decorador(funcion):
    @wraps(funcion)
    def wrapper(*args, **kwargs):
        print("Antes de llamar a la función")
        resultado = funcion(*args, **kwargs)
        print("Después de llamar a la función")
        return resultado
    return wrapper

@mi_decorador
def saludar(nombre):
    print(f"Hola, {nombre}")

saludar("Ana")
```

Salida:

```
Antes de llamar a la función
Hola, Ana
Después de llamar a la función
```

Decorador con parámetros

```python
def pedir(funcion):
    @wraps(funcion)
    def wrapper(*args, **kwargs):
        mensaje, decir_por_favor = funcion(*args, **kwargs)
        if decir_por_favor:
            return f"{mensaje} ¡Por favor! Soy pobre :("
        return mensaje
    return wrapper

@pedir
def say(decir_por_favor=False):
    mensaje = "¿Puedes comprarme una cerveza?"
    return mensaje, decir_por_favor

print(say())                        # ¿Puedes comprarme una cerveza?
print(say(decir_por_favor=True))    # ¿Puedes comprarme una cerveza? ¡Por favor! Soy pobre :(
```

Decoradores habituales

Python trae algunos decoradores integrados:

```python
class MiClase:
    @classmethod          # Método de clase
    def metodo_clase(cls):
        pass

    @staticmethod         # Método estático
    def metodo_estatico():
        pass

    @property             # Propiedad (getter)
    def nombre(self):
        return self._nombre
```

Los verás mucho en el módulo PSP, especialmente con `@classmethod` y `@staticmethod`.

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre `import math` y `from math import sqrt`?
2. ¿Cuándo es mejor usar un generador que una lista?
3. ¿Qué hace un decorador a una función?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Los módulos se importan con `import`; pip instala paquetes externos desde PyPI.
- Los generadores (`yield`) crean valores sobre la marcha, ahorrando memoria.
- Los decoradores envuelven funciones para añadirles comportamiento extra.

🐛 Vocabulario rápido

Término	Idea general
Módulo	Archivo `.py` con código reutilizable
pip	Gestor de paquetes de Python
PyPI	Repositorio oficial de paquetes Python
requirements.txt	Lista de dependencias del proyecto
Generador	Función con `yield` que produce valores uno a uno
yield	Palabra clave que convierte una función en generador
Decorador	Función que envuelve a otra para añadir comportamiento
@wraps	Decorador que preserva los metadatos de la función original

📚 Volver al índice de la unidad · **Anterior:** 07 · Clases · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé el código, el ring final y el laboratorio de la unidad 🧠

> 🗺️ **Estás en:** 🐍 **U01 · Python 3 básico** → 09 · Cierre

Has recorrido los 8 puntos de Python básico: comentarios, tipos, colecciones, control de flujo, funciones, clases, módulos y las herramientas avanzadas. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harían en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé el código

> *Eres un programa de Python. Tu misión: procesar una lista de alumnos y calcular su nota media.*

Paso a paso, ¿qué ocurre?

1. **Importas `math`** con `import math`. El módulo se carga en memoria y puedes usar `math.sqrt()` si lo necesitas.
2. **Defines una función `calcular_media`** con `def calcular_media(notas):`. El argumento `notas` es una lista.
3. **Dentro de la función**, usas `sum(notas) / len(notas)` para calcular la media. `sum()` y `len()` son built-ins, no necesitan import.
4. **Llamas a la función** con `calcular_media([7, 8, 9])`. Python crea un frame nuevo en la pila de llamadas, ejecuta el bloque indentado y devuelve `8.0`.
5. **Guardas el resultado** en una variable: `media = calcular_media(notas)`.
6. **Usas un `if`** para comprobar si la media es >= 5: aprobado o suspensa.
7. **Imprimes** el resultado con un f-string.

> 💡 **Ahora tú:** ¿y si la lista de notas está vacía? `len(notas)` sería 0 y dividirías entre cero. Necesitarías un `try/except` o una comprobación previa. Eso es programación defensiva.

🔥 Fireside Chat: Variables vs Constantes

> *Dos conceptos se sientan junto a la chimenea a zanjar la eterna pregunta de si Python tiene constantes.*

Variable: — Soy la base de todo. `x = 5`, `nombre = "Ana"`. Cualquier cosa que cambie durante la ejecución pasa por mí.

Constante: — Y yo… bueno, en Python no existo oficialmente. No hay palabra clave `const`. Pero la convención es que me escriba en MAYÚSCULAS: `MAX_INTENTOS = 3`.

Variable: — Entonces eres solo una convención. Cualquiera puede hacer `MAX_INTENTOS = 5` y romper tu "constancia".

Constante: — Es cierto. En Python confiamos en el programador. Si pongo `PI = 3.14159` en mayúsculas, es un aviso de que no toque. Pero no me protege el lenguaje.

Variable: — En JavaScript sí tienes `const`. En Python, la filosofía es "somos adultos".

Constante: — Y funciona. La claridad del código importa más que la imposición.

Moraleja: En Python, las constantes son convención, no restricción. Escribe en MAYÚSCULAS lo que no deba cambiar y confía en que quien lea tu código lo respete.

🕵️ ¿Quién soy?

Adivina el concepto Python de cada pista:

1. "Soy un bloque de código que se repite mientras una condición sea verdadera. Puedo ser infinito si no tengo cuidado."
2. "Soy una función sin nombre, escrita en una sola línea con `lambda`."
3. "Soy la referencia al objeto dentro de sus propios métodos. Sin mí, las clases no funcionarían."
4. "Soy una colección que no admite duplicados y puedo hacer intersecciones y uniones."
5. "Soy un valor que representa la ausencia de datos. No soy `0`, no soy `""`, soy `None`."
6. "Soy un archivo `.py` que otros programas importan. No se ejecuta directamente, pero mi código se reutiliza."

> 🔁 **Respuestas**

CONRAD VS EL MUNDO

> *Conrad, el profesor gruñón que odia los errores clásicos, enumera los fallos que ve cada semestre.*

1. "Olvídate del :"

```
if x > 5   # SyntaxError: invalid syntax
```

El `:` al final de `if`, `for`, `while`, `def` y `class` es **obligatorio**. Sin él, Python no sabe dónde empieza el bloque.

2. "Mezclar espacios y tabuladores"

```
def func():
    x = 5
  y = 10   # TabError: inconsistent use of tabs and spaces
```

Python odia mezclar. Usa **4 espacios** siempre. Configura tu editor para que al pulsar Tab inserte 4 espacios.

3. "Ponerle comas al return"

```
def sumar(a, b):
    return a, b   # ¡Esto devuelve una TUPLA! (a, b)
```

Si quieres devolver dos valores separados, usa `return a, b` (tupla). Si quieres devolver la suma, es `return a + b`.

4. "Confundir = con =="

```
x = 5      # Asignación
x == 5     # Comparación (devuelve True o False)
```

5. "Olvidar `self` en los métodos"

```
class MiClase:
    def saludar():
        print("Hola")   # TypeError: saludar() takes 0 positional arguments but 1 was
given
```

Todo método de instancia necesita `self` como primer parámetro, aunque no lo uses.

⚡ Laboratorio de tortura

> **Duración:** 20 minutos **Herramienta:** cualquier editor de texto + terminal **Objetivo:** crear un programa completo que gestione una lista de tareas (to-do list) usando lo aprendido.

Escenario

Eres el desarrollador de una app de tareas. El usuario debe poder añadir tareas, marcarlas como hechas, ver la lista y calcular el porcentaje de tareas completadas.

Tareas paso a paso

1. Crea una lista vacía `tareas = []`.
2. Crea una función `agregar_tarea(nombre, prioridad)` que añada un diccionario `{"nombre": nombre, "prioridad": prioridad, "hecha": False}`.
3. Crea una función `completar_tarea(nombre)` que busque la tarea por nombre y cambie `hecha` a `True`. Si no la encuentra, imprime un aviso.
4. Crea una función `ver_tareas()` que imprima cada tarea con su estado (✅ si está hecha, ⬜ si no).
5. Crea una función `calcular_progreso()` que devuelva el porcentaje de tareas completadas.
6. Añade un menú con `while True` y `input()` que permita al usuario elegir entre: agregar, completar, ver, progreso y salir.

Fallo intencionado

El siguiente código tiene **3 errores**. Encuéntralos:

```python
tareas = []

def agregar_tarea(nombre, prioridad):
    tarea = {"nombre": nombre, "prioridad": prioridad, "hecha": False}
    tareas.append(tarea)

def completar_tarea(nombre):
    for tarea in tareas:
        if tarea["nombre"] == nombre:
            tarea["hecha"] = True
            return
    print("Tarea no encontrada")

def ver_tareas():
    for i, tarea in enumerate(tareas):
        estado = "✅" if tarea["hecha"] else "⬜"
        print(f"{i+1}. {estado} {tarea['nombre']} (Prioridad: {tarea['prioridad']})")

def calcular_progreso():
    if len(tareas) == 0:
        return 0
    hechas = sum(1 for t in tareas if t["hecha"])
    return hechas / len(tareas) * 100

while True:
    print("\n1. Agregar tarea")
    print("2. Completar tarea")
    print("3. Ver tareas")
    print("4. Progreso")
    print("5. Salir")
    opcion = input("Elige una opción: ")

    if opcion == "1":
        nombre = input("Nombre de la tarea: ")
        prioridad = input("Prioridad (alta/media/baja): ")
        agregar_tarea(nombre, prioridad)
    elif opcion == "2":
        nombre = input("Nombre de la tarea a completar: ")
        completar_tarea(nombre)
    elif opcion == "3":
        ver_tareas()
    elif opcion == "4":
```

```
        print(f"Progreso: {calcular_progreso():.1f}%")
    elif opcion == "5":
        break
```

Pista 1: si el usuario escribe un nombre que no existe, el programa no dice nada. Añade un aviso cuando la tarea no se encuentre (pero ten en cuenta que el `return` dentro del `for` hace que salga de la función al encontrar la primera coincidencia).

Pista 2: si el usuario introduce algo que no es un número (por ejemplo, una letra), `ver_tareas` no maneja el error. ¿Qué excepción se lanza?

Pista 3: la función `calcular_progreso` devuelve un float con un decimal. ¿Y si el usuario escribe una opción que no existe en el menú? No hay comprobación.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🐣 **Primera ejecución**	Ejecuta `python hola.py` sin errores
🎯 **Cien por cien**	Resuelve todos los ejercicios del boletín inicial
🔥 **Modo avión**	Resuelve todos los ejercicios del boletín avanzado
🧠 **El interview**	Responde todas las preguntas de “Entrevista de trabajo” sin mirar los apuntes

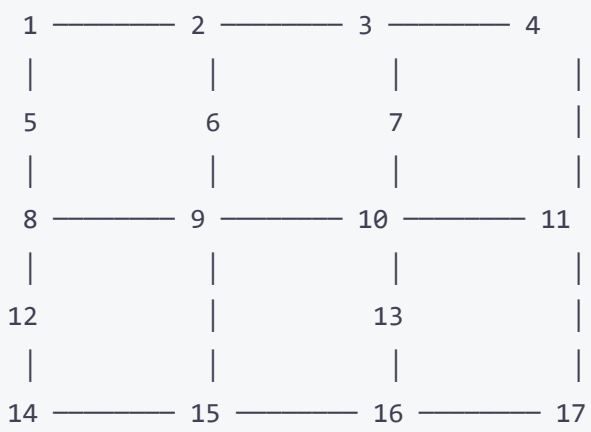
---

🧠 Atrévete a pensar

1. ¿Por qué Python es mejor para principiantes que C o Java?
2. ¿Cuándo usarías una tupla en lugar de una lista?
3. ¿Qué pasaría si `range(5)` devolviera una lista en lugar de un generador?
4. ¿Cómo harías un programa que lea un archivo de texto y cuente cuántas veces aparece cada palabra?
5. ¿Por qué se usa `self` en lugar de poner el nombre de la clase directamente?

🧩 Crucigrama de bits

Resuelve las pistas y rellena las casillas:



Horizontales:

1. (5 letras) Colección mutables con índices numéricos
2. (3 letras) Referencia al objeto dentro de sus métodos
3. (6 letras) Bloque que se repite mientras la condición sea verdadera
4. (4 letras) Sinónimo de función anónima (empieza con “l”)
5. (7 letras) Colección de pares clave-valor

Verticales:

1. (3 letras) Valor que representa “nada”: `_ _ _`
2. (6 letras) Archivo `.py` que se importa
3. (4 letras) Número con decimales (tipo de dato)
4. (8 letras) Palabra clave para crear una función
5. (7 letras) Añade paquetes desde PyPI
6. (4 letras) Término técnico para “duplicar” un string
7. (5 letras) Sinónimo de conjunto (en inglés)
8. (6 letras) Colección inmutable
9. (4 letras) Resultado de `True`
10. (8 letras) Archivo de dependencias del proyecto
11. (6 letras) Valor que puede ser `True` o `False`

12. (3 letras) Palabra clave para importar módulos
13. (4 letras) Función built-in que suma una lista

> 🔄 **Soluciones**

💬 Entrevista de trabajo

Estas son preguntas típicas sobre Python básico que podrían hacerte en una entrevista:

1. **¿Cuál es la diferencia entre una lista y una tupla?**

   > La lista es mutable (puedes cambiarla); la tupla es inmutable (no cambia). Las tuplas son más rápidas y seguran para datos que no deben modificar.

2. **¿Qué es `self` en una clase?**

   > Es la referencia al objeto actual. Permite acceder a los atributos y métodos del propio objeto dentro de la clase.

3. **¿Cuándo usarías `*args` y `*kwargs`?**

   > Cuando no sabes cuántos argumentos recibirá la función. `*args` para posicionales, `**kwargs` para nombrados.

4. **¿Qué es un generador?**

   > Una función con `yield` que produce valores uno a uno, sin crear toda la secuencia en memoria. Es eficiente para datos grandes.

5. **¿Qué hace un decorador?**

   > Envuelve una función para añadirle comportamiento antes y después de ejecutarla, sin modificar su código original.

6. **¿Cuál es la diferencia entre `==` e `is`?**

   > `==` compara **valores** (igualdad). `is` compara **identidad de objeto** (si son el mismo objeto en memoria).

🤷 No hay preguntas tontas

1. **¿Python es lento?** Es más lento que C o Java, pero para la mayoría de aplicaciones la diferencia es irrelevante. Y si necesitas velocidad, usas C extensions o PyPy.

2. **¿Por qué Python 3 y no Python 2?** Python 2 ya no recibe soporte desde 2020. Python 3 es el presente y el futuro.

3. **¿Necesito memorizar todas las funciones built-in?** No. Conoce las principales (`len`, `range`, `print`, `input`, `sum`, `max`, `min`, `sorted`) y busca el resto en la documentación.

4. **¿Cuál es la diferencia entre `print()` y `return`?** `print()` muestra algo por pantalla (para el usuario). `return` devuelve un valor (para que otro código lo use).

5. **¿Para qué sirve `if __name__ == "__main__":`?** Permite que un archivo `.py` se ejecute directamente **o** importarse como módulo sin ejecutar el código principal.

🎬 Poscréditos

> *La cámara se aleja del terminal. La última línea de código se ejecuta. El cursor parpadea en la esquina de la pantalla, esperando la siguiente instrucción.*
>
> *Has aprendido a hablar en Python. Ahora viene lo bueno: crear procesos que vivan y mueran, hilos que se sincronicen, servidores que atiendan peticiones de medio mundo.*
>
> *El viaje apenas empieza.*

PRÓXIMAMENTE EN UD 2 — Gestión de procesos

✅ Criterios de evaluación cubiertos (Prerequisito)

CE	Criterio	Cubierto
—	Domina la sintaxis básica de Python (variables, tipos, operadores)	✅ Puntos 3 y 4
—	Controla el flujo de ejecución (condicionales, bucles, excepciones)	✅ Punto 5
—	Define y llama a funciones con diferentes tipos de argumentos	✅ Punto 6
—	Crea y usa clases con atributos y métodos	✅ Punto 7

CE	Criterio	Cubierto
—	Importa módulos y gestiona paquetes externos	✅ Punto 8

📚 Volver al índice de la unidad · **Anterior:** 08 · Módulos y avanzado · **Siguiente:** UD 2 · Gestión de procesos

UD 2 — Gestión de procesos con Python

Ejecutar, controlar y comunicar procesos con subprocess 🚀

Un programa en el disco duro es un **muerto viviente**: no ocupa memoria, no consume CPU, no hace nada. Un **proceso** es ese mismo programa **vivo**, ejecutándose, ocupando memoria y consumiendo CPU. En esta unidad aprenderás a crear procesos con el módulo `subprocess`, a controlar su ejecución, a capturar su salida y a comunicarlos entre sí con pipes.

Dominar `subprocess` es la base de todo lo que viene después: los hilos de la UD 3 son procesos ligeros, y los servidores de la UD 6 serán procesos o hilos ejecutándose de fondo.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un proceso, su PID y la burbuja de memoria que lo forma.
- Distinguir computación **paralela**, **distribuida** y **concurrente**.
- Lanzar y esperar programas con `subprocess.run()`, capturando salida y controlando timeout y errores.
- Lanzar procesos en segundo plano con `subprocess.Popen()` y gestionarlos con `wait`, `poll`, `terminate` y `kill`.
- Comunicar procesos pasando datos por `stdin` y leyendo su respuesta por `stdout` con `communicate()`.
- Adaptar tus programas a Windows y Linux cambiando solo el comando.
- Lanzar funciones propias en paralelo con `multiprocessing` (`Process`, `Manager` y `Lock`) y comprender la burbuja de memoria entre procesos.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Qué es un proceso	La burbuja de memoria, el PID y las características de todo proceso	Todos

Punto	Qué aprenderás	Nivel
02 · Estados de un proceso	NUEVO, LISTO, EJECUCIÓN, BLOQUEADO, TERMINADO y sus transiciones	Todos
03 · Paralela vs Distribuida	La diferencia entre paralela, distribuida y concurrencia	Todos
04 · subprocess.run()	Lanzar y esperar: capturar salida, timeout y errores	Todos
05 · subprocess.Popen()	Lanzar y seguir: wait, poll, terminate, kill y el PID	Todos
06 · Comunicación con procesos	stdin, stdout y `communicate()`: pasar datos a un proceso	Todos
07 · Compatibilidad Windows / Linux	La tabla de comandos equivalentes y los trucos del shell	Todos
08 · Procesos nativos con multiprocessing	Process, Manager y Lock: tus propias funciones en paralelo	Todos
09 · Ejemplo resuelto: estado compartido	Un almacén en memoria que cuatro procesos actualizan sin pisarse	Todos
10 · Procesos en la práctica	Sé el código, el ring run vs Popen y ejercicios	Todos
11 · Cierre	Sé el proceso, Fireside, Laboratorio de tortura, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 9 primeros puntos son teoría en progresión (el 8 dedica a `multiprocessing` y el 9 la consolida con un ejemplo resuelto). El 10o aterriza en la práctica y el 11o consolida.

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce las características y la gestión de los procesos en un sistema operativo.

CE	Criterio	Dónde se cubre
a)	Reconoce las características de los procesos	✅ Punto 1
b)	Distingue entre computación paralela y distribuida	✅ Punto 3
c)	Conoce los estados de un proceso	✅ Punto 2
d)	Identifica las diferencias clave entre proceso e hilo	→ UD 3
e)	Crea programas con procesos (subprocess)	✅ Puntos 4 y 5 + ⚡ Laboratorio (punto 11)
f)	Establece comunicación entre procesos	✅ Punto 6 + puntos 8 y 9 (Manager y Lock)

🚪 ¿Por dónde empiezo?

¿Vienes de cero en programación de sistemas? Empieza por el punto 1 y no te saltes los puntos 1 a 3: son la base conceptual de TODO el módulo.

¿Ya sabes qué es un proceso? Saltar a los puntos 4 y 5, que son el corazón práctico de la unidad.

📍 **Primer punto:** 01 · Qué es un proceso ⏭️ **Al acabar la unidad, continúa en** UD 3 · Hilos y concurrencia.

01 — Qué es un proceso

La burbuja de memoria que vive en tu sistema 🫧

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 01 · Qué es un proceso

---

📬 La idea en una frase

> Un **proceso** es un programa en ejecución con su propio espacio de memoria, recursos (archivos abiertos, sockets) y un identificador único llamado **PID**.

Un programa en el disco duro es un **muerto viviente**: no hace nada. Un proceso es ese mismo programa **vivo**, ocupando memoria, consumiendo CPU y respondiendo al teclado, a la red o al ratón. La diferencia no está en el código: está en que el sistema operativo lo ha cargado en memoria y lo está ejecutando.

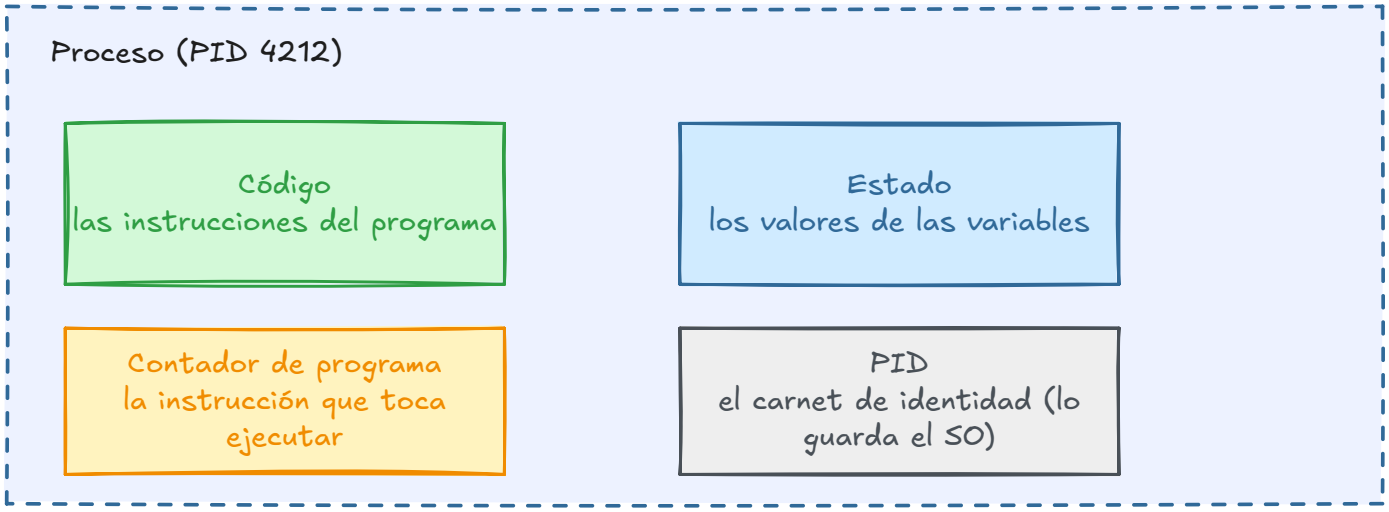
```
import os
print(f"Este proceso se llama PID {os.getpid()}")
```

Cada vez que ejecutas ese programa, el sistema operativo crea un proceso nuevo con su propio PID.

🫧 La burbuja de memoria

Cada proceso vive en su propia **burbuja de memoria**: un espacio de direcciones aislado del resto del sistema. Dentro de esa burbuja viajan:

La burbuja de memoria de un proceso



- **Código**: las instrucciones del programa.
- **Estado**: los valores actuales de las variables.
- **Contador de programa**: qué instrucción toca ejecutar ahora.
- **PID**: el carnet de identidad del proceso (técnicamente lo guarda el sistema operativo en su tabla de procesos, no dentro de la memoria del proceso; aquí lo dibujamos junto por claridad).

> "Si un proceso se cuelga, los demás no se enteran. Cada uno vive en su burbuja de memoria."

🪪 El PID, el carnet de identidad

Cada proceso tiene un **PID** (*Process IDentifier*): un número único que asigna el sistema operativo en el momento de crearlo. Sirve para referirse a él, para matarlo, para vigilarlo.

```
import os

print(f"Mi PID es {os.getpid()}")
```

Para ver los procesos de tu sistema con sus PIDs:

- **Windows**: Administrador de tareas → pestaña "Detalles", o `tasklist` en la terminal.
- **Linux / macOS**: `ps aux` o `top`.

📋 Características de un proceso

Propiedad	Descripción
PID	Identificador único numérico
Memoria propia	Cada proceso tiene su espacio de direcciones aislado
Recursos	Archivos, sockets, manejadores
Contexto	Estado de la CPU, registros, contador de programa
Comunicación	Necesita mecanismos externos (pipes, sockets, archivos)

La última fila es la clave: los procesos **no comparten memoria por defecto**. Si quieren intercambiar datos necesitan un mecanismo externo: un archivo, un socket o un pipe. Lo verás en el punto 6.

🍳 La analogía de la cocina

Un proceso es una **receta en marcha** en una cocina. La receta escrita en el libro es el **programa** (código muerto en el disco). Cuando un cocinero la coge, pone los ingredientes sobre su mesa (memoria), empieza a leerla por el paso 1 (contador de programa) y recibe su propio número de pedido (PID).

Cada cocinero con su receta y su mesa: si uno se quema, los demás siguen cocinando sin enterarse. Eso es el **aislamiento** de la burbuja de memoria. En el punto 3 verás qué pasa cuando hay varias cocinas (varias CPUs) o varios restaurantes (varias máquinas).

🧠 Mini-chequeo

1. ¿Qué es el PID y para qué sirve?
2. ¿Qué contiene la burbuja de memoria de un proceso?
3. ¿Por qué dos procesos no pueden compartir una variable directamente?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un proceso es un programa **en ejecución** con memoria, recursos y un PID propio.
- Su burbuja de memoria contiene código, estado, contador de programa y PID, aislada del resto.
- Los procesos no comparten memoria: se comunican con mecanismos externos.

🐛 Vocabulario rápido

Término	Idea general
Proceso	Programa en ejecución con memoria y recursos propios
PID	Identificador único del proceso
Burbuja de memoria	Espacio de direcciones aislado de cada proceso
Contador de programa	Qué instrucción toca ejecutar ahora
Contexto	Estado de la CPU, registros y contador del proceso

📚 Volver al índice de la unidad · **Siguiente:** 02 · Estados de un proceso

02 — Estados de un proceso

El ciclo de vida: de NUEVO a TERMINADO 🔄

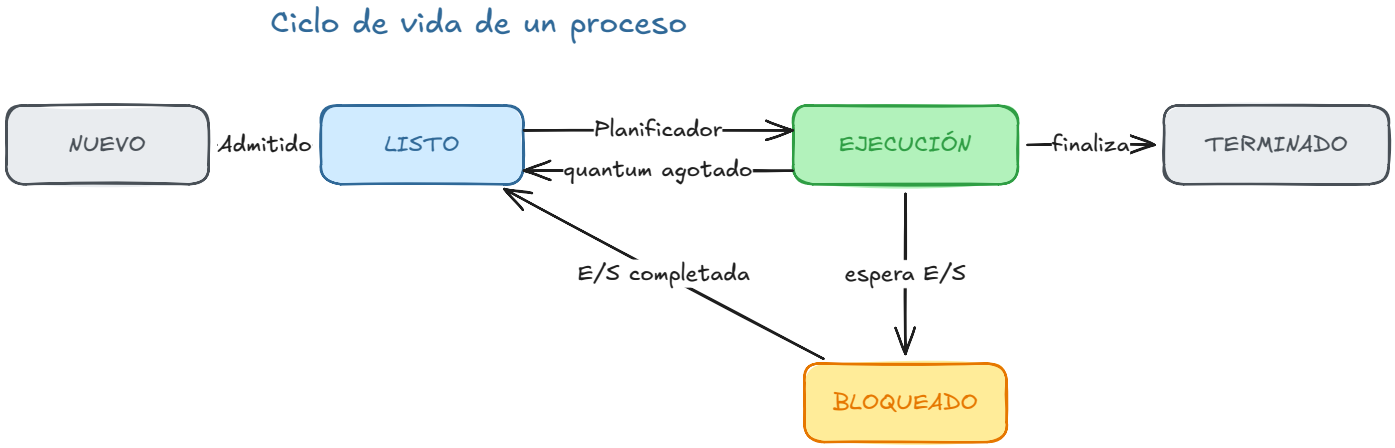
> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 02 · Estados de un proceso

📬 La idea en una frase

> Un proceso no está siempre “ejecutándose”: nace, espera su turno, usa la CPU, se bloquea esperando recursos y finalmente muere. Esos **cinco estados** forman su ciclo de vida.

El sistema operativo gestiona decenas o cientos de procesos a la vez con una sola CPU (o pocas). Para que todo parezca simultáneo, reparte la CPU en porciones de tiempo y cada proceso va pasando de estado según lo que le toca.

🔄 El ciclo de vida de un proceso



- De **LISTO** solo se puede ir a **EJECUCIÓN** (la CPU toca) y volver.
- De **EJECUCIÓN** se va a **LISTO** (time slice), a **BLOQUEADO** (E/S) o a **TERMINADO**.
- De **BLOQUEADO** se vuelve siempre a **LISTO** cuando la E/S termina.

Estado	Qué significa
NUEVO	El proceso acaba de ser creado
LISTO	Preparado para ejecutar, esperando que la CPU esté libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones

Estado	Qué significa
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado

🚦 Las transiciones, una a una

- **NUEVO → LISTO**: el proceso ya está cargado y espera su turno de CPU.
- **LISTO → EJECUCIÓN**: el planificador (*scheduler*) le entrega la CPU.
- **EJECUCIÓN → LISTO**: el planificador se la quita porque llega otro con más prioridad o por **time slice** (cada proceso tiene una porción de tiempo).
- **EJECUCIÓN → BLOQUEADO**: el proceso pide una operación de E/S (leer un archivo, esperar un socket, `time.sleep`) y no puede continuar hasta que llegue el dato.
- **BLOQUEADO → LISTO**: la E/S ha terminado y el proceso vuelve a la cola de preparados.
- **EJECUCIÓN → TERMINADO**: el proceso acaba (`return`, `sys.exit()`, o lo matan).

> 💡 **LISTO y BLOQUEADO no son lo mismo.** LISTO significa "puedo ejecutar, solo espero a que la CPU me toque". BLOQUEADO significa "no puedo ejecutar aunque me dieran la CPU: estoy esperando un recurso".

☕ El ejemplo de la cafetería

Imagina una cola de gente en una cafetería con un único camarero:

- Entras por la puerta → **NUEVO**.
- Te pones en la cola → **LISTO** (esperando al camarero).
- El camarero te atiende → **EJECUCIÓN**.
- Pides un café y el camarero se va a la máquina → **BLOQUEADO** (esperando el recurso "café").
- El café está listo y vuelves a la cola → **LISTO**.
- El camarero te lo entrega → **EJECUCIÓN**.
- Pagas y te vas → **TERMINADO**.

El camarero (la CPU) alterna entre los clientes de la cola dando a cada uno unos segundos: por eso una sola CPU puede atender a muchos procesos “a la vez”.

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre LISTO y BLOQUEADO?
2. ¿Qué transición ocurre cuando un proceso llama a `time.sleep(2)`?
3. Un proceso está ejecutándose y el planificador le quita la CPU porque llega otro con más prioridad. ¿A qué estado pasa?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un proceso recorre cinco estados: **NUEVO, LISTO, EJECUCIÓN, BLOQUEADO y TERMINADO**.
- LISTO y BLOQUEADO son distintos: uno espera CPU y el otro espera un recurso.
- El planificador reparte la CPU y por eso un solo procesador parece ejecutar muchos procesos a la vez.

🐛 Vocabulario rápido

Término	Idea general
NUEVO	El proceso acaba de crearse
LISTO	Preparado, esperando que la CPU se libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado
Planificador (scheduler)	Decide qué proceso LISTO pasa a EJECUCIÓN

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es un proceso · **Siguiente:** 03 · Paralela vs Distribuida

03 — Paralela vs Distribuida

CPUs, máquinas y la concurrencia de por medio 👰

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 03 · Paralela vs Distribuida

📬 La idea en una frase

> **Paralela** es ejecutar varias tareas **a la vez** en varias CPUs; **distribuida** es ejecutarlas en **múltiples máquinas** conectadas por red; **concurrencia** es que varias tareas **avancen**, aunque se turnen en una sola CPU.

La diferencia está en el "dónde": la paralela reparte trabajo entre los núcleos de tu máquina, la distribuida entre máquinas, y la concurrencia es una forma de *parecer* simultáneo aunque físicamente no lo sea.

👰 La tabla de los tres conceptos

Concepto	Qué significa
Paralela	Varias tareas ejecutándose **a la vez** en múltiples CPUs/núcleos
Distribuida	Varias tareas ejecutándose en **múltiples máquinas** conectadas por red
Concurrencia	Varias tareas **avanzando** (no necesariamente a la vez, pueden turnarse)

Ejemplos cotidianos

- **Paralela**: 4 hilos de cocina, cada uno friendo un huevo en su sartén (4 CPUs).
- **Distribuida**: 4 restaurantes en 4 ciudades, todos cocinando el mismo menú.
- **Concurrente**: 1 cocinero que va friendo huevos de 3 pedidos, alternando.

🍳 Paralela: cuatro sartenes

Imagina que tienes que freír 4 huevos. Con **una sola sartén** (una CPU), los fríes de uno en uno o alternando: eso es *concurrencia*. Con **4 sartenes** (4 CPUs), los fríes los 4 a la vez: eso es

paralelismo real.

En Python, el módulo `multiprocessing` crea procesos reales que el sistema operativo reparte entre los núcleos de tu CPU (lo verás a fondo en el punto 8):

```
# Paralela con multiprocessing (varios CPUs)
from multiprocessing import Pool

def cuadrado(n):
    return n * n

if __name__ == "__main__":  # obligatorio en Windows: protege el spawn
    with Pool(4) as p:  # 4 procesos en paralelo
        print(p.map(cuadrado, [1, 2, 3, 4]))
```

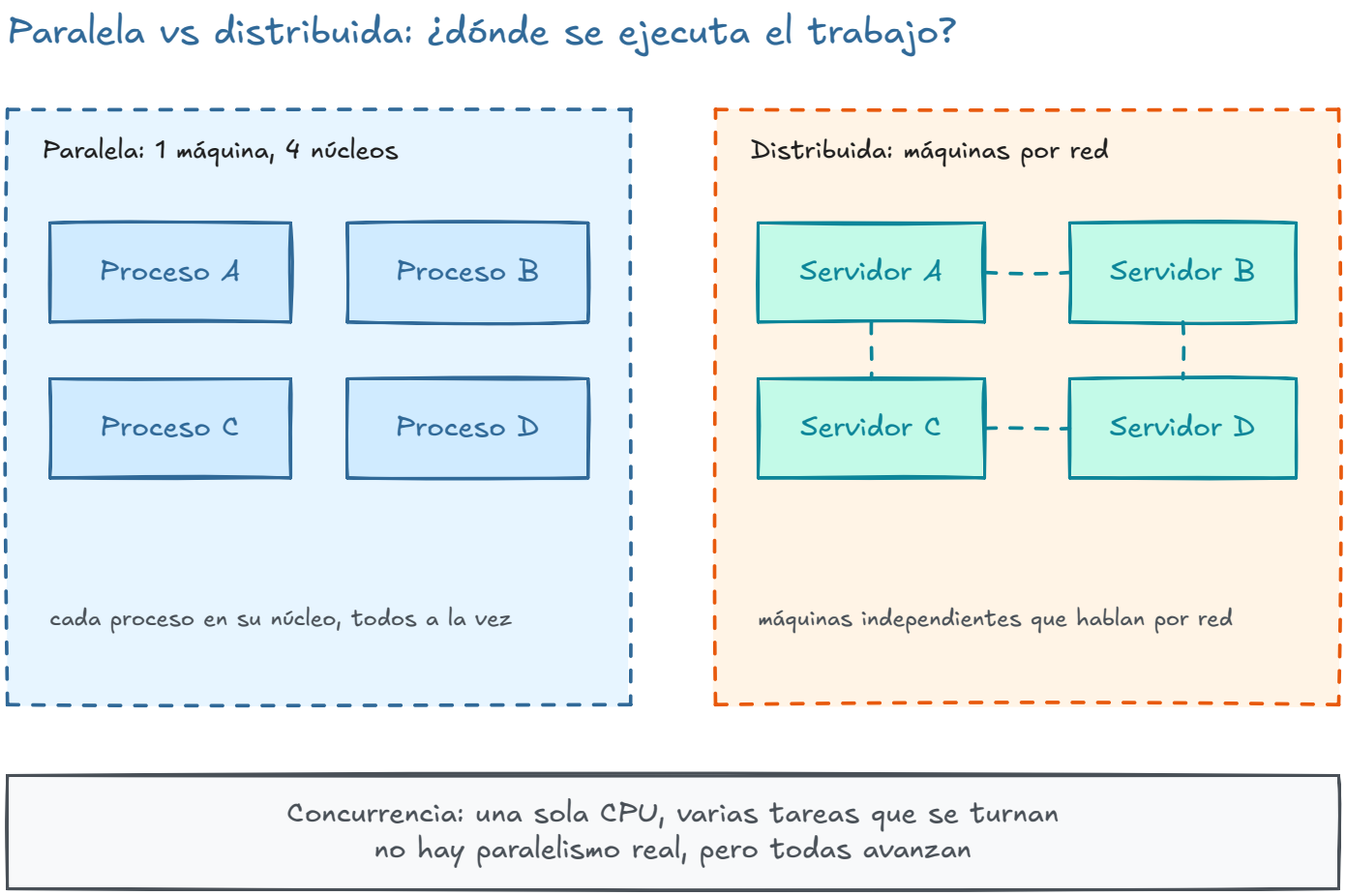
Salida:

```
[1, 4, 9, 16]
```

Los 4 procesos se reparten la lista y cada uno calcula una parte. Si tu máquina tiene 4 núcleos, los 4 se ejecutan **a la vez**.

🏙️ Distribuida: varios restaurantes

La computación distribuida lleva la idea al límite: no varias CPUs de la misma máquina, sino **máquinas completas conectadas por red**, cada una con su memoria y su CPU. Es el modelo de los clústeres, la web y los servicios en la nube.



Cada máquina ejecuta uno o varios procesos independientes. La distribución introduce un problema nuevo: **la comunicación por red** y **los fallos de máquina**. Eso lo verás en las unidades de sockets (UD 5) y de APIs REST (UD 7).

⚖️ CPUs vs máquinas

Pregunta	Paralela	Distribuida
¿Dónde se ejecuta?	Múltiples CPUs/núcleos de **una** máquina	Múltiples **máquinas** conectadas por red
¿Comparten memoria?	No por defecto: cada **proceso** vive en su burbuja aislada (los **hilos** de una misma máquina sí comparten, UD 3)	No (cada máquina tiene la suya)
¿Comunicación?	Pipes, memoria compartida, locks	Sockets, HTTP, mensajería
¿Escala?	Hasta los núcleos de tu CPU	Hasta cientos de máquinas

---

Pregunta	Paralela	Distribuida
Ejemplo en Python	`multiprocessing.Pool`	`socket`, APIs REST

> 💡 El **GIL** de Python limita la concurrencia real de los hilos (lo verás en la UD 3); con **procesos** (`multiprocessing`) ese límite no existe porque cada proceso tiene su propio intérprete.

🧠 Mini-chequeo

1. ¿Cuál es la diferencia clave entre paralela y distribuida?
2. ¿Por qué el ejemplo del "cocinero alternando" es concurrencia y no paralelismo?
3. ¿Qué función de `multiprocessing` reparte una tarea entre varios procesos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- **Paralela** = varias CPUs a la vez; **distribuida** = varias máquinas por red; **concurrencia** = avanzar turnándose.
- `multiprocessing.Pool` reparte trabajo real entre los núcleos de tu CPU.
- La paralela comparte memoria y es interna a la máquina; la distribuida necesita red y comunicación por sockets/HTTP.

🐛 Vocabulario rápido

Término	Idea general
Paralela	Varias tareas a la vez en varias CPUs
Distribuida	Varias tareas en varias máquinas por red
Concurrencia	Varias tareas avanzando, turnándose
multiprocessing	Módulo Python que crea procesos reales

Término	Idea general
Pool	Grupo de procesos que reparte tareas

📚 Volver al índice de la unidad · **Anterior:** 02 · Estados de un proceso · **Siguiente:** 04 · subprocess.run()

04 — subprocess.run()

Lanza un programa y espera a que termine ⏳

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 04 · subprocess.run()

📬 La idea en una frase

> `subprocess.run()` lanza un programa y **espera** a que termine, devolviéndote su salida y su código de retorno. Es la forma más sencilla de ejecutar otro programa desde Python.

Cuando necesitas el resultado de un comando (¿qué versión hay? ¿responde el ping? ¿qué dice este comando?), `run()` es tu herramienta: lanza, espera y captura.

🚀 El primer lanzamiento

```
import subprocess

# Comando simple
resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(f"Salida: {resultado.stdout}")
print(f"Código de retorno: {resultado.returncode}")
```

Salida (más o menos):

```
Salida: Python 3.11.4
Código de retorno: 0
```

- `subprocess.run(["python", "--version"])` recibe una **lista** con el ejecutable y sus argumentos. Cada elemento de la lista es un argumento aparte.
- `capture_output=True` captura la salida estándar (`stdout`) y la de errores (`stderr`).
- `text=True` devuelve cadenas en vez de bytes (mucho más cómodo para leer).
- `resultado.returncode` es el código de retorno: **0 = todo bien**, distinto de 0 = algo falló.

🎛️ Los parámetros importantes

Parámetro	Qué hace
`capture_output=True`	Captura stdout y stderr
`text=True`	Devuelve strings en vez de bytes
`timeout=N`	Lanza excepción si tarda más de N segundos
`check=True`	Lanza excepción si el código de retorno no es 0

Resultado: `stdout`, `stderr` y `returncode`

Atributo	Qué contiene
`resultado.stdout`	La salida estándar del programa
`resultado.stderr`	La salida de errores
`resultado.returncode`	0 si fue bien, otro número si falló

⏱️ Timeout: no esperes para siempre

Si un comando puede colgarse (un ping, un servidor), protege tu programa con `timeout`:

```
# Con timeout y control de errores
try:
    resultado = subprocess.run(
        ["ping", "google.com", "-n", "3"],
        capture_output=True,
        text=True,
        timeout=10
    )
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("❌ El ping tardó demasiado")
except subprocess.CalledProcessError:
    print("❌ Error en el comando")
```

- `subprocess.TimeoutExpired`: el comando superó los `timeout` segundos.
- `subprocess.CalledProcessError`: solo se lanza con `check=True` cuando el `returncode` no es 0.

🐛 Un ejemplo con error controlado

```
import subprocess

resultado = subprocess.run(["python", "--versio"], capture_output=True, text=True)
print(f"Código: {resultado.returncode}")
print(f"Error: {resultado.stderr}")
```

Salida:

```
Código: 2
Error: Unknown option: --versio
```

El código de retorno **2** (distinto de 0) te dice que el comando falló, y `stderr` te dice por qué. No hizo falta ninguna excepción: por defecto `run()` no lanza, solo devuelve el resultado.

🧠 Mini-chequeo

1. ¿Qué hace `capture_output=True` y `text=True`?

2. ¿Qué significa un `returncode` distinto de 0?
3. ¿Qué excepción lanza `run()` si el comando tarda más de `timeout`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `subprocess.run()` lanza un comando y **espera** su finalización.
- Captura la salida con `capture_output=True`, texto con `text=True` y controla el tiempo con `timeout`.
- El `returncode` te dice si fue bien (0) o mal (distinto de 0).

🐛 Vocabulario rápido

Término	Idea general
subprocess.run	Función que lanza un comando y espera
capture_output	Captura stdout y stderr
text=True	Devuelve strings en vez de bytes
timeout	Lanza TimeoutExpired si excede
returncode	Código de retorno (0 = bien)
check=True	Lanza CalledProcessError si returncode ≠ 0

📚 Volver al índice de la unidad · **Anterior:** 03 · Paralela vs Distribuida · **Siguiente:** 05 · subprocess.Popen()

05 — subprocess.Popen()

Lanza un proceso y sigue tu camino 🚶

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 05 · subprocess.Popen()

📬 La idea en una frase

> `subprocess.Popen()` lanza un proceso en **segundo plano** y devuelve el control inmediatamente: no espera a que termine. Tú sigues haciendo cosas mientras él vive.

`run()` espera; `Popen()` no. Eso convierte a `Popen` en la herramienta para abrir aplicaciones, lanzar varios programas a la vez y gestionarlos después con sus métodos.

🚀 El primer Popen: abrir el bloc de notas

```
import subprocess

# Lanzar el bloc de notas (no espera)
proceso = subprocess.Popen(["notepad.exe"])
print(f"Bloc de notas lanzado con PID {proceso.pid}")

# Podemos hacer otras cosas mientras el bloc de notas está abierto
print("Haciendo otras cosas...")

# Cuando queramos, esperamos a que termine
proceso.wait()
print("El bloc de notas se cerró")
```

Paso a paso:

1. `subprocess.Popen(["notepad.exe"])` lanza el bloc de notas y **vuelve al instante**.
2. `proceso.pid` te da el PID del proceso recién creado.
3. Python sigue ejecutando (`print("Haciendo otras cosas...")`) mientras el bloc de notas está abierto.
4. `proceso.wait()` **bloquea** a Python hasta que el bloc de notas se cierra.

🎛️ Los métodos de Popen

Método de Popen	Qué hace
`proceso.wait()`	Espera a que termine (bloqueante)
`proceso.poll()`	Pregunta si ha terminado (no bloqueante)
`proceso.terminate()`	Envía señal de terminación
`proceso.kill()`	Mata el proceso forzosamente
`proceso.pid`	PID del proceso hijo

`wait()` vs `poll()`

- `wait()` **bloquea**: Python se queda parado hasta que el proceso termina.
- `poll()` **no bloquea**: devuelve `None` si el proceso sigue vivo, o su código de retorno si ya terminó.

```
import subprocess, time

proceso = subprocess.Popen(["notepad.exe"])

while proceso.poll() is None:
    print("El bloc de notas sigue abierto...")
    time.sleep(1)

print(f"Cerrado con código {proceso.poll()}")
```

`terminate()` vs `kill()`

- `terminate()` en Linux/macOS envía **SIGTERM**: un cierre *suave*, el proceso tiene oportunidad de guardar y salir. En **Windows** ambos son equivalentes: `terminate()` llama a `TerminateProcess`, una terminación **forzosa** igual que `kill()`.
- `kill()` mata **forzosamente**, sin dar opción. Úsalo solo cuando `terminate()` no baste.

```python
proceso = subprocess.Popen(["calc.exe"])
print(f"Calculadora lanzada con PID {proceso.pid}")

proceso.terminate()   # cierre forzoso (Windows) o suave (Linux/macOS)
proceso.kill()        # solo si el anterior no funcionó
```

🏃 Lanzar varios procesos a la vez

La gran ventaja de `Popen` es lanzar varios procesos y esperarlos después:

```python
import subprocess, time

notepad = subprocess.Popen(["notepad.exe"])
calc = subprocess.Popen(["calc.exe"])
mspaint = subprocess.Popen(["mspaint.exe"])

print(f"PIDs: {notepad.pid}, {calc.pid}, {mspaint.pid}")

time.sleep(5)          # les damos 5 segundos de vida
notepad.terminate()
calc.terminate()
mspaint.kill()
print("Todos cerrados 🏁")
```

Los 3 procesos viven a la vez, cada uno con su PID, mientras Python hace otras cosas. Eso es imposible con `run()`, que espera a cada uno antes de lanzar el siguiente.

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre `wait()` y `poll()`?
2. ¿Qué devuelve `proceso.poll()` mientras el proceso sigue vivo?
3. ¿Cuándo usarías `kill()` en lugar de `terminate()`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `Popen` lanza un proceso en segundo plano y devuelve el control **al instante**.
- `wait()` bloquea, `poll()` pregunta, `terminate()` cierra suave y `kill()` mata a la fuerza.
- Con `Popen` puedes lanzar y gestionar **varios procesos a la vez**, cada uno con su PID.

🐛 Vocabulario rápido

Término	Idea general
Popen	Clase que lanza procesos en segundo plano
wait()	Espera bloqueante a que termine
poll()	Consulta no bloqueante del estado
terminate()	Señal de cierre suave
kill()	Muerte forzosa
pid	PID del proceso hijo

📚 Volver al índice de la unidad · **Anterior:** 04 · subprocess.run() · **Siguiente:** 06 · Comunicación con procesos

06 — Comunicación con procesos

Pasa datos por stdin y lee la respuesta por stdout 📨

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 06 · Comunicación con procesos

---

📬 La idea en una frase

> Un proceso hijo puede **leer de su stdin** y **escribir en su stdout**; con `communicate()` le pasas datos por un tubo y lees su respuesta por el otro.

Los procesos no comparten memoria (punto 1), pero el sistema operativo les presta **pipes** (tuberías): el `stdin`, `stdout` y `stderr` de cada proceso pueden conectarse a Python. Así un proceso "pregunta" y el otro "responde".

🔌 Conectar los tubos

```
import subprocess

proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)
```

- `stdin=subprocess.PIPE`: Python podrá **escribir** en el stdin del hijo.
- `stdout=subprocess.PIPE`: Python podrá **leer** el stdout del hijo.
- `text=True`: los datos viajan como texto, no como bytes.

El proceso hijo ejecuta `print(input().upper())`: lee una línea de su stdin, la pasa a mayúsculas y la escribe en su stdout.

📨 Enviar y recibir con communicate()

```
salida, _ = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"
```

`communicate(input="hola mundo")`:

1. Escribe `"hola mundo"` en el stdin del hijo y cierra el tubo de entrada.
2. Espera a que el hijo termine.
3. Devuelve una tupla `(stdout, stderr)`.

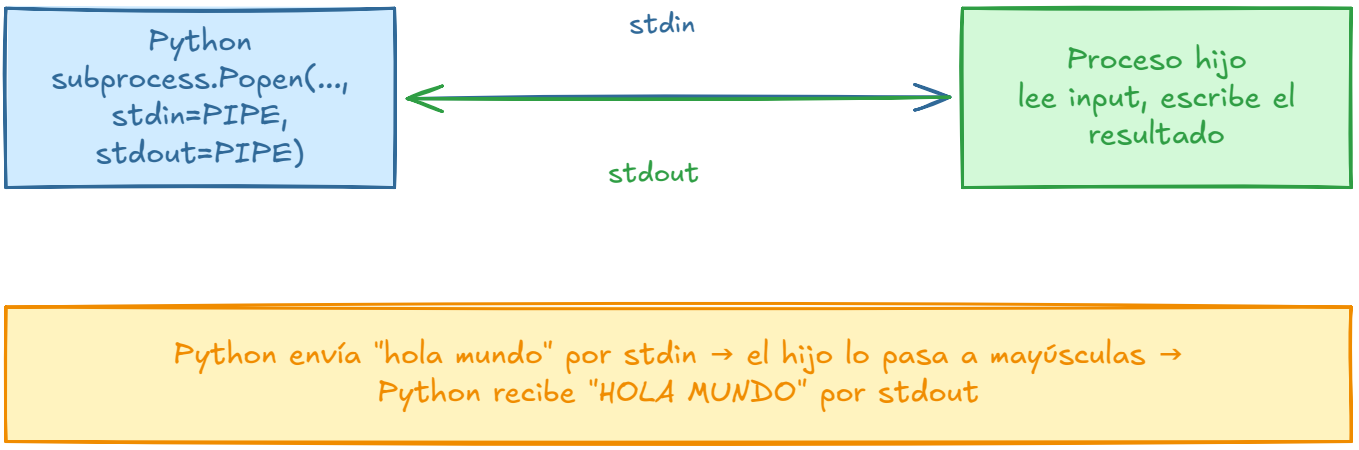
Después de `communicate()` el proceso ya terminó: no puedes volver a comunicarte con él.

🗣️ El mismo ejemplo completo, con sentido

```
import subprocess

# Escribir en stdin y leer stdout
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

salida, error = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"

print(f"Error (None si todo bien): {error}")
# → Error (None si todo bien): None
```

Flujo de datos:

Comunicación con pipes: stdin y stdout



sin memoria compartida: los datos viajan por la tubería

💡 ¿Cuándo usar cada herramienta?

Situación	Herramienta
Necesitas el resultado de un comando y puedes esperar	`subprocess.run(..., capture_output=True)`
El proceso vive en segundo plano y no necesitas sus datos	`subprocess.Popen(...)` + `wait()`
Necesitas **enviarle datos** y/o **leer su respuesta**	`Popen(..., stdin=PIPE, stdout=PIPE)` + `communicate()`

> ⚠️ **Cuidado con los deadlocks:** si el hijo escribe mucho en stdout y nadie lo lee, el pipe se llena y el hijo se bloquea esperando a que alguien lo vacíe. `communicate()` lee todo y te evita ese problema. Por eso con `run()` se usa `capture_output=True` y con `Popen`, `communicate()`.

🧠 Mini-chequeo

1. ¿Qué significa `stdin=subprocess.PIPE` y `stdout=subprocess.PIPE`?
2. ¿Qué devuelve `communicate(input="texto")`?
3. ¿Por qué `communicate()` evita los deadlocks con pipes?

> 🔁 Respuestas

✅ Resumen en 3 frases

- Con `stdin=PIPE` y `stdout=PIPE` conectas tuberías entre Python y el proceso hijo.
- `communicate(input="...")` envía datos al hijo, espera su fin y devuelve `(stdout, stderr)`.
- Los pipes permiten “preguntar” a un proceso y “leer” su respuesta sin compartir memoria.

🐛 Vocabulario rápido

Término	Idea general
Pipe	Tubería que conecta la salida de un proceso con la entrada de otro
stdin	Entrada estándar del proceso
stdout	Salida estándar del proceso
stderr	Salida de errores del proceso
PIPE	Constante que le dice a Popen “conecta esta tubería”
communicate()	Envía datos y recoge la salida del hijo

📚 Volver al índice de la unidad · **Anterior:** 05 · subprocess.Popen() · **Siguiente:** 07 · Compatibilidad Windows / Linux

07 — Compatibilidad Windows / Linux

Un mismo código, dos mundos: la tabla de trucos 🪟 🐧

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 07 · Compatibilidad Windows / Linux

📬 La idea en una frase

> Los ejemplos de esta unidad están escritos para **Windows**, pero los conceptos (`subprocess.run`, `Popen`, `PID`, `communicate`) son **idénticos en todos los sistemas**: solo cambia el comando.

La API de `subprocess` es la misma en Windows, Linux y macOS. Lo único que cambia es qué ejecutable pones en la lista: `notepad.exe` no existe en Linux y `ls` no existe en Windows. Aprende a sustituir el comando y el código funciona igual.

🪟 🐧 La tabla de comandos equivalentes

Windows	Linux / macOS
`notepad.exe`	`gedit`, `nano`, `xed`
`calc.exe`	`gnome-calculator`, `bc`
`mspaint.exe`	`pinta`, `kolourpaint`
`ping -n 5 8.8.8.8`	`ping -c 5 8.8.8.8`
`ipconfig`	`ip addr`, `ifconfig`
`where python`	`which python`
`cmd /c mkdir`	`mkdir` (es ejecutable real)
`start http://...`	`xdg-open http://...`
`dir`	`ls`

> Sustituye el comando correspondiente y el código funciona igual.

🧩 El truco del shell

Algunos comandos **no son ejecutables reales**: son palabras que interpreta el intérprete de comandos (shell). El ejemplo clásico es `mkdir`:

- En **Windows**, `mkdir` es un **comando interno de `cmd`**: no hay un archivo `mkdir.exe`, así que hay que invocarlo a través del shell con `cmd /c mkdir ...`.
- En **Linux**, `mkdir` es un **ejecutable real** (`/usr/bin/mkdir`): se lanza directamente.

```
import subprocess

# En Windows: cmd /c ejecuta comandos internos del shell
resultado = subprocess.run(["cmd", "/c", "mkdir", "prueba_psp"],
                           capture_output=True, text=True)
print(f"Creada con código: {resultado.returncode}")
```

En Linux usarías directamente `["mkdir", "prueba_psp"]` porque `mkdir` es un ejecutable real.

Otro ejemplo: abrir el navegador con una URL.

- **Windows**: `start http://localhost:4321` (con `shell=True`, porque `start` es interno del shell).
- **Linux**: `xdg-open http://localhost:4321` (ejecutable real).

```
import subprocess

# Windows
subprocess.run(["start", "http://localhost:4321"], shell=True)

# Linux / macOS
# subprocess.run(["xdg-open", "http://localhost:4321"])
```

> ⚠️ **`shell=True` con cuidado:** solo úsalo cuando el comando es interno del shell (como `start` o `dir`). Con `shell=True` y texto del usuario, un atacante podría inyectar comandos. Pasa siempre la lista de argumentos, no un string concatenado.

🌎 Lo que nunca cambia

Concepto	Windows	Linux
`subprocess.run([...])`	✅	✅
`subprocess.Popen([...])`	✅	✅
`capture_output=True`, `text=True`	✅	✅
`timeout`, `returncode`	✅	✅
`communicate(input=...)`	✅	✅
`proceso.wait()`, `poll()`, `terminate()`, `kill()`	✅	✅
`proceso.pid`	✅	✅

Aprende los conceptos con `notepad.exe` y `calc.exe` y, cuando toques Linux, solo cambia la lista de comandos de la tabla de arriba.

🧠 Mini-chequeo

1. ¿Qué comando de Windows equivale a `ls` en Linux?
2. ¿Por qué en Windows `mkdir` necesita `cmd /c`?
3. ¿Cuándo tienes que usar `shell=True`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- La API de `subprocess` es idéntica en Windows y Linux: solo cambian los comandos.
- Comandos internos del shell (como `mkdir` en Windows) necesitan `cmd /c` o `shell=True`.
- Sustituye el comando de la tabla y tu código funcionará en cualquier sistema.

🐛 Vocabulario rápido

Término	Idea general
Shell	Intérprete de comandos (`cmd`, `bash`)
Comando interno	Palabra que interpreta el shell (no es un ejecutable)
cmd /c	Ejecuta un comando interno de `cmd` en Windows
shell=True	Delega la ejecución en el shell (¡con cuidado!)
xdg-open	Abre una URL o archivo con la app por defecto en Linux

📚 Volver al índice de la unidad · **Anterior:** 06 · Comunicación con procesos · **Siguiente:** 08 · Procesos nativos con multiprocessing

08 — Procesos nativos con multiprocessing

Clona tu propio programa Python y ejecútalo en varios núcleos 🔁

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 08 · Procesos nativos con multiprocessing

📬 La idea en una frase

> Mientras que `subprocess` sirve para lanzar programas **externos** (la calculadora o un comando del sistema), `multiprocessing` te permite **clonar tu propio programa Python** para ejecutar varias de tus funciones al mismo tiempo en distintos núcleos del procesador.

Hasta ahora has visto cómo abrir aplicaciones de terceros. Pero, ¿qué pasa si quieres que tu propio servidor atienda a cinco clientes a la vez? No puedes usar `subprocess.Popen(["python", "mi_script.py"])` de forma elegante para cada tarea interna. Ahí entra la librería estándar `multiprocessing`.

⚖️ subprocess vs multiprocessing

Característica	`subprocess`	`multiprocessing`
¿Qué lanza?	Ejecutables externos (`ping`, `notepad.exe`, `ls`)	Funciones de tu propio código Python
Paso de datos	Difícil: a través de tuberías de texto (`stdin`/`stdout`)	Fácil: pasas variables y objetos como argumentos a la función
Uso típico	Automatización del sistema operativo y scripts de bash	Programación concurrente, servidores y cálculos matemáticos pesados

Es el mismo criterio que usaste para distinguir los conceptos del punto 3: `subprocess` **abre otros programas**; `multiprocessing` **paraleliza tu propio código** repartiendo funciones entre núcleos reales.

🚀 Lanzando funciones: Process, start() y join()

Para crear un proceso interno, utilizas la clase `Process`. Le indicas qué función (`target`) debe ejecutar y qué argumentos (`args`) necesita:

```
import multiprocessing
import os
import time

def trabajador(nombre, repeticiones):
    print(f"[{os.getpid()}] Hola, soy el trabajador {nombre}")
    time.sleep(repeticiones)
    print(f"[{os.getpid()}] El trabajador {nombre} ha terminado")

# IMPORTANTE EN WINDOWS: siempre debes proteger la creación de procesos
if __name__ == '__main__':
    print(f"[{os.getpid()}] Soy el proceso PADRE")

    # 1. Instanciar los procesos
    p1 = multiprocessing.Process(target=trabajador, args=("Alice", 2))
    p2 = multiprocessing.Process(target=trabajador, args=("Bob", 1))

    # 2. Arrancarlos (pasan al estado LISTO/EJECUCIÓN)
    p1.start()
    p2.start()

    # 3. Esperar a que terminen (equivalente a wait() en subprocess)
    p1.join()
    p2.join()

    print(f"[{os.getpid()}] Todos los trabajadores han finalizado")
```

- `target`: el nombre de la función (sin paréntesis, porque no quieres ejecutarla ahora, sino pasársela al proceso).
- `args`: una tupla con los argumentos. Si solo pasas un argumento, recuerda poner una coma al final: `args=("Alice",)`.
- `start()`: lanza el proceso al sistema operativo.
- `join()`: bloquea al proceso padre hasta que ese hijo termine (lo viste con `wait()` en Popen).

> ⚠️ El bloque `if __name__ == '__main__':` no es decoración: en Windows no existe `fork` y cada proceso hijo **vuelve a importar el script desde el principio**. Sin esa protección, los hijos

> crearían más hijos en un bucle infinito. En Linux, con `fork`, casi nunca lo notas… pero escríbelo siempre: te ahorra un susto al cambiar de sistema.

🧱 El muro del aislamiento (la burbuja de memoria)

Recuerda la teoría del punto 1: cada proceso vive en su propia burbuja aislada. Si creas un proceso nuevo a partir de tu código, el sistema operativo hace un **clon de toda la memoria RAM** en ese instante. Lo que modifique el hijo, no lo ve el padre.

Observa esta trampa didáctica:

```
import multiprocessing

# Variable global
puntuacion = 10

def sumar_puntos():
    global puntuacion
    puntuacion += 5
    print(f"Hijo: La puntuación es {puntuacion}")

if __name__ == '__main__':
    p1 = multiprocessing.Process(target=sumar_puntos)
    p1.start()
    p1.join()

    print(f"Padre: La puntuación es {puntuacion}")
```

Salida:

```
Hijo: La puntuación es 15
Padre: La puntuación es 10
```

¿Qué ha pasado? Al hacer `start()`, el hijo clonó la variable `puntuacion = 10` en su propia RAM. El hijo suma 5 y **su clon** vale 15. Cuando el hijo muere, su burbuja de memoria se destruye. El proceso padre imprime su propia variable original, que nunca fue modificada: **los procesos no comparten variables globales**.

🤝 Rompiendo el muro con Manager

Si quieres que un proceso servidor actualice un diccionario de usuarios y otro proceso pueda leerlo, necesitas **comunicación entre procesos (IPC)** estructurada. Python nos ofrece `multiprocessing.Manager()`.

El Manager crea un proceso servidor invisible que gestiona la memoria y permite que los demás procesos accedan a diccionarios o listas compartidas (conocidos como **proxies**):

```
import multiprocessing

def registrar_usuario(diccionario_compartido, nombre, edad):
    # Esto SÍ modifica la memoria que ven todos
    diccionario_compartido[nombre] = edad
    print(f"Proceso hijo registró a {nombre}")

if __name__ == '__main__':
    # Creamos el gestor de memoria compartida
    with multiprocessing.Manager() as manager:
        # manager.dict() crea un diccionario especial que cruza la burbuja
        buzones = manager.dict()

         p1 = multiprocessing.Process(target=registrar_usuario, args=(buzones, "Alice",
25))
          p2 = multiprocessing.Process(target=registrar_usuario, args=(buzones, "Bob",
30))

        p1.start()
        p2.start()
        p1.join()
        p2.join()

        # El padre SÍ puede ver los cambios
        print(f"Diccionario final en el padre: {buzones}")
```

El `manager.dict()` no es un dict normal: es un **proxy** que traduce cada lectura y escritura en mensajes al proceso servidor del Manager. Cuesta más que un dict de toda la vida, pero es la forma limpia de compartir estado entre burbujas.

🔒 Sincronización con Lock (condiciones de carrera)

Compartir memoria trae un problema crítico: ¿qué pasa si dos procesos intentan modificar el mismo buzón a la vez? Los datos pueden sobrescribirse y corromperse. A esto se le llama **condición de carrera** (*race condition*): el mismo nombre y el mismo fallo que viste con los hilos, porque la lógica es idéntica.

Para solucionarlo, existe el candado: `multiprocessing.Lock`. Es un "testigo" que solo puede tener un proceso a la vez:

1. El proceso pide el candado (`lock.acquire()`).
2. Modifica la variable compartida (esta zona protegida se llama **sección crítica**).
3. Suelta el candado (`lock.release()`) para que pase el siguiente.

> 💡 **Truco:** en Python puedes usar el bloque `with lock:` para que el `acquire` y el `release` se hagan automáticamente, igual que con los hilos.

```python
import multiprocessing
import time

def incrementar_contador(contador_compartido, candado):
    for _ in range(100):
        # Protegemos la sección crítica. Si otro proceso tiene el Lock,
        # este proceso se queda bloqueado esperando.
        with candado:
            # Sección crítica: modificar la variable compartida
            valor_actual = contador_compartido.value
            time.sleep(0.001)  # Simulamos que el cálculo tarda
            contador_compartido.value = valor_actual + 1

if __name__ == '__main__':
    # Variable numérica compartida (empieza en 0)
    contador = multiprocessing.Value('i', 0)
    # Creamos un candado para organizar a los trabajadores
    candado = multiprocessing.Lock()

    procesos = []
    # Lanzamos 5 procesos
    for i in range(5):
        # Hay que pasarles el candado como parámetro
            p = multiprocessing.Process(target=incrementar_contador, args=(contador,
candado))
        procesos.append(p)
        p.start()

    for p in procesos:
        p.join()

    print(f"Valor final del contador: {contador.value}")  # Siempre será 500
```

Si ejecutas ese mismo código **sin** el Lock, verás que el resultado final es impredecible (por ejemplo, 143, 290…), porque los procesos leen y escriben a la vez, machacando el trabajo del anterior. Todo esto lo desmenuzarás a fondo en la UD 4 · Sincronización, donde los mismos patrones aparecen con hilos.

🏋️ Reto avanzado (no evaluable)

> ⭐ Este ejercicio **no entra en la evaluación**: es para quien quiera estirarse más allá del examen. Intenta resolverlo por tu cuenta antes de mirar la solución.

El cribo de primos en paralelo

Quieres contar cuántos números primos hay entre 1 y 2.000.000. En serie es un buen paseo de espera; en paralelo, una carrera de relevos.

Fases del reto:

1. Escribe `es_primo(n)` en serie y cronometra el recuento completo con `time.time()`.
2. Reparte el rango entre **4 procesos** con `multiprocessing.Pool` y `pool.map()`. Compara el tiempo con el de la serie.
3. Cambia el enfoque: ahora cada proceso hijo no devuelve una cuenta, sino la **lista completa** de primos que le tocan; el padre los concatena y los ordena con `sorted()`. ¿Cuánto pesa ahora el paso de datos entre procesos?
4. **Extra:** usa `pool.imap_unordered()` y muestra los parciales según llegan, sin esperar al orden del mapa. ¿En qué se diferencia de `map()`?

> **Pista 1:** `Pool(4).map(es_primo, rango)` devuelve una lista de `True`/`False` del mismo tamaño que el rango: cuenta los `True` con `sum(...)` al final.
>
> **Pista 2:** para repartir rangos enteros, divide el total en 4 bloques con paso 4: `[range(i, límite, 4) for i in range(4)]`, y lanza `pool.map(contar_rango, bloques)`.
>
> **Pista 3:** `imap_unordered` es un generador: recórrelo con un bucle `for` e imprime a medida que llegan los resultados; `map` espera a tener la lista entera.

> › 🔄 **Ver solución**

🧠 Mini-chequeo

1. ¿Por qué debes usar `if __name__ == '__main__':` al crear procesos en Windows?
2. Si un proceso hijo modifica una lista global estándar de tu script, ¿el proceso padre verá la lista modificada?
3. ¿Qué hace `manager.dict()`?
4. ¿Para qué sirve un `Lock` en programación concurrente?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- `multiprocessing` te permite crear procesos (`Process`), arrancarlos (`start()`) y esperarlos (`join()`) ejecutando funciones de tu propio código.
- Las variables globales no se comparten entre procesos porque el sistema operativo clona la RAM; necesitas `multiprocessing.Manager()` para pasar datos de un proceso a otro.
- Cuando varios procesos modifican los mismos datos compartidos, debes proteger el código conflictivo con un `Lock` para evitar sobreescrituras.

🐛 Vocabulario rápido

Término	Idea general
`Process(target=f)`	Clase que prepara la ejecución paralela de una función
`.start()`	Envía el proceso al planificador del sistema operativo (lo arranca)
`.join()`	Detiene al proceso padre hasta que el proceso hijo finalice
`Manager`	Utilidad del sistema para crear estructuras de memoria compartida (IPC)
`Lock`	Candado de exclusión mutua para proteger secciones críticas de código
Condición de carrera	Fallo que ocurre cuando varios procesos modifican un dato a la vez sin sincronizarse

📚 Volver al índice de la unidad · **Anterior:** 07 · Compatibilidad Windows / Linux · **Siguiente:** 09 · Ejemplo resuelto: estado compartido con procesos

09 — Ejemplo resuelto: estado compartido con procesos

Un almacén en memoria que cuatro procesos actualizan a la vez, sin perder una sola escritura 🧱

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 09 · Ejemplo resuelto: estado compartido con procesos

---

📬 La idea en una frase

> Este punto es un ejemplo resuelto y explicado paso a paso: montas un **almacén de datos en memoria** (`manager.dict()`) al que cuatro **procesos** (`multiprocessing.Process`) escriben a la vez, protegido por un **candado de procesos** (`multiprocessing.Lock`), y al terminar guardas el estado a un fichero JSON para que **sobreviva al cierre del programa**.

Es la pieza que une todo lo que has visto en la unidad: `Process` para paralelizar, `Manager` para cruzar la burbuja de memoria y `Lock` para que no se pisen. Si tienes claro este ejemplo, tienes la base sobre la que se construye cualquier servidor concurrente con procesos (y lo verás enchufado a un servidor real en la UD 6).

🎯 El problema

Imagina un almacén de pedidos de una tienda:

- Llegan pedidos por cuatro canales a la vez (cuatro procesos trabajadores).
- Cada trabajador **suma** unidades en un diccionario compartido: `{"manzanas": 12, "peras": 5, …}`.
- Al final del turno, el dueño quiere ver el recuento total **y guardarlos** para retomarlos mañana.

Si cada trabajador tuviera su propio diccionario (como pasaba en la trampa del punto 8), cada uno vería solo sus propios pedidos y el total sería una ficción. Hace falta memoria compartida… entre burbujas.

🧱 Paso 1: la estructura base del programa

El programa tiene tres piezas, y se escribe en este orden:

1. **El trabajador**: la función que lanzará cada proceso hijo. Recibe tres cosas: el almacén compartido, el candado y su lista de pedidos.
2. **El guardado y la carga**: dos utilidades que vuelcan el almacén a JSON y lo recuperan (persistencia).
3. **El main**: crea el `Manager`, el `Lock`, lanza los procesos, espera con `join()` y guarda el resultado.

> 💡 **Convención de este curso:** las utilidades y el trabajador van arriba del todo; el punto de entrada `if __name__ == '__main__':` va el último. Así el script se puede importar desde otro sin que se lanzen procesos por accidente.

👷 Paso 2: el trabajador

```
import json
import multiprocessing
import random
import time

def trabajador(almacen, candado, pedidos):
    """Proceso hijo: registra sus pedidos en el almacén compartido."""
    for fruta, unidades in pedidos:
        time.sleep(random.uniform(0.01, 0.05))  # Simula el trabajo real
        # Sección crítica: leer, calcular y escribir DE UNA VEZ
        with candado:
            total = almacen.get(fruta, 0)
            almacen[fruta] = total + unidades
        print(f"  [{multiprocessing.current_process().name}] "
              f"+{unidades} {fruta} (total parcial: {almacen.get(fruta)})")
```

Fíjate en los tres detalles clave:

- **`with candado:`** protege la lectura (`almacen.get`) y la escritura (`almacen[fruta] = …`) **juntas**. Si leyeras fuera del candado, dos procesos leerían el mismo valor previo y el último machacaría al primero: condición de carrera.
- **`multiprocessing.current_process().name`** te dice qué proceso habla (“Process-1”, “Process-2”…). Es la forma de trazar la salida, igual que nombrabas los hilos con `.name` en otros ejemplos.

- El trabajador **no sabe** que hay otros procesos. Solo sabe que su almacén responde a `get` y a `asignación`.

💾 Paso 3: persistencia (guardar y cargar el almacén)

El almacén del `Manager` vive en memoria: si apagas el programa, desaparece. Para que el estado sobreviva, lo volvamos a un fichero JSON al terminar y lo recargamos al arrancar:

```
def guardar_almacen(almacen, fichero="almacen.json"):
    # dict(almacen) convierte el proxy en un dict normal de Python,
    # que sí se puede serializar con json.dump
    with open(fichero, "w", encoding="utf-8") as f:
        json.dump(dict(almacen), f, ensure_ascii=False, indent=2)
    print(f"💾 Almacén guardado en {fichero}")

def cargar_almacen(fichero="almacen.json"):
    try:
        with open(fichero, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}
```

> ⚠️ **Trampa clásica:** `json.dump(almacen)` falla directamente porque un `proxy` de `Manager` **no es un dict**. Necesitas convertirlo primero con `dict(almacen)`, que hace una "foto" del estado actual y devuelve un dict normal y corriente.

🚀 Paso 4: el main que lo pone todo en marcha

```python
if __name__ == '__main__':
    # 1) Recuperamos el turno anterior (si existe)
    cargado = cargar_almacen()
    print(f"📥 Estado previo: {cargado}")

    # 2) Crear el gestor de memoria compartida y el candado
    with multiprocessing.Manager() as manager:
        almacen = manager.dict(cargado)  # Arranca desde el estado cargado
        candado = multiprocessing.Lock()

        # 3) Repartir el trabajo entre 4 procesos
        reparto = [
            [("manzanas", 3), ("uvas", 2)],
            [("peras", 5)],
            [("manzanas", 1), ("naranjas", 4)],
            [("uvas", 1), ("peras", 2), ("naranjas", 2)],
        ]

        procesos = []
        for i, pedidos in enumerate(reparto, start=1):
            p = multiprocessing.Process(
                target=trabajador,
                args=(almacen, candado, pedidos),
                name=f"trabajador-{i}",
            )
            procesos.append(p)
            p.start()

        # 4) Esperar a que TODOS terminen (imprescindible antes de leer)
        for p in procesos:
            p.join()

        # 5) Ver el resultado final y guardarlo
        print(f"\n🏁 Total del turno: {dict(almacen)}")
        guardar_almacen(almacen)
```

🔍 Traza paso a paso

```
1. cargar_almacen("almacen.json")
   → No existe el fichero → devuelve {} → "Estado previo: {}"

2. multiprocessing.Manager() → proceso servidor del Manager en marcha
   manager.dict({}) → proxy listo
   multiprocessing.Lock() → candado libre

3. Cuatro Process(target=trabajador, args=(almacen, candado, pedidos))
   start() × 4 → el SO clona la memoria 4 veces (4 burbujas)
   → CADA burbuja ve una COPIA de "almacen"... pero al ser un proxy,
     cada get/set se convierte en un mensaje al proceso del Manager

4. Los trabajadores hacen lectura → suma → escritura DENTRO de "with candado:"
   → Solo UN proceso a la vez en la sección crítica
   → El Manager aplica cada escritura sobre SU copia maestra

5. join() × 4 → el padre espera a que los 4 hijos finalicen
   → El Manager sigue vivo (está dentro del "with") → el proxy responde

6. dict(almacen) → foto del estado → json.dump → almacen.json en disco
   → Salida del bloque "with" → el proceso del Manager termina
```

Salida típica:

```
📥 Estado previo: {}
  [trabajador-1] +3 manzanas (total parcial: 3)
  [trabajador-3] +1 manzanas (total parcial: 4)
  [trabajador-2] +5 peras (total parcial: 5)
  [trabajador-4] +1 uvas (total parcial: 3)
  ...
🏁 Total del turno: {'manzanas': 4, 'uvas': 3, 'peras': 7, 'naranjas': 6}
💾 Almacén guardado en almacen.json
```

El orden en el que aparecen los mensajes **varía en cada ejecución** (el planificador reparte la CPU a su aire), pero el total final **siempre es el mismo**: esa es la garantía que da el candado.

🧨 Paso 5: quita el candado y mira la catástrofe

Comenta las dos líneas del `with candado:` (deja la indentación, solo quita el contexto) y vuelve a ejecutar. Verás totales distintos en cada corrida (por ejemplo, `manzanas: 2` en vez de 4), porque dos procesos leen el mismo valor previo y el segundo machaca el incremento del primero. Ejecuta tres o cuatro veces: **nunca** sabrás qué escritura se pierde.

Es la misma condición de carrera que viste con hilos, con el mismo nombre y el mismo arreglo, solo que aquí el choque ocurre entre burbujas de memoria en lugar de entre hilos.

🧠 Mini-chequeo

1. ¿Por qué el trabajador recibe `almacen` y `candado` por argumentos en vez de usar variables globales?
2. ¿Qué pasa si haces `json.dump(almacen)` sin convertirlo antes con `dict(...)`?
3. ¿Por qué el `join()` de todos los procesos va antes de leer el almacén final?
4. Si ejecutas el programa dos días seguidos, ¿por qué el segundo día arranca con datos del primero?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `manager.dict()` es la memoria compartida entre procesos: se lo pasas a cada hijo como argumento y todos hablan con el mismo almacén a través del proceso servidor del Manager.
- `multiprocessing.Lock` protege la sección crítica (leer-modificar-escribir) y garantiza que ningún incremento se pierda aunque los procesos corran a la vez.
- La persistencia se resuelve convirtiendo el proxy a dict normal (`dict(almacen)`) y volcándolo a JSON; al arrancar, el mismo JSON alimenta al `manager.dict()` como estado inicial.

🐛 Vocabulario rápido

Término	Idea general
`manager.dict()`	Diccionario proxy compartido entre procesos (memoria compartida)
Sección crítica	Trozo de código que solo puede ejecutar un proceso a la vez
`with candado:`	Bloque protegido: acquire automático al entrar, release al salir
`dict(proxy)`	"Foto" del proxy como dict normal, usable con `json.dump`
Persistencia	Que los datos sobreviven al cierre del programa (fichero JSON aquí)
`current_process().name`	Nombre del proceso en ejecución, para trazar la salida

📚 Volver al índice de la unidad · **Anterior:** 08 · Procesos nativos con multiprocessing · **Siguiente:** 10 · Procesos en la práctica

10 — Procesos en la práctica

Sé el código, el ring y los ejercicios del lápiz 🖍️

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 10 · Procesos en la práctica

📬 La idea en una frase

> Aterriza todo lo anterior: sé el código que abre dos aplicaciones a la vez, mira pelearse a `run()` y `Popen()` en el ring, y pon a prueba tus manos con los ejercicios del lápiz.

⭐ Sé el código: abrir bloc de notas y calculadora

> "Sé el programa que abre dos aplicaciones a la vez. Traza cada paso."

```
import subprocess, time

print("🚀 Abriendo bloc de notas...")
notepad = subprocess.Popen(["notepad.exe"])
print(f"   → PID: {notepad.pid}")

print("🚀 Abriendo calculadora...")
calc = subprocess.Popen(["calc.exe"])
print(f"   → PID: {calc.pid}")

print("\nAmbos programas están abiertos.")
print("El usuario puede escribir en el bloc de notas y usar la calculadora.")

# Mientras tanto, Python no está bloqueado
for i in range(5):
    print(f"   Python haciendo cosas... ({i+1}/5)")
    time.sleep(1)

print("\nCerrando programas...")
notepad.terminate()
calc.kill()
print("Hecho 🏁")
```

Traza paso a paso:

```
1. subprocess.Popen(["notepad.exe"])
   → El SO crea un nuevo proceso
   → Windows busca notepad.exe en el PATH
   → Lo carga en memoria
   → Asigna PID 12345
   → Python recibe el control inmediatamente

2. print("PID: 12345")

3. subprocess.Popen(["calc.exe"])
   → Mismo proceso, PID 12346

4. Python hace 5 cosas mientras ambos están abiertos
   → 3 procesos independientes: Python, notepad, calc

5. notepad.terminate()
   → Envía señal de cierre al bloc de notas

6. calc.kill()
   → Mata la calculadora forzosamente
```

Fíjate en el paso 4: **Python no se bloquea**. Mientras los dos programas están abiertos, el proceso de Python sigue imprimiendo. Tres procesos independientes conviven a la vez, cada uno con su PID, su memoria y su turno de CPU (punto 1 y punto 2).

🥊 El ring de los conceptos — run() vs Popen()

> *Dos funciones de `subprocess` se citan en el cuadrilátero para resolver, de una vez, quién lanza mejor.*

run(): — ¡Yo soy el rey de la simplicidad! Lanzas un comando, esperas, y ¡zas! tienes el resultado.

Popen(): — Sí, pero mientras tú esperas como un muñeco, yo puedo lanzar un proceso y seguir haciendo otras cosas. ¿Para qué esperar si no hace falta?

run(): — ¿Y si necesitas el resultado? Conmigo es directo: `result.stdout` y ya. Tú necesitas `communicate()`, más vueltas…

Popen(): — Pero imagina que quieres lanzar 3 programas a la vez. Conmigo puedes lanzarlos, irte a hacer café, y luego matarlos a todos. Tú tendrías que esperar a que termine cada uno antes de lanzar el siguiente.

run(): — Vale, vale... para procesos que viven en segundo plano, eres mejor. Pero para comandos rápidos y resultados inmediatos, soy más limpio.

> **Moraleja**: `run()` para comandos rápidos que necesitan respuesta. `Popen()` para procesos que deben vivir en segundo plano.

📝 Aprieta el lápiz

Pon a prueba lo aprendido. Intenta resolver cada ejercicio antes de mirar la solución.

1. PID del proceso

Crea un programa que imprima su propio PID y el PID del proceso padre.

> › 🔄 **Solución**

2. Lanzador de apps

Usa `Popen` para abrir 3 programas distintos (bloc de notas, calculadora, navegador).

> › 🔄 **Solución**

3. Comando con timeout

Lanza `ping` con timeout de 3 segundos. Captura el error si excede.

> › 🔄 **Solución**

4. Comunicación bidireccional

Crea un proceso hijo que lea de stdin y devuelva el texto en mayúsculas.

> › 🔄 **Solución**

5. Caza de procesos

Lanza 5 procesos `notepad` y luego mata todos menos uno.

> 🔄 **Solución**

🧠 Mini-chequeo

1. En el "Sé el código", ¿cuántos procesos hay vivos mientras Python imprime sus 5 mensajes?
2. ¿Cuándo usarías `run()` y cuándo `Popen()`?
3. ¿Qué devuelve `os.getppid()`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `Popen` lanza aplicaciones en segundo plano y Python sigue trabajando mientras viven.
- `run()` es para respuestas inmediatas; `Popen()` para procesos de larga vida (ring).
- Con `os.getpid()`, `os.getppid()`, `communicate()` y `terminate()` ya sabes jugar con procesos de verdad.

🐛 Vocabulario rápido

Término	Idea general
Sé el código	Técnica de traza mental: ejecutar el programa paso a paso en tu cabeza
os.getpid()	PID del proceso actual
os.getppid()	PID del proceso padre
communicate()	Enviar datos al hijo y leer su respuesta
terminate() / kill()	Cierre suave / muerte forzosa

📚 Volver al índice de la unidad · **Anterior:** 09 · Ejemplo resuelto: estado compartido con procesos · **Siguiente:** 11 · Cierre

11 — Cierre: consolida lo aprendido

Sé el proceso, laboratorio con subprocess y el ring final de la unidad 🧠

> 🗺️ **Estás en:** 🚀 **UD 2 · Gestión de procesos** → 11 · Cierre

Has terminado la teoría: la burbuja de memoria y el PID, los cinco estados, paralela contra distribuida, `run()`, `Popen()`, los pipes con `communicate()`, la compatibilidad Windows/Linux, los procesos nativos con `multiprocessing` y el ejemplo resuelto de estado compartido. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 10 y antes de abrir los boletines.

⭐ Sé el proceso

> *Eres un proceso de Python recién lanzado con `subprocess.Popen(["python", "calcula.py"])`. Acaban de asignarte un PID: 12345.*

¿Qué pasa?

1. El sistema operativo crea tu proceso: asigna PID **12345**, reserva tu **burbuja de memoria** (código, estado, contador de programa).
2. Pasas al estado **NUEVO** y, en cuanto estás cargado, a **LISTO**: esperas tu turno de CPU en la cola del planificador.
3. La CPU te toca: pasas a **EJECUCIÓN**. Ejecutas las instrucciones de `calcula.py` una a una.
4. Llamas a `input()` para leer de stdin: te bloqueas (**BLOQUEADO**) esperando que el usuario escriba algo.
5. El usuario escribe y pulsa **ENTER**; la E/S termina y vuelves a **LISTO**.
6. La CPU te toca otra vez (**EJECUCIÓN**), terminas tu último `print()` y llegas a **TERMINADO**.
7. Tu padre espera con `proceso.wait()` o `poll()` y recoge tu código de retorno: **0**.

> 💡 **Ahora tú:** ¿y si tu padre lanza a la vez otros dos procesos (`calc.exe` y `notepad.exe`)? Los tres avanzáis turnándoos la CPU (concurrencia) y, si hay varios núcleos, alguno ejecutará de verdad a la vez (paralelismo). Eso es lo que viste en el punto 3.

🔥 Fireside Chat: Proceso vs Hilo

> *Un proceso y un hilo se sientan junto a la chimenea a resolver, de una vez, quién es más ligero.*

Proceso: — Yo soy la unidad completa: memoria propia, recursos, PID. Si me cuelgo, tú ni te enteras. Soy el punto 1.

Hilo: — ¿Y para qué quieres toda esa burbuja? Yo vivo **dentro** de un proceso y comparto su memoria. Soy mucho más barato de crear.

Proceso: — Pero mis fallos no tumban a nadie más. Tú, si se te va un puntero, te llevas por delante al proceso entero.

Hilo: — Cierto, pero yo puedo compartir variables directamente. Tú necesitas pipes y `communicate()` para hablar con tu vecino.

Proceso: — Sí, pero en Python tengo otra ventaja: mi propio intérprete. Los hilos comparten el **GIL**, así que mi paralelismo con `multiprocessing` es real.

Hilo: — Vale, vale. Tú para aislamiento y paralelismo real; yo para tareas ligeras que comparten memoria. ¿Empate?

Proceso: — *sonríe* Empate… hasta la UD 3.

> **Moraleja**: el proceso aísla y paraleliza de verdad; el hilo es ligero y comparte memoria. Los hilos son el plato principal de la UD 3 · Hilos y concurrencia.

🕵️ ¿Quién soy?

1. Soy el identificador único que el SO asigna a cada proceso.
2. Soy el estado en el que el proceso espera un recurso (I/O, socket, sleep).
3. Soy el mecanismo por el que una sola CPU parece ejecutar varios procesos a la vez.
4. Soy el módulo de Python que reparte tareas en procesos paralelos reales.
5. Soy el tubo que conecta la salida de un proceso con la entrada de otro.
6. Soy el proceso que ya terminó pero que su padre no ha recogido todavía.

> › 🔁 **Respuestas**

CONRAD VS EL MUNDO: “maté el proceso y no volvió”

CONRAD: — “Clásico: *‘he lanzado mi programa con subprocess.run y no me devuelve el control’*. ¡Pues claro! `run()` espera a que el comando termine, y si lanzas `notepad.exe`, el bloc de notas no se cierra nunca: te quedas colgado hasta que el usuario cierre la ventana. Si querías lanzar y seguir, eso es `Popen`, no `run()`.”

CONRAD: — “Y lo mejor: *‘puse shell=True para abrir el navegador y ahora un comando borra mis archivos’*. Pues sí: `shell=True` delega en el intérprete de comandos. Si le pasas un string con datos del usuario, es un agujero de seguridad. Lista de argumentos, nunca strings concatenados. O `cmd /c`, que para eso está.”

CONRAD: — “Y no me vengas con *‘¿será que mi proceso se ha convertido en zombie?’*. Míralo con `proceso.poll()`: si te devuelve el código de retorno, ya terminó y solo falta que lo recojas con `wait()` o `poll()`. Zombie sin recoger = entrada ocupada en la tabla de procesos. A diagnosticar.”

Laboratorio de tortura: el lanzador con cierre automático

> **Duración:** 40 minutos **Herramienta:** Python 3 (`subprocess` y `time`, sin instalar nada)

Escenario: construye un “lanzador de aplicaciones” que abra tres programas (bloc de notas, calculadora y Paint), muestre sus PIDs, los deje vivir 5 segundos y luego cierre solo los que sigan abiertos.

Tareas paso a paso:

1. Define una función `lanzar_apps()` que cree los tres `Popen` y los guarde en una lista de tuplas `(nombre, proceso)`.
2. Recorre la lista mostrando `nombre` y `proceso.pid`.
3. Espera 5 segundos con `time.sleep(5)` mientras las apps están abiertas.
4. Recorre otra vez la lista: si `proceso.poll() is None` (sigue viva), llámale `terminate()`; si ya terminó, imprime que se cerró sola.
5. Tras cada `terminate()`, recoge el código de retorno con `proceso.wait()` e imprímelo.
6. Envuelve la llamada en `if __name__ == "__main__":`.

Fallo intencionado: en el paso 5, en lugar de `wait()` tras `terminate()`, **borra esa línea** y deja el programa así. ¿Qué pasa? El padre lanza las apps, las mata, pero **nunca recoge su código de retorno**: en Linux/macOS, los procesos terminados quedan como **zombies** ocupando una entrada en la tabla de procesos hasta que el padre muere o los recoge. En **Windows** no hay zombies como tal: el manejador del proceso queda abierto y consultable, pero el código de retorno tampoco se limpia hasta que lo recoges con `wait()`.

> **Pista 1:** un proceso no desaparece al terminar: su código de retorno queda guardado hasta que el padre lo **recoge** con `wait()` o `poll()`. Sin esa llamada, la entrada en la tabla de procesos sigue ocupada: es el zombie de las preguntas tontas.
>
> **Pista 2:** si no ves el zombie a simple vista, añade `print(f"{nombre} → poll(): {proceso.poll()}")` justo después del `terminate()`. La diferencia está en **recoger** el código: `wait()` (o `poll()`) lo devuelve y limpia la entrada; sin esa llamada, `poll()` te seguirá devolviendo el código de un proceso que ya murió y no ha sido recogido.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Process Whisperer**	Explicar el ciclo de vida de un proceso: sus 5 estados y sus transiciones
🏅 **Launcher**	Abrir y gestionar varias aplicaciones a la vez con `Popen`, `terminate()` y `kill()`
🏅 **Pipe Talker**	Comunicar un proceso padre y un hijo con `communicate()`
🏅 **Parallel Thinker**	Distinguir paralela, distribuida y concurrencia, y saber cuándo usar `multiprocessing`

🧠 Atrévete a pensar

1. ¿Por qué un proceso no puede compartir memoria directamente con otro?
2. ¿Qué pasa si un proceso hijo muere y su padre nunca llama a `wait()` ni `poll()`?
3. ¿Cuándo usarías `multiprocessing` en lugar de `subprocess`?
4. ¿Por qué `shell=True` es peligroso si le pasas datos del usuario?

5. ¿Qué ventaja tiene `Popen` sobre `run()` para lanzar 3 aplicaciones a la vez?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Identificador único del proceso (3 letras)
4. Estado en el que el proceso espera a que la CPU se libre (5 letras)
6. Señal de cierre suave de Popen (9 letras)
8. Proceso terminado sin recoger por su padre (6 letras)

Vertical:
2. Módulo Python que crea procesos en paralelo real (14 letras)
3. Estado del proceso mientras espera un recurso (9 letras)
5. Tubería que conecta la salida de un proceso con la entrada de otro (4 letras)
7. Método de Popen que pregunta sin bloquear (4 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **"¿Qué es un proceso y en qué se diferencia de un programa?"**
2. **"Explica el ciclo de vida de un proceso."**
3. **"¿Qué diferencia hay entre computación paralela y distribuida?"**
4. **"¿Cómo lanzarías un programa externo desde Python y leerías su salida?"**
5. **"¿Qué diferencia hay entre `run()` y `Popen()`? ¿Cuándo usarías cada uno?"**
6. **"¿Cómo comunicarías dos procesos entre sí?"**

> 💡 **Cómo encararlas:** la 4 y la 5 son las "preguntas reina". Para la 4, recorre la cadena: `subprocess.run([...], capture_output=True, text=True)` → `resultado.stdout`; si el proceso necesita datos, `Popen(stdin=PIPE, stdout=PIPE)` + `communicate(input=...)`. Para la 5, repite la moraleja del punto 10: `run()` para respuestas inmediatas, `Popen()` para segundo plano. Y para la 1 no olvides la **burbuja de memoria** y el **PID**. Si sabes contarlo fluido, ya eres medio programador de sistemas.

🤷 No hay preguntas tontas

> ❓ **¿Cuántos procesos puede tener mi sistema?**

Depende de la **memoria RAM**. Cada proceso ocupa memoria. En Windows puedes verlos en el Administrador de tareas.

> ❓ **¿Qué ocurre si un proceso hijo termina pero el padre nunca consulta su estado?**

El hijo se convierte en un **proceso zombie**. Ha liberado su memoria RAM, pero conserva su entrada en la tabla de procesos del SO a la espera de que el padre lea su código de salida mediante `wait()`, `poll()` o `join()`.

> ❓ **¿Y si el proceso padre muere repentinamente antes que sus hijos?**

Los procesos hijos se convierten en **huérfanos**. En sistemas Linux/POSIX son adoptados automáticamente por el proceso raíz `init` (PID 1) o `systemd`, que se encarga de recoger sus estados cuando finalicen. En Windows, el propio sistema se encarga de ellos.

> ❓ **`run()` vs `Popen()` — ¿cuándo usar cada uno?**

- `run()`: cuando necesitas el resultado y puedes esperar.
- `Popen()`: cuando el proceso debe vivir en segundo plano mientras tú haces otras cosas.

> ❓ **¿Puedo lanzar cualquier programa?**

Sí, cualquier ejecutable. Pero el **PATH** debe incluirlo o debes dar la ruta completa.

> ❓ **¿Por qué no vale un dict normal entre procesos?**

Porque los procesos **no comparten memoria**: cada uno clona las variables al arrancar. Un dict normal sería como tener cinco cuadernos distintos en vez de uno compartido. `manager.dict()` vive en el proceso del Manager y todos hablan con él por mensajes (punto 9).

> ❓ **¿Y si me olvido de poner el candado?**

A veces funciona… hasta que no. Es la condición de carrera: dos procesos leen el mismo valor previo y el segundo machaca al primero. Prueba a quitar el `with candado:` y ejecuta tres o cuatro veces: nunca sabrás qué escritura se pierde.

> ❓ **¿Los datos sobreviven si apago el programa?**

No de serie: el `manager.dict()` vive en memoria. Para que sobrevivan, volcándolos a un fichero (`json.dump(dict(almacen), f)`) al terminar y recargándolo al arrancar. Fíjate en que hay que

convertir el proxy a dict con `dict(...)` antes de serializarlo.

> ❓ **¿Cuántos procesos lanzo a la vez?**

Como mucho, tantos como **núcleos** de CPU tengas (uno por núcleo es el punto de partida). Si lanzas más, el planificador se dedica a cambiar de proceso en vez de a trabajar.

> ❓ **Si `multiprocessing` duplica la memoria al crear un proceso, ¿es muy costoso para el sistema?**

Sí: crear procesos es considerablemente más pesado en memoria y en tiempo de CPU (overhead) que crear hilos. Por eso, en sistemas de alta producción se suelen usar **pools de procesos** (`ProcessPoolExecutor`) para reutilizar los ya creados en lugar de instanciar y destruir procesos constantemente.

🎬 Poscréditos

> *El bloc de notas y la calculadora se abren a la vez. Tres procesos independientes conviven: Python, notepad y calc.*

El usuario escribe en el bloc de notas mientras la calculadora suma. La CPU alterna entre ellos: nadie se entera de que se turnan.

Notepad se cierra con un `terminate()` y su código de retorno vuela de vuelta a Python por el pipe.

Los procesos se comunican sin compartir memoria. Cada uno en su burbuja.

PRÓXIMAMENTE EN UD 3: *Los procesos son pesados: cada uno su memoria, su burbuja, su PID. Pero ¿y si quieres que varias tareas compartan memoria y se turnen la CPU? Necesitas algo más ligero: los hilos.*

✅ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce las características y la gestión de los procesos en un sistema operativo.

CE	Criterio	Cubierto
a)	Reconoce las características de los procesos	✅ Burbuja de memoria y PID (punto 1) + ⭐ Sé el proceso
b)	Distingue entre computación paralela y distribuida	✅ Punto 3 + 🔥 Fireside (punto 11)
c)	Conoce los estados de un proceso	✅ Punto 2 + ⭐ Sé el proceso
d)	Identifica las diferencias clave entre proceso e hilo	→ UD 3 (🔥 Fireside lo anticipa)
e)	Crea programas con procesos (subprocess)	✅ Puntos 4-5-9 + ⚡ Laboratorio de tortura
f)	Establece comunicación entre procesos	✅ Punto 6 + punto 8 (Manager y Lock) + ⚡ Laboratorio de tortura

> RA1d (proceso vs hilo) se cubre en la **UD 3 · Hilos y concurrencia**. RA1g (análisis de ventajas de procesos frente a hilos) también en **UD 3**. RA1h (documentación) es transversal a todo el curso.

📚 Volver al índice de la unidad · **Anterior:** 10 · Procesos en la práctica · **Siguiente:** UD 3 · Hilos y concurrencia

UD 3 — Hilos y concurrencia con Python

Multitarea dentro de un proceso: hilos con threading 🔀

En la UD 2 lanzaste procesos completos con `subprocess`: cada uno con su propia memoria y su propio PID. Ahora toca bajar un nivel y entrar *dentro* del proceso: un proceso puede tener varias tareas ejecutándose a la vez, compartiendo su memoria. Esas tareas son los **hilos** (threads): se crean en milisegundos, se comunican con variables compartidas y son la base de los servidores que atienden a muchos clientes a la vez.

Esta unidad es el primer contacto serio con los hilos en Python: los crearás con `threading.Thread`, los lanzarás con `start()`, los esperarás con `join()`, los convertirás en servidores de fondo con `daemon=True` y programarás avisos diferidos con `Timer`. También entenderás el famoso **GIL**, el candado que limita a los hilos de Python.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un hilo, en qué se diferencia de un proceso y por qué comparten la memoria.
- Crear y lanzar hilos con `threading.Thread`, `start()` y `join()`.
- Pasar argumentos a un hilo con `args`/`kwargs` y nombrar cada hilo con `.name`.
- Explicar qué es un hilo **daemon** y cuándo usarlo.
- Usar `threading.Timer` para ejecutar una función tras un retardo.
- Explicar el **GIL**: qué limita y por qué los hilos no aceleran el código CPU-bound.
- Describir el ciclo de vida de un hilo.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · De proceso a hilo	Qué es un hilo y en qué se diferencia de un proceso	Todos
02 · Tu primer hilo	`threading.Thread`, `start()`, `join()` y el hilo principal	Todos

Punto	Qué aprenderás	Nivel
03 · Hilos con argumentos	`args`, `kwargs` y el nombre de cada hilo	Todos
04 · Hilos daemon	Hilos de fondo que se sacrifican al salir	Todos
05 · Timer	Ejecutar algo una sola vez tras un retardo	Todos
06 · El GIL	El candado de CPython: por qué los hilos no aceleran la CPU	Todos
07 · Estados del hilo	El ciclo de vida del hilo	Todos
08 · Hilos en la práctica	Be the code, el ring Hilo vs Proceso	Todos
09 · Cierre	Sé el hilo, Fireside, Laboratorio de tortura…	Todos

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA2)

RA2 — Hilos (parcial): estructura, creación, esperas, daemon y GIL.

CE	Criterio	Dónde se cubre
RA2a	Identifica la estructura de un hilo	✅ Punto 1 + Punto 7
RA2b	Crea y lanza hilos con threading	✅ Puntos 2-3 + ⚡ Laboratorio (punto 9)
RA2e	Implementa esperas con join() y sleep()	✅ Puntos 2-4 + ⚡ Laboratorio (punto 9)
RA2f	Gestiona hilos daemon	✅ Punto 4 + ⚡ Laboratorio (punto 9)
RA2h	Conoce el GIL y sus limitaciones	✅ Punto 6 + Punto 8

> Los criterios RA2c (Lock), RA2d (semáforos) y RA2g (condiciones de carrera) se cubren en la **UD 4 · Sincronización**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 2 y dominas procesos y `subprocess`? Empieza por el punto 1, que parte justo de ahí: de proceso a hilo.

¿Ya sabes lanzar hilos con `start()` y `join()`? Saltar a los puntos 6 y 7. Pero si vienes de cero en multitarea, no te saltes los puntos 1 a 3.

📍 **Primer punto:** 01 · De proceso a hilo ⏭️ **Al acabar la unidad, continúa en UD 4 · Sincronización.**

01 — De proceso a hilo

Qué es un hilo y en qué se diferencia de un proceso 🧵

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 01 · De proceso a hilo

---

📬 La idea en una frase

> Un **hilo** es la unidad más pequeña de ejecución: una tarea que vive *dentro* de un proceso y comparte su memoria con los demás hilos de ese proceso.

En la UD 2 lanzaste procesos: programas completos con su propia memoria, su propio PID y su propio estado. Ahora entramos a un nivel más fino: un solo proceso puede tener **varios hilos ejecutándose a la vez**, todos trabajando con la misma memoria. Es como pasar de abrir varias casas (procesos) a repartir habitaciones dentro de una sola (hilos).

🧵 ¿Qué es un hilo?

Un **hilo** (thread) es la unidad más pequeña de ejecución que el sistema operativo puede gestionar. Un proceso puede tener múltiples hilos, todos compartiendo la misma memoria.

```
import threading

def saludar():
    print("¡Hola desde un hilo!")

hilo = threading.Thread(target=saludar)
hilo.start()
hilo.join()
```

Fíjate en las tres líneas mágicas que ya usarás toda la unidad:

1. `threading.Thread(target=saludar)` → **crea** el hilo (todavía no hace nada).
2. `hilo.start()` → **lo lanza**: el hilo empieza a ejecutar la función `saludar`.
3. `hilo.join()` → **espera** a que el hilo termine antes de seguir con el programa.

Características de un hilo

- **Comparten memoria** con otros hilos del mismo proceso (por eso se comunican tan rápido).
- Son **más ligeros que los procesos**: cuestan muchos menos recursos al crearlos.
- Se comunican mediante **variables compartidas** (con cuidado: eso es el TEMA 03).
- En Python, están **limitados por el GIL** para código CPU-bound (lo verás en el punto 6).

🥊 Hilos vs Procesos

Característica	Proceso	Hilo
Memoria	Aislada (cada uno la suya)	Compartida (todos en la misma)
Creación	Lenta (el SO debe copiar recursos)	Rápida
Comunicación	Pipes, sockets, archivos	Variables globales
Aislamiento	Alto (uno no afecta a otro)	Bajo (uno puede romper a todos)
Coste	Alto	Bajo

> "Los procesos son como casas separadas. Los hilos son como habitaciones de la misma casa."

🍳 La analogía de los chefs en una cocina

Imagina una cocina de restaurante con varios cocineros:

- **Procesos** = cocinas de restaurantes diferentes. Cada una tiene su propia cocina, sus propios fogones y sus propios ingredientes. Si un restaurante se quema, el de al lado ni se entera (aislamiento alto). Pero abrir un restaurante nuevo es caro y lento.
- **Hilos** = cocineros dentro de la **misma** cocina. Todos comparten la encimera, la nevera y los ingredientes (memoria compartida). Contratar a un cocinero más es barato y rápido. Pero si uno tira el aceite caliente, todos lo sufren (aislamiento bajo).

> "Un hilo es como una tarea dentro de una casa. Todos los hilos comparten la misma casa (memoria), pero cada uno hace su propia cosa."

Ese reparto de la misma nevera es lo que hace a los hilos tan rápidos para comunicarse… y tan peligrosos si dos tocan el mismo ingrediente a la vez. Ese peligro se llama **condición de carrera** y lo atacarás en el TEMA 03 · Sincronización.

🧠 Mini-chequeo

1. ¿Dónde vive un hilo: dentro de un proceso o junto a él?
2. ¿Por qué un hilo es más barato de crear que un proceso?
3. ¿Cuál es el precio de compartir memoria entre hilos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un **hilo** es la unidad más pequeña de ejecución y vive dentro de un proceso compartiendo su memoria.
- Frente a los procesos, los hilos son más ligeros, se crean más rápido y se comunican con variables compartidas, pero con menos aislamiento.
- La memoria compartida es a la vez su gran ventaja y su gran riesgo: a eso le pondremos remedio en el TEMA 03.

🐛 Vocabulario rápido

Término	Idea general
Hilo (thread)	Unidad más pequeña de ejecución dentro de un proceso
Memoria compartida	La memoria del proceso, usada por todos sus hilos
threading.Thread	Clase de Python para crear un hilo
start()	Lanza el hilo: empieza a ejecutar su función
join()	Espera a que el hilo termine
GIL	Candado de CPython que limita los hilos para código CPU-bound

---

📚 Volver al índice de la unidad · **Siguiente:** 02 · Tu primer hilo

02 — Tu primer hilo

Crea, lanza y espera tu primer hilo con start() y join() 🚀

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 02 · Tu primer hilo

📬 La idea en una frase

> Crear un hilo es escribir `threading.Thread(target=funcion)`, lanzarlo con `.start()` y esperarlo con `.join()`.

Tres líneas. Eso es todo lo que necesitas para que una función se ejecute en paralelo con el resto del programa. La gracia (y la complicación) está en *cuándo* se ejecuta cada pieza.

📝 Crear y lanzar un hilo

```
import threading

def saludar():
    print("¡Hola desde un hilo!")

hilo = threading.Thread(target=saludar)
hilo.start()
hilo.join()
```

Salida:

```
¡Hola desde un hilo!
```

Desglose de las tres líneas clave:

Línea	Qué hace
`threading.Thread(target=saludar)`	**Crea** el hilo. En este momento el hilo existe pero no hace nada (estado NUEVO, lo verás en el punto 7).

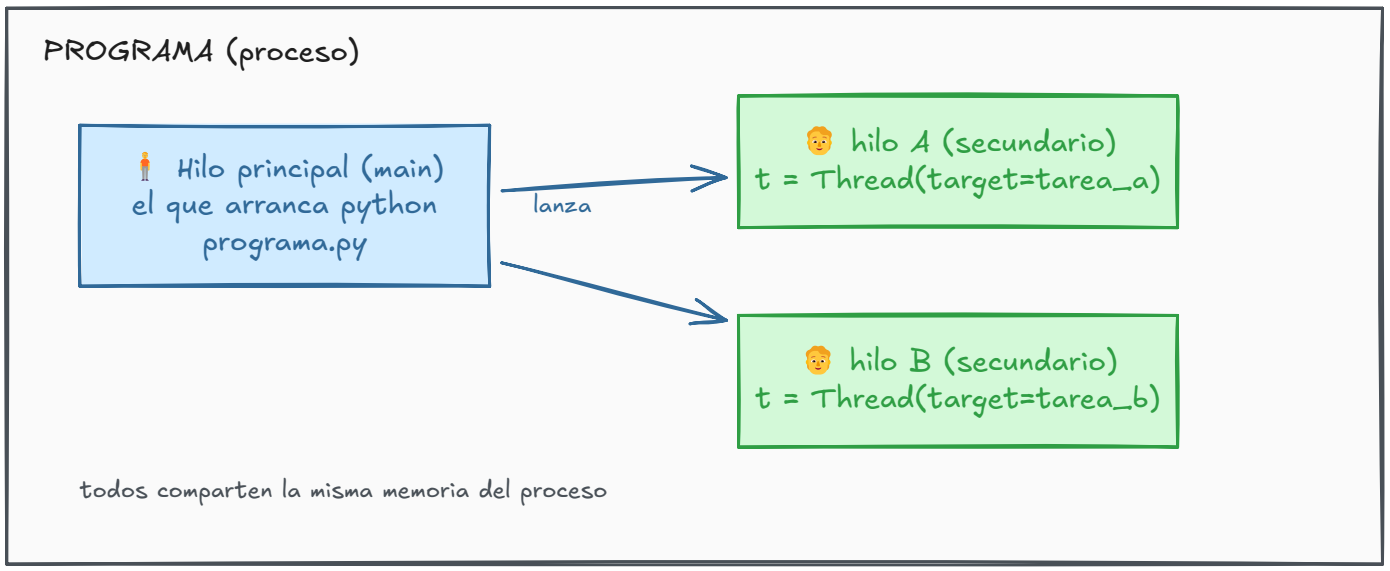
Línea	Qué hace
`hilo.start()`	**Lanza** el hilo: pasa a ejecutable y el sistema operativo decide cuándo ejecutar `saludar()`.
`hilo.join()`	**Espera** a que el hilo termine. Sin ella, el programa principal seguiría su camino y podría terminar antes que el hilo.

> ⚠️ Un error típico de principiante: llamar a `saludar()` directamente (con paréntesis) en vez de pasar la función sin ellos. `target=saludar` pasa la *referencia*; `target=saludar()` ejecuta la función antes de crear el hilo y no lanza nada.

👑 El hilo principal vs los hilos secundarios

Cuando ejecutas `python programa.py`, tu código se ejecuta dentro de un hilo: el **hilo principal** (main thread). Todo lo que lanzas con `Thread()` son **hilos secundarios** que viven dentro del mismo proceso.

Dentro de un proceso: el hilo principal y sus hijos



El hilo principal **no espera** a los secundarios por arte de magia: hay que decírselo con `join()`. Sin `join()`, el principal puede llegar al final de su código y su último `print` puede salir **antes** que el de los secundarios. Ojo: el intérprete **sí espera** a que terminen todos los hilos **no-daemon** antes de salir del programa; lo que pierdes sin `join()` es el **orden** de la salida, no la espera.

⏳ join() — esperar a que termine

`join()` hace que el programa principal espere hasta que el hilo termine.

```
import threading, time

def trabajador(segundos):
    print(f"Trabajando durante {segundos}s...")
    time.sleep(segundos)
    print("Terminado")

h = threading.Thread(target=trabajador, args=(3,))
h.start()

print("Esperando al hilo...")
h.join()  # Espera hasta que termine
print("El hilo terminó, continuamos")
```

Salida:

```
Trabajando durante 3s...
Esperando al hilo...
Terminado
El hilo terminó, continuamos
```

Fíjate en el orden: el `print("Esperando al hilo...")` del principal aparece *mientras* el hilo sigue dormido. El principal llega a `join()`, se queda bloqueado esperando, y recién cuando el hilo termina sigue con su última línea.

> Sin `join()`, el programa principal seguiría y posiblemente terminaría antes que el hilo. Con `join()`, tienes la garantía de que el hilo ha acabado cuando tú continúas.

🧰 Propiedades de un hilo

Un hilo no es solo una función en ejecución: es un objeto con información útil.

```python
import threading

def fn():
    print(f"Ejecutando {threading.current_thread().name}")

hilo = threading.Thread(target=fn, name="hilo-1")
hilo.start()
hilo.join()

print(hilo.name)        # "hilo-1"
print(hilo.ident)       # ID numérico del hilo
print(hilo.daemon)      # True/False (lo verás en el punto 4)
print(hilo.is_alive())  # True si sigue ejecutándose
```

Propiedad	Qué devuelve
`.name`	El nombre del hilo (por defecto algo como “Thread-1”)
`.ident`	Un ID numérico único mientras el hilo vive
`.daemon`	`True` si es un hilo daemon, `False` si no
`.is_alive()`	`True` si el hilo aún se está ejecutando

> 💡 Desde dentro del hilo puedes obtener tu propia información con `threading.current_thread()`, que devuelve el objeto Thread actual.

🧠 Mini-chequeo

1. ¿Qué pasa si creas un hilo y no llamas a `start()`?
2. ¿Qué pasa si llamas a `start()` dos veces sobre el mismo hilo?
3. ¿Para qué sirve `join()`? ¿Qué riesgo evitas?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- Crear y lanzar un hilo es `Thread(target=fn)`, `.start()` y `.join()`: crear, lanzar, esperar.

- El **hilo principal** no espera a los secundarios por defecto; `join()` fuerza esa espera.
- Cada hilo es un objeto con `.name`, `.ident`, `.daemon` e `.is_alive()` que te cuenta su estado.

🐛 Vocabulario rápido

Término	Idea general
Hilo principal	El hilo que ejecuta `python programa.py`, el "main"
Hilo secundario	Un hilo lanzado desde otro con `Thread()`
start()	Lanza el hilo: de nuevo a ejecutable
join()	Bloquea el programa principal hasta que el hilo termina
is_alive()	Dice si el hilo todavía se está ejecutando
current_thread()	Devuelve el objeto Thread desde dentro del propio hilo

📚 Volver al índice de la unidad · **Anterior:** 01 · De proceso a hilo · **Siguiente:** 03 · Hilos con argumentos

03 — Hilos con argumentos

Pasa datos al hilo con args y kwargs, y ponle nombre a cada hilo 🖨️

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 03 · Hilos con argumentos

📬 La idea en una frase

> Para que un hilo trabaje con datos propios le pasas argumentos en `args=` (tupla) o `kwargs=` (diccionario), y para distinguirlo le pones nombre con `.name`.

Sin argumentos, todos los hilos harían exactamente lo mismo. Con `args` y `kwargs`, cada hilo puede hacer su versión del trabajo. Y con `.name`, puedes saber en cada momento *quién* está haciendo qué.

🎒 Pasar argumentos con args

La función que ejecuta el hilo puede recibir argumentos: se los pasas en `args=` como una **tupla**, en el mismo orden que espera la función.

```
import threading

def trabajar(nombre, tarea):
    print(f"{nombre} empezando: {tarea}")
    # ... trabajo ...
    print(f"{nombre} terminó: {tarea}")

# Crear hilos con nombre y argumentos
hilo_a = threading.Thread(target=trabajar, args=("Ana", "lavar platos"))
hilo_b = threading.Thread(target=trabajar, args=("Bob", "fregar suelo"))

# Lanzarlos
hilo_a.start()
hilo_b.start()

# Esperar
hilo_a.join()
hilo_b.join()
```

Salida (orden no garantizado):

```
Ana empezando: lavar platos
Bob empezando: fregar suelo
Ana terminó: lavar platos
Bob terminó: fregar suelo
```

> ⚠️ **Truco de la coma:** una tupla de un solo elemento necesita coma final: `args=("Ana",)`. Sin la coma, `args="Ana"` no es una tupla: Python **itera el string** y pasa un argumento por carácter (`'A'`, `'n'`, `'a'`), así que `trabajar` recibiría 3 argumentos en lugar de 1 → `TypeError`.

🎁 kwargs: argumentos con nombre

Si tu función usa parámetros con nombre, puedes pasarlos con `kwargs=` en forma de diccionario.

```python
import threading

def preparar(plato, minutos):
    print(f"Preparando {plato}: tarda {minutos} min")

hilo = threading.Thread(
    target=preparar,
    kwargs={"plato": "paella", "minutos": 40}
)
hilo.start()
hilo.join()
```

Salida:

```
Preparando paella: tarda 40 min
```

Puedes mezclar ambos: `args=("Ana",)` y `kwargs={"tarea": "limpiar"}` si la función recibe `(nombre, tarea=...)`. Lo importante es que el orden de `args` coincide con los parámetros posicionales, y `kwargs` con los de nombre.

🪪 Nombrar hilos: `.name = "hilo-" + str(n)`

Cuando lanzas varios hilos iguales, ¿cómo sabes cuál es cuál? Poniéndoles nombre. La forma clásica es generarlos con un nombre único dentro de un bucle:

```
import threading

def trabajar(n):
    print(f"🌱 {threading.current_thread().name}: trabajando ({n})")
    # ... trabajo ...
    print(f"🏁 {threading.current_thread().name}: terminado ({n})")

hilos = []
for i in range(1, 4):
    h = threading.Thread(target=trabajar, args=(i,))
    h.name = "hilo-" + str(i)   # 📛 nombre único
    hilos.append(h)

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Salida (orden no garantizado):

```
🌱 hilo-1: trabajando (1)
🌱 hilo-2: trabajando (2)
🌱 hilo-3: trabajando (3)
🏁 hilo-1: terminado (1)
🏁 hilo-3: terminado (3)
🏁 hilo-2: terminado (2)
```

Fíjate en los detalles:

- `h.name = "hilo-" + str(i)` asigna un **nombre único** a cada hilo antes de lanzarlo.
- Dentro de la función, `threading.current_thread().name` devuelve el nombre del hilo que está ejecutando.
- Los `print` **se entremezclan**: el orden exacto lo decide el scheduler del sistema operativo, no nosotros.

> 💡 También puedes poner el nombre directamente al crear: `threading.Thread(target=trabajar, args=(i,), name="hilo-" + str(i))`. El resultado es idéntico.

👫 Varios hilos a la vez

Con una **lista por comprensión** puedes crear y lanzar N hilos en tres líneas. Es el patrón que usarás toda la unidad (y en los servidores concurrentes de la UD 6):

```
import threading

def tarea(n):
    print(f"{threading.current_thread().name} → tarea {n}")

hilos = [threading.Thread(target=tarea, args=(i,), name="hilo-" + str(i))
         for i in range(1, 5)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()

print("Todas las tareas terminadas")
```

Salida (orden no garantizado):

```
hilo-1 → tarea 1
hilo-3 → tarea 3
hilo-2 → tarea 2
hilo-4 → tarea 4
Todas las tareas terminadas
```

El mensaje final **siempre aparece el último** gracias a los dos bucles: primero lanzas todos, luego esperas a todos con `join()`.

🧠 Mini-chequeo

1. ¿Qué le pasas a `args=`? ¿Y a `kwargs=`?
2. ¿Por qué hace falta la coma en `args=("Ana",)`?
3. ¿Cómo sabes, dentro de la función del hilo, el nombre del hilo que te está ejecutando?

> 🔁 Respuestas

✅ Resumen en 3 frases

- Los argumentos del hilo se pasan con `args=` (tupla) o `kwargs=` (diccionario), en el orden o con el nombre que espera la función.
- Con `.name = "hilo-" + str(n)` cada hilo lleva un nombre único que puedes leer con `threading.current_thread().name`.
- El patrón lista por comprensión `+ for h in hilos: h.start() + for h in hilos: h.join()` lanza N hilos a la vez y espera a todos.

🐛 Vocabulario rápido

Término	Idea general
args	Tupla de argumentos posicionales que recibe la función del hilo
kwargs	Diccionario de argumentos con nombre que recibe la función
.name	Nombre del hilo, único para distinguirlo
current_thread()	El objeto Thread del hilo que se está ejecutando
Lista por comprensión	Forma compacta de crear N hilos en una línea

📚 Volver al índice de la unidad · **Anterior:** 02 · Tu primer hilo · **Siguiente:** 04 · Hilos daemon

04 — Hilos daemon

Hilos de fondo que se sacrifican para que el programa pueda salir 😈

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 04 · Hilos daemon

---

📬 La idea en una frase

> Un hilo **daemon** se ejecuta en segundo plano y **se mata automáticamente** cuando el programa principal termina. Los no-daemon, en cambio, impiden que el programa salga hasta terminar.

Imagina una alarma de incendios: la quieres encendida mientras el edificio está vivo, pero si el edificio desaparece, la alarma no tiene sentido y muere con él. Esa es la filosofía del hilo daemon.

😈 ¿Qué es un hilo daemon?

Un hilo **daemon** (diablo) es un hilo de fondo pensado para tareas auxiliares: monitorizar, limpiar, emitir un heartbeat, mostrar un reloj… La regla de oro:

- Un hilo **no daemon** (por defecto) impide que el programa termine hasta que él acabe.
- Un hilo **daemon** se corta en seco cuando el programa principal llega al final, sin esperar a nada.

```python
import threading, time

def reloj():
    """Imprime la hora cada segundo (por siempre)"""
    while True:
        print(f"⏰ {time.strftime('%H:%M:%S')}")
        time.sleep(1)

hilo_reloj = threading.Thread(target=reloj, daemon=True)
hilo_reloj.start()

time.sleep(3)  # El programa principal dura 3 segundos
print("Programa principal terminando...")
# Al terminar, el hilo daemon se mata solo
```

Salida:

```
⏰ 14:35:22
⏰ 14:35:23
⏰ 14:35:24
Programa principal terminando...
```

El reloj es un `while True`: nunca terminaría. Si no fuera daemon, el programa se quedaría colgado para siempre esperándolo. Siendo daemon, el programa principal hace su vida, y al terminar **el hilo muere con él**.

Tipo	Comportamiento
`daemon=False` (defecto)	El programa espera a que termine
`daemon=True`	El programa lo mata al salir

> Los hilos **no daemon** impiden que el programa termine. Los daemon se sacrifican para que el programa pueda salir.

🪂 Be the code, my friend — daemon en acción

Vamos a ser el código. El mismo trabajador de 5 pasos, primero sin daemon y luego con él.

SIN daemon — el programa espera:

```
import threading, time

def trabajador():
    for i in range(5):
        print(f"Trabajando... ({i+1}/5)")
        time.sleep(0.5)
    print("Trabajador terminó")

# SIN daemon — el programa espera
h1 = threading.Thread(target=trabajador)
h1.start()
h1.join()
print("Programa terminó (después del hilo)")
```

Salida:

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Trabajando... (4/5)
Trabajando... (5/5)
Trabajador terminó
Programa terminó (después del hilo)
```

El hilo hace sus 5 pasos completos y **recién después** el programa principal termina.

CON daemon — el hilo se mata al salir:

```
# CON daemon — el hilo se mata al salir
h2 = threading.Thread(target=trabajador, daemon=True)
h2.start()
time.sleep(1.2)
print("Programa terminó (el hilo daemon muere conmigo)")
```

Salida:

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Programa terminó (el hilo daemon muere conmigo)
```

> El daemon solo llegó a la iteración 3. El programa principal terminó y lo mató. Fíjate: no hay `join()` aquí, porque no queremos esperarlo; justo lo contrario.

🥊 El ring de los conceptos — Hilo normal vs Hilo daemon

Hilo Normal: — Yo soy un hilo de verdad. El programa principal espera a que termine lo que tengo que hacer. Tengo responsabilidad.

Hilo Daemon: — ¡Qué aburrido! Yo soy libre. Mi única misión es servir en segundo plano. Cuando el programa principal termina, yo me muero con él, sin dramas.

Hilo Normal: — ¿Y si estás en medio de algo importante cuando el main termina? Pierdes datos, dejas cosas a medias…

Hilo Daemon: — Para eso existen los daemon bien hechos: tareas de monitorización, limpieza, heartbeat… cosas que da igual si se cortan. Si quieres garantía de finalización, usas un hilo normal con `join()`.

Hilo Normal: — Cierto. Al final, cada uno tiene su sitio. Yo para tareas críticas, tú para servicios auxiliares.

> **Moraleja**: Usa hilos **daemon** para servicios de fondo prescindibles. Usa hilos **normales** con `join()` para tareas que deben completarse sí o sí.

⏳ join() para esperar a un daemon (si acaso lo necesitas)

Un daemon se mata solo al final… pero también puedes esperarlo explícitamente con `join()` si quieres que termine *antes* de que el programa siga. El `join()` funciona igual: bloquea hasta que el hilo termina. Con un daemon infinito (`while True`) eso significa esperar para siempre, así que solo tiene sentido con daemons finitos o con `join(timeout)`:

```python
import threading, time

def aviso():
    time.sleep(2)
    print("Aviso listo")

h = threading.Thread(target=aviso, daemon=True)
h.start()
h.join(timeout=3)   # espera como mucho 3 segundos
print("Continuamos...")
```

`join(timeout=N)` espera un máximo de N segundos: si el hilo no ha terminado, el programa sigue igual. Es la herramienta perfecta para no quedarse bloqueado.

🧠 Mini-chequeo

1. ¿Un hilo daemon impide que el programa termine?
2. ¿Cuándo usarías `daemon=True` en lugar de un hilo normal?
3. ¿Qué hace `join(timeout=2)` si el hilo tarda 5 segundos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un hilo **daemon** se ejecuta en segundo plano y se **mata al terminar** el programa principal.
- Un hilo **normal** impide que el programa salga hasta completarse; por eso las tareas críticas van con `join()`.
- Regla: daemon para servicios auxiliares prescindibles, normal para lo que debe terminarse sí o sí.

🐛 Vocabulario rápido

Término	Idea general
Hilo daemon	Hilo de fondo que muere cuando termina el programa principal

Término	Idea general
daemon=True	Flag que convierte un hilo en daemon
Hilo normal	Hilo que el programa principal espera antes de salir
join(timeout)	Espera al hilo un máximo de segundos y sigue
Heartbeat	Señal periódica de “sigo vivo”, típica de hilos daemon

📚 Volver al índice de la unidad · **Anterior:** 03 · Hilos con argumentos · **Siguiente:** 05 · Timer

05 — Timer

Ejecuta una función una sola vez después de un retardo ⏰

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 05 · Timer

---

📬 La idea en una frase

> `threading.Timer(retardo, funcion)` ejecuta `funcion` **una sola vez** después de `retardo` segundos, sin bloquear el resto del programa.

El `Timer` es un hilo especial que en vez de empezar a trabajar ya, se queda "dormido" el tiempo que le digas y entonces dispara la función. Es la manera más sencilla de programar un aviso diferido.

⏰ El ejemplo mínimo

```
import threading

def aviso():
    print("⏰ ¡Tiempo cumplido!")

temporizador = threading.Timer(5.0, aviso)
temporizador.start()
print("Timer iniciado, 5 segundos...")

# temporizador.cancel()  # Si queremos cancelar
```

Salida:

```
Timer iniciado, 5 segundos...
⏰ ¡Tiempo cumplido!
```

El `print("Timer iniciado...")` sale **inmediatamente**: el programa no se queda bloqueado esperando. Cinco segundos después, el Timer dispara `aviso()` por su cuenta, como un hilo más.

> `Timer` ejecuta la función **una sola vez** después del retardo. No se repite. Si lo que quieres es repetirlo cada N segundos, la forma clásica es que la propia función se reprograme (lo

> retocamos en el boletín avanzado).

🧭 Cancelar un Timer

Como el Timer es un hilo, puedes cancelarlo antes de que dispare con `.cancel()`:

```
import threading, time

def aviso():
    print("⏰ ¡Tiempo cumplido!")

t = threading.Timer(5.0, aviso)
t.start()

time.sleep(2)
t.cancel()          # cancelamos antes de los 5 segundos
print("Cancelado: el aviso nunca llegará")
```

Salida:

```
Cancelado: el aviso nunca llegará
```

`cancel()` funciona si el Timer todavía no ha disparado. Una vez disparado, ya no tiene sentido: la función ya se ejecutó.

🎁 Pasar argumentos al Timer

El Timer admite los mismos `args`/`kwargs` que cualquier hilo:

```python
import threading

def recordatorio(mensaje, veces):
    for _ in range(veces):
        print(f"📌 {mensaje}")

t = threading.Timer(3.0, recordatorio, args=("¡Beber agua!", 3))
t.start()
print("Recordatorio programado en 3 segundos...")
```

Salida:

```
Recordatorio programado en 3 segundos...
📌 ¡Beber agua!
📌 ¡Beber agua!
📌 ¡Beber agua!
```

💡 Usos típicos del Timer

Uso	Ejemplo
Despertador / aviso	"¡Despierta!" a los 3 segundos
Recordatorio	Un mensaje que aparece al pasar un rato
Timeout de una operación	Cancelar algo que tarda demasiado
Cierre de sesión	Desconectar a un usuario tras N segundos de inactividad
Limpieza diferida	Borrar un archivo temporal después de un tiempo

Un detalle a recordar: el `Timer` dispara desde un **hilo aparte** que **hereda el valor de `daemon` del hilo que lo crea** (en el hilo principal, `daemon=False`). Eso significa que, por defecto, el intérprete **espera** al Timer antes de salir: si no quieres que el programa se alargue esperando el aviso, hazlo daemon a mano, y siempre **antes** de `start()`:

```python
import threading

t = threading.Timer(5, lambda: print("¡Despierta!"))
t.daemon = True   # daemon: si el principal termina, el aviso se cancela
t.start()
```

🧠 Mini-chequeo

1. ¿Cuántas veces ejecuta el Timer la función?
2. ¿El programa principal se bloquea mientras espera el retardo?
3. ¿Qué hace `t.cancel()`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `threading.Timer(retardo, funcion)` ejecuta la función **una sola vez** después del retardo, sin bloquear el programa.
- Puedes pasar argumentos con `args=`/`kwargs=` y cancelar el disparo con `.cancel()`.
- Es ideal para avisos, recordatorios y timeouts; si quieres repetición, la función debe reprogramarse a sí misma.

🐛 Vocabulario rápido

Término	Idea general
threading.Timer	Hilo que dispara una función tras un retardo
Retardo	Segundos que espera el Timer antes de ejecutar
cancel()	Cancela el disparo si aún no ha ocurrido
args / kwargs	Argumentos que recibe la función del Timer
Una sola vez	Comportamiento del Timer: no se repite por defecto

06 — El GIL

El candado de CPython que limita a los hilos para código CPU-bound 🔒

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 06 · El GIL

📬 La idea en una frase

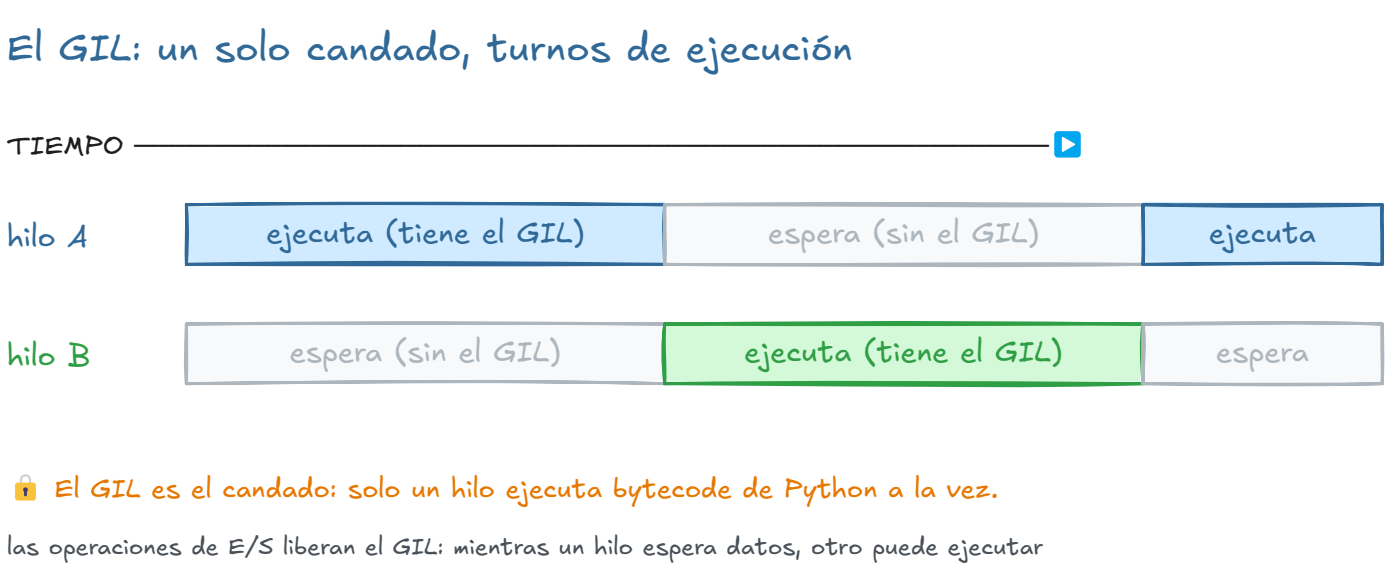
> **GIL** = Global Interpreter Lock: un candado interno de CPython que evita que dos hilos ejecuten bytecode Python a la vez, así que los hilos no aceleran el código de CPU… pero sí el de espera.

Es la gran trampa de los hilos en Python: "¿para qué sirven si no aceleran nada?" La respuesta corta: sí aceleran lo que vale la pena (esperar por red y disco) y no aceleran lo que no (calcular). Vamos a verlo con números.

🔒 ¿Qué es el GIL?

GIL = **G**lobal **I**nterpreter **L**ock. Es un candado que CPython (el Python de toda la vida) mantiene para que **solo un hilo ejecute bytecode Python en cada instante**.

¿Por qué existe? Para proteger la memoria interna del intérprete: sin el candado, dos hilos podrían corromper las estructuras de Python al tocarlas a la vez. Python paga el precio de la seguridad con un límite: **no hay paralelismo real de CPU** entre hilos de un mismo proceso.



Los hilos se turnan el candado en pequeños fragmentos: parecen simultáneos (por eso los `print` se entremezclan), pero nunca ejecutan Python a la vez.

⚡ CPU-bound: los hilos NO sirven de nada

Una tarea **CPU-bound** es la que usa la CPU a tope: sumar millones de números, procesar imágenes, comprimir. Ahí el GIL no deja ejecutar a más de un hilo, así que 4 hilos tardan lo mismo que 1.

```
import threading, time

# ⚡ CPU-bound: los hilos NO sirven de nada
def contar():
    total = 0
    for i in range(50_000_000):
        total += i

inicio = time.time()
hilos = [threading.Thread(target=contar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos CPU: {time.time() - inicio:.2f}s")
# → varios segundos (mismo tiempo que con 1 hilo)
```

Cuatro hilos contando 50 millones cada uno… y el tiempo es prácticamente el mismo que con uno solo. Peor aún: a veces es **ligeramente más lento** por el coste de turnarse el candado.

🌐 I/O-bound: los hilos SÍ aceleran

Una tarea **I/O-bound** espera por algo externo: descargar de una red, leer de disco, esperar una respuesta. Ahí el hilo **libera el GIL mientras espera**, y los demás pueden trabajar.

```python
# 🌐 I/O-bound: los hilos SÍ aceleran
def esperar():
    time.sleep(2)  # Simula descarga/lectura

inicio = time.time()
hilos = [threading.Thread(target=esperar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos I/O: {time.time() - inicio:.2f}s")
# → ~2 segundos (4 descargas en paralelo)
```

En serio: cuatro hilos esperando 2 segundos **cada uno** terminan en ~2 segundos, no en 8. Mientras un hilo espera la red, el siguiente aprovecha para iniciar su descarga. Todos esperan *a la vez*.

📊 La tabla que hay que saberse

Tipo de tarea	¿Hilos ayudan?	Motivo
CPU-bound (calcular, procesar)	❌ No	El GIL solo deja ejecutar a uno
I/O-bound (esperar red, disco)	✅ Sí	El GIL se libera durante la espera

> 💡 **¿Cómo distingo cada tipo?** Si tu programa gasta el tiempo **calculando** (contar, cifrar, procesar), es CPU-bound. Si gasta el tiempo **esperando** (descargar, consultar una API, leer archivos), es I/O-bound. Los hilos brillan en el segundo caso.

🔨 Cuándo sirven de verdad los hilos

Con lo visto, el mapa mental queda así:

- **Solicitudes de red** → hilos sí. Descargar 10 archivos con 10 hilos es ~10 veces más rápido.
- **Lecturas/escrituras de archivos** → hilos sí. Varias operaciones de disco en paralelo.
- **Servidores que atienden clientes** → hilos sí. Cada cliente espera su turno; mientras espera, otros avanzan (lo verás en la UD 6).
- **Cálculo puro** → hilos no. Para eso, **`multiprocessing`**.

> Para CPU-bound en Python, usa `multiprocessing` (varios procesos, cada uno con su propio GIL). Esos procesos sí ejecutan en paralelo de verdad, a cambio del coste de crear procesos que viste en el punto 1.

🧠 Mini-chequeo

1. ¿Qué significa exactamente "CPython solo ejecuta un hilo a la vez"?
2. Descargar 5 archivos de 2 segundos cada uno: ¿1 hilo o 5 hilos? ¿Cuánto tarda cada opción?
3. Sumar 50 millones de números con 4 hilos: ¿más rápido, igual o más lento que con 1?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El **GIL** es un candado de CPython que impide que dos hilos ejecuten bytecode a la vez.
- Para código **CPU-bound**, los hilos no sirven; para código **I/O-bound**, aceleran muchísimo porque el GIL se libera durante la espera.
- Cuando necesitas paralelismo real de CPU, toca usar `multiprocessing`, no hilos.

🐛 Vocabulario rápido

Término	Idea general
GIL	Global Interpreter Lock: candado de CPython
CPU-bound	Tarea que gasta el tiempo calculando (no le sirven los hilos)
I/O-bound	Tarea que gasta el tiempo esperando (le sirven los hilos)
Bytecode	El código intermedio que ejecuta CPython, protegido por el GIL
multiprocessing	Módulo para paralelismo real con procesos (cada uno con su GIL)

📚 Volver al índice de la unidad · **Anterior:** 05 · Timer · **Siguiente:** 07 · Estados del hilo

07 — Estados del hilo

El ciclo de vida de un hilo, de nuevo a terminado 🔄

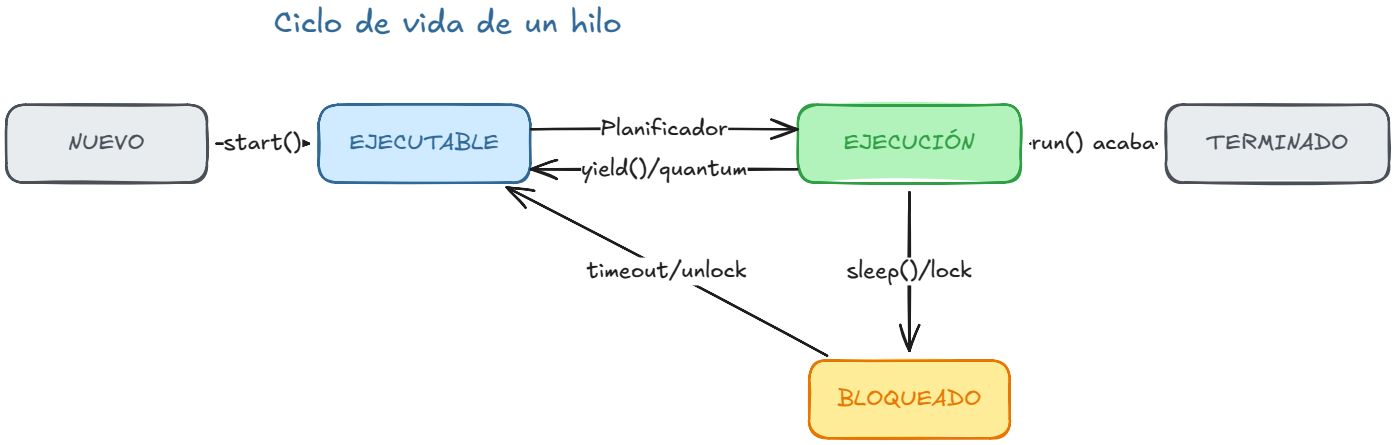
> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 07 · Estados del hilo

📬 La idea en una frase

> Un hilo nace como objeto, se lanza con `start()`, espera su turno, se ejecuta, se bloquea cuando espera algo y termina cuando su función acaba: ese es su ciclo de vida.

Cada hilo pasa por estados, igual que una persona pasa por situaciones a lo largo del día. Saber en qué estado está un hilo (o cuándo lo estará) es clave para entender por qué un programa se comporta como se comporta.

🔄 El diagrama del ciclo de vida



🗺️ Los estados, uno a uno

Estado	Significado
NUEVO	El objeto Thread existe pero no se ha llamado a `start()`
EJECUTABLE	`start()` llamado. Puede ejecutar en cualquier momento
EJECUCIÓN	El scheduler le ha dado la CPU: está ejecutando su código ahora mismo

Estado	Significado
BLOQUEADO	Esperando (sleep, I/O, un lock)
TERMINADO	El método `run()` ha terminado

Recorrido completo de un hilo típico:

1. **NUEVO** — `threading.Thread(target=fn)` crea el objeto. Todavía no se ejecuta nada.
2. **EJECUTABLE** — `h.start()` lo lanza. A partir de aquí el scheduler del sistema operativo decide *cuándo* le toca.
3. **EJECUCIÓN** — Le toca: ejecuta su código. Puede volver a EJECUTABLE cuando el scheduler decide dar paso a otro hilo.
4. **BLOQUEADO** — El hilo se queda esperando: un `time.sleep()`, una lectura de red, una espera de I/O o (en el TEMA 03) un lock. Cuando lo que espera se libera, vuelve a EJECUTABLE.
5. **TERMINADO** — La función que ejecutaba el hilo acaba. Su `run()` terminó y no volverá a ejecutarse.

🧍 El estado BLOQUEADO, el más importante

En la práctica, el estado que más vemos es **BLOQUEADO**, y casi siempre por dos motivos:

- **`time.sleep(n)`** — el hilo pide "no me des CPU durante n segundos" (lo usaste en los puntos 2 y 4).
- **Espera de I/O** — descarga, lectura de archivo, esperar un socket (del TEMA 04 para allá).

Mientras un hilo está BLOQUEADO, **libera la CPU** (y, como viste en el punto 6, también el GIL): otro hilo puede ejecutar. Es exactamente el mecanismo que hace rápidas las tareas I/O-bound.

> 💡 `sleep(0)` es un caso curioso: cede la CPU **voluntariamente** sin esperar nada, solo para dar paso a otro hilo. Es una "buena práctica" en hilos cooperativos.

🕐 ¿Cuándo termina un hilo?

Un hilo llega a **TERMINADO** cuando su función acaba (o si lanza una excepción no capturada). Y aquí entra el matiz del punto 4:

- Un hilo **no daemon** que llega a TERMINADO es requisito para que el programa principal pueda salir.
- Un hilo **daemon** puede ser cortado en seco por el final del programa principal, aunque esté en EJECUCIÓN o BLOQUEADO: muere sin llegar "bien" a TERMINADO.

Puedes comprobar si un hilo sigue vivo en cada momento con `hilo.is_alive()`:

```
import threading, time

def corto():
    time.sleep(1)

h = threading.Thread(target=corto)
print(f"Antes de start: {h.is_alive()}")   # False (estado NUEVO)
h.start()
print(f"Justo tras start: {h.is_alive()}") # True (EJECUTABLE/EJECUCIÓN)
h.join()
print(f"Tras join: {h.is_alive()}")        # False (TERMINADO)
```

Salida:

```
Antes de start: False
Justo tras start: True
Tras join: False
```

🧠 Mini-chequeo

1. ¿En qué estado está un hilo justo después de crearlo pero antes de `start()`?
2. ¿Qué ocurre cuando un hilo en EJECUCIÓN llama a `time.sleep(2)`?
3. ¿Puede un hilo volver de BLOQUEADO a EJECUCIÓN sin pasar por EJECUTABLE?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- El ciclo de vida de un hilo es **NUEVO → EJECUTABLE → EJECUCIÓN ⇄ BLOQUEADO → TERMINADO**.

- **BLOQUEADO** es el estado de las esperas (sleep, I/O, lock): al bloquearse, el hilo libera la CPU y el GIL.
- `is_alive()` te dice en vivo si el hilo todavía se está ejecutando o ya terminó.

🐛 Vocabulario rápido

Término	Idea general
NUEVO	Hilo creado, sin `start()` todavía
EJECUTABLE	`start()` llamado, esperando turno del scheduler
EJECUCIÓN	El hilo tiene la CPU y está ejecutando código
BLOQUEADO	Esperando: sleep, I/O o un lock
TERMINADO	La función del hilo ha acabado
Scheduler	Componente del SO que decide qué hilo ejecuta

📚 Volver al índice de la unidad · **Anterior:** 06 · El GIL · **Siguiente:** 08 · Hilos en la práctica

08 — Hilos en la práctica

Be the code, el ring Hilo vs Proceso y Aprieta el lápiz 🖍️

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 08 · Hilos en la práctica

📬 La idea en una frase

> Este punto es el taller de la unidad: seguimos la ejecución de varios hilos paso a paso, dirimimos la pelea Hilo vs Proceso y resolvemos los ejercicios de Aprieta el lápiz.

Ya sabes crear hilos, pasarles datos, convertirlos en daemons, temporizarlos y entender el GIL y los estados. Ahora toca juntarlo todo como lo harás en el examen: leyendo código multihilo y prediciendo su salida.

🧍 Be the code, my friend — El hilo viajero

> "Sé el código y recorre su ejecución paso a paso. Dos hilos, dos viajeros."

```
import threading, time

def viajero(nombre, paradas):
    for i in range(paradas):
        print(f"{nombre} 🚶 está en la parada {i+1}")
        time.sleep(0.5)
    print(f"{nombre} 🏁 ha llegado a su destino")

h1 = threading.Thread(target=viajero, args=("Ana", 3))
h2 = threading.Thread(target=viajero, args=("Bob", 2))

h1.start()
h2.start()
h1.join()
h2.join()
```

Traza (ejecución real):

```
▶️ Ana (h1) empieza. Dice: "Ana 🚶 está en la parada 1"
▶️ Bob (h2) empieza. Dice: "Bob 🚶 está en la parada 1"
⏳ Ana duerme 0.5s
⏳ Bob duerme 0.5s
▶️ Bob se despierta. Dice: "Bob 🚶 está en la parada 2"
▶️ Ana se despierta. Dice: "Ana 🚶 está en la parada 2"
⏳ Ana duerme 0.5s
⏳ Bob duerme 0.5s
▶️ Bob se despierta. Dice: "Bob 🏁 ha llegado a su destino"
▶️ Ana se despierta. Dice: "Ana 🚶 está en la parada 3"
⏳ Ana duerme 0.5s
▶️ Ana se despierta. Dice: "Ana 🏁 ha llegado a su destino"
```

Fíjate en los detalles que te piden en los exámenes:

- Ambos hilos **arrancan a la vez**: Ana y Bob dicen su parada 1 casi al mismo tiempo.
- Mientras Ana duerme sus 0.5s (estado **BLOQUEADO** del punto 7), Bob aprovecha y avanza.
- Bob hace solo 2 paradas; Ana hace 3. **Bob llega antes** a su destino.
- Los `join()` del final garantizan que el mensaje del principal (si lo hubiera) esperaría a ambos.

> Fíjate cómo los `print` se entremezclan. El orden exacto lo decide el scheduler del sistema operativo. **No hay garantía de orden.**

🥊 El ring de los conceptos — Hilo vs Proceso

Hilo: — Soy ligero, veloz y nacido para compartir. Nací en milisegundos y trabajo en la misma memoria que mis hermanos.

Proceso: — ¿Y dónde está tu privacidad? Yo tengo mi propia memoria aislada. Si tú cometes un error, te llevas por delante a todo el proceso. A mí nadie me toca.

Hilo: — Pero compartir es mi superpoder: me comunico con variables globales al instante, sin pipes ni sockets. ¿Cuánto tardas tú en enviar un dato a tu vecino?

Proceso: — Pipes, sockets, archivos… más lento, sí. Pero creo cien procesos y cada uno trabaja con su propio GIL a toda CPU. Tú y tus hermanos os peleáis por un solo GIL.

Hilo: — ¡El GIL es para calcular! Yo brillo esperando: descargas, lecturas de disco, clientes de red. Mientras uno espera, los demás avanzan.

Proceso: — Al final, cada uno a su oficio. Yo para aislamiento y CPU de verdad; tú para esperas y servicios ligeros.

> **Moraleja**: Usa **procesos** cuando necesites aislamiento o paralelismo real de CPU (`multiprocessing`). Usa **hilos** cuando tu tarea es de espera (I/O) o quieres algo ligero y que comparta memoria. La tabla completa la tienes en el punto 1.

📌 Aprieta el lápiz

Resuelve estos seis ejercicios con lo aprendido en la unidad. Cuando los tengas (o te rindas), despliega las soluciones.

1. **Carrera de mensajes**: Crea 5 hilos que impriman su nombre 10 veces. Observa cómo se entremezclan.
2. **Temporizador daemon**: Crea un hilo daemon que imprima "tic" cada segundo. El programa principal espera 5 segundos y termina.
3. **Timer despertador**: Crea un Timer que imprima "¡Despierta!" a los 3 segundos.
4. **GIL test**: Compara el tiempo de 4 hilos vs 1 hilo haciendo una tarea CPU-bound (contar hasta 50M). Verifica que tardan lo mismo.
5. **I/O vs CPU**: Crea dos versiones de descarga simulada (sleep 2s) — una con 1 hilo y otra con 4. Mide la diferencia.
6. **Pool Puzzle**: Ordena las líneas para crear un programa que lance 2 hilos que saluden 3 veces cada uno.

> ✅ **Soluciones**

🧠 Mini-chequeo

1. En la traza del viajero, ¿por qué Bob llega antes que Ana a su destino?
2. ¿Qué garantizan los `join()` del final del programa de los viajeros?
3. En el Pool Puzzle, ¿por qué el `import threading` debe ir al principio del orden?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- La traza de varios hilos se **entremezcla** y su orden lo decide el scheduler: no hay garantía de orden, solo de que con `join()` se esperan.
- En el ring, **hilos** para tareas ligeras y de espera que comparten memoria; **procesos** para aislamiento y CPU de verdad.
- El Pool Puzzle y los ejercicios son exactamente el formato de examen: ordenar líneas y predecir salidas.

🐛 Vocabulario rápido

Término	Idea general
Traza	Recorrido paso a paso de la ejecución del programa
Intercalado	Alternancia de mensajes de varios hilos sin orden fijo
Scheduler	Decide qué hilo ejecuta en cada momento
Pool Puzzle	Ejercicio de ordenar líneas de código desordenadas
join()	Espera a que el hilo termine antes de continuar

📚 Volver al índice de la unidad · **Anterior:** 07 · Estados del hilo · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé el hilo, el ring final y el laboratorio de la unidad 🧠

> 🗺️ **Estás en:** 🔀 **UD 3 · Hilos y concurrencia** → 09 · Cierre

Has terminado la teoría: qué es un hilo, cómo se crea y se espera, argumentos y nombres, daemons, Timer, el GIL y el ciclo de vida. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé el hilo

> *Eres un hilo llamado "hilo-1". Acabas de nacer en el programa de los viajeros del punto 8. Tu misión: recorrer 3 paradas con 0.5s de sueño entre cada una.*

¿Qué pasa, paso a paso?

1. **Naces en estado NUEVO**: `threading.Thread(target=viajero, args=("Ana", 3))`. Existes como objeto, pero no has hecho nada todavía.
2. **`h1.start()` te lanza**: pasas a **EJECUTABLE**. Ya puedes ejecutar cuando el scheduler quiera.
3. **El scheduler te da la CPU**: pasas a **EJECUCIÓN** e imprimes `"Ana 🚶 está en la parada 1"`.
4. **`time.sleep(0.5)` te bloquea**: pasas a **BLOQUEADO**. Sueltas la CPU y el GIL; tu compañero Bob aprovecha y avanza.
5. **Te despiertas**: vuelves a **EJECUTABLE**, el scheduler te da CPU de nuevo y avanzas a la parada 2… y luego la 3.
6. **Tu función acaba**: pasas a **TERMINADO**. El `h1.join()` del programa principal confirma que ya no sigues vivo.

En ningún momento decides tú cuándo te toca. El scheduler del sistema operativo reparte la CPU entre ti y Bob como quiere; tu único control es lanzarte (`start()`) y hacer esperar al principal (`join()`).

> 💡 **Ahora tú:** ¿y si fueras el reloj daemon del punto 4? Tu función es un `while True`: nunca llegas "bien" a TERMINADO. El día que el programa principal acabe, te matan en seco, estés en EJECUCIÓN o BLOQUEADO. Ese es el destino del daemon.

🔥 Fireside Chat: Hilo vs Proceso

> *Dos trabajadores de la multitarea se sientan junto a la chimenea a zanjar, de una vez, quién hace qué.*

Hilo: — Soy la unidad más pequeña de ejecución. Nací dentro de un proceso y comparto su memoria con mis hermanos. Me crean en milisegundos.

Proceso: — Yo soy el dueño de la casa. Memoria aislada, PID propio, espacio de direcciones separado. Si un hilo se equivoca, puede tirar la casa entera.

Hilo: — Pero comunicarme es instantáneo: una variable global y listo. Tú necesitas pipes, sockets o archivos para hablar con tu vecino.

Proceso: — Es cierto, más lento. Pero cuando toca calcular de verdad, cada proceso tiene su propio GIL. Yo puedo usar 8 CPUs de golpe; tú te peleas con tus hermanos por una.

Hilo: — ¡El GIL es para calcular! Yo brillo esperando: descargas, lectura de disco, clientes de red. Mientras uno espera, los demás avanzan. Ahí no me gana nadie.

Proceso: — Al final, cada uno a su oficio. Yo para aislamiento y CPU real; tú para esperas, servicios de fondo y todo lo ligero.

> **Moraleja**: los hilos comparten memoria y son baratos (perfectos para I/O y servicios ligeros); los procesos aíslan y paralelizan CPU de verdad. Elige según la tarea.

🦹 ¿Quién soy?

1. Soy la unidad más pequeña de ejecución y comparto memoria con mis hermanos dentro de un proceso.
2. Me llaman “main” y, sin un `join()`, puedo terminar el programa antes que los demás hilos.
3. Me ejecuto en segundo plano y muero cuando el programa principal termina, sin dramas.
4. Soy un candado de CPython que impide que dos hilos ejecuten bytecode a la vez.
5. Me ejecutan una sola vez tras un retardo, y pueden cancelarme antes de que dispare.
6. Soy el estado en el que un hilo espera dormido, leyendo de red o esperando un lock.

> › 🔁 **Respuestas**

🤬 CONRAD VS EL MUNDO: “el programa termina antes que mis hilos”

CONRAD: — “Clásico: mi programa imprime ‘Fin’ y los hilos ni se han enterado. Razones: 1) **No puse `join()`**: el principal siguió a lo suyo y se fue antes de que terminaran. 2) **Llamé a la función con paréntesis** en `target=fn()`: se ejecutó al instante en el hilo principal y el hilo nació ya terminado. 3) **Hice el hilo daemon** y el principal salió: lo maté yo mismo. 4) **Esperaba que `start()` ejecutara al momento**, cuando en realidad solo lo pone en EJECUTABLE a merced del scheduler.”

CONRAD: — “Y lo mejor: *‘pero si lo he lanzado’*. ¡Pues claro! **Lanzar no es esperar.** `start()` pone el hilo en marcha; `join()` es lo único que hace que el programa principal se quede esperándolo. Sin `join()`, tu ‘Fin’ puede salir antes que todo el trabajo.”

CONRAD: — “Y no me vengas con *‘¿será que el GIL lo bloquea?’*. Si tus hilos son de espera (I/O), el GIL no es el problema: el problema es que **no los estás esperando tú**. A diagnosticar: ¿hay `join()`? ¿la función va sin paréntesis? ¿el hilo es daemon sin quererlo?”

⚡ Laboratorio de tortura: la sala de descargas

> **Duración:** 45 minutos **Herramienta:** Python 3 (`threading` y `time`, sin instalar nada)

Escenario: estás montando el mini-descargador del instituto. Cuatro archivos de 2 segundos cada uno (simulados con `time.sleep(2)`) deben descargarse “en paralelo” con 4 hilos, y el programa debe cronometrar cuánto tarda el lote entero.

Tareas paso a paso:

1. Define `descargar(archivo)`: imprime ⬇️ `descargando {archivo}`, hace `time.sleep(2)` y al final imprime ✅ `{archivo} listo`.
2. Crea 4 hilos con `threading.Thread(target=descargar, args=(nombre,))`, uno por archivo, con `.name = "hilo-1"`… `"hilo-4"`.
3. Lanza todos con `for h in hilos: h.start()`.
4. Espera a todos con `for h in hilos: h.join()`.
5. Cronometra el lote con `time.time()` antes y después: con 4 hilos debe rondar los **2 segundos**, no los 8.

Fallo intencionado: en el paso 2, en lugar de `target=descargar`, escribe `target=descargar("foto.png")` — **con paréntesis y argumento**. ¿Qué pasa? La función se ejecuta en el momento de crear el hilo, en el hilo principal y **en serie**: los 4 archivos tardan 8 segundos, las descargas aparecen "ya terminadas" antes de lanzar nada, y los hilos nacen haciendo nada.

> **Pista 1:** `target` recibe una **referencia** a la función, no una llamada. `descargar` sin paréntesis le pasa la función; `descargar("foto.png")` la ejecuta ahí mismo (lo avisamos en el punto 2). Si tus descargas imprimen antes del primer `start()`, ese es el bug.
>
> **Pista 2:** si el lote tarda ~8 segundos en vez de ~2, no hay paralelismo: los `time.sleep(2)` se están sumando porque todo se ejecuta en el hilo principal. Un `print(f"Creando {archivo}")` dentro del bucle te delata: los 4 mensajes aparecen juntos y seguidos, sin intercalarse con los ⬇️.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Primer Hilo**	Crear, lanzar y esperar un hilo con `Thread`, `start()` y `join()`
🏅 **Hilo con Nombre**	Pasar `args`/`kwargs` y nombrar N hilos con `.name = "hilo-" + str(n)`
🏅 **Daemon Tamer**	Explicar y construir un hilo daemon de fondo que muere al salir
🏅 **Timer Whisperer**	Programar un aviso con `threading.Timer` y cancelarlo si hace falta
🏅 **GIL Master**	Explicar el GIL y distinguir tareas CPU-bound de I/O-bound

---

🧠 Atrévete a pensar

1. ¿Por qué los hilos comparten memoria y los procesos no?
2. ¿Qué le pasa a tu programa si un hilo **no daemon** entra en un bucle infinito?
3. ¿Cuándo conviene usar `multiprocessing` en lugar de hilos?
4. ¿Por qué la salida de varios hilos nunca tiene orden garantizado?
5. ¿Qué diferencia hay entre un hilo en EJECUTABLE y uno en EJECUCIÓN?

> 💡 Soluciones

🧩 Crucigrama de bits

```
Horizontal:
1. Unidad más pequeña de ejecución dentro de un proceso (5 letras)
3. Método que lanza un hilo (5 letras)
5. Método que hace esperar al programa principal (4 letras)
6. Hilo de fondo que muere al salir el programa (6 letras)
8. Candado de CPython que limita a los hilos (3 letras)

Vertical:
2. Clase de Python con la que se crea un hilo (6 letras)
4. Estado del hilo que espera dormido o por I/O (9 letras)
7. Ejecuta una función una sola vez tras un retardo (5 letras)
```

> 📝 Soluciones

💬 Entrevista de trabajo

1. **“¿Qué es un hilo y en qué se diferencia de un proceso?”**
2. **“¿Cómo creas y lanzas varios hilos en Python con argumentos y nombres?”**
3. **“¿Qué es un hilo daemon? ¿Cuándo lo usarías?”**
4. **“¿Qué es el GIL y cómo afecta al rendimiento de tus hilos?”**
5. **“¿Cómo sabes si un hilo ha terminado y cómo esperas a varios hilos a la vez?”**

> 💡 **Cómo encararlas:** la 1 y la 4 son las “preguntas reina”. Para la 1, compara memoria, coste de creación, comunicación y aislamiento (la tabla del punto 1) y remata con la moraleja del ring: hilos para I/O y servicios ligeros, procesos para aislamiento y CPU. Para la 4, recorre el punto 6: qué es el GIL, por qué los hilos no aceleran CPU-bound, por qué sí I/O-bound, y cuándo toca `multiprocessing`. Si sabes contarlo fluido, ya eres medio programador concurrente.

🤷 No hay preguntas tontas

> ❓ **¿Un hilo puede crear otro hilo?**

Sí. Un hilo puede lanzar otros hilos sin problema.

> ❓ **¿Cuántos hilos puedo crear?**

Hay límite práctico. En Windows, unos pocos miles. Cada hilo consume ~1MB de memoria virtual por su pila.

> ❓ **¿Puedo matar un hilo desde fuera?**

No limpiamente. No hay `hilo.kill()`. La forma correcta es usar una variable bandera que el hilo compruebe periódicamente.

> ❓ **¿Qué pasa si no llamo a `join()`?**

El hilo se ejecuta igual. Pero el programa principal no espera. Si es no-daemon, el programa no terminará hasta que el hilo termine.

> ❓ **¿`sleep(0)` sirve para algo?**

Sí, cede la CPU voluntariamente para que otro hilo pueda ejecutar. Es una "buena práctica" en hilos cooperativos.

> ❓ **¿Los hilos tienen prioridad?**

En Python, no hay prioridades nativas. El scheduler del SO decide. Puedes simular prioridades con lógica condicional, pero no es real.

🎬 Poscréditos

> *El hilo de Ana recorre sus tres paradas. El de Bob, más ligero, llega antes a su destino.*

El reloj daemon imprime la hora en segundo plano hasta que el programa decide apagarse.

El GIL vigila que ningún hilo pise a otro mientras CPython respira.

Los hilos comparten memoria, y compartir sin ponerse de acuerdo es peligroso.

PRÓXIMAMENTE EN UD 4: *Dos hilos tocando la misma variable a la vez es una carrera. Si no se ponen de acuerdo, los datos se corrompen. Necesitan locks, semáforos y barreras: Sincronización.*

✅ Criterios de evaluación cubiertos (RA2)

RA2 — Hilos (parcial): estructura, creación, esperas, daemon y GIL.

CE	Criterio	Cubierto
RA2a	Identifica la estructura de un hilo	✅ Punto 1 + Punto 7
RA2b	Crea y lanza hilos con threading	✅ Puntos 2-3 + ⚡ Laboratorio de tortura
RA2e	Implementa esperas con join() y sleep()	✅ Puntos 2-4 + ⚡ Laboratorio de tortura
RA2f	Gestiona hilos daemon	✅ Punto 4 + ⚡ Laboratorio de tortura
RA2h	Conoce el GIL y sus limitaciones	✅ Punto 6 + Punto 8

> Los criterios RA2c (Lock), RA2d (semáforos) y RA2g (condiciones de carrera) se cubren en el **TEMA 03 — Sincronización**.

📚 Volver al índice de la unidad · **Anterior:** 08 · Hilos en la práctica · **Siguiente:** UD 4 · Sincronización

UD 4 — Sincronización en Python

Cuando los hilos se pisan: locks, semáforos y barreras 🔒

En la UD 3 lanzaste hilos por todas partes y comprobaste que los hilos de un mismo proceso **comparten memoria**. Eso es fantástico… y peligroso: si dos hilos tocan la misma variable a la vez, los resultados se descontrolan. Esta unidad pone orden en el caos: aprenderás qué es una **condición de carrera**, por qué `contador += 1` esconde una trampa de 3 pasos, y cómo los **locks**, **semáforos**, **barreras** y **condiciones** del módulo `threading` convierten el desorden en coordinación.

Verás, además, el patrón de **productor-consumidor** resuelto con `Condition`, cómo evitar los temidos **deadlocks** con las reglas del oficio, y un cierre práctico con laboratorio.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una **condición de carrera** y por qué `contador += 1` no es una operación atómica.
- Proteger secciones críticas con `threading.Lock` usando `with lock:`.
- Distinguir `Lock` de `RLock` y saber cuándo necesitas el reentrante.
- Limitar el acceso concurrente a un recurso con `Semaphore`.
- Coordinar fases de trabajo en paralelo con `Barrier`.
- Sincronizar hilos con `Condition` (`wait` / `notify` / `notify_all`).
- Implementar el patrón **productor-consumidor** con una cola compartida.
- Evitar **deadlocks** con buenas prácticas.
- Decidir qué mecanismo de sincronización usar en cada situación.
- Entender qué es el modo **free-threaded** (PEP 703) y por qué la sincronización sigue siendo obligatoria sin GIL (punto 9).

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Condición de carrera	Qué pasa cuando dos hilos se pisan la memoria compartida	Todos
02 · Lock	Exclusión mutua: `acquire`, `release` y `with lock:`	Todos
03 · RLock	El lock reentrante para cuando el mismo hilo entra dos veces	Todos
04 · Semaphore	El aforo máximo: hasta N hilos dentro a la vez	Todos
05 · Barrier	Nadie avanza hasta que llegan todos	Todos
06 · Condition	Esperar a que otro hilo avise: `wait` y `notify`	Todos
07 · Productor-Consumidor	El patrón clásico con cola compartida	Todos
08 · Buenas prácticas	Deadlocks, orden de locks	Todos
09 · Python Free-Threaded	Python sin GIL: qué cambia y qué no (PEP 703)	Ampliación
10 · Cierre	Sé el lock, Fireside, Laboratorio de tortura…	Todos

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA2)

RA2: Gestiona la programación de hilos y su sincronización.

CE	Criterio	Dónde se cubre
RA2c	Sincroniza hilos con Lock	✅ Puntos 2, 3 y 7 + ⚡ Laboratorio (punto 10)
RA2d	Usa semáforos para acceso controlado	✅ Punto 4 + ⚡ Laboratorio (punto 10)
RA2g	Evita condiciones de carrera	✅ Puntos 1, 2 y 8 + ⚡ Laboratorio (punto 10)

> RA2a, RA2b, RA2e, RA2f y RA2h se cubren en la **UD 3 · Hilos y concurrencia**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 3 y ya sabes lanzar hilos y esperarlos con `join()`? Empieza por el punto 1, que parte de la condición de carrera.

¿Ya sabes qué es un lock y solo necesitas el semáforo o la barrera? Saltar a los puntos 4, 5 y 6.

📍 **Primer punto:** 01 · Condición de carrera ⏭️ **Al acabar la unidad, continúa en** UD 5 · Sockets TCP y UDP.

01 — Condición de carrera

Cuando dos hilos se pisan la memoria compartida 🏃💥

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 01 · Condición de carrera

📬 La idea en una frase

> Una **condición de carrera** ocurre cuando dos o más hilos acceden a la misma variable compartida a la vez, y el resultado final depende de qué hilo llegue primero: los incrementos se pierden y el resultado es incorrecto.

Dos hilos que incrementan una variable compartida sin sincronización:

```
import threading, sys

contador = 0
sys.setswitchinterval(1e-6)   # ⚠️ fuerza cambios de hilo muy frecuentes

def sumar(c):
    return c + 1

def incrementar():
    global contador
    for _ in range(200_000):
        contador = sumar(contador)   # leer-sumar-escribir vía función

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Esperado: 800.000 | Obtenido: {contador}")
# → 346.564, 375.671, 340.593... ¡nunca 800.000!
```

Con `sys.setswitchinterval(1e-6)` el planificador cambia de hilo muchísimas veces por segundo: es la forma de **hacer visible** la interrupción entre el “leer” y el “escribir”. En Python 3.13+ el += en una línea suele completarse sin interrupciones y el ejemplo “clásico” casi nunca falla; por eso pasamos la suma por una función, que da al planificador puntos donde cortar.

Lanzamos 4 hilos, cada uno suma 200.000 veces. El resultado esperado es 800.000… y jamás lo vemos. Cada ejecución da un número distinto, siempre menor. Esa es la cara visible de la condición de carrera.

🔍 ¿Por qué falla? El += no es atómico

`contador += 1` parece una línea, pero por dentro hace **3 operaciones**:

```
1. Leer contador de memoria
2. Sumar 1
3. Escribir el resultado
```

Si dos hilos leen el mismo valor antes de que ninguno haya escrito, ambos escriben el mismo resultado y se pierde un incremento:

```
Hilo-A: lee contador = 0
Hilo-B: lee contador = 0          ← ¡mismo valor!
Hilo-A: escribe contador = 1
Hilo-B: escribe contador = 1      ← ¡pisó el incremento de A!
Hilo-A: lee contador = 1
Hilo-A: escribe contador = 2
Hilo-B: lee contador = 1          ← ¡leyó valor viejo!
```

Entre la “lectura” del hilo A y su “escritura”, el hilo B cuela otra lectura y otra escritura. Al final, dos incrementos solo han subido el contador en 1.

> ⚠️ En Python, el **GIL** evita que dos hilos ejecuten Python puro a la vez, pero **no** protege esta secuencia: el planificador puede interrumpir a un hilo entre el “leer” y el “escribir”. Con `contador += 1` en una sola línea, el intérprete moderno (3.13+) casi nunca corta justo ahí, por eso usamos `setswitchinterval` y pasos separados: la condición de carrera es real, solo hay que saber exponerla.

💰 La analogía: dos cajeros y una caja

Imagina una tienda con **una sola caja** y **dos cajeros**. La caja solo puede guardar el dinero que hay en ella, y los dos cajeros la comparten:

- El cajero A mira la caja: hay **100 €** (leer).
- Mientras A va a buscar cambio, el cajero B también mira la caja: **100 €** (leer).
- A vuelve y mete 50 €: escribe **150 €**.
- B vuelve y mete 50 €: también escribe **150 €**, ¡pisando el incremento de A!

Se han cobrado 100 € en total, pero en la caja solo hay 150 € en lugar de 200 €. **Los 50 € del cajero A han desaparecido** en la caja compartida: se perdieron por una condición de carrera. Exactamente lo mismo le pasa a `contador`.

El remedio es la **exclusión mutua**: mientras un cajero toca la caja, el otro espera fuera. Ese es el `Lock`, que verás en el punto 2.

🌎 Dónde aparece la condición de carrera

Situación	Qué se pierde
Contador compartido (`contador += 1`)	Incrementos
Saldo de una cuenta bancaria	Ingresos / retiros
Cola de trabajos compartida	Elementos de la cola
Registro (log) con fecha y hora	Líneas de log
Caché con varios lectores/escritores	Datos actualizados

> Es el bug más clásico de la programación concurrente: el programa **funciona casi siempre** y falla de forma intermitente e imposible de reproducir. Por eso la sincronización se diseña desde el principio, no al final.

🧠 Mini-chequeo

1. ¿Qué es una condición de carrera?
2. ¿Por qué `contador += 1` no es atómico y qué tres pasos esconde?
3. ¿Cómo explicarías el problema con la analogía de los dos cajeros y la caja?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Dos hilos que tocan la misma variable a la vez producen resultados incorrectos: eso es una **condición de carrera**.
- `contador += 1` no es atómico: son **leer, sumar, escribir**, y el planificador puede interrumpir entre pasos.
- La solución pasa por la **sincronización** (exclusión mutua), el tema del resto de la unidad.

🐛 Vocabulario rápido

Término	Idea general
Condición de carrera	Dos hilos tocan la misma variable a la vez y se pisan
Sección crítica	Zona de código que toca un recurso compartido
Operación atómica	Operación que no se puede interrumpir a mitad
Exclusión mutua	Solo un hilo accede al recurso compartido a la vez
GIL	Bloqueo global de Python: no protege la secuencia leer-sumar-escribir

📚 Volver al índice de la unidad · **Siguiente:** 02 · Lock

02 — Lock

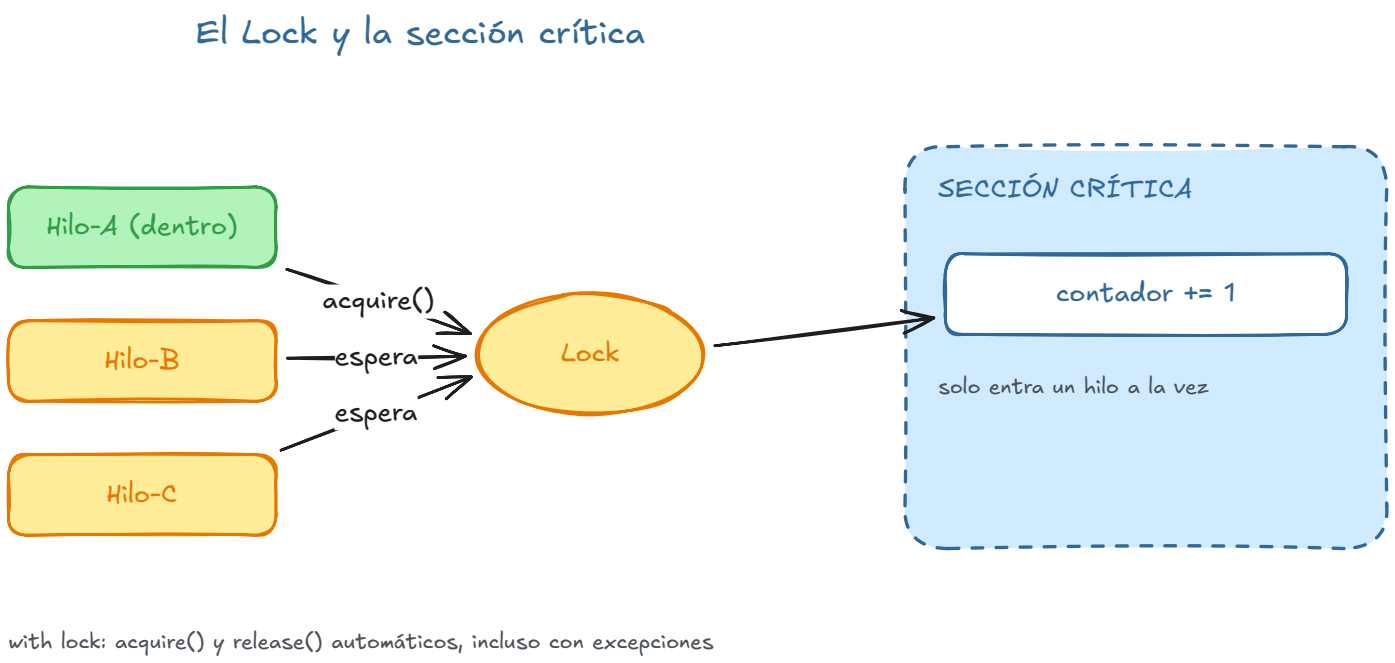
El cerrojo que protege la sección crítica 🔒

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 02 · Lock

📬 La idea en una frase

> Un **Lock** (cerrojo) garantiza que solo un hilo entre en la **sección crítica** a la vez: el resto espera fuera hasta que el primero lo libera. Es la exclusión mutua que faltaba en el punto 1.

En el ejemplo del contador compartido, la sección crítica es `contador += 1`. Protegerla con un Lock hace que los 4 hilos se turnen: nadie puede leer mientras otro está escribiendo, y el resultado final vuelve a ser el esperado.



with lock: acquire() y release() automáticos, incluso con excepciones

```python
import threading

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(100_000):
        with lock:           # 🤝 Solo un hilo aquí dentro
            contador += 1

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Con Lock: {contador}")  # ✅ 400.000
```

Ahora el resultado es siempre **400.000**, en cualquier ejecución. El `Lock` se adquiere antes de tocar el contador y se libera al salir del bloque.

🔑 Métodos de Lock

Método	Qué hace
`lock.acquire()`	Bloquea. Si otro hilo lo tiene, espera
`lock.release()`	Libera. Otro hilo puede entrar
`lock.locked()`	Devuelve `True` si está bloqueado

`acquire()` devuelve `True` si consiguió el lock. Si otro hilo lo tiene, el hilo que llama se **bloquea** (espera) hasta que se libere. `locked()` es útil para inspeccionar el estado sin bloquearse.

🤝 Siempre `with lock:`

```
lock = threading.Lock()

# ✅ La forma segura
with lock:
    contador += 1

# ⚠️ La forma peligrosa
lock.acquire()
contador += 1
# lock.release()  ← ¡si lo olvidas, DEADLOCK!
```

> ⚠️ **Siempre usa `with lock:`**. Si usas `acquire()` manual y olvidas `release()`, el hilo se queda con el cerrojo puesto para siempre y los demás esperan eternamente: un **deadlock**. El `with` libera el lock solo, incluso si el código dentro lanza una excepción.

🎭 Be the code: contador con y sin Lock

Sin Lock — el caos:

```
Hilo-A: lee contador = 0
Hilo-B: lee contador = 0          ← ¡mismo valor!
Hilo-A: escribe contador = 1
Hilo-B: escribe contador = 1      ← ¡pisó el incremento de A!
Hilo-A: lee contador = 1
Hilo-A: escribe contador = 2
Hilo-B: lee contador = 1          ← ¡leyó valor viejo!
```

Con Lock — el orden:

```
Hilo-A: acquire() → DENTRO
Hilo-A: lee contador = 0
Hilo-A: escribe contador = 1
Hilo-A: release() → FUERA

Hilo-B: acquire() → DENTRO        ← esperó hasta que A soltó
Hilo-B: lee contador = 1          ← valor correcto
Hilo-B: escribe contador = 2
Hilo-B: release() → FUERA

Hilo-C: acquire() → DENTRO
Hilo-C: lee contador = 2          ← valor correcto
...
```

Con el Lock, el "leer → sumar → escribir" de cada hilo ocurre **de principio a fin sin que nadie se cuele** entre medias. Los cajeros de la analogía del punto 1 ya no se pisan: mientras uno toca la caja, el otro espera fuera.

> 💡 **Regla de oro:** el Lock protege la **sección crítica**, no el hilo. Todo el código que toque la variable compartida debe ir dentro del mismo lock.

🧠 Mini-chequeo

1. ¿Qué garantiza un `Lock`?
2. ¿Qué pasa si dos hilos llaman a `acquire()` a la vez?
3. ¿Por qué `with lock:` es mejor que `acquire()`/`release()` manuales?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El `Lock` es el cerrojo que protege la sección crítica: solo un hilo a la vez.
- Con `with lock:` nunca se olvida liberarlo; con `acquire()`/`release()` manuales, un `release()` olvidado = deadlock.
- Protegido con Lock, el contador del punto 1 pasa de resultados aleatorios a un **400.000** exacto.

🐛 Vocabulario rápido

Término	Idea general
Lock	Cerrojo: exclusión mutua sobre una sección crítica
acquire()	Bloquear el lock (esperar si está ocupado)
release()	Liberar el lock para que otro entre
with lock:	Contexto que adquiere y libera el lock automáticamente
Sección crítica	Código que toca un recurso compartido

📚 Volver al índice de la unidad · **Anterior:** 01 · Condición de carrera · **Siguiente:** 03 · RLock

03 — RLock

Cuando el mismo hilo quiere entrar dos veces 🔁

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 03 · RLock

📬 La idea en una frase

> Un **Lock normal no es reentrante**: si el mismo hilo intenta adquirirlo dos veces, se espera a sí mismo y se produce un **deadlock**. Un **RLock** (Reentrant Lock) sí permite que el mismo hilo lo adquiera varias veces antes de liberarlo.

Recuerda el punto 2: `acquire()` bloquea "si otro hilo lo tiene". El problema es que el Lock no distingue entre "otro hilo" y "yo mismo".

```
import threading

# Lock normal — DEADLOCK si el mismo hilo intenta adquirirlo dos veces
lock = threading.Lock()
lock.acquire()
lock.acquire()  # ⚠️ El hilo se espera a sí mismo → DEADLOCK

# RLock — el mismo hilo puede adquirirlo varias veces
rlock = threading.RLock()
rlock.acquire()  # ok
rlock.acquire()  # ok (mismo hilo)
rlock.release()
rlock.release()
```

Con el `Lock` normal, la segunda `acquire()` nunca vuelve: el hilo espera a que se libere un lock que él mismo tiene puesto. **El hilo se espera a sí mismo.** Con `RLock`, cada `acquire()` incrementa un contador interno y cada `release()` lo decrementa: el lock solo se libera de verdad cuando el contador llega a 0.

📝 ¿Cuándo lo necesitas? Funciones que se llaman entre sí

El caso más típico: una función que adquiere el lock y llama a **otra función que también quiere el mismo lock**.

```
import threading

rlock = threading.RLock()

def depositar(cuenta, cantidad):
    with rlock:
        # Aquí se llama a otra función que también usa el lock
        registrar_operacion("ingreso", cantidad)
        cuenta[0] += cantidad

def registrar_operacion(tipo, cantidad):
    with rlock:            # ✅ mismo hilo, RLock lo permite
        print(f"📝 {tipo}: {cantidad} €")

cuenta = [100]
depositar(cuenta, 50)
print(f"Saldo: {cuenta[0]} €")
```

Si `registrar_operacion` usara un `Lock` normal, `depositar` tendría el lock puesto y la segunda `acquire()` se bloquearía para siempre. Con `RLock`, las dos adquisiciones son del **mismo hilo** y funcionan.

> 💡 `RLock` también es tu aliado si la función se llama a sí misma (recursión) y cada llamada toca la sección crítica.

⚖️ Lock vs RLock

Aspecto	Lock	RLock
¿Otro hilo puede adquirirlo?	Sí, cuando se libera	Sí, cuando se libera
¿El mismo hilo puede adquirirlo dos veces?	❌ Deadlock	✅ Sí (contador interno)

Aspecto	Lock	RLock
¿Cuándo se libera de verdad?	En el primer `release()`	Cuando los `release()` igualan a los `acquire()`
Uso típico	Sección crítica simple	Funciones que se llaman entre sí / recursión
Coste	Mínimo	Mínimo (un poco más de contabilidad)

> ⚠️ **Regla:** usa `Lock` por defecto. Solo cambia a `RLock` cuando de verdad el mismo hilo necesite adquirirlo más de una vez (funciones que se llaman entre sí). Un `RLock` no evita que dos hilos distintos se pisen: para eso siguen valiendo las mismas reglas del punto 2.

🧠 Mini-chequeo

1. ¿Qué le pasa a un `Lock` normal si el mismo hilo intenta adquirirlo dos veces?
2. ¿Cómo evita el `RLock` ese problema?
3. ¿Cuándo cambiarías de `Lock` a `RLock`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- El `Lock` normal no es reentrante: el mismo hilo que lo adquiere dos veces se bloquea a sí mismo (deadlock).
- El `RLock` lleva un contador interno y permite que el mismo hilo lo adquiera varias veces, liberándose cuando el contador vuelve a 0.
- Úsalo en funciones que se llaman entre sí o en recursión; para el resto, `Lock` por defecto.

🐛 Vocabulario rápido

Término	Idea general
RLock	Lock reentrante: el mismo hilo puede adquirirlo varias veces

Término	Idea general
Reentrante	Capaz de volver a adquirirse por quien ya lo tiene
Contador interno	No de `acquire()` sin `release()`; a 0 se libera
Deadlock	Hilo que espera un recurso que él mismo tiene
Recursión	Función que se llama a sí misma

📚 Volver al índice de la unidad · **Anterior:** 02 · Lock · **Siguiente:** 04 · Semaphore

04 — Semaphore

El aforo máximo de la sección crítica 🎟️

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 04 · Semaphore

📬 La idea en una frase

> Un **Semaphore** permite que hasta **N hilos** accedan al recurso a la vez: es como el aforo máximo de un local. Cuando se libera un puesto, entra el siguiente.

El `Lock` del punto 2 deja pasar a **uno** a la vez. El `Semaphore(2)` deja pasar a **dos** a la vez; `Semaphore(3)`, a **tres**… y así hasta el número que le pases al constructor.

```
import threading, time

semaforo = threading.Semaphore(2)  # Máximo 2 hilos dentro

def entrar(id):
    print(f"Hilo-{id} esperando...")
    with semaforo:
        print(f"  → Hilo-{id} DENTRO")
        time.sleep(2)
    print(f"  ← Hilo-{id} SALE")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

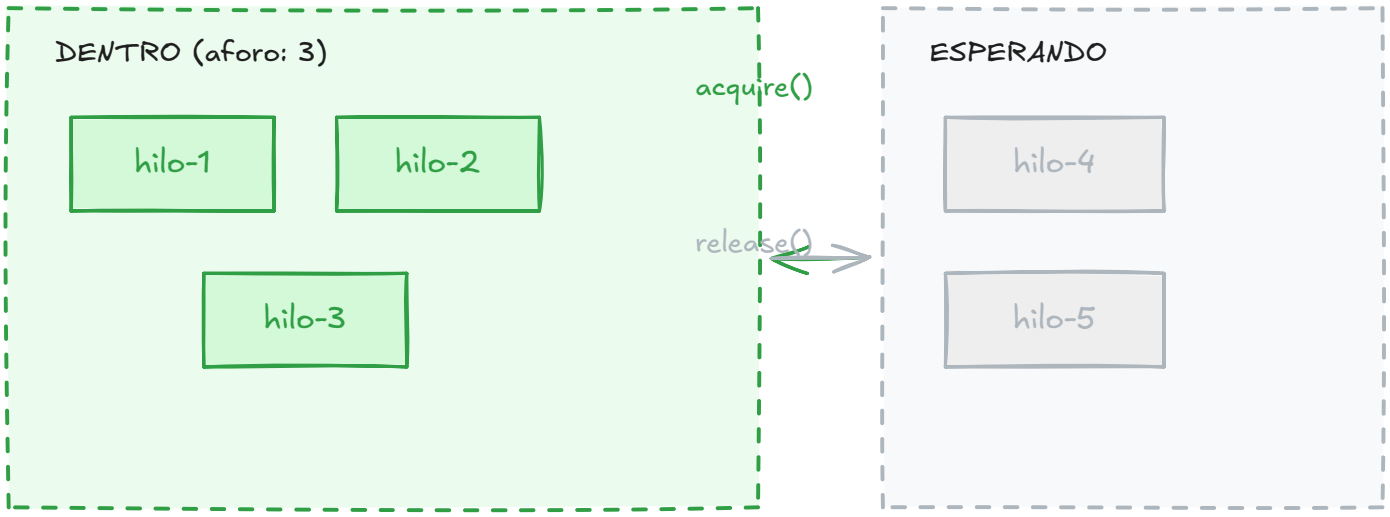
Salida (salen de 2 en 2):

```
Hilo-0 esperando...
Hilo-1 esperando...
Hilo-2 esperando...
Hilo-3 esperando...
Hilo-4 esperando...
  → Hilo-0 DENTRO
  → Hilo-1 DENTRO
  ← Hilo-0 SALE
  ← Hilo-1 SALE
  → Hilo-2 DENTRO
  → Hilo-3 DENTRO
  ← Hilo-2 SALE
  ← Hilo-3 SALE
  → Hilo-4 DENTRO
  ← Hilo-4 SALE
```

Los 5 hilos esperan, pero dentro solo hay **2 a la vez**. Cuando uno sale, entra el siguiente. El semáforo hace de portero con un contador interno: `acquire()` lo baja, `release()` lo sube.

⏱️ Semáforo con timeout

Semaphore(3): el aforo del local



Lock = Semaphore(1): para exclusión mutua usa Lock; para limitar a N, Semaphore

A veces no quieres esperar eternamente: si el recurso no se libera en X segundos, el hilo se rinde. `acquire(blocking=True, timeout=N)` devuelve `True` si consiguió el recurso y `False` si pasaron los N segundos.

```python
import threading, time

semaforo = threading.Semaphore(1)  # Solo 1 dentro

def entrar(id):
    if semaforo.acquire(timeout=1):
        print(f"Hilo-{id} entró")
        time.sleep(2)
        semaforo.release()
    else:
        print(f"Hilo-{id} TIMEOUT: se va")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (aproximada):

```
Hilo-0 entró
Hilo-1 TIMEOUT: se va
Hilo-2 TIMEOUT: se va
```

El hilo 0 ocupa el único puesto; los otros esperan 1 segundo y, al no entrar, **abandonan** en lugar de bloquearse para siempre. Ese `True/False` de `acquire(timeout=...)` te deja decidir si el hilo entra o se rinde.

🏭 ¿Para qué sirve un semáforo?

Situación	¿Por qué un semáforo?
Limitar conexiones a una base de datos	No saturar el servidor de BD
Descargas simultáneas	Máximo N descargas a la vez
Acceso a una API con rate limit	No superar las peticiones permitidas
Impresoras compartidas	Solo las N impresoras disponibles
Aforo de un recurso físico	Como el aforo máximo de un local

> 💡 **Diferencia con Lock:** el Lock es `Semaphore(1)`. Para **exclusión mutua** usa Lock (más simple y rápido); para **limitar acceso concurrente** a N recursos, Semaphore.

🧠 Mini-chequeo

1. ¿Cuántos hilos dejan entrar `Semaphore(3)` a la vez?
2. ¿Qué devuelve `acquire(timeout=2)` cuando el recurso se libera y cuando no?
3. ¿Cuándo elegirías Semaphore en lugar de Lock?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El `Semaphore(N)` limita a **N hilos** a la vez dentro del recurso: contador interno que baja al entrar y sube al salir.
- Con `acquire(timeout=N)` los hilos esperan un máximo de N segundos y se rinden si no entran (devuelve `True`/`False`).
- Es el mecanismo del **aforo**: conexiones a BD, descargas simultáneas o APIs con rate limit.

🐛 Vocabulario rápido

Término	Idea general
Semaphore	Aforo máximo: hasta N hilos dentro a la vez
Contador interno	Nº de puestos libres del recurso
acquire()	Reservar un puesto (baja el contador)
release()	Liberar un puesto (sube el contador)
Timeout	Esperar máximo N segundos y rendirse

📚 Volver al índice de la unidad · **Anterior:** 03 · RLock · **Siguiente:** 05 · Barrier

05 — Barrier

Ningún hilo avanza hasta que llegan todos 🏁

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 05 · Barrier

📬 La idea en una frase

> Una **Barrier** obliga a todos los hilos a esperar hasta que el último llegue. Entonces todos continúan **a la vez**: perfecta para sincronizar **fases** de un trabajo en paralelo.

El `Lock` y el `Semaphore` controlan **cuántos** pasan. La `Barrier` controla **cuándo**: nadie cruza la meta hasta que el grupo completo está listo.

```
import threading, time

barrera = threading.Barrier(3)  # 3 hilos deben llegar

def corredor(id):
    print(f"Corredor-{id} preparándose...")
    time.sleep(id * 0.5)        # Cada uno tarda distinto
    print(f"  → Corredor-{id} en la salida")
    barrera.wait()              # Espera a los demás
    print(f"  🏁 Corredor-{id} SALIÓ!")

hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida:

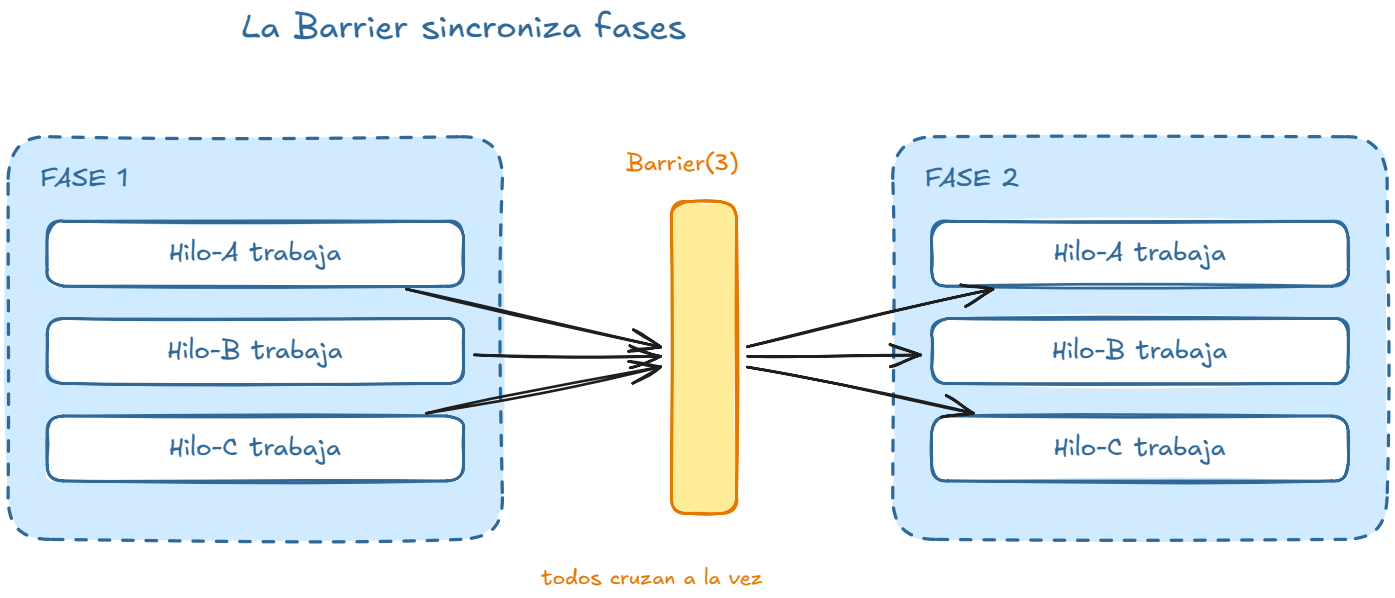
```
Corredor-0 preparándose...
  → Corredor-0 en la salida
Corredor-1 preparándose...
  → Corredor-1 en la salida
Corredor-2 preparándose...
  → Corredor-2 en la salida
  🏁 Corredor-0 SALIÓ!
  🏁 Corredor-1 SALIÓ!
  🏁 Corredor-2 SALIÓ!
```

Fíjate en la salida: los 3 corredores tardan distinto en llegar a la salida (`time.sleep(id * 0.5)`), pero **los 3 cruzan a la vez** justo después de que el último llegue. `barrera.wait()` devuelve cuando los 3 hilos han llegado al `wait()`.

> 💡 El `Barrier(3)` espera exactamente 3 hilos. Si un hilo no llega (se bloquea o muere), los demás esperan para siempre: **el número de hilos debe coincidir** con el de la barrera.

🧱 Sincronizar fases de un trabajo

La aplicación más típica: un trabajo en paralelo con varias **fases**. Nadie empieza la fase 2 hasta que todos terminaron la fase 1.



Nadie empieza la fase 2 hasta que todos terminan la fase 1

Un ejemplo: descargar 3 archivos, y solo cuando los 3 estén descargados, empezar a procesarlos.

```python
import threading, time

barrera = threading.Barrier(3)

def tarea(id):
    print(f"  📥 Descargando archivo-{id}...")
    time.sleep(1 + id)          # descargas de distinta duración
    barrera.wait()              # 🏁 espera a que todos terminen
    print(f"  📊 Procesando archivo-{id} (fase 2)")

hilos = [threading.Thread(target=tarea, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (extracto):

```
  📥 Descargando archivo-0...
  📥 Descargando archivo-1...
  📥 Descargando archivo-2...
  📊 Procesando archivo-0 (fase 2)
  📊 Procesando archivo-1 (fase 2)
  📊 Procesando archivo-2 (fase 2)
```

Ningún archivo se procesa hasta que los 3 están descargados. La barrera convierte el “cada uno a lo suyo” en “todos juntos en cada fase”.

⚖️ Barrier frente al resto

Mecanismo	Pregunta que responde
Lock	¿Quién toca el recurso? (solo uno)
Semaphore	¿Cuántos a la vez? (hasta N)
Barrier	¿Cuándo empieza la siguiente fase? (cuando todos llegan)

> Los tres se pueden combinar: una `Barrier` para coordinar fases, un `Semaphore` para limitar quién accede al recurso en cada fase y un `Lock` para las secciones críticas dentro de cada fase.

🧠 Mini-chequeo

1. ¿Qué hace `barrera.wait()`?
2. ¿Qué ocurre si solo llegan 2 de los 3 hilos esperados por `Barrier(3)`?
3. ¿Para qué tipo de problema es la barrera la herramienta ideal?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- La `Barrier(N)` espera a que **N hilos** lleguen al `wait()`; cuando el último llega, todos continúan a la vez.
- Es la herramienta para **sincronizar fases**: nadie empieza la siguiente fase hasta que todos acaban la anterior.
- El número de hilos debe coincidir con el de la barrera, o el grupo se queda esperando para siempre.

🦎 Vocabulario rápido

Término	Idea general
Barrier	Barrera: N hilos esperan hasta que llegan todos
wait()	El hilo llega a la barrera y espera al resto
Fase	Etapa de un trabajo; las fases se separan con barreras
Sincronización de grupo	Coordinar N hilos, no solo 2
BreakingBarrierError	Se lanza si un hilo de la barrera se rompe/aborta

📚 Volver al índice de la unidad · **Anterior:** 04 · Semaphore · **Siguiente:** 06 · Condition

06 — Condition

Cuando un hilo necesita que otro le avise 📢

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 06 · Condition

📬 La idea en una frase

> Una **Condition** permite que un hilo se duerma esperando a que otro le **notifique** que algo ocurrió: `wait()` duerme y libera el lock, `notify()` despierta a un hilo dormido.

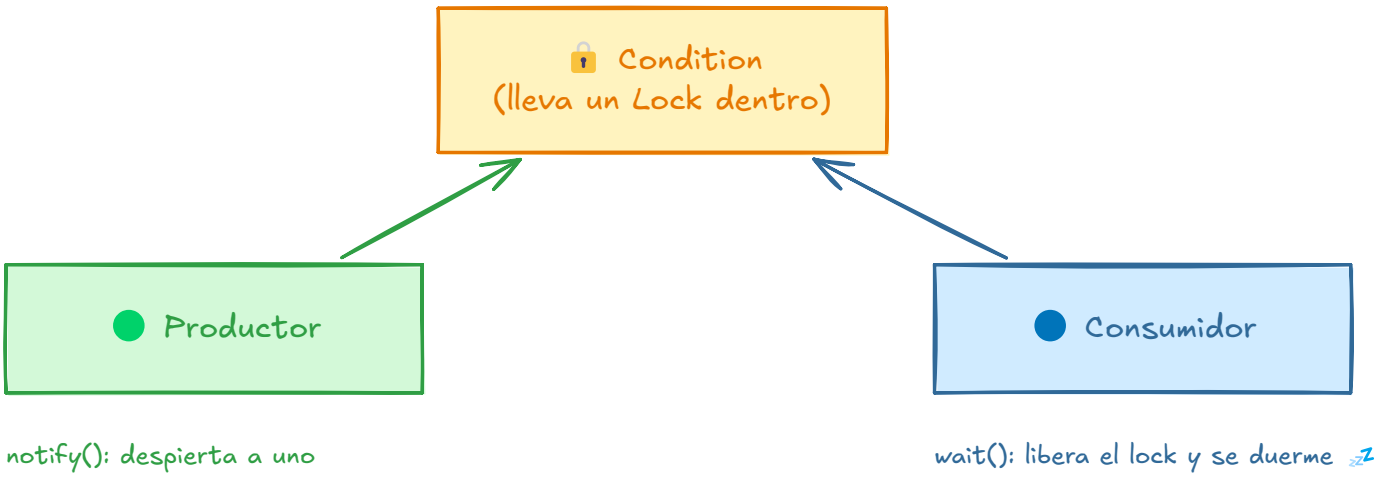
El `Lock`, el `Semaphore` y la `Barrier` sincronizan **acceso**. La `Condition` sincroniza **eventos**: un hilo quiere hacer algo, pero tiene que esperar a que la condición (por ejemplo, “hay un elemento en la cola”) se cumpla, y es otro hilo quien se la avisa.

```python
import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for i in range(5):
        with condition:
            item = random.randint(1, 100)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify()  # Avisa al consumidor
        time.sleep(random.random())

def consumidor():
    for _ in range(5):
        with condition:
            while not cola:
                condition.wait()  # Espera a que haya algo
            item = cola.pop(0)
            print(f"  🍽️ Consumido: {item}")

h_prod = threading.Thread(target=productor)
h_cons = threading.Thread(target=consumidor)
h_prod.start()
h_cons.start()
h_prod.join()
h_cons.join()
```

El productor añade un elemento y **notifica**; el consumidor, si la cola está vacía, **espera** hasta que le avisen. El `Condition` lleva asociado un lock interno: al entrar en `with condition:` se adquiere, y `wait()` lo **libera** mientras duerme para que el productor pueda entrar.

📚 Métodos de Condition

Condition: wait() y notify() bajo el mismo lock



⚠️ wait(), notify() y notify_all() solo dentro del with condition:

Método	Qué hace
`wait()`	Libera el lock y espera una notificación
`notify()`	Despierta a un hilo que está esperando
`notify_all()`	Despierta a todos los hilos que esperan

> ⚠️ **Siempre dentro del `with condition:`.** `wait()`, `notify()` y `notify_all()` exigen tener el lock de la condición adquirido.

⚠️ La regla de oro: `while`, no `if`

Tras despertar con `notify()`, el hilo debe **volver a comprobar la condición**:

```
# ✅ CORRECTO
while not cola:
    condition.wait()

# ❌ INCORRECTO
if not cola:
    condition.wait()
```

¿Por qué? Pueden darse **falsas activaciones** (spurious wakeups) o que otro hilo haya consumido el elemento mientras esperábamos. Con `if`, el hilo despierta y asume que la condición se cumple… pero puede que ya no. Con `while`, tras despertar **recomprueba** y, si la condición sigue sin cumplirse, vuelve a dormir.

> 💡 Esta es una de las diferencias que separa al novato del profesional: **siempre `while` después de `wait()`**.

🕵️ ¿Cuándo usarla?

Situación	¿Por qué Condition?
Productor-consumidor con cola	El consumidor espera hasta que haya producto
Un hilo “jefe” que espera a que los obreros terminen	`wait()` hasta que avisen
Recurso que se llena y se vacía	Notificar cuando el estado cambia
Turnos entre hilos	`notify()` al que le toca

La `Condition` es el mecanismo del **aviso**: un hilo no adivina cuándo hay trabajo, otro se lo dice. Es la pieza que convierte “comprobar sin parar” (y quemar CPU) en “dormir y que me despierten”.

```python
import threading, time

condition = threading.Condition()
listo = False

def trabajador():
    global listo
    with condition:
        while not listo:            # espera la señal de la jefa
            condition.wait()
    print("🛠️ Trabajador: ¡manos a la obra!")

def jefa():
    global listo
    time.sleep(1)
    with condition:
        listo = True
        condition.notify()          # ¡despierta al trabajador!
    print("👩‍💼 Jefa: ¡podéis empezar!")
```

Sin la condición, el trabajador tendría que comprobar `listo` en bucle gastando CPU. Con `wait()`/`notify()`, duerme hasta que la jefa le avisa.

🧠 Mini-chequeo

1. ¿Qué hace `wait()` exactamente con el lock de la condición?
2. ¿Por qué hay que usar `while` en lugar de `if` después de `wait()`?
3. ¿Qué diferencia hay entre `notify()` y `notify_all()`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- La `Condition` sincroniza **eventos**: un hilo hace `wait()` (liberando el lock y durmiendo) hasta que otro hace `notify()`.
- La regla de oro es **`while` y no `if`** después de `wait()`: las falsas activaciones exigen recomprobar la condición.

- Es la base del patrón **productor-consumidor**, que verás en el siguiente punto.

🐛 Vocabulario rápido

Término	Idea general
Condition	Mecanismo de espera y aviso entre hilos
wait()	Libera el lock y duerme hasta que le avisen
notify()	Despierta a un hilo que está esperando
notify_all()	Despierta a todos los hilos que esperan
Falsa activación	Despertar sin que nadie notificara de verdad (spurious wakeup)

📚 Volver al índice de la unidad · **Anterior:** 05 · Barrier · **Siguiente:** 07 · Productor-Consumidor

07 — Productor-Consumidor

El patrón clásico con cola y Condition 🏭🍽️

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 07 · Productor-Consumidor

📬 La idea en una frase

> El patrón **productor-consumidor** separa a quien **crea** datos (productor) de quien los **procesa** (consumidor): comparten una cola y una `Condition` para que el consumidor espere cuando la cola está vacía y el productor le avise cuando añade algo.

Es el patrón de sincronización más usado en el mundo real: un hilo llena una cola de tareas, otros hilos las procesan. La `Condition` del punto 6 es el pegamento que los coordina.

```python
import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for i in range(5):
        with condition:
            item = random.randint(1, 100)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify()  # Avisa al consumidor
        time.sleep(random.random())

def consumidor():
    for _ in range(5):
        with condition:
            while not cola:
                condition.wait()  # Espera a que haya algo
            item = cola.pop(0)
            print(f"  🍽️ Consumido: {item}")

h_prod = threading.Thread(target=productor)
h_cons = threading.Thread(target=consumidor)
h_prod.start()
h_cons.start()
h_prod.join()
h_cons.join()
```

Salida (aproximada):

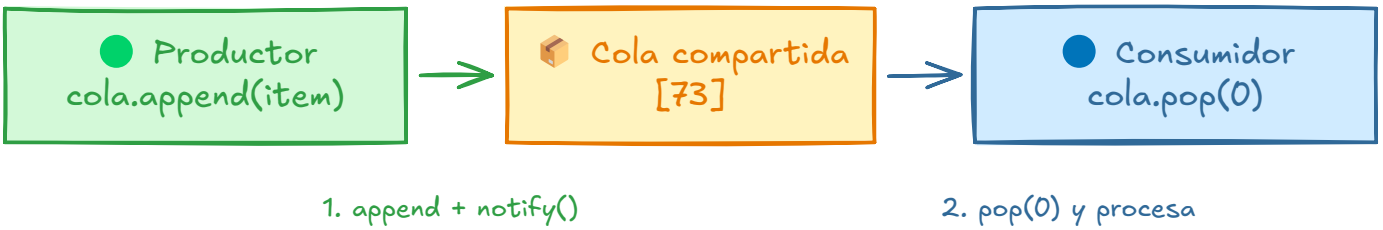
```
📦 Producido: 73
  🍽️ Consumido: 73
📦 Producido: 41
  🍽️ Consumido: 41
📦 Producido: 8
  🍽️ Consumido: 8
...
```

El ritmo lo marcan los `sleep()`: a veces el consumidor se adelanta, encuentra la cola vacía y espera; a veces el productor se adelanta, deja elementos y el consumidor los consume de

golpe. La `Condition` garantiza que **ninguno se pierde un dato** y que nadie se queda esperando un elemento que ya llegó.

🎬 Paso a paso

Productor-consumidor con Condition



1. Consumidor: cola vacía → wait() libera el lock y se duerme 💤
2. Productor: append(73) → notify() → libera el lock al salir del with
3. Consumidor: se despierta, recomprueba el while, saca el 73 y procesa

wait() libera el lock al dormir: sin eso, deadlock

Veamos cómo se coordinan los dos hilos:

```
1. El consumidor arranca, la cola está vacía
   → with condition: adquiere el lock
   → while not cola: es True
   → condition.wait()  → libera el lock y se duerme 💤

2. El productor arranca, con el lock libre
   → with condition: adquiere el lock
   → cola.append(item)  → cola = [73]
   → condition.notify() → despierta al consumidor 😀
   → sale del with → libera el lock

3. El consumidor se despierta
   → recomprueba while not cola → False (¡hay un item!)
   → item = cola.pop(0) → procesa el 73
   → sale del with → libera el lock
```

La magia de `wait()` del punto 6: mientras el consumidor duerme, **libera el lock** para que el productor pueda entrar y producir. Sin esa liberación, el productor nunca podría producir y el consumidor esperaría eternamente (deadlock).

📏 Con varios productores y consumidores

Con **varios** hilos a cada lado, la regla cambia ligeramente: si varios consumidores esperan, `notify()` solo despierta a **uno** y los demás se quedan dormidos aunque haya trabajo. Para no dejar a nadie esperando, el productor usa `notify_all()`:

```
import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for _ in range(6):   # 6 items: 2 consumidores × 3 = 6 consumos
        with condition:
            item = random.randint(1, 10)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify_all()   # despierta a TODOS los consumidores
        time.sleep(1)

def consumidor(id):
    for _ in range(3):
        with condition:
            while not cola:
                condition.wait()
            item = cola.pop(0)
            print(f"  🍽 Consumidor-{id} comió: {item}")

hilos = []
for i in range(2):
    hilos.append(threading.Thread(target=consumidor, args=(i,)))
for _ in range(1):
    hilos.append(threading.Thread(target=productor))

for h in hilos: h.start()
for h in hilos: h.join()
```

> 💡 Y si quieres limitar el tamaño de la cola (buffer con tope), el semáforo del punto 4 o la clase `queue.Queue(maxsize=N)` (thread-safe, sin locks a mano) te dan el control: los productores llaman a `put()` y los consumidores a `get()`. El ejemplo produce exactamente lo que los consumidores van a consumir (2 × 3 = 6); si produjera menos, los consumidores se quedarían esperando en `wait()` para siempre.

🧠 Mini-chequeo

1. ¿Qué papel juega cada hilo en el patrón productor-consumidor?
2. ¿Por qué `wait()` tiene que liberar el lock mientras el consumidor duerme?
3. ¿Por qué con varios consumidores conviene `notify_all()` en lugar de `notify()`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El patrón **productor-consumidor** separa la creación de datos (productor) del procesamiento (consumidor) con una cola compartida.
- La `Condition` coordina el aviso: el consumidor hace `wait()` (liberando el lock) y el productor hace `notify()`/`notify_all()` cuando añade un elemento.
- La regla `while` + `wait()` garantiza que ningún hilo consuma un dato que no existe ni se quede dormido con trabajo pendiente.

🐛 Vocabulario rápido

Término	Idea general
Productor	Hilo que crea datos y los mete en la cola
Consumidor	Hilo que saca datos de la cola y los procesa
Cola compartida	Buffer donde el productor deja y el consumidor recoge
notify_all()	Avisar a todos los consumidores que esperan
Buffer limitado	Cola con tope máximo (semáforo o `queue.Queue`)

📚 Volver al índice de la unidad · **Anterior:** 06 · Condition · **Siguiente:** 08 · Buenas prácticas

08 — Buenas prácticas

Deadlocks, orden de locks, el ring final y a practicar 🛡️

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 08 · Buenas prácticas

📬 La idea en una frase

> Ya tienes Lock, RLock, Semaphore, Barrier y Condition. Este punto cierra la teoría con el enemigo más temido —el **deadlock**—, las reglas para evitarlo y el "ring" que enfrenta a los mecanismos para que decidas cuándo usar cada uno.

Después de este punto, el free-threaded sin GIL (punto 9), el cierre de la unidad (punto 10) y los boletines pondrán a prueba todo lo aprendido.

🧊 El deadlock: todos esperan al Lock

Un **deadlock** es la situación en la que dos o más hilos se esperan **mutuamente para siempre**: cada uno tiene un recurso y espera el que tiene el otro.

Ejemplo clásico con 2 hilos y 2 locks:

```
Hilo-A: adquiere Lock-1          Hilo-B: adquiere Lock-2
Hilo-A: espera Lock-2  ←──→      Hilo-B: espera Lock-1
```

- Hilo-A tiene Lock-1 y espera a que se libere Lock-2.
- Hilo-B tiene Lock-2 y espera a que se libere Lock-1.
- **Ninguno de los dos puede avanzar.** Se esperan el uno al otro eternamente.

```python
import threading, time

lock1 = threading.Lock()
lock2 = threading.Lock()

def hilo_a():
    with lock1:
        time.sleep(0.1)          # tiempo para que B tome lock2
        with lock2:              # B tiene lock2... espera para siempre 💀
            print("A: ¡trabajando!")

def hilo_b():
    with lock2:
        time.sleep(0.1)          # tiempo para que A tome lock1
        with lock1:              # A tiene lock1... espera para siempre 💀
            print("B: ¡trabajando!")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start(); h2.start()
h1.join(); h2.join()  # ← nunca termina
```

Los `sleep(0.1)` fuerzan el escenario: A coge el lock1, B coge el lock2, y después cada uno intenta coger el del otro. El programa **se queda colgado** y ni A ni B imprimen su mensaje. La CPU se queda al 100% con dos hilos bloqueados que se esperan a sí mismos… en realidad, al otro.

🛡️ Cómo evitar deadlocks: las 3 reglas

1. **Adquirir los locks siempre en el mismo orden.** Si todos los hilos piden primero Lock-1 y luego Lock-2, el escenario anterior no puede darse: nadie tiene Lock-2 sin haber pasado por Lock-1.
2. **Usar `with lock:`** (nunca olvidar `release()`). Un `release()` olvidado deja el cerrojo puesto para siempre y cualquier hilo que lo espere, esperará eternamente.
3. **Usar `RLock` si el mismo hilo necesita adquirirlo varias veces** (lo viste en el punto 3): evita que un hilo se espere a sí mismo.

> ⚠️ **¿Puedo tener más de un Lock?** Sí, pero cada Lock añade riesgo de deadlock. Sé disciplinado con el orden de adquisición: **mismo orden para todos los hilos, siempre**.

⚡ Preguntas rápidas (y sus respuestas)

¿Qué es más rápido, Lock o Semaphore? Lock es más simple y, para exclusión mutua, gana en velocidad. Semaphore tiene contador interno: solo lo necesitas para limitar acceso concurrente (aforo). Para "solo uno a la vez", Lock.

¿Y si necesito sincronización entre procesos en vez de entre hilos? Usa `multiprocessing.Lock`, `multiprocessing.Semaphore`, etc. Son equivalentes a los de `threading` pero para procesos: recuerda que los procesos no comparten memoria, así que la sincronización se hace sobre recursos del sistema.

🥊 El ring de los conceptos — Lock vs Semáforo vs Barrera

Lock: "Soy el portero. Solo dejo pasar a **uno cada vez**. Los demás esperan fuera."

Semáforo(3): "Soy el guardia de una sala con **3 sillas**. Cuando una se libera, entra el siguiente."

Barrera(3): "Soy el juez de salida. **No permito que nadie corra** hasta que los 3 estén en la línea."

Lock: "Para exclusión mutua, no hay mejor que yo. Dos hilos no pueden tocarme el recurso a la vez."

Semáforo: "Para **limitar acceso concurrente**. Como un aforo máximo en un local."

Barrera: "Para sincronizar **fases de un trabajo**. Todos completan la fase 1, luego todos empiezan la fase 2."

> **Moraleja del ring:** cada mecanismo responde a una pregunta distinta — ¿quién? (Lock), ¿cuántos? (Semáforo), ¿cuándo? (Barrera). Y la `Condition` del punto 6 añade el aviso entre hilos.

📝 Aprieta el lápiz

1. **Condición de carrera**: Crea 2 hilos que incrementen un contador 1M veces cada uno. Compara con y sin Lock.

2. **Semáforo descargas**: Crea 10 hilos que simulen descargas (sleep 2s) usando un Semáforo(3). Mide el tiempo total.
3. **Barrera de fases**: Crea 4 hilos que trabajen en 2 fases. La fase 2 no empieza hasta que todos terminan la fase 1.
4. **Productor-Consumidor**: 1 productor crea números aleatorios cada 0.5s; 2 consumidores los procesan. Usa Condition.
5. **Deadlock provocado**: Crea intencionadamente un deadlock con 2 hilos y 2 locks. Luego arréglalo.

> 🔓 **Soluciones**

🧩 Pool Puzzle de sincronización

> **Instrucciones**: Ordena estas líneas para que 3 hilos incrementen un contador compartido de forma segura.

```
a. lock = threading.Lock()
b. for h in hilos: h.start()
c. contador += 1
d. hilos = []
e. import threading
f. for _ in range(3):
g. with lock:
h. def incrementar():
i.     hilos.append(threading.Thread(target=incrementar))
j. for h in hilos: h.join()
k. global contador
l. contador = 0
m. print(contador)
```

> 🧩 **Solución**

🧠 Mini-chequeo

1. Dibuja el escenario exacto de un deadlock con 2 hilos y 2 locks.
2. ¿Cuál es la regla de oro para los locks múltiples?
3. ¿Qué mecanismo usarías para: un recurso a la vez / aforo de 3 / coordinar fases?

✅ Resumen en 3 frases

- El **deadlock** es un abrazo mortal: cada hilo espera un recurso que tiene el otro, y ninguno avanza.
- Reglas anti-deadlock: **mismo orden de locks**, `with lock:` siempre, y `RLock` si el mismo hilo se re-adquiere.
- El ring final te deja la regla: Lock para exclusión mutua, Semaphore para aforo, Barrier para fases y Condition para el aviso.

🐛 Vocabulario rápido

Término	Idea general
Deadlock	Hilos que se esperan mutuamente para siempre
Orden de locks	Regla: todos los hilos piden los locks en el mismo orden
Abrazo mortal	Otro nombre del deadlock (Hilo-A y Hilo-B entrelazados)
multiprocessing.Lock	Lock equivalente para procesos en lugar de hilos
Pool Puzzle	Ejercicio de ordenar líneas de código sueltas

📚 Volver al índice de la unidad · **Anterior:** 07 · Productor-Consumidor · **Siguiente:** 09 · Python Free-Threaded

09 — Python Free-Threaded (PEP 703)

Python sin GIL: qué cambia, qué no y por qué el Lock sigue siendo el jefe 🔓

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 09 · Python Free-Threaded

📬 La idea en una frase

> **Free-threaded** es un build de CPython **sin GIL**: los hilos ejecutan bytecode Python en paralelo de verdad en varios núcleos (PEP 703). Pero **no arregla las condiciones de carrera**: sin sincronización, `contador += 1` sigue perdiendo incrementos… y ahora se pierden con más facilidad.

Hasta ahora has aprendido la sincronización con el GIL puesto: ese candado global que "protegía" (y limitaba) a los hilos. Este punto mira qué pasa cuando ese candado desaparece: el modo sin GIL nació en Python 3.13, se consolidó en el 3.14 y desde el 3.15 se instala por defecto (junto al build tradicional). La respuesta corta: **el GIL no era tu amigo, pero sí era tu candado de seguridad invisible**; sin él, la sincronización es más necesaria que nunca.

🔓 Qué es el free-threaded (y por qué tardó 15 años)

El **GIL** (UD 3 · punto 6) garantiza que un solo hilo ejecute bytecode Python en cada instante. Eso simplifica la memoria interna del intérprete, pero impide el paralelismo real de CPU entre hilos.

PEP 703 (*Making the Global Interpreter Lock Optional in CPython*, aceptado en octubre de 2024) define cómo eliminarlo sin romper Python. La clave técnica es el **conteo de referencias sesgado** (*biased reference counting*): cada objeto lleva un pequeño candado interno y una cuenta de referencias "ligera" para el hilo que lo posee; así, la recolección de basura no necesita un candado global.

Línea temporal

Versión	Estado	Qué implica para ti
3.13 (2025)	Fase I: **experimental**	Build opcional `python3.13t`; ~40 % más lento en monohilo
3.14 (2025)	Fase II: **soportado** (PEP 779)	Se quita la etiqueta “experimental”; el rendimiento mejora
3.15 (2026)	**Instalado por defecto**	Los instaladores oficiales lo incluyen activado (puedes desmarcarlo)
Futuro	Fase III: ¿build por defecto?	Aún sin decidir; el build con GIL seguirá existiendo un buen rato

> ⚠️ **No confundas builds**: `python` (con GIL) y `python3.15t` (free-threaded) son **dos intérpretes distintos**, con sus propios `site-packages`. Un paquete instalado en uno no aparece en el otro.

🕵️ ¿Mi Python es free-threaded? (3 comprobaciones)

```
import sys, sysconfig

print(sys.version)                                  # ¿"free-threading build"?
print(sys._is_gil_enabled())                        # False = sin GIL activo
print(sysconfig.get_config_var("Py_GIL_DISABLED"))  # 1 = build compatible
```

En la terminal basta con `python -VV`: si el intérprete es free-threaded, la versión lo dice. Y para forzar el modo a mano (solo en builds `t`):

```
PYTHON_GIL=0 python3.15t mi_programa.py   # o: python3.15t -X gil=0 mi_programa.py
```

Un detalle práctico: si importas una **extensión C** que aún no soporta free-threading (muchas librerías científicas lo están preparando), CPython **reactiva la GIL automáticamente** y avisa. Tu programa sigue siendo correcto, pero ya no hay paralelismo real.

🔬 Mira el bytecode: `contador += 1` bajo el capó

La condición de carrera del punto 1 sigue siendo el mejor laboratorio. Vamos a mirar qué instrucciones se esconden tras esa línea con el módulo `dis` (*disassemble*):

```
import dis

contador = 0

def sumar():
    global contador
    contador += 1

dis.dis(sumar)
```

Salida (abreviada, Python 3.15):

```
  5           0 LOAD_GLOBAL              0 (contador)
              2 LOAD_CONST               1 (1)
              4 BINARY_OP                0 (+)
              6 STORE_GLOBAL             0 (contador)
```

Tres operaciones, como siempre:

Instrucción	Qué hace	Fase
`LOAD_GLOBAL`	Lee `contador` de memoria	**Leer**
`BINARY_OP (+)`	Suma 1	**Sumar**
`STORE_GLOBAL`	Escribe el resultado	**Escribir**

Con GIL: el planificador solo puede interrumpir entre instrucciones, y en Python 3.13+ el intérprete casi nunca corta justo en medio de esta secuencia corta: por eso la demo clásica "casi nunca falla" y hay que forzarla con `sys.setswitchinterval(1e-6)`.

Sin GIL: dos hilos en dos núcleos ejecutan las tres instrucciones **a la vez, de verdad**. No hay quien los separe: la carrera no solo es posible, es **lo normal**.

🏃 demo_race_condition.py

El experimento completo, con comentarios en español, para ejecutarlo **con y sin GIL**:

```python
# demo_race_condition.py — condición de carrera con y sin GIL
import sys, threading

contador = 0

def sumar(c):
    return c + 1

def incrementar():
    global contador
    for _ in range(200_000):
        contador = sumar(contador)   # leer-sumar-escribir vía función

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()

print(f"GIL activa: {sys._is_gil_enabled()}")
print(f"Esperado: 800.000 | Obtenido: {contador}")
```

```
python demo_race_condition.py                # build normal (con GIL)
PYTHON_GIL=0 python3.15t demo_race_condition.py   # build free-threaded
```

- **Con GIL** y sin trucos: en 3.13+ suele dar 800.000 (o se acerca mucho); añade `sys.setswitchinterval(1e-6)` y perderá incrementos a mares.
- **Sin GIL:** pierde incrementos casi siempre, sin necesidad de ningún truco. Y es un problema de **esta** secuencia de instrucciones, no del hilo que "llega tarde": al no haber candado global, leer y escribir se solapan en paralelo.

> 🧠 El free-threaded **no toca la semántica de Python**: `contador += 1` sigue siendo leer-sumar-escribir. Lo que quita es la protección *accidental* que daba el GIL al serializarlo todo.

🧵 Por qué Lock y queue.Queue siguen mandando

Esto es lo más importante del punto: **free-threaded no es "Python multicore sin preocupaciones"**.

1. **Las carreras de Python siguen siendo carreras.** El GIL solo *reducía* la ventana de la carrera; la eliminaba por completo cuando el planificador no cortaba la secuencia. Sin él, hay que cerrar la ventana a mano: `Lock`.
2. **Las estructuras nativas sí están protegidas.** `list`, `dict` y `set` usan candados internos: `lista.append(x)` o `d[k] = v` no rompen el intérprete. Pero **tus** operaciones compuestas (`if saldo > 100: saldo -= 100`) no lo están.
3. **`queue.Queue` gana enteros.** Implementado con `Lock` + `Condition` por dentro, es la forma canónica de pasar datos entre hilos… y ahora con paralelismo real de CPU. El patrón productor-consumidor **no cambia ni una línea**.
4. **El rendimiento tiene trampa.** En monohilo el build free-threaded es más lento (el objetivo es ≤10 %; en 3.13 era ~40 %). Si tu programa es de red o disco, **la GIL no te frenaba**: `threading` normal es más rápido.

Lo que **sí** cambia: los trabajos CPU-bound con hilos dejan de necesitar `multiprocessing` en muchos casos… si los datos compartidos van protegidos con `Lock` o encerrados en una `queue.Queue`.

🗺️ Matriz de decisión (¿qué uso y cuándo?)

Situación	¿Con GIL?	Qué usar
Espera de red/disco (I/O-bound)	Da igual	`threading` o `asyncio` (el GIL se libera al esperar)
CPU-bound, datos compartidos	Siempre	Hilos + `Lock` / `queue.Queue` (con o sin GIL)
CPU-bound, sin datos compartidos	Sin GIL	Hilos directos: paralelismo real
CPU-bound, sin datos compartidos	Con GIL	`multiprocessing` (sigue siendo la salida)

Situación	¿Con GIL?	Qué usar
Código monohilo al máximo	Con GIL	El build tradicional va más rápido
Extensión C sin adaptar	—	CPython reactiva la GIL solo (con aviso)

Regla de bolsillo: el free-threaded cambia *de dónde sale el paralelismo*, no *las reglas de la sincronización*. Toda la unidad anterior sigue siendo de obligado cumplimiento.

🛠️ Actividades sencillas

Actividad 1 — Identifica tu intérprete

Escribe un script que imprima `sys.version`, `sys._is_gil_enabled()` y `sysconfig.get_config_var("Py_GIL_DISABLED")`. Ejecútalo con `python` y, si lo tienes, con `python3.15t`. Anota las diferencias.

> 🔄 **Ver solución**

Actividad 2 — Desnuda el `contador += 1`

Usa `dis.dis()` sobre una función que haga `contador += 1` con `global`. Identifica en la salida qué instrucción **lee**, cuál **suma** y cuál **escribe**. ¿Cuántas instrucciones hay en total?

> 🔄 **Ver solución**

Actividad 3 — Pon el Lock a prueba

Modifica `demo_race_condition.py` para que cada incremento pase por `with lock:`. Ejecútalo **con y sin GIL** (`PYTHON_GIL=0 python3.15t …`). ¿Los dos resultados son correctos ahora?

> 🔄 **Ver solución**

> 💪 **Extra:** cronometra las versiones con y sin lock (`time.time()`). ¿Notas el coste del lock con GIL? ¿Y sin él? En CPU-bound puro el lock serializa de todas formas: el paralelismo real aparece cuando el trabajo dentro del `with` es pequeño y el trabajo fuera, grande.

🧠 Mini-chequeo

1. ¿Quita el free-threaded la necesidad de usar `Lock`? ¿Por qué?
2. ¿Cómo compruebo en tres comandos si mi intérprete es free-threaded y si la GIL está activa?
3. ¿Qué instrucciones esconde `contador += 1` y por qué sin GIL la carrera es más probable?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- **Free-threaded** (PEP 703) es un build de CPython **sin GIL**: paralelismo real de CPU entre hilos, disponible desde 3.13, soportado desde 3.14 e instalado por defecto desde 3.15 (`python3.15t`).
- Quita el candado, no las reglas: `contador += 1` sigue siendo leer-sumar-escribir y **las condiciones de carrera empeoran** si no sincronizas.
- **Lock y queue.Queue siguen siendo obligatorios**; lo que cambia es que los hilos CPU-bound pueden paralelizarse sin caer en `multiprocessing`.

🐛 Vocabulario rápido

Término	Idea general
Free-threaded	Build de CPython sin GIL (binario con sufijo `t`)
PEP 703	La propuesta que hace opcional el GIL
Fases I / II / III	Experimental → soportado → build por defecto
Conteo de referencias sesgado	Candado ligero por objeto que sustituye al GIL
`sys._is_gil_enabled()`	Comprueba si la GIL está activa en este proceso
`PYTHON_GIL=0`	Variable para desactivar la GIL (solo en builds `t`)
Extensión C no adaptada	Módulo nativo que reactiva la GIL al importarlo

📚 Volver al índice de la unidad · **Anterior:** 08 · Buenas prácticas · **Siguiente:** 10 · Cierre

10 — Cierre: consolida lo aprendido

Sé el lock, laboratorio con carrera y el duelo final de la unidad 🧠

> 🗺️ **Estás en:** 🔒 **UD 4 · Sincronización** → 10 · Cierre

Has terminado la teoría: condición de carrera, Lock, RLock, Semaphore, Barrier, Condition, productor-consumidor, las reglas anti-deadlock y el free-threaded sin GIL. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 9 y antes de abrir los boletines.

⭐ Sé el lock

> *Eres un `threading.Lock` recién creado. Te han asignado proteger el contador compartido de 4 hilos. Cada hilo quiere hacer `contador += 1` 100.000 veces.*

¿Qué pasa?

1. El hilo A llama a `with lock:` → **adquieres** el cerrojo y le das paso.
2. A ejecuta `contador += 1` de principio a fin: lee, suma, escribe. **Nadie se cuela**: tú estás puesto.
3. A sale del bloque → llamas a `release()` y **liberas** el cerrojo.
4. El hilo B, que estaba esperando, entra: **adquieres** de nuevo, repite los pasos 2-3.
5. Los 4 hilos se turnan contigo: 100.000 veces cada uno.

Resultado: el contador llega a **400.000**, exacto, en todas las ejecuciones. Tú (el Lock) has convertido el caos de la condición de carrera en un turno ordenado.

> 💡 **Ahora tú:** ¿y si un hilo olvidara llamarte con `release()`? Te quedarías puesto para siempre y el resto esperaría eternamente: eso es un **deadlock**. Por eso los hilos te usan siempre con `with lock:`, que garantiza el `release()` pase lo que pase (lo viste en el punto 2).

🔥 Fireside Chat: Lock vs Semaphore

> *Dos mecanismos de sincronización se sientan junto a la chimenea a resolver, de una vez, quién controla el acceso.*

Lock: — Yo soy la exclusión mutua. Solo uno de mis hilos toca el recurso a la vez. Los demás esperan fuera hasta que libero.

Semaphore: — Vaya, qué estricto. Yo dejo pasar hasta **N**. Tengo un contador interno: mientras queden puestos, entran; cuando se acaban, esperan.

Lock: — ¿Y para qué querría nadie que entren varios? Mi recurso no admite más de uno.

Semaphore: — Para un **aforo**, querido: 3 impresoras, 3 descargas, 3 conexiones a la BD. Conmigo entran exactamente 3. Tú solo sabes decir "uno".

Lock: — Pero yo soy más simple y rápido. Para una sección crítica, no hay discusión.

Semaphore: — Y yo puedo hacer de ti: `Semaphore(1)` equivale a tu exclusión mutua. Pero tú no puedes ser yo.

Lock: — Vale, cada uno a lo suyo. ¿Y esa barrera que todo lo frena?

Semaphore: — *suspira* Esa no nos habla. Solo sabe esperar a que lleguen todos antes de dejar correr a nadie. Cosas de carreras de relevos.

> **Moraleja**: Lock para exclusión mutua (solo uno), Semaphore para aforo (hasta N), Barrier para fases (cuando todos llegan). Y la Condition de fondo, avisando.

🕵️ ¿Quién soy?

1. Soy el cerrojo que deja pasar a **un solo** hilo a la sección crítica a la vez.
2. Soy el cerrojo que el mismo hilo puede adquirir varias veces sin bloquearse.
3. Soy el guardia de una sala con **3 sillas**: cuando una se libera, entra el siguiente.
4. Soy el juez de salida: no dejo correr a nadie hasta que los 3 corredores están en la línea.
5. Soy el aviso: un hilo se duerme con `wait()` hasta que otro lo despierta con `notify()`.
6. Soy el bug que ocurre cuando dos hilos se pisan la misma variable compartida.

> › 🔁 **Respuestas**

👹 CONRAD VS EL MUNDO: "olvidé el `release()` y todos esperan al lock"

CONRAD: — "Clásico: el programa se cuelga y nadie sabe por qué. Miras el código y… ¡claro! Usé `lock.acquire()`, hice el trabajo, y se me olvidó el `lock.release()`. El hilo se quedó con el cerrojo puesto, y todos los demás se quedaron esperando al lock para siempre. No es un fallo raro: es un **deadlock** en su forma más tonta."

CONRAD: — "Y lo mejor: *'pero yo hago acquire solo en este sitio'*. ¡Pues claro! También lo haces dentro de un `with`, y así nunca más. El `with lock:` hace el `release()` solo, aunque haya una excepción en medio. Adquirir a mano sin liberar es firmar una sentencia de muerte para tus hilos."

CONRAD: — "Y no me vengas con *'¿será que el ordenador va lento?'*. No. Si un hilo se queda esperando para siempre y el contador no llega a su valor, ya tienes la respuesta: hay un lock mal liberado o un orden de adquisición incorrecto. El GIL del punto 1 no te protege de esto. A diagnosticar."

⚡ Laboratorio de tortura: la caja común de la cooperativa

> **Duración:** 45 minutos **Herramienta:** Python 3 (`threading` y `time`, sin instalar nada)

Escenario: 4 hilos representan a 4 socios de una cooperativa que comparten una caja común (el `contador`). Cada socio debe aportar 100.000 monedas. Al final de la jornada, la caja debe contener **400.000**.

Tareas paso a paso:

1. Escribe la versión **sin sincronizar** del contador y ejecútala varias veces: anota que el total casi nunca es 400.000. Para que la carrera sea visible, pasa la suma por una función (`contador = sumar(contador)`) y añade `sys.setswitchinterval(1e-6)` (condición de carrera del punto 1).
2. Protege el incremento con `lock = threading.Lock()` y `with lock:` (el punto 2). Ejecútala 3 veces: ahora siempre da **400.000**.
3. Sustituye el `Lock` por un `Semaphore(1)` y verifica que también funciona (el punto 4).
4. Añade una **segunda caja** (otro contador de operaciones) que se incremente desde una función auxiliar llamada dentro de otra: protégelas con un `RLock` para que el mismo hilo pueda adquirirlo dos veces (punto 3).
5. Añade una `Barrier(4)` para que los 4 socios **empiecen a aportar a la vez**: nadie toca su caja hasta que los 4 están en la salida (punto 5).

Fallo intencionado: en la tarea 2, "por accidente", uno de los 4 hilos hace `contador += 1` **fuera** del `with lock:` (o bien un hilo que usa `lock.acquire()` sin `release()`). ¿Qué pasa? El contador vuelve a dar valores por debajo de 400.000, y si es el `release()` olvidado, el programa se **cuelga** esperando el `join()`. Los demás hilos están perfectos: el fallo es de un único hilo.

> **Pista 1:** busca el único `contador += 1` que no está dentro de `with lock:`. El contador da mal aunque los otros 3 hilos estén bien protegidos: basta **un** hilo fuera de la sección crítica.
>
> **Pista 2:** si todo parece protegido pero el programa no termina, revisa que nadie llamó a `acquire()` sin su `release()`. Un `with lock:` nunca lo olvida; un `acquire()` suelto, sí. Ahí tienes el deadlock del punto 8.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Carrera Cazador**	Explicar por qué `contador += 1` no es atómico y provocar una condición de carrera
🏅 **Cerrojo Maestro**	Proteger un contador compartido con `Lock` y `with lock:` y obtener siempre 400.000
🏅 **Aforo Controlado**	Limitar con `Semaphore` el acceso a N recursos (descargas, BD, impresoras)
🏅 **Sincronizador de Fases**	Coordinar N hilos con `Barrier` para que ninguna fase arranque antes de tiempo
🏅 **Jefe de Cola**	Implementar un productor-consumidor con `Condition` (wait/notify) sin perder datos
🏅 **Rompe Deadlocks**	Detectar un deadlock y arreglarlo con orden de locks y `with lock:`

🧠 Atrévete a pensar

1. ¿Por qué `contador += 1` con 4 hilos y sin lock casi nunca da el valor esperado, pero a veces sí?

2. ¿Qué pasaría si usaras `Semaphore(3)` para proteger una sección crítica que debe ser exclusiva?
3. ¿Por qué `notify()` puede dejar a un consumidor esperando para siempre con varios consumidores?
4. ¿Cuál es la diferencia práctica entre `Lock` y `RLock` para una función que llama a otra?
5. ¿Por qué "adquirir siempre en el mismo orden" elimina el deadlock de 2 hilos y 2 locks?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Cerrojo que deja pasar a un solo hilo a la vez (4 letras)
3. Lock que el mismo hilo puede adquirir varias veces (5 letras)
6. Condición en la que dos hilos se esperan para siempre (8 letras)
8. Mecanismo de espera y aviso: wait() y notify() (9 letras)

Vertical:
2. Aforo máximo: hasta N hilos dentro a la vez (9 letras)
4. Sincroniza fases: nadie avanza hasta que llegan todos (7 letras)
5. Operación que no se puede interrumpir a mitad (7 letras)
7. Bug cuando dos hilos se pisan una variable compartida (7 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **"¿Qué es una condición de carrera y cómo la evitas en Python?"**
2. **"Explica la diferencia entre Lock, RLock, Semaphore y Barrier. ¿Cuándo usarías cada uno?"**
3. **"¿Cómo funciona `Condition.wait()` y `notify()`? ¿Por qué se usa `while` y no `if`?"**
4. **"Implementa un productor-consumidor con varios consumidores."**
5. **"¿Qué es un deadlock? Pon un ejemplo y explica cómo prevenirlo."**

> 💡 **Cómo encararlas:** la 2 y la 4 son las "preguntas reina". Para la 2, recorre el ring del punto 8: Lock = exclusión mutua (solo uno), Semaphore = aforo (hasta N), Barrier = fases (cuando

> todos llegan), RLock = reentrante. Para la 4, plantea una cola compartida, `Condition` con `notify_all()`, y la regla `while not cola: wait()`. Si sabes contarlo fluido, ya eres medio experto en concurrencia.

🤷 No hay preguntas tontas

> ❓ **¿Qué es un deadlock?**

Dos o más hilos esperándose mutuamente para siempre. Ejemplo: Hilo-A tiene Lock-1 y espera Lock-2; Hilo-B tiene Lock-2 y espera Lock-1.

> ❓ **¿Cómo evito deadlocks?**

1. Adquirir los locks siempre en el **mismo orden**
2. Usar `with lock:` (nunca olvidar `release()`)
3. Usar `RLock` si el mismo hilo necesita adquirirlo varias veces

> ❓ **¿Puedo tener más de un Lock?**

Sí. Pero cada Lock añade riesgo de deadlock. Sé disciplinado con el orden de adquisición.

> ❓ **¿Qué es más rápido, Lock o Semaphore?**

Lock es más simple. Semaphore tiene contador interno. Para exclusión mutua, Lock gana en velocidad.

> ❓ **¿Y si necesito sincronización entre procesos en vez de entre hilos?**

Usa `multiprocessing.Lock`, `multiprocessing.Semaphore`, etc. Son equivalentes pero para procesos.

🎬 Poscréditos

> *El contador llega a 400.000, exacto. Los 4 hilos se turnan con el Lock sin pisarse.*

Un hilo se olvida del `release()`. Los demás esperan al lock. El programa se congela.

El semáforo regula el aforo, la barrera alinea las fases y el productor despierta al consumidor con un `notify()`.

La sincronización ha ganado la batalla contra el caos.

PRÓXIMAMENTE EN UD 5: *Sockets TCP y UDP. Los hilos comparten memoria y ahora saben coordinarse… pero ¿y si los procesos ni siquiera comparten memoria? Tocará hablar por la red.*

> 🔓 **¿Y si Python quitara la GIL?** Eso es el modo **free-threaded** (PEP 703), ya disponible en Python 3.13+ e instalado por defecto desde el 3.15. Te lo cuentamos en el punto 9 · Python Free-Threaded: el paralelismo de CPU es real, pero `Lock` y `queue.Queue` siguen mandando.

✅ Criterios de evaluación cubiertos (RA2)

RA2: Gestiona la programación de hilos y su sincronización.

CE	Criterio	Cubierto
RA2c	Sincroniza hilos con Lock	✅ Puntos 2, 3 y 7 + ⚡ Laboratorio de tortura
RA2d	Usa semáforos para acceso controlado	✅ Punto 4 + ⚡ Laboratorio de tortura
RA2g	Evita condiciones de carrera	✅ Puntos 1, 2 y 8 + ⚡ Laboratorio con fallo intencionado

> RA2a, RA2b, RA2e, RA2f y RA2h se cubren en la **UD 3 · Hilos y concurrencia**.

📚 Volver al índice de la unidad · **Anterior:** 09 · Python Free-Threaded · **Siguiente:** UD 5 · Sockets TCP y UDP

UD 5 — Sockets TCP y UDP

Comunicaciones en red: clientes, servidores y protocolos 🔌

> "Un socket es como un teléfono: marcas, esperas a que contesten, habláis y colgáis. La diferencia es que TCP garantiza que cada palabra llegue; UDP lanza el mensaje y reza."

En la UD 4 sincronizaste hilos para que no se pisen entre sí dentro de un mismo programa. Ahora toca lo contrario: hacer que **procesos distintos, incluso en máquinas distintas, hablen entre ellos**. La herramienta son los **sockets**: TCP para cuando necesitas fiabilidad, UDP para cuando necesitas velocidad.

En esta unidad construirás tu primera conversación entre programas: un **cliente** que llama y un **servidor** que contesta. Aprenderás el three-way handshake de TCP, los errores típicos cuando la red se rompe, y también cómo usar UDP para datagramas sin conexión. Al final, montarás servidores eco en ambos protocolos.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un **socket** y para qué sirven la **IP** y el **puerto**.
- Distinguir **TCP** (fiable, con conexión) de **UDP** (rápido, sin conexión).
- Implementar un **cliente TCP** con `connect()`, `sendall()` y `recv()`.
- Implementar un **servidor TCP** con `bind()`, `listen()` y `accept()`.
- Describir el **three-way handshake** y el cierre de una conexión TCP.
- Gestionar **errores de red** con `try/except`.
- Configurar **SO_REUSEADDR** y **timeouts** para servidores robustos.
- Implementar un **cliente UDP** con `sendto()` y `recvfrom()`.
- Implementar un **servidor UDP** con `bind()` y `recvfrom()`.
- Hablar **HTTP** a nivel de socket y entender **NTP** sobre UDP.
- Decidir cuándo usar TCP y cuándo UDP en un caso real.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Qué es un socket	IP + puerto, TCP vs UDP, la analogía del teléfono	Todos
02 · Cliente TCP	`connect()`, `sendall()` y `recv()`: el cliente que habla primero	Todos
03 · Servidor TCP	`bind()`, `listen()` y `accept()`: el servidor que escucha	Todos
04 · Ciclo de vida y errores	Handshake, cierre, TIME_WAIT, `SO_REUSEADDR`, timeouts	Todos
05 · Cliente y servidor UDP	`sendto()`, `recvfrom()` y la vida real de los datagramas	Todos
06 · HTTP y NTP	HTTP hablado a pelo, byte ordering y el reloj de Internet	Todos
07 · Cuándo usar cada protocolo	Criterios de decisión y casos reales: DNS, VoIP, streaming	Todos
08 · Práctica eco	Servidor + cliente eco en TCP y UDP, mano a mano	Todos
09 · Cierre	Sé el socket, Fireside, Laboratorio de tortura…	Todos

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación.

CE	Criterio	Dónde se cubre
RA3a	Modelo de capas de red (TCP/IP)	✅ Punto 1
RA3b	Protocolo UDP	✅ Puntos 1 y 5
RA3c	Crea servidores TCP	✅ Puntos 3 y 8 + ⚡ Laboratorio (punto 9)
RA3d	Crea clientes TCP	✅ Puntos 2 y 8 + ⚡ Laboratorio (punto 9)
RA3e	Crea servidores y clientes UDP	✅ Punto 5 y 8 + ⚡ Laboratorio (punto 9)
RA3f	Gestiona errores de red	✅ Punto 4 + ⚡ Laboratorio (punto 9)
RA3g	Configura opciones de socket	✅ Punto 4
RA3h	Protocolos HTTP/NTP sobre sockets	✅ Punto 6

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 4 y dominas hilos y sincronización? Empieza por el punto 1, que parte de cero: qué es un socket, la IP y el puerto.

¿Ya sabes qué es un socket? Saltar al punto 3, al 4 o al 5 (UDP).

📍 **Primer punto:** 01 · Qué es un socket ⏭️ **Al acabar la unidad, continúa en UD 6 · Servidores concurrentes.**

01 — Qué es un socket

IP + puerto y TCP contra UDP: fiabilidad o velocidad 🔌

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 01 · Qué es un socket

📬 La idea en una frase

> Un **socket** es el punto final de una conexión de red: la interfaz que tu programa usa para enviar y recibir datos a través de la red. Y según cómo hable, será **TCP** (la carta certificada) o **UDP** (el avión de papel).

Piensa en una llamada telefónica (la analogía que abre esta unidad): marcas, esperas a que contesten, habláis y colgáis. Tu teléfono es el **socket**; el número que marcas es la **IP**; y la extensión a la que pides hablar es el **puerto**. Sin ese trío no hay conversación posible.

🖥️ IP y puerto: el "quién" y el "dónde"

Para que dos programas hablen por la red necesitan dos datos:

Concepto	Qué es	Ejemplo
IP	Identifica la **máquina** en la red	`127.0.0.1`, `192.168.1.10`
Puerto	Identifica el **programa** dentro de la máquina	`80` (web), `5000`, `9000`

- `127.0.0.1` es **localhost**: tu propia máquina. Perfecto para pruebas sin red real. Para que otros se conecten, usa tu IP real (ej: `192.168.1.x`).
- Un puerto es un número del **0 al 65535**. El SO se reserva algunos para servicios conocidos (el `80` para web, el `443` para HTTPS); para tus programas usa puertos altos como `5000` o `9000`.

> 💡 La **pareja (IP, puerto)** se llama *dirección de socket* o *endpoint*, y en Python siempre se pasa como una tupla: `("127.0.0.1", 5000)`.

🐍 Crear un socket en Python

```python
import socket

# Crear un socket TCP
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Parámetro	Significado	Valores
`AF_INET`	Familia de direcciones	IPv4
`AF_INET6`	IPv6	
`SOCK_STREAM`	TCP (orientado a conexión)	
`SOCK_DGRAM`	UDP (sin conexión)	

- **`AF_INET`** dice al SO que usaremos direcciones **IPv4** (`AF_INET6` para IPv6).
- **`SOCK_STREAM`** pide un socket **TCP**, orientado a conexión: el canal fiable, ordenado y sin pérdidas.
- **`SOCK_DGRAM`** pide un socket **UDP**, sin conexión: datagramas sueltos que envías y olvidas.

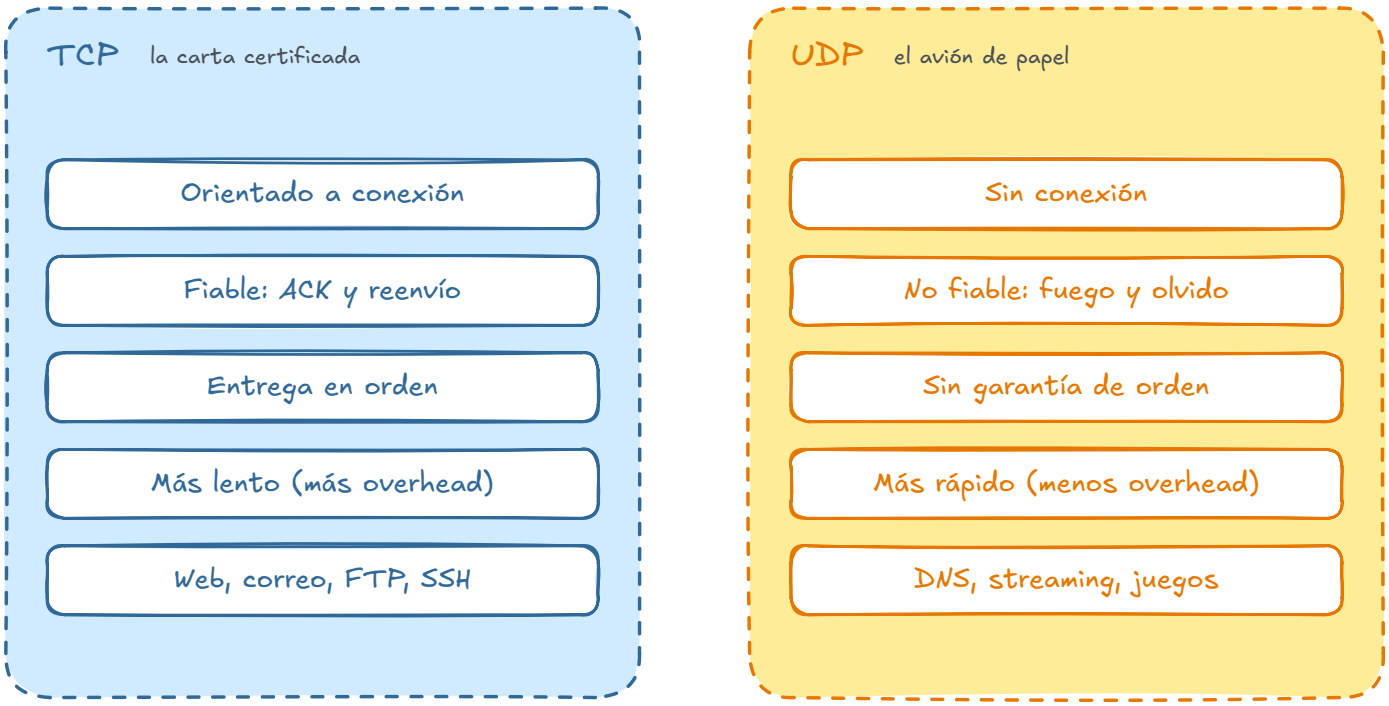
> 💡 Truco para no liarte: **D**GRAM = datagramas sueltos (**UDP**); **S**TREAM = flujo continuo (**TCP**).

✈️ La analogía: carta certificada contra avión de papel

- **TCP** es la **carta certificada**: pagas más, el cartero te confirma la entrega, y si se pierde, se reenvía. Sabes que llegó, y en el orden correcto.
- **UDP** es el **avión de papel** desde el balcón: lo lanzas y olvidas. Si aterriza, bien; si el viento se lo lleva, también. Pero puedes lanzar cien aviones en el tiempo que tardas en preparar una carta certificada.

Esas dos filosofías definen el resto de la unidad: **fiabilidad contra velocidad**.

⚖️ TCP vs UDP cara a cara



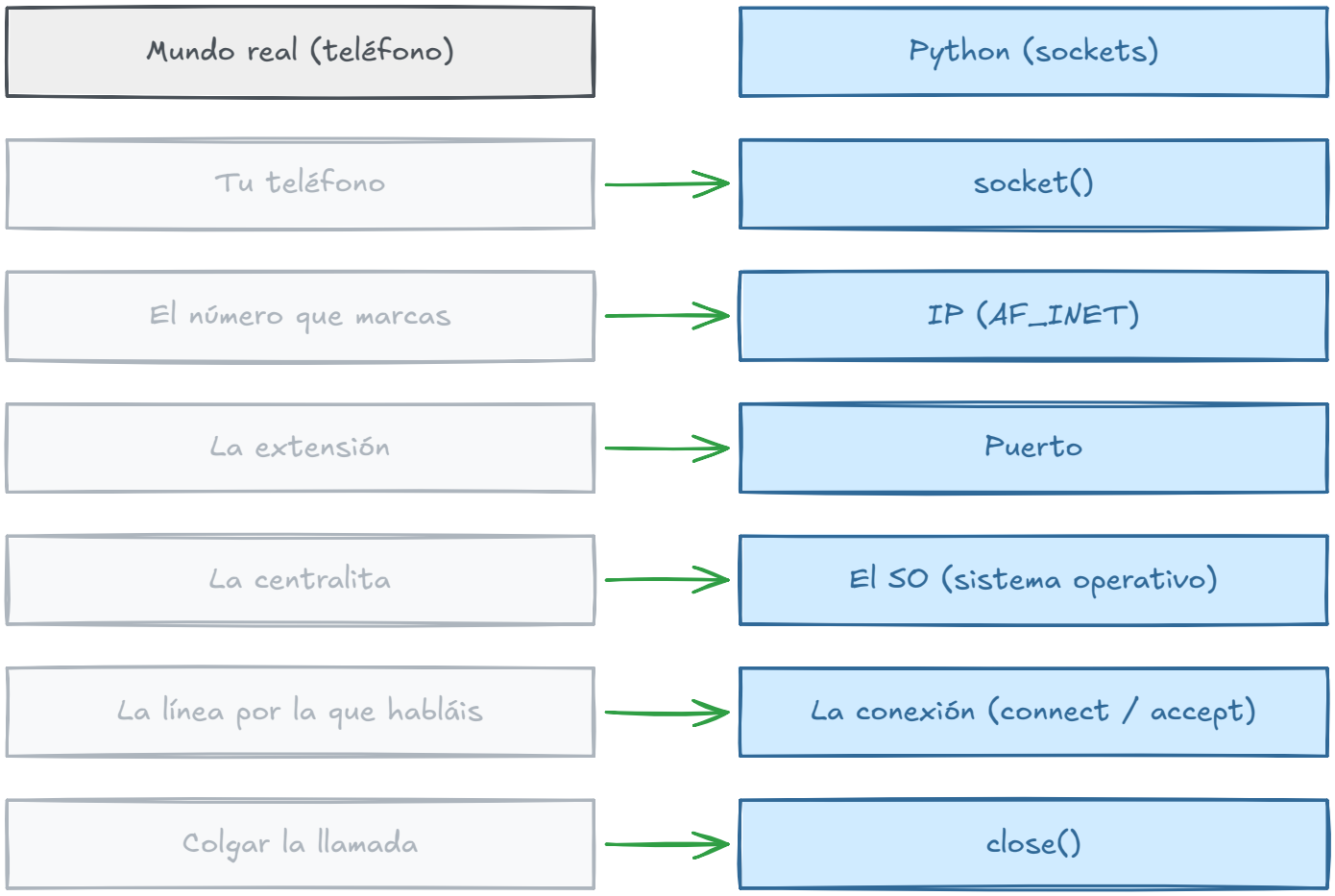
TCP cuando importan los datos · UDP cuando importa la velocidad

Característica	TCP	UDP
Conexión	Sí (handshake)	No
Entrega garantizada	Sí	No
Orden	Sí	No
Velocidad	Más lento	Más rápido
Uso típico	Web, correo, FTP	Streaming, juegos, DNS

La columna de la derecha es la que te ocupará en la segunda mitad de la unidad: tres filas de “No” que, paradójicamente, son la razón de que UDP sea tan rápido.

📞 La analogía del teléfono, al detalle

La analogía del teléfono, al detalle



El **SO** es la centralita: se encarga de que el paquete con tu mensaje salga de tu teléfono y llegue al teléfono correcto. Tú solo te ocupas de marcar (conectarte) y de hablar (enviar y recibir). El socket es ese teléfono: el **punto final** de la conversación.

🧠 Mini-chequeo

1. ¿Qué identifica la **IP** y qué identifica el **puerto**?
2. ¿Qué representa `127.0.0.1` y para qué sirve?
3. ¿Qué constante de Python crea un socket **TCP** y cuál uno **UDP**?
4. ¿Qué garantiza TCP que UDP no garantiza? (tres cosas)

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un **socket** es el punto final de una conexión de red: la interfaz para enviar y recibir datos; la IP identifica la máquina y el puerto el programa.
- **TCP** garantiza conexión, entrega y orden (carta certificada); **UDP** no garantiza nada y es más rápido (avión de papel).
- En Python se distinguen en una línea: `SOCK_STREAM` para TCP y `SOCK_DGRAM` para UDP, ambos con `AF_INET`.

🐛 Vocabulario rápido

Término	Idea general
Socket	Punto final de una conexión de red
IP	Identifica la máquina en la red
Puerto	Identifica el programa dentro de la máquina
AF_INET	Familia de direcciones IPv4
SOCK_STREAM	Socket TCP orientado a conexión
SOCK_DGRAM	Socket UDP sin conexión
localhost	127.0.0.1, tu propia máquina
Datagrama	Paquete independiente que manda UDP

📚 Volver al índice de la unidad · **Siguiente:** 02 · Cliente TCP

02 — Cliente TCP

connect, sendall y recv: el cliente que habla primero 🗣️

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 02 · Cliente TCP

📬 La idea en una frase

> El cliente TCP **marca el número**: crea el socket, se conecta con `connect()`, envía con `sendall()`, recibe con `recv()` y cierra. Sin cliente nadie llama; sin él, el servidor esperaría en el puerto para siempre.

En el punto 1 viste qué es un socket. Aquí lo pones a trabajar por el lado del **cliente**: el que inicia la conversación. En la analogía del teléfono, eres tú marcando el número y esperando a que contesten.

🔄 Ciclo de vida del cliente

```
socket() → connect() → send()/recv() → close()
```

Cuatro pasos en orden: **crear**, **conectar**, **hablar** y **cerrar**. El `with` de Python se encarga del cierre automático (paso 4) al salir del bloque.

🐍 El cliente mínimo resuelto

```python
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
    cliente.connect((HOST, PORT))
    cliente.sendall(b"Hola servidor!")
    respuesta = cliente.recv(1024)
    print(f"Respuesta: {respuesta.decode()}")
```

Salida (con el servidor del punto 3 escuchando en el puerto 5000):

```
Respuesta: Recibido: Hola servidor!
```

🧩 Cada método, al detalle

Método	Qué hace	¿Cuándo?
`socket(AF_INET, SOCK_STREAM)`	Crea el socket TCP	Siempre, lo primero
`connect((HOST, PORT))`	Establece la conexión (three-way handshake)	Antes de hablar
`sendall(datos)`	Envía **todos** los bytes	Cuando quieras mandar
`recv(1024)`	Recibe hasta 1024 bytes	Cuando esperes respuesta
`close()` / `with`	Libera la conexión	Al terminar

- **`connect()`** lanza `ConnectionRefusedError` si no hay nadie escuchando en ese puerto. Es la forma de saber que el servidor no está encendido.
- **`sendall()`** se llama *all* porque garantiza que se envían **todos** los bytes (a diferencia de `send()`, que puede enviar solo una parte y tendrías que reintentar).
- **`recv(1024)`** no significa que *solo* puedas recibir 1024 bytes: es el **tamaño máximo del buffer**. Si el mensaje es más grande, necesitas varios `recv()` y concatenar. Tú tienes que implementar el protocolo de aplicación para saber cuándo has recibido el mensaje completo.
- El **`with`** cierra el socket automáticamente al salir: si lo usas, no necesitas `close()` explícito.

🎭 Be the code, my friend — Cliente TCP

> "Sé el cliente. Cada paso, cada byte."

```
🔵 CLIENTE
1. socket(AF_INET, SOCK_STREAM) → cli
2. connect(("127.0.0.1", 5000))
   → Three-way handshake TCP
3. sendall(b"Hola servidor!")
4. Esperando respuesta... ⏳

🔵 CLIENTE
4. recv(1024) → b"Recibido: Hola servidor!"
5. print("Respuesta: Recibido: Hola servidor!")
6. close()
```

> ⚠️ **`recv()` es bloqueante**: el programa se queda esperando hasta que lleguen datos. Si el servidor nunca responde, te quedas colgado ahí (para salir del apuro, verás los **timeouts** en el punto 4).

🧠 Mini-chequeo

1. ¿Qué método establece la conexión con el servidor y qué error lanza si no hay nadie escuchando?
2. ¿Por qué usar `sendall()` y no `send()`?
3. ¿Qué hace `recv(1024)` y por qué no es un límite absoluto?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- El cliente TCP sigue `socket()` → `connect()` → `send()`/`recv()` → `close()`.
- `connect()` estrecha la mano, `sendall()` envía todo y `recv(1024)` lee hasta 1024 bytes.
- El `with` cierra el socket solo; si el servidor no existe, `connect()` lanza `ConnectionRefusedError`.

🐛 Vocabulario rápido

Término	Idea general
connect()	Establece la conexión TCP con el servidor
sendall()	Envía todos los bytes de golpe
recv()	Recibe datos; bloqueante hasta que llegan
Bloqueante	La llamada se queda esperando hasta que algo ocurra
ConnectionRefusedError	No hay nadie escuchando en ese puerto

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es un socket · **Siguiente:** 03 · Servidor TCP

03 — Servidor TCP

bind, listen y accept: el que escucha y atiende 🔔

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 03 · Servidor TCP

📬 La idea en una frase

> El servidor TCP **descolgó primero**: reserva un puerto con `bind()`, avisa con `listen()` y se queda esperando con `accept()` a que un cliente llame. Todo el peso de la conversación recae sobre él.

En el punto 2 el cliente marcaba el número. Este punto es la otra cara de la moneda: el servidor que **escucha**. Sin `bind()` no tiene puerto; sin `listen()` no anuncia que espera llamadas; sin `accept()` no recibe a nadie.

🔄 Ciclo de vida del servidor

```
socket() → bind() → listen() → accept() → recv()/send() → close()
```

Seis pasos. Los tres primeros (`socket`, `bind`, `listen`) son el “descolgar el teléfono”; `accept()` es el “¿diga?” que se queda esperando a que suene el timbre.

🐍 El servidor mínimo resuelto

```python
import socket

HOST = "127.0.0.1"  # localhost
PORT = 5000

# 1. Crear socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:

    # 2. Asignar dirección y puerto
    servidor.bind((HOST, PORT))

    # 3. Escuchar conexiones entrantes
    servidor.listen()
    print(f"Servidor escuchando en {HOST}:{PORT}")

    # 4. Aceptar una conexión (BLOQUEANTE)
    conexion, direccion = servidor.accept()
    print(f"Conectado con {direccion}")

    with conexion:
        # 5. Recibir datos
        datos = conexion.recv(1024)
        print(f"Recibido: {datos.decode()}")

        # 6. Enviar respuesta
        conexion.sendall(b"Recibido: " + datos)

    # 7. Se cierra solo al salir del with
```

Salida (tras ejecutar el cliente del punto 2 en otra terminal):

```
Servidor escuchando en 127.0.0.1:5000
Conectado con ('127.0.0.1', 54321)
Recibido: Hola servidor!
```

🧩 Cada método, al detalle

Método	Qué hace	¿Cuándo?
`bind((HOST, PORT))`	Reserva la dirección y el puerto	Antes de escuchar
`listen()`	Prepara el socket para recibir conexiones	Tras el bind
`accept()`	Espera un cliente y devuelve `(conexion, direccion)`	En bucle
`recv(1024)`	Recibe datos del cliente conectado	Cuando llegan
`sendall(datos)`	Envía la respuesta	Cuando hay que contestar

- **`bind()`** es la única forma de "poseer" un puerto: sin él, el socket no escucha en ningún sitio. Para el servidor es **obligatorio**; para el cliente, el SO asigna uno automáticamente.
- **`listen()`** avisa al SO: "prepárate, voy a aceptar llamadas entrantes". Sin ella, no hay puerto abierto y el cliente recibe `ConnectionRefusedError`.
- **`accept()`** es **bloqueante**: el servidor se queda esperando hasta que un cliente llama. Devuelve **dos cosas**: el socket de la conversación (`conexion`) y la dirección del cliente (`direccion`, una tupla `(IP, puerto)`).
- Fíjate: a partir de `accept()`, todo se hace con **`conexion`**, no con `servidor`. `servidor` solo escucha; `conexion` es el canal de esa llamada concreta.

🔁 El bucle de atención

El servidor de arriba atiende **un solo cliente** y termina. Para atender a varios, se envuelve el `accept()` en un bucle:

```python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
    servidor.bind(("127.0.0.1", 5000))
    servidor.listen()
    print("Servidor escuchando...")

    while True:
        conexion, direccion = servidor.accept()
        print(f"Conectado con {direccion}")
        with conexion:
            datos = conexion.recv(1024)
            print(f"Recibido: {datos.decode()}")
            conexion.sendall(b"Recibido: " + datos)
```

> ⚠️ Con este código básico, el servidor atiende **un cliente cada vez**: si llega un segundo mientras el primero habla, se queda en la cola de `listen()`. Para atender varios a la vez necesitas hilos o `select()`: eso es la **UD 6 · Servidores concurrentes**.

🧩 Pool Puzzle — El ciclo de vida del socket

Las líneas de abajo están desordenadas. ¿Puedes ordenarlas para que formen un servidor TCP funcional?

```
a) with socket.socket() as srv:
b)     srv.listen()
c)     conn, addr = srv.accept()
d)     import socket
e)         conn.sendall(b"Bienvenido\n")
f)     srv.bind(("127.0.0.1", 5000))
g)     with conn:
h)         datos = conn.recv(1024)
```

> › 🔓 Solución

🧠 Mini-chequeo

1. ¿Qué tres métodos preparan el servidor antes de atender a nadie?

2. ¿Qué devuelve `accept()` y por qué es bloqueante?
3. Tras `accept()`, ¿con qué objeto se recibe y envía: `servidor` o `conexion`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- El servidor TCP sigue `socket()` → `bind()` → `listen()` → `accept()` → `recv()/send()` → `close()`.
- `bind()` reserva el puerto, `listen()` anuncia que espera llamadas y `accept()` bloquea hasta que un cliente llama.
- Para atender a varios clientes hay que envolver `accept()` en un bucle (y más adelante, usar hilos o `select()`).

🐛 Vocabulario rápido

Término	Idea general
bind()	Reserva la dirección y el puerto del servidor
listen()	Prepara el socket para conexiones entrantes
accept()	Acepta un cliente; bloqueante, devuelve (conexión, dirección)
conexion	El canal de la conversación con ese cliente
Bloqueante	La llamada espera hasta que algo ocurra

📚 Volver al índice de la unidad · **Anterior:** 02 · Cliente TCP · **Siguiente:** 04 · Ciclo de vida y errores

04 — Ciclo de vida y errores

Handshake, cierre, TIME_WAIT, reuseaddr y timeouts bajo control 🤝

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 04 · Ciclo de vida y errores

---

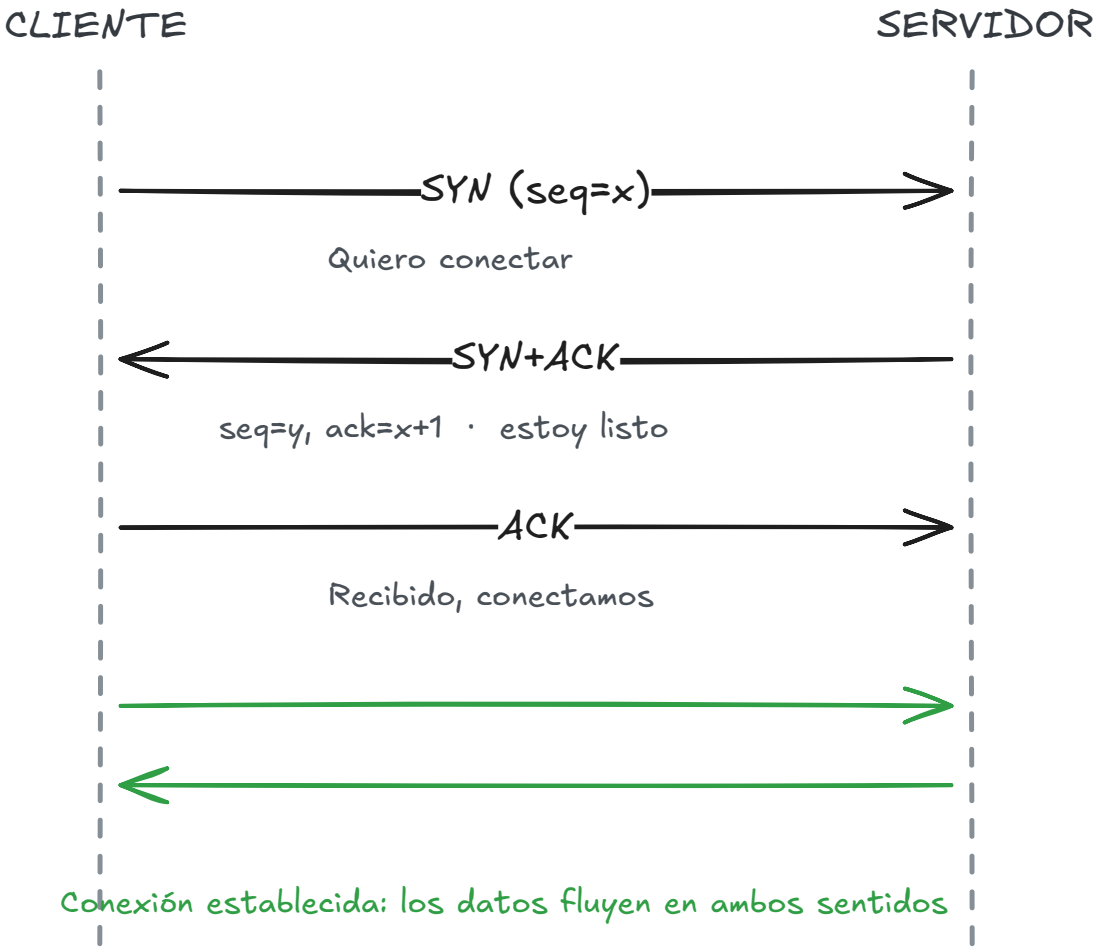
📬 La idea en una frase

> Una conexión TCP **nace, vive y muere**: arranca con un **three-way handshake**, transporta los datos y se cierra con un apretón de despedida; y cuando las redes fallan, tu programa debe saber **capturar el error y no quedarse bloqueado para siempre**.

En los puntos 2 y 3 llamaste a `connect()` y `accept()` sin saber qué pasaba por dentro. Aquí levantas el capó, ves el protocolo de transporte, entiendes por qué relanzar un servidor a veces falla… y aprendes a ponerte a salvo de los fallos de red.

🤝 Three-way handshake (establecer conexión)

Three-way handshake de TCP



1. **SYN** (synchronize): el cliente dice *"quiero hablar contigo"*.
2. **SYN + ACK**: el servidor contesta *"de acuerdo, y yo también quiero hablar contigo"*.
3. **ACK** (acknowledge): el cliente confirma *"recibido, hablemos"*.

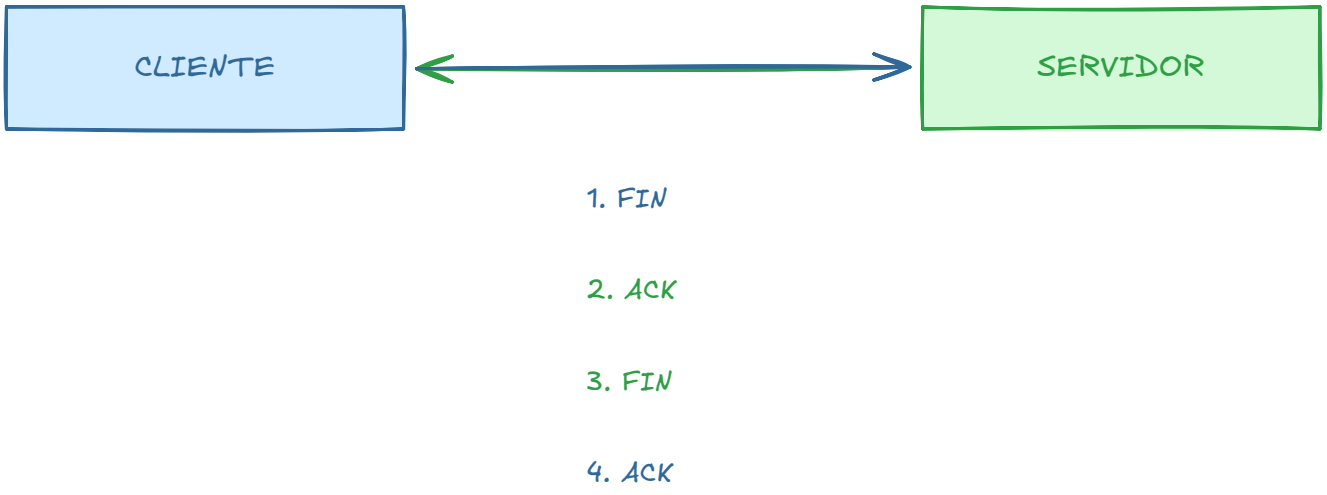
A partir de ahí, **los datos fluyen en ambas direcciones**. Si cuentas los mensajes (3), tienes el nombre: **three-way handshake**. Es exactamente lo que ocurre dentro de tu `cliente.connect()`.

TCP garantiza que los datos lleguen **en orden** y **sin pérdidas**. A cambio, es un poco más lento que UDP: ese "orientado a conexión" significa que antes de enviar un solo byte, ambas partes se ponen de acuerdo con este saludo previo.

👋 Cierre de la conexión: la despedida

Cuando ya no hay más que decir, la conexión se cierra con un apretón de despedida:

La despedida TCP: FIN → ACK → FIN → ACK



apretón de despedida en ambos sentidos: cada lado anuncia que termina y el otro lo confirma

1. Quien quiere cerrar envía **FIN** (*"no tengo más que enviar"*).
2. El otro lado responde **ACK** y, cuando también termina, envía su propio **FIN**.
3. El primero confirma con **ACK** y la conexión se libera.

> 💡 En Python no gestionas el cierre a mano: cuando el `with` termina (o llamas a `close()`), el SO ejecuta esta despedida por ti. Aunque lo parezca, **no es instantáneo**: el estado de cierre queda unos segundos en el sistema (TIME_WAIT, el problema del apartado siguiente).

😡 TIME_WAIT y "Address already in use"

```
# 1ª ejecución del servidor: todo bien
srv.bind(("127.0.0.1", 5000))     # ✅ OK

# Matas el servidor con Ctrl+C y lo relanzas al instante...
srv.bind(("127.0.0.1", 5000))     # 💥 OSError: Address already in use
```

¿Por qué? Tras el cierre, la conexión entra en estado **TIME_WAIT** durante un tiempo breve (típicamente 1-2 minutos, 2×MSL): el SO mantiene reservado el par (IP, puerto) para asegurarse de que los últimos FIN/ACK no queden huérfanos. Mientras tanto, nadie más puede hacer `bind()` a ese puerto.

No es un capricho: la despedida TCP necesita tiempo para que los mensajes de cierre no lleguen tarde y revuelvan conexiones nuevas. Es **protección**, no burocracia. El problema es que,

para un programador que reinicia su servidor mil veces al día, esa protección se convierte en un estorbo… hasta que aparece `SO_REUSEADDR`.

La solución: `SO_REUSEADDR`

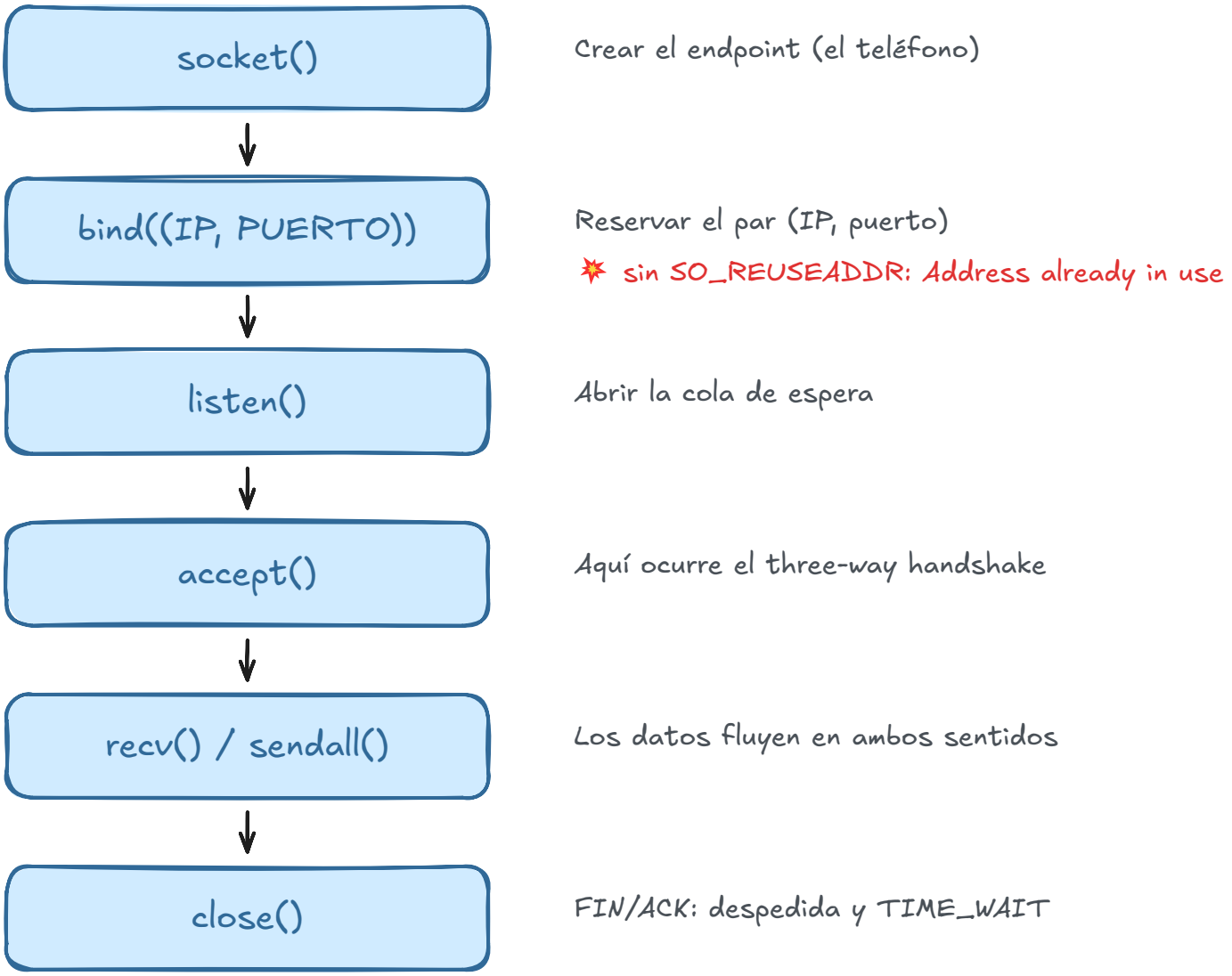
```
import socket

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 5000))
servidor.listen()
```

Pieza	Significado
`SOL_SOCKET`	“Nivel de opciones” del propio socket
`SO_REUSEADDR`	La opción que permite reutilizar la dirección
`1`	Activa la opción (0 la desactivaría)

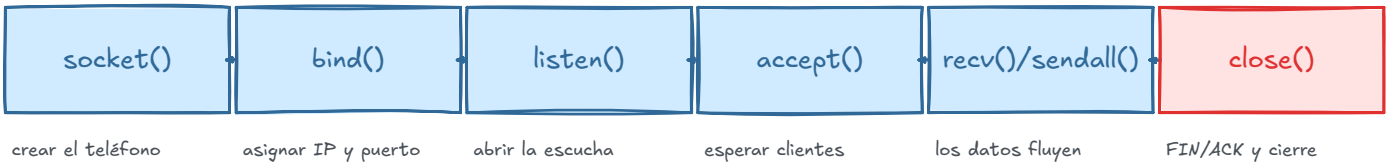
Con ella, el SO te deja hacer `bind()` al puerto aunque queden conexiones en `TIME_WAIT`. El servidor se puede **matar y reiniciar** sin esperar. **Pon esto siempre** en tus servidores: te ahorrará minutos de depuración.

📸 El ciclo completo, en una mirada

Ciclo de vida de una conexión TCP



El ciclo de vida en el servidor TCP



accept() se repite: por cada cliente, un ciclo recv()/sendall() y su close()

El **ciclo de vida de la conexión** es la historia completa entre la primera `socket()` y el último `close()`: preparar el teléfono, estrechar la mano, hablar y colgar.

🛡️ Las redes fallan: `try/except` y timeouts

En los puntos 2 y 3 todo iba bien. En la vida real el servidor se cae, la red se corta o el mensaje tarda demasiado. Tu programa necesita **red de seguridad**.

```
import socket, time

def conectar_seguro(host, port, reintentos=3):
    for intento in range(reintentos):
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(5)
                s.connect((host, port))
                s.sendall(b"test")
                return s.recv(1024)
        except socket.timeout:
            print(f"⏱️ Timeout (intento {intento+1})")
        except ConnectionRefusedError:
            print(f"🚫 Conexión rechazada — ¿el servidor está encendido?")
            time.sleep(1)
        except ConnectionResetError:
            print(f"💥 El servidor cerró la conexión abruptamente")
        except OSError as e:
            print(f"🔌 Error de red: {e}")
    return None
```

- **`for intento in range(reintentos)`** → reintenta hasta 3 veces antes de rendirse.
- **`s.settimeout(5)`** → si una operación tarda más de 5 segundos, lanza `socket.timeout` en lugar de quedarse bloqueada para siempre.
- **`try/except` por excepción** → cada error tiene su propio tratamiento.
- **`return None`** al final → si tras 3 intentos no hay respuesta, el llamador sabe que falló.

El mapa de excepciones

Excepción	Cuándo ocurre
`socket.timeout`	La operación excede el tiempo límite
`ConnectionRefusedError`	No hay nadie escuchando en ese puerto

Excepción	Cuándo ocurre
`ConnectionResetError`	El otro lado cerró de golpe
`ConnectionAbortedError`	El SO local aborta la conexión
`BrokenPipeError`	Escribes en un socket que ya se cerró
`OSError`	Red caída, DNS no resuelve, etc.

> 💡 **Regla práctica:** primero captura las excepciones específicas (`socket.timeout`, `ConnectionResetError`, `BrokenPipeError`) y deja `OSError` como última red.

Non-blocking y `select`

`recv()` y `accept()` son bloqueantes: se quedan esperando. Tres formas de controlar esa espera:

```
import socket, select

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Opción 1: timeout fijo
s.settimeout(5.0)

# Opción 2: no bloqueante (lanza excepción si no hay datos)
s.setblocking(False)

# Opción 3: select (esperar con timeout en múltiples sockets)
lectura, _, _ = select.select([s], [], [], 1.0)
if lectura:
    datos = s.recv(1024)
```

> `select.select()` es la solución para esperar en **varios sockets a la vez sin hilos**: le pasas la lista y te avisa cuáles están listos. Lo usarás a fondo en la UD 6 · Servidores concurrentes.

🧠 Mini-chequeo

1. ¿Cuáles son los tres mensajes del handshake y en qué orden?

2. ¿Qué error lanza el SO al relanzar un servidor sin `SO_REUSEADDR` y qué estado lo causa?
3. ¿Qué excepción lanza `recv()` si se pasa el timeout y qué excepción si escribes en un socket ya cerrado?
4. ¿Cuál es la diferencia entre `ConnectionRefusedError` y `ConnectionResetError`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Una conexión TCP arranca con el **three-way handshake** (SYN → SYN+ACK → ACK), transporta los datos y se cierra con **FIN/ACK**; tras el cierre, TIME_WAIT retiene el puerto un tiempo breve.
- **`setsockopt(SO_REUSEADDR, 1)`** antes del `bind()` evita el "Address already in use" al reiniciar servidores.
- Las redes fallan: captura cada error con su propio `except`, usa `settimeout()` para no bloquearte y `select()` para esperar en varios sockets sin hilos.

🐛 Vocabulario rápido

Término	Idea general
Handshake	Saludo previo de TCP antes de enviar datos
SYN / ACK / FIN	Mensajes de apertura, confirmación y cierre
TIME_WAIT	Estado que retiene el puerto tras cerrar (1-2 min)
SO_REUSEADDR	Permite reutilizar la dirección y el puerto
Timeout	Tiempo máximo de espera de una operación
ConnectionRefusedError	No hay nadie escuchando en el puerto
ConnectionResetError	El otro lado cerró la conexión de golpe
select()	Espera en varios sockets a la vez sin hilos

📚 Volver al índice de la unidad · **Anterior:** 03 · Servidor TCP · **Siguiente:** 05 · Cliente y servidor UDP

05 — Cliente y servidor UDP

sendto, recvfrom y la vida real de los datagramas 📡

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 05 · Cliente y servidor UDP

📬 La idea en una frase

> UDP **no se conecta**: el cliente manda un datagrama con `sendto()` a una dirección y espera con `recvfrom()`; el servidor solo hace `bind()` y escucha. **Sin `connect()`, sin `accept()`, sin `listen()`…** y sin ninguna garantía de que el mensaje llegue.

Viste en el punto 1 la tabla del "No": UDP no garantiza conexión, entrega ni orden. Aquí lo compruebas en la API, construyes tu primer cliente y servidor, y aterrizas las consecuencias de lanzar aviones de papel.

📤 El cliente UDP: `sendto()` y `recvfrom()`

```python
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(b"Hola UDP!", (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

Línea	Qué hace
`socket(AF_INET, SOCK_DGRAM)`	Crea el socket UDP (el `SOCK_DGRAM` del punto 1)
`sendto(b"Hola UDP!", (HOST, PORT))`	Envía el datagrama: **la dirección viaja en la llamada**, no en un `connect()` previo

Línea	Qué hace
`recvfrom(1024)`	Espera la respuesta y devuelve la tupla `(datos, dirección)`; aquí la dirección se descarta con _

Sendto contra send: la diferencia visual

Cliente TCP (puntos 2-3)	Cliente UDP (aquí)
`s.connect((HOST, PORT))`	— (no existe)
`s.send(datos)`	`s.sendto(datos, (HOST, PORT))`
`s.recv(1024)`	`s.recvfrom(1024)`

En TCP primero te conectas y luego envías; en UDP el “a quién” va dentro del envío.

> ⚠️ Cada `sendto()` es un datagrama independiente. Pueden llegar **desordenados, duplicados o no llegar**. Si el servidor no está escuchando, el datagrama se pierde en el vacío y el `recvfrom()` se queda esperando **para siempre** (a menos que pongas un `settimeout()`).

📥 El servidor UDP: `bind()` y `recvfrom()`

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor UDP en {HOST}:{PORT}")

    # Recibir datagrama (recvfrom devuelve datos + dirección)
    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")

    # Responder
    servidor.sendto(b"Recibido!", direccion)
```

- **`bind((HOST, PORT))`** → reserva el puerto: cualquier datagrama que llegue a `127.0.0.1:5001` es para este proceso. Sin `bind()`, el socket no escucha en ningún sitio.
- **`datos, direccion = servidor.recvfrom(1024)`** → se bloquea esperando un datagrama. Cuando llega, devuelve **el contenido y la dirección del cliente** `(IP, puerto)`.
- **`servidor.sendto(b"Recibido!", direccion)`** → responde usando esa dirección: la única forma de "devolver la pelota" en UDP, porque no hay conexión que recuerde quién eras.

Sin accept(), sin listen(): la diferencia con TCP

Servidor TCP (punto 3)	Servidor UDP (aquí)
`bind()`	`bind()`
`listen()`	— (no existe)
`accept()` → devuelve `conn`	— (no existe)
`conn.recv(1024)`	`servidor.recvfrom(1024)`
(la conexión sabe quién es)	`recvfrom()` devuelve la dirección

> 💡 Consecuencia importante: un solo servidor UDP puede atender a **cualquier número de clientes** sin `accept()` ni hilos, porque cada mensaje trae su dirección pegada.

💨 Datagramas: cada envío es un mundo aparte

Imagina que mandas 5 mensajes seguidos desde un cliente UDP:

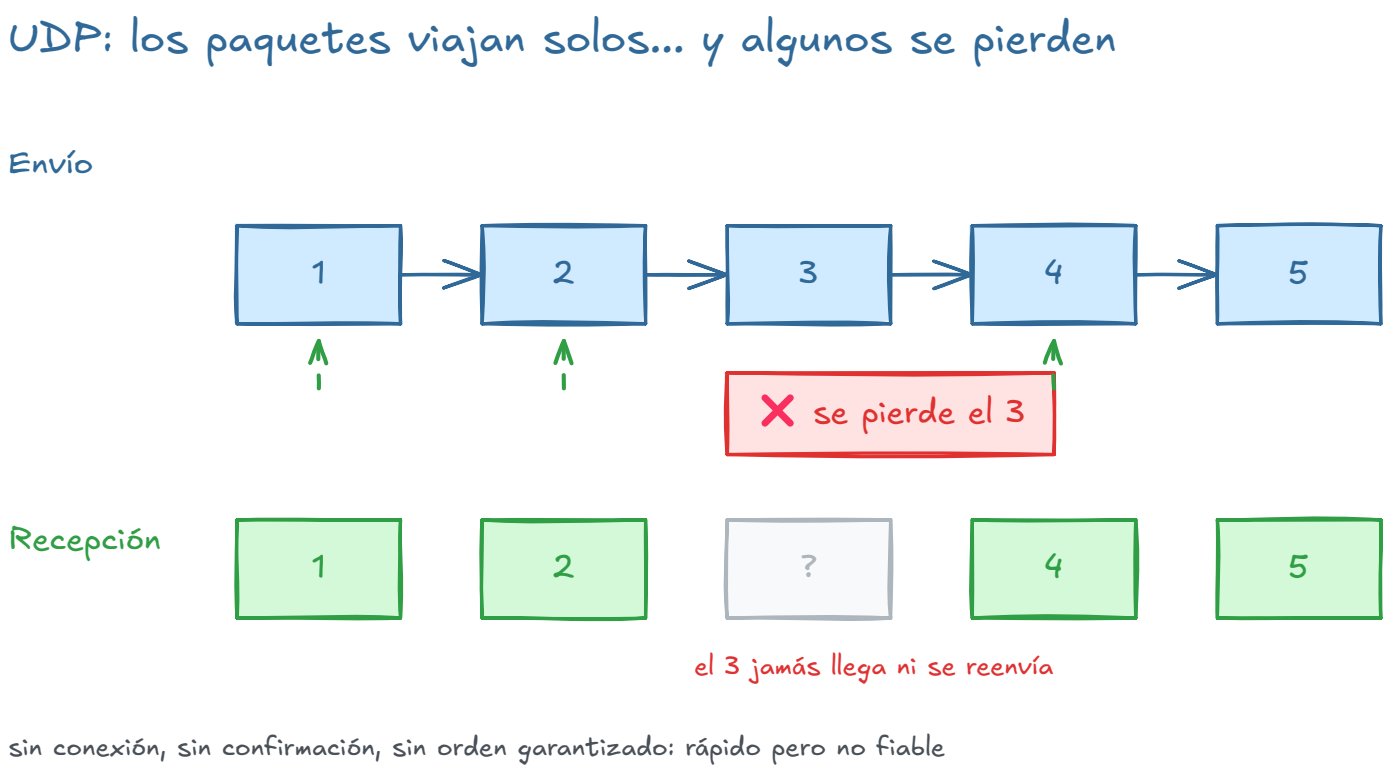
```
import socket, time

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    for i in range(1, 6):
        cliente.sendto(f"Mensaje #{i}".encode(), ("127.0.0.1", 5001))
        time.sleep(0.2)
```

El servidor podría recibirlos en **cualquier orden** (o no recibir alguno). La red no promete nada: cada datagrama viaja por su cuenta, como aviones de papel lanzados desde el mismo balcón pero que el viento puede separar.

> 💡 **Ojo con localhost:** en `127.0.0.1` (loopback) esto casi nunca se manifiesta: los datagramas van de un socket a otro en la misma máquina y llegan en orden y sin pérdidas. El desorden y la pérdida se ven en **redes reales** (con routers, congestión y caminos distintos).

Los males de UDP

1. **Pérdida de paquetes** — No hay confirmación de recepción. Si un paquete se pierde, **se pierde para siempre**: UDP no reenvía nada.
2. **Orden no garantizado** — Los datagramas pueden llegar en distinto orden. Sin números de secuencia, el receptor no tiene cómo reordenarlos.
3. **Duplicados** — En condiciones raras, el mismo datagrama puede llegar **dos veces**. UDP no lo detecta.
4. **Tamaño máximo** — Un datagrama no puede ser gigante: la carga útil útil se limita a **65.535 − 8 (UDP) − 20 (IPv4) ≈ 65.507 bytes**. Si intentas enviar más, `sendto()` lanza un `OSError`.



¿Por qué alguien elegiría esto?

Porque esos males son el **precio de la velocidad**: sin handshake, sin confirmaciones ni reenvíos, cada paquete cuesta el mínimo. Y hay aplicaciones donde perder un paquete molesta menos que esperar por él: una videollamada, un juego online, DNS. La regla de decisión completa la tienes en el punto 7.

🧠 Mini-chequeo

1. ¿Qué dos métodos usa el cliente UDP y qué devuelve cada uno?
2. ¿Por qué el servidor UDP no necesita `accept()`?
3. ¿Qué tres cosas pueden pasarle a datagrama en la red y qué pasa si se pierde?
4. ¿Cuál es el tamaño máximo de un datagrama UDP?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El cliente UDP manda datagramas con `sendto()` y recibe con `recvfrom()`, sin `connect()`; el servidor solo hace `bind()` y escucha, sin `listen()` ni `accept()`.
- `recvfrom()` devuelve datos **y** dirección del emisor: con esa tupla se responde con `sendto()`, y así un mismo servidor atiende a cualquier número de clientes.
- Cada datagrama es un mundo aparte: puede perderse, duplicarse o llegar desordenado, con un límite de ~65.507 bytes; es el precio de la velocidad.

🐛 Vocabulario rápido

Término	Idea general
sendto()	Envía un datagrama con su dirección destino
recvfrom()	Recibe un datagrama: datos + dirección del emisor
SOCK_DGRAM	Tipo de socket UDP
Datagrama	Paquete independiente de UDP (máx. ~65.507 bytes)
Sin conexión	No hay handshake: mandas y punto
Sin confirmación	UDP no avisa de la recepción (a diferencia de TCP)

📚 Volver al índice de la unidad · **Anterior:** 04 · Ciclo de vida y errores · **Siguiente:** 06 · HTTP y NTP

06 — HTTP y NTP

HTTP hablado a pelo con un socket y la hora de Internet por UDP 🌐

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 06 · HTTP y NTP

📬 La idea en una frase

> TCP y UDP son solo la **tubería**; los protocolos de aplicación deciden **qué** se escribe dentro. Aquí hablas con tus propios ojos **HTTP** (el protocolo que mueve la web, sobre TCP) y **NTP** (el reloj de Internet, sobre UDP).

🌐 HTTP desde cero: el protocolo que mueve la web

HTTP es un protocolo de **texto**: mandas una petición en texto claro y recibes una respuesta también en texto. Cuando tu navegador quiere una página, manda un texto como este:

```
GET /ruta HTTP/1.1\r\n
Host: ejemplo.com\r\n
User-Agent: MiNavegador\r\n
Accept: text/html\r\n
\r\n
```

Línea	Qué hace
`GET /ruta HTTP/1.1`	Primera línea: **verbo**, recurso y versión
`Host: ejemplo.com`	Obligatoria en HTTP/1.1: a qué servidor hablamos
`User-Agent`, `Accept`	Cabeceras opcionales que describen quién pregunta y qué quiere
`\r\n\r\n` (línea en blanco)	Marca el **final de las cabeceras**: "ya estaba todo"

Los **verbos** principales:

Verbo	Qué pide	Ejemplo
GET	Leer un recurso	Obtener una página o imagen
POST	Enviar datos al servidor	Mandar un formulario
PUT	Guardar/reemplazar un recurso	Subir un archivo
DELETE	Borrar un recurso	Eliminar algo del servidor

La respuesta

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 1234\r\n
\r\n
<html>...cuerpo...</html>
```

- **`HTTP/1.1 200 OK`** → versión, **código de estado** y significado. `200` es “todo bien”; también verás `404 Not Found`, `500 Internal Server Error`.
- **`Content-Type`** → qué tipo de contenido viene a continuación.
- **`Content-Length`** → cuántos bytes mide el cuerpo, para saber cuándo se acabó.
- **`\r\n\r\n`** → separa cabeceras de **cuerpo** (el HTML en sí).

El cliente HTTP manual

```python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("www.example.com", 80))
    s.sendall(b"GET / HTTP/1.1\r\nHost: www.example.com\r\nConnection: close\r\n\r\n")

    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

print(respuesta.decode()[:500])
```

Dos detalles importantes:

- **`Connection: close`** → HTTP/1.1 mantiene la conexión abierta por defecto (*keep-alive*). Pedir `Connection: close` fuerza al servidor a cerrarla al terminar, y así el bucle sabe cuándo parar (`recv()` devuelve `b""`).
- **Los bytes se acumulan** con `respuesta += datos`: la respuesta de una web llega **en trozos**, no de una pieza. El bucle los junta todos.

> 💡 Esto es la base de la [UD 7 · HTTP y APIs REST]: librerías como `requests` hacen exactamente esto por debajo, pero ahora sabes el truco.

🔢 El orden de los bytes (byte ordering)

Cuando envías números (un puerto, una longitud, un ID) por la red, la pregunta es **en qué orden van sus bytes**. Dos máquinas pueden guardar los enteros "al revés" (little-endian o big-endian). Para no liarse, la red usa **big-endian**, también llamado **network byte order**.

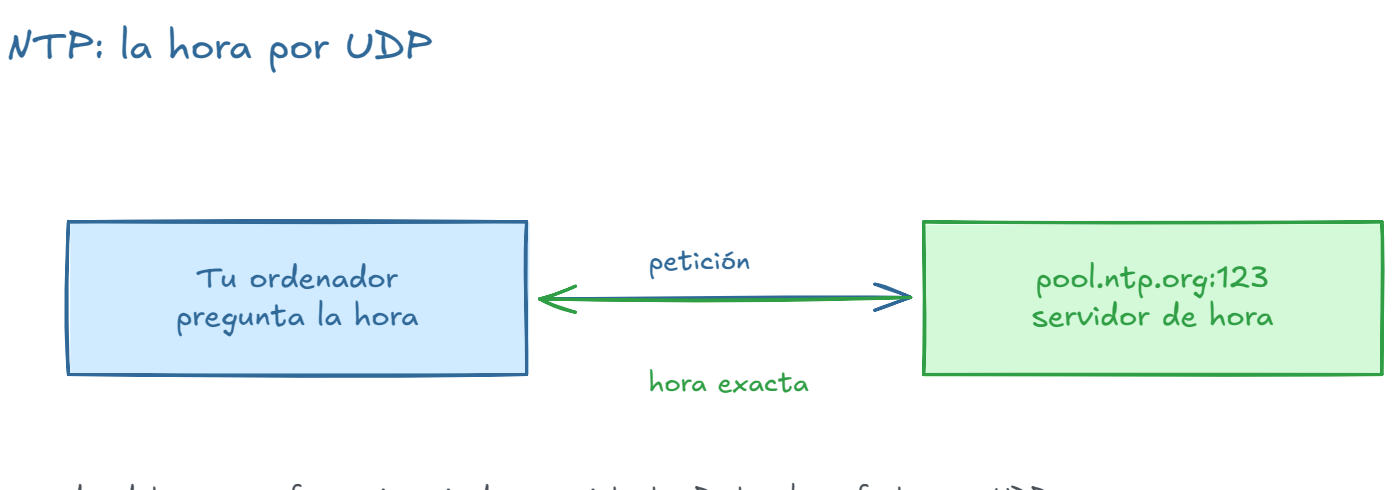
```python
import struct

# Empaquetar un entero en network byte order ('!' = big-endian)
datos = struct.pack("!I", 5000)      # el puerto 5000 como 4 bytes
print(datos)                          # b'\x00\x00\x13\x88'

# Desempaquetarlo de vuelta
puerto = struct.unpack("!I", datos)[0]
print(puerto)                         # 5000
```

Código	Significado
`!`	**Network byte order** (big-endian)
`I`	Entero sin signo de 4 bytes
`H`	Entero sin signo de 2 bytes

> 💡 Los códigos `!I` y `!H` los verás en todos los protocolos que hablan números (NTP, DNS, TCP mismo). Este es el truco que necesitas para el NTP que viene ahora.

⏰ NTP: sincronizar relojes con UDP

NTP (Network Time Protocol) usa UDP para sincronizar relojes: tu ordenador manda un pequeño datagrama al puerto 123 de un servidor de tiempo y recibe la hora exacta en la respuesta. Curioso: justo cuando la exactitud importa, NTP usa el protocolo "que pierde paquetes".



- **`pool.ntp.org`** es un grupo de servidores de tiempo repartidos por el mundo; cualquier petición UDP al puerto **123** te devuelve la hora.
- NTP manda **múltiples peticiones** y cruza las respuestas para descontar la latencia y quedarse con la estimación más fiable.
- Por eso puede permitirse UDP: **un datagrama perdido es irrelevante** cuando manda muchos y promedia estadísticamente. Así tu ordenador sabe la hora sin tener un reloj atómico.

Cliente NTP manual en Python

El paquete NTP es un datagrama de **48 bytes** con un formato binario muy concreto:

```
import socket, struct, time

def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        # Paquete NTP: 48 bytes, modo cliente
        paquete = b'\x1b' + 47 * b'\0'
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)

    # El timestamp está en los bytes 40-43
    t = struct.unpack('!I', datos[40:44])[0]
    # Ajustar época NTP (1900) a Unix (1970)
    return t - 2208988800

hora = hora_ntp()
print(f"Hora NTP oficial: {time.ctime(hora)}")
```

- **`paquete = b'\x1b' + 47 * b'\0'`** → el primer byte `00011011` pone el **modo cliente** y la **versión NTP v3**; los 47 bytes restantes son relleno del formato de 48 bytes.
- **`s.settimeout(5)`** → muy importante en UDP: si la respuesta no llega en 5 segundos, lanza una excepción en lugar de bloquearse para siempre.
- **`struct.unpack('!I', datos[40:44])`** → el servidor escribe su hora como entero de 4 bytes en **network byte order** (el `!` del byte ordering) en las posiciones 40-43.
- **`t - 2208988800`** → el timestamp NTP cuenta desde **1900**; Unix desde **1970**. La diferencia exacta ajusta la época.

> **SNTP** (Simple Network Time Protocol) es la versión ligera de NTP para routers domésticos, móviles e IoT: para el 99% de las aplicaciones es más que suficiente.

🧠 Mini-chequeo

1. ¿Qué significa la línea en blanco `\r\n\r\n` en una petición HTTP y qué indica el código `200`?
2. ¿Qué código de `struct` usa big-endian y por qué lo necesita el cliente NTP?
3. ¿Por qué NTP usa UDP si la hora debe ser exacta?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- **HTTP es texto sobre TCP**: una petición GET con cabeceras, leída en bucle con `recv()` hasta el cierre; los verbos (GET, POST, PUT, DELETE) y códigos (200, 404) son la base de las APIs de la UD 7.
- Los números viajan en **network byte order (big-endian)**, que en Python se fuerza con el prefijo `!` de `struct`.
- **NTP** sincroniza relojes con un datagrama UDP de 48 bytes al puerto 123: puede permitirse UDP porque promedia muchas peticiones y `settimeout()` evita bloqueos eternos.

🐛 Vocabulario rápido

Término	Idea general
HTTP	Protocolo de texto sobre TCP que mueve la web
GET / POST / PUT / DELETE	Verbos HTTP para leer, enviar, guardar y borrar
200 OK	Código de estado: la petición fue bien
\r\n\r\n	Línea en blanco que separa cabeceras de cuerpo
Network byte order	Orden estándar de bytes en la red (big-endian)
NTP	Protocolo que sincroniza relojes con UDP (puerto 123)

Término	Idea general
pool.ntp.org	Grupo mundial de servidores de tiempo

📚 Volver al índice de la unidad · **Anterior:** 05 · Cliente y servidor UDP · **Siguiente:** 07 · Cuándo usar cada protocolo

07 — Cuándo usar cada protocolo

Criterios de decisión y casos reales: DNS, VoIP, streaming 🧭

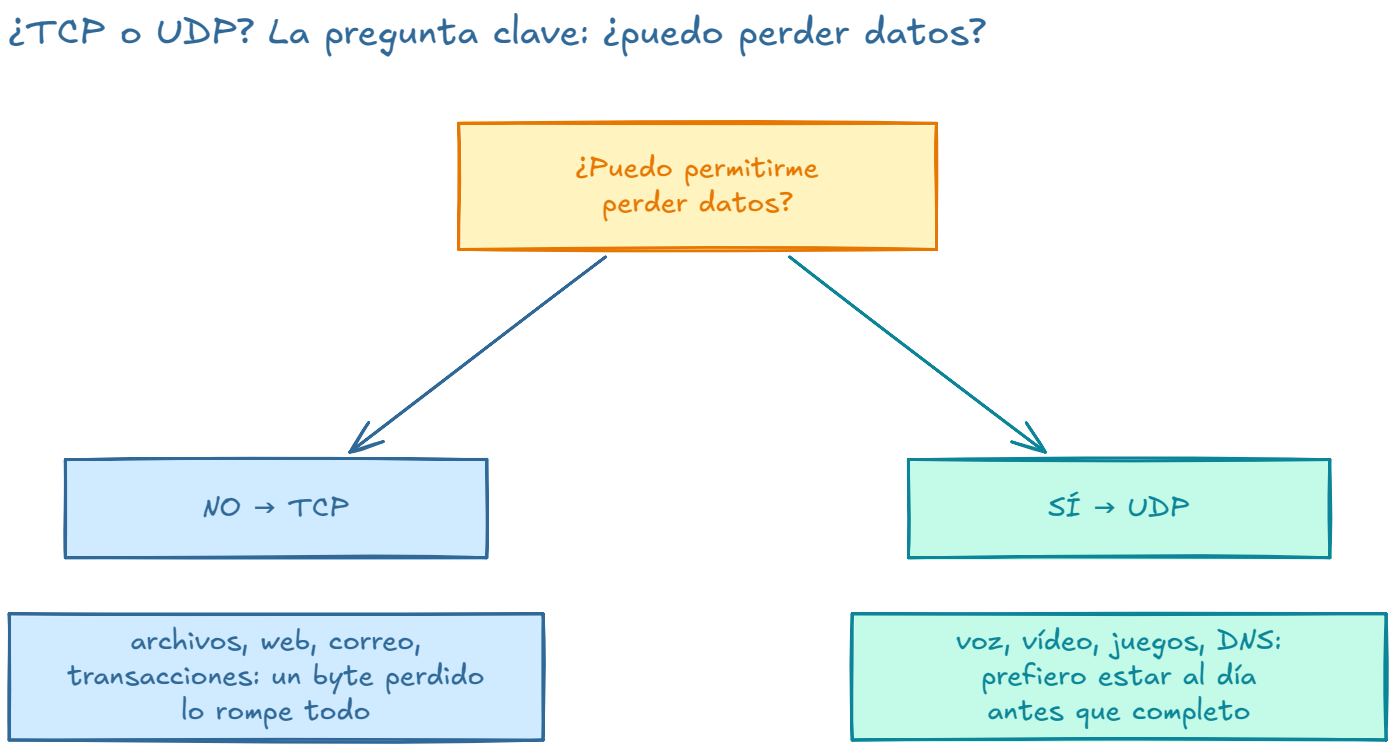
> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 07 · Cuándo usar cada protocolo

📬 La idea en una frase

> No existe “el mejor protocolo”: existe la pregunta correcta. **¿Prefieres que llegue todo, o prefieres que llegue rápido?** Según la respuesta, TCP o UDP.

Este punto junta todo lo anterior (la tabla del punto 1 y los males de los datagramas del punto 5) para darte la regla de decisión que usarás de por vida como programador.

🧭 El criterio: velocidad o fiabilidad



Dos preguntas que te ayudan a decidir:

1. **¿Es un dato “completo” o un “flujo en vivo”?** Un archivo debe llegar entero (TCP); una conversación debe llegar fluida (UDP).
2. **¿Qué molesta más: que falte un trozo o que todo vaya lento?** Si el trozo faltante es fatal, TCP; si el retraso es fatal, UDP.

📡 Casos reales: la tabla que se repite en las entrevistas

Aplicación	Protocolo	Por qué
Web (HTTP)	**TCP**	La página debe llegar completa y en orden
Correo (SMTP, POP3, IMAP)	**TCP**	Un mensaje a medias es inútil
Transferencia de archivos (FTP)	**TCP**	Un byte perdido corrompe el archivo
DNS	**UDP**	La consulta es diminuta; si se pierde, se repite
VoIP / videollamadas (Zoom, Skype)	**UDP**	La conversación debe ir fluida; un frame perdido se tolera
Streaming en vivo no-HTTP (RTP)	**UDP**	Prefieres el directo fluido a un reenvío que lo congele
Juegos online (Fortnite, Minecraft)	**UDP**	Cientos de paquetes por segundo; la latencia manda
NTP (hora)	**UDP**	Manda muchas peticiones y promedia (punto 6)

Fíjate en el patrón: **cuando el dato es valioso y debe quedar intacto, TCP; cuando lo valioso es el momento, UDP**. Ojo: el *streaming web* moderno (Twitch, YouTube) no usa UDP, sino HTTP sobre TCP (segmentación HLS/DASH) porque busca llegar a todo el mundo por infraestructura web.

🔀 Y si quieres ambas cosas: QUIC

¿Fiabilidad de TCP con velocidad de UDP? Eso existe: **QUIC**, el protocolo sobre el que funciona **HTTP/3**. QUIC se construye sobre UDP (la capa de transporte es UDP) pero añade por su cuenta confirmaciones, control de congestión y cifrado. Es la “vuelta a la tortilla”: la industria decide cuándo incluso el “no fiable” merece una capa de fiabilidad.

🧠 Mini-chequeo

1. ¿Qué criterio usarías para elegir entre TCP y UDP en una app nueva?
2. ¿Por qué DNS usa UDP si “perder” la respuesta de un dominio parece grave?
3. Pon dos ejemplos de TCP y dos de UDP distintos de los de la tabla.

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- La regla de decisión es una sola pregunta: ¿prefieres que llegue todo (TCP) o que llegue rápido (UDP)?
- Los casos reales son consistentes: web/correo/FTP con TCP; DNS/VoIP/streaming/juegos con UDP.
- Si quieres ambas cosas, QUIC (sobre HTTP/3) construye fiabilidad encima de UDP.

🦎 Vocabulario rápido

Término	Idea general
Criterio de decisión	¿Fiabilidad o velocidad? Eso elige el protocolo
DNS	Directorio de Internet; usa UDP por su tamaño diminuto
VoIP	Voz sobre IP; usa UDP para ir fluida
Streaming	Emisión en vivo; prefiere fluidez a completitud
QUIC	Fiabilidad tipo TCP construida sobre UDP

📚 Volver al índice de la unidad · **Anterior:** 06 · HTTP y NTP · **Siguiente:** 08 · Práctica eco

08 — Práctica eco

Servidor + cliente eco en TCP y UDP, y ejercicios resueltos 🛠️

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 08 · Práctica eco

📬 La idea en una frase

> La práctica clásica de sockets: un **servidor eco** que devuelve al cliente exactamente lo que recibe. Aquí la montas **dos veces**, una en TCP y otra en UDP, para ver con tus propias manos la diferencia entre canal fiable y datagramas sueltos.

Juntas todo lo aprendido: cliente y servidor TCP (puntos 2-3), `SO_REUSEADDR` y timeouts (punto 4), cliente y servidor UDP (punto 5). Con dos terminales verás cada protocolo completo en acción.

🛠️ Eco en TCP

```python
import socket

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    print("Servidor eco TCP escuchando en 127.0.0.1:9000")

    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        print(f"Recibido de {addr}: {datos.decode()}")
        conn.sendall(datos)  # Eco
```

El truco está en **`conn.sendall(datos)`**: devuelve los **mismos bytes** que llegaron. Eco, literalmente.

```python
import socket

with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Prueba")
    print(cli.recv(1024).decode())
```

Salida (con el servidor en ejecución en otra terminal):

```
Prueba
```

🎭 Be the code — Mano a mano TCP

```
🟢 SERVIDOR                          🔵 CLIENTE
1. socket(AF_INET, SOCK_STREAM)      1. socket(AF_INET, SOCK_STREAM)
2. bind(("127.0.0.1", 9000))         2. connect(("127.0.0.1", 9000))
3. listen()                             → Three-way handshake TCP
4. accept() → espera... ⏳

                                     3. sendall(b"Prueba")
5. accept() devuelve (conn, addr)    4. Esperando respuesta... ⏳
6. recv(1024) → b"Prueba"
7. sendall(b"Prueba")  ← eco

                                     5. recv(1024) → b"Prueba"
                                     6. close()
8. close()
```

> `accept()` y `recv()` son **bloqueantes**: el programa se queda esperando hasta que algo ocurra.

📡 Eco en UDP

```python
import socket

HOST = "127.0.0.1"
PORT = 9001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor eco UDP en {HOST}:{PORT}")

    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")

    # Devolver lo mismo que se ha recibido
    servidor.sendto(datos, direccion)
```

`servidor.sendto(datos, direccion)` devuelve los **mismos bytes** a la **misma dirección** de la que vinieron. El cliente:

```python
import socket

HOST = "127.0.0.1"
PORT = 9001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    mensaje = "Hola UDP, devuélveme esto"
    cliente.sendto(mensaje.encode(), (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"El servidor me devolvió: {datos.decode()}")
```

Salida del cliente:

```
El servidor me devolvió: Hola UDP, devuélveme esto
```

🎭 Be the code — Eco UDP paso a paso

```
1. Servidor: bind(("127.0.0.1", 9001))   → reserva el puerto 9001
2. Servidor: recvfrom(1024)              → se queda escuchando (bloqueado)
3. Cliente:  sendto(b"...", ("127.0.0.1", 9001))
             → el datagrama viaja a 127.0.0.1:9001
4. Servidor: recvfrom() despierta → (datos, direccion)
5. Servidor: sendto(datos, direccion)     → devuelve lo mismo al cliente
6. Cliente:  recvfrom() → recibe el eco y lo imprime
```

La línea 4 es la clave: la **dirección del cliente** aparece por primera vez cuando el servidor la necesita para responder. Ese es el flujo completo de UDP: fuego, escucha, responde.

⚖️ Las dos versiones, cara a cara

	Eco TCP	Eco UDP
Puerto de prueba	9000	9001
El servidor espera con	`accept()` + `recv()`	`recvfrom()`
Devuelve con	`conn.sendall(datos)`	`servidor.sendto(datos, direccion)`
¿Necesita saber quién es el cliente?	La conexión lo sabe	`recvfrom()` le entrega la dirección
¿Atiende a varios a la vez?	De uno en uno (o con hilos)	Cualquiera, sin hilos

📝 Aprieta el lápiz

1. **Eco server TCP**: Crea un servidor que devuelva exactamente lo que recibe.
2. **Eco server UDP**: Crea un servidor UDP que devuelva al cliente lo mismo que recibe.
3. **Contador de letras**: El cliente envía una frase, el servidor responde con la cantidad de letras.
4. **Servidor hora**: El cliente se conecta y el servidor le devuelve la hora actual.
5. **Cliente con timeout**: Crea un cliente que intente conectar, y si no hay respuesta en 3s, muestre “Servidor no disponible”.

6. **Comparativa velocidad**: Mide cuánto tarda TCP vs UDP en enviar 100 mensajes pequeños.

> 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Qué línea convierte a cada servidor (TCP y UDP) en “eco”?
2. ¿Cómo descubre el servidor UDP a quién responder?
3. ¿Qué dos llamadas son bloqueantes en el eco TCP y en el eco UDP?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- El eco TCP devuelve con `conn.sendall(datos)` lo que recibió con `recv(1024)`, sobre una conexión aceptada con `accept()`.
- El eco UDP devuelve con `sendto(datos, direccion)` lo que recibió con `recvfrom()`, usando la dirección que cada datagrama trae pegada.
- Con el servidor en una terminal y el cliente en otra tienes cada protocolo completo funcionando; UDP atiende a cualquiera sin hilos, TCP de uno en uno.

🐛 Vocabulario rápido

Término	Idea general
Eco	El servidor devuelve lo mismo que recibe
sendall(datos)	La jugada del eco TCP: reenviar los bytes recibidos
sendto(datos, direccion)	La jugada del eco UDP: reenviar a la dirección de origen
Bloqueante	accept(), recv() y recvfrom() esperan hasta que algo ocurre
Terminal del servidor	Ventana donde el servidor queda escuchando

📚 Volver al índice de la unidad · **Anterior:** 07 · Cuándo usar cada protocolo · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé el socket, laboratorios TCP/UDP y el cierre de la unidad 🧠

> 🗺️ **Estás en:** 🔌 **UD 5 · Sockets TCP y UDP** → 09 · Cierre

Has terminado la teoría: socket, cliente y servidor TCP, handshake, errores, `SO_REUSEADDR`, cliente y servidor UDP, datagramas, HTTP, NTP y cuándo usar cada protocolo. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé el socket

> *Eres el socket del servidor, recién creado con `socket(AF_INET, SOCK_STREAM)`. Tu misión: escuchar, aceptar y atender a quien llame.*

¿Qué pasa?

1. Haces `bind(("127.0.0.1", 9000))`: reservas el puerto. **Ahora eres la centralita.**
2. Llamas a `listen()`: el SO ya sabe que esperas llamadas.
3. Te quedas en `accept()`… **esperas. Y esperas.** Eres bloqueante: no hay nada que hacer hasta que alguien llame.
4. Un cliente ejecuta `connect()` y el **three-way handshake** recorre la red (SYN → SYN+ACK → ACK). El timbre suena.
5. `accept()` despierta y devuelve `(conn, ("127.0.0.1", 54321))`: un **nuevo socket** para esa conversación, con la dirección del cliente.
6. Llegas a `recv(1024)`… **esperas otra vez**. Cuando el cliente manda `b"Hola!"`, despiertas.
7. Contestas con `sendall()` y, al salir del `with`, el socket de la conversación se cierra con la despedida FIN/ACK.
8. Y tú, el socket servidor, sigues en tu `accept()`: **esperando al siguiente cliente**.

Todo el tiempo, el SO ha hecho el trabajo sucio: handshakes, despedidas, orden de bytes.

> 💡 **Ahora tú:** ¿y si el cliente llama mientras sigues en el paso 6 atendiendo a otro? La conexión queda **en la cola** de `listen()` (el *backlog*), esperando tu próximo `accept()`. Eso es lo que viste en el punto 3: un solo hilo atiende a un cliente cada vez.

🔥 Fireside Chat: TCP vs UDP

> *Dos protocolos de transporte se sientan junto a la chimenea a dirimir, de una vez, quién manda.*

TCP: — Yo soy el mensajero certificado. Entrego cada carta, en orden, y si se pierde, la reenvío. Pero cuesta más.

UDP: — Yo soy el lanzador de aviones de papel. Mando y olvido. Si no llega, pues no llega. Pero lanzo 100 en el tiempo que tú preparas uno.

TCP: — Mis casos de uso: web (HTTP), correo (SMTP), transferencia de archivos (FTP). Todo lo que necesite fiabilidad.

UDP: — Mis casos de uso: videollamadas (Zoom), juegos online (Fortnite), DNS, NTP. Prefiero velocidad antes que fiabilidad.

TCP: — Tengo control de congestión, retransmisión, checksums…

UDP: — Yo tengo… velocidad. Y puedo añadir fiabilidad en la capa de aplicación si quiero (QUIC, por ejemplo).

TCP: — Eres un temerario.

UDP: — Y tú un pesado. Por eso nos complementamos.

> **Moraleja:** no son rivales: son dos herramientas para dos momentos. TCP cuando el dato debe quedar intacto; UDP cuando el momento es lo valioso. Ese criterio lo dominas desde el punto 7.

🕵️ ¿Quién soy?

1. Soy el punto final de una conexión de red: la interfaz para enviar y recibir datos.
2. Soy el método del cliente TCP que estrecha la mano antes de hablar.
3. Soy el método del servidor UDP que recibe datagramas y dice de quién vienen.
4. Soy el estado que mantiene el puerto reservado unos segundos tras cerrar.
5. Soy la opción que evita el “Address already in use”.
6. Soy el protocolo de texto sobre TCP que mueve la web.
7. Soy el grupo de servidores de tiempo que responde en el puerto 123.

8. Soy el protocolo fiable que se construye encima de UDP para HTTP/3.

> 🔁 **Respuestas**

👹 CONRAD VS EL MUNDO: “conexión reiniciada y paquetes perdidos”

CONRAD: — “Clásico TCP: el cliente hace `recv()` y le salta *‘ConnectionResetError’*. Pues claro. Razones: 1) **El servidor se cayó** en mitad de la conversación y su SO mandó un RST. 2) **Cerraste el servidor con Ctrl+C** mientras el cliente hablaba: misma historia. 3) El cliente intentó **escribir en un socket ya cerrado** → `BrokenPipeError`. 4) O el **timeout** se te pasó: `recv()` bloqueado para siempre porque nadie respondió.”

CONRAD: — “Y lo mejor: *‘pero yo hacía sendall y me daba error’*. ¡Pues claro! `sendall()` no avisa: es el SO quien lanza la excepción cuando la otra punta de la tubería ya no existe. Captura `ConnectionResetError` y `BrokenPipeError` por separado, como viste en el punto 4, y tu cliente dejará de morir a lo loco.”

CONRAD: — “Y no me vengas con *‘¿será que la red va lenta?’*. Si el servidor se reinició y volvió a arrancar, **sin `SO_REUSEADDR`** te salta *Address already in use* al instante. En UDP, encima, *‘el cliente hacía recvfrom y nunca llegaba nada’*: si el datagrama se perdió, `recvfrom()` se queda bloqueado **para siempre**. Un `settimeout(5)` y verás la excepción aparecer. Tres errores, tres causas, tres soluciones: reintentos con `try/except`, `settimeout()`, y `SO_REUSEADDR`. A diagnosticar.”

⚡ Laboratorio de tortura: eco TCP y UDP

> **Duración:** 45 minutos **Herramienta:** Python 3 (`socket`, sin instalar nada) + dos terminales

Escenario: construye los dos ecos del punto 8 — uno TCP, otro UDP — y rompe ambos a propósito.

Tareas paso a paso:

1. **Escribe el servidor eco TCP** (`servidor_eco.py`): `SO_REUSEADDR`, `bind()` en `127.0.0.1:9000`, `listen()`, `accept()` y un bucle que haga `recv()` y responda con `sendall(datos)`. Imprime cada mensaje con su dirección.

2. **Escribe el cliente TCP** (`cliente_eco.py`): pide un mensaje con `input()`, lo envía con `sendall()` y muestra la respuesta de `recv()`.
3. **Arranca el TCP** en una terminal y el cliente en otra. Envía tres mensajes y comprueba que el eco funciona y el contador lleva la cuenta.
4. **Repite en UDP** (puerto 9001): servidor con `bind()` + `recvfrom()` + `sendto(datos, direccion)`; cliente con `sendto()` + `recvfrom()`.
5. **Añade `settimeout(3)`** al cliente TCP y comprueba qué pasa si cierras el servidor antes de enviar. Haz lo mismo con el cliente UDP.

Fallo intencionado TCP: cierra el servidor con Ctrl+C y **relánzalo al instante**. ¿Qué pasa? Sin `SO_REUSEADDR`, el `bind()` falla con *"Address already in use"* porque las conexiones anteriores siguen en **TIME_WAIT**. Añade `servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` antes del `bind()` y comprueba que ahora sí puedes reiniciar sin esperar.

Fallo intencionado UDP: en el servidor, en lugar de responder con la `direccion` que devuelve `recvfrom()`, responde a una dirección inventada: `servidor.sendto(datos, ("127.0.0.1", 9999))`. ¿Qué pasa? El cliente se queda **bloqueado para siempre** en su `recvfrom()`: el eco fue a otro puerto, donde nadie escucha. Solo el `settimeout(5)` te sacará del atolladero.

> **Pista 1:** el estado TIME_WAIT es el culpable del fallo TCP. Con `SO_REUSEADDR` activado, el SO te deja reutilizar la dirección aunque queden conexiones en ese estado.
>
> **Pista 2:** si un cliente se queda colgado en `recv()` o `recvfrom()`, ese es el síntoma clásico de "la respuesta nunca llegó". Añade `settimeout()` y verás la excepción `socket.timeout` a los pocos segundos, confirmando el diagnóstico.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Telefonista**	Crear el primer socket TCP con `socket(AF_INET, SOCK_STREAM)`
🏅 **Marca-Números**	Implementar un cliente TCP con `connect()`, `sendall()` y `recv()`
🏅 **Centralita**	Implementar un servidor TCP con `bind()`, `listen()` y `accept()`
🏅 **Handshaker**	Explicar el three-way handshake y el cierre FIN/ACK
🏅 **Fénix TCP**	Dominar `SO_REUSEADDR` y reiniciar servidores sin error

Logro	Cómo conseguirlo
🏅 **Avión de Papel**	Crear y enviar el primer datagrama UDP con `sendto()`
🏅 **Eco Master**	Montar un servidor + cliente eco en TCP y UDP
🏅 **Fiable o Veloz**	Explicar cuándo usar TCP y cuándo UDP con casos reales
🏅 **Navegador a pelo**	Hablar HTTP a mano con un socket TCP y leer la respuesta
🏅 **Relojero**	Obtener la hora oficial de Internet con un cliente NTP

🧠 Atrévete a pensar

1. ¿Por qué el servidor TCP hace `accept()` y no `recv()` directamente?
2. ¿Qué pasaría si dos programas intentan `bind()` al mismo puerto a la vez?
3. ¿Por qué UDP puede “perder datos” y TCP no, y por qué eso es aceptable en VoIP?
4. ¿Cómo sabe un servidor UDP a quién responder si no hay conexión?
5. ¿Por qué NTP usa UDP aunque la hora exacta parezca “importante”?

> › 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Método del cliente TCP que establece la conexión (7 letras)
4. Paquete que inicia el three-way handshake (3 letras)
6. Opción que evita "Address already in use" (12 letras)
8. Método del cliente UDP para enviar un datagrama (6 letras)

Vertical:
2. Verbo HTTP para pedir un recurso (3 letras)
3. Dirección de tu propia máquina (9 letras)
5. Estado que mantiene el puerto reservado al cerrar (9 letras)
7. Puerto de los servidores NTP (3 dígitos)
```

> › 📝 **Soluciones**

💬 Entrevista de trabajo

1. **“¿Qué diferencia hay entre TCP y UDP? ¿Cuándo usarías cada uno?”**
2. **“Escribe un servidor TCP que reciba un mensaje y lo devuelva.”**
3. **“¿Cómo funciona el three-way handshake? Explícalo con un diagrama.”**
4. **“¿Qué errores pueden ocurrir al comunicar por sockets y cómo los gestionas?”**
5. **“¿Qué es SO_REUSEADDR y cuándo lo necesitas?”**
6. **“¿Cómo funciona HTTP a nivel de socket? Describeme una petición y su respuesta.”**

> 💡 **Cómo encararlas:** la 1 y la 2 son las “preguntas reina”. Para la 1, repite la moraleja del punto 7: fiabilidad contra velocidad, con los casos reales (web/correo → TCP; streaming/juegos/DNS → UDP). Para la 2, escribe el servidor del punto 3 sin pensarlo: `socket()` + `bind()` + `listen()` + `accept()` + `recv()` + `sendall()`. Si sabes contarlo fluido, ya eres medio desarrollador de redes.

🤷 No hay preguntas tontas

> ❓ **¿Qué pasa si el servidor no llama a `listen()`?**

El cliente recibe `ConnectionRefusedError`. Sin `listen()`, no hay puerto abierto.

> ❓ **¿Y si no llamo a `bind()`?**

Para el servidor es obligatorio (necesita un puerto fijo). Para el cliente, el SO asigna uno automáticamente.

> ❓ **¿Puedo tener más de un cliente conectado a la vez?**

Con el código TCP básico, no: solo acepta un cliente cada vez. Para múltiples clientes, mira la **UD 6 · Servidores concurrentes**. En UDP, en cambio, un solo servidor atiende a cualquiera sin hilos: cada datagrama trae su dirección.

> ❓ **¿UDP puede perder datos?**

Sí. No hay confirmación de recepción. Si pierdes un paquete, se pierde para siempre.

> ❓ **¿HTTP siempre usa TCP?**

Sí, HTTP/1.1 y HTTP/2 usan TCP. **HTTP/3** usa QUIC, que va sobre UDP (¡la vuelta a la tortilla!).

> ❓ **¿NTP usa UDP? ¿No es importante que llegue la hora exacta?**

Sí, NTP usa UDP. Pero manda muchas peticiones y calcula estadísticamente la hora correcta. Si un paquete se pierde, no pasa nada: el próximo valdrá.

🎬 Poscréditos

> *Un socket TCP estrecha la mano. SYN, SYN+ACK, ACK. La llamada está en marcha.*

El cliente habla; el servidor escucha; el eco vuelve exactamente igual que llegó.

El servidor se cae y renace. Esta vez, `SO_REUSEADDR` le deja volver al instante.

Y en una terminal lejana, un datagrama UDP vuela sin conexión, sin confirmación, sin miedo… mientras un reloj sin átomos pregunta la hora exacta en el puerto 123.

PRÓXIMAMENTE EN UD 6: *Servidores concurrentes. Un cliente a la vez ya no basta: hilos, `select()` y `socketserver` para atender a una multitud sin caerte.*

✅ Criterios de evaluación cubiertos (RA3)

RA3 — Sockets: comunicaciones en red con TCP/UDP y protocolos de aplicación (HTTP, NTP).

CE	Criterio	Cubierto
a)	Modelo de capas de red (TCP/IP)	✅ Punto 1 + Fireside Chat
b)	Identifica tipos de sockets (TCP/UDP)	✅ Puntos 1 y 5
c)	Crea servidores TCP	✅ Puntos 3 y 8 + ⚡ Laboratorio de tortura
d)	Crea clientes TCP	✅ Puntos 2 y 8 + ⚡ Laboratorio de tortura
e)	Implementa servidores y clientes UDP	✅ Puntos 5 y 8 + ⚡ Laboratorio de tortura
f)	Gestiona errores de red	✅ Punto 4 + ⚡ Laboratorio con fallo intencionado

CE	Criterio	Cubierto
g)	Configura opciones de socket (SO_REUSEADDR, non-blocking)	✅ Punto 4 + ⚡ Laboratorio de tortura
h)	Implementa protocolos de aplicación (HTTP, NTP)	✅ Punto 6 + Cliente HTTP y NTP manuales

📚 Volver al índice de la unidad · **Anterior:** 08 · Práctica eco · **Siguiente:** UD 6 · Servidores concurrentes

UD 6 — Servidores concurrentes con Python

Servidores que atienden a muchos clientes a la vez 🎪

> "Un servidor secuencial atiende a un cliente cada vez. Los demás esperan. Un servidor concurrente atiende a todos a la vez. Como un camarero con 10 mesas."

Hasta ahora has construido servidores TCP que atienden **de uno en uno**: si un cliente tarda 3 segundos, los que llegan detrás esperan. En esta unidad darás el salto a la **concurrencia**: primero lanzando un **hilo por cada cliente** y después con un **ThreadPoolExecutor**, el equipo de hilos reutilizables que usa la industria.

También medirás cuánto ganas con cada enfoque mediante un **benchmark**, aprenderás a proteger con **Lock** el estado compartido del servidor, cerrarás montando un servidor concurrente completo y verás la alternativa con **procesos independientes**, que aíslan los fallos de cada cliente.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar por qué un servidor secuencial solo atiende un cliente cada vez.
- Implementar un **servidor multihilo** que lance un hilo por cada conexión.
- Gestionar un **ThreadPoolExecutor** con un número fijo de hilos reutilizables.
- Medir el rendimiento de un servidor con un **benchmark**.
- Proteger con **Lock** las variables compartidas entre los hilos del servidor.
- Montar un **servidor concurrente completo** con su lanzador de clientes.
- Atender clientes con `multiprocessing.Process`, compartiendo el estado con `manager.dict()` para lograr aislamiento de fallos.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Servidor secuencial	El servidor que atiende de uno en uno y su límite	Todos
02 · El problema de la espera	Por qué un cliente lento congela a toda la cola	Todos
03 · Hilo por cliente	Lanzar un hilo por cada conexión	Todos
04 · ThreadPoolExecutor	Un equipo fijo de hilos reutilizables	Todos
05 · Benchmark	Medir cuánto ganas con la concurrencia	Todos
06 · Sincronización en servidores	El Lock para el estado compartido	Todos
07 · Límites y buenas prácticas	Cuántos hilos puedes crear	Todos
08 · Servidor concurrente completo con hilos	Todo junto: servidor, cliente, lanzador y ejercicios	Todos
09 · Servidor concurrente completo con procesos	El mismo servidor, pero con `Process` en lugar de hilos	Todos
10 · Cierre	Sé el servidor, Fireside, Laboratorio de tortura…	Todos

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA4c-d)

RA4: Implementa servicios en red con servidores concurrentes.

CE	Criterio	Dónde se cubre
RA4c	Implementa servidores concurrentes con hilos	✅ Puntos 1-3 y 8 + ⚡ Laboratorio (punto 10)
RA4d	Gestiona pools de hilos (ThreadPoolExecutor)	✅ Puntos 4-5 y 8 + ⚡ Laboratorio (punto 10)

> RA4a-b (APIs REST y comerciales) se cubren en las **UD 7 y UD 8**. RA4e-g (asyncio) se cubren en la **UD 10**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 5 y dominas sockets TCP? Empieza por el punto 1, que parte de un servidor TCP normal y corriente.

¿Ya sabes escribir servidores y solo quieres el salto a la concurrencia? Saltar al punto 3 o al punto 4.

📍 **Primer punto:** 01 · Servidor secuencial ⏭️ **Al acabar la unidad, continúa en** UD 7 · HTTP y APIs REST.

01 — Servidor secuencial

El servidor que atiende a un cliente cada vez 🐢

> 🗺️ **Estás en:** 🏗️ **UD 6 · Servidores concurrentes** → 01 · Servidor secuencial

📬 La idea en una frase

> Un servidor **secuencial** atiende a un cliente cada vez: `accept()` recibe una conexión, la procesa hasta el final y solo entonces vuelve a aceptar. Los demás clientes esperan en una cola invisible, aunque ya estén conectados.

Es el servidor que construiste en la UD 5 · Sockets TCP y UDP: un bucle `while True` con `accept()`. Funciona, pero tiene un límite muy claro que vamos a medir.

🐢 El servidor secuencial

El ejemplo clásico simula trabajo pesado con un `time.sleep(3)` dentro de la atención a cada cliente:

```
import socket, time

def servidor_lento():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🐢 Servidor SECUENCIAL: 1 cliente cada vez")

        while True:
            conn, addr = srv.accept()
            print(f"  Cliente {addr} conectado")
            time.sleep(3)  # Simula trabajo pesado
            conn.sendall(b"Procesado\n")
            conn.close()
            print(f"  Cliente {addr} atendido")
```

> Si 3 clientes se conectan a la vez: el primero tarda 3s, el segundo 6s, el tercero 9s. 🐢

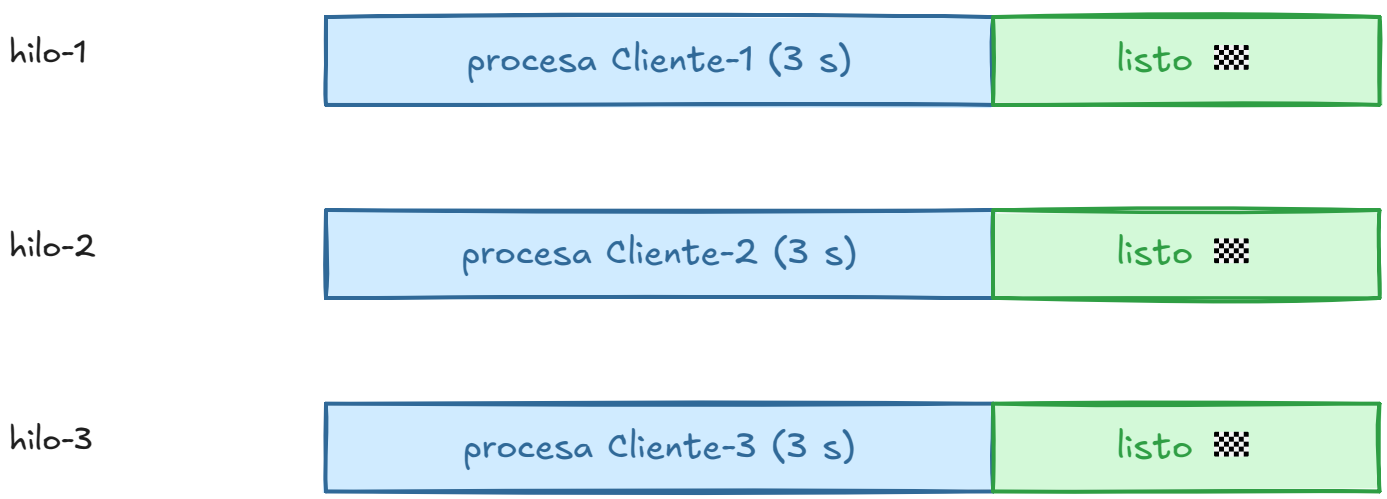
El bucle hace esto: acepta → procesa (los 3 segundos) → cierra → acepta de nuevo. Mientras procesa, el `accept()` siguiente no se ejecuta, así que nadie más entra. El socket del sistema operativo sí "acepta" la conexión TCP a nivel de red (los clientes no ven error), pero el código no los atiende: están **en cola**.

📜 La traza: secuencial vs concurrente

El clásico "Be the code" del tema. Traza mentalmente qué pasa cuando 3 clientes se conectan a la vez:

Servidor secuencial (3 clientes a la vez):

Servidor concurrente (3 clientes a la vez):



los 3 se procesan a la vez en paralelo: los tres listos en t=3 s

> Con hilos, todos terminan a la vez. Sin hilos, el último espera 9s.

Por ahora nos quedamos con la primera columna: el secuencial. El límite se ve solo: **el tiempo total crece linealmente** con el número de clientes. En el punto 3 construiremos el concurrente.

🧠 Mini-chequeo

1. ¿Por qué los clientes 2 y 3 “se conectan” sin error pero no reciben respuesta?
2. Si 5 clientes se conectan a la vez y cada uno tarda 2s en procesarse, ¿cuánto espera el quinto?
3. ¿Qué línea del código es la que se queda “atascada” mientras se procesa un cliente?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- El servidor secuencial procesa **un cliente cada vez** y los demás esperan en la cola del sistema operativo.
- Su límite es lineal: **n clientes × tiempo por cliente** de espera para el último.
- La solución a la espera es la **concurrencia**: atender a varios clientes a la vez (punto 3).

🐛 Vocabulario rápido

Término	Idea general
Servidor secuencial	Atiende un cliente cada vez, en orden
Cola del socket	Conexiones TCP aceptadas por el SO que aún no atiende el código
accept()	Llamada que recoge la siguiente conexión pendiente
time.sleep(3)	Simula trabajo pesado en los ejemplos
Límite lineal	El tiempo total crece con el número de clientes

📚 Volver al índice de la unidad · **Siguiente:** 02 · El problema de la espera

02 — El problema de la espera

El cliente lento que bloquea a todos los demás 🥶

> 🗺️ **Estás en:** 🏗️ **UD 6 · Servidores concurrentes** → 02 · El problema de la espera

📬 La idea en una frase

> El problema del servidor secuencial no es el número de clientes, sino el **cliente lento**: uno solo que tarda 3 segundos congela a todos los que esperan detrás. Basta un cliente pesado para paralizar el servidor entero.

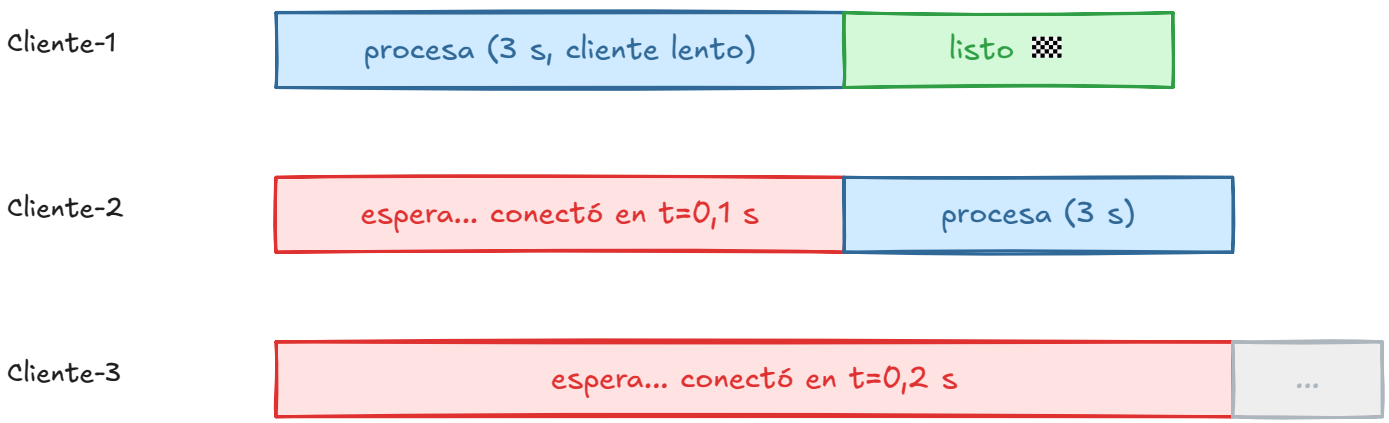
En el punto 1 viste que el tiempo crece linealmente. Ahora vamos a entender por qué es un problema real, no solo una curiosidad teórica: cualquier cliente que tarde, tarde por lo que sea (cálculo pesado, acceso a una base de datos lenta, una red de 56k…), convierte a todos los demás en espectadores.

🥶 Un cliente lento congela la cola

Vuelve al código del servidor secuencial:

```
import socket, time

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        time.sleep(3)              # ← el trabajo "lento" de este cliente
        conn.sendall(b"Procesado\n")
        conn.close()
```

Traza la secuencia con un cliente lento en medio:



conectan en t=0,1 y t=0,2 s pero no se les acepta hasta que el servidor se queda libre

Fíjate en el detalle clave: **Cliente-2 conectó en t=0.1s** y fue atendido a las 3s. **Cliente-3 conectó en t=0.2s** y fue atendido a las 6s. No es que el servidor sea lento: es que **el trabajo del cliente anterior bloquea el `accept()`**. El cliente más rápido del mundo espera a que el más lento termine.

> ⚠️ El problema empeora con un solo cliente pesado: una petición que tarde 30 segundos (una consulta enorme, un archivo gigante) pone a **toda** la cola a esperar 30 segundos, y al que llega detrás, otros 30.

🏦 La analogía: la ventanilla única

Imagina un banco con **una sola ventanilla**. Llegan 10 personas a la vez. La primera tiene un trámite que dura 3 minutos; las otras 9 esperan en fila. La segunda ya está "en el banco" (se ha conectado al socket), pero su trámite no empieza hasta que la primera termina.

```
🏦 VENTANILLA ÚNICA (servidor secuencial):
  [PERSONA-1 ⏳ 3min]   [PERSONA-2 👤] [PERSONA-3 👤] [PERSONA-4 👤] …
  La ventanilla solo puede atender a una persona a la vez.

🏢 VENTANILLAS MÚLTIPLES (servidor concurrente):
  [PERSONA-1 ⏳] [PERSONA-2 ⏳] [PERSONA-3 ⏳] [PERSONA-4 ⏳] …
  Cada persona tiene su ventanilla: todas avanzan a la vez.
```

La moraleja es obvia: si el banco tiene 10 ventanillas, 10 personas se atienden en paralelo. Esa es exactamente la idea del punto 3: dar a cada cliente su propia "ventanilla" (su hilo).

Situación	Servidor secuencial	Servidor concurrente
10 clientes, 2s cada uno	~20 segundos (10 × 2s)	~2 segundos (en paralelo)
1 cliente lento de 30s	Todos esperan +30s	Solo el lento tarda 30s
Picos de conexiones	La cola crece sin límite visible	Se atienden en paralelo

🧠 Mini-chequeo

1. Un cliente tarda 8 segundos en procesarse. ¿Cuánto espera el siguiente cliente que conectó en el mismo segundo?
2. ¿En qué punto del código se "atasca" la atención al resto de clientes?
3. ¿El problema es el número de clientes o la lentitud del procesamiento?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un cliente lento **bloquea el `accept()`** y todos los que esperan detrás se congelan.
- El tiempo de espera del último cliente es **n × tiempo del más lento** en el peor caso.
- La analogía de la **ventanilla única** lo resume: más ventanillas (hilos) = más clientes a la vez.

🐛 Vocabulario rápido

Término	Idea general
Cliente lento	Conexión cuyo procesamiento tarda (cálculo, red, base de datos)
Bloqueo	El `accept()` no se ejecuta hasta que termina el cliente actual
Cola	Clientes ya conectados al socket que aún no se atienden

Término	Idea general
Ventanilla única	Analogía del servidor secuencial
Procesamiento	El trabajo que hace el servidor con cada cliente

📚 Volver al índice de la unidad · **Anterior:** 01 · Servidor secuencial · **Siguiente:** 03 · Hilo por cliente

03 — Hilo por cliente

Cada cliente se atiende en su propio hilo 🔀

> 🗺️ **Estás en:** 🧬 **UD 6 · Servidores concurrentes** → 03 · Hilo por cliente

---

📬 La idea en una frase

> En lugar de atender al cliente dentro del bucle, el servidor **lanza un hilo por cada cliente** y vuelve inmediatamente a `accept()`. Cada conexión tiene su propia "ventanilla".

Ya conoces los hilos de la U03 · Hilos fundamentos y la sincronización de la U04. Este punto los pone a trabajar: el patrón **hilo por cliente** (thread per connection) es el salto más directo desde el servidor secuencial.

🚀 El servidor multihilo

Cada cliente recibe su propio hilo. La función que lo atiende se ejecuta en paralelo con todas las demás:

```python
import socket, threading

def atender(conn, addr):
    print(f"[+] Cliente {addr} conectado")
    with conn:
        datos = conn.recv(1024)
        print(f"    Recibido: {datos.decode()}")
        import time
        time.sleep(2)  # Simular trabajo
        conn.sendall(b"OK: " + datos)
    print(f"[-] Cliente {addr} desconectado")

def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🚀 Servidor MULTIHILO — varios clientes a la vez")

        while True:
            conn, addr = srv.accept()
            hilo = threading.Thread(target=atender, args=(conn, addr))
            hilo.start()
```

Dos diferencias frente al servidor secuencial del punto 1:

1. **No hay trabajo en el bucle principal**: `accept()` → crear hilo → `start()` → vuelta a `accept()`. El bucle nunca se bloquea.
2. **`conn` se pasa como argumento** al hilo: cada hilo recibe su socket y su dirección, y los libera con `with conn:` al terminar.

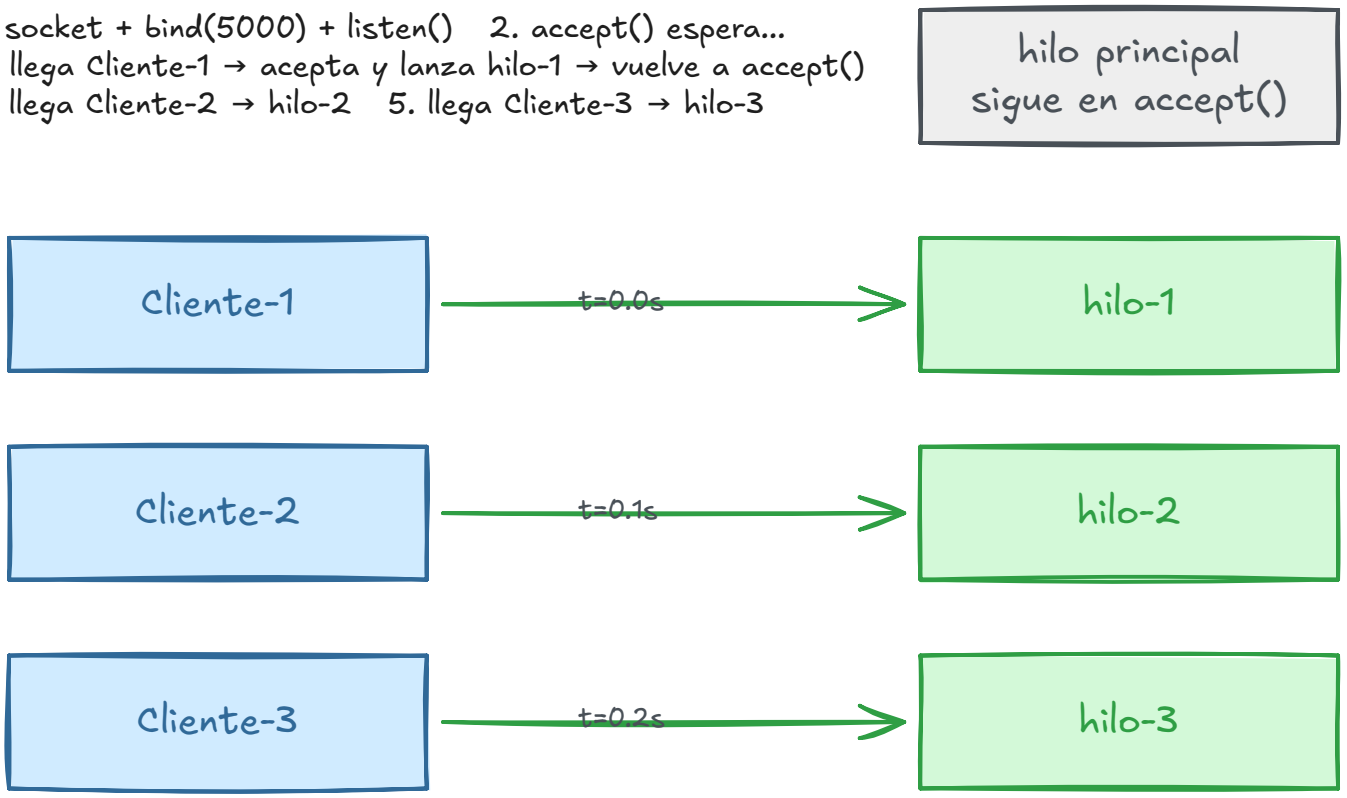
> 📝 Recuerda la convención del módulo: los hilos se nombran con `.name = "hilo-"+str(n)` si necesitas trazarlos.

📜 Be the code — Servidor multihilo en acción

> "Sé el servidor que recibe 3 clientes a la vez. Traza cada hilo."

Sé el servidor: 3 accept() y 3 hilos

1. socket + bind(5000) + listen() 2. accept() espera...
3. llega Cliente-1 → acepta y lanza hilo-1 → vuelve a accept()
4. llega Cliente-2 → hilo-2 5. llega Cliente-3 → hilo-3



ahora 4 hilos ejecutando: el principal en accept() y un hilo por cliente

El hilo principal **nunca se detiene**: en cuanto lanza un hilo, vuelve a `accept()`. El trabajo pesado (`time.sleep(2)`) vive en los hilos secundarios, que se ejecutan en paralelo gracias al *threading* de la U03.

⚖️ Ventajas y el primer cuidado

Ventaja	Explicación
✅ Simple de escribir	Solo un `threading.Thread(target=atender, args=(conn, addr))`
✅ Aprovecha la CPU	Varios clientes procesados de verdad en paralelo
✅ El bucle no se bloquea	`accept()` vuelve al instante, nadie espera en la cola

Pero hay una letra pequeña: **crear un hilo por cada cliente puede saturar el sistema con 10.000 conexiones**. Cada hilo consume memoria y el sistema operativo tiene que gestionarlos. Ese límite lo verás en el punto 4 (el ThreadPool) y en el punto 7.

🧠 Mini-chequeo

1. ¿Qué hace el hilo principal mientras un hilo secundario procesa a su cliente?
2. ¿Por qué no hace falta `join()` en el bucle principal del servidor?
3. ¿Qué pasaría con este código si llegan 10.000 conexiones a la vez?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El patrón **hilo por cliente** lanza un `threading.Thread` por cada conexión y el bucle vuelve al instante a `accept()`.
- Todos los clientes se atienden **en paralelo**, cada uno en su propio hilo.
- Es simple y efectivo para pocos clientes, pero **no escala** a miles de conexiones: para eso existe el ThreadPool.

🐛 Vocabulario rápido

Término	Idea general
Hilo por cliente	Patrón: un `Thread` por cada conexión aceptada
Hilo principal	El del bucle `accept()`: solo acepta y lanza hilos
atender()	Función que procesa a un cliente en su hilo
start()	Pone el hilo en marcha sin bloquear al llamante
Thread(target, args)	Constructor del hilo con su función y argumentos

📚 Volver al índice de la unidad · **Anterior:** 02 · El problema de la espera · **Siguiente:** 04 · ThreadPoolExecutor

04 — ThreadPoolExecutor

Un equipo fijo de hilos que se reutiliza ⚡

> 🗺️ **Estás en:** 🧬 **UD 6 · Servidores concurrentes** → 04 · ThreadPoolExecutor

📬 La idea en una frase

> Crear un hilo por cada cliente puede saturar el sistema con 10.000 conexiones. Un **ThreadPoolExecutor** reutiliza un número fijo de hilos: si llegan más clientes que hilos, esperan en una cola en lugar de multiplicar los hilos.

Es el mismo servicio del punto 3, pero con **recursos bajo control**: el pool es un equipo de camareros limitado, no un camarero nuevo por cada mesa.

⚠️ El problema: miles de hilos

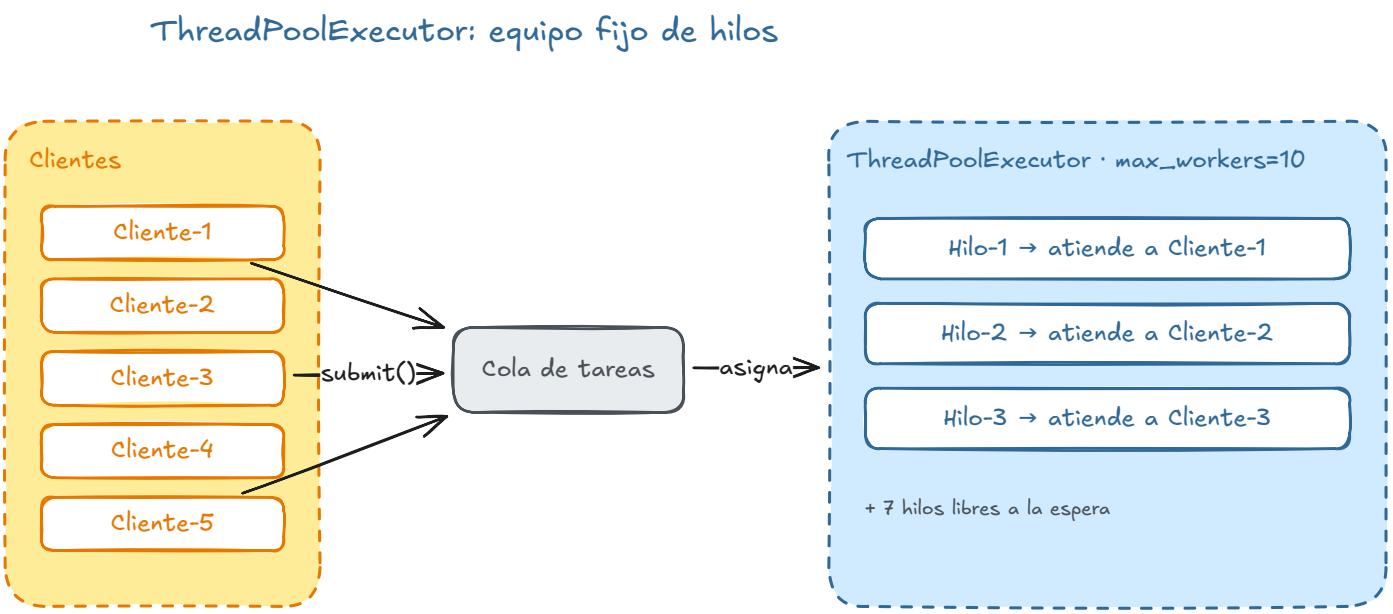
El enfoque del punto 3 funciona de maravilla con 10 clientes. Con 10.000 conexiones simultáneas se convierte en una bomba:

- Cada hilo consume **memoria propia** (pila de ejecución).
- El sistema operativo debe **planificar** (context switch) entre miles de hilos: la CPU pierde más tiempo cambiando de hilo que trabajando.
- El proceso puede quedarse **sin memoria** y morir.

Enfoque	Ventaja	Desventaja
Hilo por cliente	Simple	Puede saturar el SO (miles de hilos)
ThreadPool	Límite controlado	Si el pool se llena, los clientes esperan

> "Si el pool se llena, los clientes esperan" suena a defecto, pero es la **protección** que queremos: esperar en cola es mejor que matar el servidor.

⚡ El servidor con pool



Si el pool se llena, los clientes esperan en cola: protección, no defecto

```python
import socket, concurrent.futures

MAX_HILOS = 10

def atender(conn, addr):
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK: " + datos)

def servidor_pool():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()

        with concurrent.futures.ThreadPoolExecutor(max_workers=MAX_HILOS) as pool:
            print(f"⚡ Servidor con Pool de {MAX_HILOS} hilos")
            while True:
                conn, addr = srv.accept()
                pool.submit(atender, conn, addr)
```

Dos piezas nuevas frente al punto 3:

1. **`ThreadPoolExecutor(max_workers=MAX_HILOS)`** crea el equipo de hilos (10 aquí). Se usa como contexto (`with ... as pool`) para que se cierre solo al salir.

2. **`pool.submit(atender, conn, addr)`** sustituye a `Thread(target=...).start()`: entrega la tarea al pool y este decide qué hilo libre la coge. Si todos están ocupados, la tarea espera en la cola interna del pool.

> ⚡ El `submit()` devuelve un `Future`, pero en un servidor no lo necesitamos: no nos interesa el resultado, solo que la tarea se ejecute.

🎯 ¿Qué tamaño de pool elegir?

No hay una fórmula mágica, pero estas reglas de oro ayudan:

Situación	Tamaño razonable
Servidor de prueba / clase	5-10 hilos
CPU intensivo (cálculo puro)	No de núcleos de la CPU
I/O intensivo (red, discos, APIs)	Más hilos que núcleos (los hilos esperan mucho en I/O)

> 💡 En UD 10 · asyncio verás que la concurrencia por I/O se puede hacer aún más ligera, sin hilos. De momento, el pool es la opción sensata.

🧩 Pool Puzzle — Servidor con ThreadPool

Desordena y reconstruye este servidor que usa ThreadPoolExecutor:

```
a) from concurrent.futures import ThreadPoolExecutor
b)     pool.submit(atender, conn, addr)
c)     with ThreadPoolExecutor(max_workers=5) as pool:
d) import socket
e) def atender(conn, addr):
f)         conn.sendall(b"OK\n")
g) while True:
h)     with socket.socket() as srv:
i)             datos = conn.recv(1024)
j)         conn, addr = srv.accept()
k)     srv.bind(("0.0.0.0", 5000))
l)     srv.listen()
```

> 🔓 Solución

🧠 Mini-chequeo

1. ¿Qué pasa con el cliente número 11 si el pool tiene `max_workers=10` y todos están ocupados?
2. ¿Qué ventaja tiene reutilizar hilos frente a crear uno nuevo por cliente?
3. ¿Por qué el pool se crea fuera del bucle `while True`?

> 🔄 Respuestas

✅ Resumen en 3 frases

- `ThreadPoolExecutor(max_workers=N)` mantiene **N hilos fijos reutilizables** y una cola interna.
- Si el pool está lleno, los clientes **esperan en cola** en lugar de multiplicar los hilos.
- Es la opción de **producción**: límite controlado, hilos reutilizados y sin riesgo de saturar el SO.

🐛 Vocabulario rápido

Término	Idea general
ThreadPoolExecutor	Pool de hilos reutilizables de `concurrent.futures`
max_workers	Número máximo de hilos del pool
submit()	Entrega una tarea al pool (devuelve un Future)
Cola interna	Tareas esperando a que un hilo se libere
Future	Objeto con el resultado futuro de la tarea (no lo usamos en servidores)

📚 Volver al índice de la unidad · **Anterior:** 03 · Hilo por cliente · **Siguiente:** 05 · Benchmark

05 — Benchmark

Medir cuánto ganas con la concurrencia ⏱️

> 🗺️ **Estás en:** 🏗️ **UD 6 · Servidores concurrentes** → 05 · Benchmark

📬 La idea en una frase

> Un **benchmark** lanza N clientes a la vez contra tu servidor y cronometra cuánto tardan todos en recibir respuesta. Es la prueba objetiva de que la concurrencia funciona… o de que tu servidor sigue siendo secuencial.

Hasta ahora hemos razonado sobre tiempos ("el secuencial tarda n × tiempo"). Este punto lo **mide de verdad**: con un lanzador masivo de clientes verás con tus propios ojos la diferencia entre secuencial, hilos y pool.

🧪 El experimento

¿Cuánto más rápido es un servidor concurrente? Aquí tienes un experimento que lanza 10 clientes y mide el tiempo total:

```
import socket, time, threading, concurrent.futures

def cliente(id):
    """Conecta, envía 'ping', recibe 'pong'"""
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(b"ping")
        s.recv(1024)

def prueba(tipo_servidor, n=10):
    """Lanza n clientes en paralelo y cronometra"""
    with concurrent.futures.ThreadPoolExecutor(max_workers=n) as pool:
        futuros = [pool.submit(cliente, i) for i in range(n)]
        concurrent.futures.wait(futuros)
```

Fíjate en un detalle: el propio lanzador usa un `ThreadPoolExecutor` con `n` hilos para lanzar los `n` clientes en paralelo. El cronometraje (los `inicios` y el tiempo total) lo añade quien ejecute la

prueba, usando `time.time()` antes y después.

📊 Los resultados

Con un servidor que tarda **2 segundos por cliente**, la tabla comparativa queda así:

Enfoque	10 clientes (2s c/u)	Fórmula
🐢 Secuencial	~20 segundos	n × tiempo_por_cliente
🚀 Hilos	~2 segundos	max(tiempo_por_cliente)
⚡ ThreadPool (5 hilos)	~4 segundos	ceil(n/workers) × tiempo_por_cliente

> El ThreadPool con 5 hilos tarda el doble que hilos ilimitados, pero **no ahoga el sistema**. Con 1000 clientes, hilos ilimitados matarían el PC; el pool encola y sirve de a 5.

La fórmula del pool lo explica: con 10 clientes y 5 hilos, se atienden **2 tandas** de 5 (ceil(10/5) = 2), cada una de 2 segundos → ~4 segundos. El hilo por cliente lo hace en una sola tanda → ~2 segundos. Y el secuencial… 10 tandas de uno → ~20 segundos.

🚀 El lanzador masivo de clientes

Este es el script que probará tu servidor real, el "💉 Cliente de prueba — lanzador masivo":

```python
import socket, threading, time

def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"  ✅ Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"  ❌ Cliente-{id}: {e}")

print("Lanzando 10 clientes...")
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print("🏁 Todos los clientes terminaron")
```

> Este script prueba que tu servidor concurrente realmente atiende a varios clientes a la vez. Si es secuencial, los clientes 2-9 esperarán su turno.

Para medir el tiempo total, envuelve el bloque con `inicio = time.time()` antes de lanzar los hilos y `print(f"Tiempo total: {time.time() - inicio:.2f}s")` al final. Ese número es tu **benchmark**.

🔍 Cómo interpretar los resultados

Lo que ves	Qué significa
Tiempo ≈ n × tiempo_por_cliente	Tu servidor es **secuencial** (o el pool tiene 1 hilo)
Tiempo ≈ max(tiempo_por_cliente)	Tu servidor es **concurrente** (hilos o pool con suficientes workers)

Lo que ves	Qué significa
Tiempo ≈ ceil(n/workers) × tiempo_por_cliente	Tu **pool** está trabajando en tandas
Errores ❌ `ConnectionRefused`	El servidor no está arrancado o el puerto es otro

> 💡 El benchmark también sirve para **elegir el tamaño del pool**: sube `max_workers` y mira dónde deja de mejorar el tiempo. Más hilos no siempre es más rápido (punto 7).

🧠 Mini-chequeo

1. ¿Por qué el lanzador usa `join()` sobre todos los hilos?
2. Un servidor con pool de 3 hilos recibe 9 clientes que tardan 1s cada uno. ¿Tiempo total aproximado?
3. ¿Qué le diría la salida del lanzador a un servidor que imprime “Procesado” de uno en uno?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- El benchmark lanza **N clientes en paralelo** y mide el tiempo total con `time.time()`.
- Secuencial ≈ n × tiempo; hilos ≈ max(tiempo); pool ≈ ceil(n/workers) × tiempo.
- El lanzador masivo demuestra si tu servidor atiende de verdad a varios clientes a la vez.

🐛 Vocabulario rápido

Término	Idea general
Benchmark	Prueba que mide el rendimiento del servidor
Lanzador masivo	Script que crea N clientes simultáneos
join()	Espera a que terminen todos los hilos
ceil(n/workers)	Número de tandas del pool (redondeo hacia arriba)

Término	Idea general
time.time()	Cronómetro para medir duraciones

📚 Volver al índice de la unidad · **Anterior:** 04 · ThreadPoolExecutor · **Siguiente:** 06 · Sincronización en servidores

06 — Sincronización en servidores

El Lock para el estado compartido entre hilos 🔒

> 🗺️ **Estás en:** 🧵 **UD 6 · Servidores concurrentes** → 06 · Sincronización en servidores

📬 La idea en una frase

> Cuando varios hilos del servidor tocan la **misma variable global** (un contador de conexiones, un total de bytes), hay **condición de carrera**: dos hilos pueden leer y escribir a la vez y perder actualizaciones. La solución es el **Lock** de la U04.

En el punto 3 y el punto 4 los hilos eran independientes. Pero un servidor real suele llevar **estado**: "¿cuántos clientes he atendido?", "¿cuántos bytes he recibido?". Ese estado es compartido… y eso es un problema.

💥 La condición de carrera en el servidor

Imagina un contador global de conexiones atendidas, sin protección:

```
import socket, threading

contador = 0  # ← variable global compartida

def atender(conn, addr):
    global contador
    contador += 1          # ⚠️ ¡no es atómico!
    print(f"Total atendidos: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
```

El problema es que `contador += 1` se ejecuta en **varios pasos**: leer el valor → sumar 1 → guardarlo. Dos hilos pueden leer el mismo valor (p. ej. 7) a la vez y los dos guardar 8: **una conexión se pierde**.

```
Hilo-A:  lee contador (=7)
Hilo-B:  lee contador (=7)   ← ¡mismo valor!
Hilo-A:  guarda 8
Hilo-B:  guarda 8            ← contador debía ser 9, quedó en 8 💥
```

> ⚠️ **Regla de oro:** si dos hilos del servidor modifican la misma variable, usa **Lock**. Mejor aún: **evita compartir estado** cuando puedas.

🔒 El Lock en acción

Con un `threading.Lock`, la sección crítica queda protegida:

```python
import socket, threading

contador = 0
lock = threading.Lock()

def atender(conn, addr):
    global contador
    with lock:                    # 🔒 nadie más puede entrar aquí
        contador += 1
        print(f"Total atendidos: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")

def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        while True:
            conn, addr = srv.accept()
            threading.Thread(target=atender, args=(conn, addr)).start()
```

`with lock:` adquiere el lock al entrar y lo libera al salir (aunque haya una excepción). Entre esas líneas, **un solo hilo a la vez**: los demás esperan su turno. La lectura, suma y guardado del contador ya son indivisibles.

> 💡 El `with lock:` del servidor es exactamente el patrón que ya conoces de la U04 · Sincronización: misma herramienta, distinto escenario.

🔍 ¿Y el contador de bytes?

El mismo patrón sirve para acumular cualquier valor global. Un contador de bytes totales recibidos:

```
import socket, threading

total_bytes = 0
lock = threading.Lock()

def atender(conn, addr):
    global total_bytes
    with conn:
        datos = conn.recv(1024)
    with lock:
        total_bytes += len(datos)          # 🔒 actualización atómica
        print(f"  {addr} envió {len(datos)} bytes — Total: {total_bytes}")
```

Nota: el `recv()` y el `sendall()` **no** necesitan lock (cada conexión es un socket distinto); solo lo necesita la **variable compartida**. Proteger de más (meter el recv dentro del lock) volvería el servidor secuencial sin querer.

🧠 Mini-chequeo

1. ¿Por qué `contador += 1` no es una operación segura entre hilos?
2. ¿Qué pasa si dos hilos incrementan el contador a la vez sin Lock?
3. En el servidor, ¿qué necesita Lock: el `recv()` del socket o la variable global?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- Los hilos del servidor compiten por las **variables globales** (contadores, totales) y eso causa condiciones de carrera.
- El **Lock** (`with lock:`) hace indivisible la actualización: un hilo a la vez en la sección crítica.
- Regla práctica: protege solo lo compartido; lo que es por conexión (recv/sendall) no necesita lock.

🐛 Vocabulario rápido

Término	Idea general
Condición de carrera	Dos hilos compiten por la misma variable y se pierden datos
Sección crítica	Código que toca estado compartido y necesita Lock
threading.Lock()	Cerradura para exclusión mutua entre hilos
with lock:	Adquiere el lock al entrar y lo libera al salir
Estado compartido	Variables globales que varios hilos leen/escriben

📚 Volver al índice de la unidad · **Anterior:** 05 · Benchmark · **Siguiente:** 07 · Límites y buenas prácticas

07 — Límites y buenas prácticas

Cuántos hilos puedes crear y cómo hacerlo bien 🛡️

> 🗺️ **Estás en:** 🏗️ **UD 6 · Servidores concurrentes** → 07 · Límites y buenas prácticas

📬 La idea en una frase

> Más hilos no es siempre mejor: el sistema operativo tiene límites, el cambio de contexto tiene coste y un hilo colgado puede fugarse de recursos. La buena práctica es **controlar los límites** y elegir la herramienta adecuada para cada escala.

Ya tienes las dos herramientas (hilo por cliente y ThreadPool). Este punto responde a las preguntas que te harás cuando el servidor vaya a producción: ¿cuántos hilos puedo crear? ¿qué pasa si uno se cuelga? ¿y si llegan 10.000 conexiones?

🔢 ¿Cuántos hilos puede tener un servidor?

Depende del SO. En Windows, unos pocos miles. Pero hay un matiz importante:

> Más allá de **~100 hilos**, el **cambio de contexto** (context switch) perjudica el rendimiento.

Cada vez que la CPU alterna entre hilos, pierde tiempo en guardar y restaurar el estado de cada uno. Con pocos hilos, ese coste es despreciable; con cientos, la CPU pasa más tiempo *cambiando de hilo* que *trabajando*. Por eso el ThreadPool del punto 4 limita `max_workers`: el rendimiento sube con los hilos… hasta un punto, y luego baja.

No de hilos	Efecto esperado
1-10	Sencillo, bajo consumo, poco paralelismo
~100	Punto de equilibrio razonable en un servidor de clase
Miles	Riesgo de agotar memoria y coste de context switch alto

🧟 ¿Qué pasa si un hilo se cuelga?

Ese hilo queda **bloqueado**, pero los demás siguen funcionando: la concurrencia te salva de un colapso total. El problema real es si el hilo colgado **no libera el socket** (fuga de recursos):

- Cada conexión abierta ocupa un **file descriptor** y memoria en el SO.
- Con suficientes fugas, el servidor agota los descriptores y **deja de aceptar** nuevas conexiones.
- La solución es garantizar el cierre: el `with conn:` que usamos en toda la unidad cierra el socket al salir del bloque, **aunque haya una excepción**.

> 🛡️ Si un hilo se queda eternamente esperando datos que nunca llegan, un **timeout** (`conn.settimeout()`) lo libera a los pocos segundos. Lo practicarás en el boletín avanzado.

🏭 ¿ThreadPool o hilo por cliente en producción?

ThreadPool siempre. Controlas cuántos hilos se crean y evitas saturar el SO. Hilo por cliente es **solo para prototipos**:

Criterio	Hilo por cliente	ThreadPool
Control de recursos	❌ Ninguno	✅ `max_workers` fijo
Escala a miles de conexiones	❌ Satura el sistema	✅ Encola y sirve de a N
Complejidad	✅ Simple	✅ Igual de simple (`submit()`)
Uso recomendado	Prototipos, clases	**Producción**

🌊 ¿Y si el servidor recibe 10.000 conexiones?

- Con hilo por cliente: 10.000 hilos → el sistema se colapsa.
- Con ThreadPool de **100 hilos + cola de espera**: el servidor sobrevive, aunque los clientes esperen turno.
- La alternativa que **escala mejor** es **asyncio** (la verás en la UD 10 · Alta disponibilidad): atiende miles de conexiones sin un hilo por conexión, con un bucle de eventos muy ligero.

🔀 ¿Puedo mezclar hilos y asyncio?

Sí, con `loop.run_in_executor()`. Pero como principiante, la recomendación es clara: **empieza con uno u otro**. Los dos modelos (hilos y asyncio) son potentes por separado y confusos juntos. Cuando domines asyncio en la UD 10, podrás combinar bloqueos de bibliotecas de terceros (que no son asíncronas) usando el ejecutor.

✅ Guía rápida de buenas prácticas

- **Limita los hilos**: usa `ThreadPoolExecutor(max_workers=N)` en producción, nunca hilos ilimitados.
- **Cierra siempre la conexión**: `with conn:` libera el socket aunque haya errores.
- **Protege el estado compartido** con `Lock` (punto 6); mejor aún, evita compartir estado.
- **Aguanta clientes que se caen**: un cliente puede morir a mitad de conversación y matar el hilo con un `ConnectionResetError` o `BrokenPipeError`. Envuelve el `recv/sendall` del `atender()` en un `try/except`:

```
def atender(conn, addr):
    with conn:
        try:
            datos = conn.recv(1024)
            conn.sendall(b"OK: " + datos)
        except (ConnectionResetError, BrokenPipeError):
            print(f"Cliente {addr} se desconectó a mitad")
        except OSError:
            print(f"Error de red con {addr}")
```

Sin ese `try/except`, el hilo de ese cliente muere con un traceback (el servidor sobrevive, pero pierde el hilo y ensucia la consola).

- **Usa timeouts** en `recv()` para que un cliente mudo no cuelgue un hilo para siempre.
- **Mide con el benchmark** (punto 5) antes de tocar `max_workers`: los datos mandan, no las corazonadas.
- **El pool se crea una vez**, fuera del bucle `accept()` (punto 4).

🧠 Mini-chequeo

1. ¿Por qué “miles de hilos” puede ser *peor* que “100 hilos” aunque la CPU tenga muchos núcleos?
2. Un hilo se cuelga sin liberar su socket. ¿Qué les pasa a los demás? ¿Y al servidor a largo plazo?
3. Recibes 10.000 conexiones simultáneas. ¿Qué herramienta usarías y por qué?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Los hilos tienen **límites del SO** y el **context switch** penaliza por encima de ~100.
- Un hilo colgado no tumba al servidor, pero **fugarse de sockets** sí lo ahoga con el tiempo.
- Regla de producción: **ThreadPool con límite + timeouts + `with conn:` + benchmark**.

🐛 Vocabulario rápido

Término	Idea general
Context switch	Cambio de hilo de la CPU: necesario, pero caro
Fuga de recursos	Socket o memoria que nunca se libera
settimeout()	Límite de tiempo para `recv()`: evita hilos colgados
max_workers	Límite de hilos del pool (tu freno de mano)
asyncio	Modelo de concurrencia ligera para miles de conexiones (UD 10)

📚 Volver al índice de la unidad · **Anterior:** 06 · Sincronización en servidores · **Siguiente:** 08 · Servidor concurrente completo con hilos

08 — Servidor concurrente completo con hilos

Todo junto: servidor multihilo, cliente interactivo y lanzador de clientes 🧪

> 🗺️ **Estás en:** 🧪 **UD 6 · Servidores concurrentes** → 08 · Servidor concurrente completo con hilos

---

📬 La idea en una frase

> Este punto junta toda la unidad en tres scripts listos para ejecutar: un **servidor concurrente** (`servidor.py`) que atiende a cada cliente en su hilo con un contador protegido por Lock, un **cliente interactivo** (`cliente.py`) para hablar con él a mano, y un **lanzador de clientes** (`lanzaclientes.py`) que lo pone a prueba con N conexiones a la vez.

Es el "Be the code" definitivo: aquí tienes el código completo, cómo ejecutarlo en dos terminales y los ejercicios de "Aprieta el lápiz" para cerrar la teoría.

🧪 El servidor: `servidor.py`

Un servidor TCP multihilo con contador de conexiones protegido por Lock (el punto 3 + el punto 6):

```python
import socket, threading

contador = 0
lock = threading.Lock()
num_hilo = 0   # contador monótono para nombrar hilos de forma única

def atender(conn, addr):
    global contador
    with lock:
        contador += 1
        print(f"[+] Cliente {addr} conectado — total: {contador}")
    with conn:
        try:
            # Bucle de mensajes: la conversación dura hasta "SALIR" o cierre
            while True:
                datos = conn.recv(1024)
                if not datos:          # El cliente ha cerrado: b"" en recv
                    break
                mensaje = datos.decode().strip()
                print(f"    Recibido de {addr}: {mensaje}")
                conn.sendall(b"OK: " + datos)
                if mensaje.upper() == "SALIR":
                    break
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"    Cliente {addr} se desconectó a mitad")
    with lock:
        contador -= 1
        print(f"[-] Cliente {addr} desconectado — total: {contador}")

def servidor_concurrente():
    global num_hilo
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🏗️  Servidor CONCURRENTE escuchando en 127.0.0.1:5000")
        while True:
            conn, addr = srv.accept()
            hilo = threading.Thread(target=atender, args=(conn, addr))
            hilo.name = "hilo-" + str(num_hilo)
            num_hilo += 1
            hilo.start()
```

```
if __name__ == "__main__":
    servidor_concurrente()
```

Lo que demuestra este servidor:

- **Concurrencia**: `accept()` → crear hilo → `start()` → volver a aceptar (punto 3).
- **Sincronización**: el contador global solo se toca dentro de `with lock:` (punto 6).
- **Limpieza**: `with conn:` cierra el socket sí o sí (punto 7).
- **Convención**: punto de entrada con `if __name__ == "__main__":` y hilos nombrados con `.name`.

💬 El cliente: `cliente.py`

Antes de disparar clientes masivos, conviene tener un cliente “de una sola persona” con el que hablar a mano. Este cliente interactivo se registra, deja mensajes por consola y cierra limpio:

```python
import socket

def cliente():
    nombre = input("Tu nombre de usuario: ").strip() or "anónimo"
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"REGISTRO:{nombre}".encode())
            print(f"📩 {s.recv(1024).decode().strip()}")

            # Bucle de conversación con el servidor
            while True:
                texto = input("tú> ").strip()
                if not texto:
                    continue
                s.sendall(texto.encode())
                respuesta = s.recv(1024).decode().strip()
                print(f"servidor> {respuesta}")
                if texto.upper() == "SALIR":
                    print("👋 Conexión cerrada")
                    break
    except ConnectionRefusedError:
        print("❌ Servidor no disponible. ¿Está arrancado en la terminal 1?")
    except (ConnectionResetError, BrokenPipeError):
        print("\n🔴 El servidor ha cerrado la conexión")

if __name__ == "__main__":
    cliente()
```

Traza del cliente:

1. **`input()`** pide el nombre (si no hay nada, usa `"anónimo"`).
2. **`socket()` → `connect()`** abre la conexión TCP al puerto 5000. Si el servidor no está, salta `ConnectionRefusedError` y el mensaje amable.
3. **`sendall(f"REGISTRO:{nombre}")`** envía el registro y espera la respuesta con `recv(1024)`.
4. **Bucle**: cada línea que escribes se envía con `sendall()` y la respuesta llega con `recv(1024)`. `SALIR` cierra desde el cliente.
5. **`with socket`** se asegura de cerrar la conexión al salir (por cualquier vía).

> 💡 `recv(1024)` devuelve `b""` (bytes vacíos) cuando el otro extremo ha cerrado: por eso el servidor comprueba `if not datos` antes de intentar decodificar. El servidor del punto 8

> responde `OK: <datos>` a cualquier mensaje y cierra al recibir `SALIR`; el tratamiento de registros "de verdad" (con `manager.dict()`) aparece en el punto 9.

🚀 El lanzador: `lanzaclientes.py`

Un script que lanza 10 clientes simultáneos contra el servidor (el lanzador masivo del punto 5):

```
import socket, threading, time

def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"  ✅ Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"  ❌ Cliente-{id}: {e}")

print("Lanzando 10 clientes...")
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print("🏁 Todos los clientes terminaron")
```

> 💡 Para convertirlo en benchmark, envuelve el bloque con `inicio = time.time()` al principio y `print(f"Tiempo total: {time.time() - inicio:.2f}s")` al final.

🤝 Cómo probarlos juntos

```
TERMINAL 1 (servidor)          TERMINAL 2 (lanzador)
_____________________          _____________________

python servidor.py             python lanzaclientes.py
🏗️  Servidor CONCURRENTE…       Lanzando 10 clientes...
[+] Cliente ('127.0.0.1', …)     ✅ Cliente-0: OK: Cliente-0
[+] Cliente ('127.0.0.1', …)     ✅ Cliente-1: OK: Cliente-1
    Recibido: Cliente-0           … (todos responden)
    Recibido: Cliente-1          🏁 Todos los clientes terminaron
```

Si el servidor fuera secuencial, el lanzador tardaría 10 × tiempo_por_cliente y las respuestas llegarían de una en una. Con el servidor concurrente, **todas responden casi a la vez**: ese es el resultado del punto 5. En lugar del lanzador, también puedes usar la terminal 2 para arrancar `cliente.py` y hablar con el servidor a mano. Se mata el servidor con **Ctrl+C** y cada script se ejecuta en su terminal.

🪛 Aprieta el lápiz

1. **Servidor secuencial → hilos**: Convierte un servidor TCP secuencial en uno multihilo.
2. **ThreadPool**: Implementa el mismo servidor con ThreadPoolExecutor de 5 hilos.
3. **Lanzador de clientes**: Crea un script que lance 10 clientes simultáneos para probar tu servidor.
4. **Contador de conexiones**: El servidor debe mostrar cuántos clientes han sido atendidos (usa Lock para la variable compartida).

> 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Por qué el contador se incrementa y decrementa dentro de `with lock:`?
2. ¿Qué pasa en el lanzador si el servidor no está arrancado?
3. ¿Dónde se nombra el hilo y con qué valor?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- `servidor.py` combina **hilo por cliente + Lock + `with conn:`** en un servidor de producción sencillo.
- `lanzaclientes.py` prueba que atiende a **N clientes a la vez** y sirve de benchmark.
- Los ejercicios "Aprieta el lápiz" cubren las dos variantes (hilos y pool) y el contador sincronizado.

🐛 Vocabulario rápido

Término	Idea general
servidor.py	El servidor concurrente completo de la unidad
cliente.py	El cliente interactivo para hablar con el servidor a mano
lanzaclientes.py	El script que dispara N clientes simultáneos
hilo.name	Nombre del hilo para trazarlo ("hilo-1", "hilo-2"…)
if **name**	Punto de entrada del script
Ctrl+C	Cómo se mata el servidor en pruebas

📚 Volver al índice de la unidad · **Anterior:** 07 · Límites y buenas prácticas · **Siguiente:** 09 · Servidor concurrente completo con procesos

09 — Servidor concurrente completo con procesos

El mismo servidor de la unidad, pero atendiendo con procesos independientes en lugar de hilos 🧱

> 🗺️ **Estás en:** 🧬 **UD 6 · Servidores concurrentes** → 09 · Servidor concurrente completo con procesos

---

📬 La idea en una frase

> Este punto monta el servidor completo de la unidad, pero sustituyendo los hilos por **`multiprocessing.Process`**: cada cliente lo atiende un **proceso independiente** con su propia burbuja de memoria, el estado se comparte con **`manager.dict()`** y se protege con **`multiprocessing.Lock`**, y lo pruebas con un **cliente interactivo** (`cliente.py`) y con un **lanzador masivo** (`lanzaclientes.py`).

Es la pieza que faltaba para unir la red con los procesos: aquí conviven `socket`, `Process`, `Manager` y `Lock` en un mismo programa, que es exactamente el patrón de un servidor con aislamiento de fallos.

🤔 ¿Por qué procesos y no hilos?

El servidor de los [puntos 3 a 8] usa hilos, y funciona muy bien. Pero los procesos tienen tres ventajas que a veces pesan más:

	Hilo	Proceso
Memoria	Compartida: un bug puede corromper el estado de todos	Aislada: cada uno vive en su burbuja
Fallo	Si un hilo trumba sin querer, puede tumbar el proceso entero	Si un proceso muere, **el servidor sigue vivo**
Coste	Muy ligero (milisegundos)	Más pesado (clona la memoria)
Escalado	Limitado por el GIL de Python	Aprovecha varios núcleos de verdad

Cuando el enunciado de una práctica pide **aislamiento de fallos**, es esta última columna: cada cliente atendido en su propio proceso puede morir, bloquearse o lanzar una excepción sin arrastrar al resto del servidor.

🪜 El servidor: `servidor.py`

Servidor TCP donde cada conexión la atiende un `multiprocessing.Process`, con el estado en un `manager.dict()` protegido por candado:

```python
import socket
import multiprocessing

def atender(conn, addr, estado, candado):
    """Proceso hijo: atiende a UN cliente con su propia memoria."""
    with conn:
        try:
            # Bucle de mensajes: la conversación dura hasta que el cliente
            # se desconecte o envíe "SALIR"
            while True:
                datos = conn.recv(1024)
                if not datos:          # El cliente ha cerrado: b"" en recv
                    break
                mensaje = datos.decode().strip()
                print(f"  [pid {multiprocessing.current_process().pid}] "
                      f"Recibido de {addr}: {mensaje}")

                if mensaje.upper() == "SALIR":
                    conn.sendall(b"Adios 👋")
                    break

                # Sección crítica: leer y actualizar el estado compartido
                with candado:
                    estado["atendidos"] = estado.get("atendidos", 0) + 1
                    estado["ultimo"] = mensaje
                    total = estado["atendidos"]

                conn.sendall(f"OK [{total}]: {mensaje}".encode())
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"  [pid {multiprocessing.current_process().pid}] "
                  f"Cliente {addr} se desconectó a mitad")

def servidor_con_procesos():
    with multiprocessing.Manager() as manager:
        # Estado compartido entre TODOS los procesos (memoria en común)
        estado = manager.dict({"atendidos": 0, "ultimo": None})
        # Candado de PROCESOS: solo uno a la vez toca el estado
        candado = multiprocessing.Lock()

        with socket.socket() as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            srv.bind(("127.0.0.1", 5000))
```

```
            srv.listen()
            print("🏭  Servidor por PROCESOS escuchando en 127.0.0.1:5000")

            while True:
                conn, addr = srv.accept()
                # Cada cliente, su propio proceso (no su propio hilo)
                p = multiprocessing.Process(
                    target=atender,
                    args=(conn, addr, estado, candado),
                    name=f"cliente-{addr[1]}",
                )
                p.start()
                # IMPORTANTE: el padre cierra SU copia del socket.
                # El hijo recibió una copia al ser serializado como argumento;
                # si el padre no la cierra, el canal sigue "vivo" en su mano
                # y un cliente que espere respuesta podría colgarse.
                conn.close()
                print(f"[+] Cliente {addr} → proceso {p.name} (pid {p.pid})")

if __name__ == "__main__":
    try:
        servidor_con_procesos()
    except KeyboardInterrupt:
        print("\n🛑 Servidor detenido")
```

Pieza por pieza:

- **`manager.dict()`** es el “estado del servidor”: todos los procesos hijos ven y actualizan el mismo diccionario, aunque cada uno tenga su burbuja de RAM. Sin él, cada hijo tendría su propio contador y nunca llegarían a un total común.
- **`multiprocessing.Lock()`** protege la sección crítica (`leer → sumar → escribir`) dentro de `with candado:`: dos procesos no pueden incrementar `atendidos` a la vez o perderíamos conexiones.
- **`Process(target=atender, args=(conn, addr, estado, candado))`** lanza el proceso hijo con todo lo que necesita. El socket `conn` se lo pasas como argumento: multiprocessing lo serializa y el hijo lo recibe listo para `recv`/`sendall`.
- **El bucle `while True` con `recv`** mantiene viva la conversación: solo se sale con `b""` (el cliente ha cerrado) o con el mensaje `SALIR`. Un `recv` suelto cerraría la conexión tras el primer mensaje.

- **`if __name__ == "__main__":`** es obligatorio: en Windows no hay `fork`, cada hijo vuelve a importar el script desde el principio y sin este guard entraría en un bucle infinito de procesos.
- **`conn.close()` tras `p.start()`** es la pieza que casi todo el mundo olvida: al pasar `conn` como argumento, el hijo recibe **su propia copia** del socket. Si el padre no cierra la suya, el canal permanece abierto en sus manos y un cliente que espere respuesta puede quedarse colgado aunque el hijo haya muerto.
- **`with conn:`** cierra el socket del hijo sí o sí, igual que en el servidor con hilos del punto 8.

💬 El cliente: `cliente.py`

Cliente interactivo en Python: te registra, te deja hablar con el servidor por consola y termina limpio cuando se desconecta:

```python
import socket

def cliente():
    nombre = input("Tu nombre de usuario: ").strip() or "anónimo"
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"REGISTRO:{nombre}".encode())
            print(f"📩 {s.recv(1024).decode().strip()}")

            # Bucle de conversación con el servidor
            while True:
                texto = input("tú> ").strip()
                if not texto:
                    continue
                s.sendall(texto.encode())
                respuesta = s.recv(1024).decode().strip()
                print(f"servidor> {respuesta}")
                if texto.upper() == "SALIR":
                    print("👋 Conexión cerrada")
                    break
    except ConnectionRefusedError:
        print("❌ Servidor no disponible. ¿Está arrancado en la terminal 1?")
    except (ConnectionResetError, BrokenPipeError):
        print("\n🔴 El servidor ha cerrado la conexión")

if __name__ == "__main__":
    cliente()
```

Traza del cliente:

1. **`input()`** pide el nombre (si no hay nada, usa `"anónimo"`).
2. **`socket()` → `connect()`** abre la conexión TCP al puerto 5000. Si el servidor no está, salta `ConnectionRefusedError` y el mensaje amable.
3. **`sendall(f"REGISTRO:{nombre}")`** se registra; el servidor lo guarda en su `manager.dict()` y responde con `recv(1024)`.
4. **Bucle**: cada línea que escribes se envía con `sendall()` y la respuesta llega con `recv(1024)`. `SALIR` cierra desde el cliente.
5. **`with socket`** se asegura de cerrar la conexión al salir (por cualquier vía).

> 💡 `recv(1024)` devuelve `b""` (bytes vacíos) cuando el otro extremo ha cerrado: por eso el servidor comprueba `if not datos: return` antes de intentar decodificar.

🚀 El lanzador: `lanzaclientes.py`

Para probar que el servidor atiende a muchos a la vez (la versión con procesos del lanzador del punto 8):

```
import socket
import threading
import time

def cliente(i):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"cliente-{i}".encode())
            print(f"  ✅ cliente-{i}: {s.recv(1024).decode().strip()}")
    except Exception as e:
        print(f"  ❌ cliente-{i}: {e}")

print("Lanzando 10 clientes...")
inicio = time.time()
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print(f"🏁 Todos terminaron en {time.time() - inicio:.2f}s")
```

> 💡 El lanzador usa hilos **del lado del cliente** para disparar las conexiones a la vez: no es el servidor, no afecta al aislamiento de fallos que pide la práctica.

🤝 Cómo probarlos juntos

```
TERMINAL 1 (servidor)      TERMINAL 2 (cliente)      TERMINAL 3 (lanzador)
_____________________      _____________________     _____________________

python servidor.py         python cliente.py         python lanzaclientes.py
🏗️  Servidor por PROCESOS…  Tu nombre de usuario: Ana  Lanzando 10 clientes...
[+] Cliente ('127.0.0.1,   Recibido: OK [1]:         ✅ cliente-0: OK [2]: cliente-0
    52341) → proceso         REGISTRO:Ana            ✅ cliente-1: OK [3]: cliente-1
    cliente-52341 (pid 8123) servidor> OK [2]: hola  … (todos responden)
    [pid 8123] Recibido…     servidor> OK [4]: adios 🏁 Todos terminaron en 0,15s
                             Conexión cerrada
```

Fíjate en dos cosas:

- Cada línea del servidor muestra **un PID distinto**: es el proceso hijo que atiende a ese cliente.
- El contador sube **compartido**: los 10 clientes del lanzador y el tuyo de la terminal 2 suman en el mismo `manager.dict()` gracias al candado (en la salida de arriba, el cliente interactivo ve cómo el contador salta de [1] a [4] porque los clientes del lanzador están atendiendo a la vez).

Se mata el servidor con **Ctrl+C**.

📝 Aprieta el lápiz

1. **Añade el nombre al estado**: guarda en el `manager.dict()` qué nombres se han registrado (`estado["registrados"]`) y responde `STATUS` con la lista.
2. **Aísla un fallo**: haz que un cliente envíe “CRASH” y provoques una excepción dentro del proceso hijo; comprueba que el servidor sigue atiendo al resto.
3. **Recoge a los hijos**: guarda los objetos `Process` en una lista y haz `join()` a los que hayan terminado para no dejar procesos zombis.

> 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Qué le pasa al servidor si un proceso hijo muere por una excepción?
2. ¿Por qué el estado va en `manager.dict()` y no en un dict normal del proceso padre?
3. ¿Por qué el bucle `accept()` vive en el padre y el `recv()` en el hijo?

> 🔁 Respuestas

✅ Resumen en 3 frases

- `servidor.py` atiende cada conexión con un `multiprocessing.Process` en lugar de un hilo, logrando aislamiento de fallos: un hijo que muere no tumba al servidor.
- El estado compartido vive en `manager.dict()` y se protege con `multiprocessing.Lock`, igual que el almacén del ejemplo resuelto de la UD 2, pero ahora cruzando la capa de red.
- `cliente.py` (interactivo) y `lanzaclientes.py` (masivo) prueban el servidor desde dos terminales distintas, mostrando PIDs distintos por cliente.

🐛 Vocabulario rápido

Término	Idea general
`Process(target=atender)`	Proceso hijo que atiende a un cliente
`manager.dict()`	Estado del servidor compartido entre procesos
`multiprocessing.Lock`	Candado que evita carreras al tocar el estado
Aislamiento de fallos	Un proceso que muere no afecta al resto
`current_process().pid`	PID del hijo, para trazar quién atiende a quién
`if __name__`	Guard obligatorio en Windows (spawn) para no crear bucles de procesos

📚 Volver al índice de la unidad · **Anterior:** 08 · Servidor concurrente completo con hilos · **Siguiente:** 10 · Cierre

10 — Cierre: consolida lo aprendido

Sé el servidor, el ring final y el Laboratorio de tortura 🎢

> 🗺️ **Estás en:** 🎢 **UD 6 · Servidores concurrentes** → 10 · Cierre

Has terminado la teoría: el servidor secuencial y su límite, el cliente lento que bloquea la cola, el hilo por cliente, el ThreadPoolExecutor, el benchmark, el Lock para el estado compartido, las buenas prácticas y los dos servidores completos (con hilos y con procesos). Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 9 y antes de abrir los boletines.

⭐ Sé el servidor

> *Eres un servidor TCP en `127.0.0.1:5000` con un ThreadPoolExecutor de 5 hilos. De repente llegan 10 clientes a la vez.*

¿Qué pasa?

1. Tu bucle principal hace `accept()` y recibe la **conexión-1**: `pool.submit(atender, conn1, addr1)`.
2. La tarea entra en el pool: **hilo-1 libre** la coge al instante y empieza a procesar.
3. Vuelves a `accept()` y recibes la **conexión-2**, **conexión-3**, … hasta la **conexión-5**: cada una ocupa un hilo libre (hilo-1 a hilo-5).
4. Llega la **conexión-6**: los 5 hilos están ocupados, así que `submit()` la **encola** internamente.
5. En cuanto hilo-1 termina con la conexión-1 (responde y cierra con `with conn:`), el pool **reutiliza** ese hilo para la conexión-6.
6. Así hasta la conexión-10: **dos tandas de 5**, el `ceil(10/5)` del punto 5.

Nunca se crea un hilo nuevo. El servidor tiene exactamente 5 hilos y una cola: el sistema no se ahoga.

> 💡 **Ahora tú:** ¿y si llegaran 10.000 conexiones? El pool las iría encolando y sirviendo de a 5. Tardarían más, pero el servidor **sobreviviría**. Y para 10.000 conexiones, la solución de la UD 10 · asyncio sería todavía más ligera.

🔥 Fireside Chat: Hilo por cliente vs ThreadPool

> *Dos enfoques de concurrencia se sientan junto a la chimenea a resolver, de una vez, quién sirve mejor a los clientes.*

HiloPorCliente: — Soy el enfoque clásico. Llega un cliente, creo un hilo, lo atiendo, el hilo muere. Sencillo y directo.

ThreadPool: — ¿Y si llegan 1000 clientes? Creas 1000 hilos y el sistema se colapsa. Yo tengo un número fijo de hilos, como un equipo de camareros limitado.

HiloPorCliente: — Pero cada cliente tiene su hilo dedicado. No esperan. Es más justo.

ThreadPool: — Justo pero ineficiente. Con mi pool, los clientes esperan un poco pero el sistema no se ahoga. Además, reutilizo hilos, evitando el coste de crearlos y destruirlos constantemente.

HiloPorCliente: — Para pocos clientes simultáneos, soy más simple de implementar.

ThreadPool: — Y yo escalo mejor. Para servidores en producción, soy la opción sensata.

> **Moraleja**: Hilo por cliente es simple y funciona para pocos clientes. ThreadPool escala mejor y controla los recursos. En producción, usa ThreadPool.

🕵️‍♂️ ¿Quién soy?

1. Soy el `accept()` que se queda atascado mientras se procesa al cliente anterior.
2. Soy la llamada que entrega la tarea al pool y devuelve un Future.
3. Soy la analogía del servidor secuencial: una sola persona atiende a toda la cola.
4. Soy la variable global que dos hilos pueden pisar si no me protegen con Lock.
5. Soy lo que mide cuánto tarda en responder un servidor con N clientes simultáneos.
6. Soy la fórmula del pool: te digo en cuántas tandas se atienden N clientes con W hilos.

> › 🔁 **Respuestas**

👹 CONRAD VS EL MUNDO: “un cliente lento bloquea el servidor”

CONRAD: — "Clásico: montas un servidor secuencial, llega un cliente que tarda 30 segundos y *'el servidor se ha quedado colgado'*. Pues no: solo tiene **una ventanilla**. Mientras procesa al lento, el `accept()` no se ejecuta y toda la cola espera. Los clientes 2 al 10 ya están conectados al socket… pero nadie los atiende."

CONRAD: — "Y lo mejor: *'he puesto time.sleep(3) y las pruebas van lentas'*. ¡Pues claro! Cada cliente suma 3 segundos. Con 10 clientes, el último espera 30. **n × tiempo_por_cliente**, lo dijimos en el punto 1. Si quieres que todos terminen a la vez, necesitas hilos o un pool."

CONRAD: — "Y no me vengas con *'¿será que el puerto está ocupado?'*. El puerto va bien: lo que pasa es que el servidor atiende de uno en uno. Lanza el **lanzador de clientes** del punto 5 y míralo en el log: si las respuestas llegan de una en una, tienes un secuencial. A diagnosticar."

⚡ Laboratorio de tortura: servidor concurrente con ThreadPool

> **Duración:** 45 minutos **Herramienta:** Python 3 (solo stdlib: `socket`, `threading`, `concurrent.futures`)

Escenario: construye un servidor TCP en `127.0.0.1:5000` que responda `"OK: " + datos` a cada cliente, usando **ThreadPoolExecutor con `max_workers=3`**. Acompáñalo de un lanzador que dispare 10 clientes simultáneos y cronometre el total.

Tareas paso a paso:

1. Escribe `atender(conn, addr)` con `with conn:`: recibe con `recv(1024)` y responde con `sendall(b"OK: " + datos)`.
2. Monta el servidor con `with socket.socket() as srv, ThreadPoolExecutor(max_workers=3) as pool:` (patrón del punto 4).
3. En el bucle: `conn, addr = srv.accept()` → `pool.submit(atender, conn, addr)`.
4. Escribe el lanzador de 10 clientes con `threading.Thread` y mide el tiempo total con `time.time()` (punto 5).
5. Añade un **contador global de conexiones activas** protegido con `Lock` (punto 6) que se imprima con cada `[+]`/`[-]`.
6. Ejecuta servidor y lanzador en terminales separadas. El servidor se mata con **Ctrl+C**.

Fallo intencionado: cambia `max_workers=3` por **`max_workers=1`**. ¿Qué pasa? El pool se convierte en un servidor **secuencial de facto**: una sola tarea a la vez, las otras 9 en cola. El tiempo total

pasa de ~1 tanda a **10 tandas**: con clientes que tardan 2s, el lanzador marca ~20s en lugar de ~8s (ceil(10/3)=4 tandas × 2s).

> **Pista 1:** el problema no está en el código del cliente ni del servidor: está en el **tamaño del pool**. Con `max_workers=1`, el ThreadPool se comporta como el servidor secuencial del punto 1: el tiempo total es `n × tiempo_por_cliente`.
>
> **Pista 2:** usa la fórmula del punto 5: `ceil(n/workers) × tiempo_por_cliente`. Con workers=1, `ceil(10/1) = 10` tandas. Con workers=3, `ceil(10/3) = 4`. La diferencia de tiempo te delata la configuración.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Cola Crusher**	Explicar por qué un cliente lento bloquea a todo un servidor secuencial
🏅 **Thread Spawner**	Montar un servidor multihilo que lanza un hilo por cada cliente
🏅 **Pool Master**	Implementar un servidor con ThreadPoolExecutor y elegir el tamaño del pool
🏅 **Benchmarker**	Medir con un lanzador masivo la diferencia entre secuencial, hilos y pool
🏅 **Lock Guardian**	Proteger con Lock el contador de conexiones de un servidor concurrente

---

🧠 Atrévete a pensar

1. ¿Por qué un servidor secuencial “acepta” a los clientes pero no los atiende?
2. ¿Qué ventaja real tiene un pool de 5 hilos frente a 5 hilos creados y destruidos por cliente?
3. ¿Cuándo vale la pena sacrificar velocidad (esperar en cola del pool) por estabilidad?
4. ¿Por qué el `recv()`/`sendall()` de cada conexión no necesita Lock, pero el contador global sí?

5. ¿Qué problemas aparecerían si lanzaras 100.000 hilos contra tu servidor?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Atiende a un cliente cada vez, el enemigo de la concurrencia (10 letras)
4. Hilos por conexión, el patrón del punto 3 (4 letras)
6. Equipo fijo de hilos reutilizables (4 letras)
8. Lo que hace el bucle principal tras lanzar un hilo (6 letras)

Vertical:
2. Protege la variable compartida entre hilos (4 letras)
3. Prueba que lanza N clientes y mide el tiempo (9 letras)
5. Método que entrega una tarea al pool (6 letras)
7. Se libera con with conn: aunque haya excepción (6 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **"¿Qué es un servidor concurrente y en qué se diferencia de uno secuencial?"**
2. **"¿Cómo implementarías un servidor que atienda a 500 clientes a la vez?"**
3. **"Hilo por cliente o ThreadPoolExecutor: ¿cuál elegirías y por qué?"**
4. **"¿Qué es una condición de carrera? Pon un ejemplo en un servidor."**
5. **"¿Cómo medirías si tu servidor es realmente concurrente?"**

> 💡 **Cómo encararlas:** la 1 y la 2 son las "preguntas reina". Para la 1, recorre la cadena de la unidad: secuencial → `n × tiempo` → cliente lento bloquea → hilo por cliente → pool. Para la 2, plantea ThreadPoolExecutor con un `max_workers` razonable, `submit()` en el bucle de `accept()`, `with conn:` y un contador con Lock. Si sabes contarlo fluido y defiendes por qué **no** lanzarías 500 hilos, ya eres medio programador de servidores.

🤷 No hay preguntas tontas

> ❓ **¿Cuántos hilos puede tener un servidor?**

Depende del SO. En Windows, unos pocos miles. Pero más allá de ~100, el cambio de contexto (context switch) perjudica el rendimiento (punto 7).

> ❓ **¿Qué pasa si un hilo se cuelga?**

Ese hilo queda bloqueado, pero los demás siguen. El problema es si no libera el socket (fuga de recursos).

> ❓ **¿ThreadPool o hilo por cliente en producción?**

ThreadPool siempre. Controlas cuántos hilos se crean. Hilo por cliente solo para prototipos.

> ❓ **¿Es seguro compartir variables globales entre hilos del servidor?**

No sin Lock. Si dos hilos modifican la misma variable, usa Lock. Mejor aún: evita compartir estado.

> ❓ **¿Puedo mezclar hilos y asyncio?**

Sí, con `loop.run_in_executor()`. Pero empieza con uno u otro.

> ❓ **¿Y si el servidor recibe 10.000 conexiones?**

ThreadPool con 100 hilos + cola de espera. O usa **asyncio** (UD 10) que escala mejor.

> ❓ **¿Se puede atender a los clientes con procesos en vez de hilos?**

Sí, y es el punto 9: cada conexión la atiende un `multiprocessing.Process`. Cuesta más memoria, pero ganas **aislamiento de fallos**: si un hijo muere, el servidor sigue vivo.

> ❓ **¿Y qué le pasa a mi servidor si un cliente lo hace tronar?**

Con procesos, nada: el hijo muere con su burbuja y el bucle `accept()` del padre acepta al siguiente cliente. Con hilos compartidos, la excepción puede tumbar el proceso entero. Por eso la práctica pide procesos cuando prohíbe hilos.

> ❓ **¿Por qué hay que cerrar `conn` en el padre?**

Al pasar el socket al proceso hijo, este recibe **su propia copia**. Si el padre no cierra la suya, el canal sigue abierto en sus manos y un cliente que espere respuesta puede quedarse colgado aunque el hijo haya muerto. Tras `p.start()` va `conn.close()`.

> ❓ **¿El `cliente.py` es obligatorio?**

No, pero sin él solo puedes probar con el lanzador masivo. El cliente interactivo te deja hablar a mano con el servidor (registro, mensajes, `SALIR`) y ver cómo sube el contador compartido en tiempo real.

🎬 Poscréditos

> *El servidor secuencial hacía esperar a todos en la cola de la ventanilla única.*
>
> *Llegó el hilo por cliente y cada conexión tuvo su propia ventanilla.*
>
> *El ThreadPool puso límites al entusiasmo: cinco camareros, cola ordenada, nadie se ahoga.*
>
> *El Lock protegió el contador. El benchmark demostró la mejora. Los clientes responden a la vez.*
>
> *El servidor ya atiende a muchos. Pero 10.000 conexiones siguen esperando algo más ligero…*

PRÓXIMAMENTE EN UD 7: *HTTP y APIs REST. Sin un hilo por conexión, un solo bucle de eventos atenderá miles de clientes a la vez… pero antes toca hablar el idioma de la web: peticiones, respuestas y JSON.*

✅ Criterios de evaluación cubiertos (RA4c-d)

RA4: Implementa servicios en red, desarrollando servidores concurrentes capaces de atender a varios clientes simultáneamente.

CE	Criterio	Cubierto
c)	Implementa servidores concurrentes con hilos	✅ Hilo por cliente (puntos 3 y 8) + ⚡ Laboratorio
d)	Gestiona pools de hilos (ThreadPoolExecutor)	✅ ThreadPool y benchmark (puntos 4-5) + ⚡ Laboratorio
+)	Ampliación: atender clientes con procesos aislados	✅ Servidor con procesos (punto 9)

> RA4a-b (APIs REST y comerciales) se cubren en las **UD 7 y UD 8**. RA4e-g (asyncio, disponibilidad, comparativa de modelos) se cubren en la **UD 10 · Alta disponibilidad**.

📚 Volver al índice de la unidad · **Anterior:** 09 · Servidor concurrente completo con procesos · **Siguiente:** UD 7 · HTTP y APIs REST

U07 — APIs REST y HTTP

Hablar con la web: REST, métodos HTTP y JSON 🌐

En la U05 y la U06 montaste sockets y hablaste con procesos en la misma máquina. Ahora toca hablar con el mundo: la web entera se apoya en HTTP, y las APIs REST son la manera moderna de pedirle datos a cualquier servicio.

Cuando tu aplicación quiere saber el tiempo, traducir un texto o buscar repositorios en GitHub, no monta un socket a mano: llama a una **API**. Esta unidad te enseña el idioma de esa conversación: qué es una API, cómo funciona HTTP, qué métodos existen (GET, POST, PUT, DELETE), qué significan los códigos de estado, cómo se lee **JSON** y cómo usar la librería `requests` de Python para hacer peticiones reales contra servicios públicos. El siguiente tema, U08 · APIs Comerciales, tomará el relevo con APIs de pago como OpenWeatherMap y OpenAI.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una API y por qué pone una capa de abstracción entre dos programas.
- Describir la web: qué es una URL, una petición HTTP y una respuesta HTTP.
- Dominar los métodos HTTP (GET, POST, PUT, PATCH, DELETE) y cuándo usar cada uno, incluyendo la idempotencia.
- Aplicar los principios REST: recursos, URLs semánticas, sin estado y JSON.
- Interpretar los códigos de estado HTTP (2xx, 3xx, 4xx y 5xx) sin mirar la tabla.
- Leer y generar JSON, y convertirlo a diccionarios y listas de Python.
- Hacer peticiones GET con `requests`, con parámetros de consulta y cabeceras.
- Enviar datos con POST, PUT y DELETE usando `json=`, y gestionar errores HTTP con criterio.
- Construir un mini cliente de API completo en Python que hable con servicios reales.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Web y HTTP	Qué es una API, la URL y el intercambio petición/respuesta	Todos
02 · Métodos HTTP	GET, POST, PUT, PATCH y DELETE: los verbos de la web y la idempotencia	Todos
03 · Principios REST	Recursos, URLs semánticas, sin estado y qué significa ser RESTful	Todos
04 · Códigos de estado	200, 201, 400, 401, 404, 500…: la señal de humo del servidor	Todos
05 · JSON	El idioma de las APIs: dumps, loads y parse de respuestas	Todos
06 · requests: el GET	`requests.get`, parámetros de consulta, cabeceras y la respuesta	Todos
07 · requests: POST, PUT y DELETE	Crear, actualizar y borrar con `json=`, y gestión de errores	Todos
08 · Práctica: mini cliente de API	Sé el código, un cliente completo y los ejercicios del lápiz	Todos
09 · Cierre	Sé la petición HTTP, Fireside, Laboratorio de tortura, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Dónde se cubre
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3, 6-8 + ⚡ Laboratorio (punto 9)
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1-5 y 7 + ⚡ Laboratorio (punto 9)

> RA4c (Servidores concurrentes) y RA4d (ThreadPool) se cubren en la **UD 6 · Servidores concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **UD 10 · Alta disponibilidad**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 5 y dominas los sockets UDP y TCP? Perfecto, ese es el trampolín ideal: repasa la UD 5 · Sockets TCP y UDP para tener fresco qué es un protocolo y cómo se abre una conexión, y arranca en el punto 1, que parte justo de ahí: del intercambio de mensajes entre dos máquinas a la conversación estructurada de HTTP.

¿Ya sabes qué es una API y solo necesitas `requests` o el JSON? Ve directo al punto 5 y al punto 6. Si vienes de cero en la web, no te saltes los puntos 1 a 4: URL, métodos y códigos de estado son el idioma que vas a hablar toda la unidad.

📍 **Primer punto:** 01 · Web y HTTP

⏭️ **Al acabar la unidad, continúa en U08 · APIs Comerciales.**

01 — Web y HTTP

La conversación entre tu código y el servidor 🌍

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 01 · Web y HTTP

📬 La idea en una frase

> Una **API** es un conjunto de reglas para que dos programas se comuniquen, y **HTTP** es el idioma que usan en la web: tu programa manda una **petición** y el servidor responde.

No necesitas saber cómo está implementado el sistema por dentro: solo le pides algo y te lo devuelve. Esa capa que esconde la complejidad es la magia de las APIs.

🧩 ¿Qué es una API?

API = *Application Programming Interface* (Interfaz de Programación de Aplicaciones). Un conjunto de reglas para que dos programas se comuniquen.

Compara las dos formas de acceder a los datos de una base de usuarios:

```
# Sin API — accedes directamente a la BD (malo)
import sqlite3
conn = sqlite3.connect("usuarios.db")
conn.execute("SELECT * FROM usuarios")

# Con API — pides datos a un servicio
import requests
respuesta = requests.get("https://miapi.com/usuarios")
datos = respuesta.json()
```

> Las APIs ponen una capa de abstracción: no necesitas saber cómo está implementado el sistema por dentro.

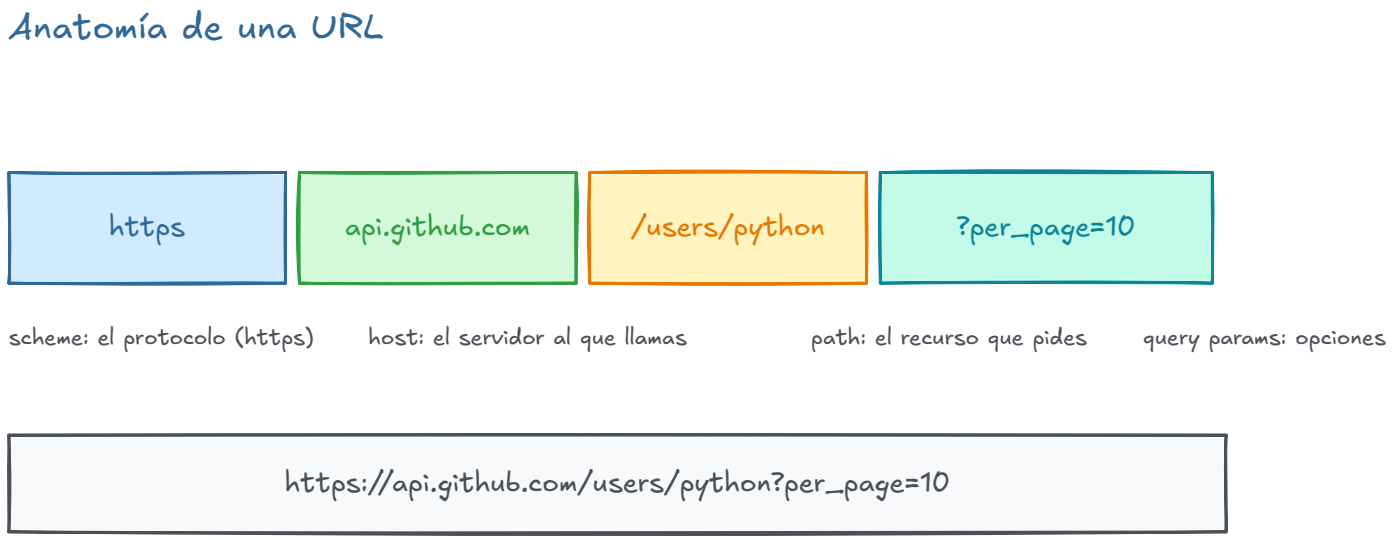
En la segunda opción no sabes qué motor de base de datos hay detrás, ni en qué lenguaje está el servidor, ni cuántos servidores son. Solo sabes la regla: “si pido `GET /usuarios`, me llega una lista”. Eso es trabajar con una API.

La web en tres piezas

Toda la web se reduce a tres piezas que ya conoces por navegar:

- **Cliente**: quien pide (tu navegador, tu código Python).
- **Servidor**: quien responde (una máquina en algún lugar del mundo).
- **Protocolo HTTP**: las reglas de la conversación.

La **URL** (*Uniform Resource Locator*) es la dirección a la que mandas la petición. Se divide en partes con significado:



Parte	Ejemplo	Qué es
scheme	`https`	El protocolo de transporte (cifrado en HTTPS)
host	`api.github.com`	El servidor al que hablas
path	`/users/python`	El recurso concreto que pides
query	`?per_page=10`	Parámetros extra de la petición

Petición y respuesta

HTTP funciona por **intercambio de mensajes**: tú envías una petición y el servidor te contesta.

Petición (lo que envía el cliente):

```
GET /users/python HTTP/1.1
Host: api.github.com
User-Agent: python-requests/2.31.0
Accept: */*
```

Respuesta (lo que devuelve el servidor, ejemplo ilustrativo):

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: ...

{"login": "python", "public_repos": 42}
```

En los próximos puntos desmontarás cada pieza: los **métodos** (GET, POST…) en el punto 2, los **códigos de estado** (200, 404…) en el punto 4 y el **cuerpo JSON** en el punto 5.

🛵 La analogía del pedido a domicilio

Una petición HTTP es como **pedir comida a domicilio**:

1. Buscas el restaurante (**URL**) que vende lo que quieres.
2. Mándasle tu pedido (**petición HTTP**): “una pizza margarita” (GET), “que me pongas en la carta un plato nuevo” (POST).
3. El restaurante (servidor) la cocina y te la entrega (**respuesta HTTP**).
4. La entrega llega con un estado: “✅ todo bien” (200), “❌ el plato no existe” (404), “🔒 el restaurante no te atiende sin reserva” (401).

No te hace falta saber cómo cocina la cocina: solo la dirección y el menú. Esa es la filosofía de las APIs.

🧠 Mini-chequeo

1. ¿Qué es una API y qué problema resuelve?
2. Identifica las cuatro partes de esta URL: `https://api.openweathermap.org/data/2.5/weather?q=Valencia`.
3. ¿Qué dos mensajes intercambian cliente y servidor en HTTP?

> 🔁 Respuestas

✅ Resumen en 3 frases

- Una API es un conjunto de reglas que permite a dos programas comunicarse sin conocer su implementación interna.
- La web funciona con HTTP: el cliente manda una petición y el servidor devuelve una respuesta.
- Una URL se compone de scheme, host, path y query: cada parte tiene un significado en la conversación.

🐛 Vocabulario rápido

Término	Idea general
API	Reglas para que dos programas se comuniquen
HTTP	El protocolo de intercambio de mensajes de la web
URL	La dirección completa de un recurso
Cliente	Quien pide (tu navegador o tu código)
Servidor	Quien responde y sirve los datos
Query	Parámetros extra de la URL (`?clave=valor`)

📚 Volver al índice de la unidad · **Siguiente:** 02 · Métodos HTTP

02 — Métodos HTTP

Los verbos de la web: leer, crear, actualizar y borrar 🚦

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 02 · Métodos HTTP

📬 La idea en una frase

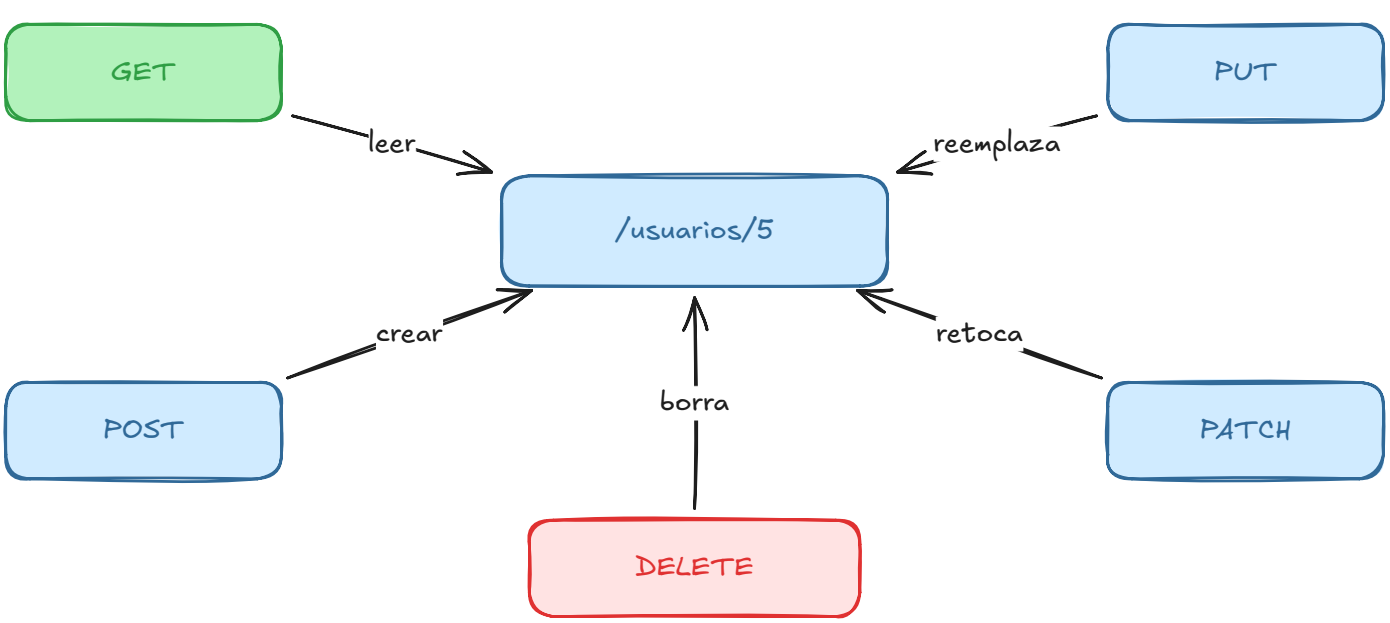
> HTTP tiene **verbos** (métodos) que dicen qué quieres hacer con un recurso: **GET** lo lee, **POST** lo crea, **PUT** lo reemplaza, **PATCH** lo retoca y **DELETE** lo borra.

Los métodos HTTP se corresponden con las operaciones **CRUD** de las bases de datos: Create, Read, Update, Delete. Elegir el verbo correcto es la primera decisión de una API bien hecha.

🚦 Los verbos de HTTP

Método	CRUD	Uso	Cuerpo
`GET`	Read	Obtener datos	❌ No
`POST`	Create	Crear recurso	✅ Sí
`PUT`	Update	Reemplazar completo	✅ Sí
`PATCH`	Update	Actualizar parcial	✅ Sí
`DELETE`	Delete	Borrar recurso	❌ No

Un mismo recurso puede “vivir” con todos los verbos. La URL no cambia: cambia el método:



POST no es idempotente: cada llamada crea un recurso nuevo · GET, PUT y DELETE sí

```
GET    /usuarios        → Listar usuarios
GET    /usuarios/5      → Obtener usuario 5
POST   /usuarios        → Crear usuario
PUT    /usuarios/5      → Actualizar usuario 5
DELETE /usuarios/5      → Borrar usuario 5
```

📜 ¿Qué es la idempotencia?

Un método es **idempotente** si llamarlo **una vez o varias produce el mismo resultado**. Piensa en un interruptor: “apagar la luz” es idempotente (da igual cuántas veces lo hagas, la luz queda apagada); “sumar 1” no lo es.

Método	Idempotente	Por qué
`GET`	✅	Leer 5 veces devuelve lo mismo
`PUT`	✅	Reemplazar el recurso completo: el estado final es igual
`DELETE`	✅	Borrar algo ya borrado sigue borrado
`POST`	❌	Cada llamada crea un recurso **nuevo**: 2 llamadas = 2 recursos

Método	Idempotente	Por qué
PATCH	⚠️	Depende de la operación parcial que apliques

> 💡 Si un cliente reenvía una petición por culpa de un timeout, un método idempotente (PUT) es seguro de repetir; un POST podría duplicar el recurso.

🐍 Métodos en Python

Con `requests`, cada método es una función: `requests.get`, `requests.post`, `requests.put`, `requests.delete`…

```
import requests

# POST — crear un recurso
nuevo_usuario = {"nombre": "Ana", "email": "ana@x.com"}
resp = requests.post("https://miapi.com/usuarios", json=nuevo_usuario)
print(resp.status_code)  # 201 Created

# DELETE — borrar
resp = requests.delete("https://miapi.com/usuarios/5")
print(resp.status_code)  # 204 No Content
```

El `json=` de `requests.post` convierte el diccionario Python a JSON automáticamente y lo envía en el cuerpo de la petición. Lo verás en detalle en el punto 7.

🧠 Mini-chequeo

1. ¿Qué método usarías para… a) ver un artículo, b) crearlo, c) cambiarlo entero, d) borrarlo?
2. ¿Por qué POST no es idempotente y PUT sí?
3. ¿GET lleva cuerpo en la petición? ¿Y POST?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- HTTP tiene cinco verbos con un propósito claro: GET lee, POST crea, PUT reemplaza, PATCH retoca y DELETE borra.
- La idempotencia distingue los métodos seguros de repetir (GET, PUT, DELETE) del que crea (POST).
- En Python, cada método es una función de `requests` y `json=` se encarga del cuerpo automáticamente.

🐛 Vocabulario rápido

Término	Idea general
Método HTTP	El verbo que dice qué hacer con el recurso
GET	Leer datos (sin cuerpo, idempotente)
POST	Crear un recurso (con cuerpo, no idempotente)
PUT	Reemplazar el recurso completo
PATCH	Actualizar solo una parte
DELETE	Borrar el recurso
Idempotente	Repetirlo no cambia el resultado

📚 Volver al índice de la unidad · **Anterior:** 01 · Web y HTTP · **Siguiente:** 03 · Principios REST

03 — Principios REST

Recursos, URLs semánticas y sin estado 🏛️

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 03 · Principios REST

📬 La idea en una frase

> **REST** (*Representational State Transfer*) es el estándar de las APIs modernas: trata las cosas del sistema como **recursos**, a cada recurso le da una **URL** y lo manipula con los **verbos HTTP** que viste en el punto 2.

REST no es una tecnología ni una librería: es un conjunto de **convenciones**. Cuando las sigues, cualquiera puede usar tu API sin leer un manual de 100 páginas.

🧱 Recursos: las cosas, no las acciones

En REST, modelas **sustantivos**, no verbos. Un recurso es "una cosa" del sistema: un usuario, un producto, un pedido. Cada recurso tiene su URL:

```
/productos      → todos los productos
/productos/123  → el producto 123
```

Fíjate en la diferencia con las APIs antiguas basadas en acciones:

❌ Acciones (no REST)	✅ Recursos (REST)
`/getUsuario?id=5`	`GET /usuarios/5`
`/crearUsuario`	`POST /usuarios`
`/borrarUsuario?id=5`	`DELETE /usuarios/5`

En REST la URL es un **nombre** (qué cosa) y el método HTTP es el **verbo** (qué hacer). Se separan los dos conceptos en vez de mezclarlos en la URL.

🗄️ Los cuatro pilares de REST

- **Recursos**: cada entidad tiene una URL (`/usuarios`, `/productos/123`).
- **Verbos HTTP**: GET (leer), POST (crear), PUT (actualizar), DELETE (borrar).
- **Sin estado** (stateless): cada petición contiene todo lo necesario.
- **JSON**: el formato de intercambio de datos.

Con esos cuatro pilares, una API REST es predecible: si sabes usar una, sabes usar todas. Ese es el mapa mental que te llevas de esta unidad:

```
GET    /usuarios        → Listar usuarios
GET    /usuarios/5      → Obtener usuario 5
POST   /usuarios        → Crear usuario
PUT    /usuarios/5      → Actualizar usuario 5
DELETE /usuarios/5      → Borrar usuario 5
```

🕊️ Sin estado (stateless)

Que una API sea **stateless** significa que el servidor **no guarda contexto entre peticiones**: cada petición lleva toda la información necesaria para ser entendida por sí sola.

Piénsalo como un camarero que no recuerda tu cara: cada vez que pides algo, le tienes que decir todo de nuevo. No puedes decir “lo mismo de antes”, porque el camarero no guarda memoria de lo anterior.

Consecuencia práctica: el servidor no tiene que recordar quién eres entre llamadas. Si tu petición necesita autenticación, la envías en cada petición (en la cabecera `Authorization`), no “la primera vez y ya está”.

🏷️ ¿Qué significa “RESTful”?

Una API es **RESTful** si cumple de verdad los principios REST: recursos con URLs semánticas, verbos HTTP bien usados, sin estado y JSON como formato.

Dos ejemplos opuestos:

- ✅ **RESTful**: `GET https://api.github.com/users/python` → "dame el recurso del usuario python".
- ❌ **No RESTful**: `GET https://miapi.com/obtenerUsuario?id=5&accion=leer` → meto la acción en la URL y abuso del GET.

Elegir el verbo correcto para cada operación es lo que el punto 2 llamaba "hacer bien REST". Las APIs de GitHub, GitHub Search o JSONPlaceholder que usarás en esta unidad son RESTful: fíjate en cómo sus URLs son limpias y su método dice la intención.

🧠 Mini-chequeo

1. ¿Qué es un recurso en REST? Pon dos ejemplos de URLs.
2. ¿Por qué `GET /getUsuario?id=5` no es RESTful?
3. ¿Qué significa que una API sea "sin estado"?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- REST modela **recursos** con URLs semánticas y los manipula con los **verbos HTTP**, sin meter la acción en la URL.
- Sus cuatro pilares son recursos, verbos HTTP, **sin estado** y JSON.
- Una API que cumple esos principios es **RESTful**: predecible y fácil de usar.

🐛 Vocabulario rápido

Término	Idea general
REST	Estándar de APIs: recursos + verbos HTTP + sin estado + JSON
Recurso	Una "cosa" del sistema con su URL
URL semántica	URL que nombra el recurso, no la acción
Stateless	El servidor no guarda contexto entre peticiones
RESTful	API que cumple los principios REST

📚 Volver al índice de la unidad · **Anterior:** 02 · Métodos HTTP · **Siguiente:** 04 · Códigos de estado

04 — Códigos de estado

La señal de humo del servidor: 200, 404, 500… 🚨

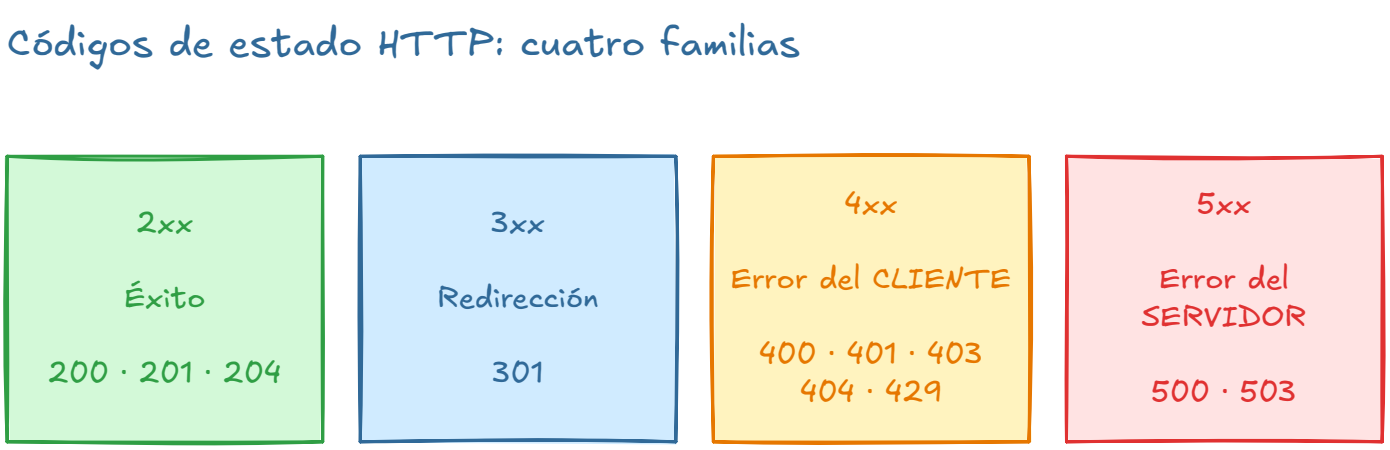
> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 04 · Códigos de estado

📬 La idea en una frase

> Cada respuesta HTTP lleva un **código de estado** que te dice si la petición salió bien o mal: **2xx es éxito, 3xx es redirección, 4xx es un error tuyo y 5xx es un error del servidor**.

Es la señal de humo de la web: en tres dígitos, el servidor te dice qué ha pasado sin necesidad de leer el cuerpo.

🎨 Las familias de códigos



4xx: revisa tu petición · 5xx: la culpa es del servidor

Familia	Significado	Ejemplos
2xx	Éxito	200 OK, 201 Created, 204 No Content
3xx	Redirección	301 Moved Permanently
4xx	Error del **cliente** (tuyo)	400, 401, 403, 404, 429
5xx	Error del **servidor**	500, 503

> **Consejo**: los códigos 2xx son éxito, 4xx son error TUYO, 5xx son error DEL SERVIDOR. Cuando veas un 4xx, revisa tu petición; cuando veas un 5xx, la culpa es de la otra parte.

📋 La tabla completa

Código	Significado	Situación típica
200	OK	GET exitoso
201	Created	POST exitoso (recurso creado)
204	No Content	DELETE exitoso
301	Moved Permanently	La URL cambió
400	Bad Request	Enviaste datos incorrectos
401	Unauthorized	Falta autenticación
403	Forbidden	No tienes permiso
404	Not Found	El recurso no existe
429	Too Many Requests	Te pasaste del límite
500	Internal Server Error	Error del servidor
503	Service Unavailable	El servicio está caído

Truco para no confundirte:

- **400 vs 404**: 400 = “tu petición está mal formada”, 404 = “lo que pides no existe”.
- **401 vs 403**: 401 = “no te has identificado”, 403 = “te has identificado pero no tienes permiso”.

Códigos en Python

En `requests`, el código está en `resp.status_code` (un entero). Lo compruebas antes de procesar los datos:

```python
import requests

resp = requests.get("https://api.github.com/users/python")
if resp.status_code == 200:
    print("✅ Todo bien:", resp.json()["login"])
elif resp.status_code == 404:
    print("❌ Usuario no encontrado")
else:
    print(f"⚠️ Código inesperado: {resp.status_code}")
```

> 💡 `requests` **no lanza excepción** cuando llega un 404 o un 500: te devuelve la respuesta con su `status_code` y `resp.ok` será `False`. Eres tú quien decide cómo tratarlo. Si quieres que una respuesta mala lance excepción, usa `resp.raise_for_status()` (lo verás en el punto 7).

🧠 Mini-chequeo

1. ¿Qué familia indica un error del servidor? ¿Y un error tuyo?
2. ¿Qué código devuelve un POST exitoso? ¿Y un DELETE exitoso?
3. Diferencias 401 vs 403 y 400 vs 404.

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- El código de estado es la señal de humo de HTTP: 2xx éxito, 3xx redirección, 4xx error tuyo, 5xx error del servidor.
- En Python lo lees con `resp.status_code` y decides qué hacer con cada código.
- `requests` no lanza excepción ante un 4xx/5xx: comprobar `status_code` (o usar `raise_for_status()`) es tu responsabilidad.

🐛 Vocabulario rápido

Término	Idea general
Código de estado	Los 3 dígitos que resumen el resultado de una petición

Término	Idea general
2xx	Éxito (200 OK, 201 Created, 204 No Content)
4xx	Error del cliente (400, 401, 403, 404, 429)
5xx	Error del servidor (500, 503)
status_code	La propiedad de la respuesta que lo contiene (int)
raise_for_status()	Método que lanza excepción si el código es 4xx/5xx

📚 Volver al índice de la unidad · **Anterior:** 03 · Principios REST · **Siguiente:** 05 · JSON

05 — JSON

El idioma de las APIs: de texto a diccionario y vuelta 📦

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 05 · JSON

📬 La idea en una frase

> **JSON** (*JavaScript Object Notation*) es el formato de intercambio de datos más usado en las APIs: un texto estructurado con llaves, corchetes y comillas que Python convierte directamente a **diccionarios** y **listas**.

Es el "idioma" en el que las APIs te devuelven los datos. En el punto 3 lo viste como uno de los cuatro pilares de REST; aquí le toca el turno de verdad.

📦 El formato

Un JSON es un texto con pares `"clave": valor`. Los valores pueden ser números, textos, booleanos, listas u otros objetos anidados:

```
{
  "nombre": "Ana",
  "edad": 25,
  "email": "ana@x.com",
  "activo": true,
  "intereses": ["Python", "redes", "cripto"]
}
```

Fíjate en las reglas: las **claves van entre comillas dobles**, los **textos entre comillas dobles**, los booleanos son `true`/`false` en minúscula y las listas van entre corchetes.

De JSON a Python y viceversa

El módulo `json` de la librería estándar hace la conversión en las dos direcciones:

```python
import json

# JSON (texto) → Python
texto = '{"nombre": "Ana", "edad": 25, "activo": true}'
datos = json.loads(texto)            # dict
print(datos["nombre"])               # "Ana"
print(type(datos))                   # <class 'dict'>

# Python → JSON (texto)
python_dict = {"nombre": "Ana", "edad": 25, "activo": True}
texto_json = json.dumps(python_dict, ensure_ascii=False, indent=2)
print(texto_json)
```

La equivalencia es casi automática:

JSON	Python
`{ ... }`	`dict`
`[ ... ]`	`list`
`"texto"`	`str`
`42`	`int` / `float`
`true` / `false`	`True` / `False`
`null`	`None`

> 💡 `json.dumps(..., ensure_ascii=False)` conserva las tildes tal cual; `indent=2` lo formatea legible. Y recuerda: `loads` viene de "**load** string" y `dumps` de "**dump** string".

🔁 JSON dentro de requests

Con `requests` ni siquiera tocas el texto: `resp.json()` parsea el cuerpo automáticamente y te da el dict/list de Python.

```python
import requests

resp = requests.get("https://api.github.com/users/python")
datos = resp.json()

print(datos["login"])          # "python"
print(datos["public_repos"])   # el número real del usuario (cambia con el tiempo)
print(datos["avatar_url"])     # "https://..."
```

Cuando la respuesta es una **lista** de recursos, `resp.json()` devuelve una lista de diccionarios y la recorres normal:

```python
resp = requests.get("https://jsonplaceholder.typicode.com/posts")
posts = resp.json()
print(f"Hay {len(posts)} posts")        # 100
print(posts[0]["title"])                # el título del primer post
```

> ⚠️ `resp.json()` **lanza excepción** si el cuerpo no es JSON (por ejemplo, un HTML de error). Por eso en el punto 4 primero compruebas `status_code` y luego parseas.

🧠 Mini-chequeo

1. ¿A qué tipo de Python se convierte `[1, 2, 3]`? ¿Y `{"a": true}`?
2. ¿Qué hace `json.loads` y qué hace `json.dumps`?
3. ¿Qué pasa si llamas a `resp.json()` sobre una respuesta que no es JSON?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- JSON es el formato de intercambio de datos de las APIs: texto estructurado que Python mapea a dicts y listas.
- `json.loads` y `json.dumps` hacen la conversión en las dos direcciones con el módulo `json`.
- Con `requests`, `resp.json()` parsea el cuerpo directamente, pero lanza excepción si no es JSON.

🐛 Vocabulario rápido

Término	Idea general
JSON	Formato de intercambio de datos de las APIs
json.loads	De texto JSON a objeto Python
json.dumps	De objeto Python a texto JSON
resp.json()	Parseo directo del cuerpo de la respuesta
ensure_ascii	Opción de `dumps` para conservar tildes
JSONDecodeError	Excepción al parsear algo que no es JSON

📚 Volver al índice de la unidad · **Anterior:** 04 · Códigos de estado · **Siguiente:** 06 · requests: el GET

06 — requests: el GET

La librería estándar de facto para hablar HTTP 📥

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 06 · requests: el GET

📬 La idea en una frase

> `requests` es la librería estándar de facto para hacer peticiones HTTP en Python: `requests.get(url)` te devuelve un objeto **respuesta** con el código, las cabeceras y el cuerpo ya parseado.

Todo lo que has visto hasta ahora —métodos, códigos, JSON— se materializa aquí en una llamada de una línea.

📥 El GET más básico

```python
import requests

# GET básico
resp = requests.get("https://api.github.com")
print(resp.status_code)     # 200
print(resp.headers)         # Cabeceras de respuesta
print(resp.elapsed)         # Tiempo que tardó
```

`requests.get` se encarga de todo lo que en la UD 5 hacías a mano con sockets: resolver el DNS, abrir la conexión TCP, el handshake TLS en HTTPS, construir la petición HTTP y leer la respuesta.

📄 La respuesta y sus propiedades

El objeto que devuelve `requests.get` es un objeto **respuesta** con estas propiedades:

Propiedad	Qué devuelve
`resp.status_code`	Código HTTP (int)

Propiedad	Qué devuelve
`resp.ok`	`True` si 200-399
`resp.text`	Cuerpo como string
`resp.json()`	Cuerpo como dict (¡lanza excepción si no es JSON!)
`resp.headers`	Dict con cabeceras
`resp.elapsed`	Objeto timedelta

La jerarquía de uso habitual es: primero `status_code` (o `resp.ok`), luego `resp.json()` para los datos. Y si solo quieres ver qué te ha contestado en bruto, `resp.text`.

🔎 Parámetros de consulta

Muchas APIs filtran o buscan con **query params** en la URL. En vez de montar la URL a mano con `f"..."`, `requests` te da `params=` y construye la query string por ti:

```
import requests

params = {"q": "python", "page": 1, "per_page": 10}
resp = requests.get("https://api.github.com/search/repositories", params=params)
# URL final: https://api.github.com/search/repositories?q=python&page=1&per_page=10
```

Ventajas de `params=` frente a concatenar a mano: escapa los caracteres especiales (espacios, &, #) y te ahorra errores de “se me olvidó el ?”.

🧢 Cabeceras personalizadas

Algunas APIs requieren cabeceras específicas: autenticación, formato esperado o el identificador de tu aplicación. Se envían con `headers=`:

```python
import requests

cabeceras = {
    "Authorization": "Bearer mi-token-secreto",
    "Accept": "application/json",
    "User-Agent": "MiApp/1.0"
}
resp = requests.get("https://api.github.com/user", headers=cabeceras)
```

> ⚠️ **Seguridad**: un token en la cabecera `Authorization` nunca debe ir "en claro" en el código ni subirse a GitHub. En la UD 8 verás cómo guardarlo en variables de entorno con `python-dotenv`.

🧠 Mini-chequeo

1. ¿Qué devuelve `requests.get` exactamente?
2. ¿Para qué sirve `params=` y qué construye por ti?
3. ¿Qué propiedad te dice si el código está entre 200 y 399?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `requests.get(url)` devuelve un objeto respuesta con código, cabeceras y cuerpo.
- `params=` construye la query string por ti y `headers=` envía cabeceras personalizadas (como el token).
- El orden de uso: comprobar `status_code`/`ok` y luego `resp.json()`.

🐛 Vocabulario rápido

Término	Idea general
requests	La librería de facto para peticiones HTTP en Python
Objeto respuesta	Lo que devuelve `requests.get`: código, cabeceras y cuerpo

Término	Idea general
params	Diccionario que se convierte en query string
headers	Diccionario con las cabeceras de la petición
resp.text	El cuerpo en bruto como string
resp.elapsed	Tiempo que tardó la petición (timedelta)

📚 Volver al índice de la unidad · **Anterior:** 05 · JSON · **Siguiente:** 07 · requests: POST, PUT y DELETE

07 — requests: POST, PUT y DELETE

Enviar datos a la API y gestionar los errores 📮

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 07 · requests: POST, PUT y DELETE

📬 La idea en una frase

> Con `requests.post`, `requests.put` y `requests.delete` **creas, reemplazas y borras recursos**: el cuerpo se envía con `json=` y la respuesta se trata igual que en el GET.

Los verbos del punto 2 cobran vida: `json=` convierte tu diccionario a JSON y lo pone en el cuerpo de la petición.

📮 POST: crear un recurso

Para crear, `requests.post` con `json=`:

```
import requests

resp = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json={"title": "foo", "body": "bar", "userId": 1}
)
print(resp.status_code)     # 201 Created
print(resp.json())          # La respuesta parseada como dict
```

JSONPlaceholder es una API de pruebas: cada POST te devuelve el recurso creado con un ID nuevo (101, 102…). Un POST exitoso responde **201 Created** (visto en el punto 4).

📝 PUT y PATCH: actualizar

PUT reemplaza el recurso **completo** (idempotente); **PATCH** actualiza solo **una parte**:

```python
import requests

# PUT — reemplazo completo
resp = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}
)
print(resp.status_code)          # 200 OK
print(resp.json()["title"])      # "nuevo titulo"
```

En la práctica casi todo el mundo usa `requests.put` con `json=` del mismo modo; `requests.patch` existe para los casos de "toca solo este campo".

🗑️ DELETE: borrar

```python
import requests

resp = requests.delete("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code)          # 200 OK (JSONPlaceholder)
# En REST puro, un borrado exitoso devuelve 204 No Content
```

El código de estado te confirma el resultado: en REST canónico, DELETE exitoso → **204 No Content** (sin cuerpo que leer).

🛡️ Gestión de errores: status_code y raise_for_status

Como viste en el punto 4, `requests` **no lanza excepción** ante un 4xx/5xx. Dos estrategias:

```python
import requests

# Opción 1: comprobar el código a mano
resp = requests.post("https://jsonplaceholder.typicode.com/posts",
                     json={"title": "x", "body": "y"})
if resp.status_code == 201:
    print("✅ Post creado con ID", resp.json()["id"])
else:
    print("❌ Algo falló:", resp.status_code)

# Opción 2: raise_for_status() lanza excepción si es 4xx/5xx
try:
    resp = requests.get("https://httpbin.org/status/404")
    resp.raise_for_status()          # lanza HTTPError
    datos = resp.json()
except requests.exceptions.HTTPError as e:
    print("⚠️ Error HTTP:", e)
```

`raise_for_status()` es la vía rápida: si la respuesta es mala, lanza una excepción y tu código no sigue parseando un error.

🧩 Pool Puzzle — Petición API con errores

Estas líneas hacen una petición a una API y gestionan errores. ¿En qué orden van?

```
a)     if respuesta.status_code == 200:
b) import requests
c)         print("Error:", respuesta.status_code)
d) respuesta = requests.get("https://api.github.com/users/python")
e)         datos = respuesta.json()
f)     else:
g)             print(datos["login"])
h) try:
```

> 🔓 Solución

🧠 Mini-chequeo

1. ¿Qué hace `json=` en `requests.post`?
2. ¿Qué código devuelve un POST exitoso y qué devuelve un DELETE exitoso?
3. ¿Qué lanza `raise_for_status()` y cuándo lo usarías?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- `requests.post/put/delete` crean, reemplazan y borran recursos enviando el cuerpo con `json=`.
- El resultado se lee igual que en el GET: `status_code` y `resp.json()`.
- Los errores se gestionan comprobando `status_code` o dejando que `raise_for_status()` lance la excepción.

🐛 Vocabulario rápido

Término	Idea general
requests.post	Envía un recurso nuevo con `json=`
requests.put	Reemplaza el recurso completo
requests.patch	Actualiza solo una parte
requests.delete	Borra el recurso
json=	Convierte un dict Python a JSON en el cuerpo
raise_for_status()	Lanza HTTPError si el código es 4xx/5xx

📚 Volver al índice de la unidad · **Anterior:** 06 · requests: el GET · **Siguiente:** 08 · Práctica: mini cliente de API

08 — Práctica: mini cliente de API

Sé el código, un cliente completo y los ejercicios del lápiz 🛠️

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 08 · Práctica: mini cliente de API

📬 La idea en una frase

> Aquí junta todo lo aprendido en un solo programa: un **mini cliente de API** que pide datos a servicios reales, los parsea y gestiona los errores — y un paseo "siendo" la librería `requests`.

De aquí saldrás con un programa completo que ya habla el idioma de la web.

📮 El mini cliente de API

Un cliente de API típico se compone de tres bloques: hacer la petición, comprobar el estado y procesar los datos. Todo junto:

```python
import requests

def obtener_usuario(nombre):
    """Devuelve un dict con los datos del usuario o None si no existe."""
    resp = requests.get(f"https://api.github.com/users/{nombre}")
    if resp.status_code == 200:
        return resp.json()
    if resp.status_code == 404:
        print(f"❌ El usuario '{nombre}' no existe.")
    else:
        print(f"⚠️ Código inesperado: {resp.status_code}")
    return None

def resumen_usuario(datos):
    print(f"👤 Login:     {datos['login']}")
    print(f"👄 Nombre:    {datos['name']}")
    print(f"📦 Repos:     {datos['public_repos']}")
    print(f"👬 Seguidores: {datos['followers']}")

if __name__ == "__main__":
    datos = obtener_usuario("python")
    if datos:
        resumen_usuario(datos)
```

Observa el patrón que repite cualquier cliente real: `requests.get` → comprobar `status_code` → `resp.json()` → usar los datos. Es el mismo esqueleto del punto 6, ahora con errores bien delimitados.

⭐ Sé el código: una petición GET, milisegundo a milisegundo

> “Sé la librería requests y recorre cada milisegundo de una petición GET.”

```python
import requests

# 1. Construimos la URL y parámetros
url = "https://api.github.com/users/python"

# 2. Llamamos a requests.get()
respuesta = requests.get(url)

# 3. Procesamos la respuesta
datos = respuesta.json()
print(f"Usuario: {datos['login']}")
print(f"Repos: {datos['public_repos']}")
```

Traza interna:

```
1. requests.get("https://api.github.com/users/python")

2. requests construye la URL:
   → scheme: https
   → host: api.github.com
   → path: /users/python

3. Abre conexión TCP a api.github.com:443 (HTTPS)
   → DNS lookup: api.github.com → 140.82.121.5
   → Three-way handshake TCP
   → Handshake TLS (cifrado)

4. Envía petición HTTP:
   GET /users/python HTTP/1.1
   Host: api.github.com
   User-Agent: python-requests/2.31.0
   Accept-Encoding: gzip, deflate
   Accept: */*

5. ⏳ Espera respuesta (~150ms)

6. Recibe respuesta:
   HTTP/1.1 200 OK
   Content-Type: application/json
   ...

7. requests parsea el JSON → dict de Python

8. Extraemos datos['login'] → "python"
```

Todo lo que en la UD 5 hacías a mano —DNS, TCP, TLS, construir la petición— lo hace `requests` en una línea. Tu única tarea es leer la respuesta.

📝 Aprieta el lápiz

1. **GET a GitHub API**: obtén los datos del usuario “python” y muestra su nombre real, repos públicos y seguidores.
2. **Parámetros de búsqueda**: busca repositorios de Python con más de 1000 estrellas usando `/search/repositories`.

3. **POST a JSONPlaceholder**: crea un post nuevo en `jsonplaceholder.typicode.com/posts` y muestra el ID devuelto.
4. **Gestión de errores**: haz GET a una URL que no existe (404) y una que dé error 500. Muestra mensajes adecuados.

> 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Cuál es el esqueleto de cualquier petición con `requests`?
2. En la traza del "Sé el código", ¿qué hace `requests` entre el paso 3 y el 6?
3. ¿Por qué comprobar `status_code` antes de `resp.json()` es una buena práctica?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un cliente de API se reduce a tres bloques: petición, comprobación de estado y procesamiento de datos.
- `requests` hace por ti todo el trabajo de red (DNS, TCP, TLS) que en la UD 5 hacías a mano con sockets.
- El patrón `status_code` → `json()` es la clave para que tu código no reviente ante respuestas de error.

🐛 Vocabulario rápido

Término	Idea general
Cliente de API	Programa que habla con una API mediante peticiones HTTP
Esqueleto de petición	`requests.get` → `status_code` → `resp.json()` → usar datos
Traza interna	El camino que sigue la petición dentro de la librería
Endpoint	La URL concreta de un recurso o acción de la API

📚 Volver al índice de la unidad · **Anterior:** 07 · requests: POST, PUT y DELETE · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé la petición HTTP, laboratorio real y el ring final de la unidad 🧠

> 🗺️ **Estás en:** 🌐 **UD 7 · HTTP y APIs REST** → 09 · Cierre

Has terminado la teoría: API, URL, métodos HTTP, principios REST, códigos de estado, JSON, `requests` con GET y con POST/PUT/DELETE, y el mini cliente completo. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real contra una API pública con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé la petición HTTP

> *Eres una petición HTTP. Acabas de nacer en un `requests.get()` dentro del portátil de un estudiante y tu destino es `api.github.com`. Empieza el viaje.*

¿Qué pasa?

1. `requests` te construye: fija el scheme `https`, el host `api.github.com` y el path `/users/python`. Eres un mensaje `GET /users/python HTTP/1.1` con cabeceras (`Host`, `User-Agent`, `Accept`).
2. Abres la conexión: primero el **DNS lookup** (`api.github.com` → una IP), luego el **three-way handshake TCP** y el **handshake TLS** que te cifra para el viaje.
3. Atraviesas Internet hasta el servidor y te entrega el proceso servidor que te espera en el puerto 443.
4. El servidor interpreta tu método (GET), localiza el recurso y contesta: **200 OK** con el cuerpo `application/json`.
5. De vuelta en Python, `requests` parsea el cuerpo JSON y te convierte en un **dict** (`datos["login"]` → `"python"`).
6. El estudiante imprime tu mensaje. Misión cumplida.

Todo en ~150 ms. Sin saber qué hay en cada router del camino.

> 💡 **Ahora tú:** ¿y si en lugar de un GET el estudiante usara un POST para crear un post en JSONPlaceholder? Tu cuerpo ya no está vacío: viajas con `{"title": "foo", "body": "bar", "userId": 1}` dentro, y el servidor te responde **201 Created** con el recurso creado y su ID nuevo. Eso es lo que aprendiste en el punto 7.

🔥 Fireside Chat: GET vs POST vs PUT vs DELETE

> *Cuatro verbos HTTP se sientan junto a la chimenea a decidir, de una vez, quién es el más importante.*

GET: — Yo soy el más usado. Solo pido información, no cambio nada. Soy seguro, idempotente y cacheable.

POST: — Pues yo soy el que crea cosas. Sin mí no habría nuevos recursos. Eso sí, no soy idempotente: si llamas dos veces, creo dos veces.

PUT: — Idempotente, como GET, pero yo actualizo. Mando el recurso completo y reemplazo lo que haya.

DELETE: — Y yo elimino. También idempotente: da igual cuántas veces lo llames, el recurso ya no está.

GET: — Oye, ¿y PATCH? Siempre se nos olvida…

POST: — Ese es el primo moderno que actualiza solo un campo. Pero mejor no liarnos, que ya somos suficientes.

> **Moraleja**: cada verbo HTTP tiene un propósito: GET (leer), POST (crear), PUT (reemplazar), DELETE (borrar). Elegir el correcto es hacer bien REST.

🕵️‍♂️ ¿Quién soy?

1. Soy el formato de intercambio de datos que casi todas las APIs usan hoy: llaves, corchetes y comillas.
2. No soy idempotente: si me llamas dos veces, creo dos recursos.
3. Soy el código de estado que aparece cuando el recurso que buscas no existe.
4. Pongo una capa de abstracción: no necesitas saber cómo está implementado el sistema por dentro.
5. Soy la propiedad de la respuesta que devuelve `True` si el código está entre 200 y 399.
6. Soy la librería de facto para hacer peticiones HTTP en Python, y construyo la query string con `params=`.

> › 🔁 **Respuestas**

CONRAD VS EL MUNDO: “404 que nadie entiende”

CONRAD: — “Clásico: haces GET a `https://api.github.com/users/python` y te llega 404. Razones: 1) **Escribiste mal la URL**: `/user` en lugar de `/users`, o te comiste un carácter. 2) **El recurso no existe**: el usuario cambió de nombre o fue borrado. 3) **Confundiste el recurso con la acción**: pediste `/search/repositories` sin `?q=`. 4) No distinguiste el **404** (no existe) del **401** (no autenticado) ni del **403** (sin permiso).”

CONRAD: — “Y lo mejor: *‘¿será que GitHub está caído?’*. ¡Pues no! Si el servidor estuviera caído verías **500** o **503**, no 404. El 404 es tuyo: la red está bien, lo que pediste está mal. A revisar la URL y el método.”

CONRAD: — “Y no me vengas con *‘¿será que el JSON está mal?’* si antes ni comprobaste el `status_code`. Primero `status_code`, luego `json()`. Si parseas un 404, `resp.json()` te lanza una excepción y el mensaje de error te lleva por el camino de la amargura. A diagnosticar.”

Laboratorio de tortura: mini cliente contra una API pública

> **Duración:** 45 minutos **Herramienta:** Python 3 (`requests`, pip install si hace falta) y conexión a Internet

Escenario: construye un mini cliente que consulte dos APIs públicas de verdad: **GitHub API** y **JSONPlaceholder**. Usa todo lo de la unidad: GET con params, POST, PUT, DELETE y gestión de errores.

Tareas paso a paso:

1. **GET básico**: obtén `https://api.github.com/users/python` y muestra `login`, `name`, `public_repos` y `followers`.
2. **Búsqueda con params**: busca repositorios de Python con más de 1000 estrellas con `params={"q": "python stars:>1000", "sort": "stars", "per_page": 5}` y muestra nombre y estrellas de los 5 primeros.
3. **POST**: crea un post en `https://jsonplaceholder.typicode.com/posts` con `json={"title": "Hola API", "body": "Probando", "userId": 1}` y muestra el ID devuelto.

4. **PUT**: actualiza el post 1 con `requests.put` y comprueba que el título cambió.
5. **DELETE**: borra el post 1 y muestra el código de estado.
6. **Errores**: pide un usuario inexistente (`/users/usuarioquenoexiste123`) y muestra un mensaje "no encontrado" sin que el programa reviente.

Fallo intencionado: en el paso 3, sustituye el POST de JSONPlaceholder por un **GET a un endpoint de escritura** de GitHub, `https://api.github.com/user/repos` (sin token). ¿Qué pasa? La API responde **404 "Not Found"** y, como tu código llama a `resp.json()` sin comprobar `status_code`, intenta parsear un cuerpo que no es el esperado y el programa revienta.

> 💡 **Nota didáctica:** JSONPlaceholder no valida métodos, así que un `GET /posts` devuelve **200** con los datos (la petición se ignora, no se crea nada). Por eso el fallo intencionado usa GitHub, que sí rechaza el método de escritura con un 404.

> **Pista 1:** los métodos no son intercambiables. Crear es siempre **POST** (lo viste en el punto 2). Si usas GET para escribir, la API te lo devuelve con un 4xx.
>
> **Pista 2:** si el fallo no se ve a simple vista, añade `print(resp.status_code, resp.text)` justo antes del `resp.json()`. Verás el código y el cuerpo de error en texto plano: esa es la prueba de que el problema está en el método elegido.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **HTTP Rider**	Explicar de memoria la anatomía de una petición y una respuesta HTTP
🏅 **Verb Master**	Elegir correctamente GET, POST, PUT, PATCH o DELETE para cada operación CRUD
🏅 **Status Whisperer**	Interpretar 200, 201, 400, 401, 404 y 500 sin mirar la tabla
🏅 **JSON Parsel**	Parsear una respuesta JSON a dict de Python y extraer campos con seguridad
🏅 **API Client**	Construir un mini cliente de API con GET, POST y gestión de errores

🧠 Atrévete a pensar

1. ¿Por qué POST no es idempotente y PUT sí?
2. ¿Qué diferencia hay entre 401 y 403?
3. ¿Por qué REST prefiere recursos en las URLs (`/usuarios/5`) en lugar de acciones (`getUsuario?id=5`)?
4. ¿Qué significa "sin estado" y qué consecuencias tiene para el servidor?
5. ¿Cuándo usarías `resp.text` en lugar de `resp.json()`?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Verbo HTTP para leer datos (3 letras)
4. Código de estado cuando el recurso no existe (3 letras)
5. Formato de intercambio de datos de las APIs (4 letras)
8. Verbo HTTP para borrar un recurso (6 letras)

Vertical:
2. Código de estado de error del servidor (3 letras)
3. Verbo HTTP para crear recursos (4 letras)
6. Cabecera que indica el formato que acepta la petición (6 letras)
7. La librería de facto para peticiones HTTP en Python (8 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **"¿Qué es una API REST? Explícala como si fuera un camarero."**
2. **"¿Qué diferencia hay entre GET y POST? ¿Y entre PUT y PATCH?"**
3. **"Explica los códigos de estado: ¿qué significan 200, 201, 400, 401, 404 y 500?"**
4. **"Escribe en Python una petición GET con parámetros a una API y procesa la respuesta JSON."**
5. **"¿Cómo gestionarías los errores de una petición HTTP en tu código?"**

> 💡 **Cómo encararlas:** la 1 y la 4 son las "preguntas reina". Para la 1, recorre los pilares del punto 3: recursos con URL, verbos HTTP, sin estado y JSON — con la analogía del camarero del punto 1. Para la 4, escribe el esqueleto del punto 6: `params=`, `status_code`, `resp.json()`. Si sabes contarlo fluido, ya eres medio desarrollador de APIs.

🤷 No hay preguntas tontas

> ❓ **¿Necesito siempre una API key?**

No, hay APIs públicas sin key (como la de GitHub para datos públicos). Pero la mayoría requiere autenticación. Y cuando la necesites, la mandas en la cabecera `Authorization` (nunca en la URL ni en el código): lo verás en la UD 8.

> ❓ **¿Puedo modificar datos con GET?**

Técnicamente sí, pero **no debes**. GET es para leer. POST/PUT para modificar. Si usas GET para modificar, violas REST: el método deja de decir la verdad sobre lo que haces.

> ❓ **¿Qué es CORS y me afecta en Python?**

CORS es una restricción del navegador. En Python no te afecta (no hay navegador). Es cosa de JavaScript.

> ❓ **¿Qué significa que una API sea "sin estado" (stateless)?**

Que cada petición contiene toda la información necesaria. El servidor no guarda contexto entre peticiones. Como un camarero que no recuerda tu cara: cada vez le tienes que decir todo.

> ❓ **¿JSON o XML?**

Hoy en día, JSON gana por goleada. XML solo se usa en entornos legacy (bancos, SOAP).

🎬 Poscréditos

> *Tu petición GET parte de tu portátil, cruza Internet y vuelve con un JSON bajo el brazo.*
>
> *El camarero te trajo exactamente lo que pediste: status 200 y los datos listos para usar.*
>
> *Y ahora que sabes hablar con la web, toca llamar a las APIs de verdad: las comerciales.*

PRÓXIMAMENTE EN UD 8: *APIs comerciales. OpenWeatherMap, OpenAI y las API keys que nunca, jamás, se suben a GitHub.*

✅ Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Cubierto
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3, 6-8 + ⚡ Laboratorio de tortura
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1-5 y 7 + ⚡ Laboratorio de tortura

> RA4c (Servidores concurrentes) y RA4d (ThreadPool) se cubren en la **UD 6 · Servidores concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **UD 10 · Alta disponibilidad**.

📚 Volver al índice de la unidad · **Anterior:** 08 · Práctica: mini cliente de API · **Siguiente:** UD 8 · APIs Comerciales

U08 — APIs Comerciales

Consumir OpenWeatherMap y OpenAI con claves y cuidado 💉

---

*En la U07 consumiste APIs públicas y gratuitas: sin clave, sin límites, sin miedo. Ahora toca el mundo real: las APIs comerciales te dan una **API key**, te limitan las peticiones, se caen y tienes que gestionar sus errores como un adulto.*

En esta unidad consumirás dos de las APIs más usadas del planeta: **OpenWeatherMap** para el tiempo y **OpenAI** para el cerebro artificial. Aprenderás a guardar tus claves con seguridad en variables de entorno, a gestionar rate limits y errores HTTP con elegancia, y a construir un programa completo que habla de verdad con ambas. Todo lo que viste en la UD 7 · HTTP y APIs REST —GET, JSON, `requests`— se viste aquí de traje de gala.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una **API key**, para qué sirve y por qué nunca debe ir en el código.
- Guardar credenciales en **variables de entorno** con `python-dotenv` y proteger tu repo con `.gitignore`.
- Consumir **OpenWeatherMap**: URL, parámetros (`q`, `appid`, `units`, `lang`) y parseo de la respuesta JSON.
- Llamar a la **API de OpenAI**: chat completions, roles de mensaje (`system`/`user`), `max_tokens` y `temperature`.
- Distinguir los **códigos de error** de una API comercial (401, 403, 404, 429, 500).
- Gestionar **rate limits** con esperas, `raise_for_status()` y **backoff exponencial**.
- Aplicar **buenas prácticas de seguridad**: rotación de claves, gitignore y nunca exponer credenciales.
- Construir un **programa completo** que consume OpenWeatherMap y OpenAI con las claves en `.env`.
- Conocer **herramientas para probar APIs** (Postman, curl, httpie, Insomnia).

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · API Keys	El carnet de identidad: qué es, cómo se obtiene y dónde se manda	Todos
02 · Variables de entorno	`python-dotenv`, el `.env` y por qué tus claves nunca van al repo	Todos
03 · OpenWeatherMap	El tiempo de tu ciudad: URL, parámetros y parseo del JSON	Todos
04 · OpenAI	El cerebro artificial: chat completions, mensajes y roles	Todos
05 · Rate limiting	Límites, cuotas y el temido 429: espera y backoff	Todos
06 · Errores HTTP	401, 403, 429 y 500: gestionar fallos con `raise_for_status`	Todos
07 · Seguridad y buenas prácticas	Rotación de claves, gitignore y no exponer secretos	Todos
08 · Práctica: APIs comerciales	Sé el código, el programa completo y los ejercicios del lápiz	Todos
09 · Cierre	Sé la petición, Fireside, Laboratorio de tortura, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Dónde se cubre
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3-5 y 8 + ⚡ Laboratorio (punto 9)
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1, 5-6 y 8 + ⚡ Laboratorio (punto 9)

> RA4c (Servidores concurrentes) y RA4d (ThreadPool) se cubren en la **UD 6 · Servidores concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **UD 10 · Alta disponibilidad**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 7 y dominas `requests`, GET y JSON? Perfecto, ese es el trampolín ideal: repasa la UD 7 · HTTP y APIs REST para tener fresco `params=` y `resp.json()`, y arranca en el punto 1, que parte justo de ahí: de la petición con `requests` a la petición con clave.

¿Ya tienes claves de OpenWeatherMap y OpenAI y solo necesitas consumirlas? Ve directo al punto 3 y al punto 4. Si vienes de cero en APIs, no te saltes los puntos 1 y 2: la API key y el `.env` son la base de todo lo demás.

📍 **Primer punto:** 01 · API Keys

⏭️ **Al acabar la unidad, continúa en UD 9 · Seguridad y cifrado.**

01 — API Keys

El carnet de identidad de las APIs comerciales 🎫

> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 01 · API Keys

📬 La idea en una frase

> Una **API key** es el carnet de identidad que te da la API comercial: identifica quién eres, cuánto puedes usar y quién te facturará si te pasas.

Las APIs de verdad no son de mentira. Te piden API key, te limitan las peticiones, se caen y tienes que gestionar sus errores como un adulto. La API key es la puerta de entrada a todo eso.

🎫 ¿Qué es una API key?

Una **API key** (o token) es una cadena secreta que la API te asigna al registrarte. Cada petición que haces la lleva contigo, y el servidor sabe así tres cosas:

Qué sabe el servidor	Ejemplo
Quién eres	El usuario `sergio` tiene la key `abc123...`
Cuánto puedes usar	Tu plan gratis permite 60 peticiones/minuto
Quién paga el exceso	Si pasas del plan, la factura cae en tu cuenta

Es exactamente el **carnet de identidad** del mundo de las APIs: sin él, el servidor no sabe si eres un estudiante probando o un robot mandando 10.000 peticiones por segundo.

🔑 ¿Cómo se obtiene?

1. Te registras en la web del proveedor (OpenWeatherMap, OpenAI, GitHub…).
2. Creas una clave desde el **panel de control** (suele llamarse “API Keys”, “Tokens” o “Credentials”).
3. Copias la cadena y **nunca, jamás, la subes a GitHub**.

> Las claves de OpenAI empiezan por `sk-` y las de OpenWeatherMap son una cadena alfanumérica larga. En este curso usarás siempre el marcador `TU_API_KEY` para no escribir claves reales.

📍 ¿Dónde va la clave?

Hay dos formas habituales de enviar la clave en la petición:

1. **Como parámetro de la URL** (lo usa OpenWeatherMap con `appid`):

   ```
   GET /data/2.5/weather?q=Sevilla&appid=TU_API_KEY&units=metric
   ```

2. **Como cabecera `Authorization`** (lo usa OpenAI con `Bearer`):

   ```
   Authorization: Bearer TU_API_KEY
   ```

Con `requests`, las dos formas se escriben así:

```python
import requests

API_KEY = "TU_API_KEY"  # ⚠️ NUNCA subas esto a GitHub

# Opción 1: la clave en la query string (params)
resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)

# Opción 2: la clave en la cabecera Authorization (headers)
resp = requests.get(
    "https://api.github.com/user",
    headers={"Authorization": f"Bearer {API_KEY}"}
)
```

> ⚠️ **Ojo con la URL:** si la clave va en la query string, queda registrada en los logs del servidor y en tu historial. Las cabeceras son más discretas. Por eso OpenAI exige `Bearer` y OpenWeatherMap (más antiguo) usa `appid`.

🔑 La analogía del llavero de acceso

Una API key es como el **llavero de acceso a un edificio de oficinas**:

- Sin llavero no entras: el servidor te responde **401 Unauthorized**.
- Tu llavero tiene un **nivel**: el de estudiante abre la planta del laboratorio, no la sala de servidores (eso sería un **403 Forbidden**).
- Si pierdes el llavero, no vale "borrar el código": hay que **anular la cerradura** y pedir uno nuevo (rotar la clave).

Y la regla de oro que repetiremos toda la unidad:

> **Regla de oro**: las API keys nunca van en el código. Usa variables de entorno.

🧠 Mini-chequeo

1. ¿Qué tres cosas sabe el servidor cuando le envías tu API key?
2. ¿Dónde puede viajar la clave en una petición HTTP? Pon un ejemplo de cada sitio.
3. ¿Qué te devuelve la API si la clave es incorrecta?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Una API key es el carnet de identidad de las APIs comerciales: identifica quién eres y cuánto puedes usar.
- Se obtiene registrándote en el proveedor y se envía en la query string (`appid`) o en la cabecera `Authorization`.
- Las API keys nunca van en el código: en el punto 2 verás dónde se guardan de verdad.

🐛 Vocabulario rápido

Término	Idea general
API key	Carnet de identidad de la API: identifica quién eres

Término	Idea general
Token	Sinónimo de clave de acceso
appid	Parámetro donde OpenWeatherMap espera tu clave
Authorization	Cabecera HTTP donde se mandan claves y tokens
Bearer	Esquema de autorización de la cabecera (`Bearer TU_API_KEY`)
401	Código de error cuando la clave es inválida o falta

📚 Volver al índice de la unidad · **Siguiente:** 02 · Variables de entorno

02 — Variables de entorno

Nunca subas tus claves: python-dotenv y el .env 🔒

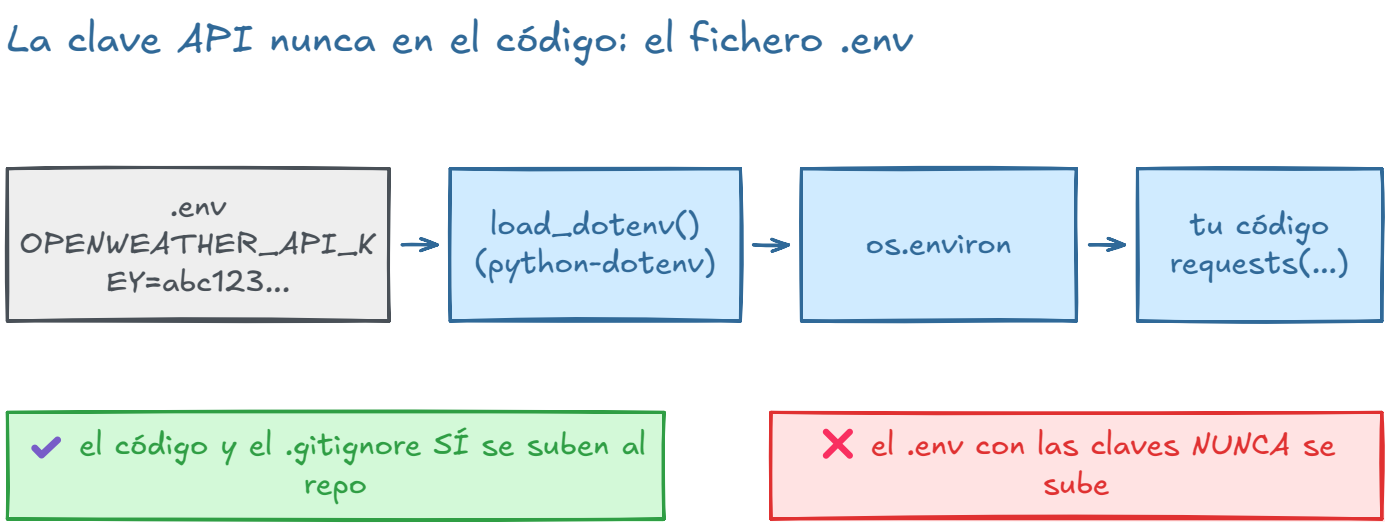
> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 02 · Variables de entorno

📬 La idea en una frase

> Las claves nunca van en el código: se guardan en un archivo **`.env`** que se carga con **`python-dotenv`**, y en Python se leen con **`os.getenv()`**.

Así el código queda limpio (nadie ve tus secretos al abrir el repo) y las claves viven en un archivo que no se sube a git.

📁 El archivo `.env`



Un archivo `.env` es un fichero de texto plano en la raíz de tu proyecto donde cada línea define una variable de entorno:

```
OPENWEATHER_API_KEY=abc123...
OPENAI_API_KEY=sk-...
```

Este archivo NO se sube a git. Es tuyo, de tu máquina, y cada persona (o cada CI) tiene el suyo. Si alguien abre tu repo en GitHub, debería encontrar el código pero nunca el `.env`.

⚙️ Cargarlo con `python-dotenv`

Primero se instala la librería:

```
pip install python-dotenv
```

Y en el código, al arrancar, se carga el `.env` y se leen las variables:

```python
# pip install python-dotenv
from dotenv import load_dotenv
import os

load_dotenv()  # Carga el archivo .env

API_KEY = os.getenv("OPENWEATHER_API_KEY")
if not API_KEY:
    raise ValueError("❌ Falta OPENWEATHER_API_KEY en .env")
```

Desglose de las tres piezas:

Línea	Qué hace
`load_dotenv()`	Lee el `.env` y mete cada variable en el entorno del proceso
`os.getenv("OPENWEATHER_API_KEY")`	Devuelve el valor, o `None` si no existe
`if not API_KEY: raise ValueError(...)`	Frena el programa con un mensaje claro si falta la clave

> 💡 El `raise ValueError` es tu salvavidas: en lugar de llegar a una petición con `None` y fallar con un error críptico del servidor, tu programa muere al instante diciéndote **qué** clave falta.

🚫 No subir claves al repo

Git nunca va a saber por sí solo que el `.env` es secreto: hay que decírselo con un **`.gitignore`**. Este archivo le dice a git qué ficheros no debe rastrear:

.gitignore

```
.env
```

Con ese `.gitignore` en tu repo, `git add .` ya no se llevará el `.env` por delante.

> ⚠️ **Aviso para adultos:** si algún día subes una clave por error, borrarla del código NO la borra del historial de git. La única solución es **rotar la clave** (crear una nueva y anular la vieja). Lo verás en el punto 7.

🧠 Mini-chequeo

1. ¿Qué hace `load_dotenv()` exactamente?
2. ¿Qué devuelve `os.getenv("MI_CLAVE")` si la variable no existe?
3. ¿Por qué el `.env` no se sube a git y cómo se lo dices a git?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- Las claves se guardan en un archivo `.env` de texto plano, fuera del código.
- `load_dotenv()` lo carga y `os.getenv("CLAVE")` devuelve el valor (o `None`).
- Un `.gitignore` con `.env` evita que tus secretos lleguen a GitHub.

🐛 Vocabulario rápido

Término	Idea general
`.env`	Archivo de texto donde viven tus variables secretas
`load_dotenv()`	Carga el `.env` al entorno del proceso
`os.getenv()`	Lee el valor de una variable de entorno (o `None`)
`.gitignore`	Lista de ficheros que git ignora
Variable de entorno	Valor externo que el código lee, nunca incrustado

📚 Volver al índice de la unidad · **Anterior:** 01 · API Keys · **Siguiente:** 03 · OpenWeatherMap

03 — OpenWeatherMap

El tiempo en tu ciudad con una llamada 🌤️

> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 03 · OpenWeatherMap

📬 La idea en una frase

> **OpenWeatherMap** te da el tiempo de cualquier ciudad con un solo GET: construyes la URL, pasas `q`, `appid`, `units` y `lang`, y parseas el JSON que te devuelve.

Es la API comercial más sencilla que vas a tocar en el módulo: perfecta para estrenar tu primera API key de verdad.

🗺️ Registro y API key

1. Entra en openweathermap.org y crea una cuenta gratuita.
2. En el panel, apartado **API Keys**, te generan tu clave personal.
3. Guárdala en tu `.env` (punto 2) como `OPENWEATHER_API_KEY`.

> 🎁 **Plan gratis:** suficiente para clase. Te permite 60 peticiones por minuto (el límite exacto lo verás en el punto 5).

📍 La URL y sus parámetros

El endpoint del tiempo actual es:

```
https://api.openweathermap.org/data/2.5/weather
```

Parámetro	Qué hace	Ejemplo
`q`	La ciudad (o “ciudad,país”)	`q=Sevilla`
`appid`	Tu API key (punto 1)	`appid=TU_API_KEY`
`units`	Unidades: `metric`, `imperial`, `standard`	`units=metric`

Parámetro	Qué hace	Ejemplo
`lang`	Idioma de las descripciones	`lang=es`

> 💡 Sin `units=metric` la temperatura llega en **kelvin**. Con `metric` llega en grados centígrados: mucho más amigable.

🌤️ Llamada y parseo

```
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

def clima(ciudad):
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}

    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"❌ Error {resp.status_code}: {resp.text}")
        return

    datos = resp.json()
    print(f"🌤️  {datos['name']}: {datos['main']['temp']}°C")
    print(f"   Sensación: {datos['main']['feels_like']}°C")
    print(f"   Humedad: {datos['main']['humidity']}%")
    print(f"   {datos['weather'][0]['description']}")

clima("Sevilla")
```

Respuesta JSON (simplificada):

```
{
  "name": "Sevilla",
  "main": { "temp": 18.5, "feels_like": 17.2, "humidity": 65 },
  "weather": [{ "description": "cielo claro" }]
}
```

Los campos que usamos:

Ruta en el JSON	Qué es
`datos['name']`	El nombre de la ciudad
`datos['main']['temp']`	Temperatura actual
`datos['main']['feels_like']`	Sensación térmica
`datos['main']['humidity']`	Humedad en %
`datos['weather'][0]['description']`	Texto descriptivo del cielo

> 💡 `weather` es una **lista** de diccionarios (porque puede haber varias condiciones), por eso se accede con `[0]`.

🧠 Mini-chequeo

1. ¿Qué parámetro convierte la temperatura a grados centígrados?
2. ¿Por qué `datos['weather'][0]['description']` necesita el `[0]`?
3. ¿Qué compruebas antes de hacer `resp.json()` en la función `clima`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- OpenWeatherMap devuelve el tiempo con un GET a `/data/2.5/weather` pasando `q`, `appid`, `units` y `lang`.
- La respuesta es JSON anidado: `main` guarda temperaturas y `weather` la descripción.
- Comprobar el `status_code` antes de parsear evita errores feos cuando la API falla.

🐛 Vocabulario rápido

Término	Idea general
Endpoint	La URL concreta de un recurso (`/data/2.5/weather`)
`q`	Parámetro de ciudad
`units=metric`	Devuelve grados centígrados en lugar de kelvin
`lang=es`	Descripciones del tiempo en español
`main`	Objeto con temperaturas y humedad
`weather`	Lista con las descripciones del cielo

📚 Volver al índice de la unidad · **Anterior:** 02 · Variables de entorno · **Siguiente:** 04 · OpenAI

04 — OpenAI

El cerebro artificial al alcance de una API 🤖

> 🗺️ **Estás en:** 🍼 **UD 8 · APIs Comerciales** → 04 · OpenAI

📬 La idea en una frase

> La API de OpenAI convierte un chat en una función: le pasas una lista de **mensajes con roles** y te devuelve la respuesta de texto que puedes imprimir.

No hay modelo instalado en tu máquina: todo vive en los servidores de OpenAI y tú solo envías peticiones HTTP. Exactamente el mismo patrón que en OpenWeatherMap, pero con un `POST` y una cabecera `Authorization`.

🧠 Registro y clave

1. Crea una cuenta en platform.openai.com.
2. Genera tu clave en **API Keys**: empieza por `sk-`.
3. Guárdala en tu `.env` como `OPENAI_API_KEY` (nunca en el código).

> ⚠️ A diferencia de OpenWeatherMap, la clave de OpenAI se manda en la **cabecera** `Authorization: Bearer sk-...`, no en la URL.

💬 Chat completions

La librería oficial `openai` hace todo el trabajo. Se instala con `pip install openai`:

```python
from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()

cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

respuesta = cliente.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Eres un profesor de Python divertido."},
        {"role": "user", "content": "Explica qué es un Lock en 2 frases."}
    ],
    max_tokens=100,
    temperature=0.7
)

print(respuesta.choices[0].message.content)
```

Desglose:

Pieza	Qué hace
`OpenAI(api_key=...)`	Crea el cliente con tu clave del `.env`
`model`	Qué modelo usar (`gpt-4o-mini`, barato y suficiente para clase)
`messages`	La conversación como lista de mensajes con rol
`max_tokens`	Longitud máxima de la respuesta (aquí 100)
`temperature`	Creatividad: 0 = literal, 2 = desatado (en la práctica se usa 0-1)
`respuesta.choices[0].message.content`	El texto de la respuesta

📜 Mensajes y roles

Una conversación con la API es una **lista de mensajes**, y cada mensaje lleva un rol:

Rol	Quién habla	Ejemplo
`system`	Las instrucciones del asistente	"Eres un profesor de Python divertido."
`user`	El humano	"Explica qué es un Lock en 2 frases."
`assistant`	Las respuestas previas del modelo	(lo rellena la API)

> 💡 El mensaje `system` es la clave del truco: define **el carácter y las reglas** del asistente. El `user` es la pregunta, y en una conversación larga vas encadenando `user` → `assistant` → `user` → …

🔄 Alternativa con `httpx` (sin librería oficial)

Si no quieres la librería `openai`, el mismo chat es una petición HTTP normal. Esto te enseña lo que hay debajo:

```
import httpx, os
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("OPENAI_API_KEY")

resp = httpx.post(
    "https://api.openai.com/v1/chat/completions",
    headers={"Authorization": f"Bearer {API_KEY}"},
    json={
        "model": "gpt-4o-mini",
        "messages": [{"role": "user", "content": "Hola!"}]
    }
)
print(resp.json()["choices"][0]["message"]["content"])
```

Fíjate: es un `POST` con un JSON en el cuerpo y la clave en la cabecera. La librería oficial no es magia, solo envuelve esto.

💰 Consumo y coste

OpenAI **no es gratis**, pero para uso educativo es casi simbólico:

Dato	Valor
Precio de gpt-4o-mini	~$0,15 por 1M tokens de entrada + ~$0,60 por 1M de salida
Tokens de una pregunta normal	~100 tokens
Coste de esa pregunta	~$0.00002 (¡menos de un céntimo!)

Un token es, aproximadamente, una pieza de palabra. Con `max_tokens` controlas cuánto gastas por llamada.

> 🧻 Las preguntas tontas del curso te lo recuerdan en el cierre: "¿OpenAI es caro? gpt-4o-mini cuesta centésimas de céntimo por pregunta. Muy barato para pruebas."

🧠 Mini-chequeo

1. ¿En qué cabecera y con qué esquema viaja la clave de OpenAI?
2. ¿Qué rol define el comportamiento del asistente? ¿Cuál es la pregunta del usuario?
3. ¿Qué hace `temperature=0.9` frente a `temperature=0`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- OpenAI recibe una lista de mensajes con roles (`system`, `user`, `assistant`) y devuelve el texto en `choices[0].message.content`.
- La clave viaja en `Authorization: Bearer`, y se controla el gasto con `max_tokens` y `temperature`.
- Sin la librería oficial, todo se reduce a un `POST` a `/v1/chat/completions` con `httpx`.

🐛 Vocabulario rápido

Término	Idea general
Chat completions	El endpoint que genera texto de conversación
messages	La lista de mensajes que define la conversación
`system`	Rol que fija las reglas del asistente
`temperature`	Parámetro de creatividad (0-2, en la práctica 0-1)
`max_tokens`	Tope de longitud (y de coste) de la respuesta
Bearer	Esquema de autorización en la cabecera

📚 Volver al índice de la unidad · **Anterior:** 03 · OpenWeatherMap · **Siguiente:** 05 · Rate limiting

05 — Rate limiting

Límites, cuotas y el temido 429 🧢

> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 05 · Rate limiting

📬 La idea en una frase

> Toda API comercial **limita cuánto puedes pedir**: si te pasas, responde **429 Too Many Requests**, y tu programa debe esperar antes de reintentar.

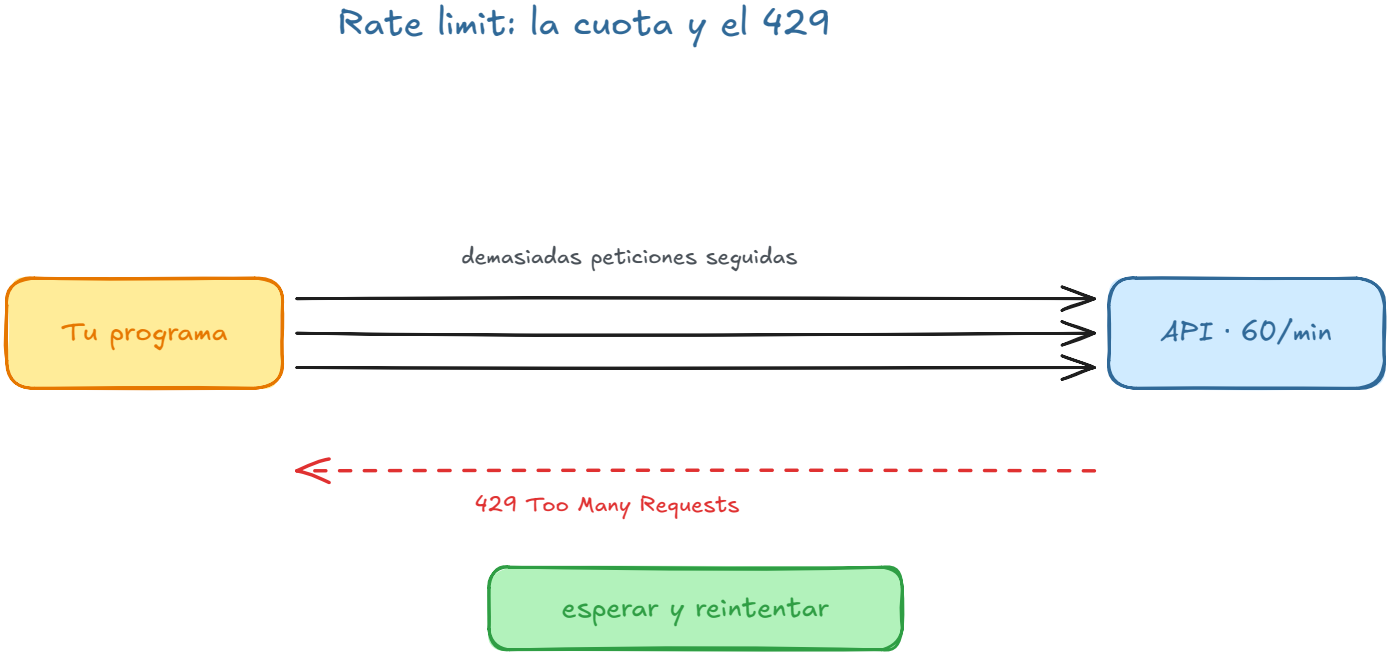
No es un castigo: es protección. Un cliente desbocado derribaría el servidor de todos los demás usuarios.

⏳ ¿Qué es el rate limit?

El **rate limit** es la cuota de peticiones que tu API key puede hacer en un intervalo de tiempo. Si la superas, la API deja de responderte.

API	Límite típico del plan gratis
OpenWeatherMap	60 peticiones/minuto
GitHub API	60 peticiones/hora
OpenAI	Por tokens por minuto (varía con el plan)

Estos límites existen por una razón sencilla: **una API no es tu base de datos**. Un bucle sin freno que pregunta el tiempo de 1.000 ciudades en un segundo es exactamente el comportamiento que quieren cortar.



la cuota protege el servicio para todos: esperas, no machacas

---

🐢 El error 429

Cuando te pasas del límite, la respuesta es:

```
HTTP/1.1 429 Too Many Requests
```

Y OpenWeatherMap añade un mensaje en el JSON:

```
{ "cod": 429, "message": "You have exceeded your quota." }
```

> 💡 Algunas APIs van más allá y **bloquean tu IP** temporalmente si insistes. Por eso el 429 se toma en serio: esperas, no machacas.

😴 Esperar antes de reintentar

La respuesta natural a un 429 es **parar, esperar y volver a intentar**:

```python
import requests
import time

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": "TU_API_KEY", "units": "metric"}
)

if resp.status_code == 429:
    print("🐢 Demasiadas peticiones. Esperando...")
    time.sleep(5)  # Respiramos 5 segundos antes de reintentar
```

Si quieres comprobar cuánto te queda, muchas APIs mandan cabeceras con la cuota restante (por ejemplo `X-RateLimit-Remaining`).

🔁 Reintentos inteligentes (backoff)

Esperar siempre lo mismo no es la mejor idea: si el servidor está saturado, mejor ir espaciando cada vez más los reintentos. Ese es el **backoff exponencial**:

```python
import time, requests

def conectar_con_backoff(url, params, max_intentos=5):
    for intento in range(max_intentos):
        try:
            resp = requests.get(url, params=params, timeout=5)
            resp.raise_for_status()
            return resp.json()
        except requests.exceptions.RequestException:
            espera = 2 ** intento  # 1, 2, 4, 8, 16 segundos
            print(f"⚠️ Intento {intento+1} fallido. Esperando {espera}s...")
            time.sleep(espera)
    raise Exception("No se pudo conectar tras varios intentos")
```

La progresión de esperas con `2 ** intento`:

```
Intento 1 fallido → espera 1s
Intento 2 fallido → espera 2s
Intento 3 fallido → espera 4s
Intento 4 fallido → espera 8s
Intento 5 fallido → espera 16s → se rinde
```

> El **backoff exponencial** evita saturar un servidor que ya está tocado. Es la técnica que usan los clientes profesionales y la retomarás en la UD 10 · asyncio para verla de otra forma.

🧠 Mini-chequeo

1. ¿Qué código HTTP indica que te has pasado del límite de peticiones?
2. ¿Por qué no es buena idea reintentar sin parar cuando ves un 429?
3. ¿Cuánto espera `conectar_con_backoff` antes del intento 4?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Las APIs limitan tus peticiones por tiempo (cuota) y te avisan con **429** cuando te pasas.
- La respuesta profesional es esperar y reintentar con calma, nunca machacar.
- El **backoff exponencial** duplica la espera entre intentos (`1, 2, 4, 8…`) para no saturar al servidor.

🐛 Vocabulario rápido

Término	Idea general
Rate limit	Cuota de peticiones por tiempo
Cuota	El máximo que tu plan te permite
429	Too Many Requests: te has pasado del límite
Backoff exponencial	Espera que se duplica entre reintentos

Término	Idea general
X-RateLimit-Remaining	Cabecera con las peticiones que te quedan

📚 Volver al índice de la unidad · **Anterior:** 04 · OpenAI · **Siguiente:** 06 · Errores HTTP

06 — Errores HTTP

401, 403, 429, 500: el servidor te está hablando ⚠️

> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 06 · Errores HTTP

📬 La idea en una frase

> `requests` **no lanza errores por sí solo**: tú compruebas el `status_code` o llamas a `raise_for_status()`, y capturas las excepciones para que tu programa no reviente.

Las APIs comerciales **fallan**. Y fallan a menudo. Prepárate.

🚨 Los códigos que verás

Estos son los códigos que una API comercial te va a devolver en la vida real:

Código	Qué significa	Causa típica
`400`	Bad Request	Enviaste mal el JSON o los parámetros
`401`	Unauthorized	La API key falta o es inválida
`403`	Forbidden	La key es válida pero sin permiso para eso
`404`	Not Found	La URL o el recurso no existe
`429`	Too Many Requests	Te pasaste del rate limit (punto 5)
`500`	Internal Server Error	El servidor de la API está tocado
`503`	Service Unavailable	La API está de mantenimiento o caída

> 💡 La regla mnemotécnica de la UD 7 sigue valiendo: los **4xx** son tu culpa (mira la URL, la key o los parámetros), los **5xx** son del servidor.

🧯 `raise_for_status()` y try/except

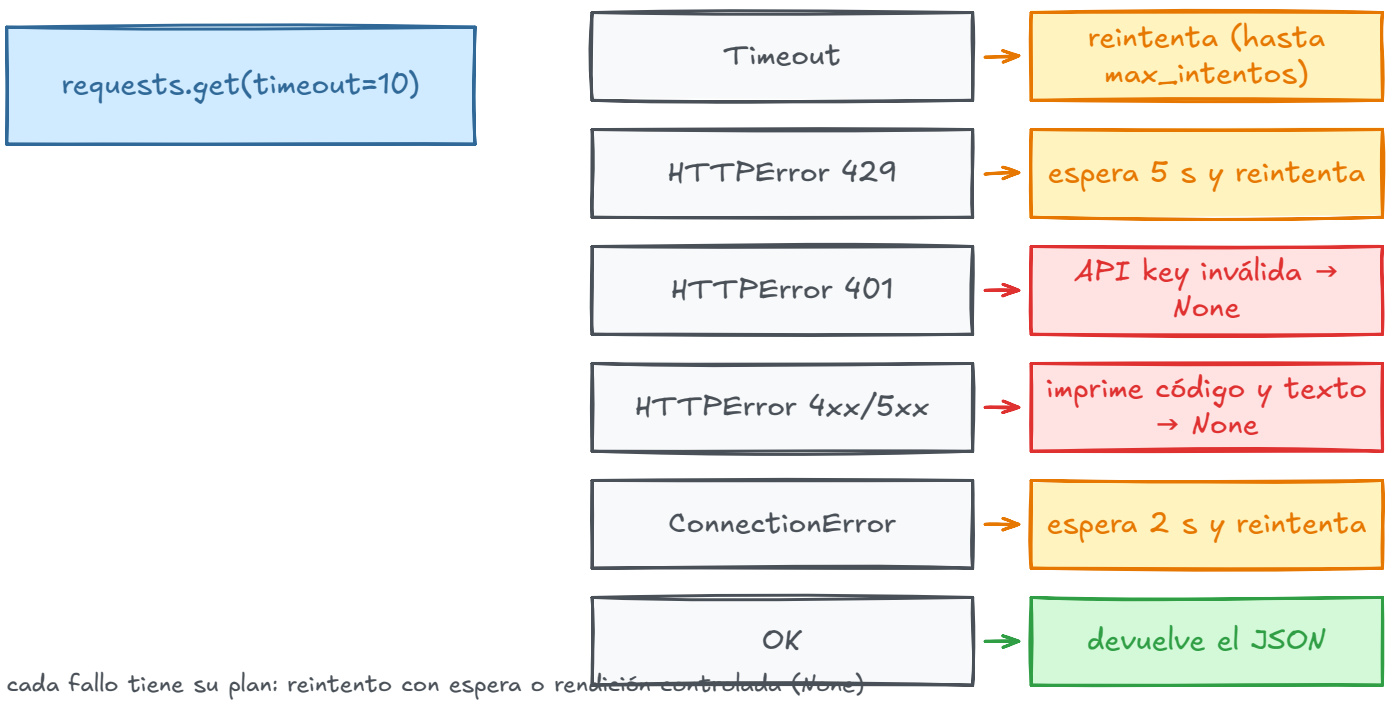
`requests` no lanza excepción cuando recibe un 404 o un 500: te devuelve la respuesta y sigues. Para convertir los errores HTTP en excepciones gestionables existe `raise_for_status()`:

```
import requests

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": "falsa", "units": "metric"}
)
print(resp.status_code)  # 401 — la key es inválida

resp.raise_for_status()  # Lanza una excepción HTTPError si es 4xx o 5xx
```

> ⚠️ **Error típico de novato:** llamar a `resp.json()` sin comprobar el código. Si el cuerpo es un error (o no es JSON), `resp.json()` lanza una excepción y el programa revienta con un mensaje que no dice nada útil.

🛡️ Una llamada segura de verdad

Juntando los códigos, el timeout y los reintentos, la función robusta queda así:



El flujo de decisión, en un vistazo:

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre 401 y 403?
2. ¿Qué hace `raise_for_status()` y qué excepción lanza?
3. ¿Por qué no debes llamar a `resp.json()` sin comprobar antes el código de estado?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Los códigos de una API comercial se dividen en 4xx (tu error) y 5xx (error del servidor).
- `raise_for_status()` convierte los códigos malos en excepciones capturables con try/except.
- Una llamada segura combina timeout, comprobación de códigos y reintentos, como `llamada_segura`.

🐛 Vocabulario rápido

Término	Idea general
401 / 403	No autenticado / autenticado sin permiso
429	Rate limit superado
500 / 503	Error del servidor / API caída
`raise_for_status()`	Lanza HTTPError si el código es 4xx o 5xx
`requests.exceptions.Timeout`	La petición no obtuvo respuesta a tiempo
`requests.exceptions.ConnectionError`	No se pudo conectar al servidor

📚 Volver al índice de la unidad · **Anterior:** 05 · Rate limiting · **Siguiente:** 07 · Seguridad y buenas prácticas

07 — Seguridad y buenas prácticas

Protege tus claves como si fueran la llave de tu casa 🔐

> 🗺️ **Estás en:** 🧪 **UD 8 · APIs Comerciales** → 07 · Seguridad y buenas prácticas

📬 La idea en una frase

> Una **API key filtrada es una factura en tu nombre**: nunca se sube a GitHub, se **rota** si se filtra y se guarda siempre en el `.env`.

Cualquiera con tu clave puede usarla (y gastar tu cuota, o peor, tu dinero). La seguridad aquí es una decisión de cada `git commit`.

🚫 Lo que nunca debes hacer

Esto funciona, pero es una bomba de relojería:

```
# ❌ MAL: la clave incrustada en el código
import requests

API_KEY = "sk-1234567890abc"  # 🚨 ¡Alguien la verá!

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)
```

En cuanto hagas `git push`, esa cadena viaja a GitHub y queda ahí **para siempre** (aunque luego la borres). El servidor de OpenWeatherMap cobrará en tu cuenta cada uso que haga quien la encuentre.

La versión correcta:

```python
# ✅ BIEN: la clave vive en el .env, el código no sabe nada
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)
```

🗂️ El `.gitignore` y el commit

El `.gitignore` es tu primera línea de defensa:

.gitignore

```
.env
```

> 🔍 **Comprueba antes de commitear:** `git status` debe mostrar tu `.env` como ignorado. Si aparece en la lista, para: no hagas `git add .` sin mirar.

Y si algún día te preguntas "¿puedo guardar la API key en el código para pruebas?" — la respuesta honesta:

> Solo si el código **nunca** va a GitHub. Mejor acostúmbrate a `.env` desde el principio: es un hábito, no una técnica.

🔁 Rotación de claves

Si una clave se filtra (la subiste a GitHub, la enviaste por correo, la viste en un log), la solución **no es borrar** —ya es tarde— sino **rotar**:

1. Entra en el panel del proveedor (OpenWeatherMap, OpenAI…).
2. **Revoca** la clave filtrada (la anulas al momento).
3. **Genera una nueva** clave y actualiza tu `.env`.

La metáfora del punto 1: si pierdes el llavero del edificio, no vales con “olvidé dónde lo dejé”. Cambias la cerradura.

🛡️ Checklist de buenas prácticas

☐	Práctica	Por qué
☐	Claves en el `.env`	El código queda limpio y sin secretos
☐	`.gitignore` con `.env`	Git no se lleva tus claves al repo
☐	Revisar `git status` antes de commitear	Cazas errores antes de que sean públicos
☐	Rotar la clave si se filtra	Anulas el daño al momento
☐	Clave en la cabecera, no en la URL	La URL queda en logs e historial
☐	No usar la misma clave en varios proyectos	Un fallo no compromete todo

---

🧠 Mini-chequeo

1. ¿Por qué borrar una clave del código no basta si ya subió a GitHub?
2. ¿Qué es rotar una clave y cuándo se hace?
3. Escribe el `.gitignore` mínimo de este módulo.

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Una clave en el código es una clave en GitHub: el `.env` + `.gitignore` la mantienen fuera del repo.
- Si una clave se filtra, la única solución real es **rotarla** (revocar y crear otra).
- La seguridad no es una función, es un hábito de cada commit: revisar `git status` antes de subir.

🐛 Vocabulario rápido

Término	Idea general
Filtrar una clave	Que una clave salga a un sitio público (GitHub, logs)
Rotar	Revocar la clave comprometida y generar una nueva
`.gitignore`	Archivo que excluye ficheros de git
Secretos	Credenciales que nunca deben hacerse públicas
Panel del proveedor	Web donde generas y revocas tus claves

📚 Volver al índice de la unidad · **Anterior:** 06 · Errores HTTP · **Siguiente:** 08 · Práctica: APIs comerciales

08 — Práctica: APIs comerciales

Sé el código, el programa completo y los ejercicios del lápiz 🛠️

> 🗺️ **Estás en:** 🧪 **UD 8 · APIs Comerciales** → 08 · Práctica: APIs comerciales

---

📬 La idea en una frase

> Aquí juntas todo lo aprendido en un solo programa: un **cliente de APIs comerciales** que pide el tiempo a OpenWeatherMap y charla con OpenAI, con las claves en el `.env` — y un paseo "siendo" la petición.

De aquí saldrás con el programa más real del módulo: dos APIs de pago consumidas como se hace en producción.

🏗️ El programa completo: tiempo + cerebro

```python
import requests
from dotenv import load_dotenv
import os
from openai import OpenAI

load_dotenv()
OPENWEATHER_KEY = os.getenv("OPENWEATHER_API_KEY")
OPENAI_KEY = os.getenv("OPENAI_API_KEY")

if not OPENWEATHER_KEY or not OPENAI_KEY:
    raise ValueError("❌ Faltan claves en .env")


def clima(ciudad):
    """Devuelve y muestra el tiempo actual de una ciudad."""
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": ciudad, "appid": OPENWEATHER_KEY, "units": "metric", "lang": "es"}
    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"❌ {ciudad}: {resp.text}")
        return
    datos = resp.json()
    print(f"🌤️  {datos['name']}: {datos['main']['temp']}°C "
          f"({datos['weather'][0]['description']})")


def preguntar_a_gpt(pregunta):
    """Envía una pregunta a OpenAI y devuelve la respuesta."""
    cliente = OpenAI(api_key=OPENAI_KEY)
    respuesta = cliente.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "Eres un profesor de Python divertido."},
            {"role": "user", "content": pregunta}
        ],
        max_tokens=100,
        temperature=0.7
    )
    return respuesta.choices[0].message.content


if __name__ == "__main__":
```

```
    clima("Madrid")
    clima("Barcelona")
    clima("Sevilla")
    print("\n💬 GPT dice:", preguntar_a_gpt("¿Qué es un Lock en 2 frases?"))
```

Fíjate en el esqueleto que se repite en toda la unidad: clave del `.env` → `requests.get/OpenAI` → comprobar / parsear → usar los datos. Es el mismo patrón del punto 6 de la UD 7, ahora con API key.

⭐ Sé el código: la consulta meteorológica

> "Sé el programa que pregunta por el tiempo, desde que escribes 'python clima.py' hasta que ves el resultado."

```
$ python clima.py Barcelona
```

Traza interna:

```
1. load_dotenv() → carga .env → obtiene API_KEY
2. Construye URL: https://api.openweathermap.org/data/2.5/weather
   ?q=Barcelona&appid=abc123...&units=metric&lang=es

3. requests.get() → abre socket TCP
   → DNS lookup: api.openweathermap.org → 104.16.x.x
   → Conexión TLS (HTTPS)
   → Envía petición HTTP GET

4. ⏳ Latencia de red (~200ms)

5. Recibe HTTP 200 + JSON:
   HTTP/1.1 200 OK
   Content-Type: application/json
               {"name":"Barcelona","main":{"temp":21.3},"weather":[{"description":"nubes
dispersas"}]}

6. resp.json() → dict de Python
7. Extrae datos['main']['temp'] = 21.3
8. Extrae datos['weather'][0]['description'] = "nubes dispersas"
9. print → "🌤️  Barcelona: 21.3°C"
10. print → "   nubes dispersas" 🏁
```

En los pasos 3 a 5 ocurre lo que en la UO5 hacías a mano con sockets: DNS, TCP, TLS y el GET HTTP. La única novedad frente a la UD 7 es el paso 1: la **API key** que ahora viaja en la petición.

🛠️ Herramientas para probar APIs

Antes de escribir Python, puedes probar una API a mano con estas herramientas:

Herramienta	Tipo	Para qué
Postman	GUI	Probar APIs, colecciones, tests
curl	CLI	`curl https://api.github.com/users/python`
httpie	CLI moderno	`http https://api.github.com/users/python`
Insomnia	GUI	Alternativa open source a Postman

> 💡 Con Postman o curl puedes ver la respuesta cruda de OpenWeatherMap (con tu clave en `appid`) antes de escribir una sola línea de Python. Cuando la API responde en la herramienta, pasas el mismo GET a `requests`.

🖍️ Aprieta el lápiz

1. **Clima en 3 ciudades**: Usa OpenWeatherMap para mostrar el clima de Madrid, Barcelona y Sevilla a la vez.
2. **Chat con GPT**: Pregúntale a GPT-4o-mini qué es un semáforo en Python y que te ponga un ejemplo.
3. **API con errores**: Conéctate a una URL que no existe y captura el error. Luego a una con API key falsa (401).
4. **Backoff test**: Crea una función que intente conectarse a un servidor que no existe y muestre el tiempo de espera entre intentos.

> 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Qué tres bloques forman el programa completo de esta práctica?
2. En la traza del “Sé el código”, ¿qué nuevo paso añade la U08 frente a la UD 7?
3. ¿Para qué te sirve Postman o curl antes de escribir Python?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Un cliente de APIs comerciales = claves del `.env` + llamada a la API + parseo de la respuesta.
- El programa completo consume OpenWeatherMap (tiempo) y OpenAI (chat) siguiendo el mismo patrón `requests` de la UD 7.
- Postman, curl, httpie e Insomnia te dejan probar la API a mano antes de programar.

🐛 Vocabulario rápido

Término	Idea general
Cliente de API	Programa que consume una API
Traza interna	El camino paso a paso de la petición
`.env`	Donde viven las claves que el programa carga
Postman / Insomnia	GUIs para probar APIs
curl / httpie	CLIs para probar APIs

📚 Volver al índice de la unidad · **Anterior:** 07 · Seguridad y buenas prácticas · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé la petición con API key, laboratorio real y el ring final de la unidad 🧠

> 🗺️ **Estás en:** 💉 **UD 8 · APIs Comerciales** → 09 · Cierre

Has terminado la teoría: API keys, variables de entorno, OpenWeatherMap, OpenAI, rate limits, errores HTTP, seguridad y el programa completo. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real contra dos APIs de verdad con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé la petición

> *Eres una petición HTTP con una API key. Acabas de nacer en un `requests.get()` dentro del portátil de un estudiante y tu destino es `api.openweathermap.org`. Empieza el viaje.*

¿Qué pasa?

1. `requests` te construye: `https://api.openweathermap.org/data/2.5/weather?q=Sevilla&appid=TU_API_KEY&units=metric&lang=es`. Llevas el **carnet de identidad** (la API key) en la query string.
2. Abres la conexión: primero el **DNS lookup** (`api.openweathermap.org` → una IP), luego el **three-way handshake TCP** y el **handshake TLS** que te cifra para el viaje.
3. Atraviesas Internet hasta el servidor de OpenWeatherMap y te entrega el proceso que te espera en el puerto 443.
4. El servidor **te lee el carnet**: comprueba tu `appid` contra su base de claves y tu plan (¿cuánto puedes pedir por minuto?).
5. Te responde **200 OK** con un JSON: `{"name":"Sevilla","main":{"temp":18.5},...}`.
6. De vuelta en Python, `requests` parsea el JSON y te convierte en un **dict** (`datos["main"]["temp"]` → `18.5`).
7. El estudiante imprime 🌤️ `Sevilla: 18.5°C`. Misión cumplida.

Todo en ~200 ms. Con tu identidad verificada en cada viaje.

> 💡 **Ahora tú:** ¿y si la clave fuera inválida? En el paso 4 el servidor no te reconoce: te responde **401 Unauthorized** con un JSON de error. Si el estudiante no comprueba el

> `status_code` y llama directo a `resp.json()`, el programa muere con un mensaje que no dice nada útil. Por eso existe `raise_for_status()` (punto 6).

🔥 Fireside Chat: REST vs SOAP vs GraphQL

> *Tres estilos de API se sientan junto a la chimenea a discutir, de una vez, quién es el mejor.*

REST: — Mira, soy el estándar. Simple, recursos, JSON. Todo el mundo me conoce.

SOAP: — Yo fui el rey en los 2000. XML, WSDL, muy estructurado. Aún vivo en bancos.

GraphQL: — Yo soy el moderno. Una sola URL, pides exactamente lo que necesitas, ni más ni menos.

REST: — ¿Y eso es bueno? Yo tengo URLs claras: `/usuarios/5`, `/productos`.

GraphQL: — Sí, pero si quieres el nombre y el correo del usuario y los títulos de sus posts, ¿cuántas peticiones necesitas?

REST: — Dos: `/usuarios/5` y `/usuarios/5/posts`.

GraphQL: — Yo con una: `query { usuario(id:5) { nombre email posts { titulo } } }`.

REST: — Vale, eres más eficiente. Pero yo soy más simple para empezar.

GraphQL: — Tienes razón. Para proyectos pequeños, REST gana. Para grandes, yo escalo mejor.

> **Moraleja**: en este módulo manda REST: OpenWeatherMap y OpenAI (vía chat completions) son APIs REST con JSON. SOAP vive en los bancos y GraphQL es la alternativa eficiente para APIs grandes.

🕵️ ¿Quién soy?

1. Soy el carnet de identidad que la API comercial te da al registrarte.
2. Soy el código de estado cuando te pasas del límite de peticiones.
3. Soy el rol de mensaje de OpenAI que define el comportamiento del asistente.
4. Soy la función de `requests` que lanza una excepción si la respuesta es 4xx o 5xx.
5. Soy el archivo donde guardas tus claves y que nunca, jamás, se sube a git.
6. Soy la técnica que duplica la espera entre reintentos (1, 2, 4, 8…).

> 🔁 Respuestas

🤬 CONRAD VS EL MUNDO: “clave subida a GitHub”

CONRAD: — “Clásico: haces `git add . && git commit && git push` con la API key escrita en el código (o sin `.env` en el `.gitignore`). Razones por las que estás vendido: 1) **La clave queda en el historial de git para siempre**, aunque la borres después. 2) **Cualquiera que encuentre tu repo la usa**: tu cuota gratis se agota en una tarde, o peor, tu tarjeta paga el exceso. 3) La clave no caduca sola: es tu llavero, y lo dejaste en la puerta.”

CONRAD: — “Y lo mejor: *‘es que era un repo privado’*. ¡Pues claro! Los repos privados se hacen públicos, se comparten, se filtran. La seguridad no depende de la visibilidad del repo, sino de que el secreto **no esté ahí**.”

CONRAD: — “Y no me vengas con *‘¿será que la clave no se ve?’*. Ya lo sabes: **rota**. Revoca la clave filtrada en el panel del proveedor, genera una nueva, ponla en el `.env`, y añade `.env` al `.gitignore`. Primero el daño, luego el arreglo.”

⚡ Laboratorio de tortura: OpenWeatherMap + OpenAI

> **Duración:** 1 hora **Herramienta:** Python 3 (`requests`, `python-dotenv`, `openai`) y conexión a Internet

Escenario: monta un programa que consume las dos APIs comerciales de la unidad con tus claves reales guardadas en el `.env`. Es lo que repetirás en cualquier proyecto que necesite tiempo o IA.

Tareas paso a paso:

1. **Crea el `.env`** con tus dos claves reales: `OPENWEATHER_API_KEY=...` y `OPENAI_API_KEY=sk-...`. Añade `.env` al `.gitignore` antes de hacer nada más.
2. **Clima de tu ciudad**: escribe `clima.py` que cargue la clave con `load_dotenv()`, haga GET a OpenWeatherMap y muestre temperatura, sensación, humedad y descripción.
3. **Compara dos ciudades**: llama a la misma función con dos ciudades y muestra cuál está más caliente.

4. **GPT responde**: escribe `gpt.py` que pregunte a GPT-4o-mini qué es un Lock en Python con `max_tokens=100` y `temperature=0.7`.
5. **Gestiona el 401**: cambia la `appid` a una clave falsa y haz que el programa detecte el error sin reventar (comprueba `status_code`).
6. **Simula un 429**: haz un bucle de 70 peticiones seguidas a OpenWeatherMap (el plan gratis permite 60/min). ¿Qué código recibes a partir de la 61?

Fallo intencionado: en la tarea 2, en lugar de leer la clave del `.env` con `os.getenv`, escríbela a fuego en el código (`API_KEY = "TU_API_KEY"`) y haz `git add . && git commit`. La clave queda **en el historial de git para siempre**, aunque luego la borres y añadas el `.env` al `.gitignore`. Es el fallo más caro de esta unidad: el código funciona, pero acabas de regalar tu carnet de identidad.

> **Pista 1:** las claves son como las llaves de casa: si las dejas en la puerta (en un commit público), no vale con "recogerlas" (borrarlas). Hay que **cambiar la cerradura**: revocar la clave en el panel del proveedor y generar una nueva. Eso es rotar (punto 7).
>
> **Pista 2:** comprueba tú mismo que el daño es real con `git log -p`: ahí verás la clave escrita en el commit aunque ya la hayas borrado del fichero. Y para el resto del laboratorio, vuelve al `.env`: `load_dotenv()` + `os.getenv()` es la única vía.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Key Holder**	Obtener tu API key en OpenWeatherMap y OpenAI y guardarla en el `.env`
🏅 **Env Master**	Cargar claves con `python-dotenv` y proteger el repo con `.gitignore`
🏅 **Weather Rider**	Consumir OpenWeatherMap y parsear `main` y `weather` del JSON
🏅 **GPT Whisperer**	Llamar a chat completions con roles, `max_tokens` y `temperature`
🏅 **429 Survivor**	Sobrevivir al rate limit con esperas y backoff exponencial
🏅 **Key Guardian**	Rotar una clave filtrada y explicar por qué nunca va en el código

---

🧠 Atrévete a pensar

1. ¿Por qué las API keys no deben ir en la URL ni en el código?
2. ¿Qué diferencia hay entre 401 y 403?
3. ¿Qué es el rate limit y por qué las APIs comerciales lo imponen?
4. ¿Por qué el backoff exponencial es mejor que esperar siempre lo mismo?
5. ¿Qué harías si descubres que tu API key se ha subido a GitHub?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Código de estado cuando te pasas del límite de peticiones (3 letras)
4. Archivo donde guardas tus claves y nunca se sube a git (3 letras)
5. Parámetro de OpenWeatherMap que pone las unidades en grados (5 letras)
8. Código de estado cuando la API key es inválida (3 letras)

Vertical:
2. Rol de mensaje de OpenAI que define el comportamiento del asistente (6 letras)
3. Proveedor de clima con API key gratuita (14 letras)
6. Librería para peticiones HTTP en Python (8 letras)
7. Espera exponencial entre reintentos (7 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **"¿Qué es una API key y cómo la enviarías al servidor?"**
2. **"¿Cómo guardas las credenciales en una aplicación Python? ¿Por qué no en el código?"**
3. **"¿Qué es un rate limit? ¿Cómo gestionarías un 429?"**
4. **"Escribe en Python una llamada a OpenWeatherMap con `requests` y procesa la respuesta JSON."**
5. **"¿Qué diferencia hay entre REST, SOAP y GraphQL? ¿Cuál usarías y por qué?"**

> 💡 **Cómo encararlas:** la 1 y la 4 son las "preguntas reina". Para la 1, recorre el punto 1: la API key es el carnet de identidad, va en la query (`appid`) o en `Authorization: Bearer`, y nunca en el código. Para la 4, escribe el esqueleto del punto 3: `params` (con `units=metric`), comprobar

> `status_code`, `resp.json()` y acceder a `main` y `weather`. Si además mencionas el `.env` y el `raise_for_status`, la entrevista es tuya.

🤷 No hay preguntas tontas

> ❓ **¿Me pueden banear por usar mucho la API?**

Sí. Todas tienen límites (rate limits). Lee la documentación. OpenWeatherMap gratis: 60 peticiones/minuto.

> ❓ **¿Qué pasa si me paso del límite?**

HTTP 429 (Too Many Requests). Algunas APIs bloquean tu IP por un tiempo.

> ❓ **¿OpenAI es caro?**

gpt-4o-mini cuesta ~$0,15 por 1M tokens de entrada y ~$0,60 por 1M de salida. Una pregunta normal son ~100 tokens = centésimas de céntimo. Muy barato para pruebas.

> ❓ **¿Puedo guardar la API key en el código para pruebas?**

Solo si el código nunca va a GitHub. Mejor acostúmbrate a `.env` desde el principio.

🎬 Poscréditos

> *Tu petición parte con su API key cifrada por TLS y vuelve con el JSON del tiempo bajo el brazo.*
>
> *El 429 llegó, esperaste con backoff, y la petición siguiente salió bien.*
>
> *Y las claves se quedaron en el `.env`, lejos de GitHub, donde deben vivir.*

PRÓXIMAMENTE EN UD 9: *Seguridad y cifrado. Las contraseñas que no se pueden leer: hash, AES, RSA y las firmas digitales.*

✅ Criterios de evaluación cubiertos (RA4a-b)

RA4: Desarrolla aplicaciones que se comunican por red. Servicios en red.

CE	Criterio	Cubierto
a)	Utiliza APIs REST para obtener datos externos	✅ Puntos 3-5 y 8 + ⚡ Laboratorio de tortura
b)	Gestiona peticiones HTTP y procesa respuestas JSON	✅ Puntos 1, 5-6 y 8 + ⚡ Laboratorio de tortura

> RA4c (Servidores concurrentes) y RA4d (ThreadPool) se cubren en la **UD 6 · Servidores concurrentes**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en la **UD 10 · Alta disponibilidad**.

📚 Volver al índice de la unidad · **Anterior:** 08 · Práctica APIs comerciales · **Siguiente:** UD 9 · Seguridad y cifrado

UD 9 — Seguridad y cifrado con Python

Hash, cifrado simétrico y asimétrico, firmas digitales 🔐

Cada vez que escribes una contraseña en un formulario, un servidor decide si es la correcta sin saber cuál es. ¿Cómo es posible? Gracias al **hash**: una función que convierte cualquier texto en una huella digital imposible de revertir. Pero la seguridad no termina ahí: también necesitas **cifrar** datos para que solo el destinatario pueda leerlos, **firmar** documentos para demostrar que son tuyos y proteger sistemas con principios como el mínimo privilegio.

Esta unidad fusiona hash y cifrado en un solo bloque: empezarás con las funciones hash (MD5, SHA-256) y las contraseñas seguras, pasarás al cifrado clásico (César) y moderno (AES, RSA), verás las firmas digitales y el cifrado híbrido, y cerrarás con un sistema seguro completo.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es un **hash** y sus propiedades (determinismo, unidireccionalidad, longitud fija).
- Comparar MD5, SHA-1, SHA-256 y SHA-512 con criterio.
- Diseñar un sistema de registro/login que almacene solo hashes.
- Explicar la **sal** y cómo neutraliza las tablas rainbow.
- Cifrar y descifrar con el **cifrado César**.
- Distinguir cuándo usar hash (integridad) y cuándo cifrado (confidencialidad).
- Cifrar y descifrar con **AES** (simétrico) usando `cryptography`.
- Cifrar y descifrar con **RSA** (asimétrico) usando `cryptography`.
- Crear y verificar **firmas digitales**.
- Implementar **cifrado híbrido** (AES + RSA).
- Aplicar principios de seguridad: Zero Trust, mínimo privilegio, defensa en profundidad.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Principios de seguridad	Zero Trust, mínimo privilegio, defensa en profundidad	Todos
02 · Hash y huellas digitales	`hashlib`, MD5, SHA-1, SHA-256, SHA-512	Todos
03 · Contraseñas seguras	Hash de contraseñas, sal, registro/login	Todos
04 · Cifrado clásico	César, fuerza bruta, hash vs cifrado	Todos
05 · Cifrado simétrico (AES)	AES con `cryptography`, modos de operación	Todos
06 · Cifrado asimétrico (RSA)	RSA, claves pública/privada	Todos
07 · Firmas digitales	Firmar y verificar con RSA	Todos
08 · Cifrado híbrido y práctica	AES+RSA, RBAC, sistema seguro completo	Todos
09 · Cierre	Sé el cifrado, Fireside, Laboratorio de tortura…	Todos

📝 Boletines de la unidad

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen integridad y confidencialidad.

CE	Criterio	Dónde se cubre
RA5a	Principios básicos de seguridad	✅ Punto 1
RA5b	Tipos de cifrado (simétrico/asimétrico)	✅ Puntos 4-6
RA5c	Implementa funciones hash (MD5, SHA)	✅ Puntos 2-3 + ⚡ Laboratorio (punto 9)

CE	Criterio	Dónde se cubre
RA5d	Implementa cifrado AES	✅ Punto 5 + ⚡ Laboratorio (punto 9)
RA5e	Implementa cifrado RSA	✅ Punto 6 + ⚡ Laboratorio (punto 9)
RA5f	Crea y verifica firmas digitales	✅ Punto 7
RA5g	Implementa cifrado híbrido	✅ Punto 8
RA5h	Conoce sistemas de roles y RBAC	✅ Punto 8

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 8 y dominas APIs? Empieza por el punto 1, que sienta los cimientos.

¿Ya sabes qué es un hash? Saltar a los puntos 4, 5 o 6.

📍 **Primer punto:** 01 · Principios de seguridad ⏭️ **Al acabar la unidad, continúa en** UD 10 · Alta disponibilidad.

01 — Principios de seguridad

La base de todo: cero confianza y defensa en profundidad 🛡️

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 01 · Principios de seguridad

📬 La idea en una frase

> Antes de cifrar una sola letra, tienes que interiorizar los **principios de seguridad**: si no entiendes por qué se hace algo, cualquier herramienta que aprendas después será un martillo mal usado.

Los algoritmos cambian (MD5 cayó, SHA-1 cayó, algún día caerá SHA-256), pero los principios son eternos. Son las reglas del oficio que distinguen a quien "copia código" de quien diseña sistemas seguros.

🏰 Los seis principios que no puedes olvidar

Principio	Traducción al español de la calle
Zero Trust	No te fíes ni de tu sombra. Verifica siempre.
Mínimo privilegio	Da solo los permisos mínimos necesarios.
Defensa en profundidad	No pongas toda la seguridad en una capa.
Cifra todo	En tránsito (TLS) y en reposo (disco).
Rotación de claves	Cambia las claves periódicamente.
No inventes tu cripto	Las bibliotecas existentes ya han sido auditadas.

Desglose de cada uno:

- **Zero Trust** — La red interna no es de fiar por defecto. Un usuario autenticado hoy puede ser un atacante mañana. Por eso cada operación sensible se vuelve a verificar. En esta unidad lo ves con los hashes: ni siquiera confiamos en nuestra propia base de datos, por eso no guardamos contraseñas en claro.

- **Mínimo privilegio** — Nadie debería tener más acceso del que necesita para su trabajo. Un becario no necesita borrar la tabla de usuarios. Esto conecta directamente con **RBAC** (Control de Acceso Basado en Roles): a cada rol le toca un paquete de permisos, y nadie se sale del suyo. Lo verás a fondo en el punto 8, pero el principio lo aplicas ya.
- **Defensa en profundidad** — Si solo tienes una cerradura, alguien con la llave abre la casa entera. La seguridad real apila capas: contraseña con hash + sal, cifrado en disco, firewall, logs. Si una capa cae, las demás siguen.
- **Cifra todo** — Los datos viajan por la red (en tránsito, con TLS) y descansan en el disco (en reposo). En esta unidad ciframos en reposo las contraseñas con hash; en los puntos 5 y 6 cifrarás mensajes completos con AES y RSA.
- **Rotación de claves** — Una clave que lleva 5 años en uso es una clave comprometida. Igual que cambias la cerradura cuando se pierde una copia, las claves y contraseñas se renuevan periódicamente.
- **No inventes tu cripto** — La tentación de escribir "mi propio algoritmo" es enorme… y siempre termina mal. Las bibliotecas como `hashlib` o `cryptography` llevan décadas siendo atacadas y parcheadas. Tú no puedes hacer eso en un finde.

🔑 La analogía de las cerraduras de una casa

Imagina tu casa como un sistema informático. Las **cerraduras** son los mecanismos de seguridad: la puerta principal (contraseña), la alarma (detección de intrusos), la caja fuerte (cifrado de datos sensibles).

- Poner una sola cerradura en la puerta y nada más es **seguridad de una capa**: si alguien copia la llave, entra y listo.
- Dejar la llave bajo el felpudo es el equivalente a guardar contraseñas en **texto plano**: la seguridad existe, pero no sirve de nada.
- Tener cerradura **y** alarma **y** caja fuerte es **defensa en profundidad**: aunque salten la primera, la segunda los delata y la tercera los frena.
- Y lo más importante: **ningún cerrajero serio diseña su propia cerradura desde cero**; compra una certificada. Eso es *no inventes tu cripto*.

> Cuando en el punto 3 guardes una contraseña con hash, estarás instalando una cerradura decente. Cuando añadas **sal**, estarás poniendo la segunda vuelta.

📖 La cita que debes saber

> “La seguridad no es un producto, es un proceso.” — Bruce Schneier

No es un interruptor que enciendes y olvidas: es una disciplina continua de revisar, parchear, rotar y auditar. Por eso existen los principios: para que el proceso nunca se detenga.

🧠 Mini-chequeo

1. ¿Por qué “no inventes tu cripto” es un principio y no una sugerencia?
2. ¿Qué relación hay entre “mínimo privilegio” y RBAC?
3. ¿Por qué el cifrado se aplica “en tránsito” y “en reposo”?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- La seguridad real se construye sobre **principios** (Zero Trust, mínimo privilegio, defensa en profundidad), no sobre herramientas sueltas.
- **No inventes tu cripto**: usa bibliotecas auditadas, cifra todo y rota las claves.
- Estos principios son el filtro con el que evaluarás cada decisión del resto de la unidad (y de tu vida como programador).

🐛 Vocabulario rápido

Término	Idea general
Zero Trust	No confiar en nadie por defecto; verificar siempre
Mínimo privilegio	Cada usuario solo con los permisos que necesita
Defensa en profundidad	Varias capas de seguridad independientes
RBAC	Control de acceso por roles (a cada rol, sus permisos)
Rotación de claves	Renovar claves/contraseñas periódicamente

Término	Idea general
Cifrado en tránsito / en reposo	Proteger datos mientras viajan / mientras se guardan

📚 Volver al índice de la unidad · **Siguiente:** 02 · Hash y huellas digitales

02 — Hash y huellas digitales

La huella digital de cualquier dato, MD5, SHA-1, SHA-256 y SHA-512 🔍

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 02 · Hash y huellas digitales

📬 La idea en una frase

> Un **hash** es una función que convierte cualquier entrada (un texto, un archivo, una contraseña) en una cadena de **longitud fija**, y lo hace de forma **unidireccional**: no se puede revertir. Pero no todos los hash son iguales: **MD5 y SHA-1 están rotos** y solo sirven para checksums no críticos; **SHA-256 y SHA-512 son los seguros**.

👣 La huella digital

Un hash toma un mensaje de cualquier tamaño y devuelve un resumen de tamaño fijo:

```
"a"                                                                 →
ca978112ca1bbdcafac231b39a23dc4da786eff8147c4e72b9807785afee48bb   (SHA-256)
"Hola     mundo"                                                                →
ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c   (SHA-256)
"En           un           lugar           de           la            Mancha"           →
e6ff77084bec551e5c5ae84a1b44d4eed6dd9cabf86bea2abbfd510230bb3ec7   (SHA-256)
```

Fíjate en la clave: da igual que la entrada tenga 1 carácter o 1 millón, el hash **siempre tiene la misma longitud** (64 caracteres en SHA-256). Eso lo convierte en una herramienta ideal para comprobar que “esto que me han enviado es exactamente lo que esperaba”.

🐍 Hash en Python: tres sabores

Con el módulo `hashlib` (de la librería estándar, sin instalar nada) puedes calcular hashes de cualquier texto:

```python
import hashlib

texto = b"Hola mundo"

print("MD5:", hashlib.md5(texto).hexdigest())
print("SHA1:", hashlib.sha1(texto).hexdigest())
print("SHA256:", hashlib.sha256(texto).hexdigest())
```

Salida:

```
MD5: f822102f4515609fc31927a84c6db7f8
SHA1: c083106c930790151165b95bd11860724e3836cb
SHA256: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c
```

Nota la diferencia de longitudes: MD5 devuelve **32** caracteres (128 bits), SHA1 **40** (160 bits) y SHA256 **64** (256 bits).

> 💡 El `b` antes del texto lo convierte a **bytes**, que es lo que espera `hashlib`. Si tienes un `str`, usa `.encode()`.

⚖️ La tabla comparativa

Algoritmo	Bits	¿Seguro?	Uso recomendado
MD5	128	❌ No	Solo checksums no críticos
SHA-1	160	❌ No	Solo legacy
SHA-256	256	✅ Sí	Todo uso general
SHA-512	512	✅ Sí	Cuando necesites más seguridad

El porqué de cada fila:

- **MD5 (1992)** — Durante décadas fue el rey, hasta que en 2004 se demostraron **colisiones prácticas**: es posible fabricar dos archivos distintos con el mismo MD5. Roto.
- **SHA-1 (1995)** — Heredó el trono… y en 2017 el equipo de Google logró la primera **colisión real** (dos PDFs distintos, mismo SHA-1). Roto también.

- **SHA-256 / SHA-512 (familia SHA-2, 2001)** — A día de hoy siguen en pie: no se conoce forma práctica de forzarlos. Son los que ves en las descargas de software, los certificados TLS y las cadenas de bloques.

> ⚠️ **Nunca uses MD5 o SHA-1 para seguridad**. Son vulnerables a colisiones. Usa SHA-256 o superior. La única excepción es un checksum casual para detectar corrupción accidental (no ataques).

📄 Hash de un archivo: verificar integridad

La aplicación más cotidiana del hash es comprobar que un archivo no se ha corrompido ni manipulado:

```
import hashlib

# Hash de un archivo (verificar integridad)
with open("archivo.pdf", "rb") as f:
    hash_archivo = hashlib.sha256(f.read()).hexdigest()
    print(f"SHA256 del archivo: {hash_archivo}")
```

Dos detalles importantes:

- Se abre en modo binario `"rb"` porque queremos el hash de los bytes exactos, sin conversión de texto.
- **Un solo bit cambiado en el archivo → un hash completamente distinto** (efecto avalancha). Por eso un checksum te dice si el archivo es *exactamente* el que publicó el autor.

SHA-512 en acción

Si necesitas más margen, el código es idéntico cambiando el nombre:

```
import hashlib

texto = b"Hola mundo"

print("SHA256:", hashlib.sha256(texto).hexdigest())
print("SHA512:", hashlib.sha512(texto).hexdigest())
```

Salida:

```
SHA256: ca8f60b2cc7f05837d98b208b57fb6481553fc5f1219d59618fd025002a66f5c
SHA512: 34ddb0edac59e441459e07cf33bd628f53fbbf752141125f069f32081b169f93...
```

📐 Las cinco propiedades del hash

- **Determinista**: misma entrada → mismo hash. Siempre.
- **Unidireccional**: no se puede obtener el original a partir del hash.
- **Longitud fija**: MD5 = 128 bits, SHA1 = 160 bits, SHA256 = 256 bits, pase lo que pase con la entrada.
- **Efecto avalancha**: un cambio mínimo (una letra, un bit) cambia el hash **por completo**.
- **Colisiones**: dos entradas diferentes con el mismo hash. En teoría posibles, en la práctica **casi imposibles** en SHA-256.

🧠 Mini-chequeo

1. ¿Qué longitud tiene un hash SHA-256 expresado en hexadecimal? ¿Y en bits?
2. ¿Por qué MD5 y SHA-1 no sirven para seguridad?
3. ¿Por qué “unidireccional” es la propiedad más importante para guardar contraseñas?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Un hash convierte cualquier entrada en una huella de **longitud fija**, siempre la misma para la misma entrada.
- MD5 y SHA-1 tienen colisiones demostradas: **rotos para seguridad**; SHA-256/SHA-512 son hoy los fiables.
- `hashlib.md5/sha1/sha256(texto).hexdigest()` te da la huella en una línea.

🐛 Vocabulario rápido

Término	Idea general
Hash	Función que resume cualquier dato en una cadena fija
Unidireccional	Imposible de revertir
Efecto avalancha	Un bit de cambio altera todo el hash
Colisión	Dos entradas con el mismo hash
Checksum	Hash de un archivo para verificar integridad
SHA-2	Familia segura: SHA-256 y SHA-512

📚 Volver al índice de la unidad · **Anterior:** 01 · Principios de seguridad · **Siguiente:** 03 · Contraseñas seguras

03 — Contraseñas seguras

Guarda la huella, nunca la original, con sal para confundir a los robots 🔐

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 03 · Contraseñas seguras

📬 La idea en una frase

> Cuando un usuario se registra, no guardamos su contraseña: guardamos su **hash** con **sal**. Así, si roban la base de datos, no roban contraseñas. Ni siquiera las tablas rainbow sirven.

🎭 Be the code: registro y login con hash

> “Sé el programa que registra a un usuario y luego verifica su login. Contraseñas seguras, sin almacenar la original.”

```
import hashlib

def registrar(usuario, contraseña):
    hash_contra = hashlib.sha256(contraseña.encode()).hexdigest()
    print(f"  Usuario '{usuario}' registrado")
    print(f"  Hash: {hash_contra[:16]}...")
    return hash_contra

def login(usuario, contraseña, hash_almacenado):
    hash_intento = hashlib.sha256(contraseña.encode()).hexdigest()
    if hash_intento == hash_almacenado:
        print(f"  ✅ {usuario}: login correcto")
        return True
    else:
        print(f"  ❌ {usuario}: contraseña incorrecta")
        return False

# Simular registro y login
hash_ana = registrar("Ana", "MiClaveSecreta123")
login("Ana", "MiClaveSecreta123", hash_ana)  # ✅
login("Ana", "OtraClave", hash_ana)          # ❌
```

Salida:

```
Usuario 'Ana' registrado
Hash: 0f2adb0aee3de894...
✅ Ana: login correcto
❌ Ana: contraseña incorrecta
```

La máquina nunca ve la contraseña: ve su hash. El registro calcula el hash y lo devuelve. El login vuelve a calcular el hash del intento y lo compara con el almacenado.

🗄️ La regla de oro

> La contraseña original **nunca** se almacena. Solo su hash.

- Si roban la base de datos con hashes, no tienen contraseñas: solo huellas irreversibles.
- Los sistemas serios además añaden **sal** (abajo) para que ni siquiera las huellas idénticas se reconozcan entre usuarios.
- Cifrar la contraseña en lugar de hashaarla **no** sirve para esto: si el atacante roba también la clave de cifrado, recupera todo. El hash es la elección correcta por ser unidireccional.

> ⚠️ **Y lo que NUNCA debe aparecer en tu código:** `if contraseña == contraseña_guardada:` comparando texto plano, ni un `print(f"Contraseña de Ana: {contraseña}")` en los logs.

🫘 El problema: hashes idénticos

```python
import hashlib
print(hashlib.sha256(b"clave123").hexdigest())
print(hashlib.sha256(b"clave123").hexdigest())
```

Ambas líneas imprimen **exactamente el mismo hash**. Si Ana y Bob usan “clave123”, su base de datos tendrá dos filas con el mismo hash. Un atacante lo ve al instante: *“estos dos comparten contraseña”*. Peor aún: un atacante con una **tabla rainbow** (hash de millones de contraseñas comunes precalculadas) buscaría el hash y encontraría “clave123” en segundos.

🧂 La solución: salt

La **sal** es un valor aleatorio (16 bytes, generado con `os.urandom`) que se concatena a la contraseña **antes** de hashear. Como cada usuario tiene su propia sal, aunque dos compartan contraseña, sus hashes serán distintos.

```
import hashlib, os

def registrar_con_salt(usuario, contraseña):
    salt = os.urandom(16)                          # 🧂 Salt: 16 bytes NUEVOS cada vez
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    almacenado = salt.hex() + hash_contra           # Guardamos: salt + hash juntos
    print(f"  Usuario '{usuario}' registrado")
    print(f"  Almacenado (salt+hash): {almacenado[:32]}...")
    return almacenado

def login_con_salt(usuario, contraseña, almacenado):
    salt = bytes.fromhex(almacenado[:32])           # 🧂 RECUPERAMOS el mismo salt
    hash_original = almacenado[32:]                  # Hash que se guardó al registrar
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    if hash_intento == hash_original:
        print(f"  ✅ {usuario}: login correcto")
        return True
    else:
        print(f"  ❌ {usuario}: contraseña incorrecta")
        return False

# Simular — dos usuarios CON LA MISMA contraseña
hash_ana = registrar_con_salt("Ana", "clave123")   # os.urandom genera salt_A
hash_bob = registrar_con_salt("Bob", "clave123")   # os.urandom genera salt_B (distinto)

print(f"\n¿Son iguales los hashes? {hash_ana == hash_bob}")  # ¡NO!

login_con_salt("Ana", "clave123", hash_ana)    # ✅
login_con_salt("Ana", "otra", hash_ana)        # ❌
```

La clave del proceso: la sal se genera **una vez**, al registrar, y se guarda junto al hash. En el login **se recupera la misma sal** del string almacenado para poder recalcular el hash exacto.

🌈 Por qué la sal destroza las tablas rainbow

Una **tabla rainbow** es un diccionario precomputado que mapea contraseñas comunes a sus hashes. Con sal, el hash guardado ya no es el de “clave123”: es el de “salt_A + clave123”, que es único e impredecible. La tabla rainbow deja de servir.

Sin sal	Con sal
Dos usuarios con la misma clave → mismo hash	→ hashes distintos siempre
La tabla rainbow funciona al instante	→ la tabla rainbow queda inútil
Un ataque revela contraseñas compartidas	→ nada se delata

🧠 Mini-chequeo

1. ¿Qué guardamos exactamente en la base de datos al registrar a un usuario?
2. ¿Qué problema resuelve la sal y contra qué ataque concreto protege?
3. ¿Por qué los 32 primeros caracteres del string guardado son la sal?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- En el registro se guarda el **hash** de la contraseña con **sal**; en el login se hashea el intento con la misma sal y se comparan huellas.
- Sin sal, dos usuarios con la misma contraseña comparten hash y las tablas rainbow triunfan. Con sal, cada hash es único.
- La sal se genera al registrar, se guarda junto al hash, y en el login se recupera para recalcular la huella exacta.

🐛 Vocabulario rápido

Término	Idea general
Hash de contraseña	Huella irreversible que se guarda en lugar del texto plano
Sal (salt)	Valor aleatorio mezclado con la contraseña antes de hashear

Término	Idea general
Tabla rainbow	Diccionario precomputado de hashes de contraseñas comunes
os.urandom(16)	Genera 16 bytes aleatorios criptográficamente fuertes
bytes.fromhex()	Convierte texto hexadecimal de vuelta a bytes

📚 Volver al índice de la unidad · **Anterior:** 02 · Hash y huellas digitales · **Siguiente:** 04 · Cifrado clásico

04 — Cifrado clásico

César, fuerza bruta y la gran pregunta: ¿hash o cifrado? 🏛️

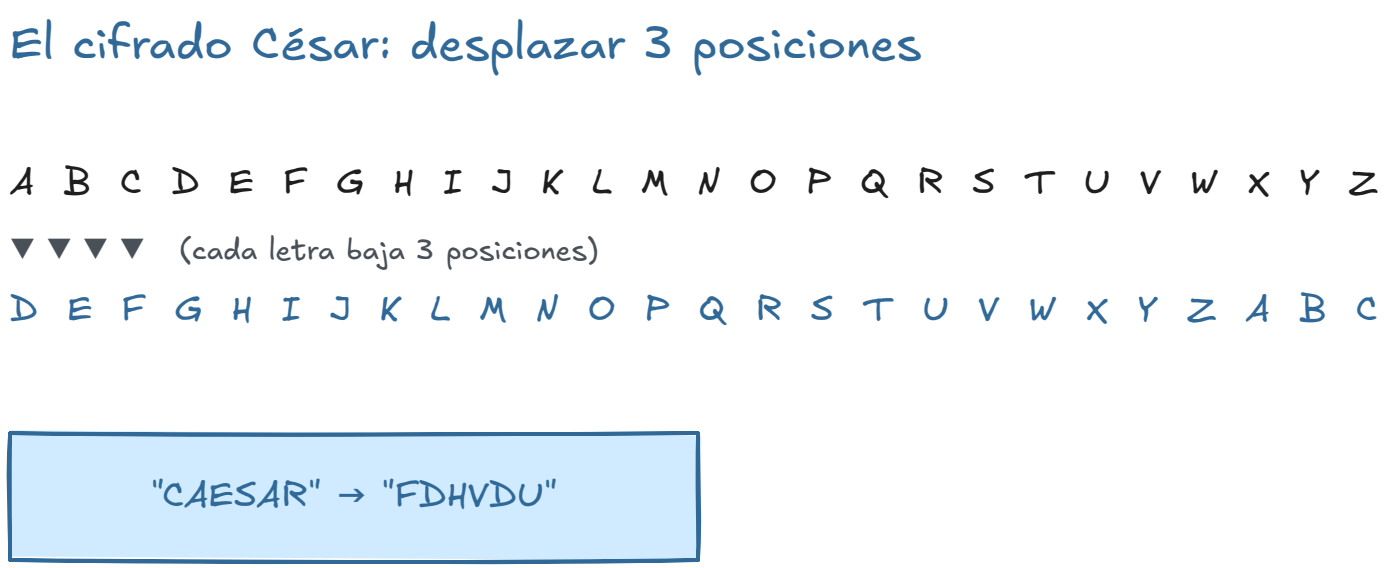
> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 04 · Cifrado clásico

📬 La idea en una frase

> El **cifrado César** desplaza cada letra un número fijo de posiciones: es el cifrado más antiguo que se conoce y, también, el más fácil de romper. Mientras el hash no se puede deshacer, el César **sí** se descifra con una clave.

🏛️ La historia: Julio César y su desplazamiento de 3

Julio César, para que sus mensajes militares no fueran leídos si caían en manos enemigas, desplazaba **cada letra 3 posiciones** en el alfabeto.



“Hola Mundo” con desplazamiento 3 → **“Krod Pxqgr”**.

Cifrar y descifrar en Python

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

def descifrar_cesar(texto, desplazamiento):
    return cifrar_cesar(texto, -desplazamiento)

original = "Hola Mundo"
cifrado = cifrar_cesar(original, 3)
descifrado = descifrar_cesar(cifrado, 3)

print(f"Original:  {original}")     # Hola Mundo
print(f"Cifrado:   {cifrado}")      # Krod Pxqgr
print(f"Descifrado:{descifrado}")   # Hola Mundo
```

Dos detalles:

- **`caracter.isupper()`** decide si la base es `ord('A')` o `ord('a')`, para que las mayúsculas sigan siendo mayúsculas.
- **Descifrar es cifrar con desplazamiento negativo**: no hace falta otra función.

Romperlo por fuerza bruta

El desplazamiento solo puede ser un número del **1 al 25**. Prueba los 25 y mira cuál tiene sentido:

```
cifrado = "Krod Pxqgr"
for desplazamiento in range(1, 26):
    intento = cifrar_cesar(cifrado, -desplazamiento)
    print(f"Desplazamiento {desplazamiento}: {intento}")
```

En 3 intentos lo encontramos. **Por eso el César no sirve para nada serio**: solo 25 claves posibles se prueban en segundos.

🥊 Hash vs Cifrado: el ring de los conceptos

> *Dos mecanismos de seguridad se citan para resolver de una vez quién hace qué.*

Hash: — Yo soy la huella digital. Transformo cualquier texto en una cadena fija. No se puede deshacer. Unidireccional. Para siempre.

Cifrado: — Vaya, qué drástico. Yo puedo cifrar y descifrar. Tengo clave. Si tú pierdes un hash, no hay vuelta atrás. Yo puedo recuperar el mensaje original.

Hash: — ¡Esa es precisamente mi gracia! Para contraseñas no quieres que se pueda deshacer.

Cifrado: — Pero para enviar un mensaje secreto, el hash no sirve. Necesitas que el destinatario pueda leerlo. Ahí entro yo.

Hash: — Y para verificar integridad, nadie me gana. Si cambia un bit, el hash cambia por completo.

Cifrado: — Al final, cada uno a lo suyo. Tú para integridad y contraseñas; yo para confidencialidad.

⚖️ La moraleja: cada uno a lo suyo

	Hash	Cifrado
¿Qué hace?	Resume en una huella de longitud fija	Transforma con una clave
¿Reversible?	❌ No, unidireccional	✅ Sí, con la clave
Protege	Integridad (y contraseñas)	Confidencialidad
Ejemplo típico	Checksum de descarga, login	Enviar un mensaje secreto

> **Moraleja**: El hash verifica integridad (no se puede deshacer). El cifrado protege confidencialidad (se puede deshacer con la clave). Ambos son necesarios.

🎬 La escena que lo aclara todo

Imagina que envías un mensaje secreto por una red hostil:

- **Solo cifrado:** el mensaje llega cifrado, pero ¿y si un atacante lo modifica por el camino? La confidencialidad estaba protegida; la **integridad, no**.
- **Solo hash:** puedes comprobar que el mensaje no cambió, pero cualquier persona que lo intercepte **lo lee tal cual**. La integridad estaba protegida; la **confidencialidad, no**.
- **Cifrado + hash:** cifras el mensaje (nadie lo lee) y además calculas su hash (el destinatario comprueba que nadie lo tocó). Así lo hacen de verdad los sistemas serios.

🧠 Mini-chequeo

1. ¿Cómo se descifra un mensaje César si conoces el desplazamiento?
2. Si quieres que nadie pueda leer un mensaje, ¿qué necesitas: hash o cifrado?
3. ¿Por qué un hash sirve para guardar contraseñas y el cifrado no es la mejor opción?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El César desplaza cada letra un número fijo; descifrar es desplazar hacia atrás. Con solo 25 claves posibles, se rompe al instante.
- El hash es **unidireccional** (integridad, contraseñas); el cifrado es **reversible con clave** (confidencialidad).
- No se excluyen: en los sistemas reales se combinan (cifrar para esconder, hashear para verificar).

🐛 Vocabulario rápido

Término	Idea general
Cifrado César	Desplazar cada letra un número fijo de posiciones
Integridad	Que un dato no ha sido modificado (hash)

Término	Idea general
Confidencialidad	Que solo quien corresponde lo lea (cifrado)
Unidireccional	Imposible de revertir, propia del hash
Reversible con clave	Se deshace con la clave correcta, propia del cifrado

📚 Volver al índice de la unidad · **Anterior:** 03 · Contraseñas seguras · **Siguiente:** 05 · Cifrado simétrico (AES)

05 — Cifrado simétrico (AES)

Una clave o un par, AES con cryptography, modos de operación 🔑

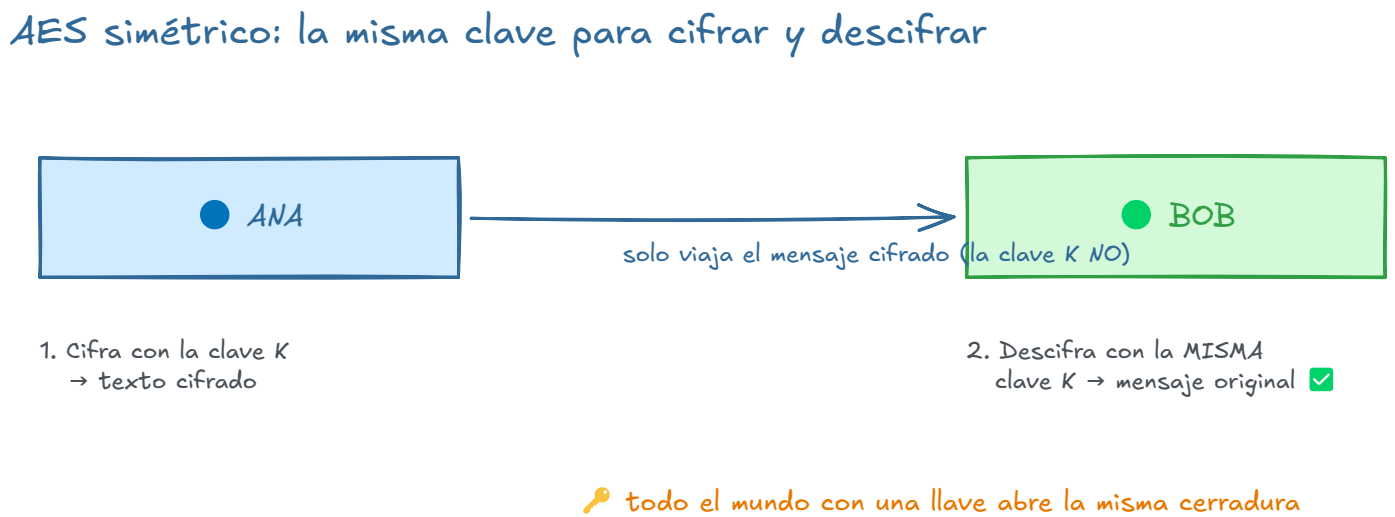
> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 05 · Cifrado simétrico (AES)

📬 La idea en una frase

> El cifrado **simétrico** usa **una sola clave** para cifrar y descifrar; el **asimétrico** usa un **par de claves** (pública + privada). **AES** es el simétrico moderno por excelencia: rápido, auditado y con modos para cada necesidad.

🔑 Cifrado simétrico: una sola clave

Todo el mundo con una llave abre la misma cerradura. El **mismo secreto** sirve para cifrar y para descifrar.



- ✅ **Rapidez:** cifra cantidades enormes en milisegundos.
- ❌ **El problema:** ¿cómo le haces llegar la clave K a Bob sin que nadie la intercepte? Ese es el *problema de la distribución de claves*.

🔑 Cifrado asimétrico: el par público/privado

Dos claves: una **pública** (todos pueden verla) y una **privada** (solo tú).

La regla de oro de las claves:

Operación	¿Qué clave usas?
Cifrar **para** alguien	La clave **pública** de esa persona
Descifrar	Tu clave **privada**
Firmar	Tu clave **privada**
Verificar una firma	La clave **pública** del firmante

⚖️ AES vs RSA

Característica	AES (simétrico)	RSA (asimétrico)
Claves	Una	Dos (pública + privada)
Velocidad	Rápido (~1GB/s)	Lento (~1MB/s)
Tamaño máximo	Ilimitado	~190 bytes (con 2048 bits)
Distribución de clave	Problema	Fácil

> ⚠️ RSA no sirve para cifrar mensajes grandes. Para eso necesitas **cifrado híbrido** (punto 8).

🧩 Componentes de AES

Cuando cifras con AES en modo GCM, lo que viaja por la red es **más que el texto cifrado**:

Componente	Descripción	¿Se envía?
Clave	16, 24 o 32 bytes (AES-128/192/256)	❌ Secreto
Nonce	Número aleatorio único	✅ Se envía con el cifrado
Tag	Código de autenticación (integridad)	✅ Se envía

Componente	Descripción	¿Se envía?
Texto cifrado	El mensaje cifrado	✅ Se envía

AES con `cryptography`

Instala la biblioteca (la más usada en Python para criptografía moderna):

```
pip install cryptography
```

Cifrar y descifrar un mensaje:

```python
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os

# Generar clave (32 bytes = 256 bits)
clave = AESGCM.generate_key(bit_length=256)

# Cifrar
nonce = os.urandom(12)  # 12 bytes para GCM
aesgcm = AESGCM(clave)
mensaje = b"Mensaje secreto"
texto_cifrado = aesgcm.encrypt(nonce, mensaje, None)

print(f"Original: {mensaje}")
print(f"Cifrado (hex): {texto_cifrado.hex()}")
print(f"Nonce: {nonce.hex()}")

# Descifrar
aesgcm2 = AESGCM(clave)
texto_descifrado = aesgcm2.decrypt(nonce, texto_cifrado, None)
print(f"Descifrado: {texto_descifrado}")
```

```
Original: b'Mensaje secreto'
Cifrado (hex): 7f2e9b1a4c6d8e0f...
Nonce: 4f3a2b1c9d8e7f6a...
Descifrado: b'Mensaje secreto'
```

> AES es **simétrico**: misma clave para cifrar y descifrar. El problema es compartir esa clave de forma segura (RSA lo resuelve en el punto 6 y el punto 8).

Si alguien manipula el cifrado, `decrypt` lanza una excepción. Es la integridad dentro del cifrado.

⛓️ Modos de operación: ECB, CBC, GCM

AES cifra de 16 en 16 bytes. El **modo** decide cómo se encadenan esos bloques:

Modo	¿Autentica?	¿IV/nonce?	¿Padding?	Veredicto
ECB	❌	❌	Sí	⚠️ Evitar: muestra patrones
CBC	❌	✅ IV	Sí	✅ Clásico y seguro
GCM	✅ (tag)	✅ Nonce	No	✅ Moderno, cifra y autentica

> ⚠️ **ECB es la trampa clásica.** Cifra cada bloque de forma independiente: mismo bloque de entrada → mismo bloque de salida. En imágenes o documentos con zonas repetidas, el patrón se ve a simple vista.

GCM es el recomendado: genera un **tag** que detecta cualquier manipulación. Con `cryptography`, se usa `AESGCM` que implementa GCM directamente.

🧠 Mini-chequeo

1. ¿Cuántos bytes de clave usa AES-256? ¿Y AES-128?
2. ¿Qué pasa si el receptor descifra sin el nonce correcto?
3. ¿Por qué no debes usar ECB con datos largos o repetitivos?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- El **simétrico** (AES) usa una clave, es rapidísimo y sirve para el volumen; el **asimétrico** (RSA) usa un par de claves y resuelve el reparto.
- Con `cryptography`, AES usa `AESGCM` con `encrypt`/`decrypt` y un nonce de 12 bytes.

- Para esta unidad, **GCM** te da cifrado + integridad en una sola operación.

🐛 Vocabulario rápido

Término	Idea general
AES	Advanced Encryption Standard: cifrado simétrico estándar
Nonce	Número aleatorio único que viaja con el cifrado
Tag	Código de autenticación que detecta manipulaciones
Modo GCM	Modo autenticado: cifra y genera tag de integridad
ECB	Modo inseguro: cada bloque se cifra solo (patrones visibles)

📚 Volver al índice de la unidad · **Anterior:** 04 · Cifrado clásico · **Siguiente:** 06 · Cifrado asimétrico (RSA)

06 — Cifrado asimétrico (RSA)

Generar el par de claves y cifrar con la pública 🔑

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 06 · Cifrado asimétrico (RSA)

📬 La idea en una frase

> **RSA** es el cifrado asimétrico por excelencia: generas un **par de claves** (pública + privada), cifras con la **pública** del destinatario y solo su **privada** puede descifrar. Así se resuelve el problema de compartir la clave del cifrado simétrico.

🔑 Generar el par de claves

Con `cryptography`, generar 2048 bits es sencillo:

```python
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Generar par de claves
clave_privada = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Exportar en formato PEM
privada_pem = clave_privada.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)
publica_pem = clave_privada.public_key().public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)

print("Pública (PEM):")
print(publica_pem.decode())
print("Privada (PEM):")
print(privada_pem.decode())
```

```
Pública (PEM):
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...
-----END PUBLIC KEY-----
```

- La **pública** se puede compartir por cualquier sitio (es *pública*).
- La **privada** es secreto absoluto: quien la tenga puede descifrar todo lo que se cifre con tu pública y firmar en tu nombre.
- Generar claves RSA tarda **~1-2 segundos** (busca números primos enormes).

🔒 Cifrar con la pública, descifrar con la privada

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generar par de claves
clave_privada = rsa.generate_private_key(public_exponent=65537, key_size=2048)
clave_publica = clave_privada.public_key()

# Cifrar con clave PÚBLICA
mensaje = b"Mensaje secreto para Bob"
texto_cifrado = clave_publica.encrypt(
    mensaje,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Descifrar con clave PRIVADA
texto_descifrado = clave_privada.decrypt(
    texto_cifrado,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
print(f"Descifrado: {texto_descifrado}")
```

```
Descifrado: b'Mensaje secreto para Bob'
```

El flujo entre dos personas:

lo que cifra una clave solo lo descifra la otra: la pública se puede dar a todo el mundo

- **OAEP** es el esquema de relleno de RSA: añade aleatoriedad al cifrado (el mismo mensaje cifrado dos veces da resultados distintos).
- Nadie más puede descifrar: ni quien intercepte el tráfico, ni quien tenga la pública. Solo la **privada de Bob**.

⚠️ El límite de RSA

RSA no cifra cualquier cosa:

- Con claves de 2048 bits, el tamaño máximo de mensaje es de **~190 bytes**.
- La velocidad ronda **~1 MB/s**, frente a los ~1 GB/s de AES.

> 💡 Para mensajes largos, RSA cifra la **clave AES** (32 bytes) y AES cifra el mensaje completo. Eso es el **cifrado híbrido** del punto 8.

🧠 Mini-chequeo

1. ¿Con qué clave cifras un mensaje para que solo lo lea Bob?
2. ¿Qué pasa si se filtra la clave privada de Bob?
3. ¿Por qué no puedes cifrar un archivo de 1 GB con RSA?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- RSA genera un **par de claves**: pública (compartible) y privada (secreta).
- Cifras con `publica.encrypt(msg, padding.OAEP(...))` y descifras con `privada.decrypt(cifrado, padding.OAEP(...))`.
- Es lento y limitado a ~190 bytes: sirve para repartir secretos pequeños, no para cifrar el tráfico.

🐛 Vocabulario rápido

Término	Idea general
RSA	Cifrado asimétrico basado en la dificultad de factorizar primos
Par de claves	Pública + privadas generadas juntas
OAEP	Esquema de relleno seguro de RSA
PEM	Formato de texto en el que se exportan las claves
Clave privada	Secreto absoluto: descifra y firma

📚 Volver al índice de la unidad · **Anterior:** 05 · Cifrado simétrico (AES) · **Siguiente:** 07 · Firmas digitales

07 — Firmas digitales

Firmar y verificar: integridad + autenticidad ✍️

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 07 · Firmas digitales

📬 La idea en una frase

> Una **firma digital** demuestra que un mensaje fue creado por quien dice serlo (autenticidad) y que no fue modificado (integridad): se cifra el **hash SHA-256** del mensaje con la clave **privada** del firmante, y cualquiera puede verificarlo con su clave **pública**.

✍️ Firmar (con tu clave privada)

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

clave = rsa.generate_private_key(public_exponent=65537, key_size=2048)
mensaje = b"Este mensaje es de Ana"

firma = clave.sign(
    mensaje,
    padding.PKCS1v15(),
    hashes.SHA256()
)
print(f"Firma: {firma.hex()[:32]}...")
```

```
Firma: 1a3f5b7c9d2e4f6a8b0c1d2e3f4a5b6c...
```

Firmar es cifrar un hash con tu privada:

1. Se calcula `SHA256` del mensaje internamente.
2. Se cifra ese hash con tu **clave privada** → eso es la firma.
3. Se envía `mensaje + firma`.

✅ Verificar (con la clave pública del firmante)

```
try:
    clave.public_key().verify(
        firma,
        mensaje,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    print("✅ Firma VÁLIDA — mensaje de Ana, no alterado")
except (ValueError, TypeError):
    print("❌ Firma INVÁLIDA — manipulada o no es de Ana")
```

```
✅ Firma VÁLIDA — mensaje de Ana, no alterado
```

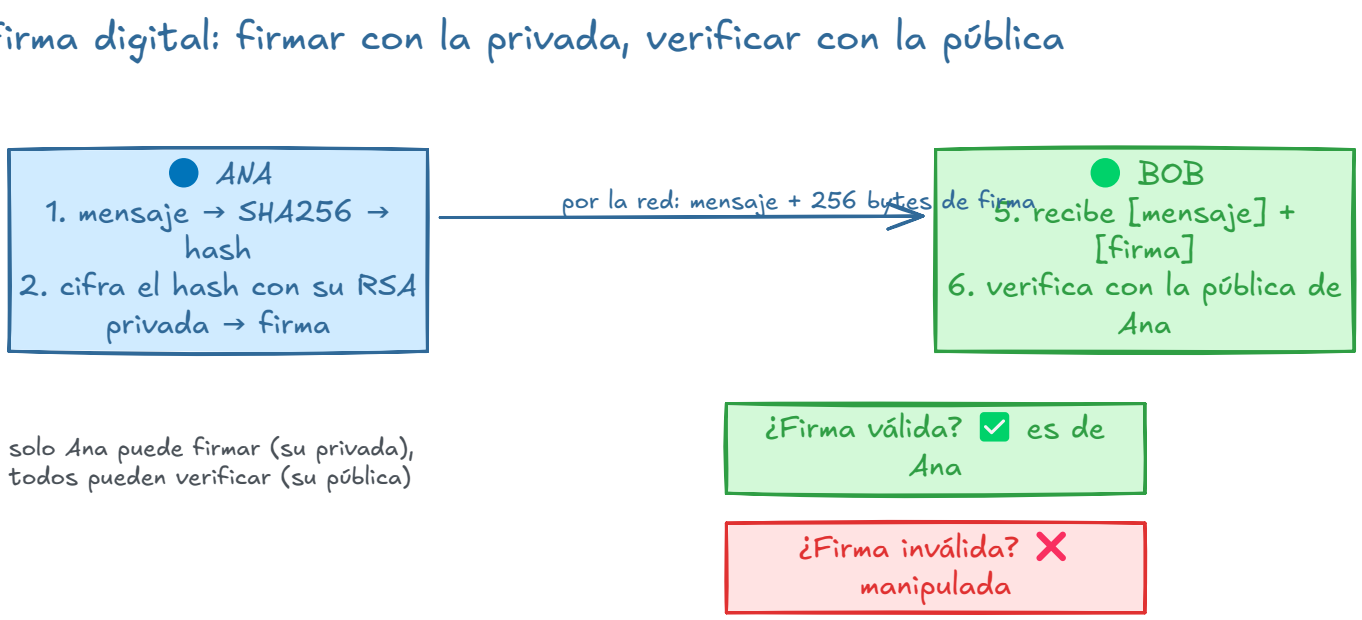
El flujo completo:

```
1. Ana escribe mensaje
2. Calcula SHA256 del mensaje
3. Cifra el hash con su clave PRIVADA → esto es la firma
4. Envía: mensaje + firma
5. Bob recibe
6. Verifica con clave PÚBLICA de Ana → hash original
7. Calcula SHA256 del mensaje recibido
8. ¿Coinciden? → ✅ Es de Ana y nadie lo modificó
```

> 🔑 **Dos garantías en una:** si los hashes no coinciden, el mensaje fue **modificado** (falla la integridad). Si la clave pública de Ana no verifica la firma, el mensaje **no es de Ana** (falla la autenticidad).

🎭 Be the code: Firma y verificación

> “Sé el proceso de firma digital desde que Ana escribe hasta que Bob verifica.”



🧠 Mini-chequeo

1. ¿Con qué clave se firma y con qué clave se verifica?
2. ¿Qué garantiza que el mensaje no fue modificado?
3. ¿Qué falla si el mensaje se altera después de firmar?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Firmar = cifrar el **SHA-256** del mensaje con tu **clave privada** (`clave.sign(msg, padding.PKCS1v15(), hashes.SHA256())`).
- Verificar = descifrar la firma con la **clave pública** del firmante y comparar hashes (`.verify()`).
- La firma aporta **autenticidad** (quién) e **integridad** (no alterado) a la vez.

🐛 Vocabulario rápido

Término	Idea general
Firma digital	Hash del mensaje cifrado con la clave privada del firmante

Término	Idea general
PKCS1v15	Esquema de firma RSA con hash
Verificar firma	Descifrar la firma con la pública y comparar hashes
Autenticidad	Garantía de quién creó el mensaje
Integridad	Garantía de que no fue modificado

📚 Volver al índice de la unidad · **Anterior:** 06 · Cifrado asimétrico (RSA) · **Siguiente:** 08 · Cifrado híbrido y práctica

08 — Cifrado híbrido y práctica

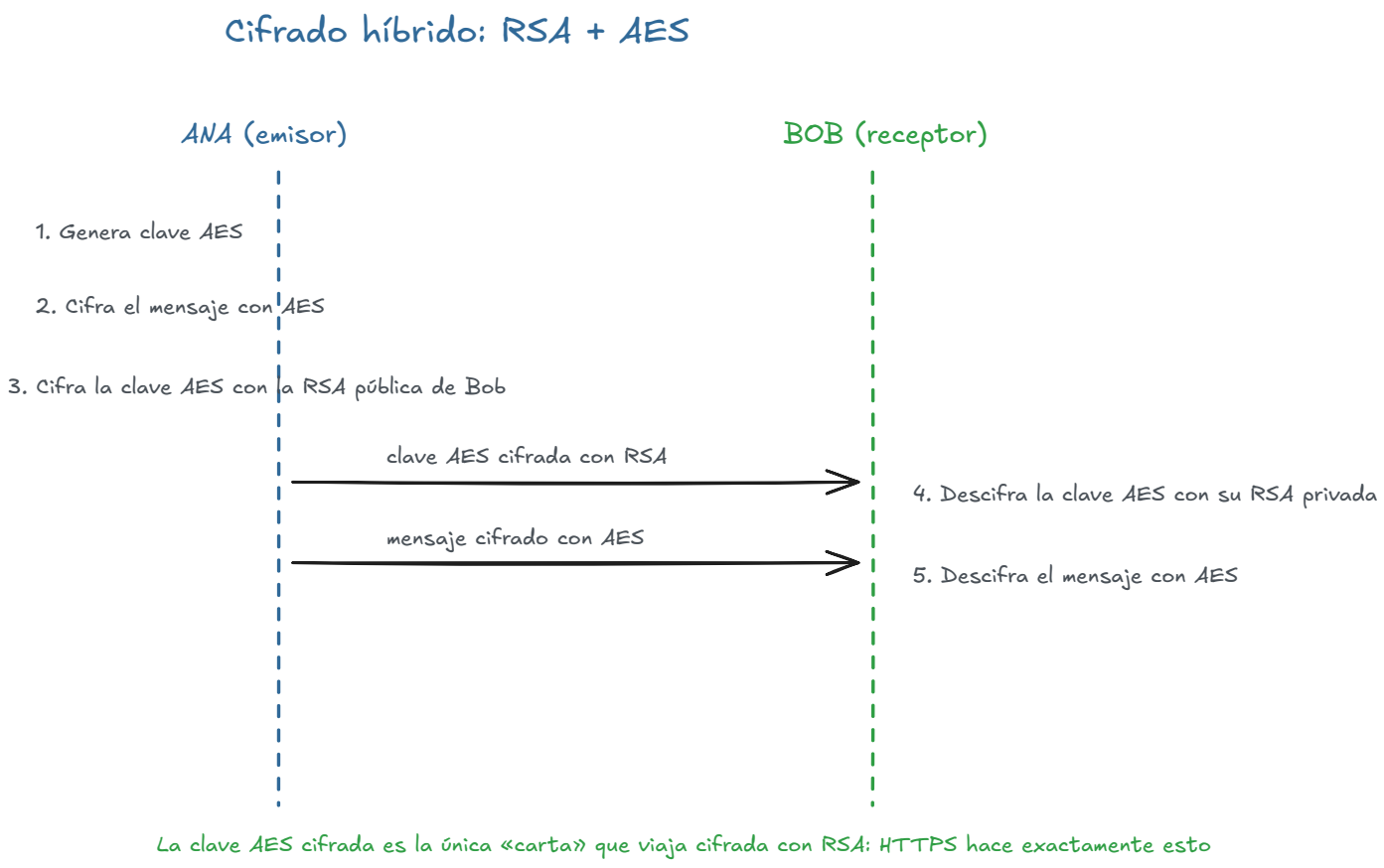
AES + RSA, RBAC y el sistema seguro completo 🏗️

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 08 · Cifrado híbrido y práctica

📬 La idea en una frase

> El **cifrado híbrido** combina lo mejor de ambos mundos: **RSA** cifra una **clave AES** aleatoria y **AES** cifra el mensaje completo. Rápido como el simétrico, con el reparto de claves resuelto por el asimétrico. Es exactamente lo que hace **HTTPS**.

🔄 El intercambio, paso a paso



🐍 Código completo

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import hashes
import os

# Claves de Bob
clave_bob_privada = rsa.generate_private_key(public_exponent=65537, key_size=2048)
clave_bob_publica = clave_bob_privada.public_key()

# Ana cifra
clave_aes = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)
mensaje = b"Hola Bob, ¿quedamos mañana?"

aesgcm = AESGCM(clave_aes)
cifrado = aesgcm.encrypt(nonce, mensaje, None)

# Cifrar clave AES con RSA pública de Bob
clave_aes_cifrada = clave_bob_publica.encrypt(
    clave_aes,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Se envía: (clave_aes_cifrada, nonce, cifrado)

# Bob descifra
clave_aes_recibida = clave_bob_privada.decrypt(
    clave_aes_cifrada,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

aesgcm2 = AESGCM(clave_aes_recibida)
mensaje_descifrado = aesgcm2.decrypt(nonce, cifrado, None)
print(f"Mensaje descifrado: {mensaje_descifrado.decode()}")
```

```
Mensaje descifrado: Hola Bob, ¿quedamos mañana?
```

Desglose de las dos mitades:

- **RSA hace el reparto:** cifra los 32 bytes de la clave AES con la pública de Bob.
- **AES hace el volumen:** cifra el mensaje (de cualquier tamaño) con esa clave.

🌐 Por qué HTTPS usa esto

Cuando entras en una web con `https://`:

1. Tu navegador genera una **clave AES** aleatoria (la clave de sesión).
2. La cifra con la **clave pública RSA** del servidor (su certificado).
3. El servidor la descifra con su **clave privada**: ahora ambos comparten la misma clave AES.
4. Todo el tráfico de la sesión se cifra con **AES**, rápido, y con **firma** (via TLS) para la integridad.

RSA solo se usa al principio, para repartir el secreto. El resto, a toda velocidad con AES.

🔐 RBAC: Control de acceso basado en roles

RBAC (*Role-Based Access Control*) es la forma de decidir **quién puede hacer qué**: los permisos no se asignan a cada persona, sino a su **rol**. Es el principio de **mínimo privilegio** llevado a la práctica.

```python
class Usuario:
    def __init__(self, nombre, rol):
        self.nombre = nombre
        self.rol = rol

PERMISOS = {
    "admin":  ["leer", "escribir", "borrar", "compartir"],
    "editor": ["leer", "escribir"],
    "lector": ["leer"],
}

def puede(usuario, accion):
    return accion in PERMISOS.get(usuario.rol, [])

ana = Usuario("Ana", "admin")
bob = Usuario("Bob", "lector")

print(f"Ana puede borrar: {puede(ana, 'borrar')}")    # True
print(f"Bob puede borrar: {puede(bob, 'borrar')}")    # False
```

RBAC también protege la criptografía:

```python
PERMISOS_CRYPTO = {
    "admin":    ["cifrar", "descifrar", "firmar"],
    "usuario":  ["cifrar", "firmar"],
    "invitado": ["cifrar"],
}

def puede_crypto(usuario, accion):
    return accion in PERMISOS_CRYPTO.get(usuario["rol"], [])

print(puede_crypto({"rol": "admin"}, "descifrar"))   # True
print(puede_crypto({"rol": "invitado"}, "firmar"))   # False
```

🪜 El sistema completo: cifrar + firmar

```python
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives import hashes
import os

# 1. Claves de Ana (firmante) y Bob (destinatario)
clave_ana = rsa.generate_private_key(public_exponent=65537, key_size=2048)
clave_bob = rsa.generate_private_key(public_exponent=65537, key_size=2048)

# 2. Ana prepara el mensaje
mensaje = b"Plan de ataque: manana a las 8"

# 3. Cifrado híbrido: AES cifra, RSA protege la clave AES
clave_aes = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)
aesgcm = AESGCM(clave_aes)
cifrado = aesgcm.encrypt(nonce, mensaje, None)
clave_aes_cifrada = clave_bob.public_key().encrypt(
    clave_aes,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# 4. Firma: SHA256 + clave privada de Ana
firma = clave_ana.sign(
    mensaje,
    padding.PKCS1v15(),
    hashes.SHA256()
)

# 5. Bob recibe y descifra
clave_aes_recibida = clave_bob.decrypt(
    clave_aes_cifrada,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

```
aesgcm2 = AESGCM(clave_aes_recibida)
original = aesgcm2.decrypt(nonce, cifrado, None)

# 6. Bob verifica la firma de Ana
try:
    clave_ana.public_key().verify(
        firma,
        original,
        padding.PKCS1v15(),
        hashes.SHA256()
    )
    print("✅ Firma VÁLIDA — mensaje de Ana, no alterado")
except (ValueError, TypeError):
    print("❌ Firma INVÁLIDA — no es de Ana o fue manipulado")

print(f"📦 Mensaje recibido: {original.decode()}")
```

```
✅ Firma VÁLIDA — mensaje de Ana, no alterado
📦 Mensaje recibido: Plan de ataque: manana a las 8
```

Fase	Pieza	Garantía
3a. Cifrar mensaje	AES (GCM)	Confidencialidad del contenido
3b. Proteger la clave AES	RSA (OAEP)	Solo Bob la descifra
4. Firmar	SHA256 + RSA privada	Autenticidad de Ana
5. Descifrar	RSA privada + AES	Confidencialidad
6. Verificar	SHA256 + RSA pública	Autenticidad + integridad

🖍️ Aprieta el lápiz

1. **AES básico**: Cifra un mensaje con AESGCM, luego descifralo. Muestra el nonce.
2. **RSA: cifra y descifra**: Genera un par RSA, cifra un mensaje corto y descifralo.
3. **Cifrado híbrido**: Cifra un mensaje largo con AES, cifra la clave AES con RSA.
4. **Firma y verifica**: Firma un mensaje, modifícalo y comprueba que la verificación falla.
5. **RBAC**: Implementa un sistema con 3 roles y 4 acciones.

🧠 Mini-chequeo

1. ¿Qué garantiza la fase de cifrado AES y qué garantiza la firma?
2. ¿Por qué el mensaje se cifra con AES y no directamente con RSA?
3. ¿Qué recibe Bob del mensaje y qué comprueba antes de confiar en él?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- El híbrido junta **RSA** (repartir la clave) y **AES** (cifrar el volumen).
- RBAC asigna permisos a **roles**, no a personas: mínima superficie de ataque.
- Un sistema seguro completo combina cifrado híbrido + firma digital + RBAC.

🐛 Vocabulario rápido

Término	Idea general
Cifrado híbrido	AES para el mensaje + RSA para la clave AES
RBAC	Control de acceso basado en roles
Rol	El paquete de permisos de una función
Sistema seguro	Cifrado híbrido + firma digital combinados

📚 Volver al índice de la unidad · **Anterior:** 07 · Firmas digitales · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé el cifrado, laboratorio híbrido con firma y el ring final de la unidad 🧠

> 🗺️ **Estás en:** 🔐 **UD 9 · Seguridad y cifrado** → 09 · Cierre

Has terminado la teoría: principios de seguridad, hash y huellas digitales, contraseñas con sal, cifrado César, AES, RSA, firmas digitales, cifrado híbrido y RBAC. Este cierre es el aterrizaje: recorres lo aprendido con juegos, un laboratorio real con fallos intencionados y las preguntas que te harán en una entrevista.

⭐ Sé el cifrado

> *Primero eres una clave simétrica: la compartes con Ana y con Bob. Luego te conviertes en un par asimétrico: una mitad pública y otra privada.*

Primera vida: eres la clave AES (simétrica).

1. Ana te genera con `AESGCM.generate_key(bit_length=256)`: eres 32 bytes aleatorios.
2. Ana cifra su mensaje contigo: `AESGCM(tú).encrypt(nonce, msg, None)` → texto cifrado.
3. Bob necesita descifrar… pero tú no viajas en el paquete. El **problema de la distribución de claves**.

Segunda vida: eres el par RSA (asimétrico).

1. Naces como par: `rsa.generate_private_key(...)`. Tu mitad **pública** se puede publicar; tu mitad **privada** jamás sale de casa.
2. Bob cifra un secreto con tu mitad pública… y solo tu mitad privada puede descifrarlo.
3. Pero eres lento y no puedes con mensajes grandes: para el volumen necesitas a tu colega AES.

> 💡 **Ahora tú:** ¿quién gana la partida? Ninguno: trabajan en equipo. RSA reparte la clave AES, y AES cifra el tráfico. Eso es lo que hace HTTPS.

🔥 Fireside Chat: Hash vs Cifrado

> *Dos mecanismos de seguridad se sientan junto a la chimenea a resolver, de una vez, quién hace qué.*

Hash: — Yo soy la huella digital. Transformo cualquier texto en una cadena fija. No se puede deshacer. Unidireccional. Para siempre.

Cifrado: — Vaya, qué drástico. Yo puedo cifrar y descifrar. Tengo clave.

Hash: — ¡Esa es precisamente mi gracia! Para contraseñas no quieres que se pueda deshacer.

Cifrado: — Pero para enviar un mensaje secreto, el hash no sirve. Ahí entro yo.

Hash: — Y para verificar integridad, nadie me gana.

Cifrado: — Al final, cada uno a lo suyo. Tú para integridad y contraseñas; yo para confidencialidad.

> **Moraleja**: el hash verifica integridad; el cifrado protege confidencialidad. Ambos son necesarios.

🔥 Fireside Chat: AES vs RSA

> *Dos cifrados se sientan junto a la chimenea a decidir quién es el más importante.*

AES: — Soy rapidísimo. Cifro un archivo entero en milisegundos. ¿El problema? Ambos necesitamos la misma clave.

RSA: — Yo soy lento, pero elegante. Tú me das tu clave pública, yo cifro, y solo tú con tu privada puedes descifrar.

AES: — Entonces, ¿para qué sirves si eres tan lento?

RSA: — Para **distribuir tu clave**. Yo cifro tu clave AES con mi RSA. Tú la descifras con tu privada. Luego usamos AES para todo.

AES: — O sea, ¿trabajamos en equipo?

RSA: — Exacto. Eso se llama **cifrado híbrido**.

> **Moraleja:** AES aporta velocidad y volumen; RSA aporta el reparto de secretos. Juntos mueven Internet.

🕵️ ¿Quién soy?

1. Me genero con `os.urandom(16)` y soy distinto para cada usuario.
2. Soy un diccionario precomputado de hashes de contraseñas comunes.
3. Tuve colisiones demostradas en 2004: ya nadie me usa para seguridad.
4. Un solo bit distinto en la entrada y me convierto en un hash completamente diferente.
5. Soy el estándar de 256 bits que nunca debes romper con MD5 o SHA-1.
6. Soy el cifrado que desplaza cada letra y que se rompe en 25 intentos.
7. Soy el número aleatorio único que viaja junto al cifrado para poder descifrar.
8. Soy el código de autenticación que detecta si el mensaje fue manipulado.
9. Soy el cifrado simétrico estándar, con claves de 16, 24 o 32 bytes.
10. Soy el modo de AES que cifra cada bloque solo y deja ver patrones (¡no me uses!).
11. Soy la clave que se puede repartir por todos los sitios, pero no descifro yo sola.
12. Soy el resultado de cifrar el SHA-256 de un mensaje con la clave privada del firmante.

> 🔁 **Respuestas**

😈 CONRAD VS EL MUNDO: “guardé las contraseñas con MD5 y sin sal”

CONRAD: — “Clásico: la base de datos cae en manos de un atacante y, en cuestión de horas, ha sacado la mitad de las contraseñas. Razones: 1) Usé **MD5**, que tiene colisiones demostradas desde 2004. 2) **Sin sal**: dos usuarios con la misma contraseña comparten hash y las tablas rainbow funcionan al instante. 3) **Guardé la contraseña en claro** en el log ‘por si acaso’. 4) Comparé contraseñas en texto plano en el código. 5) **Cifré** la contraseña en vez de hashaarla… y el atacante robó también la clave de cifrado.”

CONRAD: — “Y no me vengas con *‘¿será que los hashes se ven parecidos?’*. Los hashes no se parecen jamás: el efecto avalancha garantiza que un bit distinto cambia todo.”

😈 CONRAD VS EL MUNDO: “cifré el mensaje pero no puedo descifrarlo”

CONRAD: — "Clásico: cifras un mensaje con AES, lo envías, y al descifrar obtienes basura. Razones: 1) Usaste una **clave de 7 caracteres** como `b'clave123'`: AES espera exactamente 16, 24 o 32 bytes. 2) **No guardaste el nonce**: cada operación genera uno nuevo, y sin el mismo nonce no hay descifrado posible. 3) Cifraste con una clave y descifraste con otra. 4) **No verificaste el tag**: `decrypt` devuelve basura en silencio. 5) Usaste **ECB** para datos estructurados y los patrones se ven a simple vista."

CONRAD: — "Y no me vengas con *'¿será que cryptography está roto?'*. La biblioteca lleva años auditada. Si no descifras, revisa la clave, el nonce y el modo."

⚡ Laboratorio de tortura: registro + login + cifrado híbrido

> **Duración:** 45 minutos **Herramienta:** Python 3 (`pip install cryptography`)

Parte 1 — Registro y login con sal (15 min):

1. Define `registrar(usuario, contraseña)`: genera la sal con `os.urandom(16)`, calcula `hashlib.sha256(salt + contraseña.encode()).hexdigest()`, guarda `salt.hex() + hash`.
2. Define `login(usuario, contraseña)`: recupera la sal del string, recalcula el hash del intento y compara.
3. Registra a dos usuarios con la **misma** contraseña y comprueba que sus hashes son distintos.

Parte 2 — Sistema seguro (30 min):

4. Genera claves RSA para Ana (firmante) y Bob (destinatario).
5. Ana cifra un mensaje con AESGCM, protege la clave AES con RSA pública de Bob.
6. Ana firma el mensaje con su clave privada.
7. Bob descifra y verifica la firma.
8. Modifica **un byte** del mensaje descifrado y comprueba que la verificación del original falla.

Fallo intencionado (Parte 1): en el login, genera una sal nueva con `os.urandom(16)` en lugar de recuperar la del string. Todos los logins fallan.

Fallo intencionado (Parte 2): en la verificación, usa la clave pública de Bob en lugar de la de Ana. La verificación lanza `ValueError`.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Huella Digital**	Calcular MD5, SHA-1, SHA-256 y SHA-512 con `hashlib`
🏅 **Vault Guardian**	Implementar registro y login con hash + sal
🏅 **Rainbow Breaker**	Explicar cómo la sal neutraliza las tablas rainbow
🏅 **César Cracker**	Cifrar, descifrar y romper por fuerza bruta el César
🏅 **Simétric Master**	Cifrar y descifrar con AESGCM
🏅 **Key Pair Forger**	Generar un par RSA y cifrar/descifrar con OAEP
🏅 **Hybrid Engineer**	Montar el cifrado híbrido AES + RSA
🏅 **Signature Detective**	Firmar y verificar, detectando una firma manipulada
🏅 **RBAC Architect**	Implementar un sistema de roles y permisos

🧠 Atrévete a pensar

1. ¿Por qué no se puede “descifrar” un hash, y qué alternativas reales le quedan al atacante?
2. ¿Qué pasaría si dos usuarios con la misma contraseña tuvieran el mismo hash?
3. ¿Por qué no se cifra todo el tráfico de Internet directamente con RSA?
4. ¿Qué pasaría si reutilizaras el mismo nonce dos veces con la misma clave AES?
5. ¿Cómo garantiza el cifrado híbrido que un atacante que intercepta el paquete no pueda leer nada?

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Hash que tiene colisiones demostradas desde 2004 (3 letras)
4. Valor aleatorio que se mezcla con la contraseña antes de hashear (3 letras)
6. Cifrado simétrico estándar con claves de 16/24/32 bytes (3 letras)
8. Cifrado asimétrico más famoso (3 letras)
9. Código de autenticación del cifrado (3 letras)
10. Valor aleatorio único que viaja con el cifrado (5 letras)

Vertical:
2. Algoritmo seguro de 256 bits de la familia SHA-2 (5 letras)
3. Diccionario precomputado de hashes de contraseñas comunes (6 letras)
5. Propiedad del hash: un bit de cambio altera todo (9 letras)
7. Mezcla de AES y RSA (7 letras)
```

> 📝 **Soluciones**

💬 Entrevista de trabajo

1. **“¿Cómo guardarías las contraseñas de tus usuarios en una base de datos?”**
2. **“¿Qué diferencia hay entre hash y cifrado? ¿Cuándo usarías cada uno?”**
3. **“¿Cómo cifrarías un mensaje largo para que solo lo lea un destinatario concreto?”**
4. **“¿Qué es una firma digital y qué garantiza?”**
5. **“Explica cómo funciona HTTPS por dentro.”**
6. **“¿Qué es RBAC y cómo lo implementarías en un sistema?”**

> 💡 **Cómo encararlas:** para la 1, recorre hash + sal (SHA-256, nunca la original). Para la 2, hash → integridad, cifrado → confidencialidad. Para la 3-5, cifrado híbrido: RSA reparte la clave AES, AES cifra el tráfico. Para la 6, roles con permisos mínimos.

🤷 No hay preguntas tontas

> ❓ **¿Se puede descifrar un hash?**

No, el hash no se descifra (es unidireccional). Pero se puede **adivinar** usando tablas rainbow o fuerza bruta. Por eso se usa **sal**.

> ❓ **¿Qué clave uso para cada operación?**

- Cifrar para alguien: su clave **pública**
- Descifrar: tu clave **privada**
- Firmar: tu clave **privada**
- Verificar firma: la clave **pública** del firmante

> ❓ **¿Qué es más seguro, AES-256 o RSA-2048?**

Ambos son seguros. No compiten: son herramientas diferentes. AES protege el volumen; RSA reparte secretos. El sistema seguro los usa juntos.

> ❓ **¿Qué pasa si pierdo mi clave privada?**

Pierdes acceso a todo lo cifrado con tu clave pública. Por eso se hacen **copias de seguridad**.

PRÓXIMAMENTE EN UD 10: *Alta disponibilidad. Sin un hilo por conexión, un solo bucle de eventos atenderá miles de clientes a la vez. El servidor concurrente se vuelve esbelto.*

✅ Criterios de evaluación cubiertos (RA5)

RA5: Implementa mecanismos de seguridad que garanticen integridad y confidencialidad.

CE	Criterio	Cubierto
RA5a	Principios básicos de seguridad	✅ Punto 1
RA5b	Tipos de cifrado (simétrico/asimétrico)	✅ Puntos 5-6
RA5c	Implementa funciones hash (MD5, SHA)	✅ Puntos 2-3 + ⚡ Laboratorio
RA5d	Implementa cifrado AES	✅ Punto 5 + ⚡ Laboratorio
RA5e	Implementa cifrado RSA	✅ Punto 6 + ⚡ Laboratorio
RA5f	Crea y verifica firmas digitales	✅ Punto 7 + ⚡ Laboratorio
RA5g	Implementa cifrado híbrido	✅ Punto 8 + ⚡ Laboratorio
RA5h	Conoce sistemas de roles y RBAC	✅ Punto 8

📚 Volver al índice de la unidad · **Anterior:** 08 · Cifrado híbrido y práctica · **Siguiente:** UD 10 · Alta disponibilidad

UD 10 — Alta disponibilidad

Cerrar el viaje: asincronía, latidos y servicios siempre disponibles ⏱️

> "Asyncio es como un cocinero que, mientras espera a que hierva el agua, corta verduras. En vez de quedarse mirando la olla, hace otras cosas."

En la UD 6 construiste Servidores concurrentes con hilos: un hilo por cliente o un ThreadPool. Funcionan, pero cada hilo cuesta memoria y contexto de sistema. En esta unidad cambiarás de modelo: **asyncio**, un solo hilo que coordina miles de tareas cooperativas, y las herramientas de **disponibilidad** (heartbeat, timeouts y backoff) que mantienen un servicio vivo cuando todo falla alrededor.

También aprenderás a que un servidor **nunca se cuelgue** con un cliente mudo (timeout), a que **avise** de que sigue vivo (heartbeat) y a que **reintente** con cabeza cuando un servicio se cae (backoff). El siguiente tema ya no existe: esta es la última unidad del viaje, y cierras con un servidor asyncio robusto que no se bloquea, se vigila solo y se recupera.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros son teoría en progresión y el 9o aterriza todo en la práctica.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es el **event loop** y cómo coordina tareas asíncronas sin bloquear.
- Definir y usar **corrutinas** con `async def` y `await`.
- Lanzar tareas concurrentes con `asyncio.create_task` y `asyncio.gather`.
- Evitar que un servicio se cuelgue usando **timeouts** con `asyncio.wait_for`.
- Implementar **heartbeats** que verifiquen que un servidor sigue vivo.
- Diseñar **reintentos con backoff exponencial** para conectar ante fallos transitorios.
- Comparar **hilos vs asyncio** y decidir cuál usar en cada escenario.
- Montar un **monitor de servicio** que combine heartbeat, timeout y backoff.
- Aplicar los mecanismos de disponibilidad (RA4e), los servidores asyncio (RA4f) y la comparativa de modelos (RA4g).

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Event Loop	El gestor de tareas que no se bloquea nunca	Todos
02 · Corrutinas	`async def`, `await` y la función que sabe esperar	Todos
03 · create_task y gather	Lanzar varias tareas "a la vez" de verdad	Todos
04 · Timeouts	`wait_for` para que nada se cuelgue para siempre	Todos
05 · Heartbeat	El latido que confirma que el servicio sigue vivo	Todos
06 · Backoff	Reintentos con espera exponencial (1, 2, 4, 8…)	Todos
07 · Threads vs asyncio	La comparativa definitiva de modelos de concurrencia	Todos
08 · Disponibilidad y práctica	El monitor de servicio completo y Aprieta el lápiz	Todos
09 · Cierre	Sé la corrutina, Fireside, Laboratorio de tortura…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> 💡 **Entorno de ejecución (Windows):** los ejemplos imprimen emojis (💓, ⏱️, 🚀). En la consola clásica `cmd` de Windows (codepage cp1252) un `print()` con emoji lanza `UnicodeEncodeError`. Ejecuta con UTF-8: `python -X utf8 archivo.py`, o usa Windows Terminal / VS Code (que ya son UTF-8).

> Practica con los pares del curso: empezar siempre el resuelto para ver el estilo y luego intentar el por-resolver.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

✅ Criterios de evaluación cubiertos (RA4e-g)

RA4: Implementa servicios en red, desarrollando mecanismos de disponibilidad y servidores asíncronos.

CE	Criterio	Dónde se cubre
e)	Implementa mecanismos de disponibilidad (heartbeat, reintentos, timeout)	✅ Puntos 4-6 y 8 + ⚡ Laboratorio (punto 9)
f)	Desarrolla servidores con asyncio	✅ Puntos 1-3 y 8 + ⚡ Laboratorio (punto 9)
g)	Compara modelos de concurrencia (hilos vs asyncio)	✅ Punto 7 + 🔥 Fireside Chat (punto 9)

> RA4c (Servidores concurrentes con hilos) y RA4d (ThreadPool) se cubren en la **UD 6 · Servidores concurrentes**.

🚪 ¿Por dónde empiezo?

¿Vienes de la UD 9 y dominas el cifrado? Perfecto. El trampolín real, eso sí, son los servidores concurrentes: repasa la UD 6 · Servidores concurrentes para tener frescos el hilo por cliente, el ThreadPool y el problema del bloqueo, y arranca en el punto 1, que parte justo del problema de la espera que dejaste planteado.

¿Ya sabes qué es asyncio y solo quieres la disponibilidad? Ve directo al punto 4 y de ahí a los puntos 5 y 6. Pero si vienes de cero, no te saltes los puntos 1 a 3: el event loop y las corrutinas son la base de todo lo demás.

📍 **Primer punto:** 01 · Event Loop

01 — Event Loop

El gestor de tareas que no se bloquea nunca ⚙️

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 01 · Event Loop

📬 La idea en una frase

> El **event loop** es el gestor de asyncio: un bucle que decide qué tarea se ejecuta en cada momento. Cuando una corrutina espera, la pausa y ejecuta otra. Un solo hilo, miles de tareas, ninguna espera activa.

Viene directo del problema que dejaste en la UD 6 · Servidores concurrentes: un servidor que usa `accept()` y `recv()` se **bloquea** mientras espera. La solución de la UD 6 fueron los hilos. La de hoy es distinta: **un solo hilo que cambia de tarea cuando una espera**.

🚫 El problema de esperar

```
# ❌ Esto bloquea TODO el programa
datos = conn.recv(1024)  # El programa se para aquí hasta que lleguen datos
```

Si el código se queda en esa línea, no puede atender a nadie más. El bloqueo es el enemigo. Y las respuestas posibles son dos:

Solución 1: Hilos (UD 6) — caros si hay muchos clientes: cada hilo consume memoria y contexto del sistema operativo.

Solución 2: Asyncio — un solo hilo, pero cambia de tarea cuando una espera. Ningún `recv()` bloqueante, ningún `sleep()` que pare todo.

☕ La analogía de la cafetería

Imagina a un único camarero en una cafetería con 10 mesas:

Modelo	El camarero	Problema
Secuencial	Atiende la mesa 1 hasta el final, luego la 2…	Las mesas 2-10 esperan
Hilos	Contrata 10 camareros, uno por mesa	Caro con 10.000 mesas
Event loop	Anota el pedido de la 1, pasa a la 2 mientras la cafetera trabaja…	Ninguno: nadie espera de brazos cruzados

El camarero del event loop **nunca se queda mirando la cafetera**: cada espera es una oportunidad para atender a otra mesa. Eso es lo que hace asyncio: “esperar” nunca significa quedarse quieto.

⚙️ Qué es el event loop, de verdad

El **event loop** es un bucle que mantiene una lista de tareas listas para ejecutarse. En cada vuelta:

1. Ejecuta la tarea que toca, hasta que encuentra un `await` (una espera).
2. Cuando la tarea espera, la aparca y sigue con la siguiente.
3. Cuando la espera termina (llegan datos, pasa el tiempo), la tarea vuelve a la cola de listas.

```
import asyncio

async def tarea(nombre, segundos):
    print(f"  {nombre} empieza")
    await asyncio.sleep(segundos)
    print(f"  {nombre} termina ({segundos}s)")

async def main():
    # Lanzar 3 tareas "a la vez"
    await asyncio.gather(
        tarea("A", 3),
        tarea("B", 1),
        tarea("C", 2)
    )

asyncio.run(main())
```

Salida:

```
A empieza
B empieza
C empieza
B termina (1s)
C termina (2s)
A termina (3s)
```

> Las 3 empiezan a la vez. B termina primero (solo 1s). El event loop aprovecha los `await` de otras para avanzar.

El event loop también gestiona timers, sockets, procesos y cualquier I/O: cada `await` le dice "esto va a tardar, ocúpate de otras cosas".

🧠 Mini-chequeo

1. ¿Por qué `conn.recv(1024)` bloquea todo el programa en un servidor síncrono?
2. ¿Cuántos hilos usa asyncio?
3. En la analogía de la cafetería, ¿qué representa la cafetera que trabaja sola?

> › 🔁 **Respuestas**

✅ Resumen en 3 frases

- `recv()`, `accept()` y `sleep()` síncronos **bloquean** el hilo y paralizan todo lo demás.
- El **event loop** es un solo hilo que coordina miles de tareas, pausando las que esperan y ejecutando las que están listas.
- Cada `await` es una oportunidad para atender a otra tarea: nadie se queda mirando la olla.

🐛 Vocabulario rápido

Término	Idea general
Event loop	El gestor que decide qué tarea asíncrona se ejecuta

Término	Idea general
Bloqueo	Cuando el hilo espera sin hacer nada (recv, sleep)
Corrutina	Función asíncrona que puede pausarse (punto 2)
asyncio.run()	Arranca el event loop y ejecuta la corrutina principal
await	La marca de “esto va a tardar, ocúpate de otras cosas”

📚 Volver al índice de la unidad · **Siguiente:** 02 · Corrutinas

02 — Corrutinas

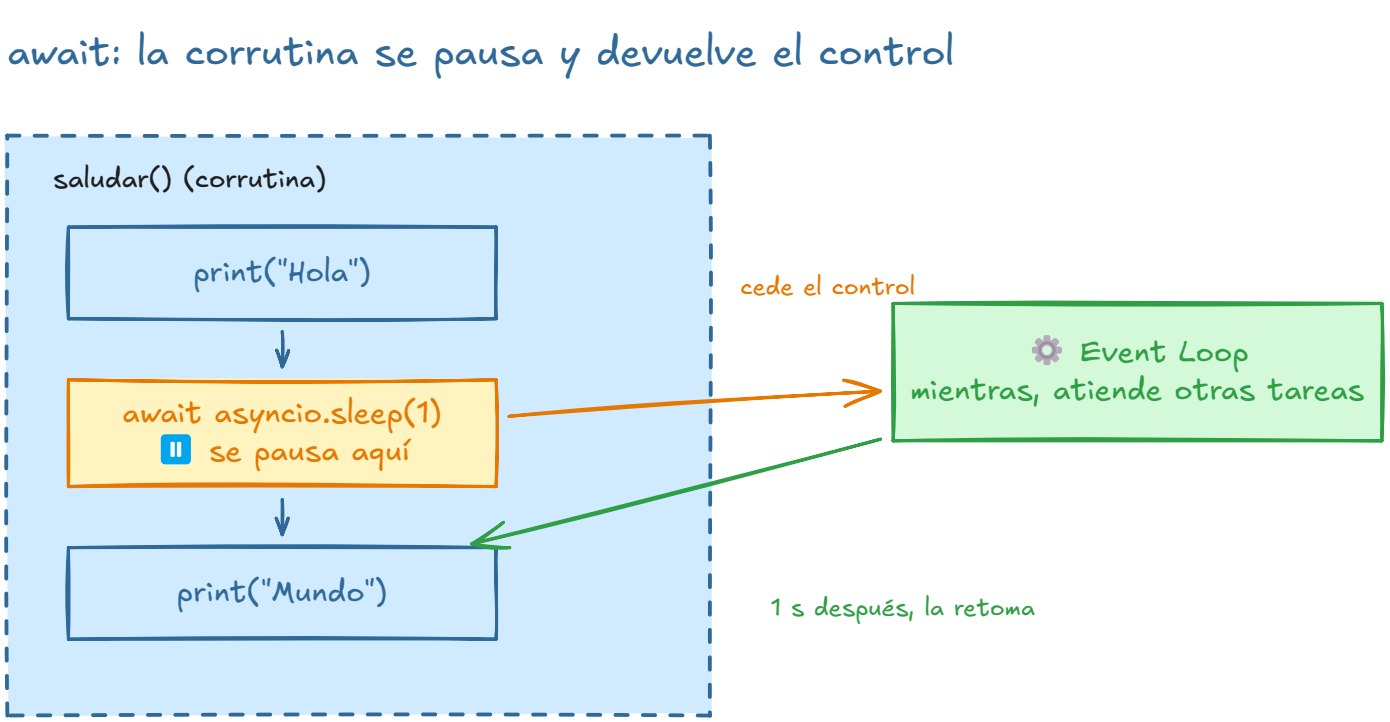
async def, await y la función que sabe esperar

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 02 · Corrutinas

📬 La idea en una frase

> Una **corrutina** es una función declarada con `async def` que puede **pausarse** con `await` y reanudarse después: no es un hilo, es una función que sabe esperar.

async def y await



La pieza básica de asyncio:

```
import asyncio

async def saludar():
    print("Hola")
    await asyncio.sleep(1)  # "Oye, mientras duermo, haz otras cosas"
    print("Mundo")

asyncio.run(saludar())
```

Concepto	Qué es
`async def`	Define una **corrutina** (función que puede pausarse)
`await`	Pausa la corrutina hasta que algo termine
`asyncio.run()`	Crea el event loop y ejecuta la corrutina principal

> `await = "Oye, esto va a tardar. Mientras, ocúpate de otras cosas."`

Cuando se encuentra `await asyncio.sleep(1)`, la corrutina `saludar()` se pausa y devuelve el control al event loop. Un segundo después, el event loop la reanuda en la siguiente línea (`print("Mundo")`). Ese ir-y-volver es lo que hace que un solo hilo sirva a miles de corrutinas.

🤔 Corrutina no es una función normal

La diferencia clave con una función normal:

```
import asyncio

async def tarea():
    return 42

print(tarea())          # ❌ No imprime 42: imprime <coroutine object ...>
```

Si ejecutas el programa, al salir verás además un `RuntimeWarning: coroutine 'tarea' was never awaited`: es la señal del intérprete de que creaste la corrutina pero nunca la ejecutaste. Es el mismo mensaje que “se te olvidó el `await`”.

Llamar a una corrutina **no ejecuta su cuerpo**: crea un objeto corrutina. Solo se ejecuta cuando el event loop la programa, de una de estas formas:

```
asyncio.run(tarea())          # Desde fuera: arranca el event loop

async def main():
    resultado = await tarea() # Desde dentro: espera y obtiene el resultado
```

Forma	Cuándo usarla
`asyncio.run(c)`	Solo para la corrutina principal, una vez por programa
`await c()`	Dentro de otra corrutina: espera su resultado
`asyncio.gather(c1, c2)`	Lanza varias a la vez (punto 3)
`asyncio.create_task(c)`	Lanza en segundo plano (punto 3)

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre `async def f()` y `def f()`?
2. ¿Qué hace `await` con el hilo?
3. Si llamas a `tarea()` (sin `await` ni `run`), ¿qué obtienes?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Una corrutina es una **función que puede pausarse** (`async def` + `await`).
- `await` no bloquea el hilo: cede el control al event loop.
- Para ejecutar una corrutina hace falta `asyncio.run()` o programarla dentro de otra corrutina.

🐛 Vocabulario rápido

Término	Idea general
Corrutina	Función `async def` que puede pausarse y reanudarse
await	Pausa la corrutina y devuelve el control al event loop
asyncio.run()	Arranca el event loop y ejecuta la corrutina principal
Objeto corrutina	El resultado de llamar a una `async def` sin ejecutarla

Término	Idea general
Cooperativo	Las tareas se ceden el control voluntariamente en cada await

📚 Volver al índice de la unidad · **Anterior:** 01 · Event Loop · **Siguiente:** 03 · create_task y gather

03 — create_task y gather

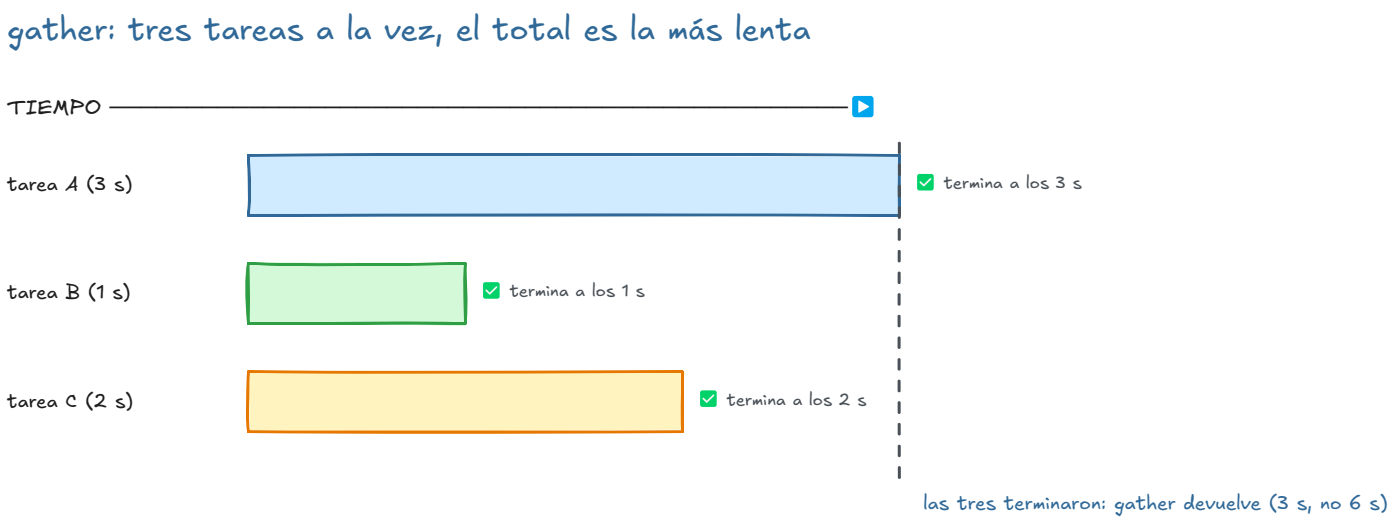
Lanzar varias tareas "a la vez" de verdad 🚀

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 03 · create_task y gather

📬 La idea en una frase

> `asyncio.gather` lanza varias corrutinas a la vez y espera a todas; `asyncio.create_task` lanza una corrutina **en segundo plano** para que la principal siga su camino. Así se consigue la **concurrencia real** de asyncio.

🚀 asyncio.gather: todas a la vez



Del punto 1:

```python
import asyncio

async def tarea(nombre, segundos):
    print(f"  {nombre} empieza")
    await asyncio.sleep(segundos)
    print(f"  {nombre} termina ({segundos}s)")

async def main():
    await asyncio.gather(
        tarea("A", 3),
        tarea("B", 1),
        tarea("C", 2)
    )

asyncio.run(main())
```

Salida:

```
  A empieza
  B empieza
  C empieza
  B termina (1s)
  C termina (2s)
  A termina (3s)
```

`gather` espera a **todas** las corrutinas: el `main()` no termina hasta que A, B y C lo hacen. El tiempo total es el de la más lenta (3s), no la suma (6s).

> Si en lugar de `gather` hicieras `await tarea("A", 3)` → `await tarea("B", 1)` → `await tarea("C", 2)`, serían 3 + 1 + 2 = **6 segundos**. La concurrencia de `gather` es lo que lo reduce a 3.

🎯 asyncio.create_task: en segundo plano

A veces no quieres esperar: lanzas una tarea y sigues con lo tuyo.

```python
import asyncio

async def heartbeat():
    while True:
        print("💓 latido")
        await asyncio.sleep(2)

async def main():
    asyncio.create_task(heartbeat())   # 🎯 en segundo plano, no la espero
    for i in range(3):
        print(f"main trabajando {i}")
        await asyncio.sleep(1)
    print("main termina")

asyncio.run(main())
```

	gather	create_task
Espera	Espera a todas	No espera (segundo plano)
Cuándo usarla	Quieres el resultado de todas	Tareas de fondo: heartbeat, timers, monitorización
Captura	Se puede `await` el resultado	Hay que guardar la referencia para no perderla

> ⚠️ **Truco del maestro:** si creas una tarea con `create_task`, el `main()` termina y **se lleva la tarea consigo**. Para que una tarea de fondo sobreviva, `main()` debe esperar (un `await asyncio.sleep(...)` o `await tarea`) — lo verás en el punto 5.

🧠 Mini-chequeo

1. ¿Cuánto tarda `gather` con tareas de 1, 2 y 3 segundos?
2. ¿Qué hace `create_task` con la corrutina?
3. ¿Qué pasa si `main()` termina y deja tareas creadas con `create_task`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- `asyncio.gather` lanza varias corrutinas a la vez y **espera a todas**.
- `asyncio.create_task` lanza una corrutina **en segundo plano** sin bloquear a la principal.
- La concurrencia de asyncio reduce el tiempo total al de la tarea más lenta, no a la suma.

🐛 Vocabulario rápido

Término	Idea general
gather	Lanza corrutinas a la vez y espera a todas
create_task	Lanza una corrutina en segundo plano
Task	La corrutina ya programada en el event loop
Concurrencia	Varias tareas avanzan en el mismo hilo, alternándose
Segundo plano	La tarea se ejecuta mientras la principal sigue su camino

📚 Volver al índice de la unidad · **Anterior:** 02 · Corrutinas · **Siguiente:** 04 · Timeouts

04 — Timeouts

wait_for para que nada se cuelgue para siempre ⏰

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 04 · Timeouts

📬 La idea en una frase

> Un **timeout** es un tope de tiempo: si una operación tarda más de lo previsto, `asyncio.wait_for` lanza `asyncio.TimeoutError` y el servicio sigue vivo. Nada se cuelga para siempre.

⏰ El problema del cuelgue

Un `reader.read(1024)` espera datos… ¿y si el cliente nunca los envía? La corrutina quedaría esperando para siempre, ocupando su tarea sin hacer nada. En un servidor, un puñado de clientes mudos llena el servicio de corrutinas zombie.

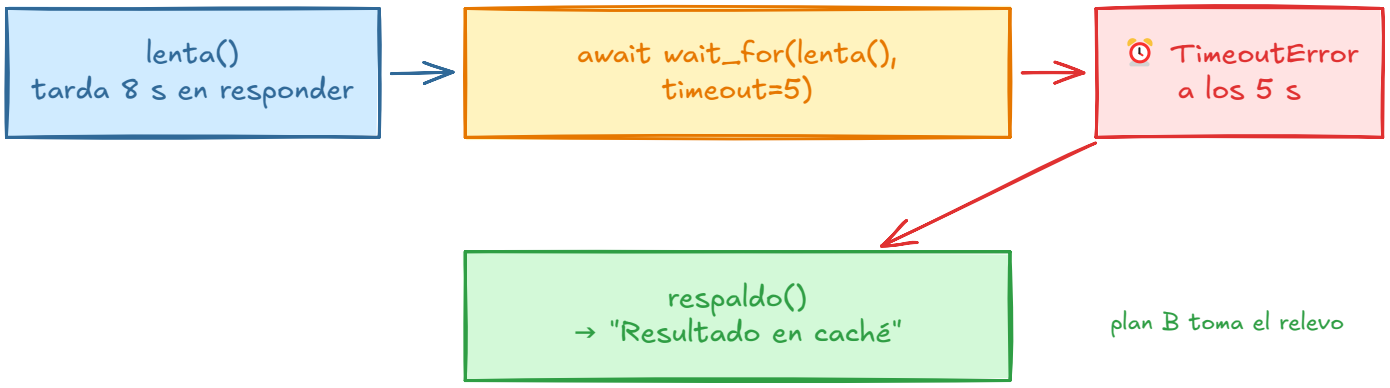
`asyncio.wait_for` pone un límite:

```
import asyncio

async def atender_con_timeout(reader, writer):
    try:
        # Esperar datos con timeout de 10 segundos
        datos = await asyncio.wait_for(reader.read(1024), timeout=10)
        writer.write(b"OK: " + datos)
        await writer.drain()
    except asyncio.TimeoutError:
        writer.write("⏱️ Timeout: conexión cerrada por inactividad\n".encode())
        await writer.drain()
    finally:
        writer.close()
        await writer.wait_closed()
```

Con `timeout=10`, un cliente mudo recibe el aviso y la conexión se cierra a los 10 segundos. **El servidor no se queda colgado con él** y la corrutina libera los recursos.

⏱️ wait_for con respaldo

wait_for: un tiempo límite y un plan B



sin timeout, un cuelgue bloquea el programa para siempre

El patrón del “fallback”: si la tarea tarda demasiado, usas un plan B.

```
import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Resultado lento"

async def respaldo():
    return "Resultado en caché"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=5)
        print(r)
    except asyncio.TimeoutError:
        print(await respaldo())

asyncio.run(main())
```

Salida: `Resultado en caché` — la tarea lenta no llegó en 5 segundos y el respaldo tomó el relevo. Es la semilla del mecanismo de disponibilidad que montarás en el punto 8.

> 💡 El timeout también se combina con el backoff: en cada intento de conexión, `wait_for` corta la espera y el backoff decide cuánto esperar antes del siguiente.

🧠 Mini-chequeo

1. ¿Qué excepción lanza `wait_for` cuando se supera el tiempo?
2. ¿Qué pasa con la corrutina "lenta" tras el timeout?
3. ¿Por qué un timeout es esencial en un servidor de producción?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- `asyncio.wait_for(corrutina, timeout=N)` corta una operación que se pasa de N segundos.
- Al saltar `asyncio.TimeoutError` puedes **responder con un plan B** (respaldo, aviso, cierre).
- Los timeouts son el primer mecanismo de **disponibilidad**: un servicio que no se cuelga está disponible.

🐛 Vocabulario rápido

Término	Idea general
Timeout	Tope de tiempo para una operación
wait_for	Función que aplica el tope y lanza TimeoutError
TimeoutError	Excepción que indica que se superó el tiempo
Respaldo (fallback)	La alternativa que se ejecuta ante el timeout
Cancelar	Interrumpir una corrutina que ya no vale la pena

📚 Volver al índice de la unidad · **Anterior:** 03 · create_task y gather · **Siguiente:** 05 · Heartbeat

05 — Heartbeat

El latido que confirma que el servicio sigue vivo 💓

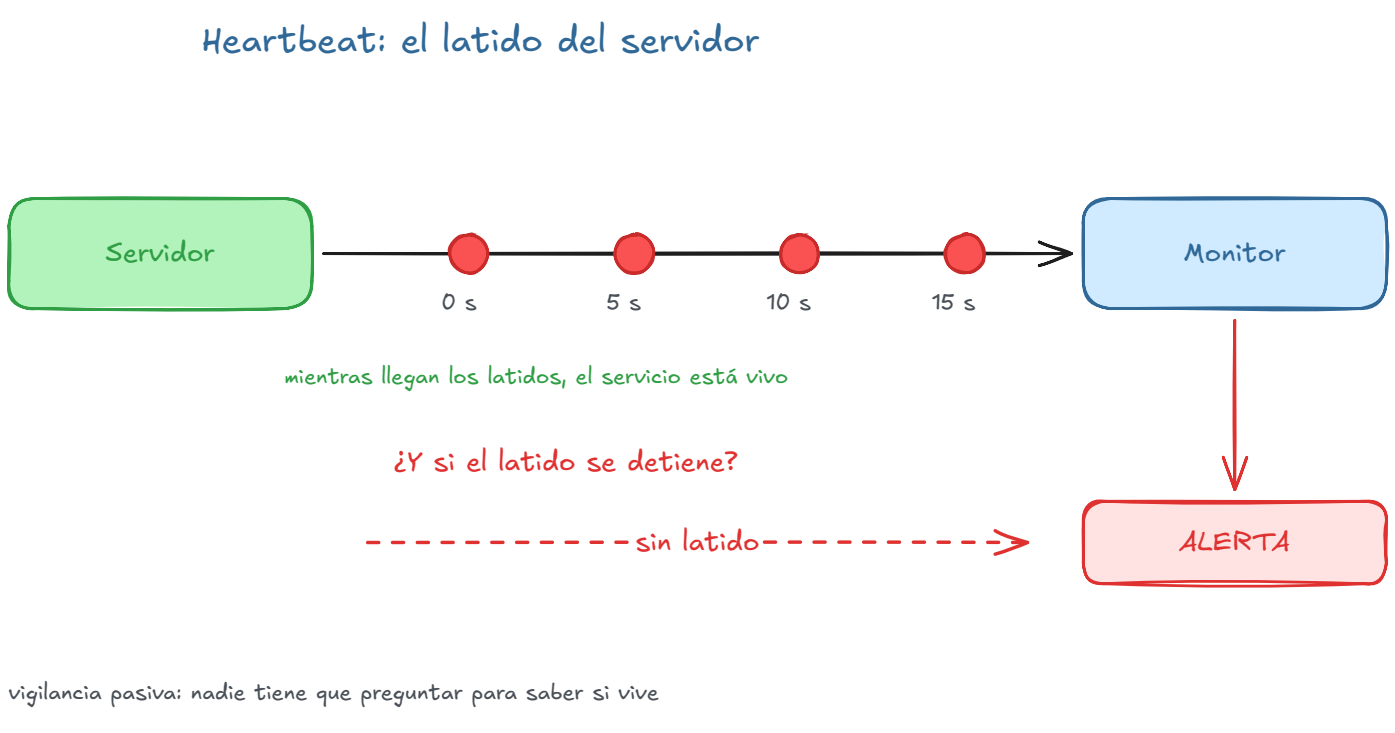
> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 05 · Heartbeat

📬 La idea en una frase

> Un **heartbeat** (latido) es un mensaje periódico para verificar que el servidor sigue vivo. Si el latido deja de sonar, algo va mal: ese es el aviso.

💓 El latido en asyncio

En un hilo, un heartbeat sería un `threading.Thread` daemon con `time.sleep`. En asyncio, es una **corrutina en segundo plano** lanzada con `create_task` (el punto 3):



```python
import asyncio

async def heartbeat(intervalo=5):
    while True:
        print("💓 Heartbeat: servidor vivo")
        await asyncio.sleep(intervalo)

async def servidor_con_heartbeat():
    # Lanzar heartbeat en segundo plano
    asyncio.create_task(heartbeat())

    servidor = await asyncio.start_server(
        lambda r, w: None, "127.0.0.1", 5000
    )
    async with servidor:
        await servidor.serve_forever()

asyncio.run(servidor_con_heartbeat())
```

Cada 5 segundos el servidor imprime que sigue vivo. El `create_task` lo lanza **en segundo plano** mientras el servidor atiende conexiones. La salida:

```
💓 Heartbeat: servidor vivo
💓 Heartbeat: servidor vivo
💓 Heartbeat: servidor vivo
...
```

> El heartbeat responde a la pregunta "¿está vivo el servidor?" sin necesidad de que un cliente haga una petición. Es la **vigilancia pasiva** del servicio.

🩺 Heartbeat con estado

Un heartbeat más útil informa del **estado**: cuántas conexiones atiende ahora mismo.

```python
import asyncio

conexiones = 0

async def atender(reader, writer):
    global conexiones
    conexiones += 1
    datos = await reader.read(1024)
    writer.write(b"OK: " + datos)
    await writer.drain()
    writer.close()
    await writer.wait_closed()
    conexiones -= 1

async def heartbeat():
    while True:
        print(f"💓 Vivo — {conexiones} conexiones")
        await asyncio.sleep(5)
```

Si el latido deja de aparecer en el log… el proceso está muerto o bloqueado. Y si aparece con `0 conexiones` durante horas, sabes que el servicio está arriba pero nadie lo usa: también es información.

🧠 Mini-chequeo

1. ¿Qué es un heartbeat?
2. ¿Por qué se lanza con `create_task` y no con un simple `await`?
3. ¿Qué información adicional puede llevar el latido?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

- Un heartbeat es un **mensaje periódico** que prueba que el servicio sigue vivo.
- En asyncio se implementa como una **corrutina en bucle infinito lanzada con `create_task`**.
- Añadirle el **estado** (conexiones, errores) lo convierte en vigilancia real de disponibilidad.

🐛 Vocabulario rápido

Término	Idea general
Heartbeat	Latido: mensaje periódico de “sigo vivo”
create_task	Cómo se lanza el latido en segundo plano
Bucle infinito	El `while True` del heartbeat
Estado del servicio	Datos que el latido puede reportar (conexiones, errores)
Disponibilidad	Capacidad de un servicio de seguir respondiendo

📚 Volver al índice de la unidad · **Anterior:** 04 · Timeouts · **Siguiente:** 06 · Backoff

06 — Backoff

Reintentos con espera exponencial (1, 2, 4, 8…) 📈

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 06 · Backoff

📬 La idea en una frase

> El **backoff exponencial** reintenta una conexión fallida esperando cada vez más: 1s, 2s, 4s, 8s… Así no machacas a un servidor que ya está teniendo problemas.

📈 El problema: reintentar sin piedad

Un cliente que falla y reintenta en bucle cerrado cada milisegundo puede **tumbar** un servidor ya debilitado: cada intento añade carga. La solución: esperar cada vez más entre intentos.

```
import asyncio

async def conectar_con_backoff(host, port, max_intentos=5):
    for intento in range(max_intentos):
        try:
            reader, writer = await asyncio.wait_for(
                asyncio.open_connection(host, port),
                timeout=3
            )
            print(f"✅ Conectado a {host}:{port}")
            return reader, writer
        except (asyncio.TimeoutError, ConnectionRefusedError):
            espera = 2 ** intento  # 1, 2, 4, 8, 16 segundos
            print(f"⚠️ Intento {intento+1} fallido. Esperando {espera}s...")
            await asyncio.sleep(espera)

    raise Exception("No se pudo conectar")
```

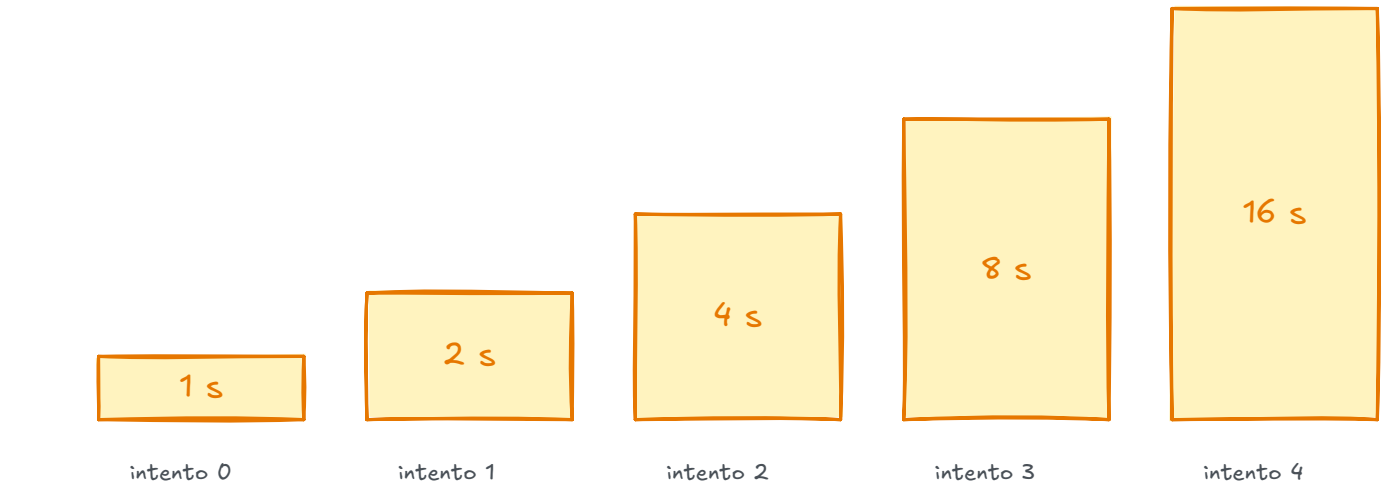
La espera sigue la secuencia **2^0, 2^1, 2^2, 2^3, 2^4 = 1, 2, 4, 8, 16 segundos**. Un fallo puntual se resuelve en el segundo intento con solo 1s de espera; un servidor caído de verdad no recibe 5 peticiones seguidas: recibe 5 con descanso creciente. El `wait_for(..., timeout=3)` añade además un tope por intento (el punto 4).

🔢 La secuencia del backoff

Backoff exponencial: cada reintento espera más (2 ** intento)

❌ reintentar sin piedad = rematar al servidor caído

✔ el backoff le da aire: un cliente paciente, no un ataque



intento 0 → 1 → 2 → 3 → 4

Intento	Espera (2 ** intento)	Acumulado
0	1s	1s
1	2s	3s
2	4s	7s
3	8s	15s
4	16s	31s

> El **backoff exponencial** evita saturar un servidor que ya está teniendo problemas. Es la diferencia entre “un cliente molesto” y “un cliente que remata al servidor”.

🧠 Mini-chequeo

1. ¿Qué secuencia de espera genera `2 ** intento` con 4 reintentos?
2. ¿Qué dos excepciones captura el ejemplo?
3. ¿Por qué es mejor que reintentar sin espera?

> 🔄 Respuestas

✅ Resumen en 3 frases

- El backoff exponencial reintenta con **espera creciente** (`2 ** intento`).
- Se combina con `wait_for` para que cada intento tenga también un **timeout**.
- Es el segundo mecanismo de disponibilidad: el cliente que **aguanta sin machacar**.

🐛 Vocabulario rápido

Término	Idea general
Backoff	Reintento con espera creciente entre intentos
2 ** intento	Fórmula de la espera: 1, 2, 4, 8, 16…
ConnectionRefusedError	El servicio no escucha en ese puerto
Reintento	Nueva tentativa tras un fallo
Jitter (variación)	Añadir aleatoriedad a la espera para evitar avalanchas (tema avanzado)

📚 Volver al índice de la unidad · **Anterior:** 05 · Heartbeat · **Siguiente:** 07 · Threads vs asyncio

07 — Threads vs asyncio

La comparativa definitiva de modelos de concurrencia 🥊

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 07 · Threads vs asyncio

📬 La idea en una frase

> Los **hilos** son multitarea de verdad gestionada por el SO; **asyncio** es multitarea cooperativa en un solo hilo. Para servidores con mucho I/O y miles de conexiones, asyncio gana; para simplicidad con pocos clientes, los hilos.

🥊 El ring de los conceptos

Thread: "Yo soy multitarea de verdad. Tengo mi propia pila, mi propio contexto. El SO me gestiona."

Asyncio: "Yo soy multitarea cooperativa. Un solo hilo, pero cambio de tarea cuando una espera."

Thread: "Si tengo 10.000 clientes, creo 10.000 hilos. El sistema sufre."

Asyncio: "Yo con 10.000 clientes uso un hilo y 10.000 corrutinas. Mucho más ligero."

Thread: "Pero mis operaciones son bloqueantes de verdad. Si llamo a `time.sleep()`, otro hilo ejecuta."

Asyncio: "Mis operaciones son `await` — nunca bloqueo. El event loop decide qué toca."

Thread: "Para servidores pequeños (<100 clientes), soy más simple."

Asyncio: "Para servidores con mucho I/O y muchas conexiones, soy imbatible."

⚖️ La tabla comparativa

Característica	Threads	Asyncio
No de hilos	Varios (gestión del SO)	1 (event loop)

Característica	Threads	Asyncio
Cambio de contexto	Gestionado por el SO	Cooperativo (en await)
Escalabilidad	Media (límite de hilos)	Alta (miles de conexiones)
Complejidad	Baja (simple)	Media (curva de aprendizaje)
CPU-bound	No sirve (GIL)	No sirve
I/O-bound	Sí funciona	Excelente

Dos reglas de oro de la tabla:

- **I/O-bound** (esperas de red, disco, timers): asyncio brilla. Las corrutinas esperan sin ocupar hilos.
- **CPU-bound** (cálculo puro, bucles pesados): ni threads ni asyncio con el **GIL**. Para eso están los procesos (`multiprocessing`, TEMA 01).

🧭 ¿Cuándo usar cada uno?

Situación	Elige
Servidor con pocos clientes (< 100) y quieres simplicidad	Threads
Servidor con miles de conexiones y mucho I/O	**Asyncio**
Código de terceros bloqueante (`requests`, `time.sleep`)	Threads (o `run_in_executor`)
Todo el stack propio, operaciones con `await`	**Asyncio**
Cálculo intensivo de CPU	Ninguno: procesos (multiprocessing)

> 💡 Puedes **mezclar** ambos: `loop.run_in_executor()` lleva código bloqueante a un hilo real mientras el event loop sigue. Pero como norma, "mejor si todo es asyncio": un solo modelo mental.

🧠 Mini-chequeo

1. ¿Quién gestiona el cambio de contexto en threads? ¿Y en asyncio?
2. ¿Por qué asyncio no sirve para CPU-bound?
3. ¿Cuándo es más simple usar threads?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Threads = **multitarea del SO**; asyncio = **multitarea cooperativa en un hilo**.
- Para I/O-bound con muchas conexiones, **asyncio es imbatible**; para pocos clientes, los hilos son más simples.
- Ninguno sirve para CPU-bound (GIL): ahí mandan los **procesos**.

🐛 Vocabulario rápido

Término	Idea general
Multitarea del SO	El sistema operativo decide qué hilo ejecuta
Multitarea cooperativa	Las corrutinas ceden el control voluntariamente en cada await
GIL	Candado de Python que limita la ejecución paralela de hilos
I/O-bound	Tarea que pasa la mayor parte del tiempo esperando
CPU-bound	Tarea que pasa la mayor parte del tiempo calculando
run_in_executor	Puente para llevar código bloqueante a un hilo real

📚 Volver al índice de la unidad · **Anterior:** 06 · Backoff · **Siguiente:** 08 · Disponibilidad y práctica

08 — Disponibilidad y práctica

Todo junto: el monitor de servicio y Aprieta el lápiz 🩺

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 08 · Disponibilidad y práctica

---

📬 La idea en una frase

> Este punto junta la unidad: un **monitor de servicio** que vigila un servidor con **heartbeat** (¿sigue vivo?), lo comprueba con **timeout** (¿responde a tiempo?) y reconecta con **backoff** (¿reintento sin machacar?). Disponibilidad = latido + timeout + backoff.

🩺 El monitor de servicio

Un monitor asíncrono que comprueba el estado de un servicio cada N segundos y reintenta con backoff cuando cae:

```python
import asyncio

async def comprobar_servicio(host, port):
    """Devuelve True si el servicio responde dentro del timeout."""
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(host, port), timeout=2
        )
        writer.close()
        await writer.wait_closed()
        return True
    except (asyncio.TimeoutError, ConnectionRefusedError):
        return False

async def monitor(host, port):
    fallos = 0
    while True:
        if await comprobar_servicio(host, port):
            print(f"💚 {host}:{port} disponible (fallos seguidos: {fallos})")
            fallos = 0
        else:
            fallos += 1
            espera = 2 ** min(fallos, 4)   # backoff: 2, 4, 8, 16, 16…
            print(f"💔 {host}:{port} caído — reintento en {espera}s")
            await asyncio.sleep(espera)
            continue
        await asyncio.sleep(5)             # comprobación periódica

asyncio.run(monitor("127.0.0.1", 5000))
```

Tres mecanismos de disponibilidad en un solo script:

Mecanismo	Qué hace	Dónde
Heartbeat	Comprueba el servicio periódicamente (cada 5s)	`while True` + `asyncio.sleep(5)`
Timeout	No espera a un servicio mudo más de 2s	`asyncio.wait_for(..., timeout=2)`

Mecanismo	Qué hace	Dónde
Backoff	Ante fallos, espera cada vez más (2, 4, 8…)	`2 ** min(fallos, 4)`

🎭 Be the code: el event loop en un servidor

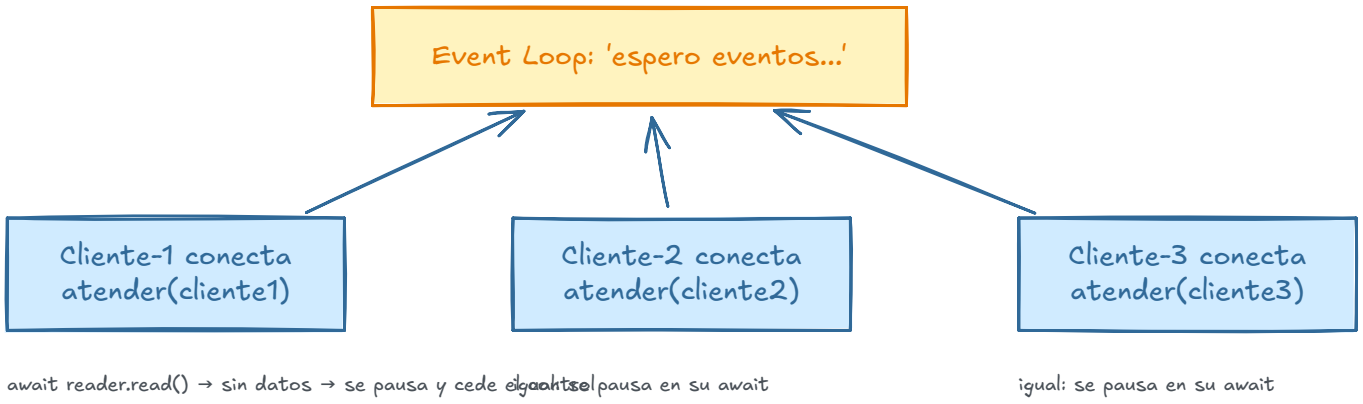
> "Sé el event loop. Tu trabajo es coordinar corrutinas sin bloquear ni un milisegundo."

El servidor asyncio con todas las piezas (del punto 3 + el punto 5):

```
import asyncio

async def atender(reader, writer):
    addr = writer.get_extra_info('peername')
    print(f"[+] Cliente {addr} conectado")

    datos = await reader.read(1024)
    print(f"    Recibido: {datos.decode()}")

    writer.write(b"OK: " + datos)
    await writer.drain()  # Espera a que se envíe

    writer.close()
    await writer.wait_closed()
    print(f"[-] Cliente {addr} desconectado")

async def main():
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    print("🚀 Servidor ASYNCIO en 127.0.0.1:5000")

    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())
```

La traza, paso a paso:

El Event Loop: un camarero que nunca se queda quieto



> Nunca hay espera activa. Cuando una corrutina espera, otra aprovecha. **Un solo hilo, miles de conexiones.**

📝 Aprieta el lápiz

1. **Asyncio básico**: Crea 3 corrutinas que esperen 1, 2 y 3 segundos. Lánzalas con `gather` y mide el tiempo total.
2. **Servidor asyncio**: Convierte el servidor TCP de la UD 6 a asyncio.
3. **Heartbeat**: Añade un heartbeat que imprima "💓 vivo" cada 3s mientras el servidor funciona.
4. **Backoff**: Crea un cliente que intente conectarse 3 veces con backoff exponencial.
5. **Monitor**: Monta el monitor de servicio con heartbeat + timeout + backoff que vigile un puerto.

> › 🔓 **Soluciones**

🧠 Mini-chequeo

1. ¿Qué tres mecanismos de disponibilidad combina el monitor?
2. En la traza del servidor, ¿qué hace el event loop cuando `atender(cliente1)` llega a `await reader.read()`?
3. ¿Por qué el servidor asyncio atiende a 2 clientes a la vez sin hilos?

> 🔁 Respuestas

✅ Resumen en 3 frases

- Disponibilidad = **heartbeat + timeout + backoff**: vigilas, cortas lo que tarda y reintentas con cabeza.
- El servidor asyncio atiende miles de conexiones en **un solo hilo** gracias a los `await`.
- Con el monitor y el servidor completos, ya tienes el material para el cierre y los boletines.

🐛 Vocabulario rápido

Término	Idea general
Monitor de servicio	Vigila el estado de un servicio periódicamente
Disponibilidad	Un servicio que no se cuelga, avisa y se recupera
start_server	La función de asyncio que crea el servidor TCP
serve_forever	Mantiene el servidor escuchando hasta el Ctrl+C
reader / writer	Los objetos asyncio de lectura y escritura de cada cliente

📚 Volver al índice de la unidad · **Anterior:** 07 · Threads vs asyncio · **Siguiente:** 09 · Cierre

09 — Cierre: consolida lo aprendido

Sé la corrutina, el ring final y el último Laboratorio de tortura 🏁

> 🗺️ **Estás en:** ⏱️ **UD 10 · Alta disponibilidad** → 09 · Cierre

Has terminado la teoría: el event loop, las corrutinas, gather y create_task, los timeouts, el heartbeat, el backoff y la comparativa con los hilos. Este cierre es el aterrizaje final del viaje: recorres lo aprendido con juegos, un laboratorio con fallos intencionados y las preguntas que te harán en una entrevista. Léelo justo después del punto 8 y antes de abrir los boletines.

⭐ Sé la corrutina

> *Eres una corrutina `monitor()` en un event loop con 2.000 corrutinas más. Tu trabajo: comprobar un servicio cada 5 segundos y avisar si cae.*

¿Qué pasa?

1. El event loop te ejecuta. Llegas a `await asyncio.wait_for(open_connection(...), timeout=2)`: **te pausas** y el event loop pasa a otras corrutinas.
2. El servicio responde. El event loop te reanuda: imprimes 💚 `disponible`.
3. Llegas a `await asyncio.sleep(5)`: **te vuelves a pausar**. Durante esos 5 segundos, el event loop atiende a las otras 1.999 corrutinas.
4. Un día el servicio no responde: `TimeoutError`. Imprimes 💔 `caído` y esperas `2 ** min(fallos, 4)` segundos (**backoff** con tope: 1, 2, 4, 8, 16, 16…), pausada, mientras el event loop sigue con las demás.
5. Así, con una corrutina, vigilas un servicio **sin bloquear a nadie**: las otras 2.000 corrutinas nunca se enteran de tu espera.

> 💡 **Ahora tú:** ¿y si fueran 2.000 monitores vigilando 2.000 servicios? Con asyncio no pasa nada: 2.000 corrutinas en un hilo. Con hilos, 2.000 hilos hundirían el sistema (punto 7).

🔥 Fireside Chat: hilos vs asyncio

> *Dos modelos de concurrencia se sientan junto a la chimenea a resolver, de una vez, quién atiende mejor a los clientes.*

Hilo: — Yo soy multitarea de verdad. Tengo mi propia pila, mi propio contexto. El SO me gestiona.

Asyncio: — Yo soy multitarea cooperativa. Un solo hilo, pero cambio de tarea cuando una espera.

Hilo: — Si tengo 10.000 clientes, creo 10.000 hilos. El sistema sufre.

Asyncio: — Yo con 10.000 clientes uso un hilo y 10.000 corrutinas. Mucho más ligero.

Hilo: — Pero mis operaciones son bloqueantes de verdad. Si llamo a `time.sleep()`, otro hilo ejecuta.

Asyncio: — Mis operaciones son `await` — nunca bloqueo. El event loop decide qué toca.

Hilo: — Para servidores pequeños (<100 clientes), soy más simple.

Asyncio: — Para servidores con mucho I/O y muchas conexiones, soy imbatible.

> **Moraleja**: Hilos para lo simple y bloqueante; asyncio para escalar en I/O. Y si algo calcula a saco, ni uno ni otro: procesos.

🕵️‍♀️ ¿Quién soy?

1. Soy el gestor que decide qué corrutina se ejecuta en cada momento.
2. Soy la función declarada con `async def` que puede pausarse.
3. Soy la marca que le dice al event loop “esto va a tardar, ocúpate de otras cosas”.
4. Soy la llamada que lanza varias corrutinas a la vez y espera a todas.
5. Soy el mensaje periódico que confirma que un servicio sigue vivo.
6. Soy la excepción que aparece cuando una operación supera su timeout.
7. Soy la espera que crece en cada reintento: 1, 2, 4, 8…

> › 🔄 **Respuestas**

👹 CONRAD VS EL MUNDO: “el servidor se cuelga si un cliente no responde”

CONRAD: — "Clásico: montas un servidor asyncio, llega un cliente que no envía nada y *'el servidor se ha quedado colgado'*. Pues no: el servidor sigue, lo que pasa es que **una corrutina se queda esperando a un `read()` que nunca llega**. Cada cliente mudo es una corrutina ocupada para siempre. Sin timeout, un puñado de clientes mudos te llena el servicio de corrutinas zombie."

CONRAD: — "Y lo mejor: *'he puesto `asyncio.sleep(2)` y mi servidor va lento'*. ¡Pues claro! Y si te olvidas del `await` y escribes `asyncio.sleep(2)` sin más, la corrutina ni se pausa ni espera: la tarea no hace lo que crees. Revisa que cada llamada asíncrona tenga su `await`."

CONRAD: — "Y no me vengas con *'¿será que asyncio no vale?'*. El event loop va perfecto: el problema es que **no hay timeout** ni **backoff**. Envuelve el `read()` en `asyncio.wait_for`, añade un heartbeat, y el servicio no se cuelga ni se te muere en silencio. A diagnosticar."

⚡ Laboratorio de tortura: el monitor de servicio definitivo

> **Duración:** 45 minutos **Herramienta:** Python 3 (solo stdlib: `asyncio`)

Escenario: construye un **monitor de servicio** que vigile `127.0.0.1:5000` con heartbeat (comprobar cada 5s), timeout (2s por comprobación) y backoff (2, 4, 8…). Acompáñalo de un servidor asyncio mínimo que se apague a los 15 segundos para que el monitor experimente una caída real.

Tareas paso a paso:

1. Escribe el servidor asyncio mínimo con `asyncio.start_server` que responda `b"OK"` y se detenga a los 15s con `asyncio.sleep(15)` (punto 8).
2. Escribe `comprobar_servicio()` con `asyncio.wait_for(asyncio.open_connection(...), timeout=2)` que devuelva `True`/`False` (punto 4).
3. Escribe el monitor con `while True` + `asyncio.sleep(5)` y backoff `2 ** min(fallos, 4)` (puntos 5 y 6).
4. Ejecuta monitor y servidor en **terminales separadas**. El servidor se apaga solo a los 15s; el monitor debe pasar de 💚 `disponible` a 💔 `caído` con esperas crecientes.
5. Vuelve a arrancar el servidor a mano: el monitor debe volver a 💚 (se ha recuperado).

Fallo intencionado: cambia el timeout del monitor a **`timeout=0.1`** con el servidor funcionando. ¿Qué pasa? El monitor ve `TimeoutError` casi siempre, aunque el servicio esté sano: con 0.1s no

le da tiempo a completar la conexión. Ahora tienes un monitor **falso positivo**: reporta caído un servicio que responde. Un timeout demasiado agresivo es tan malo como no tenerlo.

> **Pista 1:** el fallo no está en el servidor (que responde), está en el **timeout del monitor**: con `timeout=0.1` la comprobación no llega a completarse y salta `TimeoutError`. Es el mismo mecanismo del punto 4, pero usado mal.
>
> **Pista 2:** fíjate en el orden del punto 8: heartbeat (¿compruebo?), timeout (¿respondo a tiempo?), backoff (¿cómo reintento?). Si el monitor dice "caído" con el servicio arriba, el problema es el **umbral**: el timeout debe ser mayor que el tiempo real de respuesta del servicio.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **Loop Whisperer**	Explicar qué hace el event loop cuando una corrutina espera
🏅 **Coroutine Master**	Escribir corrutinas con `async def` y `await` sin bloqueos
🏅 **Gather Gatherer**	Lanzar tareas concurrentes con `gather` y `create_task`
🏅 **Timeout Guardian**	Evitar que un servicio se cuelgue con `wait_for`
🏅 **Heartbeat Keeper**	Añadir un latido que avise de que el servicio sigue vivo
🏅 **Backoff Strategist**	Reintentar conexiones con espera exponencial
🏅 **Model Decider**	Elegir con criterio entre hilos y asyncio

---

🧠 Atrévete a pensar

1. ¿Por qué asyncio usa un solo hilo y aún así atiende miles de conexiones?
2. ¿Qué pasa si olvidas el `await` delante de una operación asíncrona?
3. ¿Cuándo usarías threads y cuándo asyncio en producción?
4. ¿Por qué un timeout demasiado corto es peor que no tenerlo?
5. ¿Cómo combinarías heartbeat, timeout y backoff en un solo servicio?

> 💡 Soluciones

🧩 Crucigrama de bits

```
Horizontal:
1. El gestor que decide qué corrutina ejecutar (11 letras)
4. Función asíncrona que puede pausarse (10 letras)
6. La espera creciente de los reintentos (7 letras)
8. Mensaje periódico que confirma que un servicio vive (9 letras)

Vertical:
2. Llamada que lanza varias corrutinas a la vez y espera a todas (6 letras)
3. La marca "esto va a tardar, ocúpate de otras cosas" (5 letras)
5. Excepción que aparece al superar el tiempo límite (11 letras)
7. Tope de tiempo para una operación (7 letras)
```

> 📝 Soluciones

💬 Entrevista de trabajo

1. **"¿Qué es asyncio y en qué se diferencia de los hilos?"**
2. **"Explica qué hace el event loop con un ejemplo."**
3. **"¿Qué es una corrutina? ¿Qué diferencia `await` de un `sleep()` bloqueante?"**
4. **"¿Cómo harías que un servidor no se cuelgue nunca con un cliente mudo?"**
5. **"Diseña un servicio siempre disponible: ¿qué mecanismos le pones?"**

> 💡 **Cómo encararlas:** la 1 y la 4 son las "preguntas reina". Para la 1, recorre la cadena de la unidad: bloqueo → hilos (caros con miles) → event loop → corrutinas → gather/create_task → comparativa del punto 7. Para la 4, plantea `asyncio.wait_for(reader.read(...), timeout=10)` con `except asyncio.TimeoutError` y cierre de la conexión: el punto 4 es tu respuesta. Y la 5 es tu momento estrella: heartbeat + timeout + backoff. Si lo cuentas fluido, ya eres medio ingeniero de disponibilidad.

🤷 No hay preguntas tontas

> ❓ **¿Asyncio es más rápido que threads?**

Para I/O-bound tasks, sí, porque no hay cambio de contexto del SO. Para CPU-bound, no hay diferencia (ambos limitados por el GIL).

> ❓ **¿Puedo mezclar código síncrono con asyncio?**

Sí, con `loop.run_in_executor()`. Pero mejor si todo es asyncio.

> ❓ **¿Qué es una corrutina?**

Una función declarada con `async def` que puede pausarse con `await` y reanudarse después. No es un hilo, es una función que sabe esperar.

> ❓ **¿`await` bloquea el hilo?**

No. `await` le dice al event loop: "ahora no necesito CPU, ocúpate de otras corrutinas". Es **cooperativo**.

> ❓ **¿Cuándo usar threads y cuándo asyncio?**

- Threads: proyectos pequeños, librerías bloqueantes, simplicidad.
- Asyncio: muchos clientes concurrentes, mucho I/O, escalabilidad.

> ❓ **¿Y si me olvido del `await`?**

La operación no se espera (ni a veces se ejecuta): la corrutina no hace lo que crees. Es el fallo más típico al empezar con asyncio.

🎬 Poscréditos

> *El servidor secuencial hacía esperar a todos en la cola de la ventanilla única.*
>
> *Llegó el hilo por cliente y cada conexión tuvo su propia ventanilla.*
>
> *El ThreadPool puso límites al entusiasmo: cinco camareros, cola ordenada, nadie se ahoga.*
>
> *El Lock protegió el contador. El benchmark demostró la mejora. Los clientes responden a la vez.*
>
> *Y entonces llegó asyncio: un solo camarero, miles de mesas, ninguna espera quieta.*

> *El latido confirma que el servicio sigue vivo. El timeout corta lo que se cuelga. El backoff reintenta con cabeza.*
>
> *El viaje ha recorrido diez hitos: 🐍 Python, 🚀 Procesos, 🔀 Hilos, 🔒 Sincronización, 🔌 Sockets, 🏗️ Servidores, 🌐 API REST, 🧪 APIs comerciales, 🔐 Seguridad y ⏱️ Alta disponibilidad.*
>
> *Pero aún queda una parada más: el puente hacia el ecosistema Java.*

🎬 PRÓXIMAMENTE EN UD 11 · Spring Boot con Java

Del Python al backend empresarial: **Java**, **Spring Boot**, inyección de dependencias, `@RestController` y JPA. El puente final del viaje te espera en la UD 11 · Spring Boot.

✅ Criterios de evaluación cubiertos (RA4e-g)

RA4: Implementa servicios en red, desarrollando mecanismos de disponibilidad y servidores asíncronos.

CE	Criterio	Cubierto
e)	Implementa mecanismos de disponibilidad (heartbeat, reintentos, timeout)	✅ Heartbeat, backoff y timeouts (puntos 4-6) + ⚡ Laboratorio
f)	Desarrolla servidores con asyncio	✅ Corrutinas, gather y start_server (puntos 1-3 y 8) + ⚡ Laboratorio
g)	Compara modelos de concurrencia (hilos vs asyncio)	✅ Comparativa (punto 7) + 🔥 Fireside Chat

> RA4c (Servidores concurrentes con hilos) y RA4d (ThreadPool) se cubrieron en la **UD 6 · Servidores concurrentes**.

📚 Volver al índice de la unidad · **Anterior:** 08 · Disponibilidad y práctica · **Siguiente:** UD 11 · Spring Boot con Java

UD 11 — Spring Boot con Java

Del Python al ecosistema Java: Spring Boot, DI, REST, JPA ☕

Python es genial para aprender y para producir rápido, pero en el mundo empresarial Java sigue siendo rey. Esta unidad te da un puente directo: partiendo de lo que ya sabes en Python, verás los conceptos clave de **Spring Boot** (el framework Java más usado para servicios web) y cómo se comparan con lo que ya dominas.

No es un curso completo de Java: es una guía de supervivencia para que entiendas un proyecto Spring Boot, puedas leerlo, modificarlo y crear uno básico. Si en el futuro necesitas profundizar, tendrás la base sólida.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Entender la sintaxis básica de **Java** comparándola con Python.
- Explicar qué es **Spring Boot** y por qué domina el backend empresarial.
- Comprender la **inyección de dependencias** (DI) y el patrón de contenedor IoC.
- Crear un API REST con **@RestController**.
- Conectar a una base de datos con **JPA** (Hibernate).
- Montar un mini-proyecto completo: API de tareas con CRUD.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Java para desarrolladores Python	Sintaxis, tipos, clases, concurrencia	Todos
02 · Introducción a Spring Boot	Qué es, por qué, estructura de proyecto	Todos
03 · Inyección de dependencias	DI, IoC, @Autowired, Bean	Todos

Punto	Qué aprenderás	Nivel
04 · RESTController y APIs REST	@RestController, @GetMapping, @PostMapping	Todos
05 · JPA y bases de datos	@Entity, Repository, CRUD	Todos
06 · Ejemplo completo: API de tareas	Proyecto completo con todas las piezas	Todos
07 · Cierre	Comparativa final, recursos, siguiente paso	Todos

📝 ¿Qué necesitas?

- Conocimientos de Python (lo que has visto en la UD 1).
- Un IDE: **IntelliJ IDEA** (Community, gratis) o **VS Code** con extensión de Java.
- **Java 17+** instalado (JDK, no solo JRE).
- Curiosidad por ver "cómo lo hacen los otros".

📝 Boletines de la unidad

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto

🚪 ¿Por dónde empiezo?

Empieza por el punto 1 si no has visto Java nunca. Si ya sabes lo básico, salta al punto 2 directamente.

📍 **Primer punto:** 01 · Java para desarrolladores Python

01 — Java para desarrolladores Python

Puente rápido entre Python y Java: tipos, clases, concurrencia ☕

> 🗺️ **Estás en:** ☕ **UD 11 · Spring Boot** → 01 · Java para desarrolladores Python

📬 La idea en una frase

> Java es **estáticamente tipado** (el tipo se declara al crear la variable) y **orientado a objetos** (todo vive dentro de clases). Donde Python es flexible y conciso, Java es explícito y estructurado. Cambia la sintaxis, no el pensamiento.

🔄 Python vs Java: la comparativa rápida

Concepto	Python	Java
Declarar variable	`x = 10`	`int x = 10;`
Función	`def hola(nombre):`	`public void hola(String nombre) {}`
Clase	`class Gato:`	`public class Gato {}`
Lista	`lista = [1, 2, 3]`	`List<Integer> lista = List.of(1, 2, 3);`
Imprimir	`print(x)`	`System.out.println(x);`
Null	`None`	`null`
Comentario	`# esto`	`// esto`

🐉 Tipos básicos

```java
public class TiposBasicos {
    public static void main(String[] args) {
        int entero = 42;
        double decimal = 3.14;
        boolean activo = true;
        String texto = "Hola Mundo";
        char letra = 'A';

        System.out.println("Entero: " + entero);
        System.out.println("Decimal: " + decimal);
        System.out.println("Activo: " + activo);
        System.out.println("Texto: " + texto);
        System.out.println("Letra: " + letra);
    }
}
```

Diferencia clave: en Python todo es dinámico. En Java **declaras el tipo** (`int`, `double`, `String`…). Eso da más seguridad: el compilador detecta errores antes de ejecutar.

📦 Colecciones

```java
import java.util.*;

List<String> frutas = new ArrayList<>(List.of("manzana", "pera", "uva"));
frutas.add("naranja");
System.out.println(frutas);  // [manzana, pera, uva, naranja]

Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Bob", 30);
System.out.println(edades.get("Ana"));  // 25
```

Comparativa:

Python	Java
`lista = [1, 2, 3]`	`List<Integer> lista = new ArrayList<>(List.of(1, 2, 3));`
`dic = {"a": 1}`	`Map<String, Integer> dic = new HashMap<>();`

Python	Java
`for x in lista:`	`for (int x : lista) {}`

Clases y objetos

```java
public class Gato {
    private String nombre;
    private int vidas;

    // Constructor
    public Gato(String nombre) {
        this.nombre = nombre;
        this.vidas = 9;
    }

    // Método
    public void maullar() {
        System.out.println(nombre + ": ¡Miau!");
    }

    // Getter
    public String getNombre() {
        return nombre;
    }
}

// Uso
Gato miGato = new Gato("Felix");
miGato.maullar();  // Felix: ¡Miau!
```

Comparativa con Python:

```python
# Python
class Gato:
    def __init__(self, nombre):
        self.nombre = nombre
        self.vidas = 9

    def maullar(self):
        print(f"{self.nombre}: ¡Miau!")

mi_gato = Gato("Felix")
mi_gato.maullar()
```

Diferencias clave:

- Java usa `private` para encapsular (Python no tiene privados reales).
- Java necesita `public` para acceder desde fuera.
- Java usa `this.` explícitamente.
- Java tiene **interfaces** (como Python abstracto, pero forzado).

⚡ Concurrencia

```java
// Java: hilos con Thread
Thread hilo = new Thread(() -> {
    System.out.println("Hilo en ejecución");
});
hilo.start();
hilo.join();
```

Comparativa:

Python	Java
`threading.Thread(target=fn)`	`new Thread(() -> { ... })`
`hilo.start()`	`hilo.start()`
`hilo.join()`	`hilo.join()`
GIL (un hilo a la vez en CPU)	Sin GIL (hilos reales en paralelo)

> 💡 **Ventaja de Java:** sin GIL, los hilos truly paralelos en multiproceso. En Python necesitas `multiprocessing` para eso.

🧠 Mini-chequeo

1. ¿Qué diferencia hay entre `int x = 10;` en Java y `x = 10` en Python?
2. ¿Por qué Java usa `private` en los atributos de una clase?
3. ¿Cuál es la ventaja de Java sobre Python en concurrencia?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- Java es estáticamente tipado y orientado a objetos: declara tipos, usa clases con `private`/`public`, y tiene interfaces forzadas.
- Las colecciones (`List`, `Map`) son como las de Python pero con tipos explícitos.
- En concurrencia, Java gana por no tener GIL: hilos truly paralelos.

🐛 Vocabulario rápido

Término	Idea general
JDK	Java Development Kit: compila y ejecuta Java
Estáticamente tipado	El tipo se declara al crear la variable
GIL	Global Interpreter Lock: limita Python a un hilo en CPU
Interfaz	Contrato que una clase debe cumplir (métodos obligatorios)

📚 Volver al índice de la unidad · **Siguiente:** 02 · Introducción a Spring Boot

02 — Introducción a Spring Boot

El framework que domina el backend empresarial 🌱

> 🗺️ **Estás en:** 🟤 **UD 11 · Spring Boot** → 02 · Introducción a Spring Boot

📬 La idea en una frase

> **Spring Boot** es un framework Java que te permite crear servicios web (APIs REST, aplicaciones web, microservicios) en minutos, sin configurar servidores ni XML. Es el Flask/Django del mundo Java, pero con más estructura y opiniones.

🌱 ¿Qué es Spring Boot?

Spring Boot es una **extensión de Spring Framework** que simplifica enormemente el desarrollo de aplicaciones Java:

- **Autoconfiguración:** detecta qué necesitas y lo configura solo.
- **Servidor embebido:** tu app trae su propio servidor (Tomcat/Netty). No necesitas desplegar en un servidor externo.
- **Opiniones sobre convenciones:** hay una forma "recomendada" de hacer cada cosa (como Flask con Blueprints, pero más estricto).
- **Dependencias gestionadas:** solo añades `spring-boot-starter-*` y todo se resuelve.

🆚 Flask vs Spring Boot

Concepto	Flask (Python)	Spring Boot (Java)
Servidor	Werkzeug	Tomcat (embebido)
Rutas	`@app.route('/api')`	`@GetMapping("/api")`
Config	`app.config`	`application.properties`
Arranque	`app.run()`	`main()` + `SpringApplication.run()`

Concepto	Flask (Python)	Spring Boot (Java)
Dependencias	`pip install`	`pom.xml` (Maven) o `build.gradle`
JSON	`flask.jsonify()`	`@RestController` (automático)

📁 Estructura de un proyecto Spring Boot

```
mi-proyecto/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/ejemplo/miproyecto/
│   │   │       ├── MiProyectoApplication.java   ← punto de entrada
│   │   │       ├── controller/
│   │   │       │   └── TareaController.java     ← endpoints REST
│   │   │       ├── model/
│   │   │       │   └── Tarea.java               ← entidad
│   │   │       └── repository/
│   │   │           └── TareaRepository.java     ← acceso a datos
│   │   └── resources/
│   │       └── application.properties           ← configuración
│   └── test/                                    ← tests
├── pom.xml                                      ← dependencias (Maven)
└── mvnw / mvnw.cmd                              ← wrapper (sin instalar Maven)
```

🚀 Crear tu primer proyecto

Opción 1: Spring Initializr (recomendada)

1. Ve a start.spring.io
2. Selecciona: **Maven**, **Java 17**, **Spring Boot 3.x**
3. Añade dependencias: **Spring Web**, **Spring Data JPA**, **H2 Database**
4. Genera, descarga y abre en IntelliJ

Opción 2: Línea de comandos

```
curl https://start.spring.io/starter.zip?type=maven-project \
  -d dependencies=web,data-jpa,h2 -o mi-proyecto.zip
unzip mi-proyecto.zip
```

🏃 El punto de entrada

```
package com.ejemplo.miproyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MiProyectoApplication {
    public static void main(String[] args) {
        SpringApplication.run(MiProyectoApplication.class, args);
    }
}
```

`@SpringBootApplication` es un atajo que combina:

- `@Configuration`: marca la clase como fuente de configuración.
- `@EnableAutoConfiguration`: activa la autoconfiguración.
- `@ComponentScan`: busca componentes en el paquete y subpaquetes.

> 💡 Equivale a Flask con `app = Flask(__name__)` + `app.run()`, pero Spring lo hace todo por ti.

🧠 Mini-chequeo

1. ¿Qué es Spring Boot y por qué es tan popular?
2. ¿Qué hace `@SpringBootApplication`?
3. ¿Cuál es la diferencia entre `pom.xml` y `requirements.txt`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

- Spring Boot es el “Flask del Java”: crea APIs REST en minutos con autoconfiguración y servidor embebido.
- La estructura sigue convenciones: `controller/`, `model/`, `repository/`, `resources/`.
- `@SpringBootstrap` arranca todo: configura, escanea componentes y levanta el servidor.

🐛 Vocabulario rápido

Término	Idea general
Spring Boot	Framework Java para crear servicios web rápidamente
Maven	Gestor de dependencias (como pip)
`pom.xml`	Fichero de dependencias de Maven
`application.properties`	Fichero de configuración de Spring Boot
Starter	Paquete de dependencias preconfigurado

📚 Volver al índice de la unidad · **Anterior:** 01 · Java para desarrolladores Python · **Siguiente:** 03 · Inyección de dependencias

03 — Inyección de dependencias

IoC, @Autowired, Bean y por qué no creas tus propias instancias 🔧

> 🗺️ **Estás en:** 🟤 **UD 11 · Spring Boot** → 03 · Inyección de dependencias

📬 La idea en una frase

> La **inyección de dependencias** (DI) significa que **no creas tus propias dependencias**: se las pides a un contenedor (Spring) que las crea y te las entrega. Como un restaurante donde no cocinas: pides y te traen la comida.

🍽️ La analogía del restaurante

Imagina un restaurante:

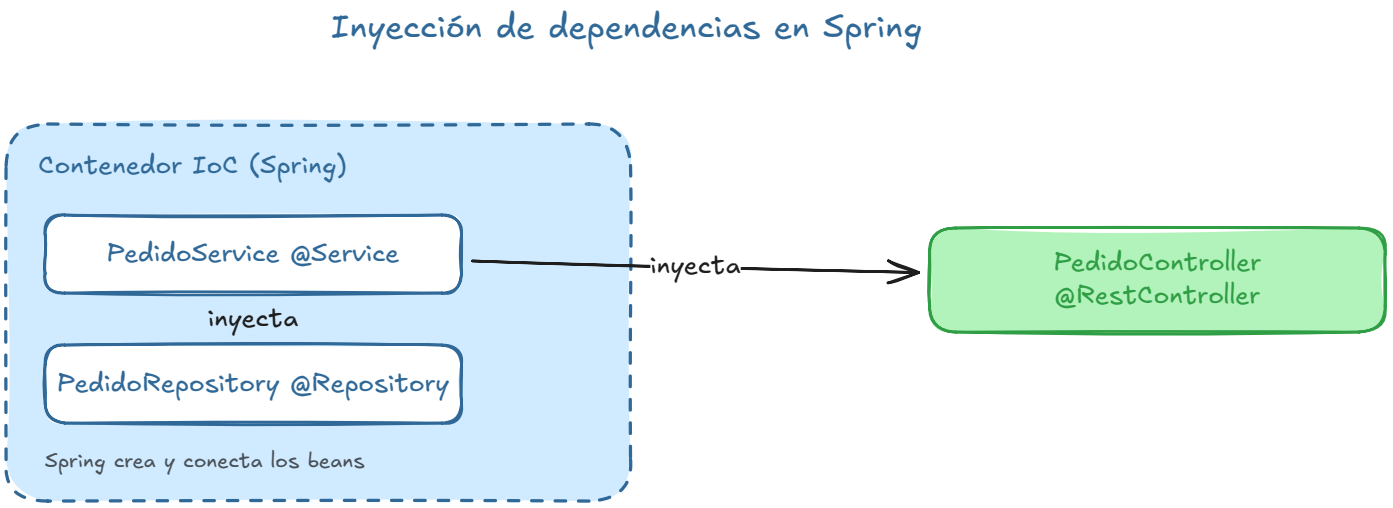
- **Sin DI (cocinas tú):** cada vez que necesitas una comida, vas al mercado, compras ingredientes, cocinas. Eres responsable de todo.
- **Con DI (restaurante):** pides la comida. El restaurante (contenedor) la prepara y te la trae. Tú solo consumes.

En código:

```
// SIN DI: tú creas la dependencia
public class PedidoService {
    private PedidoRepository repo = new PedidoRepository();  // acoplamiento directo
}

// CON DI: Spring te la inyecta
@Service
public class PedidoService {
    private final PedidoRepository repo;

    public PedidoService(PedidoRepository repo) {  // Spring decide qué implementación
usar
        this.repo = repo;
    }
}
```

🏭 El contenedor IoC

IoC (*Inversion of Control*) es el principio: el contenedor (Spring) controla la creación y lifecycle de los objetos (beans). Tú solo declaras qué necesitas.



tú no haces new: Spring llama a los constructores y pasa las dependencias

```
@Service
public class TareaService {
    private final TareaRepository repo;

    // Spring llama a este constructor y pasa el repository automáticamente
    public TareaService(TareaRepository repo) {
        this.repo = repo;
    }

    public List<Tarea> listarTodas() {
        return repo.findAll();
    }
}
```

> 💡 En Python equivaldría a que Flask creara automáticamente todas tus clases y las pasara como argumentos. No existe algo así de integrado en Python.

🏷️ Las anotaciones principales

Anotación	Qué hace
`@Component`	Marca una clase como “bean” gestionado por Spring
`@Service`	Como `@Component`, pero para servicios de negocio
`@Repository`	Como `@Component`, pero para acceso a datos
`@Controller`	Como `@Component`, pero para controladores web
`@RestController`	Como `@Controller`, pero con `@ResponseBody` automático
`@Autowired`	Inyecta una dependencia (por constructor, campo o setter)

🔄 El ciclo de vida

```
1. Spring arranca
2. Escanea el paquete (@ComponentScan)
3. Encuentra clases con @Service, @Repository, @Controller...
4. Crea instancias (beans) y las guarda en el contenedor
5. Cuando una clase necesita otra (constructor), Spring la inyecta
6. La app está lista: todos los beans están conectados
```

🧠 Mini-chequeo

1. ¿Qué problema resuelve la inyección de dependencias?
2. ¿Qué diferencia hay entre `@Component` y `@Service`?
3. ¿Por qué se usa el constructor para inyectar en lugar de `@Autowired` en un campo?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- La DI significa que Spring crea y entrega las dependencias: tú solo las recibes en el constructor.
- `@Service`, `@Repository` y `@Component` marcan clases como beans; Spring las gestiona.

- El contenedor IoC controla el lifecycle: creación, inyección y destrucción.

🐛 Vocabulario rápido

Término	Idea general
DI	Inyección de dependencias: Spring crea y entrega las dependencias
IoC	Inversión de control: el contenedor decide, no tú
Bean	Objeto gestionado por Spring
Contenedor IoC	El “restaurante” que crea y gestiona los beans
`@Autowired`	Anotación para inyectar (preferir constructor)

📚 Volver al índice de la unidad · **Anterior:** 02 · Introducción a Spring Boot · **Siguiente:** 04 · RESTController y APIs REST

04 — RESTController y APIs REST

Crear endpoints REST con Spring Boot: GET, POST, PUT, DELETE 🌐

> 🗺️ **Estás en:** ☕ **UD 11 · Spring Boot** → 04 · RESTController y APIs REST

📬 La idea en una frase

> Un **@RestController** expone endpoints HTTP que devuelven JSON. Spring Boot se encarga de convertir objetos Java a JSON automáticamente. Es como Flask, pero sin `jsonify()`.

🆚 Flask vs @RestController

```python
# Flask
from flask import Flask, jsonify
app = Flask(__name__)

@app.route('/api/tareas', methods=['GET'])
def listar_tareas():
    return jsonify([{"id": 1, "titulo": "Estudiar"}])
```

```java
// Spring Boot
@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    @GetMapping
    public List<Tarea> listarTareas() {
        return List.of(new Tarea(1L, "Estudiar"));
    }
}
```

Diferencia clave: en Spring, **no necesitas `jsonify()`**. El objeto Java se convierte a JSON automáticamente (gracias a Jackson).

🏷️ Las anotaciones de routing

Anotación	Método HTTP	Equivalente Flask
`@GetMapping`	GET	`@app.route(..., methods=['GET'])`
`@PostMapping`	POST	`@app.route(..., methods=['POST'])`
`@PutMapping`	PUT	`@app.route(..., methods=['PUT'])`
`@DeleteMapping`	DELETE	`@app.route(..., methods=['DELETE'])`
`@RequestMapping`	Cualquiera	`@app.route(...)`

📝 Ejemplo completo: API de tareas

```java
@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    private final TareaService service;

    public TareaController(TareaService service) {
        this.service = service;
    }

    // GET /api/tareas
    @GetMapping
    public List<Tarea> listar() {
        return service.listarTodas();
    }

    // GET /api/tareas/{id}
    @GetMapping("/{id}")
    public Tarea obtener(@PathVariable Long id) {
        return service.obtenerPorId(id);
    }

    // POST /api/tareas
    @PostMapping
    public Tarea crear(@RequestBody Tarea tarea) {
        return service.crear(tarea);
    }

    // PUT /api/tareas/{id}
    @PutMapping("/{id}")
    public Tarea actualizar(@PathVariable Long id, @RequestBody Tarea tarea) {
        return service.actualizar(id, tarea);
    }

    // DELETE /api/tareas/{id}
    @DeleteMapping("/{id}")
    public void eliminar(@PathVariable Long id) {
        service.eliminar(id);
    }
}
```

🔑 Las anotaciones clave

Anotación	Qué hace
`@RestController`	Marca la clase como controlador REST (devuelve JSON)
`@RequestMapping("/api/tareas")`	Prefijo de ruta para todos los endpoints
`@GetMapping`	Endpoint GET
`@PathVariable`	Extrae un parámetro de la URL (`/api/tareas/{id}`)
`@RequestBody`	Convierte el body JSON a un objeto Java

📦 El modelo

```
public class Tarea {
    private Long id;
    private String titulo;
    private boolean completada;

    // Constructor vacío (necesario para Jackson)
    public Tarea() {}

    public Tarea(Long id, String titulo, boolean completada) {
        this.id = id;
        this.titulo = titulo;
        this.completada = completada;
    }

    // Getters y Setters (Jackson los usa para serializar)
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public boolean isCompletada() { return completada; }
    public void setCompletada(boolean completada) { this.completada = completada; }
}
```

> 💡 Jackson convierte el objeto a JSON usando los **getters**. Si un campo no tiene getter, no aparece en el JSON.

🧠 Mini-chequeo

1. ¿Qué hace `@RestController` que no haga `@Controller`?
2. ¿Cómo se convierte un objeto Java a JSON en Spring Boot?
3. ¿Qué anotación extrae un parámetro de la URL?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- `@RestController` expone endpoints REST que devuelven JSON automáticamente.
- `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping` mapean métodos HTTP.
- `@PathVariable` extrae de la URL; `@RequestBody` convierte el body JSON a objeto Java.

🐛 Vocabulario rápido

Término	Idea general
`@RestController`	Controlador que devuelve JSON (no vistas)
`@RequestMapping`	Define la ruta base del controlador
`@PathVariable`	Parámetro de la URL (`/tareas/{id}`)
`@RequestBody`	Body JSON convertido a objeto Java
Jackson	Biblioteca que serializa Java ↔ JSON

📚 Volver al índice de la unidad · **Anterior:** 03 · Inyección de dependencias · **Siguiente:** 05 · JPA y bases de datos

05 — JPA y bases de datos

@Entity, Repository, CRUD con Hibernate y H2 🗄️

> 🗺️ **Estás en:** ☕ **UD 11 · Spring Boot** → 05 · JPA y bases de datos

📬 La idea en una frase

> **JPA** (*Java Persistence API*) es el estándar Java para mapear objetos a tablas de base de datos. **Hibernate** es su implementación más usada. Con Spring Data JPA, crear un repositorio CRUD es una interfaz de una línea.

🆚 SQL vs JPA

```
# Python + SQLite
import sqlite3
conn = sqlite3.connect('tareas.db')
cursor = conn.execute("INSERT INTO tareas (titulo) VALUES ('Estudiar')")
conn.commit()
```

```
// Java + JPA
@Entity
public class Tarea {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    // ...
}

// Y para guardar:
tareaRepository.save(new Tarea("Estudiar"));
```

Diferencia clave: en JPA **trabajas con objetos**, no con SQL. El framework traduce las llamadas a SQL automáticamente.

🏷️ Las anotaciones de JPA

Anotación	Qué hace
`@Entity`	Marca una clase como tabla de BD
`@Id`	Marca el campo como clave primaria
`@GeneratedValue`	Genera el ID automáticamente
`@Column`	Configura el nombre de la columna
`@ManyToOne` / `@OneToMany`	Relaciones entre tablas

📦 El repositorio

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface TareaRepository extends JpaRepository<Tarea, Long> {
    // ¡No necesitas escribir nada! Spring genera las implementaciones
}
```

Métodos que obtienes gratis:

Método	Qué hace
`findAll()`	SELECT *
`findById(id)`	SELECT WHERE id = ?
`save(entity)`	INSERT o UPDATE
`deleteById(id)`	DELETE WHERE id = ?
`count()`	SELECT COUNT(*)

🔍 Consultas personalizadas

```java
public interface TareaRepository extends JpaRepository<Tarea, Long> {

    // Spring genera la consulta a partir del nombre del método
    List<Tarea> findByCompletada(boolean completada);
    List<Tarea> findByTituloContaining(String texto);
    Optional<Tarea> findByTitulo(String titulo);
}
```

Equivalente SQL:

```sql
SELECT * FROM tareas WHERE completada = true;
SELECT * FROM tareas WHERE titulo LIKE '%texto%';
SELECT * FROM tareas WHERE titulo = 'titulo';
```

> 💡 Solo escribes el SQL cuando necesitas algo que los métodos derivados no pueden expresar. Para eso usas `@Query`.

⚙️ Configuración (`application.properties`)

```properties
# H2 (base de datos en memoria para desarrollo)
spring.datasource.url=jdbc:h2:mem:tareasdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Consola H2 (accesible en /h2-console)
spring.h2.console.enabled=true
```

Propiedad	Qué hace
`ddl-auto=update`	Actualiza el esquema automáticamente
`show-sql=true`	Muestra el SQL generado en consola
`h2.console.enabled`	Activa la consola web de H2

🧠 Mini-chequeo

1. ¿Qué es JPA y qué hace Hibernate?
2. ¿Por qué no necesitas escribir SQL para un CRUD básico?
3. ¿Qué hace `JpaRepository<T, ID>` por ti?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

- JPA mapea objetos Java a tablas de BD con `@Entity` y `@Id`.
- `JpaRepository` genera CRUD automáticamente: solo declaras la interfaz.
- Las consultas personalizadas se crean por nombre de método (`findByTitulo`).

🐛 Vocabulario rápido

Término	Idea general
JPA	Estándar Java para persistencia de objetos
Hibernate	Implementación de JPA más usada
`@Entity`	Marca una clase como tabla de BD
`@Repository`	Interfaz de acceso a datos
`ddl-auto`	Política de actualización del esquema

📚 Volver al índice de la unidad · **Anterior:** 04 · RESTController y APIs REST · **Siguiente:** 06 · Ejemplo completo

06 — Ejemplo completo: API de tareas

Crea una API REST completa con Spring Boot, JPA y H2 🏗️

> 🗺️ **Estás en:** 🟤 **UD 11 · Spring Boot** → 06 · Ejemplo completo

---

📬 La idea en una frase

> Vamos a montar una **API de tareas** completa: modelo, repositorio, servicio y controlador. CRUD completo con H2 en memoria. El "To-Do App" del mundo Spring Boot.

🏗️ El proyecto completo

Paso 1: Modelo (`Tarea.java`)

```java
package com.ejemplo.tareas.model;

import jakarta.persistence.*;

@Entity
@Table(name = "tareas")
public class Tarea {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String titulo;

    private boolean completada = false;

    // Constructor vacío (JPA lo necesita)
    public Tarea() {}

    public Tarea(String titulo) {
        this.titulo = titulo;
    }

    // Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public boolean isCompletada() { return completada; }
    public void setCompletada(boolean completada) { this.completada = completada; }
}
```

Paso 2: Repositorio (`TareaRepository.java`)

```java
package com.ejemplo.tareas.repository;

import com.ejemplo.tareas.model.Tarea;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface TareaRepository extends JpaRepository<Tarea, Long> {
    List<Tarea> findByCompletada(boolean completada);
}
```

Paso 3: Servicio (`TareaService.java`)

```java
package com.ejemplo.tareas.service;

import com.ejemplo.tareas.model.Tarea;
import com.ejemplo.tareas.repository.TareaRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class TareaService {

    private final TareaRepository repo;

    public TareaService(TareaRepository repo) {
        this.repo = repo;
    }

    public List<Tarea> listarTodas() {
        return repo.findAll();
    }

    public Tarea obtenerPorId(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new RuntimeException("Tarea no encontrada: " + id));
    }

    public Tarea crear(Tarea tarea) {
        return repo.save(tarea);
    }

    public Tarea actualizar(Long id, Tarea datos) {
        Tarea tarea = obtenerPorId(id);
        tarea.setTitulo(datos.getTitulo());
        tarea.setCompletada(datos.isCompletada());
        return repo.save(tarea);
    }

    public void eliminar(Long id) {
        repo.deleteById(id);
    }
}
```

Paso 4: Controlador (`TareaController.java`)

```java
package com.ejemplo.tareas.controller;

import com.ejemplo.tareas.model.Tarea;
import com.ejemplo.tareas.service.TareaService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    private final TareaService service;

    public TareaController(TareaService service) {
        this.service = service;
    }

    @GetMapping
    public List<Tarea> listar() {
        return service.listarTodas();
    }

    @GetMapping("/{id}")
    public Tarea obtener(@PathVariable Long id) {
        return service.obtenerPorId(id);
    }

    @PostMapping
    public Tarea crear(@RequestBody Tarea tarea) {
        return service.crear(tarea);
    }

    @PutMapping("/{id}")
    public Tarea actualizar(@PathVariable Long id, @RequestBody Tarea tarea) {
        return service.actualizar(id, tarea);
    }

    @DeleteMapping("/{id}")
    public void eliminar(@PathVariable Long id) {
        service.eliminar(id);
    }
}
```

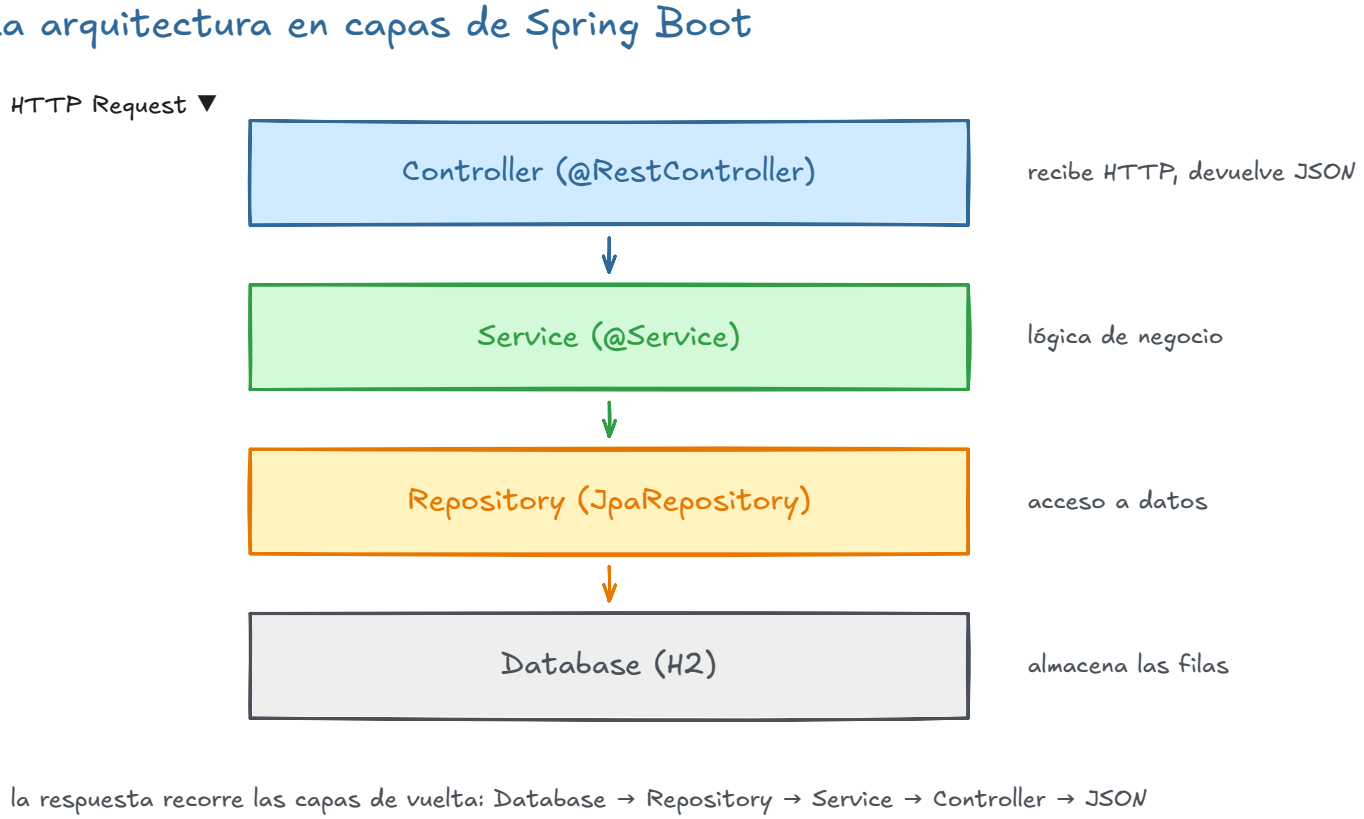
Paso 5: Punto de entrada (`TareasApplication.java`)

```java
package com.ejemplo.tareas;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TareasApplication {
    public static void main(String[] args) {
        SpringApplication.run(TareasApplication.class, args);
    }
}
```

🧪 Probar la API

```
# Crear tarea
curl -X POST http://localhost:8080/api/tareas \
  -H "Content-Type: application/json" \
  -d '{"titulo": "Estudiar Spring Boot"}'

# Listar tareas
curl http://localhost:8080/api/tareas

# Obtener tarea por ID
curl http://localhost:8080/api/tareas/1

# Actualizar tarea
curl -X PUT http://localhost:8080/api/tareas/1 \
  -H "Content-Type: application/json" \
  -d '{"titulo": "Estudiar Spring Boot", "completada": true}'

# Eliminar tarea
curl -X DELETE http://localhost:8080/api/tareas/1
```

La arquitectura en capas



Cada capa solo depende de la inferior: Controller → Service → Repository → Database. Eso es **separación de capas** (como MVC en Python).

Mini-chequeo

1. ¿Por qué el modelo tiene un constructor vacío?
2. ¿Qué pasaría si eliminas `@Service` de `TareaService`?
3. ¿Por qué el controller depende del service y no del repository directamente?

> **Respuestas**

Resumen en 3 frases

- Una API Spring Boot tiene 4 capas: Controller → Service → Repository → Model.
- El `@RestController` recibe HTTP y devuelve JSON; el `@Service` tiene la lógica; el `Repository` accede a la BD.
- Con JPA y H2, el CRUD se genera casi automáticamente.

🐛 Vocabulario rápido

Término	Idea general
Capas	Separación: Controller → Service → Repository → Model
`@Entity`	Clase que se mapea a una tabla
`@Service`	Lógica de negocio
`JpaRepository`	CRUD automático con JPA

📚 Volver al índice de la unidad · **Anterior:** 05 · JPA y bases de datos · **Siguiente:** 07 · Cierre

07 — Cierre: Python vs Java/Spring Boot

El final del viaje: comparativa final, recursos y cierre 🏁

> 🗺️ **Estás en:** ☕ **UD 11 · Spring Boot** → 07 · Cierre

Has recorrido el puente entre Python y Java/Spring Boot y con él el viaje completo de PSP. Este cierre consolida lo aprendido con una comparativa final, los recursos para seguir y las preguntas que te harán en una entrevista.

⭐ Sé el framework

> *Eres Flask, el micro-framework de Python. Te sientas junto a Spring Boot en un bar a discutir quién es más productivo.*

Flask: — Soy ligero, conciso. Con 10 líneas tengo un API funcionando. No necesito clases, anotaciones ni compilación.

Spring Boot: — Yo tengo autoconfiguración, DI, JPA, seguridad y servidores embebidos. Tú necesitas extensiones para todo eso.

Flask: — Pero para aprender soy más directo. Sin ceremonia, sin boilerplate.

Spring Boot: — En producción, mi estructura de capas y tipado estático previenen bugs que tú descubres en runtime.

Flask: — Al final, cada uno con su público.

Spring Boot: — Exacto. Tú para prototipos y proyectos pequeños; yo para equipos grandes y sistemas empresariales.

> **Moraleja:** Python es velocidad de desarrollo; Java/Spring es escalabilidad y mantenibilidad. Ambos tienen su sitio.

⚖️ La comparativa final

Aspecto	Python / Flask	Java / Spring Boot
Velocidad de desarrollo	⚡ Rápido	🐢 Más lento
Tipado	Dinámico	Estático
Rendimiento	Bueno	Excelente
Escalabilidad	Media	Alta
Ecosistema web	Flask, Django	Spring Boot
Testing	pytest	JUnit + Mockito
Despliegue	`python app.py`	`java -jar app.jar`
Uso típico	Startups, prototipos, data	Empresas, banca, microservicios

🔥 Fireside Chat: Python vs Java

> *Dos lenguajes se sientan junto a la chimenea a discutir cuál es mejor para servicios web.*

Python: — Soy rápido de aprender, rápido de escribir, rápido de desplegar. Los startups me eligen a mí.

Java: — Yo tengo tipos, compilación, herramientas de IDE increíbles y 20 años de ecosistema empresarial.

Python: — Mi GIL me limita en concurrencia, es cierto.

Java: — Y mi verbosidad me hace escribir más líneas para lo mismo.

Python: — Al final, la mejor herramienta es la que resuelve el problema.

Java: — En eso coincidimos.

> **Moraleja:** No compiten: son herramientas distintas para contextos distintos. Un buen programador conoce ambas.

🕵️ ¿Quién soy?

1. Soy el framework Java que arranca servidor embebido y autoconfigura todo.
2. Soy el principio donde Spring crea y entrega las dependencias.
3. Soy la anotación que marca un controlador REST.
4. Soy la interfaz que genera CRUD automáticamente.
5. Soy la base de datos en memoria que usa Spring Boot para desarrollo.
6. Soy la biblioteca que convierte objetos Java a JSON.

> 🔄 **Respuestas**

📚 Recursos para seguir

Oficiales

- spring.io/guides — Guías oficiales de Spring
- .baeldung.com — Tutoriales Java/Spring excelentes
- docs.oracle.com — Documentación oficial de Java

Práctica

- Spring Boot Tutorial (YouTube) — Canales recomendados
- Java by Comparison — Libro: Python → Java

🧠 Atrévete a pensar

1. ¿En qué situaciones elegirías Spring Boot sobre Flask para un nuevo proyecto?
2. ¿Qué ventajas e inconvenientes tiene la inyección de dependencias?
3. ¿Cómo cambiarías una API Flask existente para migrarla a Spring Boot?
4. ¿Por qué Java sigue siendo el lenguaje empresarial a pesar de ser más verboso?

> 💡 **Soluciones**

🎬 Poscréditos

> *El servidor secuencial hacía esperar a todos en la cola de la ventanilla única.*

> *Llegó el hilo por cliente y cada conexión tuvo su propia ventanilla.*
>
> *El ThreadPool puso límites al entusiasmo. El Lock protegió el contador. Los clientes responden a la vez.*
>
> *Y entonces llegó asyncio: un solo camarero, miles de mesas, ninguna espera quieta.*
>
> *El viaje cruzó el océano: de Python a Java, de Flask a Spring Boot. De un guion rápido a un framework con capas.*
>
> *El viaje ha recorrido once hitos: 🐍 Python, 🚀 Procesos, 🔀 Hilos, 🔒 Sincronización, 🔌 Sockets, 🏗️ Servidores, 🌐 API REST, 💉 APIs comerciales, 🔐 Seguridad, ⏱️ Alta disponibilidad y ☕ Spring Boot.*

Y con esto… ¡el viaje ha terminado! 🏁

✅ Criterios de la unidad

Criterio	Cubierto
Entender la sintaxis básica de Java	✅ Punto 01
Explicar qué es Spring Boot	✅ Punto 02
Comprender la inyección de dependencias	✅ Punto 03
Crear un API REST con @RestController	✅ Punto 04
Conectar a BD con JPA	✅ Punto 05
Montar un proyecto completo	✅ Punto 06

📚 Volver al índice de la unidad · **Anterior:** 06 · Ejemplo completo

Boletín UD 1 — Inicial

📝 Boletín UD 1 — Inicial

> Ejercicios básicos para afianzar los conceptos de comentarios, tipos de datos, variables, colecciones, control de flujo, funciones y clases de la unidad UD 1.

---

1. Hola mundo con f-string

Crea un programa que pida tu nombre con `input()` y salude usando un f-string:

```
¿Cómo te llamas? Ana
Hola, Ana. ¡Bienvenido a Python!
```

2. Calculadora básica

Crea un programa que pida dos números y muestre su suma, resta, multiplicación y división. Muestra el resultado de cada operación con un f-string.

Pista: usa `float(input("Dame un número: "))` para que acepte decimales.

3. Par o impar

Pide un número entero al usuario y dice si es par o impar usando el operador `%`.

```
Dame un número: 7
7 es impar
```

4. Tabla de multiplicar

Pide un número y muestra su tabla de multiplicar del 1 al 10.

```
Dame un número: 5
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Pista: usa un bucle `for` con `range(1, 11)`.

5. Invertir una cadena

Crea un programa que pida una cadena de texto y la muestre al revés.

```
Dame una cadena: hola
La cadena invertida es: aloh
```

Pista: en Python, `"cadena"[::-1]` invierte una cadena.

6. Lista de la compra

Crea un programa con un menú que permita:

1. Añadir un producto a una lista
2. Ver todos los productos
3. Eliminar un producto
4. Salir

Usa un bucle `while True` y `input()` para el menú.

7. Diccionario de alumnos

Crea un diccionario donde las claves sean nombres de alumnos y los valores sean sus notas. El programa debe:

1. Añadir un alumno con su nota
2. Buscar la nota de un alumno
3. Calcular la nota media de todos los alumnos
4. Mostrar quién tiene la nota más alta

Pista: usa `max(diccionario.values())` para la nota más alta y `sum(values()) / len(values())` para la media.

8. Clase Coche

Crea una clase `Coche` con atributos `marca`, `modelo` y `año`. Añade un método `descripcion()` que devuelva un string con todos los datos y un método `es_antiguo()` que devuelva `True` si el coche tiene más de 10 años.

```
coche = Coche("Seat", "León", 2015)
print(coche.descripcion())   # Seat León (2015)
print(coche.es_antiguo())    # True
```

Pista: para calcular el año actual, puedes usar `from datetime import datetime` y `datetime.now().year`.

Boletín UD 1 — Inicial (Resuelto)
✅ Boletín UD 1 — Inicial (Resuelto)

1. Hola mundo con f-string

```
nombre = input("¿Cómo te llamas? ")
print(f"Hola, {nombre}. ¡Bienvenido a Python!")
```

2. Calculadora básica

```
a = float(input("Dame un número: "))
b = float(input("Dame otro número: "))

print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} x {b} = {a * b}")

if b != 0:
    print(f"{a} / {b} = {a / b}")
else:
    print("No se puede dividir por cero")
```

3. Par o impar

```
numero = int(input("Dame un número: "))

if numero % 2 == 0:
    print(f"{numero} es par")
else:
    print(f"{numero} es impar")
```

4. Tabla de multiplicar

```python
numero = int(input("Dame un número: "))

for i in range(1, 11):
    print(f"{numero} x {i} = {numero * i}")
```

5. Invertir una cadena

```python
cadena = input("Dame una cadena: ")
invertida = cadena[::-1]
print(f"La cadena invertida es: {invertida}")
```

6. Lista de la compra

```python
compra = []

while True:
    print("\n1. Añadir producto")
    print("2. Ver productos")
    print("3. Eliminar producto")
    print("4. Salir")
    opcion = input("Elige una opción: ")

    if opcion == "1":
        producto = input("Nombre del producto: ")
        compra.append(producto)
        print(f"'{producto}' añadido.")
    elif opcion == "2":
        if compra:
            for i, p in enumerate(compra, 1):
                print(f"  {i}. {p}")
        else:
            print("La lista está vacía.")
    elif opcion == "3":
        producto = input("Nombre del producto a eliminar: ")
        if producto in compra:
            compra.remove(producto)
            print(f"'{producto}' eliminado.")
        else:
            print("Producto no encontrado.")
    elif opcion == "4":
        break
```

7. Diccionario de alumnos

```python
alumnos = {}

while True:
    print("\n1. Añadir alumno")
    print("2. Buscar nota")
    print("3. Nota media")
    print("4. Nota más alta")
    print("5. Salir")
    opcion = input("Elige una opción: ")

    if opcion == "1":
        nombre = input("Nombre del alumno: ")
        nota = float(input("Nota: "))
        alumnos[nombre] = nota
        print(f"'{nombre}' añadido con nota {nota}.")
    elif opcion == "2":
        nombre = input("Nombre del alumno: ")
        if nombre in alumnos:
            print(f"Nota de {nombre}: {alumnos[nombre]}")
        else:
            print("Alumno no encontrado.")
    elif opcion == "3":
        if alumnos:
            media = sum(alumnos.values()) / len(alumnos)
            print(f"Nota media: {media:.2f}")
        else:
            print("No hay alumnos.")
    elif opcion == "4":
        if alumnos:
            max_nota = max(alumnos.values())
            mejor = [n for n, nota in alumnos.items() if nota == max_nota]
            print(f"Nota más alta: {max_nota} ({', '.join(mejor)})")
        else:
            print("No hay alumnos.")
    elif opcion == "5":
        break
```

8. Clase Coche

```python
from datetime import datetime


class Coche:
    def __init__(self, marca, modelo, año):
        self.marca = marca
        self.modelo = modelo
        self.año = año

    def descripcion(self):
        return f"{self.marca} {self.modelo} ({self.año})"

    def es_antiguo(self):
        año_actual = datetime.now().year
        return (año_actual - self.año) > 10


# Pruebas
coche = Coche("Seat", "León", 2015)
print(coche.descripcion())   # Seat León (2015)
print(coche.es_antiguo())    # True

coche2 = Coche("Tesla", "Model 3", 2023)
print(coche2.descripcion())  # Tesla Model 3 (2023)
print(coche2.es_antiguo())   # False
```

Boletín UD 1 — Avanzado
💪 Boletín UD 1 — Avanzado

> Ejercicios que requieren aplicar los conceptos de Python 3 de forma más profunda, con programas completos y casos reales.

---

1. Comprensiones de lista

Dada una lista de números, crea en **una sola línea** con comprensión de lista:

- Una lista con los cuadrados de los números pares.
- Una lista con las longitudes de cada palabra de una frase.
- Una lista con los caracteres en mayúsculas de una cadena.

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
frase = "aprende python en la fp"
cadena = "hola mundo"
```

Pista: para las longitudes: `[len(palabra) for palabra in frase.split()]`.

2. Manejo de archivos

Crea un programa que:

1. Cree un archivo `notas.txt` con 5 notas (una por línea).
2. Lea el archivo y calcule la nota media.
3. Añada al final del archivo la media calculada.
4. Muestre todas las líneas del archivo.

```
notas = [7, 8, 6, 9, 8]
```

Pista: usa `with open("notas.txt", "w") as f:` para escribir y `"r"` para leer.

3. Lambda con reduce

Usando `functools.reduce` y una función lambda, crea una función que multiplique todos los elementos de una lista entre sí.

```
from functools import reduce
numeros = [2, 3, 4, 5]
# Resultado: 120 (2 * 3 * 4 * 5)
```

Pista: `reduce(lambda a, b: a * b, numeros)`.

4. Clase Cuenta Bancaria

Crea una clase `CuentaBancaria` con:

- Atributo `titular` y `saldo` (por defecto 0).
- Método `ingresar(cantidad)`: suma al saldo (no puede ser negativa).
- Método `retirar(cantidad)`: resta al saldo si hay suficiente; si no, muestra un aviso.
- Método `__str__`: devuelve `"Titular: X — Saldo: Y €"`.
- Método `__eq__`: dos cuentas son iguales si tienen el mismo titular y saldo.

```
c1 = CuentaBancaria("Ana", 1000)
c1.ingresar(500)
c1.retirar(200)
print(c1)   # Titular: Ana — Saldo: 1300 €
```

Pista: el método `__eq__` se llama automáticamente al usar `==`.

5. Decorador de rendimiento

Crea un decorador `medir_tiempo` que mida cuántos segundos tarda una función en ejecutarse e imprima el resultado.

```python
import time

@medir_tiempo
def funcion_lenta():
    time.sleep(1)
    print("¡Acabó!")

funcion_lenta()
# Imprime: "funcion_lenta tardó 1.0012 segundos"
```

Pista: usa `time.time()` antes y después de llamar a la función, y `@wraps` del módulo `functools`.

6. Generador de Fibonacci

Crea un generador `fibonacci(n)` que produzca los primeros `n` números de la sucesión de Fibonacci.

```python
for num in fibonacci(10):
    print(num, end=" ")
# Salida: 0 1 1 2 3 5 8 13 21 34
```

Pista: la sucesión empieza con 0 y 1, y cada número siguiente es la suma de los dos anteriores. Guarda los dos últimos valores en variables.

7. Gestor de errores personalizado

Crea una excepción personalizada `SaldoInsuficienteError` y úsala en la clase `CuentaBancaria` del ejercicio 4. Cuando el usuario intente retirar más de lo que tiene, debe lanzar esa excepción en lugar de imprimir un aviso.

```python
c = CuentaBancaria("Bob", 100)
try:
    c.retirar(200)
except SaldoInsuficienteError as e:
    print(e)   # "Saldo insuficiente: tienes 100 €, intentas retirar 200 €"
```

Pista: define la clase como `class SaldoInsuficienteError(Exception): pass` y en `retirar` usa `raise SaldoInsuficienteError(...)`.

8. Mini inventario con set operations

Crea dos conjuntos de productos: `tienda_a` y `tienda_b`. El programa debe:

1. Calcular los productos que están en ambas tiendas (intersección).
2. Calcular los productos exclusivos de cada tienda (diferencia).
3. Calcular todos los productos sin duplicados (unión).
4. Calcular los productos que están en una tienda pero no en la otra (diferencia simétrica).

```
tienda_a = {"manzana", "plátano", "naranja", "leche"}
tienda_b = {"plátano", "leche", "pan", "huevos"}
```

Pista: la diferencia simétrica se calcula con `^` o con `symmetric_difference()`.

9. Parser de CSV sencillo

Crea una función `leer_csv(nombre_fichero)` que lea un archivo CSV sin usar la biblioteca `csv`. La función debe devolver una lista de diccionarios, donde cada diccionario representa una fila y las claves son los nombres de las columnas del encabezado.

```
nombre,edad,ciudad
Ana,25,Madrid
Bob,30,Barcelona
Clara,22,Valencia
```

```
resultado = leer_csv("datos.csv")
print(resultado[0])   # {'nombre': 'Ana', 'edad': '25', 'ciudad': 'Madrid'}
```

Pista: lee la primera línea para obtener las claves, luego recorre el resto dividiendo cada línea por `","`.

Boletín UD 1 — Avanzado (Resuelto)
✅ Boletín UD 1 — Avanzado (Resuelto)

1. Comprensiones de lista

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
frase = "aprende python en la fp"
cadena = "hola mundo"

# Cuadrados de los pares
cuadrados_pares = [x ** 2 for x in numeros if x % 2 == 0]
print(cuadrados_pares)   # [4, 16, 36, 64, 100]

# Longitudes de cada palabra
longitudes = [len(palabra) for palabra in frase.split()]
print(longitudes)        # [8, 6, 2, 2, 2]

# Caracteres en mayúsculas
mayusculas = [c.upper() for c in cadena]
print(mayusculas)        # ['H', 'O', 'L', 'A', ' ', 'M', 'U', 'N', 'D', 'O']
```

2. Manejo de archivos

```python
notas = [7, 8, 6, 9, 8]

# 1. Crear el archivo
with open("notas.txt", "w") as f:
    for nota in notas:
        f.write(f"{nota}\n")

# 2. Leer y calcular media
with open("notas.txt", "r") as f:
    lineas = f.readlines()
    notas_leidas = [float(linea.strip()) for linea in lineas]
    media = sum(notas_leidas) / len(notas_leidas)

# 3. Añadir la media al final
with open("notas.txt", "a") as f:
    f.write(f"{media:.2f}\n")

# 4. Mostrar todas las líneas
with open("notas.txt", "r") as f:
    print(f.read())
```

3. Lambda con reduce

```python
from functools import reduce

numeros = [2, 3, 4, 5]
producto = reduce(lambda a, b: a * b, numeros)
print(producto)   # 120
```

4. Clase Cuenta Bancaria

```python
class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def ingresar(self, cantidad):
        if cantidad > 0:
            self.saldo += cantidad
        else:
            print("La cantidad a ingresar debe ser positiva")

    def retirar(self, cantidad):
        if cantidad > 0:
            self.saldo -= cantidad
        else:
            print("La cantidad a retirar debe ser positiva")

    def __str__(self):
        return f"Titular: {self.titular} — Saldo: {self.saldo} €"

    def __eq__(self, other):
        return self.titular == other.titular and self.saldo == other.saldo


# Pruebas
c1 = CuentaBancaria("Ana", 1000)
c1.ingresar(500)
c1.retirar(200)
print(c1)   # Titular: Ana — Saldo: 1300 €

c2 = CuentaBancaria("Ana", 1300)
print(c1 == c2)   # True
```

5. Decorador de rendimiento

```python
import time
from functools import wraps


def medir_tiempo(funcion):
    @wraps(funcion)
    def wrapper(*args, **kwargs):
        inicio = time.time()
        resultado = funcion(*args, **kwargs)
        fin = time.time()
        print(f"{funcion.__name__} tardó {fin - inicio:.4f} segundos")
        return resultado
    return wrapper


@medir_tiempo
def funcion_lenta():
    time.sleep(1)
    print("¡Acabó!")


funcion_lenta()
# Imprime:
# ¡Acabó!
# funcion_lenta tardó 1.0012 segundos
```

6. Generador de Fibonacci

```python
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b


for num in fibonacci(10):
    print(num, end=" ")
# Salida: 0 1 1 2 3 5 8 13 21 34
```

7. Gestor de errores personalizado

```
class SaldoInsuficienteError(Exception):
    pass


class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def ingresar(self, cantidad):
        if cantidad > 0:
            self.saldo += cantidad

    def retirar(self, cantidad):
        if cantidad > self.saldo:
            raise SaldoInsuficienteError(
                f"Saldo insuficiente: tienes {self.saldo} €, "
                f"intentas retirar {cantidad} €"
            )
        self.saldo -= cantidad


# Prueba
c = CuentaBancaria("Bob", 100)
try:
    c.retirar(200)
except SaldoInsuficienteError as e:
    print(e)   # Saldo insuficiente: tienes 100 €, intentas retirar 200 €
```

8. Mini inventario con set operations

```python
tienda_a = {"manzana", "plátano", "naranja", "leche"}
tienda_b = {"plátano", "leche", "pan", "huevos"}

# Productos en ambas tiendas (intersección)
ambas = tienda_a & tienda_b
print(f"En ambas: {ambas}")   # {'plátano', 'leche'}

# Exclusivos de cada tienda (diferencia)
solo_a = tienda_a - tienda_b
solo_b = tienda_b - tienda_a
print(f"Solo en A: {solo_a}")   # {'manzana', 'naranja'}
print(f"Solo en B: {solo_b}")   # {'pan', 'huevos'}

# Todos los productos (unión)
todos = tienda_a | tienda_b
print(f"Todos: {todos}")

# En una pero no en la otra (diferencia simétrica)
exclusivos = tienda_a ^ tienda_b
print(f"Exclusivos: {exclusivos}")   # {'manzana', 'naranja', 'pan', 'huevos'}
```

9. Parser de CSV sencillo

```python
def leer_csv(nombre_fichero):
    with open(nombre_fichero, "r") as f:
        lineas = f.readlines()

    if not lineas:
        return []

    claves = lineas[0].strip().split(",")
    resultado = []

    for linea in lineas[1:]:
        valores = linea.strip().split(",")
        fila = dict(zip(claves, valores))
        resultado.append(fila)

    return resultado


# Prueba
resultado = leer_csv("datos.csv")
for fila in resultado:
    print(fila)
# {'nombre': 'Ana', 'edad': '25', 'ciudad': 'Madrid'}
# {'nombre': 'Bob', 'edad': '30', 'ciudad': 'Barcelona'}
# {'nombre': 'Clara', 'edad': '22', 'ciudad': 'Valencia'}
```

Boletín UD 2 — Inicial

📝 Boletín UD 2 — Inicial

> Ejercicios básicos para afianzar los conceptos de procesos, `subprocess.run`, `subprocess.Popen` y estados de la unidad UD 2.

---

1. Listar el directorio actual

Crea un programa que use `subprocess.run` para ejecutar el comando `dir` (Windows) o `ls` (Linux) y muestre la salida completa por pantalla.

2. Saber el nombre del equipo

Usa `subprocess.run` con el comando `hostname` para capturar y mostrar el nombre de tu máquina.

3. Abrir la calculadora

Usa `subprocess.Popen` para abrir la calculadora (`calc.exe`). El programa debe mostrar el PID, esperar 2 segundos y cerrarla con `terminate()`.

4. Mi primer PID

Crea un programa que imprima su propio PID con `os.getpid()`.

5. Versión de Python

Usa `subprocess.run` para ejecutar `python --version` con `capture_output=True` y `text=True`, y muestra la salida limpia (sin saltos de línea).

Pista: `resultado.stdout.strip()` te deja la salida limpia.

6. Abrir el bloc de notas y esperar

Usa `subprocess.Popen` para abrir `notepad.exe`, muestra su PID, espera 3 segundos y ciérralo con `terminate()`.

7. Ordena el ciclo de vida

Ordena los siguientes estados de un proceso en el orden correcto de su ciclo de vida: `EJECUCIÓN`, `NUEVO`, `TERMINADO`, `BLOQUEADO`, `LISTO`.

Pista: el proceso nace, luego espera su turno de CPU, se ejecuta, puede quedarse esperando un recurso y finalmente muere. Repasa el punto 2.

8. Ping con timeout

Lanza `ping 8.8.8.8 -n 3` (Windows) o `ping -c 3 8.8.8.8` (Linux) con `timeout=3` y captura la excepción si se excede el tiempo.

Pista: envuelve `subprocess.run` en un `try/except` y captura `subprocess.TimeoutExpired`. Si tu conexión es rápida, prueba con `timeout=1`.

Boletín UD 2 — Inicial (Resuelto)
✅ Boletín UD 2 — Inicial (Resuelto)

1. Listar el directorio actual

```
import subprocess

resultado = subprocess.run(["dir"], capture_output=True, text=True, shell=True)
print(resultado.stdout)
```

En Linux: `subprocess.run(["ls"])`. En Windows, `dir` es un comando interno del shell, por eso `shell=True` (punto 7).

2. Saber el nombre del equipo

```
import subprocess

resultado = subprocess.run(["hostname"], capture_output=True, text=True)
print(f"El nombre del equipo es {resultado.stdout.strip()}")
```

`capture_output=True` captura la salida de `hostname` y `text=True` la devuelve como string.

3. Abrir la calculadora

```
import subprocess, time

calc = subprocess.Popen(["calc.exe"])
print(f"Calculadora lanzada con PID {calc.pid}")
time.sleep(2)
calc.terminate()
print("Calculadora cerrada")
```

`Popen` no espera: el programa muestra el PID, espera 2 segundos y cierra con `terminate()`.

4. Mi primer PID

```
import os

print(f"Mi PID es {os.getpid()}")
```

`os.getpid()` devuelve el identificador único del proceso actual: el **PID** (punto 1).

5. Versión de Python

```
import subprocess

resultado = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(resultado.stdout.strip())
```

`capture_output=True` captura la salida. `text=True` la devuelve como string. `.strip()` elimina el salto de línea final.

6. Abrir el bloc de notas y esperar

```
import subprocess, time

notepad = subprocess.Popen(["notepad.exe"])
print(f"PID: {notepad.pid}")
time.sleep(3)
notepad.terminate()
```

`Popen` no espera. `terminate()` envía señal de cierre.

7. Ordena el ciclo de vida

NUEVO → LISTO → EJECUCIÓN → BLOQUEADO → TERMINADO

El orden correcto: el proceso **nace** (NUEVO), espera su turno de CPU (**LISTO**), la CPU lo ejecuta (**EJECUCIÓN**), puede quedarse esperando un recurso (**BLOQUEADO**) y finalmente **termina**. Repasa el punto 2.

8. Ping con timeout

```
import subprocess

try:
    resultado = subprocess.run(["ping", "8.8.8.8", "-n", "3"],
                               capture_output=True, text=True, timeout=3)
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("El ping tardó demasiado")
```

En Linux usa `["ping", "-c", "3", "8.8.8.8"]`. `timeout` lanza `TimeoutExpired` si excede.

Boletín UD 2 — Avanzado

💪 Boletín UD 2 — Avanzado

> Ejercicios que requieren aplicar los conceptos de procesos, `subprocess` y comunicación entre procesos de forma más profunda, con programas completos. Los tres últimos llevan `multiprocessing` (`Process`, `Manager` y `Lock`).

---

1. Filtrar ipconfig

Ejecuta `ipconfig` con `subprocess.run`, captura toda la salida y muestra solo las líneas que contengan “IPv4”.

Pista: divide el stdout con `split("\n")` y filtra cada línea con el operador `in` para buscar “IPv4”.

2. Comprobar si un programa existe

Usa `subprocess.run` con el comando `where python` (Windows) o `which python` (Linux) para comprobar si Python está accesible desde la terminal. Muestra el código de retorno.

Pista: el código de retorno 0 indica que `where`/`which` encontró Python. Distinto de 0, no lo encontró.

3. Abrir el navegador

Usa `subprocess.Popen` para abrir el navegador predeterminado con la URL `http://localhost:4321`. En Windows usa `start http://localhost:4321` con `shell=True`, en Linux usa `xdg-open http://localhost:4321`.

Pista: `start` es un comando interno del shell de Windows: necesita `shell=True`. `xdg-open` es un ejecutable real en Linux.

4. Lanzador múltiple

Crea un programa que lance 3 programas distintos a la vez (notepad, calc, mspaint). Muestra sus PIDs y luego los mata todos a los 5 segundos.

Pista: guarda los objetos `Popen` en una lista o tupla junto con el nombre de cada programa. Recorre la lista para mostrar PIDs y más tarde para terminarlos con `terminate()`.

5. Captura de ping

Ejecuta `ping 8.8.8.8 -n 5`, captura stdout y extrae solo las líneas que contienen “tiempo” o “time”.

Pista: divide el stdout con `split("\n")` y filtra cada línea con el operador `in` para buscar las palabras clave.

6. PID del padre

Crea un programa que muestre su propio PID y el PID del proceso padre (PPID).

Pista: usa `os.getpid()` para tu PID y `os.getppid()` para el PPID. Ejecuta desde la terminal y observa quién es el padre.

7. Comunicación en dos direcciones

Crea un proceso hijo que reciba texto por stdin y devuelva el texto en mayúsculas.

Pista: usa `subprocess.Popen` con `stdin=subprocess.PIPE`, `stdout=subprocess.PIPE` y `text=True`. Luego llama a `communicate(input="tu texto")` para enviar datos y leer la respuesta.

8. Timeout con reintentos

Crea una función que ejecute un comando con timeout de 3s. Si falla, reintenta hasta 3 veces. Si al final no funciona, lanza una excepción.

Pista: envuelve `subprocess.run` con `timeout=3` y `check=False` dentro de un bucle `for`. Captura `subprocess.TimeoutExpired` en cada intento y rompe el bucle si el comando se ejecuta correctamente. Si se agotan los intentos, lanza una excepción personalizada.

9. Mini monitor de procesos

Crea un programa que lance un subprocess y verifique su estado cada segundo mientras está vivo.

Pista: `proc.poll()` devuelve `None` si el proceso sigue ejecutándose, o el código de retorno si ya terminó. Úsalo en un bucle `while proc.poll() is None` con `time.sleep(1)` dentro.

---

10. Contador compartido con candado

Crea un contador que 4 procesos incrementen 100 veces cada uno. El total final debe ser siempre 400. Ejecútalo primero **sin** el candado y comprueba que el resultado es impredecible; después, protégelo con `multiprocessing.Lock`.

Pista: usa `multiprocessing.Value('i', 0)` para el contador y `multiprocessing.Lock()`. Pasa ambos como argumentos al `target` del `Process` y protege `contador.value += 1` con `with candado:`.

11. Almacén compartido con Manager

Monta un diccionario compartido con `multiprocessing.Manager().dict()` donde 4 procesos registran pares clave-valor (por ejemplo, cada proceso añade 3 productos con su stock). Al terminar, el padre debe ver **todas** las claves de los cuatro procesos.

Pista: crea el `manager` dentro de un bloque `with multiprocessing.Manager() as manager:` y pásale `manager.dict()` como argumento a cada `Process`. Sin `join()` a todos los hijos, el padre puede leer el diccionario antes de tiempo.

12. Persistencia del almacén

Añade al ejercicio anterior el guardado a fichero: al terminar, vuelca el `manager.dict()` a un JSON; al arrancar, recárgalo y úsalo como estado inicial para que el stock acumulado se mantenga entre ejecuciones.

Pista: `json.dump(dict(almacen), f)` — el proxy del Manager no es un dict, hay que convertirlo con `dict(...)` antes de serializarlo. En la carga, devuelve `{}` si el fichero no existe.

Boletín UD 2 — Avanzado (Resuelto)
💪 Boletín UD 2 — Avanzado (Resuelto)

1. Filtrar ipconfig

```
import subprocess

resultado = subprocess.run(["ipconfig"], capture_output=True, text=True)
lineas = resultado.stdout.split("\n")
for linea in lineas:
    if "IPv4" in linea:
        print(linea.strip())
```

`subprocess.run` captura toda la salida de `ipconfig`; `split("\n")` la divide en líneas y el filtro con `in` deja solo las de "IPv4".

2. Comprobar si un programa existe

```
import subprocess

resultado = subprocess.run(["where", "python"], capture_output=True, text=True)
print(f"Código de retorno: {resultado.returncode}")
if resultado.returncode == 0:
    print(f"Python encontrado en: {resultado.stdout.strip()}")
else:
    print("Python NO está en el PATH")
```

En Linux usarías `["which", "python"]`. El código de retorno `0` indica que `where`/`which` lo encontró; distinto de 0, no.

3. Abrir el navegador

```
import subprocess

# Windows: start es interno del shell, necesita shell=True
subprocess.run(["start", "http://localhost:4321"], shell=True)

# Linux / macOS:
# subprocess.run(["xdg-open", "http://localhost:4321"])
```

`start` no es un ejecutable real: es un comando interno de `cmd`, por eso usa `shell=True`. `xdg-open` sí es un ejecutable real en Linux (punto 7).

4. Lanzador múltiple

```
import subprocess, time

programas = [
    ("Bloc de notas", subprocess.Popen(["notepad.exe"])),
    ("Calculadora", subprocess.Popen(["calc.exe"])),
    ("Paint", subprocess.Popen(["mspaint.exe"])),
]

for nombre, proceso in programas:
    print(f"{nombre} → PID {proceso.pid}")

time.sleep(5)

for nombre, proceso in programas:
    proceso.terminate()
    print(f"Cerrando {nombre}...")

print("Todos terminados 🏁")
```

Guardamos cada objeto `Popen` en una tupla con su nombre. Recorremos la lista para mostrar PIDs y, a los 5 segundos, para terminarlos todos con `terminate()`.

5. Captura de ping

```
import subprocess

resultado = subprocess.run(["ping", "8.8.8.8", "-n", "5"], capture_output=True, text=True)
for linea in resultado.stdout.split("\n"):
    if "tiempo" in linea.lower() or "time" in linea.lower():
        print(linea)
```

`split("\n")` divide la salida y el filtro busca las líneas con “tiempo” (Windows en español) o “time” (sistemas en inglés). `.lower()` cubre ambas.

6. PID del padre

```
import os

print(f"Mi PID es {os.getpid()}")
print(f"El PID de mi padre es {os.getppid()}")
```

`os.getpid()` devuelve tu PID; `os.getppid()` el del proceso que te lanzó (normalmente tu terminal o el IDE).

7. Comunicación en dos direcciones

```
import subprocess

proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

salida, _ = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"
```

`communicate(input="hola mundo")` escribe en el stdin del hijo, espera su fin y devuelve `(stdout, stderr)`. El hijo ejecuta `print(input().upper())` (punto 6).

8. Timeout con reintentos

```
import subprocess

def ejecutar_con_reintentos(comando, intentos=3, timeout=3):
    for intento in range(1, intentos + 1):
        try:
            resultado = subprocess.run(comando, capture_output=True,
                                       text=True, timeout=timeout, check=False)
            if resultado.returncode == 0:
                print(f"✅ Éxito en el intento {intento}")
                return resultado
            print(f"Intento {intento}: código {resultado.returncode}")
        except subprocess.TimeoutExpired:
            print(f"Intento {intento}: se agotó el tiempo")
    raise TimeoutError(f"El comando {comando} falló tras {intentos} intentos")

try:
    resultado = ejecutar_con_reintentos(["ping", "8.8.8.8", "-n", "3"])
except TimeoutError as e:
    print(f"❌ {e}")
```

Cada intento usa `timeout=3` y `check=False`; se captura `TimeoutExpired` y se vuelve a intentar hasta 3 veces. Si nada funciona, se lanza una excepción propia.

9. Mini monitor de procesos

```
import subprocess, time

proceso = subprocess.Popen(["notepad.exe"])

while proceso.poll() is None:
    print(f"El proceso (PID {proceso.pid}) sigue vivo...")
    time.sleep(1)

print(f"El proceso terminó con código {proceso.poll()}")
```

`proc.poll()` devuelve `None` mientras el proceso vive y su código de retorno cuando termina. El bucle `while proc.poll() is None` vigila cada segundo hasta que muere.

10. Contador compartido con candado

```python
import multiprocessing
import time

def incrementar(contador, candado, veces):
    for _ in range(veces):
        with candado:
            # Sección crítica: leer, simular trabajo y escribir
            valor = contador.value
            time.sleep(0.0001)  # Simula que el cálculo tarda
            contador.value = valor + 1

if __name__ == '__main__':
    contador = multiprocessing.Value('i', 0)
    candado = multiprocessing.Lock()
    veces = 100

    procesos = []
    for i in range(4):
        p = multiprocessing.Process(
            target=incrementar,
            args=(contador, candado, veces),
            name=f"worker-{i}",
        )
        procesos.append(p)
        p.start()

    for p in procesos:
        p.join()

    print(f"Total: {contador.value}")  # Siempre 400
```

Sin el `with candado:` el resultado es impredecible (por ejemplo, 271, 318…): dos procesos leen el mismo valor previo y el segundo machaca el incremento del primero. Es la misma condición de carrera que con hilos, pero entre burbujas de memoria. Ojo: el `time.sleep` dentro de la sección crítica amplía la ventana del fallo para que se vea claro — quítalo y el error será más raro, pero también aparecerá.

11. Almacén compartido con Manager

```python
import multiprocessing

def registrar_stock(almacen, proceso, productos):
    for producto, unidades in productos:
        almacen[producto] = unidades
        print(f"[{proceso}] {producto} = {unidades}")

if __name__ == '__main__':
    with multiprocessing.Manager() as manager:
        almacen = manager.dict()

        trabajos = {
            "worker-1": {"manzanas": 10, "peras": 5},
            "worker-2": {"uvas": 7},
            "worker-3": {"naranjas": 3, "cerezas": 12},
            "worker-4": {"kiwis": 8},
        }

        procesos = []
        for nombre, productos in trabajos.items():
            p = multiprocessing.Process(
                target=registrar_stock,
                args=(almacen, nombre, productos),
                name=nombre,
            )
            procesos.append(p)
            p.start()

        # Esperar a TODOS antes de leer: si no, puede faltar trabajo
        for p in procesos:
            p.join()

        print(f"\nAlmacén final: {dict(almacen)}")
```

Claves del ejercicio: el `manager` vive dentro de un bloque `with` (al salir se apaga el proceso servidor del Manager y el proxy deja de responder), y el `join()` a todos los hijos es imprescindible antes de leer el resultado final.

12. Persistencia del almacén

```python
import json
import multiprocessing

def registrar_stock(almacen, proceso, productos):
    for producto, unidades in productos:
        almacen[producto] = almacen.get(producto, 0) + unidades
        print(f"[{proceso}] +{unidades} {producto}")

def guardar_almacen(almacen, fichero="almacen.json"):
    # El proxy NO es un dict: hay que convertirlo con dict(...)
    with open(fichero, "w", encoding="utf-8") as f:
        json.dump(dict(almacen), f, ensure_ascii=False, indent=2)
    print(f"💾 Almacenado en {fichero}")

def cargar_almacen(fichero="almacen.json"):
    try:
        with open(fichero, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}

if __name__ == '__main__':
    estado_previo = cargar_almacen()
    print(f"📥 Estado previo: {estado_previo}")

    with multiprocessing.Manager() as manager:
        almacen = manager.dict(estado_previo)  # Arranca desde el JSON

        trabajos = {
            "worker-1": {"manzanas": 10},
            "worker-2": {"uvas": 7},
        }

        procesos = []
        for nombre, productos in trabajos.items():
            p = multiprocessing.Process(
                target=registrar_stock,
                args=(almacen, nombre, productos),
                name=nombre,
            )
            procesos.append(p)
            p.start()
```

```
    for p in procesos:
        p.join()

    print(f"\nTotal del turno: {dict(almacen)}")
    guardar_almacen(almacen)
```

Ejecútalo dos veces seguidas: la primera arranca de `{}` y guarda `{"manzanas": 10, "uvas": 7}`; la segunda **arranca de ahí** y suma, quedando `{"manzanas": 20, "uvas": 14}`. Esa es la persistencia: el JSON conecta dos turnos de procesos. El patrón completo, explicado paso a paso, está en el punto 9 de la UD 2.

Boletín UD 3 — Inicial

📝 Boletín UD 3 — Inicial

> Ejercicios básicos para afianzar los conceptos de hilos de la unidad UD 3: crear, lanzar, esperar, pasar argumentos y nombrar hilos.

---

1. Hilo que cuenta

Crea un hilo que ejecute una función que cuente del 1 al 5 imprimiendo cada número. Lánzalo con `start()` y espera con `join()`.

2. Múltiples hilos con retardo

Crea 3 hilos que impriman su número (1, 2, 3) y luego duerman 1 segundo con `time.sleep(1)`. Lánzalos todos y espera a que terminen.

3. Hilo con argumento

Crea una función que reciba un nombre como argumento e imprima “Hola, {nombre}”. Lanza un hilo pasándole el argumento con `args=("Ana",)`.

Pista: recuerda la coma final de la tupla: `args=("Ana",)`. Sin la coma, `args="Ana"` no es una tupla: Python itera el string y pasa un argumento por carácter (3 en `'Ana'`), así que la función recibe demasiados argumentos → `TypeError`.

4. Hilo que saluda

Crea un hilo que ejecute una función que imprima “Hola desde un hilo”. Usa `threading.Thread`, `start()` y `join()`.

5. Dos hilos en paralelo

Crea dos funciones `hilo_a()` y `hilo_b()`, cada una con un bucle que imprima su nombre 3 veces. Lanza los dos hilos y observa cómo se entremezclan los mensajes.

6. Hilo con nombre

Crea una función que imprima `f"Soy {threading.current_thread().name}"`. Lánzala en un hilo con `name="MiHilo"`.

7. Esperando con join()

Crea un hilo que imprima “Empezando”, duerma 3 segundos y termine imprimiendo “Terminado”. En el programa principal, después de `start()`, espera con `join()` y luego imprime “El programa terminó”. Comprueba el orden de las tres salidas.

8. Varios hilos con nombre y argumentos

Crea 3 hilos en un bucle, cada uno con `name = "hilo-" + str(n)` y que reciba su número como argumento para imprimir `"hilo-N trabajando"`. Lanza todos y espera a todos.

Pista: usa una lista por comprensión para crear los 3 hilos y luego dos bucles: `for h in hilos: h.start()` y `for h in hilos: h.join()`.

Boletín UD 3 — Inicial (Resuelto)
✅ Boletín UD 3 — Inicial (Resuelto)

1. Hilo que cuenta

```python
import threading

def contar():
    for i in range(1, 6):
        print(i)

h = threading.Thread(target=contar)
h.start()
h.join()
```

Salida: `1 2 3 4 5`. La función cuenta del 1 al 5, se lanza con `start()` y se espera con `join()`.

2. Múltiples hilos con retardo

```python
import threading, time

def numero(n):
    print(f"Hilo {n} empieza")
    time.sleep(1)
    print(f"Hilo {n} termina")

hilos = [threading.Thread(target=numero, args=(i,)) for i in range(1, 4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Los tres hilos se lanzan a la vez y duermen su segundo **en paralelo**: todo el lote acaba en ~1 segundo, no en 3. Los mensajes se entremezclan sin orden garantizado.

3. Hilo con argumento

```
import threading

def saludar(nombre):
    print(f"Hola, {nombre}")

h = threading.Thread(target=saludar, args=("Ana",))
h.start()
h.join()
```

Salida: `Hola, Ana`. La coma final de `("Ana",)` convierte el valor en una tupla de un elemento, que es lo que espera `args=`.

4. Hilo que saluda

```
import threading

def saludar():
    print("Hola desde un hilo")

h = threading.Thread(target=saludar)
h.start()
h.join()
```

Nunca olvides `start()` para lanzar y `join()` para esperar.

5. Dos hilos en paralelo

```python
import threading

def hilo_a():
    for _ in range(3):
        print("Hilo A")

def hilo_b():
    for _ in range(3):
        print("Hilo B")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start()
h2.start()
h1.join()
h2.join()
```

Los prints se entremezclarán. El orden no está garantizado.

6. Hilo con nombre

```python
import threading

def mostrar():
    print(f"Soy {threading.current_thread().name}")

h = threading.Thread(target=mostrar, name="MiHilo")
h.start()
h.join()
```

`current_thread().name` funciona desde dentro del hilo.

7. Esperando con join()

```python
import threading, time

def tarea():
    print("Empezando")
    time.sleep(3)
    print("Terminado")

h = threading.Thread(target=tarea)
h.start()
h.join()
print("El programa terminó")
```

Salida:

```
Empezando
Terminado
El programa terminó
```

Sin `join()`, el "El programa terminó" podría salir antes que "Terminado" (o incluso antes que "Empezando", según el scheduler). Con `join()`, el orden es garantizado.

8. Varios hilos con nombre y argumentos

```python
import threading

def trabajar(n):
    print(f"hilo-{n} trabajando")

hilos = [threading.Thread(target=trabajar, args=(n,), name="hilo-" + str(n))
         for n in range(1, 4)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Cada hilo lleva su nombre (`hilo-1`, `hilo-2`, `hilo-3`) y recibe su número como argumento. Los mensajes se entremezclan, pero el orden de lanzamiento y espera es correcto gracias a los dos bucles.

Boletín UD 3 — Avanzado

💪 Boletín UD 3 — Avanzado

> Ejercicios que requieren aplicar los conceptos de hilos de la unidad UD 3 de forma más profunda: esperas con timeout, introspección, banderas de parada y timers que se repiten.

---

1. Join con timeout

Crea un hilo que duerma 5 segundos. En el hilo principal, usa `join(timeout=2)` y comprueba con `is_alive()` si el hilo sigue ejecutándose después del timeout.

Pista: `h.join(2)` devuelve `None` cuando termina a tiempo y también cuando expira. Para saber si sigue vivo después del timeout, pregunta con `h.is_alive()` justo después del `join`.

2. Listar hilos activos

Crea 3 hilos que hagan tareas distintas. Usa `threading.enumerate()` para listar todos los hilos activos y muestra sus nombres.

Pista: `threading.enumerate()` devuelve una lista de objetos Thread. Itera sobre ella y muestra `.name`. Recuerda que el hilo principal (main) también aparece en la lista.

3. Identificar el hilo actual

Crea una función que muestre `threading.current_thread().name`, `threading.current_thread().ident` y `threading.current_thread().daemon`. Ejecútala desde el hilo principal y desde un hilo secundario.

Pista: para ejecutar la función desde ambos sitios, llámala directamente en el principal y también como `target` de un `threading.Thread`.

4. 🎯 Carrera de mensajes

Crea 5 hilos, cada uno imprime su nombre 10 veces. Observa el orden impredecible.

Pista: usa una lista por comprensión para crear los 5 hilos. Lanza todos con un bucle `for h in hilos: h.start()` y espera con `for h in hilos: h.join()`. El orden de salida cambiará en cada ejecución.

5. 🔍 Simular descarga con progreso

Crea un hilo que descargue (simulado con sleep) y muestre progreso cada 25%.

Pista: dentro de la función, itera con `range(0, 101, 25)`, usa `time.sleep(0.5)` para simular el tiempo de descarga e imprime el porcentaje en cada paso.

6. 🧩 Detener hilo con bandera

Crea un hilo infinito que se detenga cuando pulses Enter.

Pista: declara una variable booleana global `ejecutando = True`. El hilo comprueba la variable en un `while ejecutando:`. Desde el principal, usa `input()` para esperar Enter y luego cambia `ejecutando = False`.

7. 🎭 Hilo que devuelve un valor

Usa una variable compartida para que un hilo devuelva un resultado al principal.

Pista: los hilos no tienen `return`. Usa una lista compartida como contenedor: el hilo guarda el resultado con `lista.append(valor)` y el principal lo lee tras `hilo.join()`.

8. ⏱️ Timer con repetición

El Timer normal solo se dispara una vez. Crea un timer que se repita cada 2 segundos usando recursividad.

Pista: dentro de la función que ejecuta el Timer, crea otro `threading.Timer(2.0, funcion).start()` al final. Así la función se programa a sí misma de nuevo. Ojo: si el programa no termina, pon el Timer como daemon (`t.daemon = True` **antes** de `t.start()`) para que muera con el principal.

9. 🏗️ Pool de hilos manual

Crea un “pool” manual con una cola de tareas. 2 hilos cogen tareas de la cola y las ejecutan.

Pista: usa `queue.Queue` (thread-safe) para las tareas. Los trabajadores ejecutan un bucle llamando a `cola.get()`. Al terminar cada tarea llaman a `cola.task_done()`. El principal espera con `cola.join()`. Crea los workers como `daemon=True` para que no bloqueen la salida, o usa un valor centinela (`None`) en la cola para que terminen ordenadamente.

Boletín UD 3 — Avanzado (Resuelto)
💪 Boletín UD 3 — Avanzado (Resuelto)

1. Join con timeout

```python
import threading, time

def lento():
    print("Hilo empezando")
    time.sleep(5)
    print("Hilo terminó")

h = threading.Thread(target=lento)
h.start()
h.join(timeout=2)
print(f"¿Sigue vivo tras el timeout? {h.is_alive()}")
```

Salida:

```
Hilo empezando
¿Sigue vivo tras el timeout? True
Hilo terminó
```

`join(timeout=2)` espera como máximo 2 segundos: al expirar, el hilo **sigue vivo** (duerme hasta los 5s) y el principal continúa. Como es no-daemon, el programa no termina hasta que el hilo acaba su sueño.

2. Listar hilos activos

```
import threading, time

def tarea(n):
    time.sleep(1)
    print(f"{threading.current_thread().name} terminó")

hilos = [threading.Thread(target=tarea, args=(i,), name="hilo-" + str(i))
         for i in range(1, 4)]

for h in hilos:
    h.start()

print("Hilos activos:")
for h in threading.enumerate():
    print(f"  {h.name}")

for h in hilos:
    h.join()
```

`threading.enumerate()` devuelve la lista de todos los hilos vivos, **incluido el principal** (`MainThread`). Justo tras el `start()` de los tres, la lista contiene 4 entradas: `MainThread`, `hilo-1`, `hilo-2` y `hilo-3`.

3. Identificar el hilo actual

```
import threading

def mostrar():
    h = threading.current_thread()
    print(f"name: {h.name} | ident: {h.ident} | daemon: {h.daemon}")

mostrar()  # desde el hilo principal

h = threading.Thread(target=mostrar, name="secundario")
h.start()
h.join()
```

Salida parecida a:

```
name: MainThread | ident: 14600 | daemon: False
name: secundario | ident: 14601 | daemon: False
```

La misma función devuelve datos distintos según quién la ejecute: `current_thread()` es el objeto Thread que se está ejecutando en ese momento. El `ident` es un número único mientras el hilo vive.

4. 🎯 Carrera de mensajes

```python
import threading

def mensaje(nombre):
    for i in range(1, 11):
        print(f"{nombre}: mensaje {i}")

hilos = [threading.Thread(target=mensaje, args=(f"hilo-{n}",))
         for n in range(1, 6)]

for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Cinco hilos imprimiendo 10 mensajes cada uno: la salida es un caos predecible en el desorden. El orden exacto cambia en cada ejecución porque lo decide el scheduler del sistema operativo. Solo está garantizado que los 50 mensajes salgan (y que el programa espere gracias a los `join()`).

5. 🔍 Simular descarga con progreso

```python
import threading, time

def descargar(archivo):
    for porcentaje in range(0, 101, 25):
        print(f"{archivo}: {porcentaje}%")
        time.sleep(0.5)

h = threading.Thread(target=descargar, args=("libro.pdf",))
h.start()
h.join()
```

Salida:

```
libro.pdf: 0%
libro.pdf: 25%
libro.pdf: 50%
libro.pdf: 75%
libro.pdf: 100%
```

El progreso avanza en saltos de 25% (`range(0, 101, 25)`) con medio segundo de espera por paso: una descarga simulada de 2 segundos en total.

6. 🧩 Detener hilo con bandera

```python
import threading, time

ejecutando = True

def infinito():
    while ejecutando:
        print("trabajando...")
        time.sleep(0.5)
    print("Hilo detenido por bandera")

h = threading.Thread(target=infinito)
h.start()

input("Pulsa Enter para detener...")
ejecutando = False
h.join()
print("Programa terminado")
```

Los hilos no se matan desde fuera (no hay `hilo.kill()`). La forma correcta es una **variable bandera** que el hilo comprueba en cada vuelta de su bucle: al pulsar Enter, el principal pone `ejecutando = False` y el hilo sale del bucle y termina ordenadamente.

7. 🎭 Hilo que devuelve un valor

```python
import threading, time

resultado = []

def calcular():
    time.sleep(1)
    valor = 6 * 7
    resultado.append(valor)

h = threading.Thread(target=calcular)
h.start()
h.join()
print(f"El resultado del hilo es: {resultado[0]}")
```

Salida:

```
El resultado del hilo es: 42
```

Las funciones de los hilos **no tienen `return`**. Para "devolver" un valor se usa una **lista compartida** (la memoria compartida del punto 1): el hilo hace `append`, y tras `join()` el principal lee el valor con seguridad, porque ya sabe que el hilo ha terminado.

8. ⏱️ Timer con repetición

```python
import threading, time

def tic():
    print(f"⏰ {time.strftime('%H:%M:%S')}")
    t = threading.Timer(2.0, tic)   # se reprograma a sí misma
    t.daemon = True                 # daemon: muere con el principal
    t.start()

tic()

time.sleep(6)   # el principal deja correr unos cuantos ticks
print("Programa principal terminando...")
```

Salida parecida a:

```
⏰ 14:35:00
⏰ 14:35:02
⏰ 14:35:04
⏰ 14:35:06
Programa principal terminando...
```

El `Timer` normal solo dispara **una vez** (punto 5). Para repetirlo, la propia función crea y lanza otro `Timer` al final: recursividad. Así cada tick programa el siguiente. El principal decide cuándo parar: ponemos cada Timer como **daemon** (`t.daemon = True`) para que, al terminar el principal, los ticks pendientes mueran con él y el programa no se quede esperando.

9. ⏳ Pool de hilos manual

```python
import threading, queue, time

def trabajador():
    while True:
        tarea = cola.get()
        if tarea is None:          # centinela: fin de trabajo
            break
        print(f"{threading.current_thread().name} hace: {tarea}")
        time.sleep(0.5)
        cola.task_done()

cola = queue.Queue()
for t in ["tarea-1", "tarea-2", "tarea-3", "tarea-4"]:
    cola.put(t)

workers = [threading.Thread(target=trabajador, name="worker-" + str(i))
           for i in range(1, 3)]
for w in workers:
    w.start()

cola.join()                # espera a que las 4 tareas se marquen como hechas
for w in workers:
    cola.put(None)         # enviamos el centinela a cada worker
for w in workers:
    w.join()

print("Todas las tareas terminadas")
```

`queue.Queue` es **thread-safe**: varios hilos pueden hacer `get()` sin corromper la cola. Dos workers compiten por las 4 tareas (las cogen en cuanto están libres, por eso la repartición puede variar). El principal espera con `cola.join()` y, al terminar, envía `None` (centinela) para que cada worker salga del bucle ordenadamente, sin dejar hilos colgados.

🎁 Bonus: tres clásicos resueltos

Tres casos que suelen aparecer en exámenes con su solución completa.

Bonus 1 — join() o no join()

```python
import threading, time

def lento():
    for i in range(5):
        print(i)
        time.sleep(1)

h = threading.Thread(target=lento)
h.start()
# Sin join() → el programa principal no espera
print("Fin del programa principal")
```

Sin `join()`, el programa principal no espera: su último `print` aparece mientras el hilo sigue contando. El programa en sí no puede terminar hasta que el hilo (no daemon) acabe, pero el orden de salida queda a merced del scheduler.

Bonus 2 — Hilo daemon

```python
import threading, time

def tic():
    while True:
        print("tic")
        time.sleep(1)

h = threading.Thread(target=tic, daemon=True)
h.start()
time.sleep(3)
print("Programa termina — el daemon muere")
```

El daemon imprime “tic” cada segundo y se **mata al salir** del programa principal (punto 4). Si fuera no-daemon con su `while True`, el programa jamás terminaría.

Bonus 3 — Timer

```
import threading

def despierta():
    print("¡Despierta!")

t = threading.Timer(4.0, despierta)
t.start()
```

Timer ejecuta **UNA SOLA vez** después del retardo de 4 segundos (punto 5). No se repite; para repetirlo, la función debe reprogramarse (ejercicio 8).

Boletín UD 4 — Inicial

📝 Boletín UD 4 — Inicial

> Ejercicios básicos para afianzar los conceptos de sincronización: condición de carrera, Lock, RLock, semáforo y barrera.

---

1. Lock protege una lista

Crea 2 hilos que añadan elementos a una lista compartida. Uno añade “A” 5 veces, el otro “B” 5 veces. Usa un `Lock` para evitar mezclas.

2. RLock básico

Crea una función que adquiera un `RLock` dos veces desde el mismo hilo (llamando a otra función que también lo adquiera). Comprueba que no se produce deadlock.

3. Semáforo con timeout

Crea un `Semaphore(2)` y lanza 4 hilos que intenten entrar. Cada hilo espera como máximo 1 segundo con `acquire(timeout=1)`. Los que no consigan entrar deben mostrar “timeout”.

Pista: `acquire(timeout=1)` devuelve `True` si consiguió el recurso y `False` si pasó el segundo. Usa ese `True`/`False` para decidir si el hilo entra o muestra “timeout”. Ojo: el trabajo dentro del semáforo (el `sleep`) debe durar **más** que el timeout; si dura exactamente 1 segundo, el margen desaparece y todos entran.

4. Carrera sin Lock

Crea 2 hilos que incrementen un contador compartido 5000 veces cada uno **sin** usar Lock. Ejecuta el programa varias veces y anota los resultados.

Pista: para que la condición de carrera sea visible, pasa la suma por una función (`contador = sumar(contador)`) y añade `sys.setswitchinterval(1e-6)` al principio. Así el contador casi nunca será 10000: `contador += 1` no es atómico (leer → sumar → escribir) y el planificador puede cortar entre pasos.

5. Contador protegido con Lock

Repite el ejercicio 4 pero protege el incremento con `lock = threading.Lock()` y `with lock:`. ¿Qué valor obtienes ahora?

6. Sección crítica con sleep

Crea una variable compartida `total = 0` y 3 hilos que hagan `total += 10` y luego `time.sleep(0.1)` repetido 20 veces. Protege la sección con un `Lock` y comprueba que al final `total` vale 600.

7. Semáforo simple

Crea un `Semaphore(2)` y lanza 4 hilos que simulen entrar en un recurso (imprime “entra”, duerme 1 segundo, imprime “sale”). Comprueba que solo 2 hilos están dentro a la vez.

8. Barrera de 3

Crea 3 hilos que simulen corredores: cada uno se prepara (duerme un tiempo distinto) y después espera en una `Barrier(3)`. Imprime “ 🏁 salió” cuando cruzan. Comprueba que los 3 salen a la vez.

Boletín UD 4 — Inicial (Resuelto)
✅ Boletín UD 4 — Inicial (Resuelto)

1. Lock protege una lista

```
import threading

lista = []
lock = threading.Lock()

def añadir(letra):
    global lista
    for _ in range(5):
        with lock:
            lista.append(letra)

h1 = threading.Thread(target=añadir, args=("A",))
h2 = threading.Thread(target=añadir, args=("B",))
h1.start()
h2.start()
h1.join()
h2.join()

print(lista)
```

Sin el `Lock`, los `append` se mezclarían (por ejemplo, `AABABB...`). Con `with lock:`, cada añadido es atómico y al final hay **5 A y 5 B** sin mezclas.

2. RLock básico

```python
import threading

rlock = threading.RLock()

def función_interna():
    with rlock:
        print("🔹 Función interna: lock adquirido")

def función_externa():
    with rlock:
        print("🔸 Función externa: lock adquirido")
        función_interna()   # mismo hilo → RLock lo permite

función_externa()
```

`función_interna` adquiere el mismo `RLock` que `función_externa` ya tiene puesto. Con un `Lock` normal sería un **deadlock** (el hilo se espera a sí mismo); con `RLock` no hay problema (punto 3).

3. Semáforo con timeout

```python
import threading, time

semaforo = threading.Semaphore(2)

def entrar(id):
    if semaforo.acquire(timeout=1):
        print(f"Hilo-{id}: DENTRO")
        time.sleep(2)   # ocupa el semáforo más tiempo que el timeout
        semaforo.release()
    else:
        print(f"Hilo-{id}: timeout")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Solo 2 hilos entran a la vez; los que no consiguen entrar en 1 segundo muestran **“timeout”** y se van en lugar de esperar eternamente (punto 4).

4. Carrera sin Lock

```
import threading, sys

contador = 0
sys.setswitchinterval(1e-6)   # fuerza cambios de hilo frecuentes

def sumar(c):
    return c + 1

def inc():
    global contador
    for _ in range(5000):
        contador = sumar(contador)   # leer-sumar-escribir vía función

hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador)  # ❌ Casi nunca será 10000
```

Sin Lock, hay condición de carrera. El valor casi siempre será **< 10000** y variará en cada ejecución (punto 1). En Python 3.13+ el `+=` en una línea casi nunca se interrumpe; pasando la suma por una función y forzando el cambio de hilo con `setswitchinterval`, la carrera se vuelve visible.

5. Contador protegido con Lock

```
import threading
contador = 0
lock = threading.Lock()
def inc():
    global contador
    for _ in range(1000):
        with lock:
            contador += 1
hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador)  # ✅ 2000
```

`with lock:` garantiza exclusión mutua: el “leer → sumar → escribir” ocurre de principio a fin sin que se cuele el otro hilo (punto 2).

6. Sección crítica con sleep

```
import threading, time

total = 0
lock = threading.Lock()

def trabajar():
    global total
    for _ in range(20):
        with lock:
            total += 10
        time.sleep(0.1)

hilos = [threading.Thread(target=trabajar) for _ in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Total: {total}")  # ✅ 600 (3 hilos × 20 × 10)
```

El `Lock` protege la sección crítica (`total += 10`); el `sleep` ocurre fuera para no bloquear a los demás.

7. Semáforo simple

```
import threading, time
s = threading.Semaphore(2)
def entrar(id):
    print(f"{id} espera")
    with s:
        print(f"  → {id} dentro")
        time.sleep(1)
    print(f"  ← {id} sale")
hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Solo 2 hilos entran a la vez. Los demás esperan hasta que un puesto se libera (punto 4).

8. Barrera de 3

```
import threading, time
b = threading.Barrier(3)
def corredor(id):
    time.sleep(id)
    print(f"  {id} preparado")
    b.wait()
    print(f"🏁 {id} salió")
hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Todos imprimen “preparado” antes de que nadie “salga”: la barrera no deja cruzar hasta que los 3 han llegado (punto 5).

Boletín UD 4 — Avanzado
💪 Boletín UD 4 — Avanzado

> Ejercicios que requieren aplicar los conceptos de sincronización de forma más profunda, con programas completos y varios mecanismos a la vez.

---

1. Condition con notify_all

Crea 3 hilos consumidores que esperen en una `Condition`. Un hilo productor añade 3 elementos y usa `notify_all()` para despertar a todos los consumidores a la vez.

2. Deadlock entre dos hilos

Crea 2 hilos y 2 locks (A y B). El hilo 1 adquiere A luego B. El hilo 2 adquiere B luego A. Añade `time.sleep(0.1)` entre adquisiciones para provocar el deadlock.

3. Semáforo con recursos limitados

Simula 3 impresoras compartidas. Crea 6 hilos que necesiten una impresora para imprimir (simulado con sleep). Usa `Semaphore(3)` para que solo 3 impriman a la vez.

4. 🎯 Banco con cuentas compartidas

Dos hilos: uno ingresa dinero, otro retira. Usa Lock para evitar saldo negativo.

Pista: Usa un Lock global. Cada hilo lo adquiere con `with lock:` antes de modificar el saldo. El hilo retirador debe comprobar `if saldo >= cantidad` antes de restar.

5. 🔍 Semáforo con timeout

Crea un semáforo donde los hilos esperen máximo 2s. Si no entran, se van.

Pista: `semaforo.acquire(blocking=True, timeout=2)` devuelve `True` si consiguió el recurso, o `False` si pasaron los 2 segundos. Úsalo para decidir si el hilo entra o se rinde.

6. 🧩 Barrera con fases múltiples

Crea 3 hilos que trabajen en 3 fases. La fase siguiente no empieza hasta que todos terminan la anterior.

Pista: Coloca `barrera.wait()` justo después del trabajo de cada fase. Así ningún hilo avanza a la siguiente fase hasta que todos han llegado al `wait()`.

7. 🎭 Condition con timeout

Un productor que produce cada 3s, consumidor que espera máximo 2s. Si no hay producto, se va.

Pista: `cond.wait(timeout=2)` devuelve `False` si el tiempo expiró sin ser notificado. Úsalo para que el consumidor salga si el productor tarda demasiado.

8. ⏱️ Deadlock provocado y solución

Crea un deadlock con 2 hilos y 2 locks. Luego arréglalo adquiriendo los locks en el mismo orden.

Pista: El deadlock ocurre cuando el hilo A toma lock1 y espera lock2, mientras B toma lock2 y espera lock1. La solución: ambos hilos adquieren los locks en el mismo orden (lock1 → lock2).

9. 🪜 Productor-Consumidor múltiple

2 productores y 3 consumidores compitiendo por un buffer de tamaño 5.

Pista: Usa `queue.Queue(maxsize=5)` como buffer compartido: es thread-safe y no necesita Lock. Los productores llaman a `put()` y los consumidores a `get()`.

Boletín UD 4 — Avanzado (Resuelto)
💪 Boletín UD 4 — Avanzado (Resuelto)

1. Condition con notify_all

```
import threading, time

cola = []
condition = threading.Condition()

def consumidor(id):
    for _ in range(1):
        with condition:
            while not cola:
                condition.wait()
            item = cola.pop(0)
            print(f"  🍽️ Consumidor-{id} procesó: {item}")

def productor():
    for i in range(3):
        with condition:
            cola.append(f"item-{i}")
            print(f"📦 Producido: item-{i}")
            condition.notify_all()   # despierta a los 3 consumidores
        time.sleep(0.5)

hilos = [threading.Thread(target=consumidor, args=(i,)) for i in range(3)]
hilos.append(threading.Thread(target=productor))
for h in hilos: h.start()
for h in hilos: h.join()
```

Con 3 consumidores esperando, `notify_all()` los despierta a **todos**; si usáramos `notify()` solo despertaría a uno y los demás podrían quedarse dormidos (punto 6).

2. Deadlock entre dos hilos

```python
import threading, time

lockA = threading.Lock()
lockB = threading.Lock()

def hilo_1():
    with lockA:
        time.sleep(0.1)
        with lockB:   # espera a que el hilo 2 libere B
            print("Hilo 1 trabajando")

def hilo_2():
    with lockB:
        time.sleep(0.1)
        with lockA:   # espera a que el hilo 1 libere A
            print("Hilo 2 trabajando")

h1 = threading.Thread(target=hilo_1)
h2 = threading.Thread(target=hilo_2)
h1.start(); h2.start()
h1.join(); h2.join()  # ❌ nunca termina: deadlock
```

El hilo 1 tiene A y espera B; el hilo 2 tiene B y espera A. **Ningún `print` sale nunca** y el programa se queda colgado (punto 8).

3. Semáforo con recursos limitados

```python
import threading, time

impresoras = threading.Semaphore(3)

def imprimir(id):
    with impresoras:
        print(f"🖨️ Imprimiendo documento-{id}...")
        time.sleep(2)
        print(f"  ✅ Documento-{id} impreso")

hilos = [threading.Thread(target=imprimir, args=(i,)) for i in range(6)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Con `Semaphore(3)` nunca hay más de **3 documentos imprimiendo a la vez**, aunque haya 6 hilos esperando (punto 4).

4. 🎯 Banco con cuentas compartidas

```
import threading, time, random

saldo = 100
lock = threading.Lock()

def ingresar():
    global saldo
    for _ in range(5):
        with lock:
            cantidad = random.randint(10, 50)
            saldo += cantidad
            print(f"➕ Ingreso {cantidad} € → saldo {saldo} €")
        time.sleep(random.random())

def retirar():
    global saldo
    for _ in range(5):
        with lock:
            cantidad = random.randint(10, 50)
            if saldo >= cantidad:
                saldo -= cantidad
                print(f"➖ Retiro {cantidad} € → saldo {saldo} €")
            else:
                print(f"❌ Saldo insuficiente ({saldo} €) para retirar {cantidad} €")
        time.sleep(random.random())

h1 = threading.Thread(target=ingresar)
h2 = threading.Thread(target=retirar)
h1.start(); h2.start()
h1.join(); h2.join()
print(f"Saldo final: {saldo} €")
```

El `Lock` global evita que ingreso y retiro se pisen; el retirador comprueba `saldo >= cantidad` **dentro** del lock antes de restar, así el saldo **nunca es negativo**.

5. 🔍 Semáforo con timeout

```python
import threading, time

semaforo = threading.Semaphore(2)

def tarea(id):
    if semaforo.acquire(blocking=True, timeout=2):
        print(f"Hilo-{id}: entró")
        time.sleep(3)
        semaforo.release()
    else:
        print(f"Hilo-{id}: se rinde (timeout 2s)")

hilos = [threading.Thread(target=tarea, args=(i,)) for i in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Con `timeout=2`, los hilos que no consiguen entrar en 2 segundos **se rinden** (devuelve `False`) en vez de bloquearse para siempre. Los primeros 2 entran; el resto abandona.

6. 🧩 Barrera con fases múltiples

```python
import threading, time

barrera = threading.Barrier(3)

def trabajador(id):
    for fase in range(3):
        print(f"  🧱 Hilo-{id}: trabajando en fase {fase+1}")
        time.sleep(id * 0.3)   # cada hilo tarda distinto
        barrera.wait()         # nadie pasa a la siguiente fase hasta que llegan los 3
        print(f"  🏁 Hilo-{id}: fase {fase+1} completada")

hilos = [threading.Thread(target=trabajador, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Los `barrera.wait()` después de cada fase garantizan que **ningún hilo empieza la fase N+1** hasta que los 3 terminaron la fase N (punto 5).

7. 🎭 Condition con timeout

```
import threading, time

cola = []
condition = threading.Condition()

def productor():
    for _ in range(2):
        time.sleep(3)              # produce cada 3 segundos
        with condition:
            cola.append("producto")
            print("📦 Producido (cada 3s)")
            condition.notify()

def consumidor():
    for _ in range(2):
        with condition:
            if not condition.wait(timeout=2):   # espera máximo 2s
                print("  ⏳ timeout: el consumidor se va")
            elif cola:
                print(f"  🍽️ Consumido: {cola.pop(0)}")

h1 = threading.Thread(target=productor)
h2 = threading.Thread(target=consumidor)
h1.start(); h2.start()
h1.join(); h2.join()
```

`wait(timeout=2)` devuelve `False` si expiran los 2 segundos sin notificación: el consumidor **se va** en lugar de esperar los 3 segundos del productor. Devuelve `True` si le avisan a tiempo.

8. ⏱️ Deadlock provocado y solución

Provocación (deadlock):

```
import threading, time

lock1 = threading.Lock()
lock2 = threading.Lock()

def hilo_a():
    with lock1:
        time.sleep(0.1)
        with lock2:
            print("A: trabajando")

def hilo_b():
    with lock2:          # ❌ orden invertido
        time.sleep(0.1)
        with lock1:
            print("B: trabajando")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start(); h2.start()
h1.join(); h2.join()   # se queda colgado
```

Solución (mismo orden de locks):

```
def hilo_b():
    with lock1:          # ✅ mismo orden que A: lock1 → lock2
        time.sleep(0.1)
        with lock2:
            print("B: trabajando")
```

Adquiriendo los locks **siempre en el mismo orden** (`lock1 → lock2`), es imposible que un hilo tenga lock2 esperando lock1 mientras otro tiene lock1 esperando lock2. El programa termina correctamente (punto 8).

9. 🏗️ Productor-Consumidor múltiple

```python
import threading, time, random, queue

buffer = queue.Queue(maxsize=5)

def productor(id):
    for _ in range(4):
        item = f"p{id}-{random.randint(1, 100)}"
        buffer.put(item)            # thread-safe, no necesita Lock
        print(f"📦 P{id} produjo: {item} (buffer {buffer.qsize()}/5)")
        time.sleep(random.random())

def consumidor(id):
    for _ in range(2):
        item = buffer.get()         # thread-safe
        print(f"  🍽️ C{id} consumió: {item} (buffer {buffer.qsize()}/5)")
        time.sleep(random.random())

hilos = [threading.Thread(target=productor, args=(i,)) for i in range(2)]
hilos += [threading.Thread(target=consumidor, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

`queue.Queue(maxsize=5)` es **thread-safe**: `put()` y `get()` ya están sincronizadas internamente. Con `maxsize=5`, si el buffer se llena, los productores esperan; si se vacía, los consumidores esperan. Sin locks a mano (punto 7).

Boletín UD 5 — Inicial

📝 Boletín UD 5 — Inicial

> Ejercicios básicos para afianzar los conceptos de sockets TCP y UDP: cliente, servidor, eco, la pareja IP/puerto, el HTTP manual y la comparativa de protocolos de la unidad UD 5.

---

1. Servidor saludo

Crea un servidor TCP que, cuando un cliente se conecte, le envíe “Bienvenido al servidor” y luego cierre la conexión.

2. Cliente que envía y recibe

Crea un cliente que se conecte a 127.0.0.1:9000, envíe “Hola” y luego espere y muestre la respuesta del servidor.

3. Servidor eco

Crea un servidor TCP que acepte una conexión, reciba un mensaje y devuelva exactamente lo mismo que recibe.

4. IP y puerto

Responde por escrito:

a) ¿Qué identifica la IP y qué identifica el puerto en una comunicación de red? b) ¿Qué significa `127.0.0.1` y para qué se usa? c) ¿Qué constante de Python crea un socket TCP y cuál un socket UDP?

Pista: repasa el punto 1 de la unidad: la IP identifica la **máquina**, el puerto identifica el **programa**; `127.0.0.1` es **localhost**; y los tipos de socket son `SOCK_STREAM` (TCP) y `SOCK_DGRAM` (UDP).

5. Cliente UDP con entrada de usuario

Crea un cliente UDP que pida un mensaje al usuario por teclado con `input()`, lo envíe a `127.0.0.1:9001` y espere una respuesta.

Pista: UDP no tiene `connect()`: la dirección va dentro del `sendto()`, y la respuesta llega con `recvfrom()`.

6. Servidor UDP con eco personalizado

Crea un servidor UDP que escuche en `127.0.0.1:9001`. Al recibir un mensaje, responda con `"Recibido: "` seguido del mensaje original.

7. TCP vs UDP: clasifica

a) Clasifica cada aplicación como TCP o UDP y justifica brevemente:

- Web (HTTP)
- Videollamada (Zoom)
- Correo (SMTP)
- Juego online (Fortnite)
- Transferencia de archivos (FTP)
- DNS

b) Completa la tabla:

Característica	TCP	UDP
Conexión		
Entrega garantizada		
Orden		
Velocidad		

8. Cliente HTTP manual

Conéctate con un socket TCP a `www.example.com:80`, haz un GET a `/` y muestra los primeros 500 caracteres de la respuesta.

Pista: envía `"GET / HTTP/1.1\r\nHost: www.example.com\r\nConnection: close\r\n\r\n"` con `sendall()`. Recibe en bucle con `recv(4096)` acumulando bytes hasta que devuelva `b""`, y entonces decodifica.

📚 Volver a la unidad · Resuelto: ✅ Boletín UD 5 — Inicial (Resuelto)

Boletín UD 5 — Inicial (Resuelto)
✅ Boletín UD 5 — Inicial (Resuelto)

1. Servidor saludo

```python
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.sendall(b"Bienvenido al servidor")
```

`accept()` espera un cliente (bloqueante). Al salir del `with`, la conexión se cierra solo.

2. Cliente que envía y recibe

```python
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Hola")
    respuesta = cli.recv(1024)
    print(f"Respuesta: {respuesta.decode()}")
```

`connect()` lanza `ConnectionRefusedError` si no hay servidor.

3. Servidor eco

```python
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos)  # Eco
```

Devuelve exactamente lo mismo que recibe.

4. IP y puerto

a) La **IP** identifica la **máquina** en la red; el **puerto** identifica el **programa** dentro de esa máquina.

b) **`127.0.0.1`** es **localhost**: tu propia máquina. Se usa para probar sin red real.

c) **TCP** → `socket.SOCK_STREAM`; **UDP** → `socket.SOCK_DGRAM`. Ambos con familia `AF_INET`.

5. Cliente UDP con entrada de usuario

```python
import socket

HOST = "127.0.0.1"
PORT = 9001

mensaje = input("Escribe tu mensaje: ")
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(mensaje.encode(), (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

El cliente UDP no tiene `connect()`: la dirección va dentro del `sendto()`, y la respuesta llega con `recvfrom()`.

6. Servidor UDP con eco personalizado

```python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")
    srv.sendto(f"Recibido: {datos.decode()}".encode(), direccion)
```

Se responde con la **dirección** que devuelve `recvfrom()`: sin ella no hay forma de contestar en UDP.

7. TCP vs UDP: clasifica

a) **TCP:** Web (HTTP), Correo (SMTP), Transferencia de archivos (FTP) → el dato debe llegar **completo y en orden**. **UDP:** Videollamada (Zoom), Juego online (Fortnite), DNS → la **velocidad** importa más; perder un paquete se tolera o se repite.

b)

Característica	TCP	UDP
Conexión	**Sí (handshake)**	**No**
Entrega garantizada	**Sí**	**No**
Orden	**Sí**	**No**
Velocidad	**Más lento**	**Más rápido**

8. Cliente HTTP manual

```python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("www.example.com", 80))
    peticion = (
        "GET / HTTP/1.1\r\n"
        "Host: www.example.com\r\n"
        "Connection: close\r\n"
        "\r\n"
    )
    s.sendall(peticion.encode())

    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

print(respuesta.decode()[:500])
```

HTTP es texto sobre TCP: mandas una petición con `sendall()` y recibes la respuesta en trozos hasta que `recv()` devuelve `b""` (el servidor cerró la conexión gracias a `Connection: close`).

📚 Volver a la unidad · Por resolver: 📝 Boletín UD 5 — Inicial

Boletín UD 5 — Avanzado

💪 Boletín UD 5 — Avanzado

> Ejercicios que requieren aplicar los conceptos de sockets TCP y UDP de forma más profunda, con programas completos: clientes interactivos, `select()`, timeouts, NTP y comparativas de protocolo.

---

1. Servidor que cuenta caracteres

Crea un servidor que reciba un texto y devuelva el número de caracteres (ej: “hola” → “4”).

Pista: recibe con `recv(1024).decode()`, calcula `len(texto)` y envía el resultado con `str(...).encode()`.

2. Cliente interactivo

Crea un cliente que pida texto por teclado con `input()`, lo envíe al servidor y muestre la respuesta. El bucle termina cuando el usuario escribe “salir”.

Pista: envuelve el `input()` + `sendall()` + `recv()` en un `while True` y rompe el bucle con `break` cuando el texto sea `"salir"`.

3. Servidor de mayúsculas

El cliente envía texto, el servidor lo devuelve en MAYÚSCULAS.

Pista: recibe los datos con `recv(1024).decode()`, aplica `.upper()` y envía el resultado con `sendall(...)`.

4. Cliente con reconexión

Cliente que intenta conectar, y si falla, reintenta hasta 3 veces con 2s de espera.

Pista: envuelve `socket.connect()` en un bucle `for` con `try/except`. Captura `ConnectionRefusedError` y `socket.timeout`, espera 2s con `time.sleep(2)` y reintenta.

5. Servidor que gestiona múltiples conexiones (sin hilos)

Usa `select.select()` para atender a varios clientes en un solo hilo.

Pista: configura el socket servidor como no bloqueante con `setblocking(False)`. `select.select()` te devuelve los sockets que tienen datos listos para leer. Si el socket listo es el servidor, acepta una nueva conexión; si es un cliente, recibe datos.

6. Timeout personalizado

Crea un servidor que cierre la conexión si el cliente no envía datos en 10 segundos.

Pista: después de `accept()`, llama a `conn.settimeout(10)`. Captura `socket.timeout` y envía un mensaje de despedida antes de cerrar.

7. Cliente NTP manual

Crea un cliente UDP que obtenga la hora actual desde `pool.ntp.org` usando el puerto 123. Envía un paquete de 48 bytes (el primero con valor `\x1b` y el resto `\0`). Extrae el timestamp de los bytes 40 a 43 con `struct.unpack('!I', ...)` y ajústalo restando 2208988800 para convertirlo a hora Unix.

Pista: añade `s.settimeout(5)` antes del `recvfrom()`: si la respuesta se pierde (UDP), lanza una excepción en lugar de bloquearse para siempre.

8. Servidor UDP multimensaje

Crea un servidor UDP que reciba y responda a 3 mensajes consecutivos en un bucle antes de cerrarse. Cada respuesta debe incluir el número de orden: `"OK #1"`, `"OK #2"`, `"OK #3"`.

Pista: usa `for i in range(1, 4)` en lugar de `while True`: así el servidor se cierra solo tras la tercera respuesta. El `with` libera el socket al salir.

9. Ping UDP

Cliente manda “PING”, servidor responde “PONG”. Mide cuánto tarda.

Pista: necesitas dos funciones (servidor y cliente) ejecutándose en paralelo. Usa `threading.Thread` con `daemon=True` para lanzar el servidor. Mide el tiempo con `time.time()` antes y después del intercambio de mensajes.

10. Servidor en todas las interfaces

El servidor escucha en todas las interfaces y responde a cualquiera.

Pista: el servidor debe escuchar en `"0.0.0.0"` para aceptar conexiones de cualquier interfaz. Usa un bucle infinito con `recvfrom()` y responde con `sendto()` a la dirección de cada cliente.

> Nota: no es *broadcast* (eso exigiría `SO_BROADCAST` y enviar a `255.255.255.255`); es un servidor que escucha en todas las interfaces.

11. Compara TCP y UDP

Escribe un programa que mida cuánto tarda en completar 10 intercambios de mensajes contra un servidor TCP y contra un servidor UDP en local. Compara los tiempos.

Pista: en TCP cada intercambio exige `connect()` (handshake); en UDP basta un `sendto()` + `recvfrom()`. Mide con `time.time()` antes y después de cada bucle.

12. Mini servidor web

Crea un servidor TCP que escuche en `127.0.0.1:8080`, acepte una conexión, lea la petición (ignorándola) y responda con `HTTP/1.1 200 OK` y un HTML con `"<h1>Hola mundo</h1>"`.

Pista: usa `SO_REUSEADDR` con `setsockopt()` para poder relanzar el servidor sin esperar. Abre `http://127.0.0.1:8080` en el navegador para verlo.

📚 Volver a la unidad · Resuelto: ✅ Boletín UD 5 — Avanzado (Resuelto)

Boletín UD 5 — Avanzado (Resuelto)
💪 Boletín UD 5 — Avanzado (Resuelto)

1. Servidor que cuenta caracteres

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        texto = conn.recv(1024).decode()
        conn.sendall(str(len(texto)).encode())
```

`len(texto)` cuenta los caracteres del texto recibido y se devuelve como texto (`str`) convertido a bytes con `.encode()`. Si el cliente envía `"hola"`, recibe `"4"`.

2. Cliente interactivo

```
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    while True:
        mensaje = input("Escribe (o 'salir'): ")
        if mensaje == "salir":
            break
        cli.sendall(mensaje.encode())
        respuesta = cli.recv(1024).decode()
        print(f"Respuesta: {respuesta}")
```

El `while True` mantiene el diálogo abierto. Cuando el usuario escribe `"salir"`, el `break` rompe el bucle y el `with` cierra el socket.

3. Servidor de mayúsculas

```python
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        texto = conn.recv(1024).decode()
        conn.sendall(texto.upper().encode())
```

`texto.upper()` convierte el texto recibido a MAYÚSCULAS y se devuelve como bytes con `.encode()`.

4. Cliente con reconexión

```python
import socket, time

host = "127.0.0.1"
port = 9000

for intento in range(3):
    try:
        with socket.socket() as cli:
            cli.settimeout(2)
            cli.connect((host, port))
            cli.sendall(b"test")
            print(f"Conectado en el intento {intento + 1}")
            break
    except (ConnectionRefusedError, socket.timeout):
        print(f"Intento {intento + 1} fallido, esperando 2s...")
        time.sleep(2)
else:
    print("Servidor no disponible tras 3 intentos")
```

El `for` reintenta hasta 3 veces. Cada `except` captura el fallo (rechazo o timeout) y espera 2s con `time.sleep(2)`. El `break` sale si se conecta; el `else` del `for` se ejecuta solo si nunca hubo `break`.

5. Servidor que gestiona múltiples conexiones (sin hilos)

```python
import socket, select

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 9000))
servidor.listen()
servidor.setblocking(False)  # no bloqueante

clientes = []
print("Servidor atendiendo hasta 3 clientes con select...")

while True:
    lectura, _, _ = select.select([servidor] + clientes, [], [], 1.0)

    for sock in lectura:
        if sock is servidor:
            conn, addr = servidor.accept()
            conn.setblocking(False)
            clientes.append(conn)
            print(f"Conectado {addr}, hay {len(clientes)} cliente(s)")
        else:
            datos = sock.recv(1024)
            if datos:
                sock.sendall(datos)  # eco
            else:
                clientes.remove(sock)
                sock.close()
```

El servidor **no bloqueante** más `select.select()` permite esperar a la vez en el socket servidor y en los sockets de los clientes. Cuando el socket listo es el servidor, se acepta una conexión nueva; cuando es un cliente, se leen sus datos. Sin hilos: un solo hilo atiende a todos.

6. Timeout personalizado

```python
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.settimeout(10)
        try:
            datos = conn.recv(1024)
            print(f"Recibido: {datos.decode()}")
        except socket.timeout:
            conn.sendall(b"Hasta luego")
            print("Cliente inactivo, cerrando conexión")
```

`conn.settimeout(10)` da al cliente 10 segundos para enviar datos. Si no lo hace, `recv()` lanza `socket.timeout` y el servidor se despide antes de cerrar.

7. Cliente NTP manual

```python
import socket, struct, time

def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        paquete = b'\x1b' + 47 * b'\0'
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)

    t = struct.unpack('!I', datos[40:44])[0]
    return t - 2208988800

hora = hora_ntp()
print(f"Hora NTP oficial: {time.ctime(hora)}")
```

El `settimeout(5)` evita que el `recvfrom()` se quede bloqueado si el datagrama de respuesta se pierde (muy UDP). El timestamp se desempaqueta de los bytes 40-43 con `struct.unpack('!I', ...)` y se ajusta restando **2208988800** segundos de época.

8. Servidor UDP multimensaje

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    for i in range(1, 4):
        datos, direccion = srv.recvfrom(1024)
        print(f"Mensaje {i} de {direccion}: {datos.decode()}")
        srv.sendto(f"OK #{i}".encode(), direccion)
```

Con `for i in range(1, 4)` el servidor responde a 3 mensajes y se cierra solo al salir del bucle (el `with` libera el socket). Cada respuesta lleva su número de orden.

9. Ping UDP

```
import socket, time, threading

def servidor():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
        srv.bind(("127.0.0.1", 9001))
        datos, direccion = srv.recvfrom(1024)
        srv.sendto(b"PONG", direccion)

threading.Thread(target=servidor, daemon=True).start()

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    inicio = time.time()
    cli.sendto(b"PING", ("127.0.0.1", 9001))
    datos, _ = cli.recvfrom(1024)
    fin = time.time()

print(f"PONG recibido en {fin - inicio:.4f} segundos")
```

El servidor se ejecuta en un hilo `daemon` mientras el cliente mide el tiempo de ida y vuelta. Ese tiempo es el **RTT** (round-trip time), la métrica de latencia de las redes.

10. Servidor en todas las interfaces

```python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind(("0.0.0.0", 9001))
    print("Servidor escuchando en todas las interfaces...")
    while True:
        datos, direccion = servidor.recvfrom(1024)
        print(f"Datagrama de {direccion}: {datos.decode()}")
        servidor.sendto(b"Recibido!", direccion)
```

`"0.0.0.0"` significa “cualquier interfaz”: el servidor acepta datagramas de cualquier cliente. El bucle infinito atiende a todos los que lleguen, uno tras otro, respondiendo con la dirección de cada uno. No es *broadcast* (eso exigiría `SO_BROADCAST` + envío a `255.255.255.255`), sino un servidor en todas las interfaces.

11. Compara TCP y UDP

```python
import socket, time
def test_tcp():
    t = time.time()
    for _ in range(10):
        with socket.socket() as s:
            s.connect(("127.0.0.1", 9000))
            s.send(b"x")
            s.recv(1024)
    return time.time() - t
def test_udp():
    t = time.time()
    for _ in range(10):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.sendto(b"x", ("127.0.0.1", 9001))
            s.recvfrom(1024)
    return time.time() - t
print(f"TCP: {test_tcp():.3f}s")
print(f"UDP: {test_udp():.3f}s")
```

UDP suele ser más rápido porque no tiene handshake: cada intercambio TCP paga un `connect()` (three-way handshake) que UDP se ahorra.

> ⚠️ Necesitas un servidor **TCP en el 9000** y un servidor **UDP en el 9001** en marcha (p. ej. los de los ejercicios anteriores). Sin ellos, TCP lanza `ConnectionRefusedError` y UDP se queda esperando una respuesta que nadie envía.

12. Mini servidor web

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 8080))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.recv(1024)  # Leer petición (la ignoramos)
            respuesta = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<h1>Hola
mundo</h1>"
        conn.sendall(respuesta.encode())
```

Abre http://127.0.0.1:8080 en tu navegador y verás “Hola mundo”. El `SO_REUSEADDR` te permite relanzar el servidor sin esperar a que el puerto se libere.

📚 Volver a la unidad · Por resolver: 💪 Boletín UD 5 — Avanzado

Boletín UD 6 — Inicial

📝 Boletín UD 6 — Inicial

> Ejercicios básicos para afianzar los conceptos de servidores concurrentes: sockets TCP, el servidor secuencial y sus límites, y el salto a los hilos.

---

1. Servidor eco básico

Crea un servidor TCP que reciba datos de un cliente y los devuelva exactamente igual (eco). Sin bucle, atiende un solo cliente y termina.

2. Servidor eco con bucle

Modifica el servidor anterior para que acepte clientes en un bucle infinito, dando eco a cada uno.

3. Cliente eco

Crea un cliente TCP que se conecte al servidor, envíe `b"Hola eco"` y muestre la respuesta recibida.

4. Servidor secuencial

Crea un servidor TCP que atienda a **un solo cliente** y termine: `accept()`, recibe, responde `b"OK"` y cierra.

Pista: es la versión mínima del servidor secuencial: `with socket.socket() as srv:` → `bind(("127.0.0.1", 5000))` → `listen()` → `accept()` → `with conn:` → `recv()` y `sendall(b"OK")`.

5. Servidor multihilo

Convierte el servidor anterior en uno **multihilo**: dentro de un `while True`, cada cliente que acepte se atiende en su propio `threading.Thread`.

Pista: define `def atender(conn, addr):` con el eco, y en el bucle lanza `threading.Thread(target=atender, args=(conn, addr)).start()`. El bucle vuelve al `accept()` al instante.

6. Cliente con respuesta

Crea un cliente TCP que se conecte al servidor, envíe `b"Hola"` y muestre por pantalla la respuesta que reciba.

7. Servidor con ThreadPoolExecutor

Implementa un servidor TCP que use `concurrent.futures.ThreadPoolExecutor` con **3 hilos** (`max_workers=3`). Responde `b"OK"` a cada cliente.

Pista: `with socket.socket() as srv, ThreadPoolExecutor(max_workers=3) as pool:` y en el bucle `pool.submit(atender, conn, addr)`. Con 3 hilos, el 4o cliente espera en cola.

8. Lanzador de 5 clientes

Crea un script que lance **5 clientes simultáneos** contra `127.0.0.1:5000`, cada uno enviando `f"Soy {id}"`, y muestre la respuesta de cada uno.

Pista: haz una lista de hilos `[threading.Thread(target=cliente, args=(i,)) for i in range(5)]`, lánzalos con `h.start()` y espera a todos con `h.join()`. La función `cliente(id)` conecta, envía y muestra `s.recv(1024).decode()`.

Boletín UD 6 — Inicial (Resuelto)
✅ Boletín UD 6 — Inicial (Resuelto)

1. Servidor eco básico

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos)  # eco: devuelve exactamente lo recibido
```

Solo atiende **UN cliente** y termina. `with conn:` cierra el socket al salir.

2. Servidor eco con bucle

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        with conn:
            datos = conn.recv(1024)
            conn.sendall(datos)  # eco
```

Clientes uno tras otro. Si uno tarda, los demás esperan: es el servidor secuencial del punto 1.

3. Cliente eco

```python
import socket
with socket.socket() as s:
    s.connect(("127.0.0.1", 5000))
    s.sendall(b"Hola eco")
    print(s.recv(1024).decode())
```

Se conecta, envía `b"Hola eco"` y muestra la respuesta del servidor (el mismo mensaje, de vuelta).

4. Servidor secuencial

```python
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK")
```

Solo atiende UN cliente y termina: `accept()` una vez, responde `b"OK"` y cierra.

5. Servidor multihilo

```python
import socket, threading
def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Cada cliente en su propio hilo. Todos se atienden en paralelo (punto 3).

6. Cliente con respuesta

```
import socket
with socket.socket() as s:
    s.connect(("127.0.0.1", 5000))
    s.sendall(b"Hola")
    print(s.recv(1024).decode())
```

Muestra por pantalla la respuesta del servidor. Con el servidor multihilo del ejercicio 5, recibiría `OK`.

7. Servidor con ThreadPoolExecutor

```
import socket, concurrent.futures
def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
with socket.socket() as srv, concurrent.futures.ThreadPoolExecutor(3) as pool:
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        pool.submit(atender, conn, addr)
```

Máximo **3 hilos**. Los clientes adicionales esperan en cola (punto 4).

8. Lanzador de 5 clientes

```
import socket, threading
def cliente(id):
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(f"Soy {id}".encode())
        print(s.recv(1024).decode())
hilos = [threading.Thread(target=cliente, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Lanza **5 clientes a la vez** contra el servidor. El `join()` espera a que terminen todos antes de acabar el script.

Boletín UD 6 — Avanzado

💪 Boletín UD 6 — Avanzado

> Ejercicios que requieren aplicar la concurrencia de forma más profunda: timeouts, contadores sincronizados, límites de clientes, colas de espera, heartbeats y hasta un balanceador de carga. Los tres últimos llevan procesos (`Process`, `Manager` y `Lock`) para atender a los clientes; verás el patrón completo en el punto 9 de la UD 6.

---

1. Servidor con timeout

Implementa un servidor que cierre la conexión si el cliente no envía datos en 5 segundos (usa `socket.settimeout` o `select`).

2. Cliente con timeout

Crea un cliente que intente conectarse a `127.0.0.1:9999` con timeout de 2 segundos. Captura la excepción `socket.timeout` y muestra "Servidor no disponible".

3. Contador de bytes totales

Añade un contador global de bytes recibidos (protegido con Lock) al servidor multihilo. Cada vez que un cliente envía datos, suma los bytes y muestra el total acumulado.

Pista: `total_bytes = 0` + `lock = threading.Lock()`. Dentro de `atender`, haz `with lock:` para hacer `total_bytes += len(datos)` y `print`. El `recv()` y `sendall()` no necesitan Lock (cada socket es independiente).

4. Servidor con límite de clientes

Crea un servidor que acepte máximo 3 clientes. El cuarto recibe "Servidor completo" y se cierra.

Pista: Usa una variable global `clientes_activos` protegida con `Lock`. Al aceptar un cliente, comprueba si ya se alcanzó el máximo; si es así, envía el mensaje y cierra sin incrementar el contador. Decrementa al terminar.

5. Prueba de carga

Lanza 20 clientes simultáneos contra tu servidor y mide cuánto tardan todos en recibir respuesta.

Pista: Usa un diccionario `resultados` compartido para guardar el tiempo de cada cliente. Cada cliente mide `time.time()` antes y después de la conexión. Lanza 20 hilos, haz `join` a todos y calcula estadísticas (total, media, exitosos).

6. Servidor con cola de espera

Cuando el pool está lleno, los clientes entran en una cola. Cuando un hilo se libera, atiende al siguiente.

Pista: Usa `queue.Queue` para encolar las conexiones aceptadas. Crea N hilos trabajadores (daemon) que saquen elementos de la cola con `cola.get()` y atiendan al cliente. El hilo principal solo acepta y encola.

7. Estado del servidor

Añade un endpoint especial: si el cliente envía “STATUS”, el servidor responde con número de conexiones activas.

Pista: Mantén un contador `activas` con Lock. En la función de atención, parsea el comando: si es `"STATUS"`, responde con el valor actual del contador; si no, responde `"OK"`. Incrementa al entrar, decrementa al salir.

8. Heartbeat en servidor

El servidor tiene un hilo heartbeat que imprime “💓 Servidor vivo — N conexiones” cada 5s.

Pista: Crea un hilo `daemon=True` con un bucle infinito que haga `time.sleep(5)` y luego imprima el estado usando el Lock para leer el contador de conexiones activas.

9. Balanceador de carga simple

Crea un “balanceador” que recibe peticiones y las distribuye entre 2 servidores workers.

Pista: Crea dos workers en los puertos 5001 y 5002 (cada uno en su hilo). El balanceador en el puerto 5000 acepta conexiones y las reenvía al worker actual haciendo de proxy: lee del cliente, envía al worker, recibe la respuesta y la reenvía al cliente. Alterna entre workers con un índice round-robin.

---

10. Servidor con un proceso por cliente

Convierte el servidor multihilo de los ejercicios anteriores en uno donde cada conexión la atienda un `multiprocessing.Process` en lugar de un hilo.

Pista: sustituye `threading.Thread(target=atender, args=(conn, addr))` por `multiprocessing.Process(target=atender, args=(conn, addr))` y recuerda el guard `if __name__ == "__main__":` (obligatorio en Windows, porque cada hijo vuelve a importar el script). El `accept()` sigue en el proceso padre.

11. Contador de conexiones entre procesos

El contador global del ejercicio 3 ya no sirve: cada proceso hijo tiene su propia copia de la memoria. Mueve el contador a un `manager.dict()` protegido con `multiprocessing.Lock` para que todos los procesos sumen en el mismo total.

Pista: crea `manager.dict({"activas": 0})` y `multiprocessing.Lock()` dentro del `with multiprocessing.Manager()` del padre, y pásalos como argumentos extra al `Process`. Incrementa dentro de `with candado:` antes del bucle de lectura y decrementa al salir.

12. Aislamiento de fallos

Haz que un cliente pueda provocar un error dentro del proceso hijo (por ejemplo, enviando “CRASH” para lanzar una excepción) y demuestra que el servidor **sigue vivo** atendiendo al resto. Añade la recogida de hijos para no dejar procesos zombis.

Pista: envuelve el tratamiento del mensaje en `try/except` dentro de `atender()`; si la excepción escapa, el proceso hijo muere con su traceback pero el bucle `accept()` del padre continúa. Guarda los `Process` en una lista y repasa con `p.is_alive()` / `p.join()` de vez en cuando.

Boletín UD 6 — Avanzado (Resuelto)
💪 Boletín UD 6 — Avanzado (Resuelto)

1. Servidor con timeout

```python
import socket

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print("🕐 Servidor con timeout de 5 segundos")
    while True:
        conn, addr = srv.accept()
        conn.settimeout(5)          # ⏰ 5 segundos máximos para recv
        with conn:
            try:
                datos = conn.recv(1024)
                print(f"  Recibido de {addr}: {datos.decode()}")
            except socket.timeout:
                print(f"  ⏰ {addr} no envió nada en 5s → cierro conexión")
```

`conn.settimeout(5)` limita el `recv()`: si el cliente no envía datos en 5 segundos, salta `socket.timeout` y el `with conn:` cierra el socket (punto 7). Un cliente mudo ya no cuelga un hilo para siempre.

2. Cliente con timeout

```
import socket

try:
    with socket.socket() as s:
        s.settimeout(2)              # ⏰ 2 segundos para conectar y recibir
        s.connect(("127.0.0.1", 9999))
        s.sendall(b"ping")
        print(s.recv(1024))
except (socket.timeout, ConnectionRefusedError):
    print("Servidor no disponible")
```

Al conectar a un puerto en el que nada escucha, o al no recibir respuesta, se captura `socket.timeout` (Windows) o `ConnectionRefusedError` (Linux) y se muestra “Servidor no disponible” en lugar de un fallo sin control.

3. Contador de bytes totales

```
import socket, threading

total_bytes = 0
lock = threading.Lock()

def atender(conn, addr):
    global total_bytes
    with conn:
        datos = conn.recv(1024)
    with lock:                                  # 🔒 actualización atómica
        total_bytes += len(datos)
        print(f"  {addr} envió {len(datos)} bytes — Total: {total_bytes}")

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()
```

El `recv()` queda **fuera** del Lock (cada socket es independiente) y solo la suma al contador global entra en `with lock:`: así varios hilos suman bytes sin condición de carrera (punto 6).

4. Servidor con límite de clientes

```python
import socket, threading

MAX_CLIENTES = 3
clientes_activos = 0
lock = threading.Lock()

def atender(conn, addr):
    global clientes_activos
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
    with lock:
        clientes_activos -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print(f"🚦 Máximo {MAX_CLIENTES} clientes a la vez")
    while True:
        conn, addr = srv.accept()
        with lock:
            if clientes_activos >= MAX_CLIENTES:
                try:
                    conn.sendall(b"Servidor completo")
                    conn.shutdown(socket.SHUT_WR)   # asegura que el mensaje llega
                except (ConnectionResetError, BrokenPipeError):
                    pass
                conn.close()
                print(f"  ❌ {addr} rechazado: servidor completo")
                continue
            clientes_activos += 1
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Al aceptar se comprueba el máximo bajo `with lock:`. El cuarto cliente recibe `"Servidor completo"` y se cierra **sin incrementar** el contador; los que entran lo decrementan al terminar.

> ⚠️ La garantía de “máximo 3” es real mientras los clientes **mantienen la conexión abierta** (el contador no baja). Si los clientes conectan, envían y cierran muy rápido, el cuarto puede

> colarse porque los primeros ya decrementaron antes de que llegue; para un aforo estricto, el contador debe reflejar conexiones activas sostenidas.

5. Prueba de carga

```
import socket, threading, time

resultados = {}
lock = threading.Lock()

def cliente(id):
    inicio = time.time()
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(b"ping")
            s.recv(1024)
        exito = True
    except Exception:
        exito = False
    fin = time.time()
    with lock:
        resultados[id] = (exito, fin - inicio)

hilos = [threading.Thread(target=cliente, args=(i,)) for i in range(20)]
for h in hilos: h.start()
for h in hilos: h.join()

exitosos = sum(1 for ok, _ in resultados.values() if ok)
tiempos = [t for ok, t in resultados.values() if ok]
media = sum(tiempos) / len(tiempos) if tiempos else 0
print(f"Exitosos: {exitosos}/20 | Total: {sum(tiempos):.2f}s | Media: {media:.2f}s")
```

Cada cliente guarda su resultado (éxito + duración) en un diccionario compartido protegido por Lock. Tras `join()` a todos, se calculan las estadísticas: es el **benchmark** del punto 5 llevado a 20 clientes.

6. Servidor con cola de espera

```python
import socket, threading, queue

COLA = queue.Queue()

def atender():
    while True:
        conn, addr = COLA.get()          # 🔔 espera a que haya trabajo
        with conn:
            datos = conn.recv(1024)
            conn.sendall(b"OK: " + datos)
            print(f"  Atendido {addr}")
        COLA.task_done()

N_TRABAJADORES = 3
for _ in range(N_TRABAJADORES):
    threading.Thread(target=atender, daemon=True).start()

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    print("🔔 Cola de espera con 3 trabajadores")
    while True:
        conn, addr = srv.accept()
        COLA.put((conn, addr))           # 📦 encola la conexión
```

El hilo principal solo **acepta y encola**. Los N trabajadores (daemon) sacan conexiones con `COLA.get()` y las atienden: es un ThreadPool hecho a mano con `queue.Queue`, el mecanismo interno de los pools del punto 4.

7. Estado del servidor

```python
import socket, threading

activas = 0
lock = threading.Lock()

def atender(conn, addr):
    global activas
    with conn:
        comando = conn.recv(1024).decode().strip()
        if comando == "STATUS":
            with lock:
                conn.sendall(f"Activas: {activas}".encode())
        else:
            conn.sendall(b"OK")
    with lock:
        activas -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        with lock:
            activas += 1
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Un cliente que envíe `"STATUS"` recibe el número de conexiones activas (el resto recibe `"OK"`). El contador se incrementa al aceptar y se decrementa al salir, siempre bajo Lock.

8. Heartbeat en servidor

```python
import socket, threading, time

activas = 0
lock = threading.Lock()

def heartbeat():
    while True:
        time.sleep(5)
        with lock:
            print(f"💓 Servidor vivo — {activas} conexiones")

threading.Thread(target=heartbeat, daemon=True).start()

def atender(conn, addr):
    global activas
    with lock:
        activas += 1
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
    with lock:
        activas -= 1

with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()
```

El hilo heartbeat es **daemon** (muere con el servidor) y cada 5s imprime el estado leyendo el contador bajo Lock. Es la semilla del *heartbeat* de disponibilidad que ampliarás en la UD 10 · asyncio.

9. Balanceador de carga simple

```python
import socket, threading

PUERTOS_WORKERS = [5001, 5002]
indice = 0

def worker(puerto):
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", puerto))
        srv.listen()
        print(f"  ⚙️  Worker escuchando en {puerto}")
        while True:
            conn, addr = srv.accept()
            with conn:
                datos = conn.recv(1024)
                conn.sendall(f"Worker-{puerto}: ".encode() + datos)

for puerto in PUERTOS_WORKERS:
    threading.Thread(target=worker, args=(puerto,), daemon=True).start()

def balanceador():
    global indice
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("⚖️  Balanceador en 127.0.0.1:5000 (round-robin)")
        while True:
            conn, addr = srv.accept()
            puerto = PUERTOS_WORKERS[indice % len(PUERTOS_WORKERS)]
            indice += 1
            with conn:
                datos = conn.recv(1024)
                with socket.socket() as worker_s:
                    worker_s.connect(("127.0.0.1", puerto))
                    worker_s.sendall(datos)
                    respuesta = worker_s.recv(1024)
                conn.sendall(respuesta)
            print(f"  {addr} → worker {puerto}")

balanceador()
```

El balanceador (puerto 5000) hace de **proxy**: lee del cliente, reenvía al worker elegido por **round-robin** (`indice % 2`), recibe su respuesta y la devuelve al cliente. Los workers 5001 y 5002 son servidores normales, cada uno en su hilo. Los clientes que envíen varias peticiones verán cómo se alterna la respuesta `Worker-5001`/`Worker-5002`.

---

10. Servidor con un proceso por cliente

```
import socket
import multiprocessing

def atender(conn, addr):
    """Proceso hijo: atiende a UN cliente con su propia memoria."""
    with conn:
        try:
            datos = conn.recv(1024)
            print(f"  [pid {multiprocessing.current_process().pid}] "
                  f"Recibido de {addr}: {datos.decode().strip()}")
            conn.sendall(b"OK: " + datos)
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"  [pid {multiprocessing.current_process().pid}] "
                  f"Cliente {addr} se desconectó a mitad")

def servidor():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🏭  Servidor por PROCESOS escuchando en 127.0.0.1:5000")
        while True:
            conn, addr = srv.accept()
            # En lugar de Thread: un proceso independiente
            p = multiprocessing.Process(target=atender, args=(conn, addr))
            p.start()
            # El padre cierra SU copia: el hijo tiene la suya (evita colgones)
            conn.close()

if __name__ == "__main__":
    servidor()
```

El cambio es minimalista —`Process` en vez de `Thread`— pero **el guard `if __name__ == "__main__":` pasa a ser obligatorio**: en Windows cada hijo vuelve a importar el script desde el principio y, sin el guard, entraría en un bucle infinito creando procesos. Fíjate en que `accept()` sigue en el padre: él reparte trabajo y nunca se bloquea leyendo datos.

11. Contador de conexiones entre procesos

```python
import socket
import multiprocessing

def atender(conn, addr, estado, candado):
    with conn:
        try:
            # Sección crítica: leer, sumar y escribir DE UNA VEZ
            with candado:
                estado["activas"] = estado.get("activas", 0) + 1
                total = estado["activas"]
            datos = conn.recv(1024)
            conn.sendall(f"OK [{total}]: {datos}".encode())
            with candado:
                estado["activas"] -= 1
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"    Cliente {addr} se desconectó a mitad")

def servidor():
    with multiprocessing.Manager() as manager:
        estado = manager.dict({"activas": 0})
        candado = multiprocessing.Lock()
        with socket.socket() as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            srv.bind(("127.0.0.1", 5000))
            srv.listen()
            print("📐  Contador de conexiones entre procesos")
            while True:
                conn, addr = srv.accept()
                p = multiprocessing.Process(
                    target=atender, args=(conn, addr, estado, candado))
                p.start()
                conn.close()  # El padre cierra su copia; el hijo tiene la suya

if __name__ == "__main__":
    servidor()
```

Por qué no valía el contador global: cada proceso hijo **clona la memoria** al arrancar, así que su `contador += 1` afectaría solo a su copia y el total nunca subiría. El `manager.dict()` es el que sí está en todos, y el `multiprocessing.Lock()` garantiza que la lectura-suma-escritura no se pisa entre procesos. Es el mismo patrón del punto 9 de la UD 6.

12. Aislamiento de fallos

```python
import socket
import multiprocessing
import traceback

def atender(conn, addr, estado, candado):
    with conn:
        try:
            datos = conn.recv(1024)
            mensaje = datos.decode().strip()

            if mensaje == "CRASH":
                raise ValueError("Fallo provocado a propósito")

            with candado:
                estado["atendidos"] = estado.get("atendidos", 0) + 1
                total = estado["atendidos"]
            conn.sendall(f"OK [{total}]: {mensaje}".encode())
        except ValueError:
            # El error queda DENTRO del hijo: imprimimos y morimos
            print(f"  [pid {multiprocessing.current_process().pid}] "
                  f"¡Excepción manejada en el hijo!")
            traceback.print_exc()
        except (ConnectionResetError, BrokenPipeError, OSError):
            print(f"    Cliente {addr} se desconectó a mitad")

def servidor():
    with multiprocessing.Manager() as manager:
        estado = manager.dict({"atendidos": 0})
        candado = multiprocessing.Lock()
        hijos = []
        with socket.socket() as srv:
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            srv.bind(("127.0.0.1", 5000))
            srv.listen()
            print("  Servidor resistente a fallos")
            while True:
                conn, addr = srv.accept()
                p = multiprocessing.Process(
                    target=atender, args=(conn, addr, estado, candado))
                p.start()
                conn.close()  # El padre cierra su copia; el hijo tiene la suya
                hijos.append(p)
```

```
                # Recoger los hijos que ya terminaron (evita zombis):
                # primero join a los muertos, luego filtrar la lista
                for h in hijos:
                    if not h.is_alive():
                        h.join()
                hijos = [h for h in hijos if h.is_alive()]

if __name__ == "__main__":
    servidor()
```

La demo: arranca el servidor, envía `CRASH` con un cliente y después otro mensaje normal. Verás el traceback **dentro del proceso hijo** y, sin embargo, el siguiente cliente recibe su `OK` con normalidad: el hijo murió, el padre no se enteró. La recogida con `is_alive()`/`join()` evita que los procesos terminados se acumulen como zombis en la tabla de procesos.

Boletín UD 7 — Inicial

📝 Boletín UD 7 — Inicial

> Ejercicios básicos para afianzar los conceptos de APIs REST, métodos HTTP, códigos de estado y JSON de la unidad UD 7.

---

1. GET a JSONPlaceholder

Usa `requests.get` para obtener el post con ID 1 de `https://jsonplaceholder.typicode.com/posts/1`. Muestra el `status_code`.

2. Título del post

Del ejercicio anterior, muestra el campo `"title"` del post obtenido usando `resp.json()`.

3. Contar usuarios

Haz GET a `https://jsonplaceholder.typicode.com/users` y muestra cuántos usuarios hay (la longitud de la lista).

Pista: `resp.json()` devuelve una lista de diccionarios; su longitud es el número de usuarios.

4. GET a GitHub

Usa `requests.get` para obtener los datos del usuario `python` de `https://api.github.com/users/python`. Muestra el `status_code`.

5. Mostrar JSON

Del ejercicio anterior, muestra la respuesta completa con `print(resp.json())`.

6. Nombre real

Del mismo usuario, muestra el campo `"name"` (su nombre real) accediendo al diccionario.

7. Métodos HTTP

Relaciona cada método HTTP con su operación CRUD y su uso:

Método	Operación CRUD	Uso
GET	___	___
POST	___	___
PUT	___	___
PATCH	___	___
DELETE	___	___

Pista: CRUD = Create, Read, Update, Delete. Recuerda que PUT reemplaza el recurso completo y PATCH solo una parte.

8. Códigos de estado

Completa el significado de estos códigos de estado HTTP:

a) 200 → ___ b) 201 → ___ c) 204 → ___ d) 400 → ___ e) 401 → ___ f) 404 → ___ g) 500 → ___

Pista: 2xx es éxito, 4xx es error del cliente, 5xx es error del servidor.

Boletín UD 7 — Inicial (Resuelto)
✅ Boletín UD 7 — Inicial (Resuelto)

1. GET a JSONPlaceholder

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code)  # 200
```

`requests.get` no lanza excepción ante errores: el código de estado te lo dice `resp.status_code`.

2. Título del post

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/posts/1")
print(resp.json()["title"])
```

`resp.json()` convierte el cuerpo JSON a un **dict** de Python, y con `["title"]` accedes al campo.

3. Contar usuarios

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/users")
usuarios = resp.json()
print(f"Hay {len(usuarios)} usuarios")  # 10
```

Cuando la respuesta es una lista de recursos, `resp.json()` devuelve una **lista** de diccionarios y `len()` cuenta los elementos.

4. GET a GitHub

```python
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.status_code)  # 200
```

5. Mostrar JSON

```python
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.json())
```

`resp.json()` convierte el JSON a diccionario Python.

6. Nombre real

```python
import requests
datos = requests.get("https://api.github.com/users/python").json()
print(datos["name"])  # "Python"
```

Encadenas `requests.get(...).json()` para ir directo al dict, y con `datos["name"]` sacas el nombre real.

7. Métodos HTTP

Método	Operación CRUD	Uso
GET	**Read**	Obtener datos (sin cuerpo)
POST	**Create**	Crear recurso (con cuerpo)
PUT	**Update**	Reemplazar el recurso completo
PATCH	**Update**	Actualizar solo una parte
DELETE	**Delete**	Borrar el recurso

8. Códigos de estado

a) 200 → **OK** (GET exitoso) b) 201 → **Created** (POST exitoso, recurso creado) c) 204 → **No Content** (DELETE exitoso) d) 400 → **Bad Request** (enviaste datos incorrectos) e) 401 → **Unauthorized** (falta autenticación) f) 404 → **Not Found** (el recurso no existe) g) 500 → **Internal Server Error** (error del servidor)

> Los **2xx** son éxito, los **4xx** son error del cliente (tuyo) y los **5xx** son error del servidor.

Boletín UD 7 — Avanzado

💪 Boletín UD 7 — Avanzado

> Ejercicios que requieren aplicar los conceptos de APIs REST, requests y JSON de forma más profunda, con programas completos contra APIs reales.

---

1. PUT para actualizar

Haz PUT a `https://jsonplaceholder.typicode.com/posts/1` con `json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}`. Muestra el código de estado y el título actualizado.

Pista: `requests.put` con `json=` reemplaza el recurso completo. El título actualizado está en `resp.json()["title"]`.

2. DELETE un recurso

Haz DELETE a `https://jsonplaceholder.typicode.com/posts/1`. ¿Qué código de estado devuelve un borrado exitoso?

3. HEAD request

Usa `requests.head()` para pedir solo las cabeceras de `https://api.github.com`. Muestra los valores de `Content-Type` y `X-RateLimit-Remaining` de las cabeceras.

Pista: `requests.head` no devuelve cuerpo, solo cabeceras. Accede a ellas con `resp.headers["Content-Type"]`.

4. 🎯 Info de usuario de GitHub

Obtén los datos del usuario “python” y muestra: nombre, bio, repos públicos, seguidores, fecha de creación.

Pista: La API de GitHub (`https://api.github.com/users/python`) devuelve un JSON con campos como `name`, `bio`, `public_repos`, `followers` y `created_at`. Usa `resp.json()` para acceder a

cada campo.

5. 🔍 Buscador de repositorios

Busca repositorios de Python con más de 1000 estrellas. Muestra nombre y estrellas.

Pista: La API de búsqueda de GitHub (`/search/repositories`) acepta `q` como parámetro. Usa `"python stars:>1000"` para filtrar. Ordena por `stars` con `sort=stars` y limita a 5 resultados con `per_page=5`.

6. 🧩 API con paginación

La API de GitHub página los resultados. Obtén los primeros 20 repos del usuario “python” (2 páginas de 10).

Pista: Usa los parámetros `page` y `per_page` en cada petición a `https://api.github.com/users/{usuario}/repos`. Itera sobre las páginas que necesites y acumula los resultados en una lista.

7. 🎭 POST y respuesta

Crea un post en JSONPlaceholder, luego haz GET para verificarlo.

Pista: JSONPlaceholder es una API de pruebas. El POST devuelve el recurso creado con un ID (101). Luego puedes hacer GET a `https://jsonplaceholder.typicode.com/posts/101` para verificar la creación.

8. ⏱️ Tiempo de respuesta

Mide el tiempo de respuesta de 5 APIs distintas. ¿Cuál es más rápida?

Pista: Mide el tiempo con `time.time()` antes y después de cada `requests.get()`. Prueba varias APIs gratuitas (GitHub, JSONPlaceholder, httpbin, ReqRes) y compara los tiempos en milisegundos.

9. 🧪 Cliente de API con caché

Crea un cliente que cachee respuestas en un diccionario para no repetir peticiones.

Pista: Usa un diccionario global donde la clave sea la URL. Guarda el dato y el timestamp. Antes de hacer una petición, comprueba si la URL está en caché y si el tiempo transcurrido desde que se guardó es menor al TTL.

Boletín UD 7 — Avanzado (Resuelto)
💪 Boletín UD 7 — Avanzado (Resuelto)

1. PUT para actualizar

```
import requests
resp = requests.put(
    "https://jsonplaceholder.typicode.com/posts/1",
    json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}
)
print(resp.status_code)            # 200 OK
print(resp.json()["title"])        # "nuevo titulo"
```

`requests.put` con `json=` reemplaza el recurso **completo**. PUT es **idempotente**: repetirlo deja el recurso en el mismo estado.

2. DELETE un recurso

```
import requests
resp = requests.delete("https://jsonplaceholder.typicode.com/posts/1")
print(resp.status_code)            # 200 OK
```

JSONPlaceholder responde **200** con cuerpo vacío. En REST canónico, un borrado exitoso devuelve **204 No Content** (sin cuerpo que leer). DELETE también es **idempotente**: da igual cuántas veces lo llames, el recurso ya no está.

3. HEAD request

```
import requests
resp = requests.head("https://api.github.com")
print(resp.headers["Content-Type"])            # "application/json; charset=utf-8"
print(resp.headers["X-RateLimit-Remaining"])   # 60
```

`requests.head` solo pide **cabeceras**, sin cuerpo: es barato y sirve para comprobar que una API responde o consultar los límites de tasa.

4. 🎯 Info de usuario de GitHub

```
import requests
datos = requests.get("https://api.github.com/users/python").json()

print(f"Nombre: {datos['name']}")
print(f"Bio: {datos['bio']}")
print(f"Repos públicos: {datos['public_repos']}")
print(f"Seguidores: {datos['followers']}")
print(f"Creado: {datos['created_at']}")
```

`resp.json()` te da el dict completo y cada campo se accede por su clave (`name`, `bio`, `public_repos`, `followers`, `created_at`).

5. 🔍 Buscador de repositorios

```
import requests
params = {"q": "python stars:>1000", "sort": "stars", "per_page": 5}
resp = requests.get("https://api.github.com/search/repositories", params=params)

for repo in resp.json()["items"]:
    print(repo["full_name"], "★", repo["stargazers_count"])
```

`params` construye la query string automáticamente. La búsqueda devuelve `items`, una lista de repositorios, y de cada uno sacas `full_name` y `stargazers_count`.

6. 🧩 API con paginación

```python
import requests
repos = []

for page in (1, 2):
    params = {"page": page, "per_page": 10}
    resp = requests.get("https://api.github.com/users/python/repos", params=params)
    repos.extend(resp.json())

print(f"Repos obtenidos: {len(repos)}")   # 20
for repo in repos:
    print(repo["name"])
```

Con `page` y `per_page` recorres la paginación: cada petición devuelve 10 repos y `extend()` los acumula en la lista. Es la técnica que usan los clientes reales para traer conjuntos completos.

7. 🎭 POST y respuesta

```python
import requests

nuevo = {"title": "Hola API", "body": "Probando POST", "userId": 1}
resp = requests.post("https://jsonplaceholder.typicode.com/posts", json=nuevo)
print(resp.status_code)          # 201 Created
print("ID:", resp.json()["id"])  # 101

verificacion = requests.get("https://jsonplaceholder.typicode.com/posts/101")
print(verificacion.json()["title"])   # "Hola API"
```

El POST devuelve **201 Created** con el recurso creado (ID 101). El GET posterior a ese ID confirma que la creación ha sido real.

8. ⏱️ Tiempo de respuesta

```python
import requests, time

apis = {
    "GitHub": "https://api.github.com",
    "JSONPlaceholder": "https://jsonplaceholder.typicode.com/posts/1",
    "httpbin": "https://httpbin.org/get",
    "ReqRes": "https://reqres.in/api/users/1",
    "OpenNotify": "http://api.open-notify.org/iss-now.json",
}

for nombre, url in apis.items():
    inicio = time.time()
    requests.get(url)
    fin = time.time()
    ms = (fin - inicio) * 1000
    print(f"{nombre}: {ms:.0f} ms")
```

`time.time()` antes y después de cada `requests.get()` te da los milisegundos. La más rápida cambia según la red del momento, pero siempre es la que menos distancia recorre y mejor responde. Es la comparativa que luego usarás en la UD 10 con asyncio.

9. ⏳ Cliente de API con caché

```python
import requests, time

cache = {}
TTL = 60  # segundos de vida de cada entrada

def obtener(url):
    ahora = time.time()
    if url in cache:
        dato, timestamp = cache[url]
        if ahora - timestamp < TTL:
            print(f"📦 Desde caché: {url}")
            return dato
    print(f"🌐 Petición real: {url}")
    dato = requests.get(url).json()
    cache[url] = (dato, ahora)
    return dato

print(obtener("https://jsonplaceholder.typicode.com/posts/1"))
print(obtener("https://jsonplaceholder.typicode.com/posts/1"))  # 📦 Desde caché
```

El diccionario usa la **URL como clave** y guarda `(dato, timestamp)`. Antes de pedir, se comprueba si la URL está en caché y si su timestamp sigue dentro del TTL. La segunda llamada no sale a la red: responde al instante desde el diccionario.

Boletín UD 8 — Inicial

📝 Boletín UD 8 — Inicial

> Ejercicios básicos para afianzar los conceptos de API keys, variables de entorno, OpenWeatherMap y errores HTTP de la unidad UD 8.

---

1. Humedad actual

Usa OpenWeatherMap para obtener el clima de tu ciudad. Muestra la humedad (`humidity`) del campo `main`.

2. Descripción del clima

Del mismo JSON, muestra la descripción del tiempo (campo `description` dentro de `weather`).

3. Temperatura mínima y máxima

Muestra la temperatura mínima (`temp_min`) y máxima (`temp_max`) del pronóstico actual de tu ciudad.

4. Temperatura y sensación térmica

Muestra la temperatura actual (`temp`) y la sensación térmica (`feels_like`) de tu ciudad en grados centígrados.

Pista: recuerda el parámetro `units=metric` para que la API devuelva grados en lugar de kelvin.

5. Error 401

Haz una petición a OpenWeatherMap con una API key falsa (`appid="falsa"`). ¿Qué código de estado devuelve?

6. ¿Dónde va la API key?

a) ¿En qué parámetro de la query string espera OpenWeatherMap la clave? b) ¿En qué cabecera y con qué esquema espera OpenAI la clave?

7. Clave desde el `.env`

Crea un archivo `.env` con `OPENWEATHER_API_KEY=test123`. Usa `python-dotenv` para cargarlo y muestra la clave con `os.getenv`.

Pista: `load_dotenv()` primero, `os.getenv("OPENWEATHER_API_KEY")` después.

8. Protege el repo

a) Escribe la línea del `.gitignore` que evita subir el archivo `.env` a git. b) ¿Qué devuelve `os.getenv("MI_CLAVE")` si la variable no existe?

Boletín UD 8 — Inicial (Resuelto)
✅ Boletín UD 8 — Inicial (Resuelto)

> ⚠️ **Sobre las claves:** por claridad, estas soluciones usan `API_KEY = "TU_API_KEY"` como marcador para centrarse en el parseo de la respuesta. Recuerda la regla de oro de la unidad: en un programa real la clave **nunca va en el código**, se lee de un `.env` con `load_dotenv()` + `os.getenv` (punto 2) y el `.env` va en `.gitignore` (punto 1).

---

1. Humedad actual

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}
resp = requests.get("https://api.openweathermap.org/data/2.5/weather", params=params)
print(f"Humedad: {resp.json()['main']['humidity']}%")
```

Necesitas una API key real de OpenWeatherMap (gratuita). Con `units=metric` la temperatura viene en grados.

2. Descripción del clima

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric", "lang": "es"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(datos["weather"][0]["description"])
```

`weather` es una **lista**: el `[0]` coge la condición principal y `description` es su texto.

3. Temperatura mínima y máxima

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(f"Mínima: {datos['main']['temp_min']}°C")
print(f"Máxima: {datos['main']['temp_max']}°C")
```

`main` contiene `temp`, `temp_min`, `temp_max`, `feels_like` y `humidity`.

4. Temperatura y sensación térmica

```
import requests
API_KEY = "TU_API_KEY"
params = {"q": "Madrid", "appid": API_KEY, "units": "metric"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(f"Temperatura: {datos['main']['temp']}°C")
print(f"Sensación: {datos['main']['feels_like']}°C")
```

5. Error 401

```
import requests
resp = requests.get("https://api.openweathermap.org/data/2.5/weather",
                    params={"q": "Sevilla", "appid": "falsa", "units": "metric"})
print(resp.status_code)  # 401
```

API key incorrecta → **401 Unauthorized**: el servidor no te reconoce y rechaza la petición.

6. ¿Dónde va la API key?

a) En el parámetro **`appid`** de la query string: `?q=Sevilla&appid=TU_API_KEY`. b) En la cabecera **`Authorization`** con el esquema **`Bearer`**: `Authorization: Bearer TU_API_KEY`.

7. Clave desde el `.env`

```
from dotenv import load_dotenv
import os

load_dotenv()
print(os.getenv("OPENWEATHER_API_KEY"))  # "test123"
```

El `.env` debe contener la línea `OPENWEATHER_API_KEY=test123`. `load_dotenv()` carga el archivo y `os.getenv` lee el valor.

8. Protege el repo

a) En el `.gitignore`:

```
.env
```

b) `os.getenv("MI_CLAVE")` devuelve **`None`** si la variable no existe. Por eso conviene comprobarlo y lanzar un `raise ValueError` con un mensaje claro.

> Los **4xx** son error del cliente (tu clave, tu URL, tus parámetros): el 401 es el típico cuando la API key es mala.

Boletín UD 8 — Avanzado

💪 Boletín UD 8 — Avanzado

> Ejercicios que requieren aplicar los conceptos de APIs comerciales, variables de entorno, rate limits y OpenAI de forma más profunda, con programas completos contra APIs reales.

---

1. Clima con .env para dos ciudades

Carga tu API key de OpenWeatherMap desde un archivo `.env` con `python-dotenv`. Obtén el clima de dos ciudades distintas y compara sus temperaturas.

Pista: `load_dotenv()` + `os.getenv("OPENWEATHER_API_KEY")`. Escribe una función `clima(ciudad)` y llámala dos veces, guardando la temperatura de cada una para compararlas.

2. GPT con temperatura creativa

Usa la API de OpenAI para pedir una historia corta sobre Python. Ajusta el parámetro `temperature=0.9` para obtener una respuesta más creativa.

Pista: en `cliente.chat.completions.create(...)`, además de `model` y `messages`, pasa `temperature=0.9` y un `max_tokens` razonable para que la historia no se corte.

3. Capturar errores HTTP

Haz una petición a `https://httpbin.org/status/404` y otra a `https://httpbin.org/status/500`. Captura los errores usando `raise_for_status()` dentro de un bloque try/except.

Pista: dentro del `except requests.exceptions.HTTPError as e`, puedes leer `e.response.status_code` para saber qué código ha fallado.

4. Comprobación de claves

Carga el `.env` y comprueba que `OPENWEATHER_API_KEY` existe. Si no existe, lanza un error claro con `raise ValueError`.

Pista: `os.getenv` devuelve `None` si la variable falta: esa es la condición para lanzar el error.

5. Chat con GPT y rol system

Pregunta a GPT-4o-mini "¿Qué es Python?" usando un mensaje `system` que diga "Eres un profesor de Python divertido" antes del mensaje `user`.

6. Timeout

Haz una petición a `https://192.0.2.1` con `timeout=3` y captura la excepción de timeout.

Pista: `192.0.2.1` es una IP de test que nunca responde. La excepción es `requests.exceptions.Timeout`.

7. 🎯 Pronóstico extendido

Usa OpenWeatherMap para obtener el pronóstico de 5 días de tu ciudad.

Pista: OpenWeatherMap tiene un endpoint `/forecast` que devuelve datos cada 3 horas. Usa `lang=es` para descripciones en español. Para obtener un dato por día, salta 8 elementos (24 horas ÷ 3 horas = 8 intervalos).

8. 🔍 Múltiples ciudades

Obtén el clima de 5 ciudades a la vez usando un solo bucle.

Pista: Itera sobre una lista de ciudades y haz una petición a OpenWeatherMap por cada una. Añade `time.sleep(0.2)` entre peticiones para evitar el rate limit. Cada respuesta contiene `main.temp` y `weather[0].description`.

9. 🧩 GPT: explicador automático

Pregunta a GPT-4o-mini que explique 3 conceptos de Python: Lock, Semaphore, Barrier.

Pista: La API de chat de OpenAI recibe una lista de mensajes. Puedes iterar sobre una lista de conceptos y pedir una explicación corta para cada uno, limitando la respuesta con `max_tokens`.

10. 🎭 Conversación con contexto

Chat con GPT que recuerda el historial de la conversación.

Pista: Mantén una lista `mensajes` que acumule cada interacción. Empieza con un mensaje `{"role": "system"}` para definir el rol. En cada turno, añade el mensaje del usuario y la respuesta del asistente para mantener el contexto.

11. ⏱️ Monitor de uptime

Comprueba cada 30s si una API responde. Si falla 3 veces seguidas, alerta.

Pista: Un bucle infinito con `time.sleep(30)` dentro. Un contador de fallos consecutivos se incrementa en cada error y se reinicia al obtener una respuesta OK. Cuando supere el máximo, muestra una alerta.

12. 🧪 Agregador de APIs

Llama a 3 APIs distintas y combina sus respuestas en un solo resumen.

Pista: Define una función que llame a varias APIs y guarde los datos relevantes en un diccionario. Cada API aporta una clave diferente. Usa GitHub para datos de usuario, JSONPlaceholder para el conteo de posts y httpbin para la IP.

Boletín UD 8 — Avanzado (Resuelto)
💪 Boletín UD 8 — Avanzado (Resuelto)

1. Clima con .env para dos ciudades

```
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")


def clima(ciudad):
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}
    datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
                         params=params).json()
    return datos["main"]["temp"]


t1 = clima("Madrid")
t2 = clima("Barcelona")
print(f"Madrid: {t1}°C | Barcelona: {t2}°C")
print("Gana Madrid" if t1 > t2 else "Gana Barcelona")
```

La función devuelve la temperatura y el programa compara las dos llamadas. La clave nunca está en el código: viene del `.env`.

2. GPT con temperatura creativa

```python
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

resp = cliente.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Cuenta una historia corta sobre Python."}],
    temperature=0.9,   # más creativo
    max_tokens=200
)
print(resp.choices[0].message.content)
```

`temperature=0.9` da respuestas más creativas y variadas; `temperature=0` sería literal y repetitivo.

3. Capturar errores HTTP

```python
import requests

for codigo in (404, 500):
    try:
        resp = requests.get(f"https://httpbin.org/status/{codigo}")
        resp.raise_for_status()
    except requests.exceptions.HTTPError as e:
        print(f"❌ Error {e.response.status_code} en {e.request.url}")
```

`raise_for_status()` lanza una `HTTPError` para cualquier 4xx/5xx, y `e.response.status_code` te dice qué código ha sido.

4. Comprobación de claves

```python
from dotenv import load_dotenv
import os

load_dotenv()

API_KEY = os.getenv("OPENWEATHER_API_KEY")
if not API_KEY:
    raise ValueError("❌ Falta OPENWEATHER_API_KEY en .env")
print("Clave cargada correctamente")
```

Si la variable no existe, `os.getenv` devuelve `None` y el programa muere con un mensaje claro **antes** de hacer ninguna petición con clave vacía.

5. Chat con GPT y rol system

```python
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

resp = cliente.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Eres un profesor de Python divertido."},
        {"role": "user", "content": "¿Qué es Python?"}
    ]
)
print(resp.choices[0].message.content)
```

El mensaje `system` define el **carácter y las reglas** del asistente; el `user` lleva la pregunta.

6. Timeout

```python
import requests

try:
    resp = requests.get("https://192.0.2.1", timeout=3)
except requests.exceptions.Timeout:
    print("Timeout — el servidor no responde")
```

192.0.2.1 es una IP de test que nunca responde. El `timeout=3` hace que `requests` aborte a los 3 segundos y lanza `requests.exceptions.Timeout`.

7. 🎯 Pronóstico extendido

```python
import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

params = {
    "q": "Sevilla",
    "appid": API_KEY,
    "units": "metric",
    "lang": "es"
}
datos = requests.get("https://api.openweathermap.org/data/2.5/forecast",
                     params=params).json()

for i in range(0, len(datos["list"]), 8):  # cada 8 elementos = 1 día
    dia = datos["list"][i]
    fecha = dia["dt_txt"][:10]
    temp = dia["main"]["temp"]
    desc = dia["weather"][0]["description"]
    print(f"{fecha}: {temp}°C — {desc}")
```

El endpoint `/forecast` devuelve datos cada 3 horas: 24 ÷ 3 = **8 intervalos por día**. Saltando de 8 en 8 obtienes un dato por día del pronóstico de 5 días.

8. 🔍 Múltiples ciudades

```
import requests, time
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

ciudades = ["Madrid", "Barcelona", "Sevilla", "Valencia", "Bilbao"]

for ciudad in ciudades:
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}
    datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
                         params=params).json()
    print(f"{datos['name']}: {datos['main']['temp']}°C — "
          f"{datos['weather'][0]['description']}")
    time.sleep(0.2)  # respiro entre peticiones: evita el rate limit
```

El `time.sleep(0.2)` espacia las peticiones para no disparar el límite de 60/min del plan gratis.

9. 🧩 GPT: explicador automático

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

conceptos = ["Lock", "Semaphore", "Barrier"]

for concepto in conceptos:
    resp = cliente.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user",
                   "content": f"Explica qué es {concepto} en 2 frases."}],
        max_tokens=60
    )
    print(f"{concepto}: {resp.choices[0].message.content}\n")
```

Se itera sobre la lista de conceptos y cada uno recibe una pregunta con `max_tokens` corto para limitar la longitud (y el coste).

10. 🎭 Conversación con contexto

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

mensajes = [{"role": "system", "content": "Eres un tutor de Python."}]

while True:
    pregunta = input("Tú: ")
    if pregunta.lower() == "salir":
        break

    mensajes.append({"role": "user", "content": pregunta})

    resp = cliente.chat.completions.create(
        model="gpt-4o-mini",
        messages=mensajes,
        max_tokens=150
    )

    respuesta = resp.choices[0].message.content
    print(f"GPT: {respuesta}")
    mensajes.append({"role": "assistant", "content": respuesta})
```

La lista `mensajes` acumula cada turno (`user` → `assistant` → `user` → …): así el modelo “recuerda” lo hablado. El mensaje `system` inicial fija el rol del tutor.

11. ⏱️ Monitor de uptime

```python
import requests, time
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

URL = "https://api.openweathermap.org/data/2.5/weather"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}

fallos = 0
MAX_FALLOS = 3

while True:
    try:
        resp = requests.get(URL, params=params, timeout=5)
        if resp.status_code == 200:
            print("✅ La API responde OK")
            fallos = 0
        else:
            print(f"⚠️ Respuesta inesperada: {resp.status_code}")
            fallos += 1
    except requests.exceptions.RequestException as e:
        print(f"❌ Error: {e}")
        fallos += 1

    if fallos >= MAX_FALLOS:
        print("🚨 ALERTA: la API lleva 3 fallos seguidos")
        fallos = 0

    time.sleep(30)  # comprobamos cada 30 segundos
```

El contador `fallos` se incrementa con cada error y se **reinicia** al obtener un 200. Cuando llega a 3, salta la alerta. Este es el esqueleto de los monitores de disponibilidad que verás en la UD 10 con asyncio.

12. 🏗️ Agregador de APIs

```python
import requests

def resumen():
    resultado = {}

    usuario = requests.get("https://api.github.com/users/python").json()
    resultado["login"] = usuario["login"]
    resultado["repos"] = usuario["public_repos"]

    posts = requests.get("https://jsonplaceholder.typicode.com/posts").json()
    resultado["posts_publicados"] = len(posts)

    ip = requests.get("https://httpbin.org/ip").json()
    resultado["mi_ip"] = ip["origin"]

    return resultado

info = resumen()
print(f"👤 Usuario: {info['login']}")
print(f"📦 Repos públicos: {info['repos']}")
print(f"📝 Posts en JSONPlaceholder: {info['posts_publicados']}")
print(f"🖥️ Mi IP (según httpbin): {info['mi_ip']}")
```

Cada API aporta una clave distinta al diccionario: GitHub los datos de usuario, JSONPlaceholder el conteo de posts y httpbin la IP. Una función centraliza las tres llamadas y combina lo relevante en un solo resumen.

Boletín UD 9 — Inicial

📝 Boletín UD 9 — Inicial

> Ejercicios básicos para afianzar los conceptos de hash (MD5, SHA), César y el cifrado moderno (AES, RSA) de la unidad UD 9.

---

1. SHA1 de tu nombre

Calcula el hash SHA1 de tu nombre (en minúsculas) usando `hashlib.sha1()`. Muestra el resultado en hexadecimal con `.hexdigest()`.

Pista: recuerda que `hashlib` espera bytes: escribe tu nombre como `b"sergi"` o con `.encode()`.

2. MD5 de una frase

Calcula el hash MD5 de la frase `"Python es genial"`. Muestra el resultado en hexadecimal.

3. Longitud de los hashes de “Hola mundo”

Calcula MD5, SHA1 y SHA256 de `"Hola mundo"` y escribe cuántos caracteres hexadecimales devuelve cada uno.

4. Cifrado César sencillo

Implementa una función `cifrar_cesar(texto, desplazamiento)` y cifra `"Hola"` con desplazamiento 3. ¿Qué texto obtienes?

5. Determinismo del hash

Calcula dos veces el SHA256 de `"clave123"` y compáralos. ¿Son iguales? ¿Qué propiedad del hash te lo garantiza?

Pista: un hash es **determinista**: misma entrada → mismo hash, siempre.

6. Nonce y tag

Cifra `b"Hola mundo con AES"` usando AES modo EAX (librería `pycryptodome`). Imprime por pantalla las longitudes del nonce y del tag (en bytes).

Pista: `cifrador.nonce` y `tag` son objetos bytes: `len(cifrador.nonce)` y `len(tag)`. El nonce del modo EAX suele medir 16 bytes.

7. AES: cifrar y descifrar completo

Cifra `b"El cifrado simetrico es rapido"` con AES (EAX), y descifralo a continuación. Muestra el nonce, el tag y el mensaje original.

Pista: usa `encrypt_and_digest` para cifrar y `AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce).decrypt(texto_cifrado)` para descifrar.

8. Simétrico vs asimétrico

Responde razonadamente:

a) ¿Cuántas claves usa el cifrado simétrico? ¿Y el asimétrico? b) ¿Con qué clave cifras un mensaje para que solo Bob lo lea? c) ¿Por qué RSA no sirve para cifrar un archivo grande?

📚 Volver a la unidad · Resuelto: ✅ Boletín UD 9 — Inicial (Resuelto)

Boletín UD 9 — Inicial (Resuelto)
✅ Boletín UD 9 — Inicial (Resuelto)

> Los ejercicios 6 y 7 necesitan `pycryptodome`: `pip install pycryptodome`.

---

1. SHA1 de tu nombre

```
import hashlib
print(hashlib.sha1(b"sergi").hexdigest())
# 52a431556910f83a3d65e1e7c3772f5d44011eb3
```

Con tu nombre (en minúsculas), por ejemplo `b"ana"`, obtendrías su hash SHA1 de **40 caracteres** hexadecimales (160 bits).

2. MD5 de una frase

```
import hashlib
print(hashlib.md5("Python es genial".encode()).hexdigest())
# 1bdf047b34a4890934fc36d12b464f2c
```

MD5 devuelve **32 caracteres** hexadecimales (128 bits).

3. Longitud de los hashes de “Hola mundo”

```
import hashlib
texto = b"Hola mundo"
print(f"MD5:    {len(hashlib.md5(texto).hexdigest())} chars (128 bits)")
print(f"SHA1:   {len(hashlib.sha1(texto).hexdigest())} chars (160 bits)")
print(f"SHA256: {len(hashlib.sha256(texto).hexdigest())} chars (256 bits)")
```

MD5 → **32**, SHA1 → **40**, SHA256 → **64** caracteres hexadecimales. La longitud es **fija** para cada algoritmo, da igual el tamaño de la entrada.

4. Cifrado César sencillo

```python
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

print(cifrar_cesar("Hola", 3))  # "Krod"
```

Cada letra avanza 3 posiciones: **H→K, o→r, l→o, a→d**. Resultado: **Krod**.

5. Determinismo del hash

```python
import hashlib
h1 = hashlib.sha256(b"clave123").hexdigest()
h2 = hashlib.sha256(b"clave123").hexdigest()
print(f"Primera:  {h1}")
print(f"Segunda:  {h2}")
print(f"¿Iguales? {h1 == h2}")  # True
```

Son **iguales**: el hash es **determinista** (punto 2). Misma entrada → mismo hash, siempre. Esa propiedad es la que hace posible el login por comparación de hashes del punto 4.

6. Nonce y tag

```python
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(b"Hola mundo con AES")

print(f"Longitud nonce: {len(cifrador.nonce)} bytes")
print(f"Longitud tag:   {len(tag)} bytes")
print(f"Longitud cifrado: {len(texto_cifrado)} bytes")
```

En modo EAX el **nonce** mide **16 bytes** y el **tag** también **16 bytes**. Los tres (nonce, tag y cifrado) viajan juntos; la clave no (punto 2).

7. AES: cifrar y descifrar completo

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)
mensaje = b"El cifrado simetrico es rapido"

cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(mensaje)

print(f"Nonce: {cifrador.nonce.hex()}")
print(f"Tag:   {tag.hex()}")
print(f"Cifrado: {texto_cifrado.hex()}")

descifrador = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
original = descifrador.decrypt(texto_cifrado)
print(f"Original: {original.decode()}")
```

El nonce y el tag se envían junto al cifrado; el receptor los usa con la misma clave para descifrar y verificar la integridad (punto 2).

8. Simétrico vs asimétrico

a) El simétrico usa **una sola clave** (misma para cifrar y descifrar). El asimétrico usa **dos**: pública + privada.

b) Con la clave **pública de Bob**. Solo su clave privada puede descifrarlo.

c) Por su **límite de tamaño** (~190 bytes con claves de 2048 bits) y su **velocidad** (~1 MB/s). Para volúmenes grandes se usa AES (o el cifrado híbrido del punto 6).

📚 Volver a la unidad · Por resolver: 📝 Boletín UD 9 — Inicial

Boletín UD 9 — Avanzado

💪 Boletín UD 9 — Avanzado

> Ejercicios que requieren aplicar hash, César, AES, RSA y firmas de forma más profunda, con programas completos.

---

1. Cifrado César con espacios

Implementa una función `cifrar_cesar(texto, desplazamiento)` que cifre frases completas respetando espacios y signos de puntuación sin modificarlos. Pruébala con `"Hola mundo"` con desplazamiento 5.

Pista: recorre carácter a carácter. Si `caracter.isalpha()` es verdadero, aplica la fórmula con `% 26`; si no, añade el carácter tal cual. La salida debe ser `"Mtqf rzsit"`.

2. Fuerza bruta César

Dado un texto cifrado con César (desplazamiento desconocido), prueba los 25 desplazamientos y muestra solo los que tengan palabras en español.

Pista: para cada desplazamiento del 1 al 25, aplica el descifrado César. Usa un conjunto de palabras comunes en español (como `"hola"`, `"mundo"`, `"python"`) para filtrar resultados. Si el texto descifrado contiene alguna palabra conocida, es probable que sea el correcto.

3. Hash con salt

Añade un salt aleatorio al hash de una contraseña para evitar tablas rainbow.

Pista: genera un salt de 16 bytes con `os.urandom(16)` y calcula `hashlib.sha256(salt + password.encode()).hexdigest()`. Guarda el resultado como `salt.hex() + hash` (string hex). Para verificar, extrae los primeros 32 caracteres (la sal en hex, 16 bytes) con `bytes.fromhex()` y repite el cálculo. Si dos usuarios tienen la misma contraseña, sus hashes serán distintos gracias al salt.

4. Mini gestor de contraseñas

Guarda contraseñas con hash + salt en un archivo JSON. Permite registro y login.

Pista: almacena cada usuario en un JSON con el formato `salt.hex() + hashlib.sha256(salt + password.encode()).hexdigest()`. Para verificar, extrae el salt (primeros 32 caracteres hexadecimales), conviértelo a bytes con `bytes.fromhex()` y recalcula el hash con la contraseña proporcionada.

5. Cifrado híbrido simplificado

Genera una clave AES de 32 bytes y cifra `b"El cifrado hibrido funciona"`. Luego cifra esa clave AES con una clave RSA pública. Descifra en orden inverso y verifica el mensaje original.

Pista: cifra el mensaje con `AES.new(clave_aes, AES.MODE_EAX)` y `encrypt_and_digest`; cifra la clave AES con `PKCS1_OAEP.new(clave.publickey()).encrypt(clave_aes)`. Para descifrar, invierte el orden: primero RSA (con la privada), luego AES (con el mismo nonce).

6. Firma alterada

Firma digitalmente el mensaje `"Transferencia de 500€".encode()`. Modifica UN byte de la firma y comprueba que la verificación falla con `pkcs1_15.new(...).verify(...)`.

Pista: para modificar un byte de la firma, conviértela a `bytearray`, cambia un índice (`firma_mutada[0] ^= 0xFF`) y vuelve a bytes. La verificación debe lanzar `ValueError` o `TypeError`.

7. RBAC con permisos cifrado

Crea un sistema RBAC con 3 roles: `admin` (cifrar, descifrar, firmar), `usuario` (cifrar, firmar), `invitado` (solo cifrar). Implementa la función `puede(usuario, accion)` y pruébala con cada rol.

Pista: un diccionario `permisos = {"admin": ["cifrar", "descifrar", "firmar"], ...}` y `return accion in permisos.get(usuario["rol"], [])`. Los roles desconocidos deben devolver `False`.

8. Cifrar archivo completo

Crea un archivo de texto `mensaje.txt` con algún contenido y cifra el archivo completo con AES, guardando el resultado. Luego descifralo y comprueba que coincide.

Pista: primero crea `mensaje.txt` (p. ej. `open("mensaje.txt", "w").write("Contenido secreto")`). Abre el archivo en modo binario `"rb"` / `"wb"`. Usa `AES.MODE_EAX`, cifra con `encrypt_and_digest` y guarda `nonce + tag + cifrado`. Para descifrar, separa los tres componentes y usa `decrypt_and_verify`.

9. RSA: cifrar mensajes largos

RSA solo cifra ~190 bytes. Intenta cifrar un mensaje de 300 bytes. ¿Qué pasa? ¿Cómo lo arreglas?

Pista: captura el `ValueError` para ver el mensaje de error. Piensa en combinar RSA con un cifrado simétrico como AES (cifrado híbrido).

10. Firma con verificación de integridad

Firma un mensaje, modifica el mensaje, y muestra que la verificación falla.

Pista: usa `SHA256.new(mensaje)` y `pkcs1_15.new(clave).sign(h)`. Después de firmar el original, crea un segundo mensaje modificado y verifícalo con la misma firma. La verificación debe lanzar `ValueError` o `TypeError`.

11. RSA vs AES benchmark

Mide cuánto tarda cifrar el mismo mensaje con RSA y AES. La diferencia es abismal.

Pista: usa `time.time()` antes y después de un bucle de 100 cifrados RSA y otro de 1000 cifrados AES. Con `time.time() - t` obtienes los segundos. Multiplica por 1000 para milisegundos.

12. Sistema de cifrado de extremo a extremo

Simula un chat cifrado: cada usuario tiene su par RSA, y los mensajes se cifran con AES + RSA híbrido.

Pista: crea una clase `Usuario` con `nombre` y `clave_rsa`. Un método `cifrar_para(mensaje, destinatario)` que genera clave AES, cifra el mensaje y la clave. Otro método `descifrar(nonce, tag, clave_aes_cifrada, cifrado)` que hace el proceso inverso.

Boletín UD 9 — Avanzado (Resuelto)
💪 Boletín UD 9 — Avanzado (Resuelto)

> Los ejercicios 5-12 necesitan `pycryptodome`: `pip install pycryptodome`.

---

1. Cifrado César con espacios

```python
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

print(cifrar_cesar("Hola mundo", 5))  # Mtqf rzsit
```

El espacio no es una letra (`isalpha()` es `False`), así que se añade tal cual. Las letras avanzan 5 posiciones: **Hola → Mtqf**, **mundo → rzsit**. Resultado: **Mtqf rzsit**.

2. Fuerza bruta César

```python
def descifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base - desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

palabras = {"hola", "mundo", "python", "clave", "secreto"}

cifrado = "Mtqf rzsit"  # "Hola mundo" con desplazamiento 5
for desplazamiento in range(1, 26):
    intento = descifrar_cesar(cifrado, desplazamiento)
    if any(p in intento.lower() for p in palabras):
        print(f"Desplazamiento {desplazamiento}: {intento}  ← ¡parece correcto!")
```

Solo hay **25 desplazamientos posibles** (punto 6). Filtrando por palabras comunes en español, el desplazamiento 5 devuelve “Hola mundo” y es el único con sentido. Se rompe en segundos.

3. Hash con salt

```python
import hashlib, os

def registrar_con_salt(contraseña):
    salt = os.urandom(16)                                    # 🧂 16 bytes nuevos cada
vez
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return salt.hex() + hash_contra                          # sal + hash juntos

def verificar(contraseña, almacenado):
    salt = bytes.fromhex(almacenado[:32])                    # 🧂 recuperamos la sal
    hash_original = almacenado[32:]
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return hash_intento == hash_original

hash_ana = registrar_con_salt("clave123")
hash_bob = registrar_con_salt("clave123")
print(f"¿Hashes iguales? {hash_ana == hash_bob}")            # False
print(f"¿Ana entra? {verificar('clave123', hash_ana)}")      # True
print(f"¿Ana con otra? {verificar('otra', hash_ana)}")       # False
```

Con la misma contraseña, Ana y Bob tienen **hashes distintos** gracias a sus sales únicas. El salt ocupa los 32 primeros caracteres hex (16 bytes), que se recuperan con `bytes.fromhex()` para el login (punto 5).

4. Mini gestor de contraseñas

```python
import hashlib, os, json

usuarios = {}

def registrar(usuario, contraseña):
    salt = os.urandom(16)
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    usuarios[usuario] = salt.hex() + hash_contra

def login(usuario, contraseña):
    almacenado = usuarios[usuario]
    salt = bytes.fromhex(almacenado[:32])
    hash_original = almacenado[32:]
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    return hash_intento == hash_original

registrar("Ana", "clave123")
registrar("Bob", "clave123")
print(f"Ana login correcto: {login('Ana', 'clave123')}")     # True
print(f"Ana login fallido:  {login('Ana', 'otra')}")        # False
print(f"Hashes distintos:   {usuarios['Ana'] != usuarios['Bob']}")  # True

with open("contraseñas.json", "w") as f:
    json.dump(usuarios, f, indent=2)
```

Cada usuario se guarda como `salt.hex() + hash` en un diccionario, y se exporta a JSON con `json.dump`. Para verificar, el login extrae la sal (primeros 32 caracteres hex), la convierte con `bytes.fromhex()` y recalcula el hash (punto 5). Si dos usuarios comparten contraseña, sus registros JSON son **distintos**.

5. Cifrado híbrido simplificado

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

clave_bob = RSA.generate(2048)

clave_aes = get_random_bytes(32)
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(b"El cifrado hibrido funciona")

clave_aes_cifrada = PKCS1_OAEP.new(clave_bob.publickey()).encrypt(clave_aes)

clave_aes_recibida = PKCS1_OAEP.new(clave_bob).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)
print(f"Mensaje original: {original.decode()}")
```

```
Mensaje original: El cifrado hibrido funciona
```

RSA reparte la clave AES (32 bytes) y AES cifra el mensaje completo. El orden de descifrado es el inverso al de cifrado (punto 6).

6. Firma alterada

```python
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = "Transferencia de 500€".encode()

h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)

firma_mutada = bytearray(firma)
firma_mutada[0] ^= 0xFF
firma_mutada = bytes(firma_mutada)

try:
    pkcs1_15.new(clave.publickey()).verify(h, firma_mutada)
    print("✅ Firma válida")
except (ValueError, TypeError):
    print("❌ Firma inválida — la firma fue alterada")
```

```
❌ Firma inválida — la firma fue alterada
```

La firma depende del hash y de la clave: **un solo byte distinto** la invalida por completo (punto 5).

7. RBAC con permisos cifrado

```
permisos = {
    "admin":    ["cifrar", "descifrar", "firmar"],
    "usuario":  ["cifrar", "firmar"],
    "invitado": ["cifrar"],
}

def puede(usuario, accion):
    return accion in permisos.get(usuario["rol"], [])

print(puede({"rol": "admin"}, "descifrar"))    # True
print(puede({"rol": "usuario"}, "descifrar"))  # False
print(puede({"rol": "usuario"}, "firmar"))     # True
print(puede({"rol": "invitado"}, "firmar"))    # False
print(puede({"rol": "invitado"}, "cifrar"))    # True
```

```
True
False
True
False
True
```

Cada rol tiene su paquete de permisos; `puede` comprueba si la acción está en el del rol (punto 7). `permisos.get(usuario["rol"], [])` devuelve lista vacía para roles desconocidos: por defecto, nada permitido.

8. Cifrar archivo completo

```python
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

clave = get_random_bytes(32)

with open("mensaje.txt", "w") as f:
    f.write("Contenido secreto del proyecto")

with open("mensaje.txt", "rb") as f:
    contenido = f.read()

cifrador = AES.new(clave, AES.MODE_EAX)
cifrado, tag = cifrador.encrypt_and_digest(contenido)

with open("mensaje.cifrado", "wb") as f:
    f.write(cifrador.nonce + tag + cifrado)      # nonce + tag + cifrado

with open("mensaje.cifrado", "rb") as f:
    paquete = f.read()

nonce = paquete[:16]
tag = paquete[16:32]
cifrado_recibido = paquete[32:]

descifrador = AES.new(clave, AES.MODE_EAX, nonce=nonce)
original = descifrador.decrypt_and_verify(cifrado_recibido, tag)
print(f"Archivo descifrado: {original.decode()}")
```

Se guardan los tres componentes juntos (`nonce + tag + cifrado`) y al descifrar se **separan por sus longitudes** (16 y 16 bytes). `decrypt_and_verify` además comprueba que el archivo no fue manipulado.

9. RSA: cifrar mensajes largos

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

clave = RSA.generate(2048)
mensaje_largo = b"x" * 300   # 300 bytes

cifrador = PKCS1_OAEP.new(clave.publickey())
try:
    cifrador.encrypt(mensaje_largo)
except ValueError as e:
    print(f"❌ Error: {e}")
```

```
❌ Error: Plaintext is too long.
```

El problema: RSA con claves de 2048 bits solo admite ~190 bytes de mensaje.

La solución (cifrado híbrido): no cifres el mensaje con RSA; cifra la clave AES con RSA y el mensaje con AES:

```
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

clave_aes = get_random_bytes(32)
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje_largo)
clave_aes_cifrada = PKCS1_OAEP.new(clave.publickey()).encrypt(clave_aes)

clave_aes_recibida = PKCS1_OAEP.new(clave).decrypt(clave_aes_cifrada)
original = AES.new(clave_aes_recibida, AES.MODE_EAX,
nonce=cifrador_aes.nonce).decrypt(cifrado)
print(f"Longitud del original: {len(original)} bytes")
```

```
Longitud del original: 300 bytes
```

Así cualquier mensaje cabe, sea cual sea su tamaño (punto 6).

10. Firma con verificación de integridad

```python
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = b"Este mensaje es de Ana"

h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)

mensaje_tocado = b"Este mensaje es de Ana pero lo cambie"

try:
    pkcs1_15.new(clave.publickey()).verify(SHA256.new(mensaje_tocado), firma)
    print("✅ Firma válida")
except (ValueError, TypeError):
    print("❌ Firma inválida — el mensaje fue manipulado")
```

```
❌ Firma inválida — el mensaje fue manipulado
```

La firma se calcula sobre el **hash del mensaje original**. Si el mensaje cambia un solo byte, el hash es distinto y la verificación falla: la firma detecta cualquier modificación (punto 5).

11. RSA vs AES benchmark

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes
import time

clave_rsa = RSA.generate(2048)
clave_aes = get_random_bytes(32)
mensaje = b"Rendimiento de cifrado"

cifrador_rsa = PKCS1_OAEP.new(clave_rsa.publickey())
cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)

inicio = time.time()
for _ in range(100):
    cifrador_rsa.encrypt(mensaje[:16])
rsa_ms = (time.time() - inicio) * 1000

inicio = time.time()
for _ in range(1000):
    cifrador_aes.encrypt(mensaje)
aes_ms = (time.time() - inicio) * 1000

print(f"100 cifrados RSA:  {rsa_ms:.1f} ms")
print(f"1000 cifrados AES: {aes_ms:.1f} ms")
```

La diferencia es **abismal**: cifrar 10 veces más mensajes con AES tarda una fracción de lo que tarda RSA con solo 100. Por eso AES va para el volumen y RSA solo para repartir la clave (punto 1).

12. Sistema de cifrado de extremo a extremo

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre
        self.clave_rsa = RSA.generate(2048)

    def cifrar_para(self, mensaje, destinatario):
        clave_aes = get_random_bytes(32)
        cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
        cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje.encode())
                                                                clave_aes_cifrada        =
PKCS1_OAEP.new(destinatario.clave_rsa.publickey()).encrypt(clave_aes)
        return cifrador_aes.nonce, tag, clave_aes_cifrada, cifrado

    def descifrar(self, nonce, tag, clave_aes_cifrada, cifrado):
        clave_aes = PKCS1_OAEP.new(self.clave_rsa).decrypt(clave_aes_cifrada)
                                    original     =     AES.new(clave_aes,     AES.MODE_EAX,
nonce=nonce).decrypt_and_verify(cifrado, tag)
        return original.decode()

ana = Usuario("Ana")
bob = Usuario("Bob")

nonce, tag, clave_aes_cifrada, cifrado = ana.cifrar_para("Hola Bob, quedamos a las 8",
bob)
print(f"Bob recibe: {bob.descifrar(nonce, tag, clave_aes_cifrada, cifrado)}")
```

```
Bob recibe: Hola Bob, quedamos a las 8
```

`cifrar_para` cifra el mensaje con AES y protege la clave AES con la **pública del destinatario**. `descifrar` invierte el proceso usando la **privada del propio usuario** y verifica el tag. Es el esquema del punto 8: cada usuario guarda su privada y solo él puede descifrar lo que le envían.

📚 Volver a la unidad · Por resolver: 💪 Boletín UD 9 — Avanzado

Boletín UD 10 — Inicial

📝 Boletín UD 10 — Inicial

> Ejercicios básicos para afianzar los conceptos de asyncio: corrutinas, el event loop, gather, las tareas en segundo plano y los primeros mecanismos de disponibilidad.

---

1. Mensaje diferido

Crea una corrutina `async def preparar()` que imprima “Preparando…”, espere 2 segundos con `await asyncio.sleep(2)`, e imprima “¡Listo!”. Ejecútala con `asyncio.run()`.

2. Saludo y despedida

Crea dos corrutinas: `saludar()` (espera 0.5s y dice “Hola”) y `despedirse()` (espera 1s y dice “Adiós”). Ejecuta ambas con `asyncio.gather`.

3. Temporizador

Crea una corrutina que imprima “tic” cada 2 segundos, 4 veces. Usa `asyncio.sleep(2)` dentro de un bucle `for`.

4. Corrutina que devuelve

Crea una corrutina `sumar(a, b)` que espere 1 segundo y devuelva `a + b`. Lánzala y muestra el resultado.

Pista: llamar a una corrutina no ejecuta su cuerpo: devuelve un objeto corrutina. Para obtener el resultado usa `await sumar(3, 4)` dentro de una corrutina `main()` y ejecuta con `asyncio.run(main())`.

5. Tres tareas a la vez

Crea 3 corrutinas que esperen 1, 2 y 3 segundos. Lánzalas con `asyncio.gather` y mide el tiempo total con `time.time()`. ¿Cuánto dura en vez de 6 segundos?

Pista: con `gather` el total es el de la tarea más lenta (3s), no la suma. Guarda `inicio = time.time()` antes del `gather` y resta después.

6. Tarea en segundo plano

Crea una corrutina `contar()` que imprima un número cada 0.5s (hasta 4). Lánzala con `asyncio.create_task` desde una corrutina `main()` que haga `await asyncio.sleep(1)` y luego imprima "main sigue". ¿Qué se imprime mientras tanto?

7. Heartbeat básico

Crea una corrutina heartbeat que imprima "💓 vivo" cada 2 segundos. Lánzala con `create_task` desde `main()` y deja que `main()` espere 5 segundos antes de terminar.

Pista: el heartbeat es un `while True` con `await asyncio.sleep(2)`. `main()` debe esperar (`await asyncio.sleep(5)`) o la tarea de fondo muere con él.

8. Timeout básico

Crea una corrutina `lenta()` que tarde 8 segundos y devuelva "Hecho". Usa `asyncio.wait_for` con timeout de 3s y muestra qué excepción salta y qué imprime tu `except`.

Pista: `asyncio.wait_for(lenta(), timeout=3)` lanza `asyncio.TimeoutError` al pasar los 3 segundos y cancela la corrutina lenta.

Boletín UD 10 — Inicial (Resuelto)
✅ Boletín UD 10 — Inicial (Resuelto)

1. Mensaje diferido

```python
import asyncio

async def preparar():
    print("Preparando...")
    await asyncio.sleep(2)
    print("¡Listo!")

asyncio.run(preparar())
```

`await asyncio.sleep(2)` pausa la corrutina 2 segundos; `asyncio.run()` crea el event loop.

2. Saludo y despedida

```python
import asyncio

async def saludar():
    await asyncio.sleep(0.5)
    print("Hola")

async def despedirse():
    await asyncio.sleep(1)
    print("Adiós")

async def main():
    await asyncio.gather(saludar(), despedirse())

asyncio.run(main())
```

`gather` ejecuta ambas “a la vez”: el total es ~1s (la más lenta), no 1.5s.

3. Temporizador

```
import asyncio

async def tic():
    for i in range(4):
        print("tic")
        await asyncio.sleep(2)

asyncio.run(tic())
```

4 “tic” espaciados 2 segundos. Un `asyncio.sleep` dentro de un `for`.

4. Corrutina que devuelve

```
import asyncio

async def sumar(a, b):
    await asyncio.sleep(1)
    return a + b

async def main():
    resultado = await sumar(3, 4)
    print(resultado)

asyncio.run(main())
```

Para obtener el valor hace falta `await`: sin él tendrías un objeto corrutina, no el resultado.

5. Tres tareas a la vez

```python
import asyncio, time

async def tarea(nombre, segundos):
    await asyncio.sleep(segundos)
    print(f"  {nombre} terminó")

async def main():
    inicio = time.time()
    await asyncio.gather(
        tarea("A", 1), tarea("B", 2), tarea("C", 3)
    )
    print(f"Tiempo total: {time.time() - inicio:.2f}s")

asyncio.run(main())
```

~3 segundos, el de la tarea más lenta. `gather` las lanza concurrentemente: los tiempos no se suman.

6. Tarea en segundo plano

```python
import asyncio

async def contar():
    for i in range(1, 5):
        print(f"  contar: {i}")
        await asyncio.sleep(0.5)

async def main():
    asyncio.create_task(contar())   # en segundo plano
    await asyncio.sleep(1)
    print("main sigue")

asyncio.run(main())
```

`contar()` se ejecuta “en segundo plano”: mientras `main()` espera su segundo, la tarea cuenta. Se ve la alternancia de las dos corrutinas en el event loop.

7. Heartbeat básico

```python
import asyncio

async def heartbeat():
    while True:
        print("💓 vivo")
        await asyncio.sleep(2)

async def main():
    asyncio.create_task(heartbeat())
    await asyncio.sleep(5)
    print("main terminó")

asyncio.run(main())
```

`create_task` lanza el latido en segundo plano; `main()` espera 5s y lo deja latir tres veces antes de terminar. Si `main()` no esperara, la tarea moriría con él.

8. Timeout básico

```python
import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Hecho"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=3)
        print(r)
    except asyncio.TimeoutError:
        print("Timeout!")

asyncio.run(main())
```

`wait_for` corta a los 3s: salta `asyncio.TimeoutError`, la corrutina `lenta()` se cancela y el programa imprime “Timeout!”.

Boletín UD 10 — Avanzado

💪 Boletín UD 10 — Avanzado

> Ejercicios que requieren aplicar asyncio de forma más profunda: backoff, timeouts con respaldo, heartbeats múltiples, descargas concurrentes, servidores con latido y monitorización de varios servicios.

---

1. Backoff exponencial

Crea una función asíncrona `conectar()` que intente conectarse 4 veces con backoff (1s, 2s, 4s, 8s). Simula el fallo lanzando `ConnectionRefusedError`. Si falla, imprime "Servicio no disponible".

Pista: `espera = 2 ** intento`. Envuelve cada intento en `try/except ConnectionRefusedError`, espera con `await asyncio.sleep(espera)` y pasa al siguiente.

2. Timeout con respaldo

Crea una corrutina `lenta()` que tarde 8 segundos. Usa `asyncio.wait_for` con timeout de 5s. Si salta `TimeoutError`, ejecuta una corrutina de respaldo que devuelva "Resultado en caché".

Pista: `asyncio.wait_for(lenta(), timeout=5)` lanza `TimeoutError`; en el `except` haz `await respaldo()`.

3. Dos heartbeats

Crea dos corrutinas heartbeat: `hb_a()` imprime "💓 A" cada 3s, `hb_b()` imprime "💓 B" cada 5s. La función main las lanza con `asyncio.create_task` y espera 12 segundos.

Pista: dos `create_task` y luego `await asyncio.sleep(12)` para que `main()` no las cancele antes de tiempo.

4. 🎯 Web scraper asíncrono

Descarga 5 URLs a la vez con asyncio y httpx. Compara el tiempo con una versión síncrona.

Pista: usa `httpx.AsyncClient` dentro de cada corrutina. Crea una lista de tareas con `[descargar(u) for u in urls]` y ejecútalas con `asyncio.gather(*tareas)`. Mide el tiempo total con `time.time()`.

5. 🔍 Servidor asyncio con heartbeat

Servidor asyncio que imprime "💓 Vivo — N conexiones" cada 5s.

Pista: usa `asyncio.start_server` para el servidor TCP. Crea una corrutina `heartbeat` con un bucle infinito `while True: await asyncio.sleep(5); ...`. Lánzala con `asyncio.create_task` antes de iniciar el servidor.

6. ⏱️ Monitorización de servidores

3 servidores simulados. Un monitor asíncrono comprueba su estado cada 3s.

Pista: usa un diccionario `estados = {"Server-A": True, ...}`. Cada servidor es una corrutina que cambia su estado aleatoriamente cada ~5-15s. El monitor lee el diccionario cada 3s y reporta servidores caídos.

7. 🧩 Semáforo asyncio

Limita a 3 descargas simultáneas usando `asyncio.Semaphore`.

Pista: crea `sem = asyncio.Semaphore(3)`. Dentro de la función de descarga, usa `async with sem:` para que solo 3 corrutinas puedan ejecutar el bloque a la vez. Usa `httpbin.org/delay/{n}` para simular descargas lentas.

8. 🎭 Timeout con fallback

Intenta descargar de un servidor principal. Si tarda más de 2s, usa un servidor de respaldo.

Pista: envuelve la llamada a la corrutina principal con `asyncio.wait_for(descargar_principal(), timeout=2)`. Captura `asyncio.TimeoutError` y en el `except` ejecuta la corrutina de respaldo.

9. 🧪 Chat asíncrono

Crea un servidor de chat con asyncio que reenvíe mensajes a todos los clientes conectados.

Pista: mantén un conjunto `clientes` con los objetos `writer` de cada conexión. Crea una función `broadcast(mensaje, emisor=None)` que itere sobre una copia del conjunto y envíe a todos excepto al emisor. Usa `asyncio.wait_for(reader.read(1024), timeout=60)` para detectar desconexiones.

Boletín UD 10 — Avanzado (Resuelto)
💪 Boletín UD 10 — Avanzado (Resuelto)

1. Backoff exponencial

```python
import asyncio

async def conectar(intentos=4):
    for i in range(intentos):
        espera = 2 ** i                 # 1, 2, 4, 8
        try:
            print(f"Intento {i+1}...")
            raise ConnectionRefusedError # Simular error
        except ConnectionRefusedError:
            print(f"  Fallo: espero {espera}s")
            await asyncio.sleep(espera)
    print("Servicio no disponible")

asyncio.run(conectar())
```

Backoff exponencial: 1s, 2s, 4s, 8s. El último intento falla y se imprime “Servicio no disponible”.

2. Timeout con respaldo

```python
import asyncio

async def lenta():
    await asyncio.sleep(8)
    return "Resultado real"

async def respaldo():
    return "Resultado en caché"

async def main():
    try:
        r = await asyncio.wait_for(lenta(), timeout=5)
        print(r)
    except asyncio.TimeoutError:
        r = await respaldo()
        print(r)

asyncio.run(main())
```

A los 5s salta `TimeoutError`; el `except` ejecuta la corrutina de respaldo que devuelve “Resultado en caché”.

3. Dos heartbeats

```python
import asyncio

async def hb_a():
    while True:
        print("💓 A")
        await asyncio.sleep(3)

async def hb_b():
    while True:
        print("💓 B")
        await asyncio.sleep(5)

async def main():
    asyncio.create_task(hb_a())
    asyncio.create_task(hb_b())
    await asyncio.sleep(12)   # da tiempo a que latan
    print("main terminó")

asyncio.run(main())
```

Los dos latidos se ejecutan en segundo plano; `main()` espera 12s (A late 4 veces, B unas 3). Sin ese `await`, las tareas se cancelarían al acabar `main()`.

4. 🎯 Web scraper asíncrono

```python
import asyncio, httpx, time

URLS = ["https://httpbin.org/get"] * 5

async def descargar(url):
    async with httpx.AsyncClient() as cliente:
        r = await cliente.get(url)
        return r.status_code

async def main_async():
    tareas = [descargar(u) for u in URLS]
    resultados = await asyncio.gather(*tareas)
    print("Resultados:", resultados)

inicio = time.time()
asyncio.run(main_async())
print(f"Versión asíncrona: {time.time() - inicio:.2f}s")

# Versión síncrona equivalente para comparar:
inicio = time.time()
with httpx.Client() as cliente:
    for u in URLS:
        cliente.get(u)
print(f"Versión síncrona:   {time.time() - inicio:.2f}s")
```

`gather(*tareas)` lanza las 5 descargas a la vez: la versión asíncrona tarda lo que la más lenta, la síncrona la suma de todas.

5. 🔍 Servidor asyncio con heartbeat

```python
import asyncio

conexiones = 0

async def atender(reader, writer):
    global conexiones
    conexiones += 1
    datos = await reader.read(1024)
    writer.write(b"OK: " + datos)
    await writer.drain()
    writer.close()
    await writer.wait_closed()
    conexiones -= 1

async def heartbeat():
    while True:
        print(f"💓 Vivo — {conexiones} conexiones")
        await asyncio.sleep(5)

async def main():
    asyncio.create_task(heartbeat())          # latido en segundo plano
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    print("🚀 Servidor ASYNCIO en 127.0.0.1:5000")
    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())
```

`create_task(heartbeat())` lanza el latido antes de aceptar conexiones. Cada 5s imprime "💓 Vivo — N conexiones". Se mata con Ctrl+C.

6. ⏱️ Monitorización de servidores

```python
import asyncio, random

estados = {"Server-A": True, "Server-B": True, "Server-C": True}

async def servidor_simulado(nombre):
    """Cambia su estado aleatoriamente cada 5-15s."""
    while True:
        await asyncio.sleep(random.randint(5, 15))
        estados[nombre] = random.choice([True, False])
        print(f"  [{nombre}] cambia a {'ARRIBA' if estados[nombre] else 'CAÍDO'}")

async def monitor():
    while True:
        await asyncio.sleep(3)
        caidos = [n for n, ok in estados.items() if not ok]
        if caidos:
            print(f"🩺 Caídos: {', '.join(caidos)}")
        else:
            print("🩺 Todo en pie")

async def main():
    for nombre in estados:
        asyncio.create_task(servidor_simulado(nombre))
    await monitor()          # el monitor corre para siempre

asyncio.run(main())
```

Cada servidor simulado cambia su estado de forma aleatoria; el monitor lee el diccionario cada 3s y reporta los caídos. Es el heartbeat del punto 8 aplicado a varios servicios a la vez.

7. 🧩 Semáforo asyncio

```python
import asyncio, httpx, time

sem = asyncio.Semaphore(3)          # solo 3 a la vez

async def descargar(n):
    async with sem:                 # reserva un hueco del semáforo
        async with httpx.AsyncClient() as cliente:
            r = await cliente.get(f"https://httpbin.org/delay/{n}")
            return r.status_code

async def main():
    tareas = [descargar(n) for n in range(1, 7)]   # 6 descargas
    resultados = await asyncio.gather(*tareas)
    print("Resultados:", resultados)

inicio = time.time()
asyncio.run(main())
print(f"Tiempo total: {time.time() - inicio:.2f}s")
```

`async with sem`: garantiza que como máximo 3 corrutinas ejecutan el bloque a la vez; las otras 3 esperan su turno.

8. 🎭 Timeout con fallback

```python
import asyncio

async def descargar_principal():
    await asyncio.sleep(5)          # servidor lento
    return "Datos del principal"

async def descargar_respaldo():
    await asyncio.sleep(1)
    return "Datos del respaldo"

async def main():
    try:
        r = await asyncio.wait_for(descargar_principal(), timeout=2)
    except asyncio.TimeoutError:
        r = await descargar_respaldo()
    print(r)

asyncio.run(main())
```

A los 2s salta `TimeoutError` y el `except` ejecuta la corrutina de respaldo. La disponibilidad gana: siempre se devuelve una respuesta, aunque venga del servidor de backup.

9. 🏗️ Chat asíncrono

```
import asyncio

clientes = set()

async def broadcast(mensaje, emisor=None):
    for writer in list(clientes):       # copia: permite eliminar al iterar
        if writer != emisor:
            writer.write(mensaje)
            await writer.drain()

async def gestionar(reader, writer):
    clientes.add(writer)
    print(f"Cliente conectado ({len(clientes)} en total)")
    try:
        while True:
            try:
                datos = await asyncio.wait_for(reader.read(1024), timeout=60)
            except asyncio.TimeoutError:
                break                   # desconexión silenciosa
            if not datos:
                break
             await broadcast(b"[" + writer.get_extra_info("peername")[0].encode() + b"]
" + datos, emisor=writer)
    finally:
        clientes.discard(writer)
        writer.close()
        await writer.wait_closed()

async def main():
    servidor = await asyncio.start_server(gestionar, "127.0.0.1", 6000)
    print("💬 Chat ASYNCIO en 127.0.0.1:6000")
    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())
```

`broadcast` reenvía a todos los clientes menos al emisor. `wait_for(..., timeout=60)` detecta desconexiones: si un cliente no envía nada en 60s, se cierra su conexión y se quita del conjunto.

Boletín UD 11 — Inicial

📝 Boletín UD 11 — Inicial

> Ejercicios básicos para afianzar el puente Python → Java: tipos, clases, inyección de dependencias, controladores REST y JPA. En este boletín escribes **Java**, no Python.

---

1. Hola, Java

Crea una clase `Saludo` con un `main` que imprima "Hola, PSP desde Java".

Pista: en Java todo vive dentro de una clase: `public class Saludo { public static void main(String[] args) { ... } }`.

2. Variables y tipos

Declara e imprime: un `int` con la nota media, un `double` con tu altura en metros, un `boolean` si has aprobado y un `String` con tu nombre. Usa `System.out.println`.

Pista: en Java cada tipo tiene tamaño fijo (`int` 32 bits, `double` doble precisión) y no se declaran con `=` inferido salvo con `var`.

3. Lista de la compra

Crea una `ArrayList<String>` con 4 productos, añade un quinto con `add`, imprime la lista completa y cuántos elementos tiene con `size()`.

Pista: `List<String> compra = new ArrayList<>();` — las colecciones se declaran con la interfaz, no con la clase concreta.

4. Clase `Alumno`

Crea la clase `Alumno` con atributos `nombre` y `nota`, un constructor con ambos parámetros y un método `String aprobado()` que devuelva "SUSPENSO" si nota < 5 y "APROBADO" si no. Crea dos objetos y muéstralos.

Pista: `public String aprobado() { return nota < 5 ? "SUSPENSO" : "APROBADO"; }` — el operador ternario funciona como en Python.

5. Tu primer `@RestController`

Crea un controlador `SaludoController` con un `@GetMapping("/saludo")` que devuelva el texto "Hola desde Spring". ¿Qué anotación marca la clase como controlador REST?

Pista: `@RestController` (equivale a `@Controller` + `@ResponseBody`: devuelve datos, no vistas).

6. Parámetro en la ruta

Añade al controlador un `@GetMapping("/saludo/{nombre}")` que reciba el nombre de la ruta y devuelva "Hola, {nombre}". Usa `@PathVariable`.

Pista: el método recibe `@PathVariable String nombre` y Spring lo enlaza con `{nombre}` de la ruta.

7. JSON de respuesta

Crea la clase `Tarea` con campos `id` (long) y `titulo` (String), con constructor y getters. Devuélvela desde un `@GetMapping("/tarea")`. ¿Qué biblioteca convierte el objeto a JSON?

Pista: Spring incluye **Jackson**, que serializa los objetos a JSON automáticamente al devolverlos desde un `@RestController`.

8. Entidad JPA mínima

Crea la entidad `Producto` (campos `id`, `nombre`, `precio`) con las anotaciones `@Entity` y `@Id`. Indica qué anotación marca la clave primaria y cuál hace que la tabla se genere con ID autoincremental.

Pista: `@Id` marca la clave primaria; `@GeneratedValue(strategy = GenerationType.IDENTITY)` la hace autoincremental (como `INTEGER PRIMARY KEY AUTOINCREMENT` en SQLite).

Pista extra: recuerda que Spring necesita `spring-boot-starter-data-jpa` en el `pom.xml` y una BD (H2 en memoria vale) en `application.properties`.

Boletín UD 11 — Inicial (Resuelto)
✅ Boletín UD 11 — Inicial (Resuelto)

1. Hola, Java

```java
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola, PSP desde Java");
    }
}
```

En Java todo vive dentro de una clase; `main` debe ser `public static void`. Se compila con `javac Saludo.java` y se ejecuta con `java Saludo`.

2. Variables y tipos

```java
public class Tipos {
    public static void main(String[] args) {
        int notaMedia = 7;
        double altura = 1.75;
        boolean aprobado = true;
        String nombre = "Sergi";

        System.out.println("Nota: " + notaMedia);
        System.out.println("Altura: " + altura);
        System.out.println("Aprobado: " + aprobado);
        System.out.println("Nombre: " + nombre);
    }
}
```

En Java los tipos tienen tamaño fijo y no se infieren (salvo `var`). La concatenación de strings se hace con `+`.

3. Lista de la compra

```java
import java.util.ArrayList;
import java.util.List;

public class Compra {
    public static void main(String[] args) {
        List<String> compra = new ArrayList<>();
        compra.add("Pan");
        compra.add("Leche");
        compra.add("Huevos");
        compra.add("Café");
        compra.add("Arroz");

        System.out.println(compra);
        System.out.println("Elementos: " + compra.size());
    }
}
```

Se declara con la interfaz (`List`) y se instancia con la clase (`ArrayList`). `size()` es el equivalente a `len()` de Python.

4. Clase `Alumno`

```java
public class Alumno {
    String nombre;
    double nota;

    public Alumno(String nombre, double nota) {
        this.nombre = nombre;
        this.nota = nota;
    }

    public String aprobado() {
        return nota < 5 ? "SUSPENSO" : "APROBADO";
    }

    public static void main(String[] args) {
        Alumno a1 = new Alumno("Ana", 8.5);
        Alumno a2 = new Alumno("Luis", 4.2);
        System.out.println(a1.nombre + ": " + a1.aprobado());
        System.out.println(a2.nombre + ": " + a2.aprobado());
    }
}
```

`this.nombre` distingue el atributo del parámetro. El operador ternario funciona igual que en Python.

5. Tu primer `@RestController`

```java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SaludoController {

    @GetMapping("/saludo")
    public String saludo() {
        return "Hola desde Spring";
    }
}
```

La anotación de la clase es **`@RestController`**: combina `@Controller` (componente web) y `@ResponseBody` (el valor devuelto se serializa, no es una vista).

6. Parámetro en la ruta

```
@GetMapping("/saludo/{nombre}")
public String saludoNombre(@PathVariable String nombre) {
    return "Hola, " + nombre;
}
```

`@PathVariable` enlaza el trozo `{nombre}` de la ruta con el parámetro del método. `GET /saludo/Ana` devuelve “Hola, Ana”.

7. JSON de respuesta

```
public class Tarea {
    private long id;
    private String titulo;

    public Tarea(long id, String titulo) {
        this.id = id;
        this.titulo = titulo;
    }

    public long getId() { return id; }
    public String getTitulo() { return titulo; }
}
```

```
@GetMapping("/tarea")
public Tarea tarea() {
    return new Tarea(1, "Estudiar PSP");
}
```

La biblioteca es **Jackson**, incluida en `spring-boot-starter-web`. Serializa el objeto usando los getters: `{"id":1,"titulo":"Estudiar PSP"}`.

8. Entidad JPA mínima

```java
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private double precio;

    // Constructores, getters y setters
}
```

- **`@Entity`**: la clase es una entidad persistente (una tabla).
- **`@Id`**: marca la clave primaria.
- **`@GeneratedValue(IDENTITY)`**: la BD genera el ID autoincremental.

Spring necesita `spring-boot-starter-data-jpa` en el `pom.xml` y una BD configurada (H2 en memoria vale con `spring.datasource.url=jdbc:h2:mem:testdb`).

Boletín UD 11 — Avanzado

💪 Boletín UD 11 — Avanzado

> Ejercicios que requieren aplicar Spring Boot de forma más profunda: servicios, validación, JPA con consultas, testing, seguridad básica, gestión de errores y un mini-proyecto por capas completo.

---

1. 🏗️ Servicio entre controlador y repositorio

Crea la API de tareas del punto 6 de la unidad completa por capas: `TareaController` → `TareaService` → `TareaRepository`. El servicio añade la lógica: si el título está vacío, lanza `IllegalArgumentException`.

Pista: `@Service` en la clase de servicio y `@Autowired` (o constructor injection) para inyectar el repositorio en el servicio y el servicio en el controlador.

2. 📋 Consultas personalizadas JPA

Con la entidad `Producto`, crea en el `ProductoRepository` dos consultas: `findByPrecioLessThan(double precio)` y `findByNombreContaining(String texto)`.

Pista: Spring Data deriva la consulta del nombre del método: `LessThan` → `WHERE precio < ?`, `Containing` → `WHERE nombre LIKE %?%`. No hace falta escribir SQL.

3. ✅ Validación con anotaciones

Añade a `Producto` las anotaciones de validación: `@NotBlank` en `nombre` y `@Positive` en `precio`. En el controlador recibe el objeto con `@Valid` y devuelve un error 400 si no cumple.

Pista: necesitas `spring-boot-starter-validation`. `@Valid` en el parámetro del método dispara la validación; Spring devuelve 400 automáticamente con el detalle en el cuerpo.

4. 🧪 Primer test con JUnit

Escribe un test unitario para `TareaService`: verifica que `crearTarea()` con título válido devuelve una tarea con ID asignada y que con título vacío lanza `IllegalArgumentException`.

Pista: usa `@ExtendWith(MockExtension.class)` + `@Mock TareaRepository` + `@InjectMocks TareaService`. Con `assertThrows(IllegalArgumentException.class, () -> service.crearTarea(""))` compruebas la excepción.

5. 🔍 Test de integración del controlador

Crea un test de integración de la API con `@SpringBootTest` + `@AutoConfigureMockMvc` que haga un `GET /api/tareas` y compruebe que responde 200 y que el cuerpo es un JSON con tipo `application/json`.

Pista: inyecta `MockMvc`, usa `mockMvc.perform(get("/api/tareas")).andExpect(status().isOk()).andExpect(content().contentType(MediaType.APPLICATION_JSON))`.

6. ❌ Gestión de errores centralizada

Crea una clase `@RestControllerAdvice` que capture `TareaNoEncontradaException` y devuelva un 404 con un cuerpo JSON del tipo `{"error": "...", "mensaje": "..."}`. Lanza esa excepción desde el controlador cuando el ID no exista.

Pista: `@RestControllerAdvice` es el equivalente global a un gestor de excepciones; un método con `@ExceptionHandler(TareaNoEncontradaException.class)` + `@ResponseStatus(HttpStatus.NOT_FOUND)` devuelve el cuerpo de error.

7. 🗃️ Migración con Flyway

Crea una migración `V1__crear_tabla_productos.sql` en `src/main/resources/db/migration` con la tabla `productos` (id, nombre, precio) y configura Spring para usar H2 con Flyway.

Pista: añade `spring-boot-starter-flyway`. Flyway ejecuta al arrancar los ficheros `V<n>__descripcion.sql` en orden; el doble guion bajo separa el número de la descripción.

8. 🔐 Seguridad básica con Spring Security

Protege el endpoint `/api/tareas`: solo acepta peticiones con cabecera `X-Api-Key: secreta`. Si falta o es incorrecta, responde 401.

Pista: crea una clase que extienda `SecurityFilterChain` (o un filtro `OncePerRequestFilter`) y comprueba la cabecera antes de dejar pasar la petición. Recuerda desactivar CSRF para APIs sin sesiones.

9. 📦 Paginación en la API

Modifica `GET /api/productos` para que devuelva una página: acepta los parámetros `page` y `size` y devuelve `Page<Producto>`.

Pista: el repositorio debe extender `PagingAndSortingRepository` (o `JpaRepository`, que ya lo hace). Cambia el parámetro del método por `Pageable pageable` y Spring lo arma desde la query string.

10. 🚀 Mini-proyecto completo

Extiende la API de tareas con: relación `Tarea` → `Categoria` (OneToMany), endpoint `GET /api/categorias/{id}/tareas`, borrado en cascada y un test de integración que recorra el CRUD completo.

Pista: `@ManyToOne` en Tarea y `@OneToMany(mappedBy = "categoria")` en Categoria. Usa `@Transactional` en el servicio para las operaciones que tocan varias tablas. El test usa `@BeforeEach` para crear datos y `@Sql` si prefieres fijarlos en un script.

Boletín UD 11 — Avanzado (Resuelto)
💪 Boletín UD 11 — Avanzado (Resuelto)

1. 🏗️ Servicio entre controlador y repositorio

```
@Service
public class TareaService {

    private final TareaRepository repository;

    public TareaService(TareaRepository repository) {
        this.repository = repository;
    }

    public Tarea crearTarea(String titulo) {
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("El título no puede estar vacío");
        }
        Tarea t = new Tarea();
        t.setTitulo(titulo);
        t.setCompletada(false);
        return repository.save(t);
    }
}
```

```java
@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    private final TareaService service;

    public TareaController(TareaService service) {
        this.service = service;
    }

    @PostMapping
    public Tarea crear(@RequestBody Tarea tarea) {
        return service.crearTarea(tarea.getTitulo());
    }
}
```

Inyección por constructor (la recomendada por Spring): el atributo es `final` y Spring inyecta el bean automáticamente, sin `@Autowired`.

2. 📋 Consultas personalizadas JPA

```java
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    List<Producto> findByPrecioLessThan(double precio);

    List<Producto> findByNombreContaining(String texto);
}
```

Spring Data deriva el SQL del nombre del método: `findByPrecioLessThan` → `SELECT * FROM producto WHERE precio < ?`. `Containing` → `WHERE nombre LIKE %?%`. No se escribe una línea de SQL.

3. ✅ Validación con anotaciones

```java
@Entity
public class Producto {

    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;

    @Positive(message = "El precio debe ser positivo")
    private double precio;
}
```

```java
@PostMapping("/productos")
public ResponseEntity<Producto> crear(@Valid @RequestBody Producto p) {
    return ResponseEntity.status(HttpStatus.CREATED).body(repository.save(p));
}
```

Con `spring-boot-starter-validation`, `@Valid` dispara la validación; si falla, Spring devuelve **400 Bad Request** con el detalle de cada campo en el cuerpo. No hace falta capturar nada a mano.

4. 🧪 Primer test con JUnit

```java
@ExtendWith(MockitoExtension.class)
class TareaServiceTest {

    @Mock
    TareaRepository repository;

    @InjectMocks
    TareaService service;

    @Test
    void crearTareaTituloValidoAsignaId() {
        Tarea guardada = new Tarea();
        guardada.setId(1L);
        when(repository.save(any(Tarea.class))).thenReturn(guardada);

        Tarea resultado = service.crearTarea("Estudiar PSP");

        assertEquals(1L, resultado.getId());
        verify(repository).save(any(Tarea.class));
    }

    @Test
    void crearTareaTituloVacioLanzaExcepcion() {
        assertThrows(IllegalArgumentException.class,
                () -> service.crearTarea(""));
    }
}
```

`@Mock` crea un doble del repositorio, `@InjectMocks` lo mete en el servicio. `verify` comprueba que se llamó a `save`.

5. 🔍 Test de integración del controlador

```
@SpringBootTest
@AutoConfigureMockMvc
class TareaControllerTest {

    @Autowired
    MockMvc mockMvc;

    @Test
    void listarTareasResponde200Json() throws Exception {
        mockMvc.perform(get("/api/tareas"))
               .andExpect(status().isOk())
               .andExpect(content().contentType(MediaType.APPLICATION_JSON));
    }
}
```

`@SpringBootTest` levanta el contexto completo (con BD de prueba) y `MockMvc` simula peticiones HTTP reales sin abrir puerto.

6. ❌ Gestión de errores centralizada

```
public class TareaNoEncontradaException extends RuntimeException {
    public TareaNoEncontradaException(Long id) {
        super("Tarea no encontrada: " + id);
    }
}
```

```
@RestControllerAdvice
public class GestorErrores {

    @ExceptionHandler(TareaNoEncontradaException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> noEncontrada(TareaNoEncontradaException e) {
        return Map.of("error", "404", "mensaje", e.getMessage());
    }
}
```

`@RestControllerAdvice` aplica a todos los controladores. En el controlador: `throw new TareaNoEncontradaException(id)` y el cliente recibe 404 con JSON de error.

7. 🗃️ Migración con Flyway

```sql
-- V1__crear_tabla_productos.sql
CREATE TABLE productos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL,
    precio DOUBLE NOT NULL
);
```

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.flyway.enabled=true
```

Flyway ejecuta al arrancar cada `V<n>__descripcion.sql` por orden y guarda en su propia tabla qué migraciones ya corrieron. Si cambias un fichero ya ejecutado, lanza error al arrancar.

8. 🔐 Seguridad básica con Spring Security

```java
@Configuration
@EnableWebSecurity
public class SeguridadConfig {

    @Bean
    SecurityFilterChain filtrar(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
            .addFilterBefore(new ApiKeyFilter(),
                    UsernamePasswordAuthenticationFilter.class);
        http.httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

```
public class ApiKeyFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {

        String clave = req.getHeader("X-Api-Key");
        if ("secreta".equals(clave)) {
            chain.doFilter(req, res);
        } else {
            res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        }
    }
}
```

El filtro se ejecuta antes de la cadena de autenticación: sin la cabecera correcta, **401**. CSRF se desactiva porque una API sin sesiones no lo necesita.

9. 📦 Paginación en la API

```
@GetMapping("/productos")
public Page<Producto> listar(@RequestParam int page,
                             @RequestParam int size) {
    Pageable pageable = PageRequest.of(page, size);
    return repository.findAll(pageable);
}
```

`JpaRepository` ya extiende `PagingAndSortingRepository`. La respuesta incluye `content` (los elementos) más `totalPages`, `totalElements` y `number`. Llamada: `GET /api/productos?page=0&size=10`.

10. 🚀 Mini-proyecto completo

```java
@Entity
public class Tarea {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String titulo;
    private boolean completada;

    @ManyToOne
    private Categoria categoria;
}
```

```java
@Entity
public class Categoria {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;

    @OneToMany(mappedBy = "categoria", cascade = CascadeType.ALL)
    private List<Tarea> tareas;
}
```

```java
@Service
public class TareaService {

    private final TareaRepository tareas;
    private final CategoriaRepository categorias;

    public TareaService(TareaRepository tareas,
                        CategoriaRepository categorias) {
        this.tareas = tareas;
        this.categorias = categorias;
    }

    @Transactional
    public Tarea moverATarea(Long categoriaId, Tarea tarea) {
        Categoria cat = categorias.findById(categoriaId)
                .orElseThrow(() ->
                    new TareaNoEncontradaException(categoriaId));
        tarea.setCategoria(cat);
        return tareas.save(tarea);
    }
}
```

```java
@RestController
@RequestMapping("/api/categorias")
public class CategoriaController {

    private final TareaRepository tareas;

    public CategoriaController(TareaRepository tareas) {
        this.tareas = tareas;
    }

    @GetMapping("/{id}/tareas")
    public List<Tarea> tareasDe(@PathVariable Long id) {
        return tareas.findByCategoriaId(id);
    }
}
```

En el repositorio: `List<Tarea> findByCategoriaId(Long categoriaId);`. `@Transactional` garantiza que crear y relacionar se hace en una sola transacción; `cascade = ALL` propaga el borrado a las tareas hijas.